

Tessent[®] Shell User's Manual

Software Version 2019.3

Document Revision 14

© 2012-2019 Mentor Graphics Corporation
All rights reserved.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

U.S. GOVERNMENT LICENSE RIGHTS: The software and documentation were developed entirely at private expense and are commercial computer software and commercial computer software documentation within the meaning of the applicable acquisition regulations. Accordingly, pursuant to FAR 48 CFR 12.212 and DFARS 48 CFR 227.7202, use, duplication and disclosure by or for the U.S. Government or a U.S. Government subcontractor is subject solely to the terms and conditions set forth in the license agreement provided with the software, except for provisions which are contrary to applicable mandatory federal laws.

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the owner of the Mark, as applicable. The use herein of a third-party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: mentor.com/trademarks.

The registered trademark Linux[®] is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

End-User License Agreement: You can print a copy of the End-User License Agreement from: mentor.com/eula.

Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: mentor.com
Support Center: support.mentor.com

Send Feedback on Documentation: support.mentor.com/doc_feedback_form

Revision History ISO-26262

Revision	Changes	Status/ Date
14	Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo. All technical enhancements, changes, and fixes listed in the <i>Tessent Release Notes</i> for this product are reflected in this document. Approved by Ron Press.	Released Sep 2019
13	Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo. All technical enhancements, changes, and fixes listed in the <i>Tessent Release Notes</i> for this product are reflected in this document. Approved by Ron Press.	Released May 2019
12	Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo. All technical enhancements, changes, and fixes listed in the <i>Tessent Release Notes</i> for this product are reflected in this document. Approved by Ron Press.	Released Mar 2019
11	Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo. All technical enhancements, changes, and fixes listed in the <i>Tessent Release Notes</i> for this product are reflected in this document. Approved by Ron Press.	Released Dec 2018

Author: In-house procedures and working practices require multiple authors for documents. All associated authors for each topic within this document are tracked within the Mentor Graphics Technical Publication's source. For specific topic authors, contact Mentor Graphics Technical Publication department.

Revision History: Released documents include a revision history of up to four revisions. For earlier revision history, refer to earlier releases of documentation on Support Center.

Table of Contents

Revision History ISO-26262

Chapter 1

Tessent Shell Introduction	19
What Is Tessent Shell?	19
What Can You Do With Tessent Shell?	20
Tessent Shell Tcl Interface	22
Command Conventions	22
Command Completion	23
Dofile Transcription	24
Tcl Command Registration	25

Chapter 2

Tool Invocation and Basic Concepts	27
Tool Invocation	28
Tessent Startup File	28
Tessent Shell Environment Variables	28
Contexts and System Modes	30
Contexts	30
System Modes	32
Context and System Mode Combinations	32
Design Levels	33
Design Data Models	34
Flat Design Data Model	34
Hierarchical Design Data Model	34
ICL Data Model	36
Object Attributes	36

Chapter 3

Design Introspection and Editing	39
Design Introspection	40
Object Specification Format	40
Collections	40
Introspection Examples	43
Design Introspection Command Summary	47
Bundle Object Introspection	49
Bundle Object	49
Bundle Object Data Model	54
Introspection	56
Introspection Examples	61
Fanin and Fanout Tracing of Complex Signals and Bundle Object Tracing	64
Connectivity Through Complex Signals	67

Design Editing	72
Design Editing Examples	73
Design Editing Command Summary	78
Simulation Contexts	80
Simulation Context Overview	80
Introspection and Analysis Using Simulation Contexts	81
Chapter 4	
DFT Architecture Guidelines for Hierarchical Designs	85
Hierarchical DFT: Concepts and Terms	86
Physical Layout Regions in Hierarchical Test	86
Pattern Retargeting	87
Wrapped Cores and Wrapper Cells	87
Internal Mode and External Mode	88
On-Chip Clock Controller	89
Graybox Model	90
Top-Down Planning Before Bottom-Up Implementation	92
Clocking Architecture	92
Resource Availability	93
Test Scheduling	93
Specific Tasks That May Require Planning	96
DFT Implementation Strategy	98
Internal and External Modes	98
Dedicated Test Clock	99
Sample DFT Planning Steps	99
Automated Features Within the Tessent Shell Flow	100
Chapter 5	
Tessent Shell Work Flows	103
Tessent Shell Flow for Flat Designs	104
Overview of the RTL and Scan DFT Insertion Flow	105
First DFT Insertion Pass: MemoryBIST and Boundary Scan	107
Second DFT Insertion Pass: EDT and OCC	111
Load the Design	112
Specify and Verify the DFT Requirements	114
Create the DFT Specification	117
Generate the EDT and OCC Hardware	120
Extract the ICL Module Description	120
Generate ICL Patterns and Run Simulation	121
Perform Synthesis	122
Perform Scan Chain Insertion	123
Perform ATPG Pattern Generation	124
Considerations for Using Gate-Level Verilog Netlists	127
Tessent Shell Flow for Hierarchical Designs	128
Hierarchical DFT Terminology	129
How the DFT Insertion Flow Applies to Hierarchical Designs	132
RTL and Scan DFT Insertion Flow for Physical Blocks	133
First DFT Insertion Pass: Performing Block-Level MemoryBIST	134

Table of Contents

Second DFT Insertion Pass: Block-Level EDT and OCC	136
Specify and Verify the DFT Requirements: DFT Signals for Wrapped Cores	139
Perform Scan Chain Insertion	141
Perform ATPG Pattern Generation: Wrapped Core.....	144
Run Recommended Validation Step for Pre-Layout Design Sign Off.....	148
RTL and Scan DFT Insertion Flow for the Top Chip.....	150
Top-Level DFT Insertion Example	151
First DFT Insertion Pass: Performing Top-Level MemoryBIST and Boundary Scan... ..	153
Second DFT Insertion Pass: Top-Level EDT and OCC.....	156
Top-Level Scan Chain Insertion Example	160
Top-Level ATPG Pattern Generation Example	160
Performing Top-Level ATPG Pattern Retargeting.....	162
RTL and Scan DFT Insertion Flow for Sub-Blocks.....	165
DFT Insertion Flow for the Sub-Block	165
DFT Insertion Flow for the Next Parent Level	166
RTL and DFT Insertion Flow with Third-Party Scan.....	169
DFT Insertion Flow With Third-Party Scan Insertion	169
Wrapped Core DFT Insertion with Third-Party Scan	170
Top Chip DFT Insertion with Third-Party Scan.....	184
Tessent Shell Post-Layout Validation Flow	193
Overview of the Post-Layout Validation Flow.....	193
Soft Link TSDB and Post-Layout Netlist.....	194
Verify MemoryBIST, Boundary Scan and JTAG Patterns	195
Verify the Scan-Inserted ATPG Patterns	196
Post-Layout Validation When You Have Ungrouped JTAG/OCC/EDT Logic	198
Hybrid TK/LBIST Flow for Flat Designs	202
RTL and Scan DFT Insertion Flow With Hybrid TK/LBIST	203
First DFT Insertion Pass: Hybrid TK/LBIST	203
Second DFT Insertion Pass: Hybrid TK/LBIST.....	207
Perform Test Point Insertion.....	212
Perform Scan Chain Insertion.....	214
Perform ATPG Pattern Generation: Hybrid TK/LBIST.....	216
Perform LogicBIST Fault Simulation.....	219
Perform JTAG Pattern Generation	222
Running Multi-Load ATPG on Memories for Wrapped Cores with Built-In Self Repair ..	224
Overview of Multi-Load ATPG on Memories for Wrapped Cores With Built-in Self Repair	224
Performing Multi-Load ATPG Pattern Generation	225
Performing Multi-Load Top-Level ATPG Pattern Retargeting	228
Built in Self Repair (BISR) in Hierarchical Tessent MemoryBIST Flow.....	231
Overview of BISR	231
Performing Block Level BISR Insertion	233
Assigning Memories to Power Domains	233
Controlling the BISR Chain Order	234
Disabling Insertion of BISR Registers.....	234
Excluding Child Block BISR Chains	234
Performing Chip Level BISR Insertion	236
Choosing a Functional Repair Clock	236
Connecting BISR controller to Existing BISR Chains.....	236

Connecting BISR controller to an External Fuse Box	237
Connecting BISR Controller to System Logic	238
Verifying Block and Chip Level BISR	239
Block Level BISR Verification	239
Chip Level BISR Verification	239
Chapter 6	
TSDB Data Flow for the Tessent Shell Flow	241
Core-Level or Flat TSDB Data Flow	241
Top-Level TSDB Data Flow	246
Chapter 7	
Tessent Examples and Solutions	253
How to Handle Clocks Sourced by Embedded PLLs During Logic Test	253
How to Design Capture Windows for Hybrid TK/LBIST	259
How to Use Boundary Scan in a Wrapped Core	261
TAP Configuration	264
Stand-alone TAP	264
TAP with IJTAG Host Scan Interface	266
Compliance Enable TAP with IJTAG Interface	267
Daisy chained TAP	269
Master TAP Insertion	270
Slave TAP Insertion	271
Connecting to a Third-Party TAP	273
How to Set Up Third-Party Synthesis	273
How to Set Up Support for Third-Party OCCs	276
How to Configure Files for Third-Party OCCs	276
Test Logic Insertion	277
Scan Insertion	279
Pattern Generation	279
Pattern Simulation	282
Chapter 8	
Test Procedure File	283
Test Procedure File Creation	284
Test Procedure File Syntax	284
Test Procedure File Structure	289
#include Statement	289
Set Statement	290
Alias Definition	293
Timing Variables	295
Timeplate Definition	298
Multiple-Pulse Clocks	304
The pulse_clock Statement	304
Inferred Timing	305
Differences Between Default add_clock and 1x Multiplier Clock	306
Always Block	306
Procedure Definition	307

Table of Contents

Clock Control Definition	317
The Procedures	326
Test_Setup (Optional).....	327
Shift (Required)	330
Alternate Shift Procedure (Optional)	332
Load_Unload (Required)	334
Shadow_Control (Optional)	337
Master_Observe (Sometimes Required).....	339
Shadow_Observe (Optional).....	339
Seq_Transparent (Optional)	340
Clock (Optional).....	342
Skew_Load (Optional)	343
Clock_run (Optional)	344
Capture Procedures (Optional)	346
Rules for Creating and Editing a Default Capture Procedure	347
Rules for Creating and Editing Named Capture Procedures	347
Slow and Load Types in the Cycle Statement	349
launch_capture_pair Statement	350
Clock_po (Optional).....	351
Ram_sequential (Optional).....	351
Ram_passthru (Optional)	353
Clock_sequential (Optional).....	353
Init_force (Optional).....	354
Test_end (Optional, all ATPG tools)	355
Sub_procedure	356
Additional Support for Test Procedure Files	358
Creating Test Procedure Files for End Measure Mode	359
Serial Register Load and Unload for LogicBIST and ATPG	363
Register Load and Unload Use Models	363
Static Versus Dynamic Register Variables.....	363
Test Procedure File Modifications	364
Dofile Modifications.....	367
Serial Load and Unload DRC Rules.....	370
P13 and P54	370
P66	370
W5	371
Procedure Examples.....	372
Notes About Using the stil2mgc Tool	376
Extraction of Strobe Timing Information from STIL (SPF).....	376
The STIL ClockStructures Block	376
Test Procedure File Commands and Output Formats	377

Chapter 9

Tessent Visualizer	379
Overview	379
Invoking Tessent Visualizer	380
Framework Overview	381
Common Features	385

Tessent Visualizer GUI Features	386
Common Toolbar Features	386
Tables	387
Columns and Filters Editor	388
Filters	389
Other Common Features	391
Schematics	392
Tessent Shell Attributes	405
Gate Report	407
Tessent Visualizer Preferences	407
Windows Overview	411
Hierarchical Schematic Window	411
Flat Schematic Window	421
Instance Browser	424
Cell Library Browser	428
DRC Browser	429
Pin Data Window	432
Transcript Window	434
Text/HDL Viewer	437
Diagnosis Report Viewer	437
Search Features	440
Usage Examples	443
Chapter 10	
Configuration Data Visualizer	449
Modifying the Contents of the Configuration Data Visualizer	449
Adding a Test Data Register to a SIB Example	450
Adding a Multiplexer to a SIB Example	451
Customizing the DFT Specification for EDT	452
Chapter 11	
Timing Constraints (SDC)	457
Generating and Using SDC for Tessent Shell Embedded Test IP	458
SDC File Generation with Tessent Shell	458
SDC Design Synthesis with Tessent Shell	460
Preparation Step 1: Sourcing SDC File	460
Preparation Step 2: Setting and Redefining Tessent Tcl Variables	460
Preparation Step 3: Verifying the Declaration of Functional Clocks	462
Preparation Step 4: Redefining Other Tessent Tcl Variables	463
Synthesis Step 1: Loading the Design Into Your Synthesis Tool	464
Synthesis Step 2: Applying the SDC Constraints	464
Synthesis Step 3: Preparing the DFT Logic for Synthesis	464
Synthesis Step 4: Synthesizing Your Design	465
Synthesis Step 5: Writing Out Your Final SDC	465
Synthesis Step 6: Writing Out Your Final Netlist	466
SDC for Modal Static Timing Analysis	467
Checking Your Functional Logic Alone	467
Checking Your Embedded Test Logic	467

Table of Contents

VHDL and Mixed Language Designs	469
VHDL Generate Blocks	469
SDC File Contents	470
tessent_set_default_variables	470
tessent_create_functional_clocks	471
tessent_<design_name>_set_dft_signals	471
tessent_constrain_<design_name>_mentor_ltest_disable	472
tessent_constrain_<design_name>_non_modal	472
tessent_<design_name>_kill_functional_paths	472
IJTAG Instrument	473
LOGICTEST Instruments	473
MemoryBIST Instrument	485
BoundaryScan Instrument	488
Cell and Pin Mapping Procs	488
Synthesis Helper Procs	489
 Appendix A	
The Tessent Tcl Interface	491
General Tcl Guidelines in Tessent Shell	491
Guidelines for Modifying Existing Dofiles for Use with Tcl	493
Special Tcl Characters	495
Custom Tcl Packages in Tessent Shell	497
Tcl Resources	498
 Appendix B	
Synthesis Guidelines for RTL Designs with Tessent Inserted DFT	499
DFT Insertion with Tessent	500
Synthesis Guidelines	501
 Appendix C	
Example Scripts using Tessent Tool-Generated SDC	505
Example Design Compiler Synthesis Script	505
Example Encounter Synthesis Script	507
Example PrimeTime STA Script	510
 Appendix D	
Clocking Architecture Examples	517
Clocks Driven by Primary Inputs	517
Clock Generators Outside the Cores	518
Clock Generators Inside the Cores	518
Clock Sourced From A Core With Embedded PLL	519
Clock Mesh Synthesis	520
 Appendix E	
Formal Verification	523
Constraints for Formality Scripts	523

Appendix F

Transitioning from the Classic Point Tools 525

 Transitioning from the Classic FastScan Point Tool 525

 Transitioning from the Classic TestKompress Point Tool. 526

 Transitioning from the Classic DFTAdvisor Tool. 527

 Transitioning from the Classic Diagnosis Tool 528

Appendix G

Getting Help 529

 The Tessent Documentation System 529

 Mentor Support Services 530

Index

Third-Party Information

End-User License Agreement

with EDA Software Supplemental Terms

List of Figures

Figure 2-1. Hierarchical Design Example	35
Figure 3-1. Hierarchical Design Example With Colors.....	44
Figure 3-2. Complex Signals with Tessent Visualizer	69
Figure 3-3. Hierarchical Design Example	74
Figure 3-4. Inverter Inception	76
Figure 3-5. Tessent Visualizer Flat Schematic (gate level).....	82
Figure 4-1. Wrapped Cores	87
Figure 4-2. Internal Mode	88
Figure 4-3. External Mode.....	89
Figure 4-4. On-Chip Clock Controller.....	90
Figure 4-5. Graybox Model	91
Figure 4-6. Compression Analysis Example	95
Figure 5-1. Two-Pass Insertion Flow for RTL, Flat Designs	105
Figure 5-2. DFT-Ready Design	106
Figure 5-3. After the First DFT Insertion Pass	106
Figure 5-4. After the Second DFT Insertion Pass	107
Figure 5-5. Flow for the Second DFT Insertion Pass	111
Figure 5-6. Flow for Loading the Design	112
Figure 5-7. Flow for Specifying and Verifying the DFT Requirements	114
Figure 5-8. Flow for Creating the DFT Specification	117
Figure 5-9. Flow for Inserting the Second-Pass Hardware	120
Figure 5-10. Flow for Extracting ICL	121
Figure 5-11. Flow for Generating and Simulating ICL Patterns	122
Figure 5-12. Two-Pass Insertion Flow for RTL, Gate-Level Designs	127
Figure 5-13. Hierarchical Design Levels.....	130
Figure 5-14. Two-Pass Insertion Flow for RTL, Wrapped Cores	133
Figure 5-15. Flow for Specifying and Verifying the DFT Requirements	139
Figure 5-16. ATPG Pattern Generation Flow for Wrapped Cores	144
Figure 5-17. Two-Pass Insertion Flow for RTL, Top Level	150
Figure 5-18. Top-Level Example, Before DFT Insertion	152
Figure 5-19. Top-Level Example, After DFT Insertion for Wrapped Cores.....	152
Figure 5-20. Top-Level Example, After DFT Insertion at the Top Level.....	153
Figure 5-21. Two-Pass Insertion Flow for RTL, Wrapped Cores, and Third-Party Scan...	170
Figure 5-22. DFT Signals and Multiplexer Logic Support Hierarchical ATPG	173
Figure 5-23. At-Speed Flows in the Wrapper Chain	175
Figure 5-24. Current Level Path to Child Core Wrapper Chains.....	179
Figure 5-25. Two-Pass Insertion Flow for RTL, Top Level, and Third-Party Scan	184
Figure 5-26. Top Level EDT Connection to Child Cores	188
Figure 5-27. Post-Layout Validation Flow	193
Figure 5-28. Post-Layout Validation Flow with Ungrouped IJTAG/OCC/EDT Logic	198

Figure 5-29. Generate Graybox Example	200
Figure 5-30. Run ATPG on the Core's Internal Mode Example	201
Figure 5-31. Two-Pass Insertion Flow With Hybrid TK/LBIST	203
Figure 5-32. Two-Pass Insertion Flow for RTL, Wrapped Cores	225
Figure 5-33. Two-Pass Insertion Flow for RTL, Top Level	228
Figure 6-1. TSDB Data Flow, Core Level, First Insertion Pass	243
Figure 6-2. TSDB Data Flow, Core Level, Second Insertion Pass	244
Figure 6-3. TSDB Data Flow, Core Level, Scan Insertion	245
Figure 6-4. TSDB Data Flow, Core Level, Pattern Generation	245
Figure 6-5. TSDB Data Flow, Top Level, First Insertion Pass	247
Figure 6-6. TSDB Data Flow, Top Level, Second Insertion Pass	248
Figure 6-7. TSDB Data Flow, Top Level, Scan Insertion	249
Figure 6-8. TSDB Data Flow, Top Level, ATPG Pattern Generation	250
Figure 6-9. TSDB Data Flow, Top Level, ATPG Pattern Generation with Pattern Retargeting 251	
Figure 7-1. Example Chip with PLL Embedded Inside Lower Core	255
Figure 7-2. Active OCCs During Internal Test Modes of corec and coreb	256
Figure 7-3. Active OCCs During Internal Test Modes of corea and coreb	257
Figure 7-4. Active OCCs During Test Modes of Top Level	258
Figure 7-5. Wrapped Core Boundary Scan Flow	262
Figure 8-1. 200ns Timing Waveform	305
Figure 8-2. Shift Procedure	330
Figure 8-3. Timing Diagram for Shift Procedure	332
Figure 8-4. Load_Unload Procedure	335
Figure 8-5. Timing Diagram for Load_Unload Procedure	336
Figure 8-6. Shadow_Control Procedure	337
Figure 8-7. Master_Observe Procedure	339
Figure 8-8. Shadow_Observe Procedure	340
Figure 8-9. Sequential Transparent Circuitry Example	341
Figure 8-10. Skew_Load Procedure	343
Figure 8-11. Skew_load applied within Pattern	344
Figure 8-12. Full Ram Sequential Pattern	352
Figure 8-13. Full Clock Sequential Pattern	354
Figure 8-14. Init_force Procedure Usage	354
Figure 9-1. Tessent Visualizer Default View	382
Figure 9-2. Tessent Visualizer Split View	383
Figure 9-3. Tessent Visualizer Undocked Windows	384
Figure 9-4. Common Toolbar Features	386
Figure 9-5. Search - Instances: Tabular Report	387
Figure 9-6. Columns and Filters Editor	388
Figure 9-7. Selection Colors	392
Figure 9-8. Schematic Window Elements (1 of 3)	393
Figure 9-9. Schematic Window Elements (2 of 3)	394
Figure 9-10. Schematic Window Elements (3 of 3)	395
Figure 9-11. Dragging an Object From a Browser to a Schematic Tab	397

Figure 9-12. Object Dropped Into a Schematic Tab	398
Figure 9-13. Mouse Gestures in Tessent Visualizer Schematic Windows	399
Figure 9-14. Unfolding an Instance	400
Figure 9-15. Attributes Table.	400
Figure 9-16. Properties and Components	401
Figure 9-17. Complex Component With Callout and Pin Data	402
Figure 9-18. Collapsed Buffers/Inverters With no Hidden Fanout	403
Figure 9-19. Uncollapsed Buffers/Inverters With no Hidden Fanout	404
Figure 9-20. Collapsed Buffers/Inverters With Hidden Fanout	404
Figure 9-21. Uncollapsed Buffers/Inverters With Hidden Fanout.	404
Figure 9-22. Uncollapsed Buffers/Inverters With Fanout Revealed	405
Figure 9-23. Tessent Shell Attributes Table	406
Figure 9-24. Preferences Window (Schematic Tab)	408
Figure 9-25. Light, Dark, and High-Contrast Color Themes in Tessent Visualizer Schematic Windows	408
Figure 9-26. Preferences Window (Tables Tab)	409
Figure 9-27. Preferences Window (Text Viewers Tab).	410
Figure 9-28. Hierarchical Schematic Window.	412
Figure 9-29. Hierarchical Schematic Window With Overview Window and Properties Table 413	
Figure 9-30. Overview Window	413
Figure 9-31. Hierarchical Instance With All Pins Shown	415
Figure 9-32. Hierarchical Instance With Unconnected Pins Deleted From the Schematic. .	416
Figure 9-33. Hierarchical Schematic Instances Table	417
Figure 9-34. Hierarchical Schematic Nets Table.	418
Figure 9-35. Hierarchical Schematic Symbols	419
Figure 9-36. Bus Tracing in the Hierarchical Schematic.	420
Figure 9-37. Bus Sub-Ranges	421
Figure 9-38. Flat Schematic Window (Cell Grouping On)	422
Figure 9-39. Flat Schematic Window (Cell Grouping Off).	423
Figure 9-40. Non-Hierarchical Pin	424
Figure 9-41. Collapsible Buffers and Inverters	424
Figure 9-42. Tessent Visualizer Instance Browser	426
Figure 9-43. Instance Browser Column Editor	427
Figure 9-44. Fault Coverage Debugging	428
Figure 9-45. Cell Library Browser Window	429
Figure 9-46. DRC Browser With Data Grouped By Rule	430
Figure 9-47. Flat Schematic Window Showing a DRC Violation.	431
Figure 9-48. DRC Browser Window With Filter	432
Figure 9-49. Tessent Visualizer Pin Data Window	433
Figure 9-50. A GTKWave Viewer Application Invoked From the Pin Data Viewer	434
Figure 9-51. Tessent Visualizer Transcript Window	435
Figure 9-52. Tessent Transcript Window Search Capabilities	436
Figure 9-53. Diagnosis Report Viewer (Text Mode).	438
Figure 9-54. Text Search in the Diagnosis Report Viewer	439

Figure 9-55. Diagnosis Report Viewer (Table View)	440
Figure 9-56. Search Window: Instances	441
Figure 9-57. Search Window: Pins	442
Figure 9-58. Search Window: Gate Pins	443
Figure 9-59. Listing State Elements	444
Figure 9-60. Searching for a Pin and Adding it to the Schematic Window	445
Figure 9-61. Searching for a Pin by Name	446
Figure 9-62. Single Bit Tracing	447
Figure 11-1. tessent_test_clock and tessent_shift_capture_clock Creation	475
Figure 11-2. Generated Clock Muxed Multi-Clock Memories	486
Figure B-1. Problem Occurrence Creating the Gate Level Netlist	499
Figure D-1. OCCs for Clocks Driven by Primary Inputs	517
Figure D-2. OCCs With Clock Generators at Chip Level, Asynchronous Clocks	518
Figure D-3. OCCs With Clock Generators Inside the Cores, Asynchronous Clocks	519
Figure D-4. Clock Sourced From A Core With Embedded PLL, With MUX	519
Figure D-5. Clock Sourced From A Core With Embedded PLL, Without MUX	520
Figure D-6. Clock Mesh Synthesis	521

List of Tables

Table 1-1. Tessent Shell Command Conventions	22
Table 1-2. Commands for Tcl Command Registration	25
Table 2-1. Application-Specific Environment Variables	28
Table 2-2. Tessent Shell Contexts	30
Table 2-3. Tessent Shell System Modes	32
Table 2-4. Tessent Shell Context and System Mode Commands	33
Table 3-1. Commands That Interact With Collections	42
Table 3-2. Design Introspection Commands	47
Table 3-3. Attribute Introspection Commands	48
Table 3-4. base_class and base_type System Verilog Examples	55
Table 3-5. Complex Signals Design Introspection Commands	60
Table 3-6. Design Editing Commands	72
Table 3-7. Design Editing Command Summary	78
Table 4-1. Test Schedule Example	96
Table 6-1. Core-Level TSDB Data Flow Inputs and Outputs	242
Table 6-2. Top-Level TSDB Data Flow Inputs and Outputs	246
Table 8-1. Reserved Punctuation Characters	285
Table 8-2. Procedure File Tool Command Summary	377
Table 9-1. Filtering Summary	390
Table 9-2. Actions Available From the Schematic Window Toolbar	395
Table 9-3. Marker Symbols	403
Table 9-4. Hierarchical Schematic Marker Symbols	421
Table A-1. Common Dofile Issues and Solutions	494
Table A-2. Common Tcl Characters	495

Chapter 1

Tessent Shell Introduction

Tessent Shell is a platform from which you can run all the Tessent tools. The platform includes shared design data, a common database, and powerful scripting utilities that provide a fully automated design-for-test (DFT) flow as well as the ability to customize the flow to fit specific requirements.

What Is Tessent Shell?	19
What Can You Do With Tessent Shell?	20
Tessent Shell Tcl Interface	22
Command Conventions.	22
Command Completion	23
Dofile Transcription	24
Tcl Command Registration.....	25

What Is Tessent Shell?

Tessent Shell is a design-for-test environment within which you can perform all the tasks required to insert DFT hardware, generate manufacturing test patterns, and post-silicon tasks such as diagnosis and yield analysis.

Tessent Shell provides a flexible design flow that supports a high-level of automation at the same time that it allows you to customize flows to your requirements. Key aspects of the environment that support automation as well as enhance flexibility are:

- **Shared Data Model** — Tessent Shell uses data models to store your design data. These models are shared across all tools and functions, and you can access the data stored within them for purposes of customizing your own design flow.

For standard automated flows, you do not need to be aware of this infrastructure. For those that need to customize design flow steps that are not supported by the native tool commands, you have the same access to the design data model as the Tessent Shell tools do.

For more information about the data models, refer to “[Design Data Models](#).”

- **Attributes** — Attributes are characteristics associated with design objects, such as library cell, pins and modules, that are stored within data models. Some are pre-defined, and you can also create your own attributes. Using attributes increases visibility into your design, allowing you to manipulate and query attributes on the design objects.

- **Design introspection:** Examine the design objects and attributes stored within the data models. Introspection allows you to retrieve the design data you need so that you can use Tcl scripting techniques to automate your own custom design flows.

For more information about design introspection, refer to “[Design Introspection](#).”

- **Tcl** — Use this common scripting language across all Tessent tools and functions. Through scripting, you can jump into and out of the Tessent Shell out-of-the-box flows as needed for your design requirements.

For more information about Tcl usage, refer to “[Tessent Shell Tcl Interface](#).”

- **Tessent Shell Database (TSDB)** — The TSDB is a database that stores all the directories and files that Tessent Shell generates as you move through a design flow. The TSDB enhances flow automation by acting as the central location where any Tessent tool can access the data it requires for the current task, whether that task be reading in a design, performing DRC, inserting logic test hardware, or performing ATPG pattern generation.

For more information about the TSDB, refer to “[TSDB Data Flow for the Tessent Shell Flow](#).”

- **IJTAG Automation** — By default, the DFT instruments you create and insert with Tessent Shell are controlled through an IJTAG network and protocol (IEEE 1687). Tessent Shell automatically inserts the IJTAG infrastructure, which simplifies test setup and control of all the instruments at all level of hierarchy, and also allows for reuse in hierarchical designs and test-benching at any stage of chip development.

For more information about IJTAG, refer to the “[Tessent IJTAG User’s Manual](#).”

What Can You Do With Tessent Shell?

The Tessent Shell platform is a jumping-off point for accomplishing the full array of DFT tasks available within the Tessent tool suite. Specific DFT tasks may require different licenses, but all tools may be launched from and managed via Tessent Shell.

DFT tasks include:

- **Instrument Insertion** — Generate and insert logic test hardware such as EDT (embedded deterministic testing) and OCC (on-chip clock controller), in addition to LogicBIST, MemoryBIST, in-system test, and boundary scan.
- **Scan Analysis and Insertion** — Perform scan analysis and scan chain insertion.
- **ATPG** — Generate ATPG patterns and perform fault simulation, including compression technology.

- **Defect Diagnosis and Yield Analysis** — Perform test failure diagnosis to determine a defect's most probable failure mechanism, as well as statistical analysis of diagnostic failures to find systemic defects.

Additionally, Tessent Shell provides general-purpose design editing support so that you can modify your design netlists as needed. You can work on the command line as described in “[Design Editing](#),” or use the [Tessent Visualizer](#) graphical interface.

If you are getting started with Tessent Shell, refer to “[Tessent Shell Work Flows](#)” for information about the Tessent Shell work flow for RTL and scan DFT insertion.

Tessent Shell Tcl Interface

The Tessent Shell environment uses a Tcl-based interface that you can use from the command line or by writing dofile scripts.

The Tcl interface supports Tcl constructs such as variables, command substitution, flow control, and procedures. You can embed Tcl constructs in tool commands and embed tool commands within Tcl constructs the same as with any Tcl command.

For guidelines about using Tcl in Tessent Shell, and information about converting existing dofiles and using special characters, refer to “[The Tessent Tcl Interface](#)” appendix.

Command Conventions	22
Command Completion	23
Dofile Transcription	24
Tcl Command Registration	25

Command Conventions

Tessent Shell provides a unified Tcl-style command set and naming convention. Commands that begin with a certain first word (for example, “get” in get_attribute_value_list) perform operations on the current data model.

[Table 1-1](#) provides a summary listing by command first word of the Tessent Shell commands. Refer to the [Tessent Shell Reference Manual](#) for a complete list of commands and options. You can also use the “help” command with a wildcard to see a complete list of commands that start with the same word:

```
> help get_*
```

Table 1-1. Tessent Shell Command Conventions

Command First Word	Types of Operations Performed
append_ compare_ copy_ filter_ foreach_ index_ remove_ sizeof_ sort_	Operates on collections. Refer to “ Collections ” for more information.

Table 1-1. Tessent Shell Command Conventions (cont.)

Command First Word	Types of Operations Performed
get_	Returns data strings or collections of lists, objects, or object attributes from the design object model for introspection.
read_	Reads files into memory, such as libraries and netlists.
report_	Displays information about a specific item, such as the current context or licenses currently checked out.
set_	Specifies options, contexts, modes, attributes, and the current design. Refer to “ Contexts and System Modes ” and “ Object Attributes ” for more information.
write_	Writes design data to a file or set of files.

Command Completion

In the Tessent Shell interface, you can use the Tab key to complete command names, command option names, and command option values. If the command you type is ambiguous, pressing the Tab key lists all matching commands. Additionally, within a command, pressing the Tab key can match variable names with \$.

Many Tessent Shell commands are constructed as follows:

command_name command_options command_option_values

For example:

set_config_value -in_wrapper /TopBuilder(CHIP)

Using Tab key completion, you can complete each of the command parts (name, options, and option values).

For information about Tcl shell handling in Tessent Shell, refer to the [set_tcl_shell_options](#) command.

Command Name Tab Completion

As an example of command name completion, you can abbreviate the `get_attribute_list` command in the Tessent Shell interface by typing the following:

SETUP> g_a_l <tab>

Pressing the Tab key after the string completes and expands the command so that it looks like this:

SETUP> get_attribute_list

Command Option Name Tab Completion

Tab completion for command option names is available for most commands. For example:

```
SETUP> get_config_value -m<tab>
-meta_id      -meta_name  -meta_object

SETUP> get_config_value -meta_
```

Command Option Value Tab Completion

Tab completion for command option values is available for the following value types:

- Configuration data path
- Enum (Enumerated) values

Configuration Data Path Example

In this example, pressing the Tab key on the partial configuration data path completes the path:

```
SETUP> set_config_value -in_wrapper /TopB<tab>
SETUP> set_config_value -in_wrapper /TopBuilder(CHIP)/
```

Enum Value Example

In this example, using the Tab key lists the supported time values -time_unit option to the get_config_value command:

```
SETUP> get_config_value -time_unit <tab>
as_is fs   ms   ns   ps   s   us

SETUP> get_config_value -time_unit
```

Dofile Entries

You can use abbreviated commands (for example, **g_a_l** for **get_attribute_list**) in Tessent Shell dofiles, but it is best to always use the full command name. There is no fixed minimum typing for any command or option, and current minimum typing could change in future releases due to the addition of new commands or options. Using the full command names also adds readability to your dofiles.

Dofile Transcription

By default, the transcript style is Full, which means the tool writes out all commands read from a dofile before any Tcl evaluation occurs. The command appears in the transcript as entered but is commented out.

During Tcl evaluation, the Tcl commands are written to the transcript as follows:

- The tool writes all commands from the dofile with a “// *command*.” prefix. This includes Tessent Shell commands, Tcl commands, and complete loop and if/else constructs.
- The tool writes all commands embedded in Tcl constructs to the transcript as executed with a “// *sub_command*.” prefix. Tessent Shell completes the variable and command substitutions and writes the resolved values in the loop to the transcript.

Note

This does not apply to introspection commands. Introspection commands are not written to the transcript as subcommands.

- The tool indents nested dofiles when they are written to the logfile.
- The tool can read a Tcl routine in much the same manner as a dofile. If you issue the source command (`>source my_report_env`), then the Tcl commands in the routine are not transcribed.

Control the Tessent Shell transcription behavior using the [set_transcript_style](#) command.

For information about modifying existing dofiles for use with the Tessent Shell Tcl interface, refer to “[Guidelines for Modifying Existing Dofiles for Use with Tcl.](#)”

Tcl Command Registration

You can convert any Tcl procedure into a Tessent Shell application command. This enables you to implement functionality in Tcl and register that functionality as a command so that Tessent Shell handles it like any other built-in command. Like any built-in command, newly registered Tcl commands support Tab completion, and you can control their availability relative to the context and system mode. The help system will also include the new command.

The following commands enable you to register new commands and to define arguments and options for those commands.

Table 1-2. Commands for Tcl Command Registration

Command	Description
display_message	Controls messages produced by Tcl commands.
lock_current_registration	Locks all current Tcl commands.
register_tcl_command	Registers a Tcl command.
unregister_tcl_command	Unregisters a Tcl command.

When you first execute the new command, the tool performs syntactic and semantic checks, and provides the parsed results to your Tcl implementation of the command. The tool also provides a mechanism to automatically define and register user-defined commands at tool invocation.

For more information about creating your own application commands, refer to the examples included with the [register_tcl_command](#) description in the *Tessent Shell Reference Manual*.

Chapter 2

Tool Invocation and Basic Concepts

In the Tessent Shell environment, setting the context and system mode orients the tool as to which task you will be performing. The tool uses design data models to store the relevant data about your design.

Tool Invocation	28
Contexts and System Modes	30
Design Data Models	34

Tool Invocation

When you invoke Tessent Shell, the tool starts up in setup mode.

Invoke Tessent Shell from a Linux shell using the following syntax:

```
% tessent -shell
```

To use most Tessent Shell functionality, you must load a cell library after invocation, which you can do with the [read_cell_library](#) command.

Refer to the [tessent](#) shell command description in the *Tessent Shell Reference Manual* for additional invocation options.

Tessent Startup File 28

Tessent Shell Environment Variables 28

Tessent Startup File

During invocation, Tessent Shell reads the *.tessent_startup* startup file. You can use this startup file for both general and tool-specific settings. The default location for the startup file is the home directory: *\$HOME/.tessent_startup*.

This startup file is common to the contexts listed in [Table 2-2](#) on page 30.

Tessent Shell Environment Variables

Tessent Shell provides environment variables for Tessent Shell environment operation in addition to application-specific environment variables. During invocation, the tool reads all environment variables that you have set.

[Table 2-1](#) lists the environment variables that you can set for Tessent Shell environment operation. For information about application-specific environment variables, see the [Managing Mentor Tessent Software](#) manual.

Table 2-1. Application-Specific Environment Variables

Variable	Default Value and Description
TESSENT_LICENSE_ORDER	Specifies the order that Tessent tools check out licenses. The list of licenses uses the same terminology accepted by the command “ set_context -license”.
TESSENT_STARTUP	Default: No default The pathname of a directory that contains a Tessent tool startup file.

Table 2-1. Application-Specific Environment Variables (cont.)

Variable	Default Value and Description
TESENT_TMP_LOCATION	Specifies the location at which the tool creates the <i>.tessent.tmp.hostname.process_id scratch</i> directory. For more information, see the get_scratch_directory command description in the <i>Tessent Shell Reference Manual</i> .
TESENT_UNDERSCORE_COMMANDS_ONLY	Directs the tool to only allow commands that use the underscore style. When this environment variable is set to any value, the legacy commands that used spaces are disabled. For example, the tool would accept <code>analyze_drc_violation</code> but would not accept “analyze drc violation”.

Contexts and System Modes

One of the first tasks you perform after invoking Tessent Shell is setting the context and system mode for the current session.

The term “context” refers to a broad category of functionality that often corresponds to a specific point tool, product, or license feature, such as Tessent FastScan. Each context includes several system modes, which specifies the tool’s current state of operation. By setting the context and then the system mode, you indicate the type of tasks you want Tessent Shell to perform.

In addition, you must specify the design level at which Tessent Shell will be executing tasks.

Contexts	30
System Modes	32
Context and System Mode Combinations	32
Design Levels	33

Contexts

The context specifies the functional category of tasks you want to perform with Tessent Shell.

[Table 2-2](#) lists the contexts that are currently available.

Table 2-2. Tessent Shell Contexts

Context	Description
dft	Editing and introspection of the following types of designs: gate-level Verilog, RTL Verilog, RTL System Verilog, and RTL VHDL.
dft -edt	EDT IP generation and optional insertion. This corresponds to the IP creation phase of Tessent TestKompress.
dft -logic_bist -edt	Configuration, generation, and insertion of the EDT/LBIST hybrid controller IP. This corresponds to Tessent LogicBIST.
dft -no_sub_context	Specifies that a command is only available in the dft context when no sub-context, such as -scan and -edt, was specified. See the register_tcl_command in the <i>Tessent Shell Reference Manual</i> for more information.
dft -scan	Scan analysis and scan chain insertion. This corresponds to Tessent Scan. Additionally in this context you can prepare a BIST-ready design and include tasks such as X-bounding, MCP/FP handling, and test point insertion. These features correspond to Tessent ScanPro.

Table 2-2. Tessent Shell Contexts (cont.)

Context	Description
dft -test_points	Test point identification and insertion. The test point identification algorithm focuses on either pattern count reduction or improving random pattern testability of the design.
patterns -failure_mapping	Reverse map top-level failures to the core so that you can perform diagnosis with Tessent Diagnosis. Used after performing scan pattern retargeting within the patterns -scan_retarting context.
patterns -ijtag	PDL command retargeting for IJTAG (IEEE 1687-2014) plus extraction of the ICL network from the design.
patterns -scan	Test pattern generation and good and fault simulation. This corresponds to Tessent FastScan and the test pattern generation phase of Tessent TestKompress. The patterns -scan context supports uncompressed scan ATPG/fault simulation, EDT ATPG/fault simulation, and Logic BIST fault simulation.
patterns -scan_diagnosis	Test failure diagnosis to determine a defect's failure mechanism and location. This corresponds to Tessent Diagnosis.
patterns -scan_retarting	Scan pattern retargeting for retargeting core-level test patterns at the top level.
patterns -silicon_insight	Control, simulate, debug, and characterize BIST-related memories, logic, PLLs, and SerDes. This corresponds to Tessent SiliconInsight.

Context Specification

Set the Tessent Shell context using the [set_context](#) command. For example:

```
SETUP> set_context dft -scan
```

You must set the context after you invoke Tessent Shell and before you can enter most commands. The `set_context` command is available only in setup mode. You can use the [get_context](#) command to see the current context.

Prior to setting the context, you can only run a small set of setup commands. These commands are those that you would typically place inside the startup file.

Context and Licensing

When you set the context, the tool automatically acquires the appropriate license, if available. Alternatively, you can directly specify the license Tessent Shell acquires by using the [set_context](#) command with the optional `-license` switch.

You can control the order in which Tessent Shell checks out licenses by setting the `TESSENT_LICENSE_ORDER` environment variable. You provide, as the value of the environment variable, a space-separated list of licenses using the terminology described for “`set_context -license`.” For example:

```
setenv TESSENT_LICENSE_ORDER "Scan TestKompress IJTAG"
```

Any available licenses not explicitly listed in the value of the `TESSENT_LICENSE_ORDER` environment variable are appended to the list in their original order. The tool issues a warning if the environment variable contains a license that does not exist.

For more information about specifying a license feature, refer to the [set_context](#) command description in the *Tessent Shell Reference Manual*. For a complete list of license feature names, refer to [Managing Mentor Tessent Software](#).

System Modes

A system mode in Tessent Shell defines the operational state of the tool. The default system mode is `setup`. The available system mode depends on the current context of the tool.

[Table 2-3](#) lists the available system modes.

Table 2-3. Tessent Shell System Modes

System Mode	Description
setup	Used as the entry point into the tool. Used to define the current context and specify the design information.
analysis	Used to perform design analysis, test pattern generation, PDL retargeting, and simulation.
insertion	Used to perform design editing and introspection.

Change the Tessent Shell system mode using the [set_system_mode](#) command. For example:

```
SETUP> set_system_mode insertion
```

Context and System Mode Combinations

Together, the context and system modes imply a Tessent Shell task flow from `setup` to `analysis` to `insertion` when you are working with DFT designs, and from `setup` to `analysis` when you are working with patterns. For example, if you are working on a DFT design while in `setup` mode, you cannot perform scan analysis.

The following matrix lists the actions you can perform in the available contexts and system modes. The tool filters out the functionality that does not apply to the context/mode you are currently in and issues an error when you attempt to perform a task not within the scope of the current context/system mode.

Context	Setup Mode	Analysis Mode	Insertion Mode
dft	• Read and configure design • Prepare for design editing, IP creation, scan insertion, test point analysis and insertion	• Scan analysis • Test point analysis • EDT IP creation • Hybrid TK/LBIST IP creation • MemoryBIST IP creation • Boundary scan IP creation • In-System Test IP creation	• RTL or gate-level design editing with design introspection
patterns	• Read and configure design • Define test pattern generation type to perform	• ATPG/fault simulation • PDL retargeting • Scan pattern retargeting • Execute an IJTAG test pattern • Perform SimDUT simulation	(not applicable)

The tool provides a set of commands that allow you to interact with contexts and system modes.

Table 2-4. Tessent Shell Context and System Mode Commands

Command	Description
get_context	Returns the current context as specified by the set_context command. Also returns inferred subcontexts, such as whether patterns -scan is configured to perform LogicBIST or ATPG.
set_context	Sets the current context and its options.
set_system_mode	Sets the system mode.

Design Levels

When working with DFT designs—that is, you are working in context `dft`—you must set the design level at which you are performing tasks. There is no default setting for the design level.

The available design levels are chip, physical block, and sub-block. For flat designs, you always work at the chip level. For hierarchical designs, it is crucial to differentiate whether you are work at the top-level (chip) or at lower levels within physical blocks or sub-blocks. Refer to “[Hierarchical DFT Terminology](#)” for definitions of these terms.

For complete information, refer to the [set_design_level](#) command description in the *Tessent Shell Reference Manual*.

Design Data Models

Tessent Shell has the following data models: the hierarchical design data model, the flat design data model, and the ICL data model. Each of these data models contains one or more design objects, such as pins and modules, and each design object is associated with a set of attributes, such as an ID or bit-width of a bus.

For a complete description of the various data models, object types, and attributes, refer to the “[Data Models](#)” chapter in the *Tessent Shell Reference Manual*.

Flat Design Data Model	34
Hierarchical Design Data Model	34
ICL Data Model	36
Object Attributes	36

Flat Design Data Model

The flat design data model is an internal, flattened representation of a hierarchical design that the tool creates when entering analysis mode. You can also explicitly create the flat model using the `create_flat_model` command.

The flat design data model consists of gates that are connected together. A gate is an instance of a primitive module, and a `gate_pin` object represents a pin on a gate instance. `Gate_pin` objects have no unique instance name. Instead they have a unique ID that differentiates one from another. The format of the ID is two integers separated by a period character. The first integer represents the gate ID, and the second integer represents the pin index (where 0 is the output pin, 1 is first input pin, and so on). However, this ID does not remain in place from one netlist version to the next or from one Tessent Shell invocation to the next. For this reason, you should avoid hard-coding this ID into a script.

Note



You can preserve hierarchical pins in the flat model by using the [set_attribute_options -preserve_boundary_in_flat_model](#) option.

For more information about the flat design data model and the `gate_pin` object type, refer to “[Flat Design Data Model](#)” in the *Tessent Shell Reference Manual*.

Hierarchical Design Data Model

Tessent Shell translates your design netlist into a hierarchical design data model in memory when you load the design into the tool using a command such as [read_verilog](#).

The design data remains in memory during the Tessent Shell session until you delete the model or exit the tool.

The hierarchical design data model contains the following object types:

- **Module** — A basic building block of your design. A module can be a Verilog module, a Tessent library model, or a built-in primitive.
- **Instance** — A single instantiation of a module.
- **Port** — An input, output, or inout interface of a module.
- **Pin** — An input or output interface of an instance.
- **Net** — A wire that connects the pins of instances.
- **Pseudo_port** — Represents a user-added primary input or primary output.

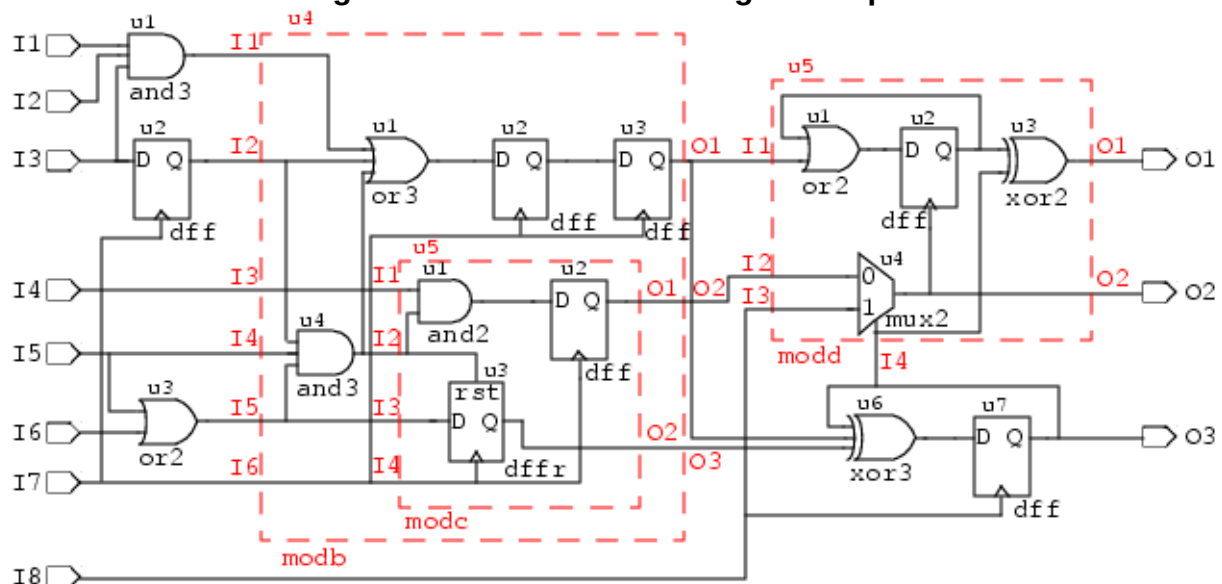
These object types are described in detail in the “[Data Models](#)” chapter of the *Tessent Shell Reference Manual*.

Use the [set_current_design](#) command to designate one module within your design as the current design. Instances, pins, and nets are hierarchical objects defined relative to the current design.

[Figure 2-1](#) shows a hierarchical design upon which many of the examples in this chapter are based. The label above the elements is the instance name while the label below the elements is the module name. For example, the module name of the AND gate in the upper-left corner is `and3`, and its instance name is `u1`.

The dashed boxes are modules instantiated in the parent module. For example, module `modc` is instantiated inside of module `modb`. Its complete instance path is `u4/u5`.

Figure 2-1. Hierarchical Design Example



ICL Data Model

The Instrument Connectivity Language (ICL) describes the elements that comprise the IJTAG network as well as their logical (though not necessarily physical) connections to each other and to the instruments at the endpoints of the network.

ICL bears a loose resemblance to a hierarchical netlist such as Verilog; it is organized by modules that may contain instances of other modules, and it describes the connections between the pins of the instances. However, it is important to note that ICL is not a complete netlist. The connections are port-to-port rather than through nets, and ICL uses abstraction rather than include the detailed physical construction of the circuitry. That is, ICL represents only the behavioral operation of the network.

The ICL data model defines objects as one of various object types, such as `icl_module`, `icl_instance`, `icl_port`, and `icl_pin`.

Each `icl_module` object has its list of objects including the following:

- `icl_port`
- `icl_instance_in_module`
- `icl_pin_in_module`
- `icl_scan_register_in_module`
- `icl_scan_mux_in_module`
- `icl_one_hot_scan_group_in_module`
- `icl_alias_in_module`
- `icl_scan_interface_of_module`

Once the current design is set using the [set_current_design](#) command, the ICL data model is elaborated downward from the current design and `icl_instance` objects are created for each instance of `icl_module`, and `icl_pin` objects are created for each port of the modules associated to the `icl_instances`. The `icl_instance` and `icl_pin` objects are hierarchical objects and therefore only exist after the current design is set. The same holds for objects of type `icl_scan_register`, `icl_scan_mux`, `icl_one_hot_scan_group`, `icl_alias`, and `icl_scan_interface`.

For more information about the ICL data model and the `icl_module`, `icl_instance`, `icl_port`, and `icl_pin` object types, refer to [“ICL Data Model”](#) in the *Tessent Shell Reference Manual*.

Object Attributes

Each design object in a design data model has a list of characteristics, called attributes, attached to that object. For example, each pin has an attribute that specifies its hierarchical name and its parent instance. There are both predefined and user-defined attributes.

Predefined attributes provide access to information known to the tool such as the name of a module or the direction of a pin. All design objects have some predefined attributes, and every design object can have user-defined attributes as well. For a complete list of attributes for each type of data model, refer to “[Data Models](#)” chapter in the *Tessent Shell Reference Manual*.

The process for creating a new user-defined attribute is called *registration*. The predefined attributes, unlike the user-defined attributes, do not need to be registered.

You must register each new user-defined attribute and specify a default value before you can use the attribute. If you want to later change the default value, you must unregister and then re-register the attribute. For more information about the registration process, refer to the description of the [register_attribute](#) command in the *Tessent Shell Reference Manual*.

You can query any attribute and change any user-defined attribute. Most predefined attributes are read-only, although some are read-write, such as the `is_hard_module` attribute.

You can filter and sort objects based on the attributes and their values. The `filter_collection` and `sort_collection` commands perform these operations. For more information about filtering attributes, refer to “[Attribute Filtering Equation Syntax](#)” in the *Tessent Shell Reference Manual*.

Intuitively named “`get_`” commands return collections of objects from the data model and the model’s attributes that you can use for design introspection of various design objects.

Chapter 3

Design Introspection and Editing

Tessent Shell provides a full range of commands for examining (introspecting) and editing designs.

Along with the command-line interface, Tessent Shell provides a graphical user interface, Tessent Visualizer, that allows you to edit and introspect designs, and adjust object attributes. For details, refer to “[Tessent Visualizer](#)” on page 379.

Design Introspection	40
Design Editing	72
Simulation Contexts.....	80

Design Introspection

Tessent Shell introspection commands allow you to examine the design objects within a design data model. They provide access to the tool's internal data structures, giving you the flexibility of Tcl scripting while keeping all compute-intensive processing in the tool's backend. The commands operate on object specifications that you include as arguments to the commands, and they return collections.

Object Specification Format	40
Collections	40
Introspection Examples	43
Design Introspection Command Summary	47
Bundle Object Introspection	49

Object Specification Format

An object specification lists the design objects (such as instances, pins, or nets) that will be acted upon by the specified command. The object specification can be the name of an object, a Tcl list of names of objects, or a collection of objects. For design objects that have unique IDs, like instances, you can use the IDs as the object names.

The following examples show the `add_schematic_objects` command with valid object specifications listed within square brackets or curly brackets.

```
add_schematic_objects [get_instances u*] -display hierarchical_schematic
```

```
add_schematic_objects [list u1 u2 u3] -display hierarchical_schematic
```

```
add_schematic_objects {u1 u2 u3} -display hierarchical_schematic
```

These commands display the results in Tessent Visualizer.

All Tessent Shell commands operating on user-specified objects accept object specifications as arguments.

Collections

Collections are an extension of Tcl that are specific to Tessent Shell applications. Native Tcl commands such as “foreach” and “puts” do not recognize collections. A collection represents a group of zero (an empty set) or more design objects.

Design introspection commands return collections of objects. The objects are stored in the internal data structures, and the tool returns a string handle to this collection. The string handle is a “@” character followed by a numeric ID (in this case, “@1”). The entire data volume remains in the tool's internal data structures, so the Tcl interface is not overloaded with large amounts of data.

Introspection commands issued directly from the shell display the “name” attribute of the first 50 elements of the collection, because the name is more useful than displaying the collection’s ID. However, names are not displayed in non-interactive mode such as when executing a dofile.

The following example demonstrates the use of the [get_instances](#) and [get_name_list](#) commands to return collections of objects:

```
SETUP>set var1 [get_instances u* -hierarchical -of_modules MOD1]
```

```
{u1 u2 u3 u4 u5 u6 u7}
```

```
SETUP>puts $var1@1
```

```
SETUP>puts [get_name_list $var1]
```

```
u1 u2 u3 u4 u5 u6 u7
```

In this example, the first command returns a collection of names. The objects are stored in internal data structures, and the tool returns a string handle to this collection which is “@” followed by a number ID (“@1”).

In the following example, the collection created by the [get_instances](#) command is stored in the variable `instCollection` (which means that the collection is referenced).

```
set instCollection [get_instances -of_type cell]
```

Tessent Shell deletes the collection when you unset the variable `instCollection` or set the variable to a new value, or when the Tcl variable(s) referring to the collection go out of scope.

In the following example, the collection created by the command [get_pins](#) is passed to the [get_attribute_value_list](#) command. This means that the collection is referenced.

```
get_attribute_value_list [get_pins u1/a*] -name direction
```

Tessent Shell deletes the collection when the command [get_attribute_value_list](#) returns a value.

Collections can refer to objects that no longer exist because they have been deleted using editing commands, such as [delete_pins](#). The built-in attribute “`is_valid`” is set to false when an object has been deleted. All commands that accept collection pointers as input automatically ignore objects with the “`is_valid`” attribute set to false.

Persistence of Collections

The tool references a collection when the collection is stored in a Tcl variable or when it is passed to a command or procedure. The tool automatically deletes a collection when it is no longer referenced.

You should not use Tcl built-in commands that expect a Tcl list, such as the `foreach` command, with collections. When you pass a collection to this type of command, the command dereferences the collection and, as a result, deletes the collection.

How to Transcript the Contents of Collections

Use the [get_name_list](#) command to return a list of the names of all the elements in the collection. So, to transcript the names in the log file, you can use the “puts” command with [get_name_list](#) (or any other Tessent [get_*](#) command):

Commands That Work With Collections or Tcl Lists

Tessent Shell provides commands that create, manipulate, and query collections. [Table 3-1](#) presents some Tessent Shell commands based on how they interact with collections.¹

Table 3-1. Commands That Interact With Collections

Commands that...	Command Name
Create collections or Tcl lists	get_attribute_list
	get_attribute_option
	get_attribute_value_list
	get_current_design
	get_fanins
	get_fanouts
	get_gate_pins
	get_instances
	get_modules
	get_name_list
	get_nets
	get_pins
	get_ports
Operate on collections	add_to_collection
	append_to_collection
	compare_collections
	copy_collection
	filter_collection
	foreach_in_collection
	index_collection
	remove_from_collection
	sort_collection

1. The table does not list all the applicable commands. Refer to the *Tessent Shell Reference Manual*.

Table 3-1. Commands That Interact With Collections (cont.)

Commands that...	Command Name
Read and write attributes of objects found within collections or Tcl lists	get_attribute_list
	get_attribute_value_list
	report_attributes
	set_attribute_value

Introspection Examples

Design introspection allows you to create procedures to show design data of interest, create and set attributes, create and manipulate collections, create custom reports, and other tasks.

Example of Creating a Procedure to Show Flip-Flop Fanouts

The following example is based on [Figure 2-1](#) on page 35 and creates a procedure called `show_fanouts` that returns the fanouts of a specified flip-flop.

```
proc show_fanouts {flop} {
    set ff_fanout [get_fanouts $flop/Q]
    puts "$flop/Q is connected to: [get_name_list $ff_fanout]"
}
```

The following two examples show how to use the `show_fanouts` procedure you just created.

> show_fanouts u2

```
u2/Q is connected to : u4/u4/A0 u4/u1/A1
```

**> foreach_in_collection ff [get_instances * -of_module dff* -hierarchical] **
{show_fanouts [get_name_list \$ff]}

```
u2/Q is connected to: u4/u1/A1 u4/u4/A0
u7/Q is connected to: O3 u6/A0 u5/u3/A1 u5/u4/S0
u4/u2/Q is connected to: u4/u3/D
u4/u3/Q is connected to: u6/A1 u5/u1/A1
u4/u5/u2/Q is connected to: u5/u4/A0
u5/u2/Q is connected to: u5/u1/A0 u5/u3/A0
u4/u5/u3/Q is connected to: u6/A2
```

Example of Displaying the Number of Input Ports

The following example displays the number of input ports on modules (objects with names that begin with “mod” in [Figure 2-1](#)).

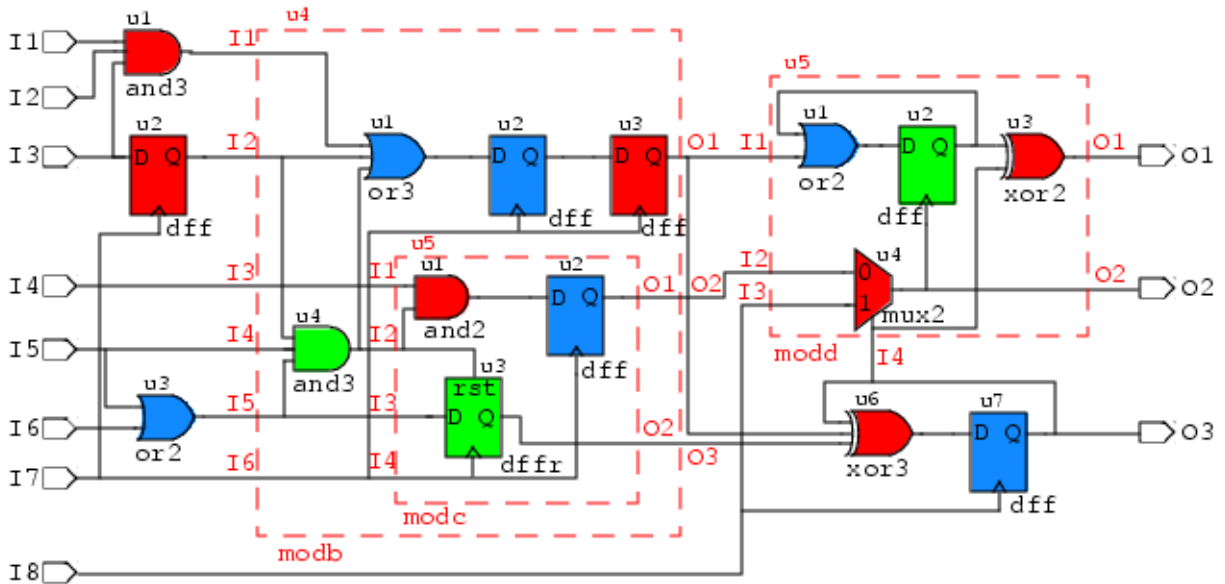
> set mods [get_modules mod* -of_type design]

**> foreach_in_collection itr \$mods {puts "The number of input ports of **
**[get_attribute_value_list \$itr -name name] **
is [sizeof_collection [get_ports -of_module \$itr -direction input]]"} }

The number of input ports of modb is 6
The number of input ports of modc is 4
The number of input ports of modd is 4

The design in Figure 3-1 is identical to Figure 2-1, with the addition of colors to help you understand the examples that follow.

Figure 3-1. Hierarchical Design Example With Colors



Example of Creating Attributes and Manipulating Collections

The following example is based on Figure 3-1 and shows the complete process of creating and setting user-defined attributes, and then creating and manipulating collections.

```
register_attribute -name color -value_type string -default "blue" -enum "red green blue" \
  -object_types instance
```

Set the “color” attribute of each instance to match Figure 3-1.

```
set_attribute_value {/u4/u4 /u4/u5/u3 /u5/u2} -name color -value "green"
```

```
{u4/u4 u4/u5/u3 u5/u2}
```

```
set_attribute_value {u1 u2 u6 /u4/u5/u1 /u4/u3 /u5/u4 /u5/u3} -name color -value "red"
```

```
{u1 u2 u6 u4/u5/u1 u4/u3 u5/u4 u5/u3}
```

```
set_attribute_value {u3 u7 /u4/u1 /u4/u2 /u4/u5/u2 /u5/u1} -name color -value "blue"
```

```
{u3 u7 u4/u1 u4/u2 u4/u5/u2 u5/u1}
```

Create collections of instances with the same color values.

```
set_all_inst [get_instances -of_type cell]
```

```
{u1 u2 u3 u6 u7 u4/u1 u4/u2 u4/u3 u4/u4 u4/u5/u1 u4/u5/u2 u4/u5/u3 u5/u1
u5/u2 u5/u3 u5/u4}
```

```
set red_inst [filter_collection $all_inst {color == "red"}]
```

```
{u1 u2 u6 u4/u3 u4/u5/u1 u5/u3 u5/u4}
```

```
set blue_inst [filter_collection $all_inst {color == "blue"}]
```

```
{u3 u7 u4/u1 u4/u2 u4/u5/u2 u5/u1}
```

```
set green_inst [filter_collection $all_inst {color == "green"}]
```

```
{u4/u4 u4/u5/u3 u5/u2}
```

Manipulate the collections and create additional collections.

```
puts "Number of red cells in the design: [sizeof_collection $red_inst]"
```

```
Number of red cells in the design: 7
```

```
puts [compare_collections $red_inst $blue_inst]
```

```
1
```

```
set red_green [add_to_collection $red_inst $green_inst]
```

```
{u1 u2 u6 u4/u3 u4/u5/u1 u5/u3 u5/u4 u4/u4 u4/u5/u3 u5/u2}
```

```
set sort_red [sort_collection $red_inst name -descending]
```

```
{u6 u5/u4 u5/u3 u4/u5/u1 u4/u3 u2 u1}
```

Example of Displaying Attribute Values of a Collection

After Tessent Shell has executed the previous commands, the following commands display the colors assigned to gates in the collection named ANDs.

```
set ANDs ""
```

```
append_to_collection ANDs [get_instances -of_type cell -filter {module_name == "AND"}]
```

```
foreach_in_collection itr $ANDs {puts "The color value for \
[get_attribute_value_list $itr -name name] is [get_attribute_value_list $itr -name color] "}
```

```
The color value for u1 is red
```

```
The color value for u4/u4 is green
```

```
The color value for u4/u5/u1 is red
```

As an alternate way to register the attribute in the previous example, you can use the `register_attribute` command as shown here:

```
register_attribute -name is_red -value_type bool -object_types "instance" \
-description "True if color is red."
```

So, instead of using the color attribute with enumerated values of red, green, and blue as shown previously, you could instead register three Boolean type attributes of `is_red`, `is_green`, and `is_blue`. This allows you to use Boolean values for filtering and tracing. For example:

```
> set red_inst [filter_collection $all_inst {is_red}]
```

Example of Changing an Attribute of a Collection

The following command example is based on [Figure 3-1](#) and shows how to change the color attribute of a collection of DFFs to green.

```
set DFFs [get_instances -of_modules dff]

# display all of the instances that have a color attribute of blue

set all_blue [filter_collection $all_inst "color == blue"]

{u7 u4/u2 u4/u5/u2}

# change all instances in the collection DFFs to have a "color" attribute value of green

set_attribute_value $DFFs -name color -value green

# now check the results
set all_green [ filter_collection [$all_inst {color == "green"}] ]

{u2 u7 u5/u2 u4/u4 u4/u2 u4/u3 u4/u5/u2 u4/u5/u3}
```

Example of Removing Duplicate Objects from a Collection

The following example shows how to create a collection containing three duplicate objects and then remove the duplicate objects from the collection.

```
set t [get_instances {u3 u3 u3}]

{u3 u3 u3}

sizeof_collection $t

3

set tt [add_to_collection "" $t -unique]

{u3}

sizeof_collection $tt

1
```

Example of Creating a Custom Report

The following dofile example creates a collection of all instance objects with a leaf name starting with “u” and then displays ten instance names per row.

```

set cnt 0
set line ""
foreach_in_collection element [get_instances u* -hierarchical] {
    if {$cnt < 10} {
        append line "[get_single_name $element] "
        incr cnt
    } else {
        puts $line
        set line ""
        append line "[get_single_name $element] "
        set cnt 1
    }
}
if {$cnt > 0} {puts $line}

```

Design Introspection Command Summary

The design introspection commands include the `get_` commands as well as various commands for manipulating attributes.

Design Introspection Command Summary

Table 3-2 lists the common design introspection commands. Refer to the *Tessent Shell Reference Manual* for more information.

Table 3-2. Design Introspection Commands

Command Name	Description
get_common_parent_instance	Returns the common ancestor of all instances, pins, or nets found in <code>object_spec</code> .
get_current_design	Returns a collection of one element that specifies the top-level design when previously set.
get_design_level	Returns the design level previously defined with the <code>set_design_level</code> command.
get_fanins	Returns a collection of all objects found in the fanin of the specified pin, net, or port objects.
get_fanouts	Returns a collection of all requested objects found in the fanout of the specified pin, net, or port object.
get_gate_pins	Returns a collection of all <code>gate_pin</code> objects found below the current design in the flat model.
get_instances	Returns a collection of instances relative to the current design.
get_modules	Returns a collection of modules.
get_nets	Returns a collection of nets relative to the current design.
get_pins	Returns a collection of pins relative to the current design.

Table 3-2. Design Introspection Commands (cont.)

Command Name	Description
get_ports	Returns a collection of ports on a given module.

Attribute Introspection Command Summary

[Table 3-3](#) lists all of the commands that introspect attributes.

Table 3-3. Attribute Introspection Commands

Command Name	Description
get_attribute_list	Lists registered attributes.
get_attribute_option	Returns the current setting of an attribute's option.
get_attribute_value_list	Returns the attribute's value.
get_name_list	Returns the name attribute of the specified objects.
get_single_name	Returns a string with the name of the element specified by the object_spec argument.
register_attribute	Registers a new attribute by adding it to the attribute manager.
report_attributes	Prints a report for the registered attributes.
reset_attribute_value	Resets the attribute to its default value.
set_attribute_options	Configures attribute options.
set_attribute_value	Defines the attribute's value.
unregister_attribute	Makes an attribute unusable by removing it from the attribute manager.

Bundle Object Introspection

The Bundle Object Hierarchy is an extension of the module instance hierarchy, and the signals (pins, ports, and nets) are considered as a tree of sub signals. The root bundle is the signal itself, the leaf objects are the bit-blasted elements of the given signal, and the nodes are the middle bundle objects.

To support this bundle object concept new object types are added to [Tessent Shell Hierarchical Design Data Model](#). The leaf objects are represented by net, pin, port object types; the sub and root bundle objects are represented by net_bundle, pin_bundle, and port_bundle objects types.

The bundle object is set of bits that represents a sub signal of a given net or port. This concept is applied to complex and simple signals for RTL and no RTL flow. The difference between simple and complex signals is in the number of levels that can be explored through the introspection commands.

Each complex signal has a select part. For example, for the following complex signal:

B [5] .c [2] .d

The portion in red ([5].c[2].d) is the select part, and the “B” is the root bundle. See “[Bundle Object](#)” on page 49 for complete details. The select part of the signals helps to explore the Bundle Object Hierarchy level-by-level.

Tessent Shell supports introspection of RTL complex signals through a bundle object, which is a set of bits that represent a sub signal of a given RTL net or port. Using Tessent Shell introspection commands allow for exploration inside the bundle object and selection of different bundle parts of the signals using the hierarchical bitselect.

Bundle Object.	49
Bundle Object Data Model	54
Introspection.	56
Introspection Examples.	61
Fanin and Fanout Tracing of Complex Signals and Bundle Object Tracing.	64
Connectivity Through Complex Signals	67

Bundle Object

A bundle object is a set of bits that represents a whole signal or its sub signals of a given RTL port or net.

The bundle is made up of Tessent Shell [Hierarchical Design Data Model](#) object types that you introspect using Tessent Shell introspection commands relative to the current design. The following illustrates this concept:

```
Root bundle (Signal itself)
|--> Node (Sub Signal)
|   |--> Leaf (Bit-blasted elements)

Example:net_bundle (Bundle Object Type)
|--> net_bundle (Bundle Object Type)
|   |--> net (Bit-blasted Object Type)

pin_bundle (Bundle Object Type)
|--> pin_bundle (Bundle Object Type)
|   |--> pin (Bit-blasted Object Type)

port_bundle (Bundle Object Type)
|--> port_bundle (Bundle Object Type)
|   |--> port (Bit-blasted Object Type)
```

The following object types contain several common attributes that are also associated with the bit-blasted object types (net, pin, and port):

- [net_bundle](#)
- [pin_bundle](#)
- [port_bundle](#)

All attributes added to port, pin, and net are also added to port_bundle, pin_bundle, and net_bundle.

For example, the naming is common between a bit-blasted object type and a bundle object type.

See “[Bundle Object Data Model](#)” on page 54 for complete information.

Naming Attributes

The following table shows the list of the principal name attributes associated with net, port, and pin object types.

Attribute	Description	RTL View	Synthesized View
name	The active name. The name depends on the selected view. This includes the select part in the case of signals (port, pin, and net). The -rtl mode has the RTL view and the synthesized view. The -no_rtl mode only has the netlist view.	The full RTL name including all the select part.	The full synthesis name including all the select part (flatten position in case of complex signals).
parent_bundle_name	The bundle parent of the port, pin, or net.	The parent name of the selected RTL object.	The parent name of the selected synthesized object.
post_synthesis_name	The synthesis name, created by the quick synthesis.	The computed equivalent post synthesis name.	Equal to “name” attribute.
pre_synthesis_name	RTL name, the original name before synthesis.	Equal to “name” attribute.	Full RTL name, the original name before synthesis.
root_bundle_name	The active name without the select part.	The base RTL name without the select part.	The base synthesis name without the select part.

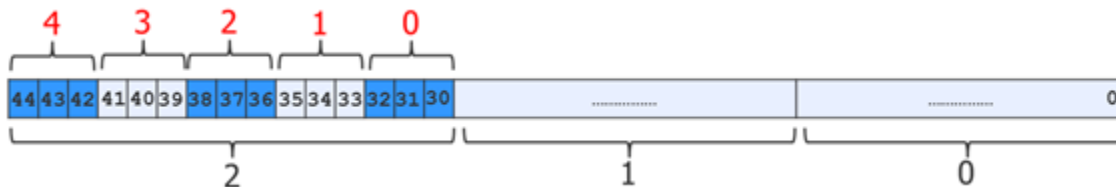
Bundle Naming Examples

Example 1

The `post_synthesis_name` attribute for complex signals is usually the base name with the flatten position as the index. System Verilog interface signals are the exception. For example, in case of the following input port “data_in” declared as array of structure in System Verilog:

```
typedef struct{
    logic a;
    logic [1:0] b;
} data_t;
input data_t [2:0][4:0] data_in;
```

The following figure illustrates the representation in the flatten dimension:



Name Attribute (RTL View)	post_synthesis_name Attribute
data_in[2][1].a	data_in[35]
data_in[2][1].b	data_in[34:33]
data_in[2][1].b[1]	data_in[34]

The following table shows the different naming of the signal “data_in[2][1].b[1]”

Attribute	RTL View	Synthesis View
name	data_in[2][1].b[1]	data_in[34]
pre_synthesis_name	data_in[2][1].b[1]	data_in[2][1].b[1]
post_synthesis_name	data_in[34]	data_in[34]
root_bundle_name	data_in	data_in

Example 2

In this example, the “net1” from Example 1 is declared inside nested generate blocks:

```
typedef struct{
    logic a;
    logic [1:0] b;
} data_t;

genvar i, j;
generate
    for (i=0; i < 2; i=i+1) begin
        for (j=0; j < 2; j=j+1) begin : MEM_j
            data_t [2:0] [4:0] net1;
            .....
        end
    end
endgenerate
```

The following table shows the different naming of “genblk1[0].MEM_j[0].net1[2][1].b[1]”

Attribute	RTL View	Synthesis View
name	genblk1[0].MEM_j[0].net1[2][1].b[1]	\genblk1[0].MEM_j[0].net1 [34]

Attribute	RTL View	Synthesis View
pre_synthesis_name	genblk1[0].MEM_j[0].net1[2][1].b[1]	genblk1[0].MEM_j[0].net1[2][1].b[1]
post_synthesis_name	\genblk1[0].MEM_j[0].net1 [34]	\genblk1[0].MEM_j[0].net1 [34]
root_bundle_name	genblk1[0].MEM_j[0].net1	\genblk1[0].MEM_j[0].net1

Example 3

This example illustrates an interface declared as a net to connect two sub instances.

```

interface intf;
  logic a;
  logic b;
  modport in (input a, output b);
  modport out (input b, output a);
endinterface

module top;
  intf i ();
  u_a m1 (.i1(i));
  u_b m2 (.i2(i));
endmodule

module u_a(intf.in i1);
endmodule
module u_b(intf.out i2);
endmodule

```

The following tables shows the different naming of the signals “i.a” and “m1/i1.a”.

For net: “i.a”

Attribute	RTL View	Synthesized View
name	i.a	\i.a
object_type	net	net
pre_synthesis_name	i.a	i.a
post_synthesis_name	\i.a	\i.a
root_bundle_name	i	i

For pin: “m1/i1.a”

Attribute	RTL View	Synthesized View
name	m1/i1.a	m1/i1.a
object_type	pin	pin
pre_synthesis_name	m1/i1.a	m1/i1.a
post_synthesis_name	m1/i1.a	m1/i1.a

Attribute	RTL View	Synthesized View
root_bundle_name	m1/i1	m1/i1.a

Example 4

The values of the name attributes are language independent and have the same values for both VHDL and Verilog RTL objects. The following example in VHDL uses the record datatype:

```
library ieee;
package record_pkg is

    -- Outputs from the FIFO.
    type t_FROM_FIFO is record
        wr_full  : std_logic;           -- FIFO Full Flag
        rd_empty : std_logic;           -- FIFO Empty Flag
        rd_dv    : std_logic;
        rd_data  : std_logic_vector(7 downto 0);
    end record t_FROM_FIFO;
    .....
end package record_pkg;
use work.record_pkg.all; -- USING PACKAGE HERE!
entity top is
    port (
        i_clk  : in  std_logic;
        i_fifo : in  t_FROM_FIFO;
        ..... );
end top;

architecture behave of top is

begin
    .....
end behave;
```

The following table shows the different naming of the port “i_fifo.rd_data(5)” in VHDL. VHDL uses “()” to represent the index, but Tessent Shell uses “[]” instead.

Attribute	RTL View	Synthesis View
name	i_fifo.rd_data[5]	i_fifo[5]
pre_synthesis_name	i_fifo.rd_data[5]	i_fifo.rd_data[5]
post_synthesis_name	i_fifo[5]	i_fifo[5]
root_bundle_name	i_fifo	i_fifo

Bundle Object Data Model

Bundle objects utilize the Tessent Shell Hierarchical Design Data Model object types.

For each object type, there are attributes associated with the design objects found on, in, and below the current design (set with the [set_current_design](#) Tessent Shell command). A bundle object has a select part and type information. Otherwise a bit-blasted object has a bit select and also type information. An object type can be a complex type in case of bundle object, like struct, enum, typedef, multi-dimensional array. In the case of a bit-blasted object it is always a 1 bit datatype. It can be register, wire, logic, bit, tri, for example. Several attributes are associated with the following Hierarchical Design Data Model object types:

- [Net](#)
- [Net Bundle](#)
- [Pin](#)
- [Pin Bundle](#)
- [Port](#)
- [Port Bundle](#)

Each of these object types is covered in detail in the [Tessent Shell Reference Manual](#).

base_class and base_type Attributes

Each bundle object type contains a base_class and base_type attribute as follows:

- **base_class** — String character that represents the class category of the base type. It helps to complete the base_type definition. Possible values are: = 4_state_signed | 4_state_unsigned | 2_state_signed | 2_state_unsigned | enum | struct_packed | struct_unpacked | union_backed | union_unpacked | interface | interface_modport | other.
- **base_type** — String character that represents the base type of the selected object. The base type is the type of the object without any dimensions. It represents the base type of the sub bundle type if the selected object is a sub bundle object. It can be a base type of the root type if the selected object is a root bundle. The base_type is a key or user name that help to understand the base type of this bundle. Possible values are: logic | register | wire | bit | supply0 | supply1 | tri | triand | trior | tri0 | tri1 | wand | wor | byte | shortint | integer | longint | *user_name* | other

The following examples shows the values for base_class and base_type bundle object type attributes in System Verilog:

Table 3-4. base_class and base_type System Verilog Examples

Examples	base_type and base_class Values
Packed struct having typedef name “data_t”	base_type= data_t, base_class= struct_packed
Unpacked struct having typedef name “data_t”	base_type= data_t, base_class= struct_unpacked

Table 3-4. base_class and base_type System Verilog Examples (cont.)

Examples	base_type and base_class Values
shortint	base_type = shortint, base_class = 2_state_signed
unsigned shortint	base_type = shortint, base_class = 2_state_unsigned
integer	base_type = integer, base_class = 4_state_signed
int	base_type = integer, base_class = 2_state_signed
unsigned integer	base_type = integer, base_class = 4_state_unsigned
unsigned int	base_type = integer, base_class = 2_state_unsigned
reg	base_type = register, base_class = 4_state_unsigned
logic	base_type= logic, base_class= 4_state_unsigned
bit	base_type= logic, base_class= 2_state_unsigned
supply0, supply1	base_type= supply0 or supply1, base_class = 2_state_unsigned
tri, wand, triand, wor, trior, tri0, tri1	base_type = tri, wand, triand.... For all those datatypes, the base_class=4-state_unsigned
module top (INTF1 Bus)	bundle “Bus” has as base_type=INTF1, base_class=interface
module top (INTF1.slave Bus)	bundle “Bus” has as base_type=INTF1.slave, base_class=interface_modport

Net Bundle

The Net Bundle object type has net_bundle datatype and attributes. For complete details, refer to [net_bundle](#) in the *Tessent Shell Reference Manual*.

Pin Bundle

The Pin Bundle object type has pin_bundle datatype and attributes. For complete details, refer to [pin_bundle](#) in the *Tessent Shell Reference Manual*.

Port Bundle

The Port Bundle object type has port_bundle datatype and attributes. For complete details, refer to [port_bundle](#) in the *Tessent Shell Reference Manual*.

Introspection

Using bundle object types, Tessent Shell allows you to introspect and explore any kind of signals within the design data model.

From a simple datatype to a complex datatype both can be explored using the introspection commands listed in “[Complex Signal Introspection Commands](#)” on page 60.

Any signal objects (ports, pins, and nets) are considered as a tree of sub signals. The given tree is referred to as a Bundle Object Hierarchy. The root bundle is the signal itself; the leaf objects are the bit-blasted elements of the given signal; the root, leaf, and middle bundle are all nodes of the tree. The middle bundles and the leaf objects are the sub bundle objects. To support this concept there are object types in the Hierarchical Design Data Model. The leaf objects are represented by net, pin, and port object types, the root and middle bundles are represented by net_bundle, pin_bundle, and port_bundle objects types. See “[Bundle Object Data Model](#)” on page 54.

The concept of bundle object type is applied to complex and simple signals for RTL and no RTL flow. The difference between simple and complex signals is in the number of levels that can be explored through the introspection commands. The select part of the signals helps to explore the Bundle Object Hierarchy level by level.

Tip

When performing introspection of complex signals, see also “[Object Specification Format](#)” on page 40 and “[Collections](#)” on page 40.

How Complex Signal Introspection Works

Consider the bundle hierarchy of the “data_in” port as follows:

```
typedef struct{
    logic a;
    logic [1:0] b;
} data_t;
input data_t [1:0][2:0] data_in;
```

The following shows the hierarchy of the “data_in” port in the Tessent Shell design data model.

```
Root bundle "data_in"
|---> data_in[1]
|   |---> data_in[1][2]
|   |   |---> data_in[1][2].a
|   |   |---> data_in[1][2].b
|   |       |---> data_in[1][2].b[1]
|   |       |---> data_in[1][2].b[0]
|   |---> data_in[1][1]
|   |   |---> data_in[1][1].a
|   |   |---> data_in[1][1].b
|   |       |---> data_in[1][1].b[1]
|   |       |---> data_in[1][1].b[0]
|   |---> data_in[1][0]
|   |   |---> data_in[1][0].a
|   |   |---> data_in[1][0].b
|   |       |---> data_in[1][0].b[1]
|   |       |---> data_in[1][0].b[0]
|---> data_in[0]
|   |---> data_in[0][2]
|   |   |---> data_in[0][2].a
|   |   |---> data_in[0][2].b
|   |       |---> data_in[0][2].b[1]
|   |       |---> data_in[0][2].b[0]
|   |---> data_in[0][1]
|   |   |---> data_in[0][1].a
|   |   |---> data_in[0][1].b
|   |       |---> data_in[0][1].b[1]
|   |       |---> data_in[0][1].b[0]
|   |---> data_in[0][0]
|   |   |---> data_in[0][0].a
|   |   |---> data_in[0][0].b
|   |       |---> data_in[0][0].b[1]
|   |       |---> data_in[0][0].b[0]
```

Referring to the figure, using the *name_patterns* argument of the introspection commands, the tool splits each pattern into the different levels so that a local sub-pattern is applied to each local level.

For the bundle part, the characters “.” and “[<index>]” allow access to sub bundle hierarchy, “:” to access field sub bundle objects of struct or SVinterface and “[<index>]” to access array element sub bundle objects. For example, the pattern “data_in[0][1].b[*]” is really five sub-patterns; “data_in”, “[0]”, “[1]”, “.b”, and “[*]”, representing five levels of bundle hierarchy.

For simple wildcard patterns, the “*” is used the exact same way as it is used in instance path matching. It does not cross hierarchy. When specified within a field name it does not cross the “.” that precedes or follows the field. When applied within [], it only matches elements within the []. In other words delimiters must be explicit.

For regular expression patterns, an individual regular expression only matches elements within the sub-pattern in which it appears. Again, it does not cross the adjacent delimiters.

Without the -regexp switch in the introspection command, the bundle hierarchy delimiters (“[” and “.”) are treated as normal delimiters and serve to break the pattern into multiple sub-

patterns. When users add the `-regex` switch in the introspection command, then, unless those delimiters are escaped, they will be considered as regular expression meta-characters. Stated differently, in `regex` mode the delimiters must be explicit and, unlike simple wildcard patterns, they must be escaped.

For example, to match `data_in[1]`, you need to use `data_in\[.*\]`. The sequence `.*` matches any character zero or more times. For more details, see the [Example With regex](#).

See “[Glob and Regular Expression Pattern Matching Syntax](#)” in the *Tessent Shell Reference Manual* for complete information on pattern matching.

Note

The examples use the “`get_ports ... -bundle`” command. For “`get_nets ... -bundle`” and “`get_pins ... -bundle`”, the behavior is the same.

Examples

These examples use the “`data_in`” hierarchy above.

```
SETUP> get_ports data_* -bundle
```

```
data_in
```

```
SETUP> get_ports data_in* -bundle
```

```
data_in
```

```
SETUP> get_ports data_in[*] -bundle
```

```
{data_in[1]} {data_in[0]}
```

```
SETUP> get_ports data_in[0][*] -bundle
```

```
{data_in[0][2]} {data_in[0][1]} {data_in[0][0]}
```

```
SETUP> get_ports data_in[0] -bundle
```

```
{data_in[0]}
```

```
SETUP> get_ports data_in[0][*].b -bundle
```

```
{data_in[0][2].b} {data_in[0][1].b} {data_in[0][0].b}
```

```
SETUP> get_ports data_in[1][2].* -bundle
```

```
{data_in[1][2].a} {data_in[1][2].b}
```

```
SETUP> get_ports data_in[1][*] -bundle
```

```
{data_in[1][2]} {data_in[1][1]} {data_in[1][0]}
```

Note

 Integer, enum, and others are bit-blasted in unbundled objects in introspection commands when the -bundle switch is omitted.

Example With regexp

In the following example the second command call is equivalent to the first, but uses -regexp. You should enclose the regexp in double braces ({{ }}) to avoid having to use the “\\” everywhere, that is, “{{data_in\[.*\]\.da.*\[.*\]\.}}”.

With wildcard syntax:

```
SETUP> puts [join [get_name_list [get_ports data_in[*].da*[*].* ]] "\n"]
```

```
data_in[1].data[1].data1[1][1]
data_in[1].data[1].data1[1][0]
data_in[1].data[1].data1[0][1]
data_in[1].data[1].data1[0][0]
data_in[1].data[1].data2[1][2]
...
data_in[0].data[0].data2[0][2]
data_in[0].data[0].data2[0][1]
data_in[0].data[0].data2[0][0]
```

With regular expression syntax:

```
SETUP> puts [join [get_name_list [get_ports {{data_in\[.*\]\.da.*\[.*\]\.}} -regexp]] "\n"]
```

```
data_in[1].data[1].data1[1][1]
data_in[1].data[1].data1[1][0]
data_in[1].data[1].data1[0][1]
data_in[1].data[1].data1[0][0]
data_in[1].data[1].data2[1][2]
...
data_in[0].data[0].data2[0][2]
data_in[0].data[0].data2[0][1]
data_in[0].data[0].data2[0][0]
```

Complex Signal Introspection Commands

The following commands support introspection of complex signals:

Table 3-5. Complex Signals Design Introspection Commands

Command Name	Description/Usage
get_bundle_objects	Returns a collection of bundle/leaf objects in the current design specified by the name_patterns list in conjunction with the specified options. Those bundle objects (leaf in case of descendants) will be ancestors or descendants of the objects which match the specified name_patterns depending upon the specified options.

Table 3-5. Complex Signals Design Introspection Commands (cont.)

Command Name	Description/Usage
<code>get_nets ... -bundle</code>	When the <code>-bundle</code> switch is specified, constrains the tool to return net bundle/leaf objects. When the pattern matches a bundle, the command returns the bundle. When it matches leaf objects, it returns those leaf objects.
<code>get_pins ... -bundle</code>	When the <code>-bundle</code> switch is specified, constrains the tool to return pin bundle/leaf objects. When the pattern matches a bundle, the command returns the bundle. When it matches leaf objects, it returns those leaf objects.
<code>get_ports ... -bundle</code>	When the <code>-bundle</code> switch is specified, constrains the tool to return port bundle/leaf objects. When the pattern matches a bundle, the command returns the bundle. When it matches leaf objects, it returns those leaf objects.

Introspection Examples

In the following System Verilog code, several kinds of datatypes are illustrated: struct, array of interface, enumeration.

```
typedef enum logic [1:0] {
    STATE_1, STATE_2, STATE_3, STATE_4, STATE_5, STATE_6
} State_t;

typedef struct packed{
    logic [2:0] eventbus;
} event_bus_packet_t;

typedef struct packed{
    State_t state_encoded;
    event_bus_packet_t dt_lcp_pulse;
    logic [3:0] eventbus_encoded;
} event_bus_split_packet_t;

interface MSBus;
    event_bus_split_packet_t Addr;
    logic [1:0] Data;
    logic RWn;
    logic Clk;
    modport Slave (input Addr, RWn, Clk, output Data);
endinterface

module top(MSBus.Slave Bus[2], input wire i_clk);
    RAM TheRAM (.MemBus(Bus));
endmodule
```

Example 1

The following command returns all bit-blasted ports of “top” module (default behavior):

SETUP> get_ports -of_modules top

```
{Bus[0].Addr.state_encoded[2]} {Bus[0].Addr.state_encoded[1]}
{Bus[0].Addr.state_encoded[0]} {Bus[0].Addr.dt_lcp_pulse.eventbus[2]}
{Bus[0].Addr.dt_lcp_pulse.eventbus[1]}
{Bus[0].Addr.dt_lcp_pulse.eventbus[0]} {Bus[0].Addr.eventbus_encoded[3]}
{Bus[0].Addr.eventbus_encoded[2]} {Bus[0].Addr.eventbus_encoded[1]}
{Bus[0].Addr.eventbus_encoded[0]} {Bus[0].RWn} {Bus[0].Clk}
{Bus[0].Data[1]} {Bus[0].Data[0]}
{Bus[1].Addr.state_encoded[2]} {Bus[1].Addr.state_encoded[1]}
{Bus[1].Addr.state_encoded[0]} {Bus[1].Addr.dt_lcp_pulse.eventbus[2]}
{Bus[1].Addr.dt_lcp_pulse.eventbus[1]}
{Bus[1].Addr.dt_lcp_pulse.eventbus[0]} {Bus[1].Addr.eventbus_encoded[3]}
{Bus[1].Addr.eventbus_encoded[2]} {Bus[1].Addr.eventbus_encoded[1]}
{Bus[1].Addr.eventbus_encoded[0]} {Bus[1].RWn} {Bus[1].Clk}
{Bus[1].Data[1]} {Bus[1].Data[0]}
```

Example 2

The following commands return all root bundle ports of “top” module or sub bundle objects according to name patterns used:

SETUP> set b [get_ports -of_modules top -bundle]

Bus i_clk

SETUP> get_ports Bus -bundle

Bus

SETUP> get_ports B* -bundle

Bus

SETUP> get_ports Bus* -bundle

Bus

SETUP> get_ports Bus[0].* -bundle

```
{Bus[0].Addr} {Bus[0].RWn} {Bus[0].Clk} {Bus[0].Data}
```

SETUP> get_ports Bus[0].Addr* -bundle

```
{Bus[0].Addr}
```

SETUP> get_ports Bus[0].Addr.* -bundle

```
{Bus[0].Addr.state_encoded} {Bus[0].Addr.dt_lcp_pulse.eventbus}
{Bus[0].Addr.eventbus_encoded}
```

SETUP> get_ports Bus[0].Addr.*

```
{Bus[0].Addr.state_encoded[2]} {Bus[0].Addr.state_encoded[1]}  
{Bus[0].Addr.state_encoded[0]} {Bus[0].Addr.dt_lcp_pulse.eventbus[2]}  
{Bus[0].Addr.dt_lcp_pulse.eventbus[1]}  
{Bus[0].Addr.dt_lcp_pulse.eventbus[0]} {Bus[0].Addr.eventbus_encoded[3]}  
{Bus[0].Addr.eventbus_encoded[2]} {Bus[0].Addr.eventbus_encoded[1]}  
{Bus[0].Addr.eventbus_encoded[0]}
```

SETUP> get_ports Bus[*] -bundle

```
{Bus[0]} {Bus[1]}
```

SETUP> get_ports Bus[*].* -bundle

```
{Bus[0].Addr} {Bus[0].RWn} {Bus[0].Clk} {Bus[0].Data} {Bus[1].Addr}  
{Bus[1].RWn} {Bus[1].Clk} {Bus[1].Data}
```

SETUP> get_ports Bus[0]

```
{Bus[0].Addr.state_encoded[2]} {Bus[0].Addr.state_encoded[1]}  
{Bus[0].Addr.state_encoded[0]} {Bus[0].Addr.dt_lcp_pulse.eventbus[2]}  
{Bus[0].Addr.dt_lcp_pulse.eventbus[1]}  
{Bus[0].Addr.dt_lcp_pulse.eventbus[0]} {Bus[0].Addr.eventbus_encoded[3]}  
{Bus[0].Addr.eventbus_encoded[2]} {Bus[0].Addr.eventbus_encoded[1]}  
{Bus[0].Addr.eventbus_encoded[0]} {Bus[0].RWn} {Bus[0].Clk}  
{Bus[0].Data[1]} {Bus[0].Data[0]}
```

SETUP> set b1 [get_bundle_objects \$b -children]

```
{Bus[0]} {Bus[1]}
```

SETUP> set b2 [get_bundle_objects [index_collection \$b1 0] -children]

```
{Bus[0].Addr} {Bus[0].RWn} {Bus[0].Clk} {Bus[0].Data}
```

SETUP> get_bundle_objects -of_objects {Bus[0].Addr} -root

```
{Bus}
```

SETUP> get_bundle_objects -of_objects {Bus[0].Addr} -leaf

```
Bus[0].Addr.state_encoded[2]} {Bus[0].Addr.state_encoded[1]}  
{Bus[0].Addr.state_encoded[0]} {Bus[0].Addr.dt_lcp_pulse.eventbus[2]}  
{Bus[0].Addr.dt_lcp_pulse.eventbus[1]}  
{Bus[0].Addr.dt_lcp_pulse.eventbus[0]} {Bus[0].Addr.eventbus_encoded[3]}  
{Bus[0].Addr.eventbus_encoded[2]} {Bus[0].Addr.eventbus_encoded[1]}  
{Bus[0].Addr.eventbus_encoded[0]}
```

SETUP> set b3 [get_bundle_objects [index_collection \$b2 0] -children]

```
{Bus[0].Addr.state_encoded} {Bus[0].Addr.dt_lcp_pulse}  
{Bus[0].Addr.eventbus_encoded}
```

SETUP> get_attribute_value_list [\$b3] -name base_class

```
enum struct_packed 4-state_unsigned
```

SETUP> get_attribute_value_list [\$b3] -name select_part_value_list

```
{0 Addr state_encoded} {0 Addr dt_lcp_pulse} {0 Addr eventbus_encoded}
```

```
SETUP> get_ports {Bus[0].Clk} -bundle
{Bus [0] .Clk}

SETUP> get_ports {Bus[0].Addr.state_encoded}
{Bus [0] .Addr.state_encoded[2]} {Bus [0] .Addr.state_encoded[1]}
{Bus [0] .Addr.state_encoded[0]}

SETUP> get_bundle_objects -of_objects {Bus[0].Addr.state_encoded} -parent
{Bus [0] .Addr}

SETUP> get_bundle_objects -of_objects {Bus[0].Addr} -parent
{Bus [0] }

SETUP> get_bundle_objects -of_objects {Bus[0]} -parent
{Bus}

SETUP> get_bundle_objects -of_objects {Bus} -parent
{ }
```

Fanin and Fanout Tracing of Complex Signals and Bundle Object Tracing

Using Tessent Shell, you can introspect and trace any kind of signals (simple and complex) through their fanins and fanouts. It is possible also to trace through bundle objects. The `get_fanins/get_fanouts` commands accept as input any kind of objects, bundle or not and with simple types or complex types. The ending and intermediate nodes can also be an object or complex type. For example, they can trace from and to, and through any kind of complex signals in System Verilog and VHDL.

The following commands support this introspection:

- [get_fanins](#)
- [get_fanouts](#)

The following shows a top-level RTL module (*top1.sv*) with SV Interface objects. The tool models SV Interfaces as a named bundle of wires. Within the tool, those are considered as normal bundle ports, pins, and nets. You access the bundle object using the [get_nets](#), [get_pins](#), and [get_ports](#) commands.


```
module top (clk, reset, ce, we, addr, datai, datao, cnto, datao2, en);

    localparam WID = 8;
    localparam AWID = 5;
    localparam LEN = 2**AWID;
    localparam GEN = 3;

    input en;
    input clk, reset, ce, we;
    input [AWID-1:0] addr;
    input [WID-1:0] datai;
    output [GEN*12-1:0] datao, datao2;
    output [GEN*WID-1:0] cnto;

    and enAnd2 (resetEn, reset, en);
    and enAnd3 (weEn, we, en);
    and enAnd4 (ceEn, ce, en);

    param_mem_if miff(
        .clk(clk),
        .reset(resetEn),
        .we(weEn),
        .ce(ceEn),
        .dati(datai),
        .addr(addr),
        .dato(datao)
    );

    genvar i;

    generate
        for (i=0; i<GEN; i++) begin: gen_mem
            SYNC_1RW_16x8 mem1 (.CLK(clk), .D(datai), .Q(datao2[(WID*(i+1)
                -1):i*WID]), .WE(we), .OE(ce), .RE(ce), .A(addr[3:0]));
        end
    endgenerate

    generate
        for (i=0; i<GEN; i++) begin: gen_mem_wrapper

            param_mem mem_inst (.mif(miff));
        end
    endgenerate

    generate
        for (i=0; i<GEN; i++) begin: gen_cnt
            param_counter #(.WID(WID)) counter_inst (
                .clk(clk),
                .reset(reset),
                .enable(ce&we),
                .count(cnto[(WID*(i+1)-1):i*WID]));
        end
    endgenerate
endmodule
```

The following shows an example command sequence to introspect the fanins and fanouts in this design:

```
SETUP> set_context dft -rtl
SETUP> read_cell_library my_cell_library.lib
SETUP> set_design_sources -format verilog -y {verilog_source_directory} -ext {v}
SETUP> set_design_sources -format tcd_memory -y {verilog_source_directory} \
-ext {lvlib lib}
SETUP> read_verilog param_mem.sv -format sv2009
SETUP> read_verilog param_counter.sv -format sv2009
SETUP> read_verilog top1.sv -format sv2009
SETUP> set_current_design top
SETUP> set_design_level physical_block
SETUP> add_input_constraints en -C1
SETUP> puts [join [get_attribute_value_list \
[get_fanins gen_mem_wrapper[0].mem_inst/mem1/CLK ] -name name] "\n"]

clk
```

Here, from SV Interface sub bundle pins, the tool will reach the output pin of primitive. (Continuing from the previous example.)

```
SETUP> puts [join [get_name_list \
[get_fanouts gen_mem_wrapper[2].mem_inst/mif.ce ]] "\n"]

enAnd4/OUT

SETUP> puts [join [get_name_list \
[get_fanouts gen_mem_wrapper[2].mem_inst/mif.ce ] -name name] "\n"]

gen_mem_wrapper[2].mem_inst/mem1/OE
gen_mem_wrapper[2].mem_inst/mem1/RE
gen_mem_wrapper[2].mem_inst/mem2/OE
gen_mem_wrapper[2].mem_inst/mem2/RE
```

Here, from SV Interface sub bundle pins, the tool will reach the SV interface sub bundle net “miff.ce”. (Continuing with the previous example.)

```
SETUP> puts [join [get_name_list \
[get_fanins gen_mem_wrapper[2].mem_inst/mif.ce -stop_on net] \
-name name] "\n"]

miff.ce

SETUP> puts [join [get_name_list \
[get_fanouts gen_mem_wrapper[2].mem_inst/mif.ce -stop_on net] -name name] "\n"]

gen_mem_wrapper[2].mem_inst/mif.ce
```

Additional Examples

The following command sequence shows introspection of bundle objects through both fanins and fanouts as well as ports and pins:

```
SETUP> set bitblasted_port [get_ports jack]
SETUP> set bitblasted_conn [get_fanouts $bitblasted_port]
SETUP> puts [join [get_name_list $bitblasted_conn] "\n"]
```

```
i1/jim[2]
i2/jim[1]
i1/jim[1]
i1/joe[2]
i1/jim[0]
i2/joe[2]
```

```
SETUP> set bundle_port [get_ports jack -bundle]
SETUP> set bundle_conn [get_fanouts $bundle_port]
SETUP> puts [join [get_name_list $bundle_conn] "\n"]
```

```
i1/jim
```

```
SETUP> set bitblasted_pin [get_pins i1/jim]
SETUP> set bitblasted_conn [get_fanins $bitblasted_pin]
SETUP> puts [join [get_name_list $bitblasted_conn] "\n"]
```

```
jack[2]
jack[1]
jack[0]
```

```
SETUP> set bundle_pin [get_pins i1/jim -bundle]
SETUP> set conn [get_fanins $bundle_pin]
SETUP> puts [join [get_name_list $bundle_conn] "\n"]
```

```
jack
```

Connectivity Through Complex Signals

Prior to the Tessent 2019.3 release, the connectivity described through complex RTL pins and nets were not visible until the RTL module were synthesized using the Quick Synthesis engine.

During ICL extraction or pre-DFT DRC, the fanin and fanout of objects to be checked were pre-traced internally and if any complex objects were found along the way, such as traversing a net or a pin that is not a simple scalar or one dimensional bus, the module was synthesized to expose the connectivity to the DRC engine. As soon as a module with a System Verilog interface was synthesized, all modules connected to this interface also needed to be synthesized which often resulted in having to synthesize the majority of the design.

With the Tessent 2019.3 release and later, the connectivity through complex signals is visible to the [get_fanins/get_fanouts](#) commands on the hierarchical design view and to the [trace_flat_model](#) command on the flat model. Only if the path pre-tracing found RTL logic that represents logic gates does the tool synthesize the module. Also, the tool is now able to maintain the RTL view, and the connectivity to and from the surrounding modules even if a module

having an SV interface needs to be synthesized. This greatly limits the amount of RTL modules needed to be synthesized to pre-DFT DRC memoryBIST and performing ICL extraction.

When Tessent Shell flattens the design, only the leaf objects of complex signals are translated to gate pins using the post synthesis names (see the `post_synthesis_name` attribute on the [Port](#), [port_bundle](#), [Pin](#), [pin_bundle](#), [Net](#), and [net_bundle](#) object types for more information).

For objects found inside unsynthesized modules, the hierarchical design objects uses the pre-synthesis names (see the `pre_synthesis_name` attribute on the [Port](#), [port_bundle](#), [Pin](#), [pin_bundle](#), [Net](#), and [net_bundle](#) object types for more information).

The gate pins objects, however, are always named using the post synthesis name of the leaf complex signal objects whether or not the module is synthesized or not.

To simplify the introspection between the hierarchical design objects and the flat model gate_pin objects, the tool provides several switches on the introspection commands. With complex signal, it is no longer possible to assume the two objects have the same name.

- [get_gate_pins](#) -of_pins *pin_objects* | -of_ports *port_objects* | -of_nets *net_objects*
- [get_nets](#) [*name_patterns*] -of_gate_pins *gate_pin_objects*
- [get_pins](#) [*name_patterns*] -of_gate_pins *gate_pin_objects*
- [get_ports](#) [*name_patterns*] -of_gate_pins *gate_pin_objects*

The following example demonstrates loading an RTL design and performing tracing on the hierarchical data module using the [get_fanins](#) command. It also shows creating and tracing the flat model using the [trace_flat_model](#) command:

```
SETUP> set_context dft -rtl
SETUP> read_cell_library my_cell_library.lib
SETUP> open_tsdb InternalCore_tsdb
SETUP> read_design InternalCore -design_id rtl
SETUP> read_verilog RTL_Directory/Interfaces.sv -format sv2009
SETUP> read_verilog RTL_Directory/Core.sv -format sv2009
SETUP> set_current_design core
SETUP> set_design_level physical_block
SETUP> puts [get_name_list [get_fanins genloop[2].icw0/ic/OutBus[1]]]

{ InBus [1] }

SETUP> create_flat_model
// Flattening process completed, gates=79, PIs=12, POs=13, CPU time=0.00 sec.
// -----
// Begin circuit learning analyses.
// -----
// Learning completed, CPU time=0.00 sec.

SETUP> puts [get_name_list [trace_flat_model -from [get_gate_pins -of_pins \
[get_pins genloop[2].icw0/ic/OutBus[1]]] -direction backward \
-controllability connected]]
```

```
{InBus[1]}
```

This example shows the use of the “[get_gate_pins](#) -of_pin” and “[get_pins](#) -of_gate_pins” to illustrate that the matching objects do not have the same name:

```
SETUP> get_gate_pins -of_pins [get_pins genloop[2].icw0/ic/OutBus[1]]
```

```
{{\genloop[2].icw0 /ic/OutBus[1]}}
```

```
SETUP> get_gate_pins -of_pins genloop[2].icw0/ic/OutBus[1]
```

```
{{\genloop[2].icw0 /ic/OutBus[1]}}
```

```
SETUP> get_pins -of_gate_pins {{\genloop[2].icw0 /ic/OutBus[1]}}
```

```
{genloop[2].icw0/ic/OutBus[1]}
```

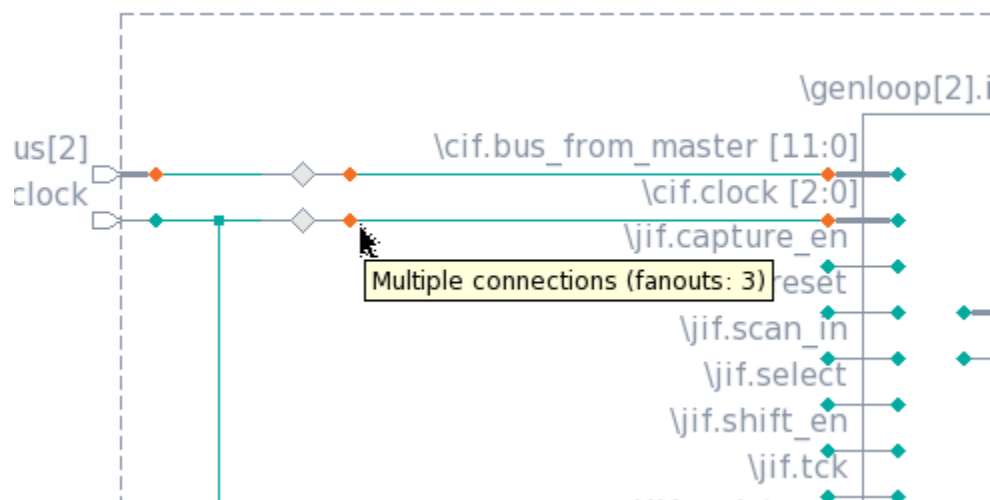
```
SETUP> get_pins -of_gate_pins [get_gate_pins {{\genloop[2].icw0 /ic/OutBus[1]}}
```

```
{genloop[2].icw0/ic/OutBus[1]}
```

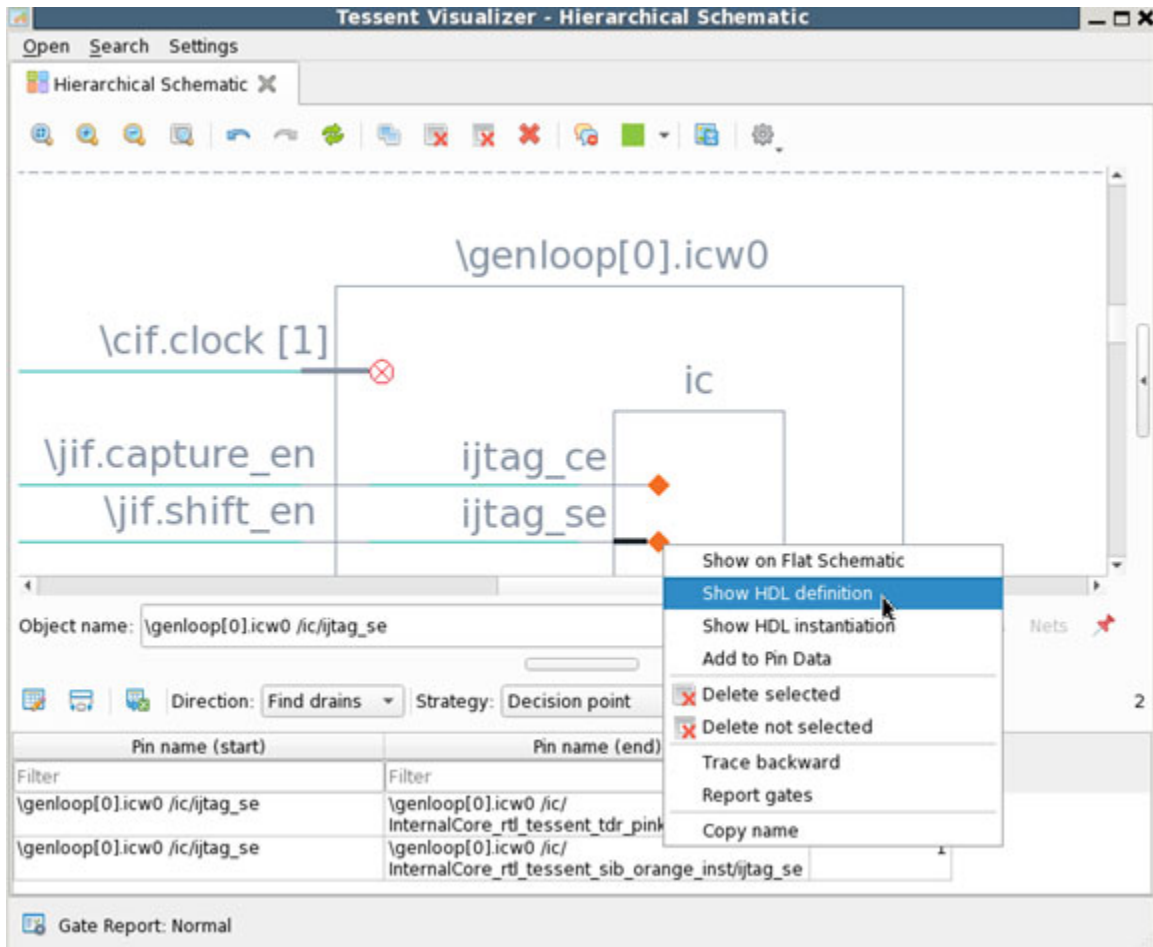
Tessent Visualizer Tracing and Visualization

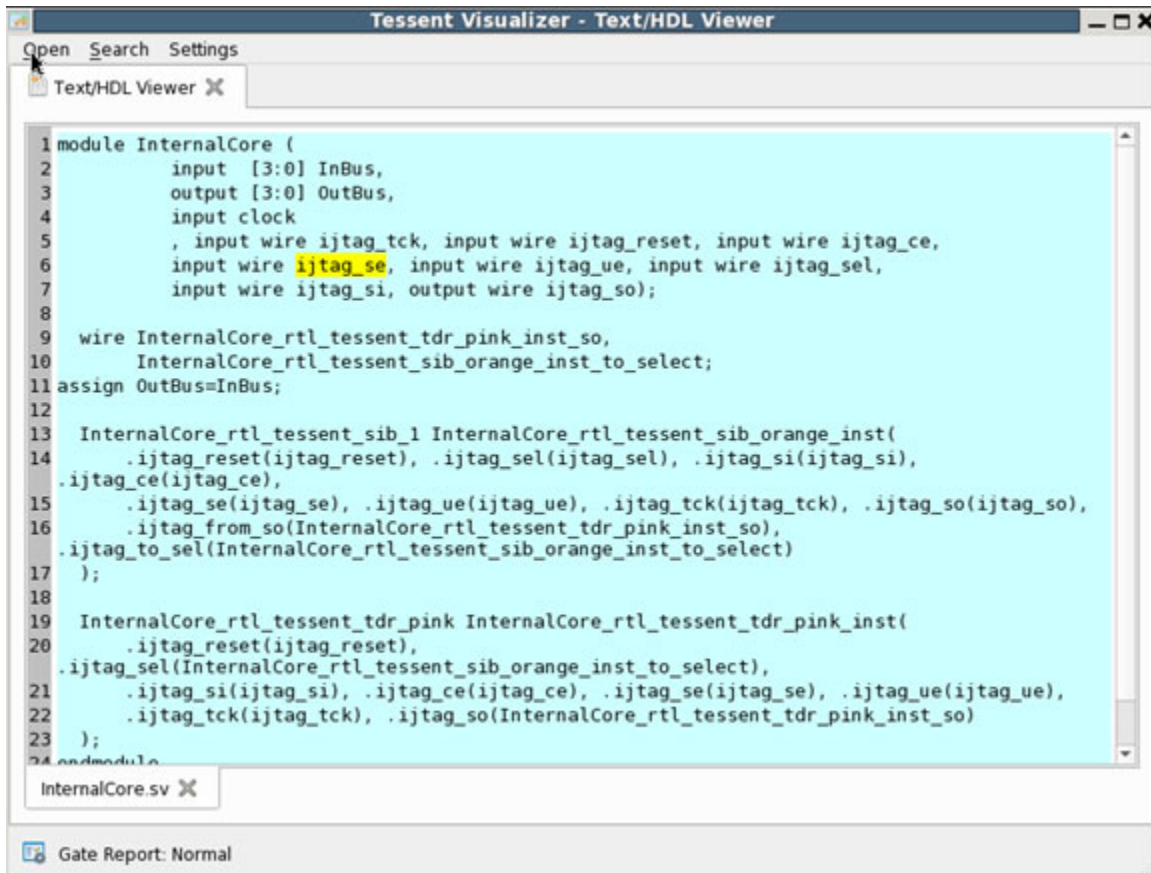
Tessent Visualizer provides a platform to trace and visualize bundle objects in the flat model. The Tessent Visualizer GUI shows the `post_synthesis_name` of the ports and pins even when the RTL view of the module is active. When you switch to the synthesized view of a module, its footprint remains unchanged in Tessent Visualizer. The RTL logic that is not shown in the RTL view becomes logic cell instances in the synthesized view.

Figure 3-2. Complex Signals with Tessent Visualizer



Tessent Visualizer facilitates cross-probing pins in the schematic to their associated RTL netlist.





For complete information on using Tessent Visualizer, see “[Tessent Visualizer](#)” on page 379.

Design Editing

Tessent Shell design editing commands allows you to modify your design after reading in the RTL or gate-level netlist. Tessent Shell supports gate-level netlist editing or RTL design editing with full language support, including multiple logical libraries, VHDL, Verilog, and System Verilog. Parameterized modules are also fully supported.

The Design editing commands allow you to manipulate a design's modules, instances, nets, ports, and pins, either interactively or through Tcl scripting.

Note



There are language restrictions. See “[HDL Limitations in the Tessent Shell Flow](#)” in the *Tessent Shell Reference Manual* for details.

Design editing commands work with collections and introspection commands, as well as native Tcl commands, to automate many tasks. [Table 3-6](#) presents the common design editing commands based on the function they perform—that is, whether they create, modify, or remove elements and so on. For more information on collections and introspection commands, see “[Design Introspection](#).”

Note



The design editing commands are not available when using some product licenses, such as Tessent FastScan and Tessent Scan.

Table 3-6. Design Editing Commands

Commands that...	Command Name
Read netlists and specify logical libraries	read_verilog
	read_vhdl
	set_logical_design_libraries
Create design elements	create_connections
	create_instance
	create_module
	create_net
	create_pin
	create_port

Table 3-6. Design Editing Commands (cont.)

Commands that...	Command Name
Remove design elements	delete_connections
	delete_instances
	delete_nets
	delete_pins
	delete_ports
Modify the design	copy_module
	intercept_connection
	replace_instances
	uniquify_instances
Set or get editing options	get_insertion_option
	set_insertion_options

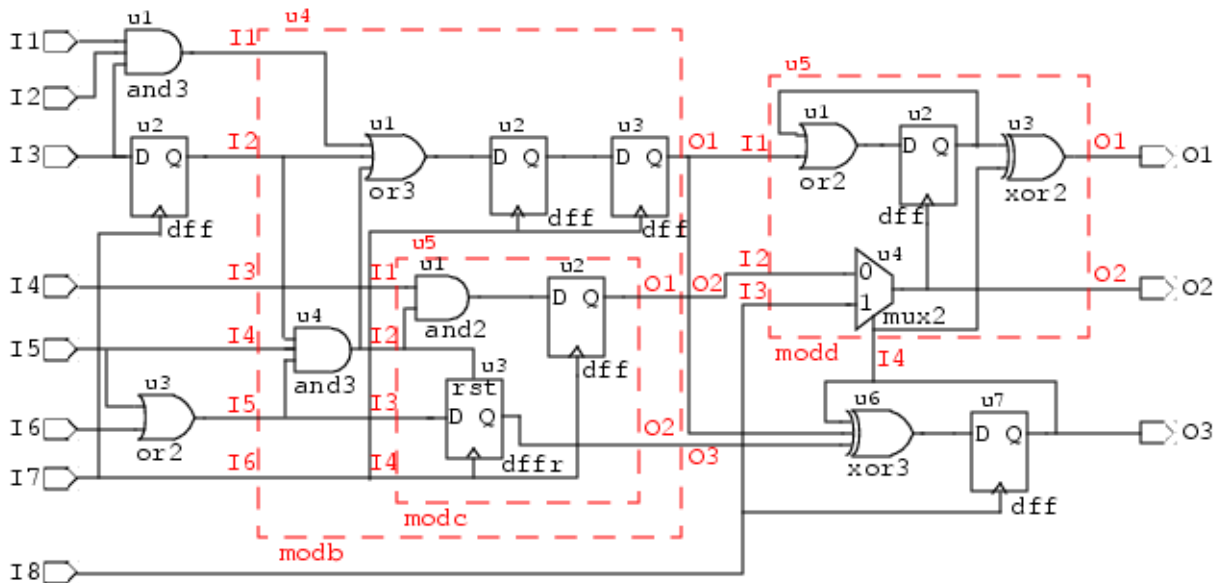
Design Editing Examples..... 73**Design Editing Command Summary..... 78**

Design Editing Examples

There are many ways to create, modify, and remove elements from your design in a Tcl scripting environment.

[Figure 3-3](#) is an example design, which is followed by related examples that show some of the ways you can edit designs within Tessent Shell.

Figure 3-3. Hierarchical Design Example



Example of Creating a Module From Scratch

The following example shows how to create module C (modc) in [Figure 3-3](#) as a standalone module.

```
set_context dft -no_rtl
set_system_mode insertion
create_module modc
set_current_design modc

create_port I1 -on_module modc -direction input
create_port I2 -on_module modc -direction input
create_port I3 -on_module modc -direction input
create_port I4 -on_module modc -direction input
create_port O1 -on_module modc -direction output
create_port O2 -on_module modc -direction output

create_instance u1 -of_module and2
create_instance u2 -of_module dff
create_instance u3 -of_module dffr

create_connection I1 u1/A0
create_connection I2 u1/A1
create_connection I2 u3/R
create_connection I3 u3/D
create_connection I4 u3/CLK
create_connection I4 u2/CLK
create_connection u1/Y u2/D
create_connection u2/Q O1
create_connection u3/Q O2

write_design -output_file MODC.v -replace
```

Example of Replacing a Module With Another Module

The following example replaces the module definition for instance u5 (modd) to modc. The original module and the new module have the same pinout, and the connecting nets remain the same after issuing the `replace_instances` command. After replacing instance u5 with modc, that instance u5/u1 changes from an or2 gate to an and02 gate.

```
INSERTION> get_attribute_value_list u5 -name module_name
modd

INSERTION> get_fanin u5/I1 -stop_on net
{u4_O1}

INSERTION> replace_instances u5 -with_module modc
{u5}

INSERTION> get_fanin u5/I1 -stop_on net
{u4_O1}

INSERTION> set_system_mode setup
SETUP> set_design_level sub_block
SETUP> set_sys_mode analysis

// Flattening process completed, cell instances=15, gates=45, PIs=8
// POs=3, CPU time=0.00 sec.
// -----
// Begin circuit learning analyses.
// -----
// Learning completed, CPU time=0.00 sec.

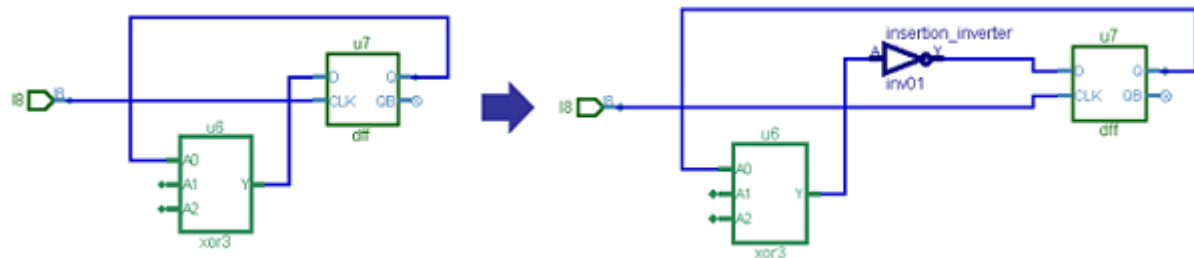
ANALYSIS> report_gates u5/u1

// /u5/u1 and02
// A0 I /u4/u3/Q
// A1 I /u4/u5/u2/Q
// Y O /u5/u2/D
```

Example of Intercepting a Connection

The following example inserts a simple inverter gate by intercepting the existing connection between the u7/D pin and the u6/Y pin, shown on the left side of the following figure. After the interception, the inverter A pin is connected to the u6/Y pin and the inverter Y pin is connected to the u7/D pin as shown on the right side of the figure.

Figure 3-4. Inverter Inception



```
INSERTION> intercept_connection u7/D -cell_function_name inverter  
{inversion_inverter}
```

Example of Adding Input and Output Pads to a Design

The following dofile example adds input and output pads to a design by creating a collection of port names from the design, creating a collection of pad names that match the ports, then connecting the ports and pads together in the design. The example also adds the TAP's IO ports, including the respective pads.

```
# Setting the context to netlist editing
set_context dft -no_rtl

# Reading the library and all necessary Verilog files
read_cell_library ../libs/libs/adk.atpg
read_verilog      ./counter_block2_edt_top_gate.v
read_verilog      ../libs/pads/*.v

# Telling the tool, which module 'top' is
set_top_module "counter_block2"
set_current_design $top_module

# Now for the editing
set_system_mode insertion

# First, insert the pad cells for all inputs of the netlist
set inputPortList [ get_ports -of_module $top_module -direction input ]
foreach_in_collection inputPort $inputPortList {

    # Creating the name of the pad cell:
    #   The following lines take care of '[' and ']' in the portname,
    #   substituting these by '_'. This makes the Tcl-life easier
    set portName [lindex [get_name_list $inputPort] 0]
    set padName   [ string map { \[ _ \] _ } ${portName}_PAD ]

    # Adding the pad cell
    create_instance $padName -of_module INPAD

# Connecting it up in the netlist
# The first line moves the existing net from the input port to the output
# of the pad cell. The second line creates a new net and connects the
# input port to the input of the pad cell
# Note: We are not connecting anything else here.
#       We leave this to boundary scan insertion
    move_connection -from $inputPort -to $padName/FP
    create_connection $inputPort $padName/IO
}

# Second, insert the pad cells for all outputs of the netlist
set outputPortList [ get_ports -of_module $top_module -direction output ]
foreach_in_collection outputPort $outputPortList {

    # Creating the name of the pad cell:
    #   The following lines take care of '[' and ']' in the portname,
    #   substituting these by '_'.
    set portName [lindex [ get_name_list $outputPort ] 0]
    set padName   [ string map { \[ _ \] _ } ${portName}_PAD ]

    # Adding the pad cell
    create_instance $padName -of_module OUTPAD_2S

# Connecting it up in the netlist
# The first line moves the existing net from the IO port to the input
# of the pad cell. The second line creates a new net and connects the
# IO port to the output of the pad cell.
# Note: We are not connecting anything else here.
#       We leave this to boundary scan insertion
    move_connection -from $outputPort -to $padName/TP
```

```
        create_connection $outputPort $padName/IO
    }

    # Next, add the JTAG IO pins and their respective PAD cell. Connecting #
    # them up.

    set inputPortList { TDI TMS TRST TCK }
    set outputPortList { TDO }

    foreach portName $inputPortList {

        # Adding the pad cell
        set padName      ${portName}_PAD
        create_instance $padName -of_module INPAD

        # Creating an Input IO pin and connecting the pad
        create_port $portName -direction input
        create_connection $portName $padName/IO
    }

    foreach portName $outputPortList {

        # Adding the pad cell
        set padName      ${portName}_PAD
        create_instance $padName -of_module OUTPAD_EN1

        # Creating an output pin and connecting the pad
        create_port $portName -direction output
        create_connection $portName $padName/IO
    }

    # Write the new netlist
    write_design -output_file "${top_module}.v" -replace
    exit
```

Design Editing Command Summary

The design editing commands allow you to work with modules, connections, instances, nets, pins, and ports.

The commands listed in the following table are commonly used design editing commands. Refer to the *Tessent Shell Reference Manual* for more information.

Table 3-7. Design Editing Command Summary

Command Name	Description
copy_module	Creates an exact copy of a design module and gives it a new name, which the tool can use as part of create_instance and replace_instances operations.
create_connections	Creates a connection between pin, net, or port objects.
create_instance	Instantiates a module (design or cell type) inside of a design module that is part of the current design.

Table 3-7. Design Editing Command Summary (cont.)

Command Name	Description
create_module	Creates a new design module.
create_net	Creates a net inside an instance of a design module.
create_pin	Creates a pin on an instance of a design module.
create_port	Creates a port on a design module.
delete_connections	Disconnects the specified pin objects.
delete_instances	Removes an instance of a module.
delete_nets	Removes net objects inside an instance of a design module.
delete_pins	Removes pin objects on an instance of a design module.
delete_ports	Removes port objects on a design module.
get_insertion_option	Introspects the default values of options affecting many design editing commands.
intercept_connection	Using the <code>get_dft_cell</code> command, obtains a cell with the specified function name and uses it to intercept a connection to a pin, port, or net.
move_connections	Moves a net connected on one pin or port to another pin or port. The first pin is left open after the move.
rename_instance	Renames the leaf name of an instance object.
replace_instances	Replaces the module object used in an instantiation.
set_insertion_options	Specifies default values of options affecting many design editing commands.
uniquify_instances	If the module of the specified instance has other instantiations in the design, the tool copies the module and the specified instance becomes an instance of the copied module.

Simulation Contexts

Tessent Shell provides simulation contexts to aid your design analysis and introspection efforts. You can use these contexts to create “simulation scratch pads” for rapid investigation of good-machine behavior of specific portions of your design.

Simulation Context Overview	80
Introspection and Analysis Using Simulation Contexts	81

Simulation Context Overview

Tessent Shell provides commands for creating and managing simulation contexts. From a particular user-defined simulation context, you can use these commands to apply stimulus forces to specified gate_pins, run simulation for a specific number of cycles, and then introspect gate_pins for simulation values.

Commands for Managing Simulation Contexts

Use the following commands to create and manage user-defined simulation contexts, as well as use predefined simulation contexts:

- [add_simulation_context](#) — Creates a new user-defined simulation context.
- [copy_simulation_context](#) — Copies the simulation values and forces from one simulation context to another.
- [delete_simulation_contexts](#) — Deletes one or more user-defined simulation contexts.
- [get_current_simulation_context](#) — Returns the name of the current simulation context.
- [get_simulation_context_list](#) — Returns the available simulation contexts in a Tcl list.
- [report_simulation_contexts](#) — Lists the available simulation contexts and indicates the current simulation context.
- [set_current_simulation_context](#) — Sets the current simulation context.

Commands for Managing Stimulus and Simulating within Simulation Contexts

The following commands are for managing stimulus (simulation forces and clock pulses) and running limited simulations within a simulation context:

- [add_simulation_forces](#) — Forces one or more gate_pin objects to the specified value.
- [delete_simulation_forces](#) — Removes forces from one or more gate_pin objects.
- [report_simulation_forces](#) — Lists the active forces on the specified gate_pin objects.

- [simulate_clock_pulses](#) — Pulses one or more clocks within the current simulation context.
- [simulate_forces](#) — Simulates the queued forces in the current simulation context.

Commands for Introspection and Analysis within Simulation Contexts

The following commands are for introspection and analysis within simulation contexts, which includes examining simulation results:

- [get_current_simulation_context](#) — Returns the name of the current simulation context.
- [get_simulation_context_list](#) — Returns the available simulation contexts in a Tcl list.
- [get_simulation_value_list](#) — Returns the simulation values on the specified gate_pin objects.
- [report_simulation_contexts](#) — Lists the available simulation contexts and indicates the current simulation context.
- [report_simulation_forces](#) — Lists the active forces on the specified gate_pin objects.
- [set_gate_report](#) — Specifies the information displayed by the report_gates command. This command allows reporting of simulated values in the current simulation context.
- [trace_flat_model](#) — Traces the flat model within the current simulation context. This command always operates within the current simulation context.

Attributes for gate_pin Objects

For a complete list of attributes, refer to the “[Data Models](#)” chapter in the *Tessent Shell Reference Manual*.

Introspection and Analysis Using Simulation Contexts

Simulation contexts allow you to perform various types of design analysis and introspection. The four predefined simulation contexts are: `stable_after_setup`, `stable_load_unload`, `stable_shift`, and `stable_capture`. After setting a simulation context, you can introspect gate_pins for simulation values based on that specific simulation context. For example, the `trace_flat_model` command can do design tracing based on values specified for the current simulation context.

In order to use any of the following techniques in this chapter, you must be doing the following:

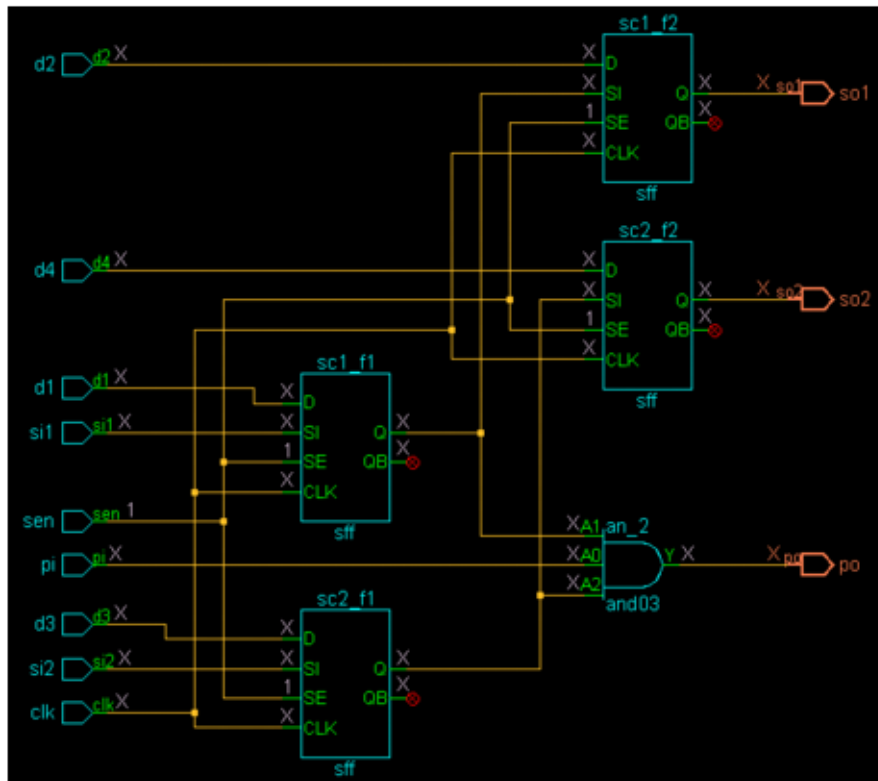
- Operating in the analysis system mode of Tessent Shell.

- Operating on a flattened design, and have read in the test procedure file (if available). At this point, the values specified in the setup, shift, capture, and load_unload procedures are in place in the design when you set the corresponding simulation context as current.

Example

For example, if you want to trace the design based on the values specified in the shift procedure, use the “set_current_simulation_context stable_shift” command. When you use this command with a small design containing two 2-cell scan chains, the values on the gate_pins display in the Tessent Visualizer Flat Schematic as shown in [Figure 3-5](#).

Figure 3-5. Tessent Visualizer Flat Schematic (gate level)



In this case, no values are forced except for `sen` (scan enable), which is set to 1. Note that the four possible simulation values are 0, 1, X, and Z.

In addition to viewing values in the Flat Schematic, you can get the current simulation values of `gate_pins` using any of the following techniques:

- Use the `get_simulation_value_list` command and specify the desired `gate_pin` object(s):

```
ANALYSIS> set_current_simulation_context stable_shift
ANALYSIS> get_simulation_value_list sc2 f1/D
```

X

- Check the value of the simulation_value attribute for the gate_pins:

```
ANALYSIS> get_attribute_value_list [get_gate_pin sc2_f1/D]
-name simulation_value
```

```
X
```

- Issue the command “set_gate_report simulation_context,” after which you can use the report_gates command and the Flat Schematic to display the simulated values from the current simulation context:

```
ANALYSIS> set_gate_report simulation_context
ANALYSIS> report_gates sc2_f1
```

```
// /sc2_f1 sff
//      D      I  (X)  /d3
//      SI      I  (X)  /si2
//      SE      I  (1)  /sen
//      CLK     I  (X)  /clk
//      Q       O  (X)  /an_2/A2  /sc2_f2/SI
//      QB      O  (X)
```

You can use simulation context functionality for different types of analysis. For example, by default the trace_flat_model command uses the stable_after_setup values as a simulation background. You can now change it to stable_capture, for example, if you want to trace based on sensitized paths during capture. Another example is to copy an existing simulation context to a new one, force some simulation value changes, then evaluate the results in the circuit after simulating the forces and/or clock pulses.

Chapter 4

DFT Architecture Guidelines for Hierarchical Designs

Most chips designed in recent times follow hierarchical place-and-route due to the increase in design sizes. For DFT, you can also use hierarchical design methodologies to perform test insertion, test generation, and diagnosis. This is called hierarchical DFT or hierarchical test.

If the design is implemented as one chip with flat place-and-route, then perform DFT implementation once for the entire chip. Hierarchical DFT implementation is only applicable for block-based designs in which the tool run time consumption is not practical for implementing DFT only from the chip level.

Hierarchical DFT has several advantages:

- Divide and conquer technique that helps with parallelization of tasks.
- Reduced memory footprint and CPU requirements to perform the given task.
- Faster pattern generation and simulation run times.
- DFT performed earlier at block level and out of design critical path.

This chapter provides guidelines that you can use to architect hierarchical test.

For information about the Tessent Shell two-pass DFT insertion flow for hierarchical designs, refer to “[Tessent Shell Flow for Hierarchical Designs](#)” on page 128.

Hierarchical DFT: Concepts and Terms	86
Top-Down Planning Before Bottom-Up Implementation	92
DFT Implementation Strategy	98

Hierarchical DFT: Concepts and Terms

Within the Tessent Shell environment, there are several concept and terms that are important for understanding how to implement DFT in hierarchical designs.

Physical Layout Regions in Hierarchical Test	86
Pattern Retargeting	87
Wrapped Cores and Wrapper Cells.	87
Internal Mode and External Mode.	88
On-Chip Clock Controller	89
Graybox Model	90

Physical Layout Regions in Hierarchical Test

Hierarchical designs have more than one layout region. There are typically several physical layout regions, and these layout regions are instantiated into other physical layout regions. In a majority of cases, the instantiations occur at the chip level, so you have two levels of hierarchy.

In some designs there may be more than two levels of physical hierarchy. From the DFT perspective, each layout region is where you perform test insertion, with the test insertion process performed independently for each region. You can implement DFT within the layout regions in parallel, thus improving throughput.

For a design with two levels of physical hierarchy, first insert DFT into the lower-level designs before inserting DFT at the next higher level, in this case the chip level.

In the Tessent Shell documentation, you will see the following terms for the layout regions of a hierarchical design:

- **Physical block, block, core, child core.** These terms are interchangeable and refer to layout regions below the chip level that you can instantiate within a chip, or across multiple chips. Blocks are logical entities that remain intact through tapeout. You perform synthesis on these blocks independent of the rest of the chip design.

The term “wrapped core” is a specialized type of block. Refer to “[Wrapped Cores and Wrapper Cells](#)” on page 87 for details.

- **Chip.** The chip is the top-level physical block—that is, the entire design—in which you typically find the pad I/O macros and clock controllers.

The chip and upper-level blocks in designs with more than two hierarchical levels are often referred to as the parent blocks and the tasks related to them as occurring at the parent level.

Pattern Retargeting

Pattern retargeting is the process by which Tessent Shell preserves the ATPG patterns associated with cores for purposes of reuse when testing the logic at the chip (or parent) level. This means you do not have to regenerate the patterns when you process the chip. Instead, you retarget the core patterns to the chip level. Every instantiation of a core includes its associated ATPG patterns.

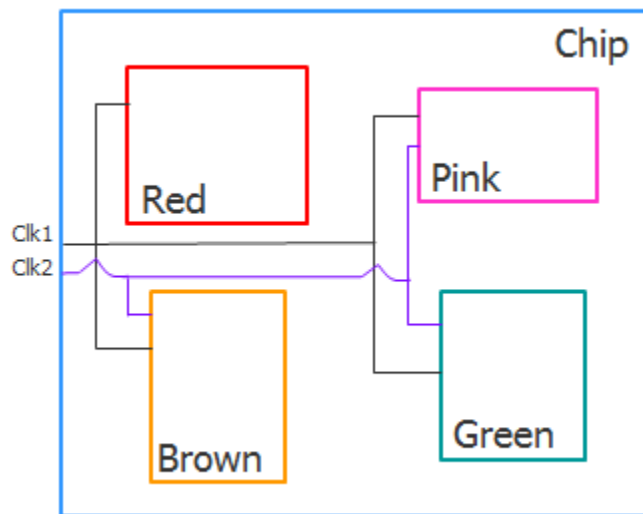
The pattern retargeting process consists of generating patterns for all cores, retargeting the core-level test patterns to the chip level, and generating patterns for the top-level chip logic only. This is also referred to as ATPG pattern retargeting or scan pattern retargeting. For details, refer to “[Scan Pattern Retargeting](#)” in the *Tessent Scan and ATPG User’s Manual*.

Wrapped Cores and Wrapper Cells

In hierarchical DFT, you must wrap physical blocks to make them reusable through pattern retargeting. The term *wrapped cores* applies to these physical blocks. The purpose of wrapping a core is to isolate its internal logic.

In the following figure, the physical layout regions from a DFT perspective are physical blocks. Tessent performs test insertion on the Red, Pink, Brown and Green physical blocks first. Clk1 and Clk2 are asynchronous clocks.

Figure 4-1. Wrapped Cores



The wrapping mechanism that you use for these blocks makes use of the functional flops that are already present at the boundary of these blocks. These flops are called shared wrapper cells (or sometimes simply wrapper cells) because they share the responsibility of their original functional use as I/O flops with isolating the core.

Optionally, you can add dedicated wrapper cells. These wrapper cells are added on primary inputs or primary outputs of a physical block that fan out or fan into large logic cones. By adding the dedicated wrapper cell, you can test the logic cone during internal testing of the core.

Shared and dedicated wrapper cells form the wrapper chains. Logic located within the wrapper chains is tested during internal testing of the core, called *intest*. Logic located outside the wrapper chains is tested during the external testing of the core, called *extest*.

Internal Mode and External Mode

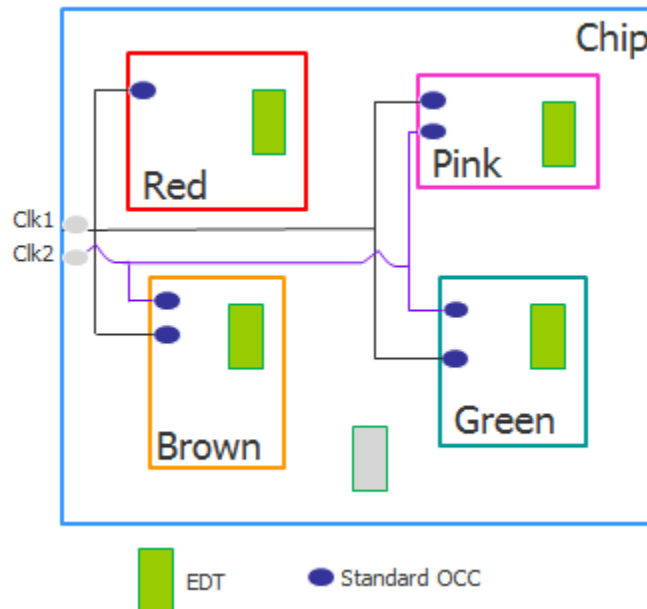
For purposes of pattern retargeting, Tessent Shell differentiates between a wrapped core's internal circuitry and its external circuitry.

Internal mode is the view into the wrapped core from the wrapper cells. That is, the logic completely internal to the core. Tessent Shell retargets internal mode ATPG patterns during ATPG pattern generation for the chip-level design.

External mode indicates the view out of the wrapped core from the wrapper cells. That is, the logic that connects the wrapped core to external logic. Tessent Shell uses the external mode to build graybox models, which are used by the internal modes of their parent physical blocks.

As shown below, based on chip pin availability, you can test the Red, Pink, Brown and Green cores in internal mode at the same time when the on-chip clock controllers (OCCs) and EDT logic blocks inside the wrapped cores are active.

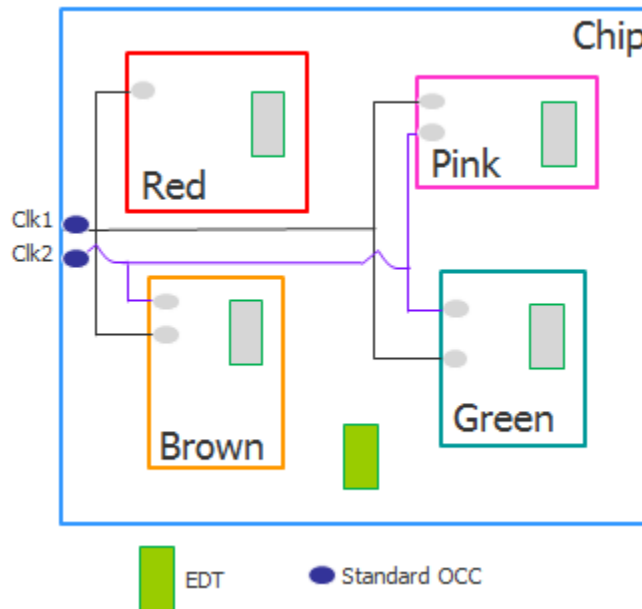
Figure 4-2. Internal Mode



You can test one or more wrapped cores in internal mode based on several factors, such as availability of chip-level pins, tester channels, tester memory, and power dissipation. The OCCs and EDT compression logic at the chip level are inactive during internal mode.

In external mode, the wrapped cores at the given physical hierarchy participate. The OCCs and EDT logic blocks inside the cores are inactive, and the OCCs and EDT logic blocks outside the wrapped cores are active. In a typical external mode configuration, one EDT logic block is used to test the logic outside the wrapper chains, as shown in the following figure.

Figure 4-3. External Mode



External mode also includes the wrapper chains within the wrapped cores. Tessent tests the logic outside of the wrapper chains during external mode.

On-Chip Clock Controller

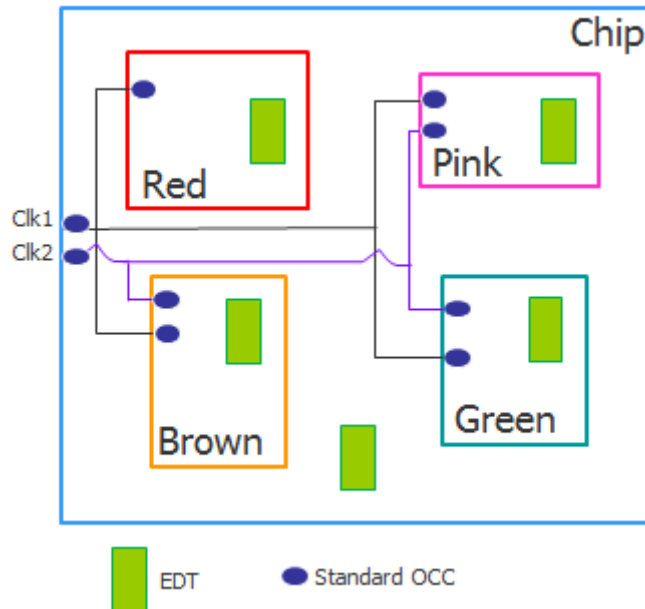
In hierarchical test, the existence of wrapped cores whose patterns will be retargeted to the chip level requires a mechanism to allow for local, self-contained clock control. This mechanism is the on-chip clock controller (OCC), which you insert into each wrapped core. OCCs inside the wrapped cores make the ATPG patterns self-contained, which allows them to be retargeted.

The OCC is initialized for a clock waveform and then applicable patterns are retargeted to the parent- or chip-level design, as applicable. Block-level patterns with block-level OCCs can be retargeted and merged with other block patterns at the parent- or chip-level design independent of the clock waveforms.

Each wrapped core can contain one or more EDT logic blocks that contain the compression logic. Similarly there may be several options for testing the external mode of the cores. In the following figure, OCC insertion occurs for each clock domain entering the cores. At the chip

level, OCCs are also required for the clock domains at this level. Insert one EDT logic block for each internal mode of the core and one EDT logic block for the core's external mode.

Figure 4-4. On-Chip Clock Controller



To help facilitate test setup and automation, use a 1149.1 TAP controller along with an IJTAG network. You may also need to insert a Test Access Mechanism (TAM) to guide how these wrapped cores may be tested.

Standard OCCs include built-in clock selection, clock-chopping, and clock-gating functionality. For more information about standard OCCs, as well as parent and child OCCs, refer “[Tessent On-Chip Clock Controller](#)” in the *Tessent Scan and ATPG User’s Manual*.

Graybox Model

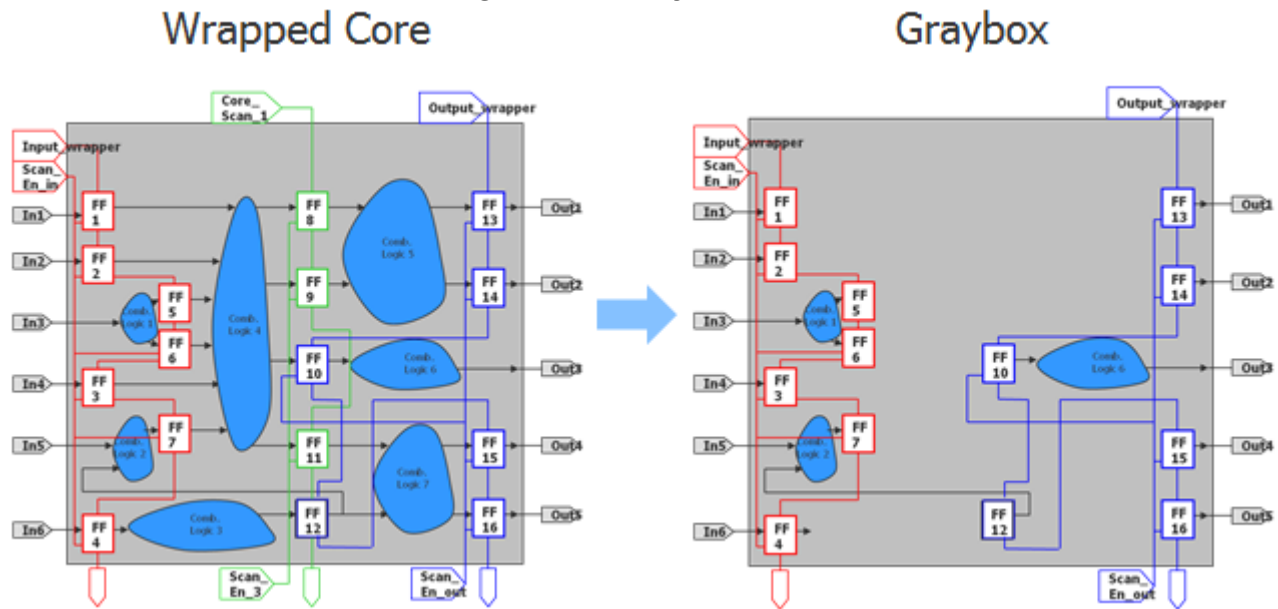
Graybox models are wrapped core models that preserve the core’s external mode logic—which includes the wrapper chains—along with the portion of the IJTAG network that needs to be tested along with the logic at the next physical level. The purpose of graybox models in hierarchical test is to retain the minimum logic required to generate ATPG patterns for the internal modes of the parent physical blocks.

For details, refer to “[Graybox Overview](#)” in the *Tessent Scan and ATPG User’s Manual*.

When testing the next physical level, using graybox models allows Tessent Shell to load the design faster than loading the full-chip design.

The following figure shows a graybox model in which the internal logic of the wrapped core is removed. This graybox model implementation has separate input and output wrapper chains with separate scan enable signals for each wrapper chain type.

Figure 4-5. Graybox Model



The input and output wrapper chains do not have to be separate, and the scan enable signals also do not need to be separate. You can use one scan enable signal with other mechanisms to enable how the wrapper chains operate.

Top-Down Planning Before Bottom-Up Implementation

To optimize your DFT implementation for hierarchical designs, top-down planning is necessary.

As described in “[Tessent Shell Flow for Hierarchical Designs](#)” on page 128, the process for inserting DFT into hierarchical designs is performed in a bottom-up process starting from the lowest level block. However, prior to insertion, it is best to plan the implementation in a top-down manner, starting at the chip level.

Top-down planning allows you to properly budget and allocate the resources required for hierarchical test. Every design has its unique challenges. Some considerations include:

- Optimizing pattern count to reduce test time.
- Optimizing scan volume to fit within allocated tester memory.
- Reducing chip pin requirements to fit within tester pin allocation.

The planning process requires you to think about your priorities. This could be test time, test quality, flow compatibility, design schedule, and so on. During the planning stage consider the priorities and also the necessary tradeoffs to meet those priorities. In addition, be aware that how you implement DFT at the chip level impacts DFT implementation at the core level.

Clocking Architecture	92
Resource Availability	93
Test Scheduling	93
Specific Tasks That May Require Planning	96

Clocking Architecture

In functional mode, the clock architecture can include clocks that are divided or multiplied. The original, divided, and multiplied clocks can be synchronous with each other within a core, and sometimes they can be synchronous across hierarchical cores. Divided clocks can also be asynchronous across hierarchical cores.

When generating ATPG patterns at the core level that will be retargeted, the core must include an OCC to control clocking as described in “[On-Chip Clock Controller](#)” on page 89. How you implement OCCs in the cores depends on various factors:

- How the clocks are balanced. The most common methods for clock balancing are clock tree synthesis and clock mesh synthesis. The method you use can influence what type of OCCs you insert (standard, parent, child).

- OCC behavior during intest and extest. How the OCCs need to behave during intest and extest of the cores also dictates how you should implement the OCCs.

Some OCCs are only required to perform clock chopping or clock-gating functionality. Sometimes OCCs may be necessary to mux clocks in for test purposes. Use chip-level clocking to determine the architecture and OCC types (standard, parent, child) that you should use within the wrapped cores.

Refer to “[Clocking Architecture Examples](#)” on page 517 for examples of clocking architectures with OCC.

Resource Availability

Resource availability plays a significant role in how you implement DFT. For example, at the chip level, the number of pins available for test limits the number of EDT channel pins that you can use. The EDT channel pins may not be associated with only one EDT controller; they could be connected to multiple EDT controllers.

Likewise, the tester may have limited channel pins available for connecting to EDT channel pins on the chip, which means that you may need to consider the tester’s memory allocation for storing the test patterns. Available memory on the tester dictates whether you will need to split the pattern set or test multiple cores in parallel.

In addition, how you will be testing the wrapped cores dictates how you create the test-access mechanism (TAM). TAMs carry scan data in and out of the chip for each group of wrapped cores you intend to run in parallel. You need a TAM if you have limited chip pins, and you are reusing those chip pins to connect to multiple EDT controllers inside the wrapped cores.

The TAM logic is dependent on how and when the cores get tested. The TAM schedules the tests for the wrapped cores at the top level, and it allows access to the chip level so that you can run these tests. The TAM also needs input from floor planning and placement of these cores to avoid routing congestion; you need to account for the proximity of the cores that will be tested in parallel and for the location of the chip pins that will connect to them.

Test Scheduling

Deciding which wrapped cores need to be tested at the same time—that is, in parallel—depends on a number of factors.

Some of these factors include:

- Number of patterns to be executed.
- Availability of chip pins.
- Tester pin storage limitations.

- Number of identical cores that need to be tested.
- Cores with similar pattern count that can be grouped together for execution.
- Power dissipation budget and allocation.
- Optimal compression at core level.
- Channel sharing of cores within modular EDT or sub-chip architectures; cores need to be tested where channels are shared.

To achieve the optimal test schedule for a chip with a given number of hierarchical cores, generate an optimized compression for the internal mode of the cores. Within each core, compression could be based on asymmetric channel configurations. For X-clean designs, fewer output channels are required.

A utility called Compression Advisor can help you obtain the optimal compression. This utility is available from Mentor Graphics application engineers. In addition, you can use the [analyze_compression](#) command to optimize compression. These tools require gate-level scan stitched netlists. BIST logic used for memories needs to be scan stitched and included for the best compression estimation.

The following example shows a schedule based on compression analysis. Suppose sixty-four pins are available at the chip level for use as channel pins. There are 12 instances of blk_a, 8 instances of blk_b, 4 instances of blk_c, and 1 instance each of blk_d, blk_e, and blk_f. The highlighted instances provide the best utilization of resources.

Figure 4-6. Compression Analysis Example

12 instances					
Block Name	Input Channels	Output Channels	Shift Length	# Patterns	# Tester Cycles
blk_a	40	2	255	2584	658920
blk_a	28	3	256	3044	779264
blk_a	52	1	255	2692	686460
blk_a	40	2	205	2505	513525
blk_a	40	2	156	2409	375804
8 instances					
Block Name	Input Channels	Output Channels	Shift Length	# Patterns	# Tester Cycles
blk_b	56	1	256	1934	495104
blk_b	48	2	256	1939	496384
blk_b	40	3	256	1915	490240
blk_b	32	4	256	1905	487680
blk_b	32	4	202	1828	369256
blk_b	32	4	154	1411	217294
4 instances					
Block Name	Input Channels	Output Channels	Shift Length	# Patterns	# Tester Cycles
blk_c	24	10	258	4475	1154550
blk_c	16	12	261	7360	1920960
blk_c	8	14	270	3895	1051650
blk_c	12	13	264	4233	1117512
Block Name	Input Channels	Output Channels	Shift Length	# Patterns	# Tester Cycles
blk_d	28	28	263	2285	600955
blk_d	36	20	260	1996	518960
blk_d	40	16	258	2005	517290
blk_d	38	18	259	1579	408961
Block Name	Input Channels	Output Channels	Shift Length	# Patterns	# Tester Cycles
blk_e	16	16	264	2233	589512
blk_e	26	6	257	2818	724226
blk_e	24	8	258	2112	544896
blk_e	20	12	261	2112	551232
blk_e	22	10	259	2033	526547
Block Name	Input Channels	Output Channels	Shift Length	# Patterns	# Tester Cycles
blk_f	16	16	264	2631	694584
blk_f	22	10	259	2462	637658
blk_f	26	6	257	2415	620655
blk_f	28	4	257	2416	620912
blk_f	31	1	255	2408	614040
blk_f	30	2	256	1918	491008

Suppose you plan to group identical cores instances for testing in parallel. For the blk_a instances, the 40 input channel pins can be broadcast. That is, you can provide the same input data to all the cores. Two output channels from each instance are required for a total of 24 additional pins. When testing the 12 blk_a instances together, you use the 64 available chip pins.

Table 4-1. Test Schedule Example

Scan Config	% Total Faults/ Group	Input Channels	Output Channels	Pins Used/ Config	Shift Length	No. of Patterns	No. of Tester Cycles
12(blk_a)	68.59	40	24	64	156	2409	375804
8(blk_b)	8.54	32	32	64	154	1411	217294
4(blk_c)	9.53	8	56	64	270	3895	1051650
blk_f, blk_e	7.33	52	12	64	259	2033	526547
blk_d	6.01	38	18	56	259	1579	408961
.	.	.	.	.	Total tester cycles		2646118

Next, test the 8 instances of blk_b together. In this case, the retargeted patterns are applied at the chip level. The 32 input channel pins broadcast to the 8 blk_b instances. In addition, each instance uses 4 output channels for a total of 64 pins. The tool proceeds to test the 4 blk_c instances with 8 input channels broadcast and 14 output channels pins per instance of blk_c. The blk_f and blk_e instances are tested together because they have similar complexity and channel requirements. Finally, the blk_d instances, which require the most channels, are tested by themselves.

Use the following guidelines when developing your test schedule:

- First, group identical core instances and execute them at the same time.
- Second, test cores of similar complexity and channel requirements.
- Last, test the cores that require the most channels by themselves.

Specific Tasks That May Require Planning

Every design has its unique set of tasks that you need to perform during test. Some tasks may involve planning while others may only require turning on or off features at the command line.

The following list provides examples of tasks that you may need to plan for ahead of time. This list is not exhaustive.

- Dual-mode configuration for Tessent TestKompress.
- Low-power mode and threshold set up.

- Multi-load ATPG patterns via the memories.
- Test point provisioning for extra scan chains.
- Support for multiple power islands and voltage islands.
- Functional timing exceptions to be read in with SDC.
- Low pin count test (LPCT) controllers.
- Test setup sequence to enter test mode.

DFT Implementation Strategy

For most designs, a few simple decisions can make a big difference to the overall DFT implementation architecture.

Internal and External Modes	98
Dedicated Test Clock.....	99
Sample DFT Planning Steps	99
Automated Features Within the Tessent Shell Flow	100

Internal and External Modes

The following considerations will help you when deciding how to test multiple cores in internal mode and external mode.

Internal Mode

When you have multiple wrapped cores, as a recommended strategy you typically fix the scan chain length at a constant value so that at the next hierarchical level when you combine cores to be tested in parallel, the shift length is identical.

In addition, another recommended strategy is to stitch OCC bits into a few dedicated compressed chains rather than distributing them across all of the chains, if the OCC bits are roughly 25% of the longest chain. If the OCC bits add up to greater than 75% of the longest chain, then stitch the OCC bits as an uncompressed chain. If there are any remaining OCC bits, connect them into a compressed chain.

If there are embedded boundary scan cells present, ensure they are stitched with the rest of the boundary scan cells at the chip level.

External Mode

The number of external mode chains from across multiple wrapped cores dictates how they should be handled. The possibilities are:

- Connect the external mode chains to one EDT controller.
- Connect the external mode chains to multiple EDT controllers.
- Connect the external mode chains directly to primary pins without any EDT compression logic. However, this can cause routing congestion.
- Connect the external mode chains when there is EDT compression logic. There may be EDT compression logic built inside the wrapped cores for the purpose of external mode, which leads to having at least 2 pins (one input and one output) for the compression logic to be used for external mode. This configuration eliminates routing congestion, but

each EDT logic block is required to have 2 pins in the external mode of each core that is connected to chip level.

If external mode chains from various cores are to be connected to one or more EDT controllers at chip level, then you must ensure that you balance the external mode chains so they are the same length. You can do this by using the chain length across multiple cores.

If there are multiple levels of hierarchy and the OCC needs to be active, it may be necessary to include the OCC bits in the external mode. That is, if you want the OCC to be used in both internal and external modes, then you need to plan for the OCC bits to be included in the external chain also. Typically, OCCs are only included in internal mode.

During ATPG, you can include the boundary scan chain at chip level when running the external mode of the wrapped cores.

Dedicated Test Clock

For hierarchical test, configure the test clock so that it does not share the functional clock port. The test clock needs an extra pulse to initialize the EDT hardware, which disturbs the scan cells and can result in D1 violations that cannot be fixed.

From the test clock, you can use DFT signals to generate the shift capture clock required for OCCs and the EDT clock required for EDT hardware.

Optionally, you can use TCK as your test clock. In this case, the TAP needs to run at the TCK rate, and the shift for ATPG is limited to the TCK frequency. Thus, you need to ensure that you are using an optimal TCK rate.

Using a dedicated test clock aids with timing closure and is beneficial for both internal test and external test because during shift when the scan enable signal is 1, the same clock path is chosen irrespective of whether the core is placed in internal or external mode of operation. For more information, refer to “Shift-Only Mode” under [Clock Control Operation Modes](#) in the *Tessent Scan and ATPG User’s Manual*.

Sample DFT Planning Steps

The following list describes key areas to think about when planning your DFT implementation. This example may not be applicable to all designs.

- Tally how many chip pins are available for scan channels and for ATE channels that are required for test. Use the smaller value when planning for the number of chip pins available for test.
- Use a dedicated test clock apart from existing functional clocks. This clock can use the TCK signal as the clock source.

- Assess the functional clock architecture for the chip and plan your OCC configuration. This includes thinking about how many OCCs you will need, their OCC types (child, parent, standard), and where they need to be inserted for wrapped cores and the chip.
- Based on the design size and pattern count, determine the number of channels required for each core.
- Determine test scheduling for cores that you can run in parallel as described in “[Test Scheduling](#)” on page 93.
- Determine the external mode configuration as described in “[External Mode](#)” on page 98.
- Confirm the top-level TAM that will be required.

Automated Features Within the Tessent Shell Flow

Tessent Shell provides many automated features that will benefit you when you are implementing your DFT strategy.

- TSDB — As described in “[What Is Tessent Shell?](#)” on page 19, the TSDB is a common database used by all the Tessent products, which means that test information generated by one tool is recognized and useable by downstream tools. For details, refer to “[TSDB Data Flow for the Tessent Shell Flow](#).”
- IJTAG automation — As described in “[What Is Tessent Shell?](#)” on page 19, IJTAG provides many benefits, including DFT setup reuse from blocks and sub-blocks to higher levels in the hierarchy. This is essential for using the bottom-up DFT insertion flow as described in “[Tessent Shell Flow for Hierarchical Designs](#)” on page 128.
- Retargetable ATPG patterns — Using OCCs inside hierarchical cores makes the ATPG patterns self-contained, which allows them to be retargeted as described in “[On-Chip Clock Controller](#)” on page 89.

Note



Using the following Tessent automated features depends on your design implementation and how you are performing the two-pass RTL and scan DFT insertion flow for hierarchical designs as described in “[Tessent Shell Flow for Hierarchical Designs](#)” on page 128.

- Reset control — Tessent automatically fixes asynchronous resets with pre-DFT DRCs that are enabled when you set `set_dft_specification_requirements -logic_test on`. Tessent deactivates the reset during shift and tests the reset during capture. Optionally, provision is provided to override the reset during capture by deactivating it. This auto-fix is beneficial when the functional reset is generated internally.
- Boundary scan chain included for ATPG — At the chip level, you can segment the boundary scan chain into smaller chains to be connected as compressed scan chains and controlled during ATPG. Optionally, you can configure these boundary scan chains to participate during capture or simply use them during shift.

- Use memory to target shadow logic faults during logic test — During logic test insertion, inserted memories get bypassed. The bypass can be built within the memories themselves or built within the MemoryBIST infrastructure. You can use IJTAG-controlled internal Test Data Registers (TDRs) to enable bypass or to use the memories during logic test. By using the memories during logic test, the shadow logic around the memories can be tested. This requires an ATPG model for the memories. For details, refer to the [Comprehensive RAM Primitive](#) information in the *Tessent Cell Library Manual*.
- Unused scan chains — During scan insertion and stitching if there are any over-specified scan chains, Tessent Scan automatically adds the unused scan chains with pipeline registers. This is beneficial when not all the scan chains of the inserted EDT compression logic are utilized.

Chapter 5

Tessent Shell Work Flows

This chapter introduces you to the basic building blocks for understanding the pre-layout DFT insertion work flow when using Tessent Shell for flat and hierarchical designs. In addition, it describes the post-layout validation flow for netlists that have completed place and routing.

Tessent Shell Flow for Flat Designs	104
Tessent Shell Flow for Hierarchical Designs.....	128
Tessent Shell Post-Layout Validation Flow.....	193
Hybrid TK/LBIST Flow for Flat Designs	202
Running Multi-Load ATPG on Memories for Wrapped Cores with Built-In Self Repair	224
Built in Self Repair (BISR) in Hierarchical Tessent MemoryBIST Flow	231

Tessent Shell Flow for Flat Designs

This discussion centers on the RTL and scan DFT pre-layout insertion flow for flat designs. Understanding this flow will aid you when working with hierarchical designs or when implementing a variation of the basic flow.

In the RTL and scan DFT insertion flow for flat designs, you perform DFT insertion for the entire chip-level design. This flat DFT implementation aligns with the physical implementation of the design. The flow consists of performing DFT hardware insertion, followed by synthesis, optional scan insertion, and ATPG pattern generation at the gate level.

Refer to the following test case for a detailed usage example of the flow described in this section:

```
tessent_example_flat_flow_<software_version>.tgz
```

You can access this test case by navigating to the following directory:

```
<software_release_tree>/share/UsageExamples/
```

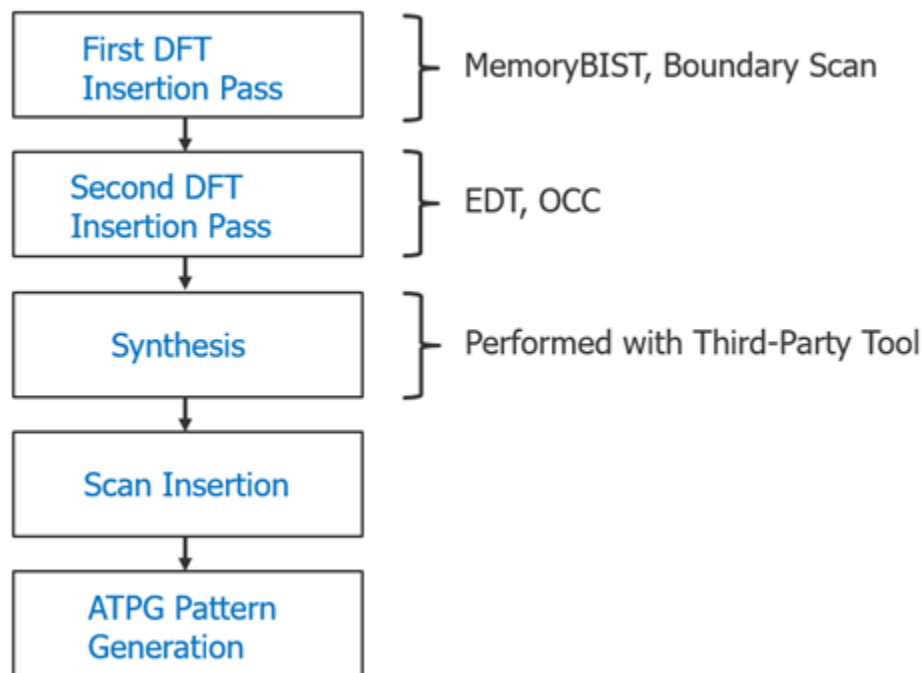
Overview of the RTL and Scan DFT Insertion Flow	105
First DFT Insertion Pass: MemoryBIST and Boundary Scan	107
Second DFT Insertion Pass: EDT and OCC	111
Load the Design	112
Specify and Verify the DFT Requirements	114
Create the DFT Specification	117
Generate the EDT and OCC Hardware	120
Extract the ICL Module Description	120
Generate ICL Patterns and Run Simulation	121
Perform Synthesis	122
Perform Scan Chain Insertion	123
Perform ATPG Pattern Generation	124
Considerations for Using Gate-Level Verilog Netlists	127

Overview of the RTL and Scan DFT Insertion Flow

Whether you are using a flat design or a hierarchical design, the RTL and scan DFT insertion flow requires you to use a two-pass insertion process to insert the DFT hardware.

As shown in the following figure, insert MemoryBIST and boundary scan prior to Embedded Deterministic Test (EDT) and the On-Chip Clock Controller (OCC). The DFT logic you insert during the first DFT insertion pass gets tested by EDT. When inserting DFT into an RTL design, you only need to run synthesis once after you have performed the two DFT insertion passes.

Figure 5-1. Two-Pass Insertion Flow for RTL, Flat Designs



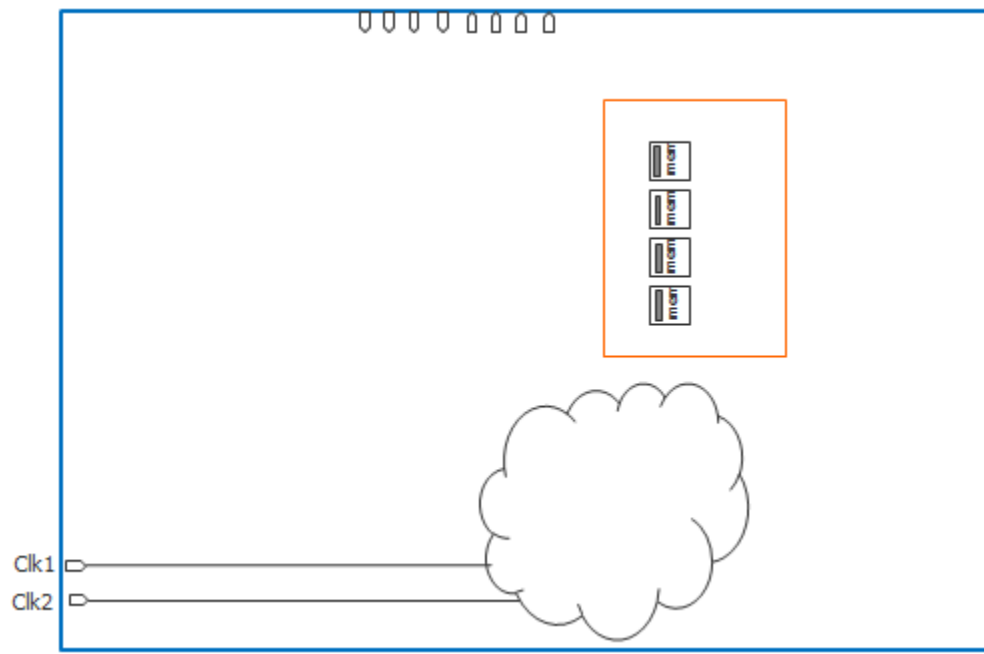
During the first pass, Tessent also inserts the IJTAG network. In addition, it inserts any IJTAG instruments you may have present in the design as long as they have ICL descriptions. Refer to the [create_dft_specification](#) command for details about this process.

During the second pass, the tool checks MemoryBIST's logic test rule compliance against the rest of the functional logic to prevent implications for the DFT signals and IJTAG network connections that could result in coverage loss and pattern count increase.

Optionally, you can perform scan chain insertion at the same time as synthesis. This does not effect how you perform DFT insertion, but you will lose some of the automation that Tessent Shell provides during ATPG pattern generation.

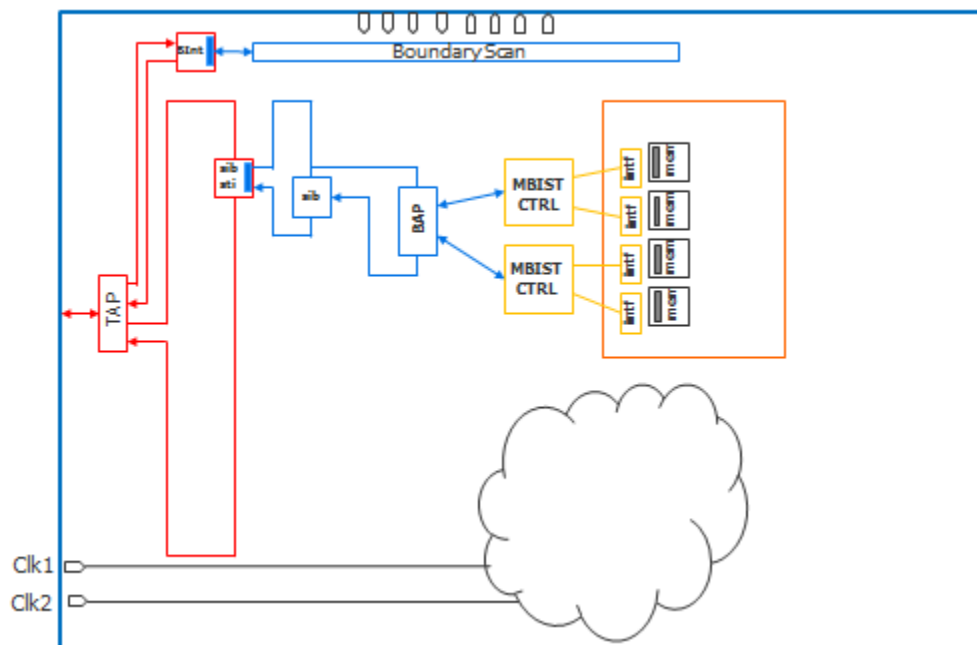
The following figures show an example of the progression of DFT hardware inserted into a DFT-ready design.

Figure 5-2. DFT-Ready Design



In the first DFT insertion pass, Tessent inserts the MemoryBIST and boundary scan hardware.

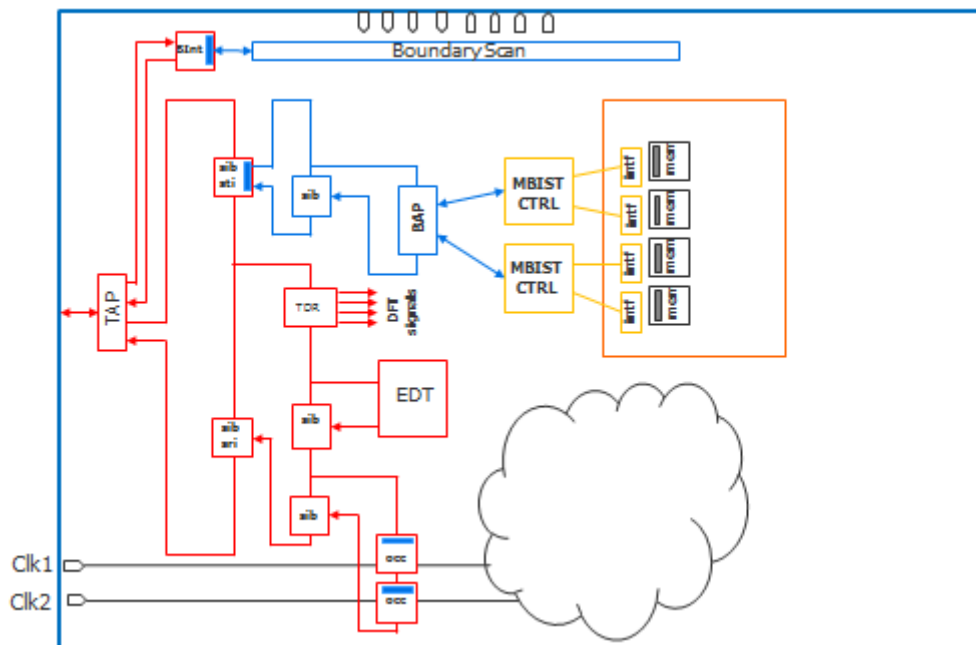
Figure 5-3. After the First DFT Insertion Pass



In the second DFT insertion pass, Tessent inserts the EDT and OCC hardware. You clock the MemoryBIST logic using the same functional clock that feeds the memories, as shown in yellow in [Figure 5-3](#). The IJTAG network (blue) will be scan tested using the IJTAG clock,

which is the TCK clock. The TAP network (red) is not scan tested or is made non-scan during ATPG.

Figure 5-4. After the Second DFT Insertion Pass



First DFT Insertion Pass: MemoryBIST and Boundary Scan

Insert MemoryBIST and boundary scan prior to EDT and OCC so that you can accurately estimate the number of scannable sequential elements (often referred to as “flops”) to be tested. The number of flops determines the size of the EDT controller that Tessent generates in the second DFT insertion pass.

The flow for inserting MemoryBIST and boundary scan together is the same as inserting them separately. For details about this insertion pass flow, refer to:

- “[Getting Started](#)” in the *Tessent MemoryBIST User’s Manual*, or
- “[Getting Started With Tessent BoundaryScan](#)” in the *Tessent BoundaryScan User’s Manual*

Note

 The line numbers used in this procedure refer to the command flow dofile in [Example 5-1](#) on page 109.

Prerequisites

- To insert boundary scan, you must have an RTL design with instantiated IO pads if you are using a chip-level design.
- For RTL netlists, you must have a Tessent cell library or the pad library for the IO pads. For more information, refer to the [Tessent Cell Library Manual](#).

Procedure

1. Load the RTL design data. (See lines 1-7.) The following step is important for the first DFT insertion pass:
 - a. Set the [set_context](#) -design_id switch to “rtl1.”

For a specified design, the -design_id switch stores all the data associated with a particular DFT insertion pass in the TSDB. For information about how Tessent interacts with the TSDB during the DFT insertion flow, refer to “[TSDB Data Flow for the Tessent Shell Flow](#)”. For the first pass, “rtl1” contains the data for the MemoryBIST and boundary scan hardware, and for the JTAG network.

Note



“rtl1” is the recommended naming convention for the design ID for the first insertion pass, but you can specify any name, as desired. Refer to “[Load the Design](#)” for more information about setting the design ID.

2. Set the [set_dft_specification_requirements](#) command to “on” for both -memory_test and -boundary_scan. (See line 11.)

This tells Tessent Shell that you are generating the MemoryBIST and boundary scan hardware at the same time. There is nothing further you need to do. The process is automated.

3. Ensure that you set the design level to “chip.” (See line 12.) Refer to “[Design Levels](#)” for more information.
4. Identify test pins and apply options to special pins. (See lines 14-26.)
5. Apply the [check_design_rules](#) command to instruct the tool to leave setup mode and enter analysis mode. If there are issues with the design, the tool remains in setup mode. (See line 31.)
6. Create the DFT specification. (See lines 33-43.)
 - a. To configure the functional pins so that they are shared as EDT channel input and output pins, add the required logic using the AuxiliaryInput ports and AuxiliaryOutput ports wrappers, respectively.

Tessent Shell allows for functional pins to be shared as EDT channel pins. You must insert auxiliary input and output logic at the same time as you insert boundary scan to avoid cascading two multiplexers along the functional output paths. For additional

information, refer to the [AuxiliaryInputOutputPorts](#) wrapper in the *Tessent Shell Reference Manual*.

Note

 You can also share test_clock, scan_en and edt_update pins with functional pins. The functional coverage is maintained when you use boundary scan during scan test.

7. Create the DFT hardware, IJTAG network connectivity, and input test patterns. (See lines 46-53.)
8. Run simulations to verify the design. (See lines 55-62.)

Results

For MemoryBIST, Tessent inserts the MemoryBIST controllers, interfaces, [BIST Access Port \(BAP\)](#), and [segment insertion bits \(SIBs\)](#). This hardware is later scan-tested using EDT, which you insert during the second insertion pass. In addition, Tessent automatically connects pre-existing scan testable instruments and scan resource instruments to the [Scan Tested Instrument \(STI\)](#) and [Scan Resource Instrument \(SRI\)](#) sides of the IJTAG network, respectively.

For boundary scan, you can segment the boundary scan chain into smaller chains that are used during logic testing with Tessent TestKompress. To segment the boundary scan chain into smaller chains to be connected to the EDT, specify [max_segment_length_for_logictest](#) within the [BoundaryScan](#) wrapper or alternatively specify the following command prior to running create_dft_specification:

```
set_defaults_value DftSpecification/BoundaryScan/max_segment_length_for_logictest
```

Examples

The following dofile example shows a typical command flow as detailed in the Getting Started discussions listed above.

The highlighted command lines illustrate additional considerations for inserting the Tessent Shell MemoryBIST and Tessent Shell Boundary Scan instruments in the first insertion pass of a two-pass DFT insertion process. The functional pins are equipped with logic so that they can be shared as EDT channel input and output pins.

Example 5-1. Dofile Example for MemoryBIST and BoundaryScan

```
1 # Load the design
2 set_context dft -rtl -design_id rtl1
3 set_tsdb_output_directory ../tsdb_rtl
4 read_verilog ../RTL/cpu_top.v
5 read_cell_library ../library/tessent/adk.tcelllib
6 set_design_sources -format tcd_memory -Y ../library/memory \
7   -extension memlib
8 set_current_design cpu_top
9
10 # Specify and verify the DFT requirements
11 set_dft_specification_requirements -memory_test on -boundary_scan on
```

```
12 set_design_level chip
13
14 # Identify TAP pins
15 set_attribute_value tck_p -name function -value tck
16 set_attribute_value tdi_p -name function -value tdi
17 set_attribute_value tms_p -name function -value tms
18 set_attribute_value trst_p -name function -value trst
19 set_attribute_value tdo_p -name function -value tdo
20 set_boundary_scan_port_options ramclk_p -cell_options clock
21 set_boundary_scan_port_options reset_p -cell_options sample
22
23 # Some pins cannot add any boundary scan cells
24 set_boundary_scan_port_options test_clock_p -cell_options dont_touch
25 set_boundary_scan_port_options edt_update -cell_options dont_touch
26 set_boundary_scan_port_options scan_en_p -cell_options dont_touch
27
28 # Specify the clocks feeding the memories
29 add_clocks 0 ramclk_p -Period 5ns
30
31 check_design_rules
32
33 # Create DFT specification
34 set_spec [create_dft_specification]
35 report_config_data $spec
36 set_config_value $spec/BoundaryScan/max_segment_length_for_logictest 150
37 read_config_data -in ${spec}/BoundaryScan -from_string {
38     AuxiliaryInputOutputPorts {
39         auxiliary_input_ports : portain_p[0], portain_p[1], portain_p[2] ;
40         auxiliary_output_ports : portbout_p[0], portbout_p[1],
41         portbout_p[2];
42     }
43 }
44 report_config_data $spec
45
46 # Create the hardware for the DFT inserted with the DFT specification
47 process_dft_specification
48
49 # Extract the IJTAG network connectivity and create ICL for the design
50 extract_icl
51
52 # Create patterns specification for validating the inserted hardware
53 create_patterns_specification
54
55 # Process the patterns specification and create simulation test benches
56 process_patterns_specification
57
58 # Run and check testbench simulations
59 set_simulation_library_sources -v ../library/verilog/adk.v \
60     -v ../library/pad_cells.v -v ../library/memory/ram.v
61 run_testbench_simulations
62 check_testbench_simulations -report_status
```

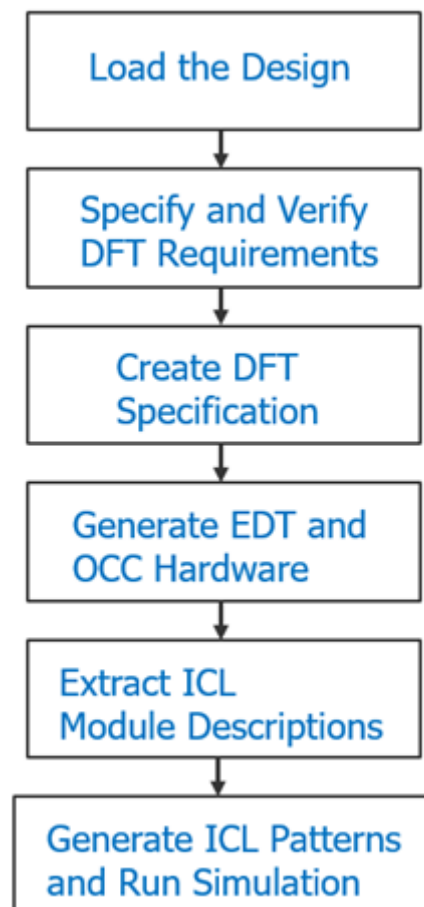
Second DFT Insertion Pass: EDT and OCC

In the second DFT insertion pass, insert the logic test hardware: EDT and OCC. This insertion pass includes requirements and consideration that differ from the first DFT insertion pass, including inserting the DFT signals.

For information about OCC, refer to “[Tessent On-Chip Clock Controller](#)” in the *Tessent Scan and ATPG User’s Manual*.

Before running this flow for chip-level designs, source nodes must be present in the RTL design so that you can define dynamic DFT signals as described in “[Specify and Verify the DFT Requirements](#).” Dynamic DFT signals are signals such as scan enable, edt_clock, edt_update, and so on, that need to change during specific tests.

Figure 5-5. Flow for the Second DFT Insertion Pass



Note

 For the second DFT insertion pass, use the same process for generating ATPG patterns and performing simulation that you used during the first DFT insertion pass.

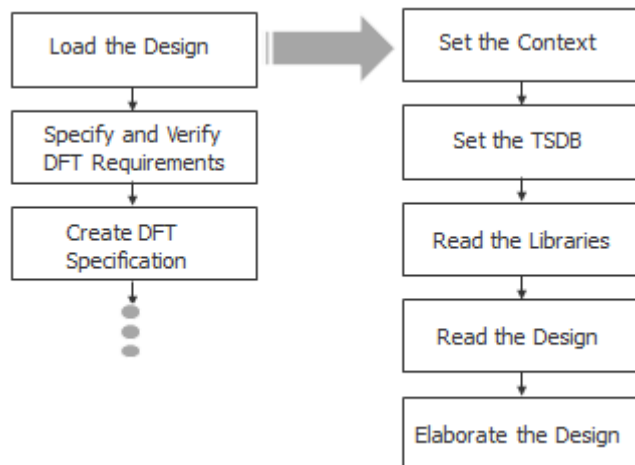
Load the Design	112
Specify and Verify the DFT Requirements	114
Create the DFT Specification	117
Generate the EDT and OCC Hardware	120
Extract the ICL Module Description	120
Generate ICL Patterns and Run Simulation.	121

Load the Design

When loading the design for the second DFT insertion pass, you must ensure that you specify a new design ID name and, optimally, use the same TSDB directory that you used in the first insertion pass. In addition, you must read in the design from the first insertion pass and elaborate the design.

The design loading process for the second pass is similar to the process you used in the first insertion pass, with a few exceptions as described in this section.

Figure 5-6. Flow for Loading the Design



Prerequisites

- For chip-level designs, source nodes must be present in the RTL design so that you can define dynamic DFT signals as described in [“Specify and Verify the DFT Requirements.”](#) Dynamic DFT signals are signals such as scan enable, edt_clock, edt_update, and so on, that need to change during specific tests.

Procedure

1. Apply the [set_context](#) command with the ID “rtl2” for the second pass. For example:

```
set_context dft -rtl design_id rtl2
```


In the first DFT insertion pass, you set the design ID to “rtl1” with the [set_context](#) command.

Note

 This manual uses recommended naming conventions for the design IDs for flat designs, which are “rtl1” for the first DFT insertion pass, “rtl2” for the second DFT insertion pass, and “gate” for the scan chain insertion pass. Refer to “[Considerations for Using Gate-Level Verilog Netlists](#)” for the naming conventions when using gate-level Verilog netlists.

For a specified design, the design ID stores all the data associated with a DFT insertion pass into the TSDB. For the first pass, “rtl1” contains the data for the MemoryBIST and boundary scan hardware. Setting the design_id to “rtl2” at the beginning of the second DFT insertion pass identifies that “rtl2” will store the EDT and OCC hardware data generated during the second pass.

The rtl2 design data is cumulative. That is, it contains the necessary rtl1 data in addition to the new data generated for EDT and OCC. The “rtl2” designation captures the fact that the second insertion pass is performed on the resulting edits of the first pass RTL data. In subsequent insertion passes, you can use either design ID to load the design and its supporting files.

Tessent generates JTAG nodes during both insertion passes and their module names are differentiated using the design ID you specified in each pass.

2. Specify the [set_tsdb_output_directory](#) command with the same directory location for both the first and second DFT insertion passes.

```
set_tsdb_output_directory ../tsdb_rtl
```

If you forget to specify the command for the second DFT insertion pass, Tessent creates a default tsdb_outdir directory in the current working directory. If you use a different output TSDB directory for the two insertion passes, ensure that you open the TSDB used by the first insertion pass by specifying the [open_tsdb](#) command.

For more information about the TSDB, refer to “[Tessent Shell Data Base \(TSDB\)](#)” in the *Tessent Shell Reference Manual*. For information about the TSDB data flow, refer to “[TSDB Data Flow for the Tessent Shell Flow](#).”

3. Specify the [read_cell_library](#) command to read in the library file for the standard cells and the pad IO macros. The following command reads the Tessent cell library file for the standard cells and pad IO macros:

```
read_cell_library ../library/adk_complete.tcelllib
```

If the pad IO library and standard cell library are separate, use the following commands to read in the atpg.lib files and the library for the pad IO description:

```
read_cell_library ../library/atpg.lib
read_cell_library ../library/pad.library
```

4. Specify the [read_design](#) command to load the design:

```
read_design cpu_top -design_id rtl1
```

The design was created in the first DFT insertion pass when you used the [read_verilog](#) and/or [read_vhdl](#) commands. This command also loads supporting files such as the TCD, ICL, and PDL, if present in the TSDB.

For example, the following command reads in the design using the design ID from the first DFT insertion. In this case, this is “rtl1”

To load the design correctly for the second insertion pass, the [read_design](#) command refers to the design source dictionary that was created during the first DFT insertion pass and stored in the [dft_inserted_designs](#) directory.

5. Specify the [set_current_design](#) command to elaborate the design.

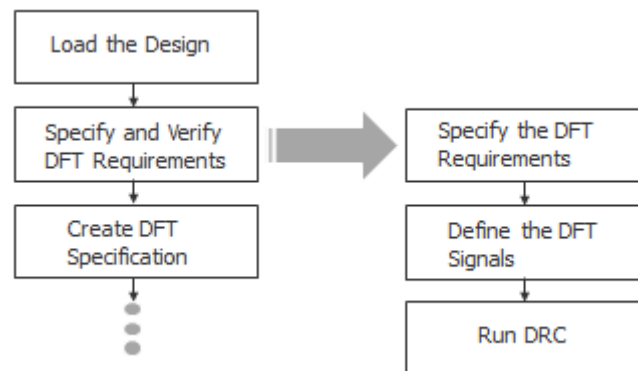
```
set_current_design cpu_top
```

If any module descriptions are missing, design elaboration identifies them. You can fix elaboration errors by adding the missing modules or by specifying the “[add_black_boxes](#) -module” command.

Specify and Verify the DFT Requirements

After loading your design data, define the DFT requirements for the EDT and OCC hardware you will be inserting. This includes adding DFT signals followed by performing design rule checking.

Figure 5-7. Flow for Specifying and Verifying the DFT Requirements



Procedure

1. Specify the [set_dft_specification_requirements](#) command to run pre-DFT design rule checking as follows:

```
set_dft_specification_requirements -logic_test on
```

Since you already specified that you were working at the chip level in the first DFT insertion pass, you do not need to re-specify this information for the second insertion pass.

2. Specify the `add_dft_signals` command to define the DFT signals. For example:

```
add_dft_signals ltest_en
...
```

See “[DFT Signals](#)” on page 115 for a more complete example.

DFT signals are used to determine the various modes of operation, control values and signal behaviors that are necessary for each test mode. The DFT signals capability in Tessent Shell automates the following tasks:

- Adding DFT signals.
- Configuring DFT signals for various modes of operation.
- Transferring information about DFT signals from one step to the next.

Based on the desired mode of operation, Tessent creates the necessary setup procedures to control DFT signals through an JTAG network.

3. Specify the [check_design_rules](#) command to run pre-DFT design rule checking.

When DRC passes, Tessent Shell shifts from setup mode to analysis mode. Pre-DFT DRC verifies that clocks are defined for all of the scannable sequential elements that will be scan tested and identifies the async sets and resets so that they can be turned off during shift operations. In addition, if you have specified the `add_dft_clock_enable` command, Tessent checks clock-gating logic and module-type clocks.

Tessent Shell generates DFT_C violations when error conditions are detected. For details, refer to “[Pre-DFT Clock Rules \(DFT_C Rules\)](#)” in the *Tessent Shell Reference Manual*.

Examples

DFT Signals

You can add static DFT signals and dynamic DFT signals. Static DFT signals include global DFT control, logic test control and scan mode signals. As described in the [add_dft_signals](#) command description, these DFT signals are typically controlled by a Test Data Register that is part of the JTAG network.

Most dynamic DFT signals originate from primary input ports. For chip-level designs, these primary input ports must already be present in the RTL and be pre-connected to a pad buffer cell. The three dynamic DFT signals that originate from primary inputs are `test_clock`, `scan_en`, and `edt_update`. To share their input ports with the functional mode, ensure that you added auxiliary input logic for them during boundary scan insertion. Tessent cannot create the nodes as ports.

The following example shows the required DFT signals for the second insertion pass when you are inserting both EDT and OCC.

```
// Create the static DFT signals
add_dft_signals ltest_en

// Generate dynamic DFT signals from source nodes
add_dft_signals scan_en edt_update test_clock \
    -source_node { porta_in1 portb_in1 porta_in2 }

// Create shift_capture_clock and edt_clock as gated versions of
// test_clock
add_dft_signals shift_capture_clock edt_clock -create_from_other_signals

// Used by top-level EDT
add_dft_signals edt_mode

// Required for boundary scan to be used with logic test without
// contacting inputs
add_dft_signals int_ltest_en output_pad_disable

// Required for scan-tested instruments (STI network) such as MemoryBIST
// and boundary scan
add_dft_signals tck_occ_en

// To bypass memories or to run multi-load ATPG for memories
add_dft_signals memory_bypass_en
```

Note



Tessent Shell automatically recognizes scan-tested instruments and stitches them into scan chains.

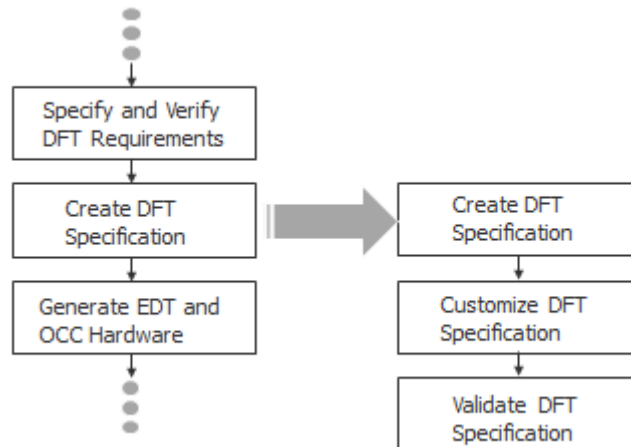
Refer to the *Tessent Shell Reference Manual* for information about the following commands:

- [add_dft_signals](#) — Insert DFT signals.
- [register_static_dft_signal_names](#) — Register your DFT signal, if, for example, you need to augment the default DFT signals for specific usage requirements.
- [report_dft_signal_names](#) — Return a list of available DFT signals.
- [report_dft_signals](#) — View a list of the DFT signals added with the `add_dft_signals` command.
- [delete_dft_signals](#) — Delete any previously specified DFT signals.
- [add_dft_modal_connections](#) — Though not normally required for flat designs, if you have multiple EDT configurations that you want to multiplex into a common set of top-level I/Os, use this command to implement the multiplexing. In addition, if the EDT channel pins are shared with functional pins, they need to include auxiliary input/output muxing logic.

Create the DFT Specification

After defining the DFT requirements for the DFT signals in setup mode, you are ready to create the DFT specification for EDT and OCC in analysis mode. The DFT specification defines how the hardware will be inserted into the design.

Figure 5-8. Flow for Creating the DFT Specification



Procedure

1. Specify the [create_dft_specification](#) command as follows:

```
create_dft_specification -sri_sib_list {occ edt}
```

For EDT and OCC, you must first generate a skeleton DFT specification that contains two empty SRI SIBs that will specify the EDT and OCC sections of the IJTAG network. Creating the DFT specification for EDT and OCC differs from the process you use for MemoryBIST and boundary scan in the first DFT insertion pass.

2. Apply commands to customize the DFT specification using one of the following interfaces:
 - **GUI** — To fill in the EDT and OCC wrapper details for the DFT specification, you can either use the Tessent GUI, known as Tessent Visualizer (see Customizing the DFT Specification for EDT), or the Tessent Shell command line.
 - **Command-Line** — To customize the DFT specification on the command line, type the EDT and OCC data as an argument to the [read_config_data -from_string](#) command. Modify the DFT specification with introspected data using the [add_config_element](#) and [set_config_value](#) commands. Tessent automatically saves modifications to the dofile/scripts directory for use in future sessions. See “[DFT Customization Example](#)” on page 118 for an example.

Note

 Tessent automatically connects the divided boundary scan segment (if present from the first DFT insertion pass) to the EDT hardware that Tessent inserts in the second insertion pass. Refer to [connect_bscan_segments_to_lsb_chains](#) for details.

3. Specify the following command to ensure that no errors exist in the DFT specification:

```
process_dft_specification -validate_only
```

This step should be done before generating the EDT and OCC hardware.

Examples

DFT Customization Example

The following example modifies a DFT specification to add EDT and OCC and saves the changes.

Note

 If you are using Tessent OCC, Tessent Scan automatically identifies and stitches the sub-chains in the OCC into scan chains.

```
read_config_data -in $spec -from_string {
  occ {
    ijtag_host_interface : Sib(occ);
  }
}
# The following illustrates a generic way to populate the OCC. The clock
# list is design specific and needs to be updated for the design to the
# OCC, scan_enable and shift_capture_clock are connected automatically
# Modify the following for your specific design requirements
set id_clk_list [list \
  clk1 clk1_p \
  clk2 clk2_p \
  clk3 clk3_p \
  clk4 clk4_p \
  ramclk_p ramclk_p \
]

foreach {id clk} $id_clk_list {
  set occ [add_config_element OCC/Controller($id) -in $spec]
  set_config_value clock_intercept_node -in $occ $clk
}
report_config_data $spec
```

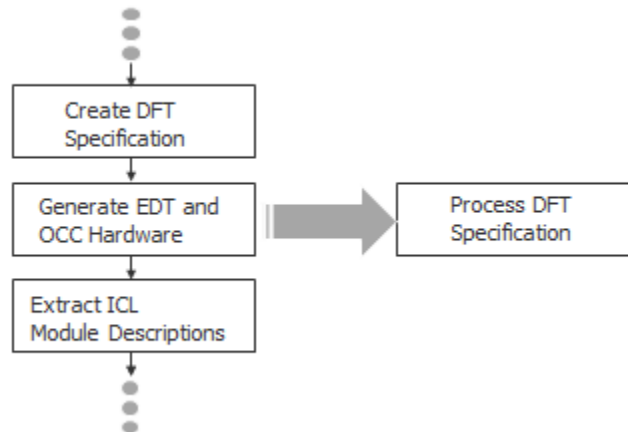
```
# To the EDT controller, the edt_clock and edt_update are connected
# automatically. The EDT controller is built-in with bypass
# Modify the following for your specific design requirements
read_config_data -in $spec -from_string {
    EDT {
        ijttag_host_interface : Sib(edt);
        Controller (c1) {
            longest_chain_range : 65, 100;
            scan_chain_count : 85;
            input_channel_count : 3;
            output_channel_count : 3;
            Connections +{
                EdtChannelsIn(1) {
                }
                EdtChannelsIn(2) {
                }
                EdtChannelsIn(3) {
                }
                EdtChannelsOut(1) {
                }
                EdtChannelsOut(2) {
                }
                EdtChannelsOut(3) {
                }
            }
        }
    }
}

# Connecting the EDT channel in to the primary input and connecting
# EDT channel out to the primary output
# You had inserted the auxiliary logic for these ports during the first
DFT insertion pass
set_config_value port_pin_name \
    -in $spec/EDT/Controller(c1)/Connections/EdtChannelsIn(1) \
[get_single_name [get_auxiliary_pins portain_p[0] -direction input]]
set_config_value port_pin_name \
    -in $spec/EDT/Controller(c1)/Connections/EdtChannelsIn(2) \
[get_single_name [get_auxiliary_pins portain_p[1] -direction input]]
set_config_value port_pin_name \
    -in $spec/EDT/Controller(c1)/Connections/EdtChannelsIn(3) \
[get_single_name [get_auxiliary_pins portain_p[2] -direction input]]
set_config_value port_pin_name \
    -in $spec/EDT/Controller(c1)/Connections/EdtChannelsOut(1) \
[get_single_name [get_auxiliary_pins portbout_p[0] -direction output] ]
set_config_value port_pin_name \
    -in $spec/EDT/Controller(c1)/Connections/EdtChannelsOut(2) \
[get_single_name [get_auxiliary_pins portbout_p[1] -direction output] ]
set_config_value port_pin_name \
    -in $spec/EDT/Controller(c1)/Connections/EdtChannelsOut(3) \
[get_single_name [get_auxiliary_pins portbout_p[2] -direction output] ]
report_config_data $spec
```

Generate the EDT and OCC Hardware

After creating the DFT specification, generate the EDT and OCC hardware. The `process_dft_specification` command generates and inserts the DFT hardware—as defined by the DFT specification—into your design.

Figure 5-9. Flow for Inserting the Second-Pass Hardware



Procedure

1. Specify the `process_dft_specification` command to insert EDT and OCC in the second pass.

```
process_dft_specification
```

2. If you want to generate the hardware, but not insert it into the design, specify the following command:

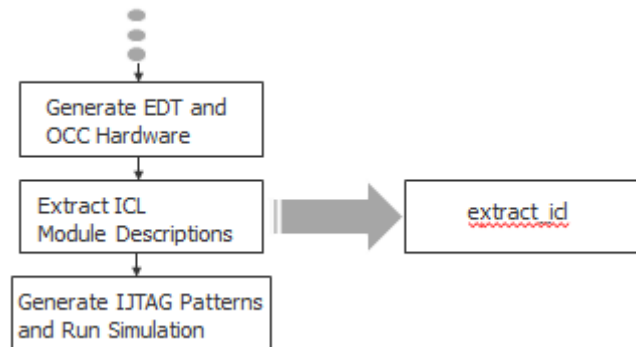
```
process_dft_specification -no_insertion
```

You can then insert the hardware into the design manually.

Extract the ICL Module Description

After inserting the EDT and OCC hardware, verify the connectivity of the ICL modules that were inserted with the `process_dft_specification` command. This is a preparatory step for ICL pattern generation.

Figure 5-10. Flow for Extracting ICL



Procedure

1. Specify the [extract_icl](#) command to find all modules (both Tessent instruments and non-Mentor Graphics instruments) and their associated ICL descriptions, and to run DRC to verify their connectivity.

The top-level ICL description corresponds to the design name you specified with the `set_current_design` command during the first insertion pass (which is also the same design name you specified when you elaborated the design at the beginning of the second insertion pass).

2. Specify the [analyze_drc_violation](#) command to debug the DRC violations.

Tessent generates I-rule errors when it detects ICL extraction errors and opens Tessent Visualizer to a schematic view of the error. For more information about the DRC I-rule errors, refer to “[ICL Extraction Rules \(I Rules\)](#)” in the *Tessent Shell Reference Manual*. For details about using Tessent Visualizer, refer to “[Tessent Visualizer](#)” on page 379.

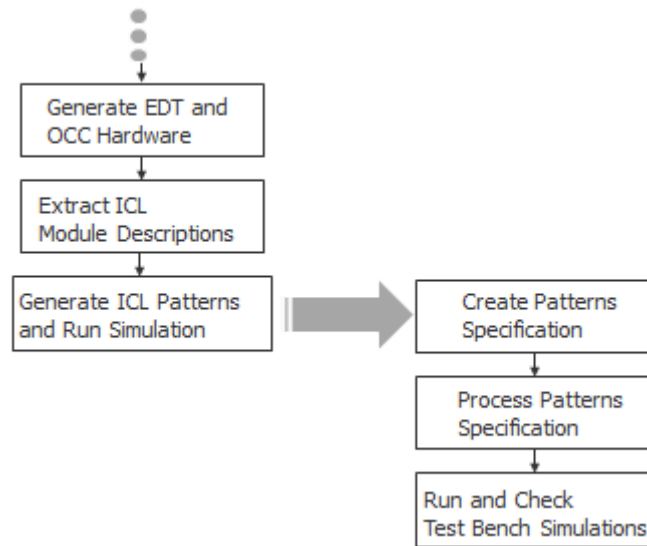
The `extract_icl` command also creates a Synopsys Design Compiler file that you can use for synthesis. Refer to “[Timing Constraints \(SDC\)](#)” for more information.

Generate ICL Patterns and Run Simulation

The process for generating and simulating the ICL verification patterns for EDT and OCC is identical to the process you used during the first DFT insertion pass for MemoryBIST and boundary scan.

The following figure shows the tasks required to complete the second insertion pass and move on to synthesis.

Figure 5-11. Flow for Generating and Simulating ICL Patterns



Procedure

1. Generate the test patterns for the design:

```
create_patterns_specification
```

2. Validate and process the content of the test patterns:

```
process_patterns_specification
```

3. Run and check testbench simulations. For example:

```
# Run and check testbench simulations
set_simulation_library_sources -v ../library/verilog/adk.v \
    -v ../library/pad_cells.v -v ../library/memory/ram.v

run_testbench_simulations
check_testbench_simulations -report_status
```

Perform Synthesis

Synthesize the original RTL as well as the DFT-inserted RTL for MemoryBIST, boundary scan, EDT and OCC. For RTL designs, perform synthesis once after performing the first and second DFT insertion passes.

Prerequisites

- Refer to “[Synthesis Guidelines for RTL Designs with Tessent Inserted DFT](#)” on page 499.
- Tessent Shell supports synthesis using Synopsys Design Compiler. For details, refer to “[Timing Constraints \(SDC\)](#)” for more information.

Procedure

Specify the [write_design_import_script](#) command to create a dc_shell design load script that you can use to load the RTL design as it exists after the two DFT insertion passes. For example:

```
write_design_import_script for_synthesis.tcl -replace
```

Note

 If you are not using Synopsys Design Compiler as your third-party synthesis tool, you can still use the command to create a dc_shell script and adjust it to match your synthesis tool's command set.

```
# You can generate the synthesis script only without running
# synthesis using the following command
# run_synthesis -startup_file ./synopsys_dc.setup
# To generate a script for third-party synthesis
# run_synthesis -generate_script_only
```

Perform Scan Chain Insertion

During scan chain insertion, Tessent inserts and stitches scan chains on the gate-level netlist that was synthesized after DFT insertion. After you run scan chain insertion, proceed to ATPG pattern generation.

During scan insertion using Tessent Scan, the non-scan instances such as EDT are automatically understood. In addition, the built-in OCC sub-chains are understood and stitched into scan chains. Tessent uses DFT signals such as the scan enable that you specified in the previous insertion passes, therefore you do not need to specify them again.

For information about scan chain insertion using Tessent Shell, refer to the “[Internal Scan and Test Circuitry Insertion](#)” in the *Tessent Scan and ATPG User's Manual*.

Procedure

1. Specify the following command to set the DFT context:

```
set_context dft -scan -design_id gate
```

Once again, when setting the context, ensure that you specify the design ID with a unique name. The recommended name is “gate” for a flat design, although you can choose a name as desired. As described in “[Load the Design](#),” for a gate-level netlist the recommended name is “gate3”.

2. Load the synthesized netlist. For example:

```
read_verilog ../Synthesis/cpu_top_synthesized.vg
```

This netlist contains the gates for the original RTL design and the DFT-inserted hardware.

3. Specify the same output directory you used in the first and second DFT passes:

```
set_tsdb_output_directory ../tsdb_rtl
```

4. Load the rtl2 design data for the DFT hardware you previously inserted. For example:

```
read_design cpu_top -design_id rtl2 -no_hdl
```

The `-no_hdl` switch specifies to read in all of the DFT data files—such as ICL, PDL and TCD—except for the design files. (You are using the synthesized design from this point forward.)

After design elaboration and design rule checking, Tessent Shell transitions from Setup mode to Analysis mode.

5. Specify the `add_scan_mode` `edt_mode` command to connect the scan chains to the EDT signals and EDT hardware that you inserted during the second insertion pass.

The use of pre-registered DFT signal “`edt_mode`” as the scan mode using the `add_scan_mode` command infers the `-enable_dft_signal` also as the `edt_mode` DFT signal.

Results

The scan stitched and inserted netlist is located in the TSDB under the design ID “gate” for RTL designs or “gate3” for gate-level netlists. Refer to “[Considerations for Using Gate-Level Verilog Netlists](#)” for details.

Examples

The following dofile shows a command flow for scan insertion and stitching:

```
set_context dft -scan -design_id gate
read_cell_library ../library/tessent/adk.tcelllib
read_verilog ../Synthesis/cpu_top_synthesized.vg
set_tsdb_output_directory ../tsdb_rtl
read_design cpu_top -design_id rtl2 -no_hdl
set_current_design cpu_top

check_design_rules

set edt_instance [get_name_list [get_instance -of_module \
                                [get_name [get_icl_module -filter \
                                tessent_instrument_type==mentor::edt]]]]
add_scan_mode edt_mode -edt_instance $edt_instance
analyze_scan_chains
report_scan_chains
insert_test_logic
exit
```

Perform ATPG Pattern Generation

After inserting scan chains, proceed to ATPG pattern generation.

Procedure

1. Do one of the following:
 - If you used Tessent Scan to insert the scan chains, specify the [import_scan_mode](#) `edt_mode` command for ATPG pattern generation.
 - If you did not use Tessent Scan for scan insertion, use the TCD IP mapping flow as described in “[Running ATPG Patterns without Tessent Scan](#)” in the *Tessent Scan and ATPG User’s Manual*.

When you specify the `import_scan_mode` command, Tessent Shell passes through the scan-insert design data for the EDT and OCC logic. This data includes the scan structures (scan chains and scan channels) that are stored in the TSDB under the “gate” design ID. The gate design ID was created during scan insertion when you specified the [insert_test_logic](#) command.

In addition, Tessent automatically creates and simulates the `test_setup` procedure cycles that are required to initialize the EDT and OCC static signals. You only need to specify non-default parameter values, if, for example, you run EDT with bypass on or set `int_test_en` to 1 to use the boundary scan as the source of the core values and isolate the ATPG test from the top-level IOs.

You can create ATPG patterns for any mode that you need, such as stuck-at and transition. These patterns are stored in the TSDB database under the [logic_test_cores](#) directory. The `import_scan_mode` command uses the same scan configuration—that is, the DFT signals—that were used for the `add_scan_mode` command during scan insertion.

2. Specify the `set_current_mode` command to indicate the type of pattern you are generating (see “[Stuck-at ATPG patterns](#)” on page 125 and “[Transition at-speed ATPG patterns](#)” on page 126).

In addition, the name you give to the generated ATPG pattern sets must differ from the mode name you specify for `import_scan_mode` (that is, “`edt_mode`”).

3. Specify the `write_patterns` command to write out the Verilog testbenches and STIL patterns.
4. Specify the `write_tsdb_data` command to save the flat model, fault list, PatDB and TCD files into the TSDB.

For details about ATPG pattern generation, refer to “[Running ATPG Patterns](#)” in the *Tessent Scan and ATPG User’s Manual*.

Examples

Stuck-at ATPG patterns

The following example generates stuck-at ATPG patterns as indicated by the `set_current_mode` `edt_stuck` command. It shows that you have a choice between using the boundary scan chains for capture during ATPG or using the primary pins of the chips (using the pads).

```
set_context patterns -scan
read_cell_library ../library/tessent/adk.tcelllib
read_cell_library ../library/memory/memory.lib
open_tsdb ../tsdb_rtl
read_design cpu_top -design_id gate
set_current_design cpu_top
set_current_mode edt_stuck
import_scan_mode edt_mode

# To apply the patterns through the boundary scan chains and not through
# the pads use:
# set_static_dft_signal_values int_ltest_en 1
# set_static_dft_signal_value output_pad_disable 1
# To allow the shift_capture_clock during capture phase on Scan Tested
# Instruments:
set_system_mode analysis
create_patterns
write_tsdb_data -replace
write_patterns patterns/cpu_top_stuck_parallel.v -verilog -parallel \
-replace -scan -parameter_list {SIM_KEEP_PATH 1}
write_patterns patterns/cpu_top_stuck_serial.v -verilog -serial -replace \
-parameter_list {SIM_KEEP_PATH 1}
exit
```

Transition at-speed ATPG patterns

The following example generates transition at-speed patterns. The `import_scan_mode` command uses the `-fast_capture_mode` switch to indicate that the OCC supplies the fast clock during capture. The `set_current_mode` command indicates `edt_transition`, and the `set_fault_type` command indicates transition faults.

```
set_context patterns -scan
read_cell_library ../library/tessent/adk.tcelllib
read_cell_library ../library/memory/memory.lib
open_tsdb ../tsdb_rtl
read_design cpu_top -design_id gate
set_current_design cpu_top
set_current_mode edt_transition
import_scan_mode edt_mode -fast_capture_mode on -verbose

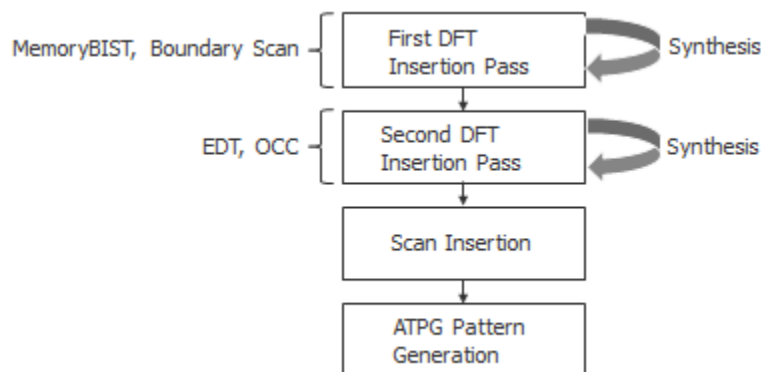
set_static_dft_signal_values int_ltest_en 1
set_static_dft_signal_value output_pad_disable 1
set_system_mode analysis
set_fault_type transition
set_external_capture_options -pll_cycles 5 [lindex [get_timeplate_list] 0]
create_patterns
write_tsdb_data -replace
write_patterns patterns/cpu_top_transition_parallel.v -verilog \
-parallel -replace -scan -parameter_list {SIM_KEEP_PATH 1}
write_patterns patterns/cpu_top_transition_serial.v -verilog -serial \
-replace -parameter_list {SIM_KEEP_PATH 1}
exit
```

Considerations for Using Gate-Level Verilog Netlists

In some cases, you may only have a gate-level Verilog netlist, not the RTL. You can use Tessent Shell to perform the DFT insertion flow when you have a gate-level netlist. The flow is similar to the RTL and scan DFT insertion flow. The main difference is that you must perform synthesis after each DFT insertion pass.

Figure 5-12 shows the gate-level DFT insertion sequence in which you insert MemoryBIST and boundary scan in the first insertion pass and then immediately run synthesis on these instruments before inserting the logic test instruments for the second DFT insertion pass (EDT, OCC, and so on). After inserting the logic test instruments, run synthesis a second time before proceeding to scan chain insertion.

Figure 5-12. Two-Pass Insertion Flow for RTL, Gate-Level Designs



The following differences apply when using a gate-level Verilog netlist rather than RTL. Other than these differences, plus performing synthesis after each DFT insertion pass, you can follow the flow as described starting with “[First DFT Insertion Pass: MemoryBIST and Boundary Scan](#).”

- Prerequisites. You must have the Tessent cell library or the ATPG library for the standard cells, in addition to the Tessent cell library for the IO pad cells.
- Load the design. During the first and second DFT insertion passes, ensure the following:
 - Specify the [set_context](#) command with the -no_rtl option rather than the -rtl option.
 - When setting the context, the recommended naming conventions for the design IDs are “gate1” for the boundary scan/MemoryBIST insertion pass and “gate2” for the EDT/OCC insertion pass. If you are following this convention, you would then use design ID “gate3” for scan insertion.
 - Use the [read_cell_library](#) command to read in the library files for both the standard cells and pad IO macros that are instantiated in the design.

Tessent Shell Flow for Hierarchical Designs

Hierarchical design methodologies provide efficiencies for large designs. Rather than design at the chip-level, chip designers break designs down into RTL blocks that allow design teams to work on different functional blocks in parallel. This method streamlines the process and allows RTL modifications, engineering change orders (ECOs) and other changes to be localized to particular blocks.

Tessent Shell uses the same “divide and conquer” methodology for hierarchical DFT by allowing you to perform RTL and scan DFT insertion at the sub-physical block level rather than only at the chip level. Implementing DFT hierarchically in a bottom-up process starting with the lowest level blocks not only helps to divide the task but also makes the task manageable.

Tip

 Before performing DFT on a hierarchical design, familiarize yourself with “[DFT Architecture Guidelines for Hierarchical Designs](#)” on page 85.

This section builds on what you learned in “[Tessent Shell Flow for Flat Designs](#)” to describe the pre-layout RTL and scan DFT insertion process for hierarchical designs.

Refer to the following testcase for a detailed usage example of the flow described in this section:

```
tessent_example_hierarchical_flow_<software_version>.tgz
```

You can access this testcase by navigating to the following directory:

```
<software_release_tree>/share/UsageExamples/
```

Hierarchical DFT Terminology	129
How the DFT Insertion Flow Applies to Hierarchical Designs	132
RTL and Scan DFT Insertion Flow for Physical Blocks	133
RTL and Scan DFT Insertion Flow for the Top Chip	150
RTL and Scan DFT Insertion Flow for Sub-Blocks	165
RTL and DFT Insertion Flow with Third-Party Scan	169

Hierarchical DFT Terminology

Tessent Shell uses a particular set of terms related to working with blocks in a hierarchical design. You must understand these terms to perform the RTL and scan DFT insertion process on hierarchical designs.

Refer to “[set_design_level](#)” in the *Tessent Shell Reference Manual* for more information.

- **Physical block.** Physical blocks are logical entities that remain intact through tapeout. They are synthesis and layout regions. Below the top level of a chip, these are blocks that you can reuse, or instantiate, within a chip or across multiple chips. You perform synthesis on these blocks independent of the rest of the chip design.

When performing DFT insertion on physical blocks, Tessent preserves the ports at the root of the physical block. Instances of the physical block that exist below the current physical block may not be preserved in the final layout when ungrouping is used.

In Tessent Shell, the hierarchical DFT insertion flow distinguishes between three types of physical blocks: wrapped cores, unwrapped cores, and chip.

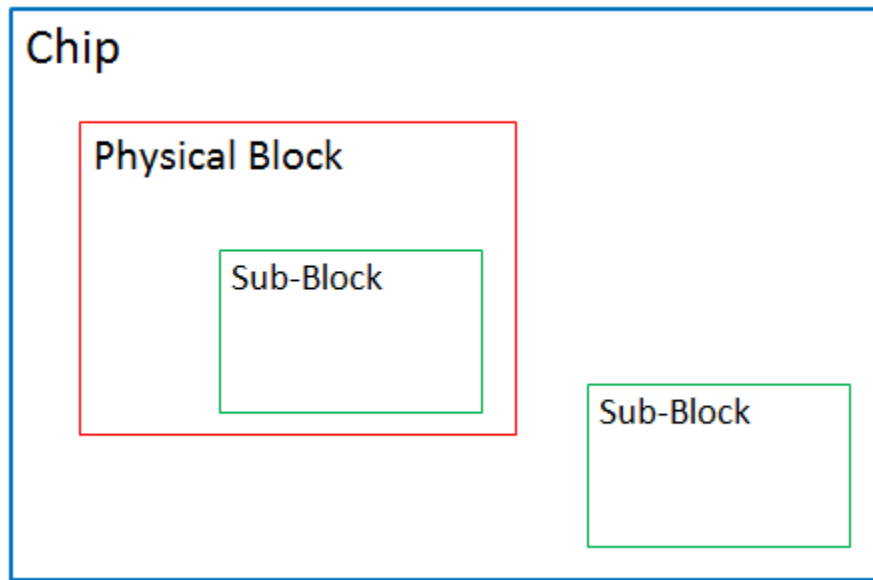
- **Wrapped core.** Wrapped cores contain wrapper cells that are used to isolate the internal logic of the core. Wrapper cells are inserted when you perform scan chain insertion. Wrapped cores are required to make the cores reusable through ATPG pattern retargeting. Wrapped cores can contain sub-blocks. (See description below.)
- **Unwrapped core.** Unwrapped cores do not contain wrapper cells but can contain sub-blocks.

For additional information, refer to “[Unwrapped Cores Versus Wrapped Cores](#)” in the *Tessent Scan and ATPG User’s Manual*.

- **Chip.** The chip is the top-level physical block—that is, the entire design—in which you typically find the pad IO macros and clock controllers. A chip may include another physical block or sub-block. Physical blocks can be wrapped cores or unwrapped cores. Unlike the other types of physical blocks, chips are layout regions.
- **Sub-block.** Sub-blocks are designs that exist within parent blocks and are synthesized with their parent blocks, which could be wrapped cores, unwrapped cores or the top level of the chip. Sub-blocks merge into their parent physical blocks during synthesis of the parent block. Refer to [set_design_level](#) for a details.

Sub-blocks are not layout physical regions. After layout is performed on the post-layout netlist, the sub-block module boundary may or may not be preserved. Sometimes the same sub-block is instantiated at both the physical block level and the chip level as shown below.

Figure 5-13. Hierarchical Design Levels



You can insert DFT hardware such as MemoryBIST, EDT, and OCC into sub-blocks, but you perform synthesis and scan insertion at the sub-block's parent physical block level (where the sub-block is instantiated). To learn about how you use sub-blocks within the Tessent Shell flow, refer to [“RTL and Scan DFT Insertion Flow for Sub-Blocks.”](#)

- **ATPG (or scan) pattern retargeting.** The process by which Tessent Shell preserves the ATPG patterns associated with wrapped cores for purposes of reuse when testing the logic at the parent instantiation level. This means you do not have to re-generate the patterns when you process the top level of the chip. Instead, you retarget the wrapped core ATPG patterns to the top level. Every instantiation of a wrapped core includes its associated ATPG patterns.

For details, refer to [“Scan Pattern Retargeting”](#) in the *Tessent Scan and ATPG User's Manual*.

For purposes of ATPG pattern retargeting and graybox modeling (see description below), Tessent Shell differentiates between a wrapped core's internal circuitry and its external circuitry.

- **Internal mode.** Internal mode is the view into the wrapped core from the wrapper cells. That is, the logic completely internal to the core. Tessent Shell retargets internal mode ATPG patterns during ATPG pattern generation for the chip-level design.
- **External mode.** External mode indicates the view out of the wrapped core from the wrapper cells. That is, the logic that connects the wrapped core to external logic. Tessent Shell uses the external mode to build graybox models, which are used by the internal modes of their parent physical blocks.

- **Graybox.** Graybox models are wrapped core models that preserve only the core's external mode logic along with the portion of the IJTAG network needed for test setup of the logic test modes. The purpose of graybox models in the bottom-up hierarchical DFT process is to retain the minimum logic required to generate ATPG patterns for the internal modes of the parent physical blocks. For details, refer to "[Graybox Overview](#)" in the *Tessent Scan and ATPG User's Manual* and "[Graybox Model](#)" on page 90.

How the DFT Insertion Flow Applies to Hierarchical Designs

Performing RTL and scan DFT insertion on hierarchical designs assumes basic knowledge of the RTL and scan insertion flow for flat designs. The process for hierarchical designs builds on the process you would use for a flat design.

As described in “[Overview of the RTL and Scan DFT Insertion Flow](#),” you perform a two-pass pre-scan insertion process for flat designs. For hierarchical designs, the same flow applies except that you perform the insertion process in a bottom-up manner for each core and sub-block in the design. After you have inserted DFT into all the lower-level physical blocks, you perform the same insertion into the parent blocks until you have reached the top level of the chip.

The following considerations apply when performing DFT insertion on a hierarchical design as opposed to a flat design:

- When performing hierarchical DFT, you must specify the hierarchical design level at which you are performing the DFT insertion process. For flat designs, the [set_design_level](#) command is always set to “chip.” For hierarchical designs, you can also specify “physical_block” or “sub_block.”
- Inserting boundary scan during the first DFT insertion pass typically applies to the chip design level. For hierarchical designs, this means that for cores and sub-blocks you insert only MemoryBIST during the first DFT insertion pass unless you have cores with embedded pad IO macros. If this is the case, then you need to insert boundary scan into the pad IOs using the [embedded boundary scan](#) flow as described in the *Tessent Boundary Scan User’s Manual*.
- Within the hierarchical flow, each physical block and sub-block has a unique design name and, ideally, should have its own TSDB.

Tip

 To facilitate data management, save each design (whether core, sub-block, or chip) in its own TSDB directory. This is the recommended practice when using Tessent Shell for DFT insertion. Using different directories ensures that you can run all sibling physical and sub-blocks in parallel without causing read-write errors into the TSDB directories between the parallel runs. Only when all the child physical and sub-blocks of a given block are completed can you then implement the DFT into the given block. Refer to “[TSDB Data Flow for the Tessent Shell Flow](#)” for more information.

- Perform ATPG pattern retargeting of core-level patterns when you process the parent physical block of the wrapper cores, as shown in section “[Top-Level ATPG Pattern Generation Example](#).”

RTL and Scan DFT Insertion Flow for Physical Blocks

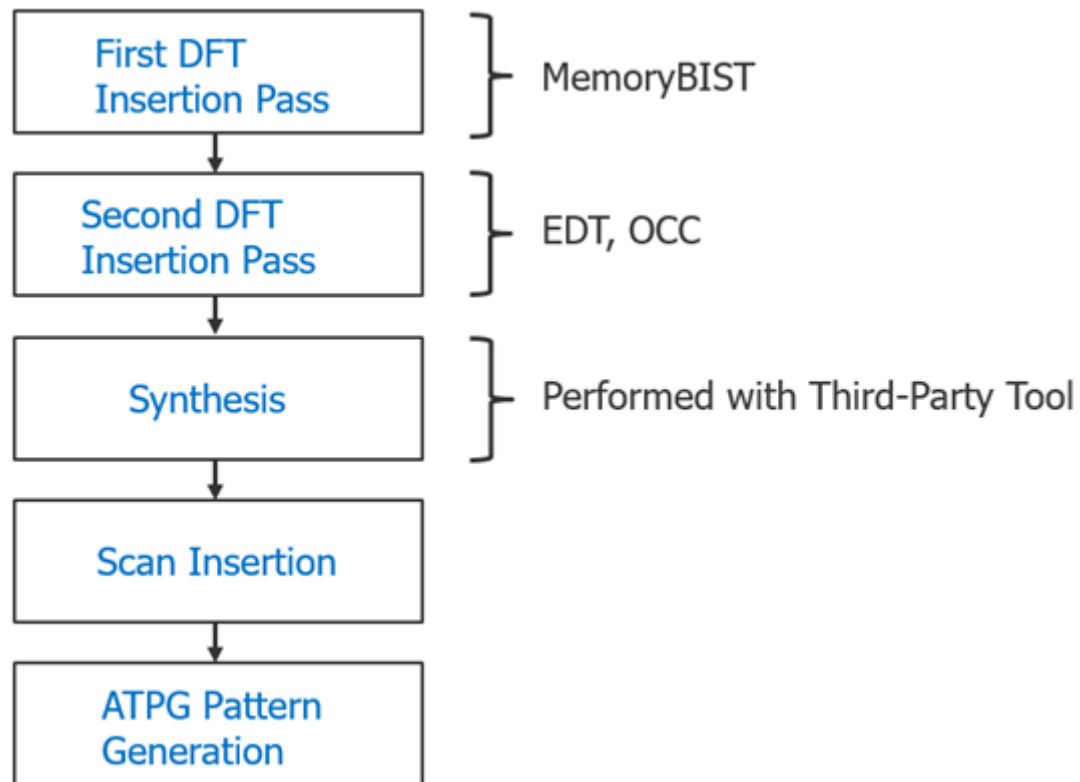
For each wrapped core, perform a two-pass pre-scan DFT insertion process as you would for a flat design except that in the first DFT insertion pass, do not insert boundary scan unless you have embedded pad IO macros present inside the core. This section describes the flow details that are unique to working with wrapped cores as opposed to flat designs.

Refer to “[Tessent Shell Flow for Flat Designs](#)” for overall flow details.

Note

This discussion assumes your design consists of wrapped cores as your lower level physical blocks, and the wrapped cores do not contain embedded pad IOs.

Figure 5-14. Two-Pass Insertion Flow for RTL, Wrapped Cores



First DFT Insertion Pass: Performing Block-Level MemoryBIST.....	134
Second DFT Insertion Pass: Block-Level EDT and OCC	136
Specify and Verify the DFT Requirements: DFT Signals for Wrapped Cores	139
Perform Scan Chain Insertion	141
Perform ATPG Pattern Generation: Wrapped Core.....	144
Run Recommended Validation Step for Pre-Layout Design Sign Off	148

First DFT Insertion Pass: Performing Block-Level MemoryBIST

Because you are not inserting boundary scan at the same time as MemoryBIST (as in the flat flow), you can follow the DFT insertion process for Tessent MemoryBIST.

Refer to “[Getting Started](#)” in the *Tessent MemoryBIST User’s Manual*.

Note



The line numbers used in this procedure refer to the command flow dofile in [Example 5-2](#).

Procedure

1. Load the RTL design data. (See lines 1-22.) The following steps are important for the first DFT insertion pass for wrapped cores:

- a. Set the [set_context](#) -design_id switch to “rtl1.”

All the data associated with MemoryBIST insertion for the wrapped core will be stored under the ID “rtl1” in the wrapped core’s TSDB.

Note



“rtl1” is the recommended naming convention for the design ID for the first insertion pass, but you can specify any name, as desired. Refer to “[Load the Design](#)” for more information about setting the design ID.

- b. Specify the [read_verilog](#) command with a list of the RTL filenames and locations for Tessent to read and compile for the design. For example:

```
../rtl/omsp_timerA_defines.v
../rtl/omsp_timerA_undefines.v
../rtl/omsp_timerA.v
../rtl/omsp_wakeup_cell.v
../rtl/omsp_watchdog.v
../rtl/openMSP430_defines.v
../rtl/openMSP430_undefines.v
../rtl/openMSP430.v
../rtl/processor_core.v
```

- c. Set the design level to “physical_block” so that the layout of this core is maintained as an independent logical entity through tapeout.
2. Create the DFT specification. (See lines 24-30.)
 3. Generate the MemoryBIST hardware and extract the ICL. (See lines 32-36.)
 4. Create the input test patterns and simulation test benches. (See lines 38-41.)
 5. Run simulations to verify the design. (See lines 43-47.)

Examples

The following dofile example shows a typical command flow as detailed in the procedure listed above.

Example 5-2. Dofile Example for MemoryBIST in Physical Blocks

```
1 # Set context to dft and indicate DFT insertion into an rtl-level design
2
3 set_context dft -rtl -design_id rtl1
4
5 # Set the TSDB directory location to be used
6
7 set_tsdb_output_directory ../tsdb_core
8
9 # Read in the memory library model
10 set_design_sources -format tcd_memory -y ../../../../library/memory \
11   -extension tcd_memory
12
13 # Read in memory Verilog model
14 set_design_sources -format verilog -v {../../../../library/memory/*.v }
15
16 # Read in the design
17 read_verilog -f rtl_file_list
18
19 set_current_design processor_core
20
21 # Set the design level as a physical_block
22 set_design_level physical_block
23
24 # Specify to insert memory test
25 set_dft_specification_requirements -memory_test on
26 add_clocks 0 dco_clk -period 3
27 check_design_rules
28 report_memory_instances
29 set spec [create_dft_specification]
30 report_config_data $spec
31
32 # Generate the memoryBIST hardware
33 process_dft_specification
34
35 # Create ICL for this design
36 extract_icl
37
38 # Generate testbenches
39 create_patterns_specification
40 process_patterns_specification
41 set_simulation_library_sources -v ../../../../library/verilog/adk.v
42
43 # Run simulations
44 run_testbench_simulations
45
46 # If simulation fails use the following command
47 //check_testbench_simulations
48 exit
```

Second DFT Insertion Pass: Block-Level EDT and OCC

In the second DFT insertion pass for wrapped cores, insert the EDT and OCC hardware. Unlike flat designs, which use standard OCCs, for wrapped cores you have a choice between inserting OCCs of type child or type standard.

Refer to “[On-Chip Clock Controller Design Description](#)” in the *Tessent Scan and ATPG User’s Manual* for details.

If your physical design implementation does not support using a clock MUX in the clock path, then you can use an OCC of type child. With the child type OCC, only clock chopping and clock gating functions occur inside the wrapped core, and these functions are sufficient for ATPG pattern retargeting (later in the flow) as long as at the chip-level you have included a parent OCC or other hardware that can perform clock selection.

Clock selection is used to select between shift and capture clocks when generating ATPG patterns for the internal mode of the core. Typically, scan chain shifting occurs at a significantly slower speed than the capture clock, hence the need for the clock.

Most of the steps for the second DFT insertion pass for wrapped cores are the same as those you would perform for a flat design. Refer to the applicable sections.

Note



The line numbers used in this procedure refer to the command flow dofile in [Example 5-3](#).

Procedure

1. [Load the Design](#). (See lines 1-10.)
2. [Specify and Verify the DFT Requirements: DFT Signals for Wrapped Cores](#). (See lines 12-32.)

Note



Wrapped cores have their own DFT requirements for the clock signals

3. [Create the DFT Specification](#). (See lines 34-38.)
4. [Generate the EDT and OCC Hardware](#). (See lines 40-78.)
5. [Extract the ICL Module Description](#). (See lines 80-84.)
6. [Generate ICL Patterns and Run Simulation](#). (See lines 86-95.)

Examples

The following dofile example shows that you set the design ID to “rtl2” for the second DFT insertion pass, set the internal mode and external mode for the wrapped core, and have chosen to specify an OCC of type child.

Example 5-3. Dofile for Second DFT Pass

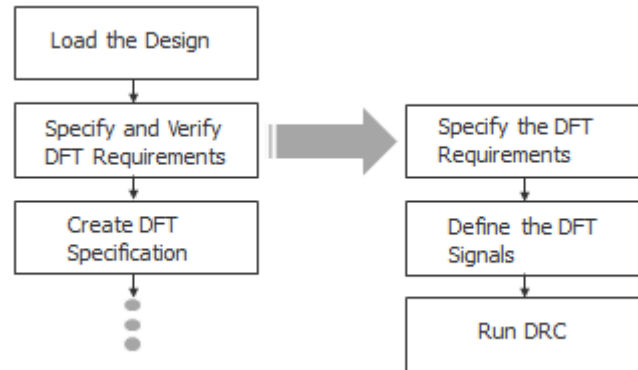
```
1 # Set context to dft and specify design_id as rtl2
2 set_context dft -rtl -design_id rtl2
3
4 # Specify where the tsdb_outdir is to be located, default is at the
5 # current working directory
6 set_tsdb_output_directory ../tsdb_core
7
8 # Use read_design with the design ID from the first DFT insertion pass
9 read_design processor_core -design_id rtl1
10 set_current_design processor_core
11
12 # No need to reset the design level, which remains "physical_block" from
13 # the first pass
14 # Add DFT signals for the wrapped core
15
16 # The following commands required for logic test
17 add_dft_signals ltest_en -create_with_tdr
18 add_dft_signals scan_en edt_update test_clock -source_node \
19 { scan_enable edt_update test_clock_u }
20 add_dft_signals edt_clock shift_capture_clock -create_from_other_signals
21
22 # Required for memories to be tested with multi-load ATPG patterns
23 add_dft_signals memory_bypass_en -create_with_tdr
24
25 # Required for STI network to be tested during logic test
26 add_dft_signals tck_occ_en -create_with_tdr
27
28 # Required for hierarchical DFT and will be used during scan insertion
29 # Specifying both the internal mode and the external mode
30 add_dft_signals int_ltest_en ext_ltest_en int_mode ext_mode
31
32 report_dft_signals
33
34 # Run pre-DFT DRC
35 set_dft_specification_requirements -logic_test on
36
37 check_design_rules
38 set_spec [create_dft_specification -sri_sib_list {edt occ} ]
39
40 # Use report_config_syntax DftSpecification/edt|occ to see full syntax
41 report_config_data $spec
42 read_config_data -in $spec -from_string {
43     occ {
44         ijtag_host_interface : Sib(occ);
45     }
46 }
```

```
47 # The following is a generic way to populate the OCC
48 # The clock list is design specific and needs to be updated for the design
49 # To the OCC - scan_enable and shift_capture_clock gets connected
50 # automatically
51 # Modify the following to your design requirements
52 set id_clk_list [list \
53     dco_clk dco_clk\
54 ]
55
56 foreach {id clk} $id_clk_list {
57     set occ [add_config_element OCC/Controller($id) -in $spec]
58     set_config_value clock_intercept_node -in $occ $clk
59 }
60 report_config_data $spec
61
62 ## To EDT controller, the edt_clock and edt_update get auto connected
63 ## The EDT controller is built-in with bypass
64 ## Modify the following to your design requirements
65
66 1 read_config_data -in $spec -from_string {
67 2     EDT {
68 3         ijtag_host_interface : Sib(edt);
69 4         Controller (cl) {
70 5             longest_chain_range : 200, 300;
71 6             scan_chain_count : 60;
72 7             input_channel_count : 3;
73 8             output_channel_count : 2;
74 9         }
75 10    }
76 11 }
77 12 report_config_data $spec
78 13 //display_spec
79 14 # Generate the hardware
80 15 process_dft_specification
81 16
82 17 # Extract the IJAG network and create ICL file for core level
83 18 extract_icl
84 19
85 20 # Write updated RTL into this new file to elaborate and synthesize later
86 21 write_design_import_script for_dc_synthesis.tcl -replace
87 22
88 23 # Generate testbench
89 24 create_patterns_specification
90 25 process_patterns_specification
91 26
92 27 set_simulation_library_sources -v ../../../../library/verilog/adk.v
93 28
94 29 # Run Verilog simulation
95 30 run_testbench_simulations
96 31 # If simulation fails, use the command below to see which pattern failed
97 32 //check_testbench_simulations
98 33
99 34 exit
```

Specify and Verify the DFT Requirements: DFT Signals for Wrapped Cores

After loading the design data from the TSDB, define the DFT requirements for the EDT and OCC hardware. This includes adding DFT signals and performing DRC.

Figure 5-15. Flow for Specifying and Verifying the DFT Requirements



Procedure

1. Specify the following command to set DFT requirements:

```
set_dft_specification_requirements -logic_test on
```

Note

 The design level as specified by `set_design_level` remains “physical_block” from the first DFT insertion pass, so you do not need to specify this command again.

2. Use the following procedure to define the DFT signals for wrapped cores in the second DFT insertion pass:

- a. Specify a global DFT signal to enter logic test mode. For example:

```
add_dft_signals ltest_en -create_with_tdr
```

- b. Define the DFT signals. For example:

```
add_dft_signals scan_en edt_update test_clock -source_node \
{ scan_enable edt_update test_clock_u }
```

```
add_dft_signals edt_clock shift_capture_clock \
-create_from_other_signals
```

If these ports do not exist, the tool creates new ports.

- c. Specify the following command to test with multi-load ATPG patterns in MemoryBIST:

```
add_dft_signals memory_bypass_en -create_with_tdr
```

- d. Specify the following command to test an STI network during logic test.

```
add_dft_signals tck_occ_en -create_with_tdr
```

- e. Specify both the internal mode and the external mode for hierarchical DFT. This is required for scan insertion.

```
add_dft_signals int_ltest_en ext_ltest_en int_mode ext_mode
```

This command specifies that wrapped cores have both internal modes and external modes, and that you must specify both. Differentiating between internal mode and external mode allows Tessent to stitch the scan chains into internal chains and external chains as described in “[Perform Scan Chain Insertion](#).”

The internal and external modes are also required for proper ATPG pattern retargeting and graybox modeling later in the insertion flow. Refer to “[Hierarchical DFT Terminology](#)” for more information.

3. After defining the DFT signals, run DRC as in the flat design flow.

If the design includes clock gating that is implemented in RTL and not with an integrated clock gating cell, you must specify their func_en and test_en ports using the [add_dft_clock_enables](#) command. Tessent checks for proper clock and asynchronous set/reset control-ability.

Results

Tessent Shell generates DFT_C errors for DRCs that are run. For details, refer to “Pre-DFT Clock Rules (DFT_C Rules)” in the *Tessent Shell Reference Manual*.

Examples

To define the DFT signals, the following example shows the required DFT signals for wrapped cores in the second DFT insertion pass:

```
# Required global DFT signal to enter logic test mode
add_dft_signals ltest_en -create_with_tdr

# If these ports do not exist, new ports will be created
add_dft_signals scan_en edt_update test_clock -source_node \
{ scan_enable edt_update test_clock_u }
add_dft_signals edt_clock shift_capture_clock -create_from_other_signals

# Required for MemoryBIST to be tested with multi-load ATPG patterns
add_dft_signals memory_bypass_en -create_with_tdr

# Required for STI network to be tested during logic test
add_dft_signals tck_occ_en -create_with_tdr

# Required for hierarchical DFT and will be used during scan insertion
# Specifying both the internal mode and the external mode
add_dft_signals int_ltest_en ext_ltest_en int_mode ext_mode
```

Perform Scan Chain Insertion

Scan chain insertion for wrapped cores includes a task for performing wrapper analysis, which prepares the functional scannable sequential elements (or “flops”) for reuse as wrapper cells. Specifically, these cells are called shared wrapper cells.

Note



Prior to scan chain insertion, perform synthesis as described in section “[Perform Synthesis.](#)”

Using shared wrapper cells reduces the number of flops required for isolating the internal test modes from the primary input ports of the wrapper core. It also reduces the number of flops required for isolating the external test modes from the internal logic. Tessent automatically generates shared wrapper cells.

The [analyze_wrapper_cells](#) command performs wrapper analysis. In addition to preparing the shared wrapper cells, the command infers dedicated wrapper cells for ports having a large fanout or fanin. You can configure the size of the fanin and fanout using the `set_wrapper_analysis_options -input_fanout_flop_threshold` and `-output_fanin_flop_threshold` options. By default, Tessent infers dedicated wrapper cells on input ports fanning out to 32 or more scannable flip-flops, and on output ports in the fanout of 32 or more scannable flip-flops.

Both shared wrapper cells and dedicated wrapper cells can coexist in the same wrapper chains, which helps Tessent maintain wrapper chains of similar length. Scan insertion uses the DFT signals `int_ltest_en` and `ext_ltest_en` along with the scan enable signal to control the wrapper cell.

For information about scan chain insertion using Tessent Shell, refer to the “[Internal Scan and Test Circuitry Insertion](#)” in the *Tessent Scan and ATPG User’s Manual*.

Note



The line numbers used in this procedure refer to the command flow dofile in [Example 5-4](#) on page 142.

Procedure

1. Specify the following command to set the DFT context:

```
set_context dft -scan -design_id gate
```

2. Load the synthesized netlist. For example:

```
read_verilog ../3.synthesis/processor_core_synthesized.vg
```

3. Specify the same output directory you used in the first and second DFT passes:

```
set_tsdb_output_directory ../tsdb_core
```

4. Load the rtl2 design data for the DFT hardware you previously inserted. For example:

```
read_design processor_core -design_id rtl2 -no_hdl
```

5. Check the design rules:

```
check_design_rules
```

6. After running DRC, set up the wrapper cells as follows (see lines 25-35):

- a. Specify the options for analyzing the wrapper cells.
- b. Add dedicated wrappers, as necessary.
- c. Analyze the wrapper cells with the [analyze_wrapper_cells](#) command.
- d. Ensure that you exclude the EDT channel IO ports from wrapper analysis.

For details, refer to “[Scan Insertion for Wrapped Core](#)” in the *Tessent Scan and ATPG User’s Manual*

7. Specify the [add_scan_mode](#) command to connect the scan chains to the EDT signals and EDT hardware that you inserted during the second insertion pass. Scan insertion for wrapper cells requires using [multi-mode scan insertion](#) as described in the *Tessent Scan and ATPG User’s Manual*. Do the following (see lines 41-46):

- a. Create one scan mode for the entire population of scan cells:

The entire population of scan cells are stitched into the first scan mode using the [add_scan_mode](#) int_mode command to generate a scan mode consisting of all the scan cells stitched together.

- b. Create a second scan mode only for the shared and dedicated wrapper cells:

In the second DFT insertion pass, you had generated a DFT signal named “int_mode” with the `add_dft_signal` command. This signal enables this scan mode. You do not need to specify the `add_scan_mode -enable_dft_signal` switch when the mode name matches the name of a DFT signal of type scan mode.

The `add_scan_mode ext_mode` command stitches the shared and dedicated wrapper cells together. Likewise, you had previously generated a DFT signal named “ext_mode” that enables this scan mode.

Examples

The following dofile shows a command flow for scan insertion. The highlighted statements illustrate additional considerations for performing scan insertion for wrapper cores. For a general overview, refer to “[Perform Scan Chain Insertion](#)” for a flat design.

Example 5-4. Scan Chain Insertion in Hierarchical Flow

```
1 set_context dft -scan -design_id gate
2
```

```

3 # You must respecify the TSDB
4 set_tsdb_output_directory ../tsdb_core
5
6 # Read Tessent library
7 read_cell_library ../../../../library/tessent/adk.tcelllib
8
9 # Read synthesized netlist
10 read_verilog ../3.synthesis/processor_core_synthesized.vg
11
12 # Use read_design to read in the DFT signals and other data from
13 # the second DFT insertion pass
14 read_design processor_core -design_id rtl2 -no_hdl
15 set_current_design processor_core
16
17 # If there are no ATPG models available for memories, use blackboxes
18 add_black_boxes -modules { \
19     SYNC_1RW_8Kx16 \
20 }
21
22 check_design_rules
23
24 report_clocks
25 # Exclude the EDT channel in and out ports from wrapper chain analysis
26 # The ijtag_* edt_update ports are automatically excluded
27 set_wrapper_analysis_options \
28     -exclude_ports [ get_ports {*_edt_channels_*} ]
29
30 # To force insertion of dedicated wrapper cell use the following command
31 # set_dedicated_wrapper_cell_options on -ports {.... }
32
33 # Perform wrapper cell analysis
34 analyze_wrapper_cells
35 report_wrapper_cells -Verbose
36
37 # Find edt_instance
38 set edt_instance [get_instances -of icl_instances \
39 [get_icl_instances -filter tessent_instrument_type==mentor::edt]]
40
41 # Specify different modes (internal and external) of the chains that need
42 # to be stitched
43 # The type internal/external and enable_dft_signal are inferred from the
44 # registered DFT signals(int_mode and ext_mode)
45 add_scan_mode int_mode -edt_instances $edt_instance
46 add_scan_mode ext_mode -chain_count 2
47
48 # Before scan insertion you can analyze the different scan modes and scan
49 # chains
50 analyze_scan_chains
51 report_scan_chains
52 //delete_scan_modes -all
53 # Insert scan chains and write the scan inserted design into the TSDB
54 insert_test_logic
55 report_scan_chains
56 report_scan_cells > scan_cells.list
57 exit

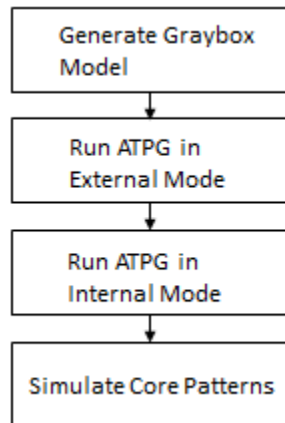
```

Perform ATPG Pattern Generation: Wrapped Core

Generating ATPG patterns for a wrapped core includes a step for generating a graybox model. In addition, you must perform ATPG twice, once for the core's external mode and once for the core's internal mode.

ATPG pattern generation at the core level involves the following four steps. Refer to “[Perform ATPG Pattern Generation](#)” for flat designs for a general description.

Figure 5-16. ATPG Pattern Generation Flow for Wrapped Cores



As described in “[Hierarchical DFT Terminology](#),” the graybox model excludes the internal mode logic of the wrapped core, preserving only the external mode logic that needs to be tested at the parent level. The IJTAG infrastructure is preserved in the graybox model also. Specifically, Tessent preserves the external logic that is present between the primary input and the input wrapper cells, plus any logic between the output wrapper cells and primary output. The parent could be another wrapper core or the top level of the chip.

You can use external mode patterns, if generated, for calculating fault coverage for the entire core (both internal and external mode). Use the internal mode ATPG patterns for ATPG pattern retargeting when performing the RTL and scan DFT insertion process on the top level of the chip.

Procedure

1. Generate graybox models (see [Example 5-5](#) on page 146 for a command flow example):
 - a. Load in the design using the same design ID as you used for scan insertion to write the graybox to the TSDB.

(Recommended) Use “gate” as the design ID if you inserted the MemoryBIST and EDT logic at the RTL level and “gate3” if the logic was also inserted into the gate level.
 - b. Specify the [analyze_graybox](#) command to generate graybox models.

When working at the parent level, using the same design ID for the graybox model as you used for scan insertion allows Tessent to access the full design view or the graybox model with the same design ID.

2. Run ATPG on the *external* mode of the wrapped core. This ATPG pattern is only used to calculate the entire core's fault coverage and cannot be reused from the chip-level. To generate ATPG patterns for external mode, do the following:
 - a. Read in the graybox model of the design with the [read_design](#) command.
 - b. Use the [set_current_mode](#) command to specify a unique ATPG mode name that represents the purpose of the pattern. The mode type is external.
 - c. Use the [import_scan_mode](#) command to retrieve the core's external mode data. Tessent uses the graybox model of the core. Using the [import_scan_mode](#) command assumes that you used Tessent Scan to perform scan chain stitching.
 - d. After running DRC and generating the ATPG patterns, use the [write_tsdb_data](#) command to store the TCD, flat model, fault list and PatDB files in the TSDB.
 - e. Use the [write_patterns](#) command to write out the testbench required to simulate the generated ATPG patterns.

See [Example 5-6](#) on page 146.

3. Run ATPG on the *internal* mode of the wrapped core. This results in the ATPG pattern that you will retarget at the top level of the chip. To generate ATPG patterns for internal mode, do the following:
 - a. Load in the graybox views for the wrapper cores that contain child wrapper cores.
 - b. If you used Tessent Scan for scan insertion, specify [import_scan_mode](#) to import the internal mode.
 - c. Specify a unique ATPG mode name with the [set_current_mode](#) command. The current mode type is internal. Typical mode names are *scan_mode_name_sa* or *scan_mode_name_tdf*.
 - d. After running DRC and generating the ATPG patterns, store the TCD, flat model, fault list and PatDB files in the TSDB using the [write_tsdb_data](#) command.
 - e. Use the [read_faults](#) command to merge the fault list from running external mode to find the total overall fault coverage of the wrapped core.

See [Example 5-7](#) on page 147.

4. Run Verilog simulation of the core-level ATPG patterns.

Performing this task ensures that the patterns will function as desired when they are retargeted at the parent level. For parallel load patterns as specified by the [write_patterns](#) -parallel command, simulate all the patterns. For serial load patterns, a handful of

patterns are sufficient; the runtime for simulating gate-level serial load patterns is significant.

Examples

Generate the Graybox Model

The following example creates a graybox model for a scan-inserted processor_core design and saves the data under the “gate” design ID.

Example 5-5. Graybox Example

```
set_context patterns -scan -design_id gate
set_tsdb_output_directory ../tsdb_core
read_cell_library ../../../../library/tessent/adk.tcelllib

## Reads in the scan inserted netlist/design
read_design processor_core -design_id gate -verbose
set_current_design processor_core
add_black_boxes -modules { \
                    SYNC_1RW_8Kx16 \
                }

report_dft_signals

import_scan_mode ext_mode
check_design_rules
analyze_graybox
write_design -tsdb -graybox -verbose

exit
```

Run ATPG on the Core's External Mode

The following example shows the command flow for running ATPG on the external mode of a core.

Example 5-6. ATPG External Mode

```
set_context patterns -scan
set_tsdb_output_directory ../tsdb_core
read_cell_library ../../../../library/tessent/adk.tcelllib

# Read in the graybox model using the -view switch
read_design processor_core -design_id gate -view graybox -verbose
set_current_design processor_core

# Specify a different name that what was used during scan insertion with
# the add_scan_mode command
set_current_mode edt_multi_SAF -type external
report_dft_signals
```

```
# Extract the external mode specified during scan insertion
import_scan_mode ext_mode
report_core_instances
report_static_dft_signal_settings

# Run DRC, Tessent Shell prompts changes to Analysis
check_design_rules
report_clocks
set_xclock_handling x

# Generate ATPG patterns
create_patterns

# Store TCD, flat_model, fault list and PatDB files in the TSDB
write_tsdb_data -replace
write_patterns patterns/processor_core_ext_stuck_parallel.v -verilog \
    -parallel -replace -parameter_list {SIM_KEEP_PATH 1}
set_pattern_filtering -sample_per_type 2
write_patterns patterns/processor_core_ext_stuck_serial.v \
    -verilog -serial -replace -parameter_list {SIM_KEEP_PATH 1}
exit
```

Run ATPG on the Core's Internal Mode

The following example shows the command flow for running ATPG on the internal mode of a core.

Example 5-7. ATPG Internal Mode

```
set_context patterns -scan
set_tsdb_output_directory ../tsdb_core
read_cell_library ../../../../library/tessent/adk.tcelllib
# Read in the full scan-inserted netlist
read_design_processor_core -design_id gate
set_current_design processor_core

# Extract the internal mode specified during scan insertion
import_scan_mode int_mode

# Use add_scan_mode to specify a different name than what was used during
# scan insertion
# Specify import_scan_mode before set_current_mode because
# import_scan_mode overrides the test mode type specified by
# set_current_mode
set_current_mode int_mode_sa -type internal

report_dft_signals
report_core_instances
report_static_dft_signal_settings

# Run DRC
check_design_rules
report_clocks
report_core_instances
add_fault -all
report_statistics -detail
```

```
# Generate ATPG patterns
create_patterns
report_statistics -detail

# Store TCD, flat_model, fault list and PatDB files in the TSDB
write_tsdb_data -replace
write_patterns patterns/processor_core_stuck_parallel.v -verilog \
  -parallel -replace -parameter_list {SIM_KEEP_PATH 1}
set_pattern_filtering -sample_per_type 2
write_patterns patterns/processor_core_stuck_serial.v \
  -verilog -serial -replace -parameter_list {SIM_KEEP_PATH 1}
exit

# To view the coverage of the faults testable by internal mode, you must
# eliminate the undetected faults, which would be detected in
# external mode. Do this by merging in the fault list generated from
# performing ATPG on the graybox in external mode
read_faults -mode ext_multi_SAF -fault_type stuck -merge

# Final coverage of the core that includes internal and external modes
report_statistics -detail
exit
```

Run Recommended Validation Step for Pre-Layout Design Sign Off

During the first pass of the hierarchical DFT insertion flow for a wrapped core, you insert and simulate the MemoryBIST hardware, mostly at RTL level. To ensure that the MemoryBIST simulation passes before the core is signed off as the pre-layout netlist, re-run MemoryBIST simulation at gate level on the core's scan-inserted netlist.

Procedure

1. Read in the scan-inserted netlist from the TSDB

Using the recommended naming conventions for the RTL and scan DFT insertion flow, the design ID would be “gate” if you inserted the MemoryBIST and EDT logic at the RTL level and “gate3” if the logic was also inserted into the gate level.

2. Elaborate the design and run DRC.
3. Create the input test patterns and simulation files.
4. Simulate the testbench.

Tessent Shell stores the generated testbench in the TSDB under the Patterns directory using the design ID “gate”.

Examples

The following example shows how to validate the MemoryBIST patterns for a scan-inserted netlist.

```
set_context_patterns -ijtag -design_id gate
set_tsdb_output_directory ../tsdb_core

##Reading the tessent cell library
read_cell_library ../../../../library/tessent/adk.tcelllib

## Reading the scan inserted netlist
read_design_processor_core -design_id gate -verbose -view interface
set_current_design processor_core
add_black_boxes -modules { \
                    SYNC_1RW_8Kx16 \
                    }
check_design_rules

create_patterns_specification
process_patterns_specification
set_simulation_library_sources \
    -v ../../../../library/memory/SYNC_1RW_8Kx16.v \
    -v ../../../../library/verilog/adk.v
run_testbench_simulation
exit
```

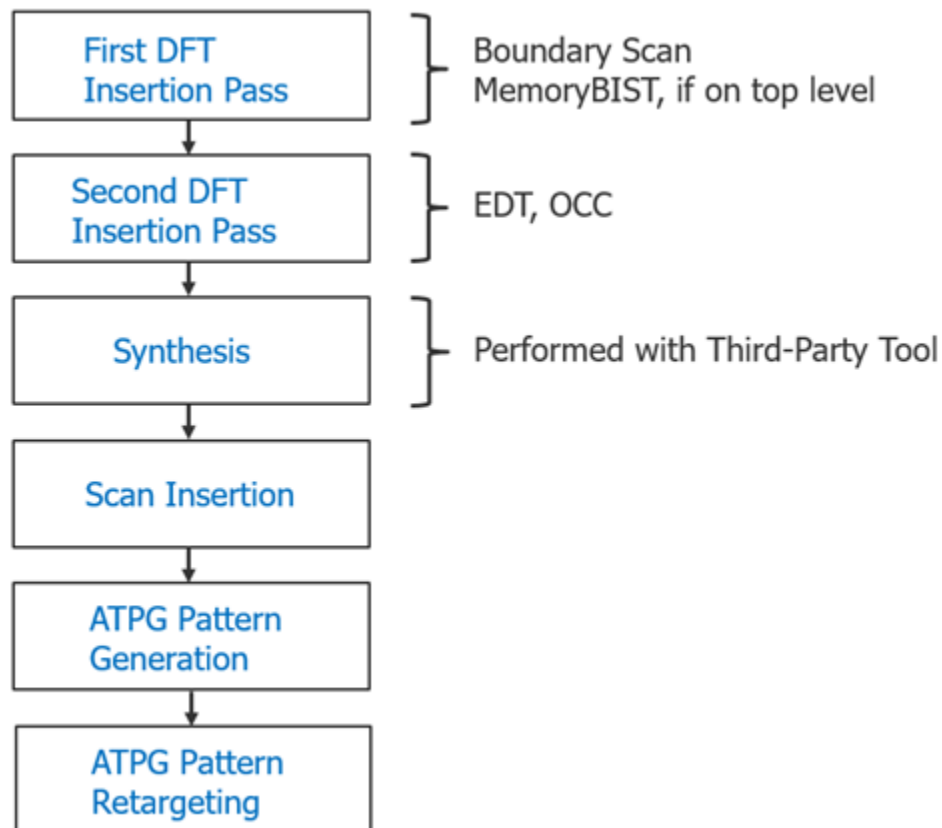
RTL and Scan DFT Insertion Flow for the Top Chip

After performing the RTL and scan DFT insertion flow for each wrapped core in your design, you can perform the DFT insertion process for the top-level chip design.

Before implementing DFT at the top level of the chip, plan how you will test the wrapped cores at the chip level. The planning for logic testing the wrapped cores is not automated, so you must decide how you will allocate the resources and organize the test schedule (especially for ATPG pattern retargeting) and specify your intent by using the [add_dft_modal_connections](#) command.

The RTL and scan DFT insertion flow for the top level of a chip follows the same basic process you used for the cores, with the addition of a step for retargeting the ATPG patterns you generated for the wrapped cores.

Figure 5-17. Two-Pass Insertion Flow for RTL, Top Level



In addition, processing at the chip level differs from wrapped cores in that you must insert a Test Access Mechanism (TAM). A TAM is a mechanism that you use to carry the scan data in and out of the chip for each group of wrapped cores you intend to run in parallel. It schedules the tests for the wrapped cores at the top level, and it allows access to the chip level so that you can run these tests.

During insertion and pattern generation, you will be opening the TSDBs that store the wrapped core design data and reading in the designs for their graybox models. The DFT insertion flow for the top level of the chip requires differentiating between three design views of the wrapped cores.

- Full netlist view. All the logic for the core. This is the default view when you do not explicitly specify a design view with the [read_design](#) command.
- Graybox model. External mode logic for the core as described in “[Hierarchical DFT Terminology](#),” including its JTAG interface. You use this view so that Tessent Shell can connect the wrapped cores at the top level for logic testing of the chip.
- Interface view. The core’s ports only. Tessent auto-loads the interface view of any sub-physical block for which you have not use [read_design](#) to load its view.

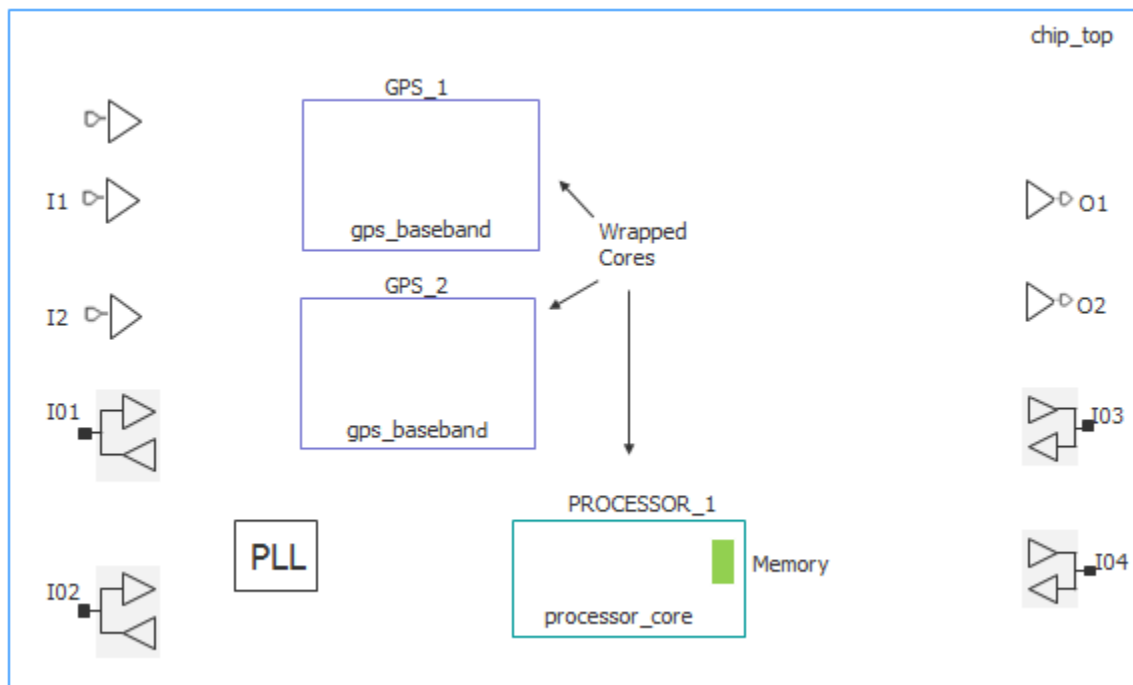
Top-Level DFT Insertion Example	151
First DFT Insertion Pass: Performing Top-Level MemoryBIST and Boundary Scan.	153
Second DFT Insertion Pass: Top-Level EDT and OCC.....	156
Top-Level Scan Chain Insertion Example.....	160
Top-Level ATPG Pattern Generation Example	160
Performing Top-Level ATPG Pattern Retargeting	162

Top-Level DFT Insertion Example

To illustrate the RTL and scan DFT insertion flow for the top level of a chip, this section uses two wrapped cores, `processor_core` and `gps_baseband`. The processor core has memory instantiated within it, and the GPS baseband is a logic-only core that is instantiated twice.

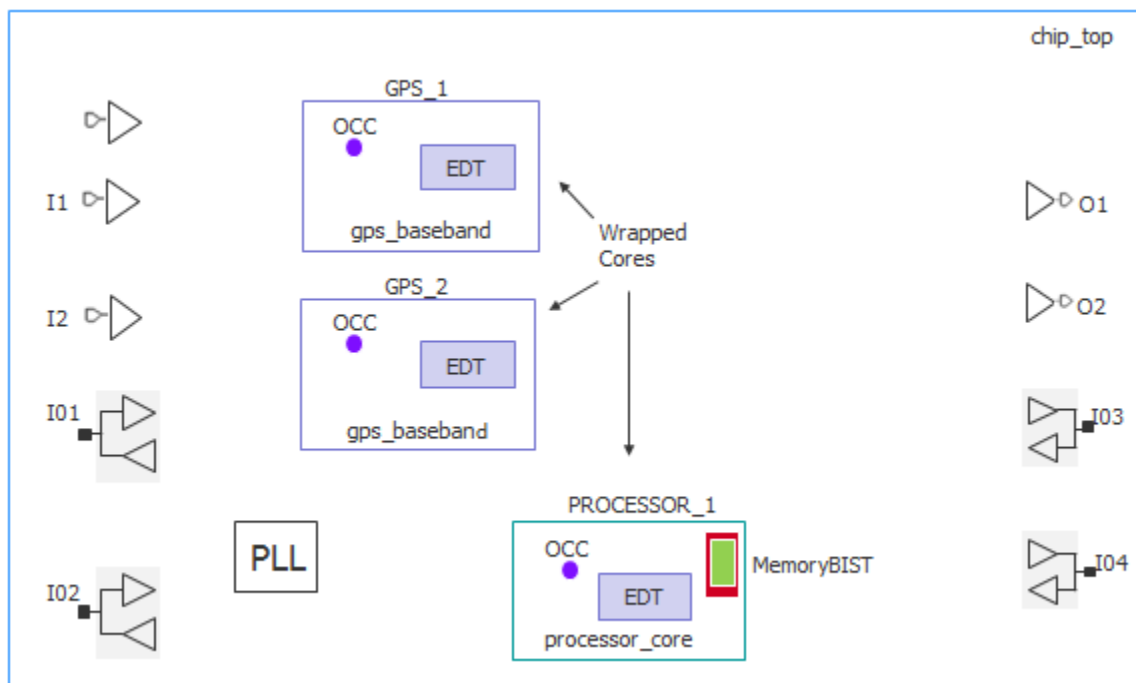
The following figure shows the view at the top level before DFT insertion.

Figure 5-18. Top-Level Example, Before DFT Insertion



After performing the RTL and scan DFT insertion process for the wrapped cores, the architecture looks as follows. Each of the cores has an EDT controller and a child OCC. The processor core has the memories with MemoryBIST already inserted.

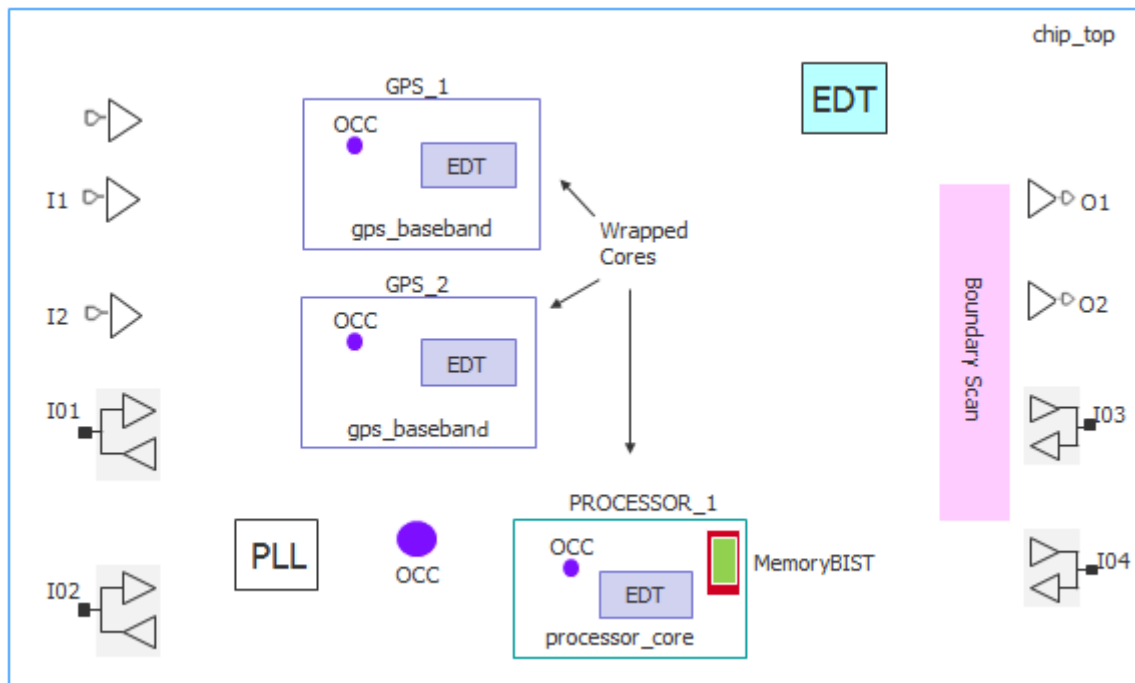
Figure 5-19. Top-Level Example, After DFT Insertion for Wrapped Cores



After inserting DFT at the top level, the design includes a TAP controller along with boundary scan, a top-level EDT controller, a parent OCC, and a simple TAM for purposes of retargeting the wrapped cores (not shown in the picture below).

In this top-level example test case, the hierarchical core starts with RTL insertion, and in addition, at the top level we would like to perform DFT at RTL as well. However, there is no RTL logic at the top level that needs to be synthesized. The PLL and pad IOs are Verilog macros. The only RTL that needs to be synthesized is the Tessent-generated RTL. Hence, this test case uses design IDs “gate1” and “gate2” for the first two DFT insertion passes, respectively.

Figure 5-20. Top-Level Example, After DFT Insertion at the Top Level



First DFT Insertion Pass: Performing Top-Level MemoryBIST and Boundary Scan

For the top level of a chip, insert boundary scan plus MemoryBIST for any memories that are present at the top level. In addition, build the JTAG network for the wrapped cores at the top level and insert a TAP controller or reuse an existing TAP controller.

As with flat designs, the flow for inserting MemoryBIST and boundary scan together is the same as inserting them separately. For details about this insertion pass flow, refer to:

- “[Getting Started](#)” in the *Tessent MemoryBIST User’s Manual*, or
- “[Getting Started With Tessent BoundaryScan](#)” in the *Tessent BoundaryScan User’s Manual*

For information about TAP controller reuse, refer to “[create_dft_specification](#)” in the *Tessent Shell Reference Manual*.

Refer to “[First DFT Insertion Pass: MemoryBIST and Boundary Scan](#)” (for flat designs) for general information and prerequisites. For example, as with flat designs, you can segment the boundary scan chain into smaller chains that are used during logic testing with Tessent TestKompress by using the [max_segment_length_for_logictest](#) command.

Note



This procedure is similar for flat designs as documented in “[First DFT Insertion Pass: MemoryBIST and Boundary Scan](#)” on page 107.

Procedure

1. Load the design.

Note



Use the [open_tsdb](#) command to open the TSDBs for all the lower-level cores. Opening their TSDBs makes their design data available.

There is no need to specify the `read_design` command because, by default, Tessent will read in the interface views of the wrapped cores, which is all that is required for DRC for the first DFT insertion pass.

2. Set the design level to “chip” for the top level of the chip. When working with the wrapped cores, you had set the design level to “physical_block.”
3. Create the DFT specification.
4. Specify enough auxiliary input and output ports for the largest retargeting wrapper core group.
5. Generate the MemoryBIST hardware and extract the ICL.
6. Create the input test patterns and simulation test benches.
7. Run simulations to verify the design.

Examples

The following dofile example shows a typical command flow for inserting boundary scan. The flow is the same as you would use for a flat design with the exception of the commands highlighted in bold. Notice that the chip-level design data exists in its own TSDB. This is recommended for the data flow as described in “[TSDB Data Flow for the Tessent Shell Flow](#).”

```

set_context dft -no_rtl -design_id gate1
set_tsdb_output_directory ../tsdb_outdir

# Open the TSDB of all the wrapped cores
open_tsdb ../../wrapped_cores/processor_core/tsdb_outdir
open_tsdb ../../wrapped_cores/gps_baseband/tsdb_outdir

# Read the Tessent cell library
read_cell_library ../../library/tessent/adk.tcelllib

# Read in PLL as a blackbox so that excluded in design list that you write
# out for synthesis
read_verilog ../rtl/noncore_blocks/pll.v -blackbox

# Point to Verilog libraries for the pad IO macros
set_design_sources -format verilog \
    -v ../rtl/noncore_blocks/pad8_io_macro.v \
    -v ../rtl/noncore_blocks/iopad_sel.v \
    ../rtl/noncore_blocks/iopad.v
read_verilog ../rtl/chip_top.v
set_current_design chip_top
set_design_level chip

# Specify and verify the DFT requirements
set_dft_specification_requirements -boundary_scan on -memory_bist on

# Identify the TAP pins
set_attribute_value TCK -name function -value tck
set_attribute_value TDI -name function -value tdi
set_attribute_value TMS -name function -value tms
set_attribute_value TRST -name function -value trst
set_attribute_value TDO -name function -value tdo

# Some pins cannot add any boundary scan cells
set_boundary_scan_port_options TEST_CLOCK_top -cell_options dont_touch
set_boundary_scan_port_options EDT_UPDATE_top -cell_options dont_touch
set_boundary_scan_port_options SCAN_ENABLE -cell_options dont_touch
set_boundary_scan_port_options RESET_N -cell_options dont_touch
set_boundary_scan_port_options INCLK -cell_options dont_touch

# Add DFT signals
add_dft_signals scan_en -source_nodes SCAN_ENABLE
report_dft_signals
add_clocks PLL_1/pll_clock_0 -reference REF_CLK -FREQ_Multiplier 16
add_clock REF_CLK -period 48
check_design_rules

# Create a dft specification
set_spec [create_dft_specification]
report_config_data $spec
# Segment the boundary scan to be used during logic test
set_config_value $spec/BoundaryScan/max_segment_length_for_logictest 80

# Equip these ports with auxiliary input/output muxes to be used by EDT
# channel pins
read_config_data -in ${spec}/BoundaryScan -from_string {
    AuxiliaryInputOutputPorts {
        auxiliary_input_ports : GPIO1_0, GPIO1_1, GPIO1_2, GPIO1_3;
    }
}

```

```

        auxiliary_output_ports : GPIO2_0, GPIO2_1, GPIO2_2, GPIO2_3, GPIO1_0,
        GPIO1_1 ;
    }
}

report_config_data $spec
process_dft_specification
extract_icl
run_synthesis
create_patterns_specification
process_patterns_specification
set_simulation_library_sources \
    -v ../../library/verilog/*.v \
    -v ../rtl/noncore_blocks/p11.v \
    -v ../rtl/noncore_blocks/iopad.v \
    -v ../../library/memory/SYNC_1RW_8Kx16.v

run_testbench_simulations
exit

```

Second DFT Insertion Pass: Top-Level EDT and OCC

For the top level of a chip, additional considerations for the second DFT insertion pass for EDT and OCC include grayboxes, DFT signals for purposes of ATPG retargeting, and TAMs.

Note



This procedure follows a similar flow as flat designs as documented in “[First DFT Insertion Pass: MemoryBIST and Boundary Scan](#)” on page 107. The unique operations are documented in the steps below.

Procedure

1. Specify the [open_tsdb](#) command to open the TSDBs for the wrapped cores, as in the first DFT insertion pass for the chip,
2. Specify the [read_design](#) command to read in the graybox models of the wrapped cores.

This allows Tessent to perform DRC on the wrapper chains in addition to the rest of the top-level logic that will be logic tested.

3. Define a retargeting mode for each group of wrapped cores whose ATPG patterns you will be retargeting to run in parallel.

For ATPG pattern retargeting purposes, Tessent requires you to include retargeting mode DFT signals in addition to the DFT signals defined in section “[DFT Signals](#).” The retargeting *X_mode* signals along with the TAM specified with the `add_dft_modal_connections` command ensure that ATPG pattern retargeting occurs correctly. Optionally, you can register your own DFT signals to be used for retargeting purposes.

[Example 5-8](#) on page 157 shows demonstrates retargeting of two wrapped cores, processor_core and gps_baseband.

4. Apply the `add_dft_modal_connections` command to specify the TAM.

During the first insertion pass, you identified the functional pins to be shared with EDT channel pins and added the required auxiliary input and auxiliary output logic. Now you use the TAM to sensitize paths to and from the top-level EDT using the `set_config_value` command.

In [Example 5-8](#) on page 157, the functional input pin GPI01_0 is shared as an EDT channel input pin. Similarly, the functional output pin GPI02_0 is shared as an EDT channel output pin.

5. Combine with top-level EDT mode signal you defined by connecting the EDT channel IOs of the wrapped cores to top-level pins via the TAM. Use the `add_dft_modal_connections` command.
6. Apply the `set_defaults_value` command to specify that Tessent Shell simulate the instruments within the wrapped cores in addition to the top-level instruments.

Examples

The following dofile example shows that the DFT insertion flow for inserting EDT and OCC into the top level of a chip follows the same basic process as for flat designs as described in “[Second DFT Insertion Pass: EDT and OCC](#),” with the exception of the highlighted commands.

Example 5-8. Top-Level Second DFT Pass

```
set_context dft -no_rtl -design_id gate2
set_tsdb_output_directory ../tsdb_outdir
open_tsdb ../../wrapped_cores/processor_core/tsdb_outdir
open_tsdb ../../wrapped_cores/gps_baseband/tsdb_outdir

read_cell_library ../../library/tessent/adk.tcelllib
read_verilog ../rtl/noncore_blocks/p11.v -blackbox
set_design_sources -format verilog -v ../rtl/noncore_blocks/iopad_sel.v \
-v ../rtl/noncore_blocks/iopad.v
read_design chip_top -design_id rtl1 -verbose
read_design processor_core -design_id gate -view graybox -verbose
read_design gps_baseband -design_id gate -view graybox -verbose
set_current_design chip_top
```

```
# Add DFT Signals
# When using boundary scan in logic test without contacting inputs
add_dft_signals int_ltest_en output_pad_disable -create_with_tdr

# Needed for Scan Tested Instruments such as MemoryBIST and boundary scan
add_dft_signals tck_occ_en -create_with_tdr

# When you are using logic test
add_dft_signals ltest_en -create_with_tdr

# Used by top-level EDT
add_dft_signals edt_mode -create_with_tdr

# These are used for the top-level EDT
add_dft_signals test_clock edt_update \
    -source_nodes {TEST_CLOCK_top EDT_UPDATE_top}
add_dft_signals shift_capture_clock edt_clock -create_from_other_signals

# Retargeting mode signals used for ATPG pattern retargeting
add_dft_signals retargeting1_mode retargeting2_mode retargeting3_mode \
retargeting4_mode

# Add simple TAM connections
# For chip-level EDT mode, the asterisk(*) means do not make connection to
the data inputs
add_dft_modal_connections -ports GPIO1_0 -input_data_destination_nodes * \
    -enable_dft_signal edt_mode
add_dft_modal_connections -ports GPIO2_0 -output_data_source_nodes * \
    -enable_dft_signal edt_mode

# Connect wrapped core EDT channel I/Os to top level for retargeting1_mode
signal
add_dft_modal_connections -ports GPIO1_2 -input_data_destination_nodes
PROCESSOR_1/processor_core_rtl2_controller_c1_edt_channels_in[0] \
    -enable_dft_signal retargeting1_mode
add_dft_modal_connections -ports GPIO2_2 -output_data_source_nodes
PROCESSOR_1/processor_core_rtl2_controller_c1_edt_channels_out[0] \
    -enable_dft_signal retargeting1_mode

# Connect wrapped core EDT channel I/Os to top level for retargeting2_mode
signal
add_dft_modal_connections -ports GPIO1_2 -input_data_destination_nodes
GPS_1/gps_baseband_rtl1_controller_c1_edt_channels_in[0] \
    -enable_dft_signal retargeting2_mode
add_dft_modal_connections -ports GPIO1_2 -input_data_destination_nodes
GPS_2/gps_baseband_rtl1_controller_c1_edt_channels_in[0] \
    -enable_dft_signal retargeting2_mode
...
add_dft_modal_connections -ports GPIO1_0 -output_data_source_nodes GPS_1/
gps_baseband_rtl1_controller_c1_edt_channels_out[0] \
    -enable_dft_signal retargeting2_mode -pipeline_stages 1
add_dft_modal_connections -ports GPIO1_1 -output_data_source_nodes GPS_1/
gps_baseband_rtl1_controller_c1_edt_channels_out[1] \
    -enable_dft_signal retargeting2_mode -pipeline_stages 1
...
```

```

report_dft_modal_connections
set_dft_specification_requirements -logic_test On
add_clocks INCLK -period 10ns
check_design_rules
report_dft_control_points
# Create DFT specification
set_spec [create_dft_specification -sri_sib_list {occ edt} ]
report_config_data $spec
read_config_data -in $spec -from_string {
    OCC {
        ijtag_host_interface : Sib(occ);
    }
}
set_id_clk_list [list \
    pll_clock_0 PLL_1/pll_clock_0 \
    INCLK INCLK \
]
foreach {id clk} $id_clk_list {
    set occ [add_config_element OCC/Controller($id) -in $spec]
    set_config_value clock_intercept_node -in $occ $clk
}
report_config_data $spec
read_config_data -in $spec -from_string {
    EDT {
        ijtag_host_interface : Sib(edt);
        ...
    }
}

set_config_value port_pin_name \
    -in $spec/EDT/Controller(c1)/Connections/EdtChannelsIn(1) \
    [get_single_name [get_auxiliary_pins GPIO1_0 -direction input]]
set_config_value port_pin_name \
    -in $spec/EDT/Controller(c1)/Connections/EdtChannelsOut(1)
    [get_single_name [get_auxiliary_pins GPIO2_0 -direction output] ]
report_config_data $spec
process_dft_specification
extract_icl

# By setting this value, all the lower level instruments in the wrapped
# cores are simulated
set_defaults_value /PatternsSpecification/SignoffOptions/
simulate_instruments_in_lower_physical_instances on
create_patterns_specification
process_patterns_specification
set_simulation_library_sources -v ../../library/verilog/*.v \
    ...

run_testbench_simulations
exit

```

Top-Level Scan Chain Insertion Example

For the top level of the chip, additional considerations for scan chain insertion include opening the TSDBs for the wrapped cores and reading in the wrapped core graybox models as you did for the second DFT insertion pass.

Note



Prior to scan chain insertion, perform synthesis as described in section “[Perform Synthesis.](#)”

Logic that is present at the top-level needs to be scan stitched along with the wrapper chains (external mode) of the cores.

Refer to “[Perform Scan Chain Insertion](#)” (for flat designs) for more information about scan chain insertion. The following dofile example shows that Tessent Shell needs to access the graybox models for each wrapped core.

Example 5-9. Top-Level Scan Chain Insertion

```
set_context dft -scan -design_id gate
set_tsdb_output_directory ../tsdb_outdir
open_tsdb ../../wrapped_cores/processor_core/tsdb_core
open_tsdb ../../wrapped_cores/gps_baseband/tsdb_core

read_cell_library ../../library/tessent/adk.tcelllib
read_verilog ../rtl/noncore_blocks/pll.v -blackbox
set_design_sources -format verilog -v ../rtl/noncore_blocks/iopad_sel.v \
    -v ../rtl/noncore_blocks/iopad.v

read_verilog ../Synthesis/chip_top_synthesized.vg
read_design chip_top -design_id rtl2 -no_hdl -verbose
read_design processor_core -design_id gate -view graybox -verbose
read_design gps_baseband -design_id gate -view graybox -verbose
set_current_design chip_top

check_design_rules
report_clocks

set edt_instance [get_name_list [get_instance -of_module \
    [get_name [get_icl_module -of_instances chip_top* \
    -filter tessent_instrument_type==mentor::edt]]] ]
add_scan_mode edt_mode -edt_instance $edt_instance
analyze_scan_chains
report_scan_chains
insert_test_logic
exit
```

Top-Level ATPG Pattern Generation Example

At the top level, Tessent Shell uses the scan-inserted design data for the chip. In the recommended flow this equates to design ID “gate.” In addition, Tessent uses the graybox

models for the wrapped cores so that you can use the external mode of the wrapper chains to run ATPG.

Refer to “[Perform ATPG Pattern Generation](#)” (for flat designs) for information about generating the ATPG patterns. The flow for the top level of a chip builds onto the process by adding in the grayboxes and blackboxes. As with flat designs, you have a choice between using the boundary scan chains for capture or using the primary pins of the chips (using the pads).

The following dofile example generates ATPG patterns at the top level using the stuck-at fault model. The dofile shows that you can read in the stuck-at fault models for the wrapped cores to calculate the total fault coverage for the chip.

Example 5-10. Top-Level ATPG Pattern Generation

```
set_context pattern -scan
set_tsdb_output_directory ../tsdb_top
open_tsdb ../../wrapped_cores/processor_core/tsdb_core
open_tsdb ../../wrapped_cores/gps_baseband/tsdb_core
read_cell_library ../../library/tessent/adk.tcelllib

# Read the Verilog
read_verilog ../rtl/noncore_blocks/pll.v -blackbox
set_design_sources -format verilog -v ../rtl/noncore_blocks/iopad.v \
    -v ../rtl/noncore_blocks/iopad_sel.v

read_design chip_top -design_id gate
read_design processor_core -design_id gate -view graybox -verbose
read_design gps_baseband -design_id gate -view graybox -verbose
set_current_design chip_top
set_current_mode edt_top_stuck

import_scan_mode edt_mode
report_core_instances

set_static_dft_signal_values tck_occ_en 1
report_static_dft_signal_settings

set_system_mode analysis
add_fault -all
report_statistics -detail
create_patterns
report_statistics -detail
write_tsdb_data -replace

write_patterns patterns/chip_top_edt_parallel.v -verilog -parallel \
    -replace -scan -parameter_list {SIM_KEEP_PATH 1}
set_pattern_filtering -sample_per_type 2
write_patterns patterns/chip_top_edt_serial.v -verilog -serial -replace \
    -parameter_list {SIM_KEEP_PATH 1}
write_patterns patterns/chip_top_edt_stuck.stil -stil -replace
```

```
# Total fault coverage for stuck-at patterns at the chip-level including
# the cores
read_faults -module processor_core -mode edt_int_stuck \
  -fault_type stuck -merge -graybox
read_faults -module gps_baseband -module edt_int_stuck \
  -fault_type stuck -merge -graybox
report_statistics -detail
exit
```

Performing Top-Level ATPG Pattern Retargeting

For each wrapped core, retarget the internal ATPG patterns for all the modes you have specified and all the fault types.

Note



This procedure is similar to “[Perform ATPG Pattern Generation: Wrapped Core](#)” on page 144.

Procedure

1. Specify the [set_context](#) patterns -retargeting command to retarget the ATPG patterns generated for the wrapped cores.
2. Use the same TSDB for ATPG retargeting as you used for ATPG pattern generation.
3. Set the current mode to a unique mode name that, ideally, indicates the core name, the fault type, and the retargeting mode DFT signal you had previously defined.
4. Specify which wrapped core ATPG patterns you are retargeting by enabling the correct retargeting mode DFT signal. Set the [set_static_dft_signal_values](#) command to the retargeting mode for this wrapped core.
5. Apply the [add_core_instances](#) command to specify the wrapped core whose internal ATPG patterns you will be retargeting.
6. Apply the [read_patterns](#) command to read in the stuck-at ATPG patterns that you will be retargeting.

Examples

The following dofile example shows how you would retarget the stuck-at ATPG patterns for the wrapped core, processor_core.

Example 5-11. Retarget Stuck-At ATPG Patterns for the Top-Level

```
set_context pattern -scan_retargeting
set_tsdb_output_directory ../tsdb_top
open_tsdb ../../wrapped_cores/processor_core/tsdb_core
open_tsdb ../../wrapped_cores/gps_baseband/tsdb_core
```

```

read_cell_library ../../library/tessent/adk.tcelllib
read_verilog ../rtl/noncore_blocks/pll.v -blackbox
set_design_sources -format verilog -v ../rtl/noncore_blocks/iopad.v \
-v ../rtl/noncore_blocks/iopad_sel.v
read_design chip_top -design_id gate
read_design processor_core -design_id gate -view graybox -verbose
read_design gps_baseband -design_id gate -view graybox -verbose

set_current_design chip_top

# Retarget processor_core stuck-at patterns
set_current_mode retarget1_processor_stuck
set_static_dft_signal_values retargeting1_mode 1
add_core_instances -instances {PROCESSOR_1} -core processor_core \
-mode edt_int_stuck

report_core_descriptions
report_clocks

set_system_mode analysis
write_tsdb_data -replace

# Read the patterns to be retargeted
read_patterns -module processor_core -fault_type stuck -mode edt_int_stuck
set_external_capture_options -pll_cycles 5 [lindex [get_timeplate_list] 0]

write_patterns patterns/processor_core_edt_stuck_retargted.v -verilog \
-parallel -replace -begin 0 -end 7 -scan \
-parameter_list {SIM_KEEP_PATH 1}
write_patterns patterns/processor_core_edt_stuck_retargted_serial.v \
-verilog -serial -replace -Begin 0 -End 2 \
-parameter_list {SIM_KEEP_PATH 1}

# Write out the STIL file that will used on the tester
write_patterns patterns/processor_core_edt_stuck_retargted.stil \
-stil -replace
exit

```

The following dofile snippet shows how you would retarget at-speed transition ATPG patterns for the wrapped core, gps_baseband. Commands that are not shown are the same as shown above for processor_core.

Example 5-12. Retarget At-Speed Transition ATPG Patterns for the Top-Level

```

# Set the context and open TSDBs ...

# Read in cell libraries, PLL, macro IO pads, designs ...

# Set the current design ...

```

```
# Retarget gps_baseband transition patterns
set_current_mode retarget2_gps_transition
set_static_dft_signal_values retargeting2_mode 1
add_core_instances -instances {GPS_1 GPS_2} -core gps_baseband \
    -mode edt_int_transition
report_core_descriptions
import_clocks -verbose
report_clocks

set_system_mode analysis
write_tsdb_data -replace
# Read the patterns to be retargeted
read_patterns -module gps_baseband -mode edt_int_transition \
    -fault_type transition
set_external_capture_options -pll_cycles 5 [lindex [get_timeplate_list] 0]

write_patterns patterns/gps_edt_transition_retargeted.v -verilog \
    -parallel -replace -begin 0 -end 7 -scan \
    -parameter_list {SIM_KEEP_PATH 1}
write_patterns patterns/gps_edt_transition_retargeted_serial.v \
    -verilog -serial -replace -Begin 0 -End 2 \
    -parameter_list {SIM_KEEP_PATH 1}

# Write out the STIL file that will used on the tester
write_patterns patterns/gps_edt_transition_retargeted.stil -stil -replace

exit
```

RTL and Scan DFT Insertion Flow for Sub-Blocks

When performing the DFT insertion flow with sub-blocks, you insert MemoryBIST and pre-DFT DRCs at the sub-block level and then move up to the sub-block's next parent physical block level (where the sub-block is instantiated) to perform synthesis and scan insertion.

Refer to “[Hierarchical DFT Terminology](#)” for more information about sub-blocks.

You may want to use the sub-block flow for any of the following reasons.

- Multiple instantiations. You only need to perform the DFT insertion flow once for a sub-block. Thereafter, every instantiation of the sub-block will include the inserted DFT hardware.
- Small size. Most sub-blocks are not big enough to be considered their own physical regions.
- Readiness. Sometimes the sub-block RTL is complete before the RTL for the physical layout region, thus you can begin DFT insertion on the sub-block as soon as RTL is ready.

DFT Insertion Flow for the Sub-Block 165

DFT Insertion Flow for the Next Parent Level 166

DFT Insertion Flow for the Sub-Block

The DFT insertion flow for sub-blocks is similar to the insertion flow for physical blocks with a few exceptions.

The main differences between this flow and that for physical blocks as described in “[RTL and Scan DFT Insertion Flow for Physical Blocks](#)” are:

- Do not perform synthesis or scan insertion at the sub-block level because the sub-block netlist may not exist after synthesis.
- During the second DFT insertion pass, you will only be inserting pre-DFT DRCs.

Typically, you do not insert EDT controllers at the sub-block level because the logic inside of the sub-block is too small and the sub-block module itself may not exist after synthesis. You can insert an EDT controller at the next parent level that you dedicate to testing the logic inside of a sub-block. In most cases, this EDT controller is active at the same time as other EDT controllers that are present at the next parent level.

Note

 The sub-block flow for inserting MemoryBIST and pre-DFT DRCs follows the same steps as discussed in the “[RTL and Scan DFT Insertion Flow for Physical Blocks](#),” with the exception of generating the EDT and OCC hardware. There are slight variations, which are described in the following procedure.

Procedure

1. [First DFT Insertion Pass: MemoryBIST and Boundary Scan](#). Ensure that you set the design level to “sub_block” rather than “physical_block”.

```
set_design_level sub_block
```
2. Second DFT insertion pass: insert pre-DFT DRCs. Follow the steps for the “[Second DFT Insertion Pass: Block-Level EDT and OCC](#)” procedure, excluding generating the EDT and OCC hardware.

Since you specified that you were working at the sub_block level in the first DFT insertion pass, you do not need to re-specify this information for the second insertion pass.

After specifying and verifying the DFT requirements, Tessent Shell performs the following tasks automatically:

- At the sub-block level, any static DFT signals you have added are implemented as IJTAG ports rather than inserted via TDRs. Tessent Shell automatically connects the IJTAG ports to the TDR at the next parent level.
- At the next parent level, adds DFT signals such as ltest_en and async_set_reset_static_disable. Tessent Shell infers the add_dft_control_point command on the sub-block pins if you have DFT DRCs that required fixing.
- At the next parent level, adds the tck_occ_en DFT signal if there is a STI-SIB network inside the sub-block.

The following command performs pre-DFT DRC.

```
set_dft_specification_requirements -logic_test on
```

During ICL extraction, Tessent Shell generates the Synopsys Design Constraints (SDC) for the sub-block.

Results

Proceed to performing the DFT insertion flow on the next parent level where this sub-block is instantiated.

DFT Insertion Flow for the Next Parent Level

The next parent level where a sub-block is instantiated may be a physical layout block that is a wrapped core or the physical layout region that is the chip. The following procedure describes the flow when you have a sub-block instantiated at the chip-level.

Prerequisites

- You have performed the DFT insertion flow as described in “[DFT Insertion Flow for the Sub-Block](#)” for the sub-blocks instantiated at this parent level so that you have the design after a clean pre-DFT DRC run.

Note

 The following procedure follows the DFT insertion process described in “[RTL and Scan DFT Insertion Flow for the Top Chip](#)”. When you have a sub-block inserted inside a wrapped core, follow the steps described in “[RTL and Scan DFT Insertion Flow for Physical Blocks](#),” ensuring that you load the sub-block design as described below.

Procedure

1. [First DFT Insertion Pass: Performing Top-Level MemoryBIST and Boundary Scan](#). When you load the design, open the sub-block’s TSDB and load the design of the sub-block that passed pre-DFT DRCs. See the example below.
2. [Second DFT Insertion Pass: Top-Level EDT and OCC](#). When you load the design, open the sub-block’s TSDB, load the full design for the sub-block, and load the design for the top-level after the first DFT insertion pass. See the example below.
3. [Synthesis](#). Synthesis of the chip-level RTL and the sub-block’s post-DFT inserted RTL occur at the same time. Tessent Shell merges the sub-block into the parent-level logic.
Refer to “[Timing Constraints \(SDC\)](#)” to learn how the synthesis constraints from the sub-block are merged at the next parent level.
4. [Scan Chain Insertion](#). Tessent Shell performs scan chain insertion on the sub-block and parent-level logic at the same time.
5. [ATPG Pattern Generation](#).

Examples

Design Loading for the First DFT Insertion Pass

The following example shows opening the TSDB for the sub-block and using read_design to read in the sub-block (processor_core) design.

```
# Set the context to insert DFT into RTL-level design
set_context dft -rtl -design_id rtl1
# Set the location of the TSDB. Default is the current working directory.
set_tsdb_output_directory ../tsdb_top

# Open the TSDBs for all the instantiated sub-blocks
open_tsdb ../../sub_block/tsdb_outdir
# Read the tessent cell library
read_cell_library ../../library/tessent/adk.tcelllib
# Read the design
read_verilog ../../rtl/chip_top.v
```

```
# Explicitly load the sub-block netlist
read_design_processor_core -design_id rtl2 -verbose
set_current_design chip_top
set_design_level chip

# Follow the rest of the flow for the first DFT insertion pass for a chip.
```

Design Loading for the Second DFT Insertion Pass

```
# Set the context to insert DFT. Define a new design ID
set_context dft -rtl -design_id rtl2

# Set the location of the TSDB. Default is the current working directory.
set_tsdb_output_directory ../tsdb_top

# Open the TSDBs for all the instantiated sub-blocks
open_tsdb ../../sub_block/tsdb_outdir

# Read the tessent cell library
read_cell_library ../../library/tessent/adk.tcelllib

# Read the verilog
read_verilog ../../rtl/noncore_blocks/p11.v -blackboxset_design_sources \
    -format verilog -v ../../rtl/noncore_blocks/iopad_sel.v \
    -v ../../rtl/noncore_blocks/iopad.v

read_design chip_top -design_id rtl1 -verbose
# Explicitly load the sub-block netlist
read_design_processor_core -design_id rtl2 -verbose

set_current_design chip_top

# Follow the rest of the flow for the second DFT insertion pass for a chip
```


RTL and DFT Insertion Flow with Third-Party Scan

The RTL and DFT insertion flow allows you to use a third-party scan insertion tool instead of Tessent Scan. Regardless of which scan insertion tool you use, you must follow a number of sequences in the DFT insertion flow that include creating the logic for internal and external mode, and the multiplexing mechanism that Tessent Scan normally inserts in order to support hierarchical ATPG.

The work flow uses the standard hierarchical DFT insertion flow, both for each wrapped core and, once you have inserted DFT in each wrapped core, for top chip.

See “[Hierarchical DFT Terminology](#)” on page 129.

Note

 This discussion assumes your design consists of wrapped cores as your lower level physical blocks, and the wrapped cores do not contain embedded pad I/Os—see “[How to Use Boundary Scan in a Wrapped Core](#)” on page 261.

DFT Insertion Flow With Third-Party Scan Insertion	169
Wrapped Core DFT Insertion with Third-Party Scan.....	170
Top Chip DFT Insertion with Third-Party Scan	184

DFT Insertion Flow With Third-Party Scan Insertion

The DFT insertion flow with third-party scan insertion requires you to manually collect and provide certain data during hierarchical ATPG. Normally when using Tessent Scan for scan insertion and stitching, the information is transferred that is needed for performing hierarchical ATPG.

- **Prerequisites** — Before starting this flow, you should identify the DFT functions you need to insert in each core as well as how many scan chains and wrappers cells for each core.

Note

 Tessent Scan automatically concatenates scan chain and wrapper chains into mixed chains to achieve the number of scan channels on the EDT logic.

Wrapped Core DFT Insertion with Third-Party Scan

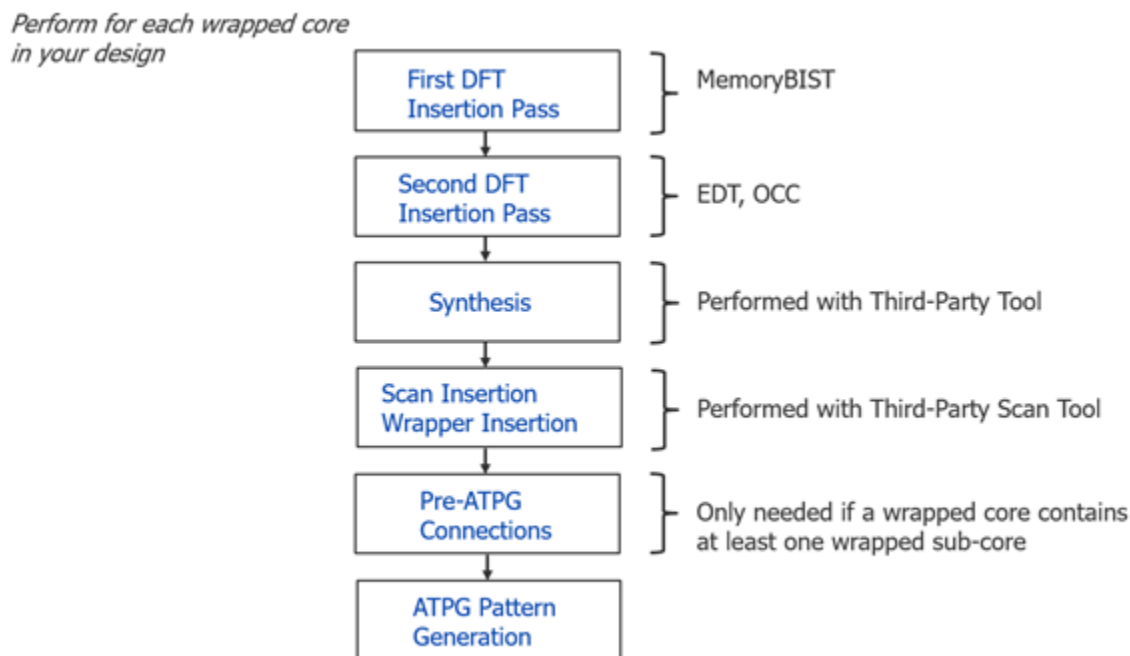
For each wrapped core, perform a two-pass pre-scan DFT insertion process as you would for a flat design except that in the first DFT insertion pass, do not insert boundary scan unless you have embedded pad IO macros present inside the core. This section describes the flow details that are unique to working with wrapped cores that you use a third-party scan insertion tool to insert the scan.

Refer to “[RTL and Scan DFT Insertion Flow for Physical Blocks](#)” on page 133 for overall details.

Note

This discussion assumes your design consists of wrapped cores as your lower level physical blocks, and the wrapped cores do not contain embedded pad IOs.

Figure 5-21. Two-Pass Insertion Flow for RTL, Wrapped Cores, and Third-Party Scan



Perform Two-Pass DFT Insertion and Synthesis for Wrapped Cores	171
Third-Party Scan Insertion for Wrapped Cores	172
Make Pre-ATPG Connections with Third-Party Scan for Wrapped Cores	179
Perform Wrapped Core Graybox Generation and ATPG Pattern Generation	181

Perform Two-Pass DFT Insertion and Synthesis for Wrapped Cores

This flow for wrapped cores is the same as the DFT insertion flow for physical blocks except you insert the scan with a third-party scan tool instead of Tessent Scan.

Procedure

1. [First DFT Insertion Pass: MemoryBIST and Boundary Scan](#). Ensure you set the design level to “physical_block”.

```
set_design_level physical_block
```

2. [Second DFT Insertion Pass: Block-Level EDT and OCC](#). Ensure you set the design level to “physical_block”.

- a. During the second insertion pass and when specifying the DFT signals, follow the procedure defined in “[Specify and Verify the DFT Requirements: DFT Signals for Wrapped Cores](#)” on page 139 noting the following for internal and external modes that are required for mode switching:

```
add_dft_signals int_ltest_en ext_ltest_en int_mode ext_mode \  
-create_with_tdr
```

```
add_dft_signals input_wrapper_scan_en output_wrapper_scan_en \  
-create_from_other_signals
```

- b. [Create the DFT Specification](#). The test case implements compressed ATPG test using Tessent TestKompress. You define the EDT logic for Tessent TestKompress and the Tessent OCC in an EDT DftSpecification wrapper as follows:

```
read_config_data -in $spec -from_string "  
  OCC {  
    ijtag_host_interface : Sib(occ);  
    type : standard;  
  }  
  EDT {  
    ijtag_host_interface : Sib(edt);  
    Controller (c1) {  
      longest_chain_range : 50, 65;  
      scan_chain_count : 80;  
      input_channel_count : 2;  
      output_channel_count : 1;  
      Connections {  
        EdtChannelsIn(2:1) {  
          port_pin_name : edt_channel_in[1:0];  
        }  
        EdtChannelsOut(1:1) {  
          port_pin_name : edt_channel_out[0:0];  
        }  
      }  
    }  
  }  
}"
```

You define the scan_chain_count as follows:

scan_chain_count = number of internal only scan chains + number of wrapper chain

This number covers all chains that the EDT logic at the current level needs to connect to. In other words the EDT hardware need to be connected to both the internal only chains and wrapper chains.

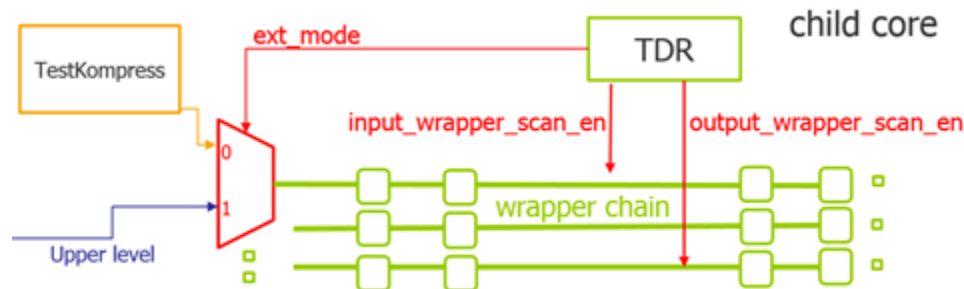
3. [Generate the EDT and OCC Hardware](#).
4. [Extract the ICL Module Description](#).
5. [Generate ICL Patterns and Run Simulation](#).
6. [Perform Synthesis](#).

Third-Party Scan Insertion for Wrapped Cores

The following guidance provides general instructions for using a third-party scan insertion tool to insert scan and perform scan stitching for wrapped cores in your design. The actual process depends on the third-party scan insertion tool you use. You must wrap all the hierarchical physical blocks and perform scan insertion and stitching in your design.

For hierarchical ATPG, you must insert multiplexers before each wrapper chain to multiplex the primary input from the top level and the current level of the EDT scan input. Figure 5-22 shows an abstract level depiction of the multiplexing logic.

Figure 5-22. DFT Signals and Multiplexer Logic Support Hierarchical ATPG



To the child cores, the pre-registered static DFT signals `ext_mode` and `int_mode` control these multiplexers as follows:

- **External Mode** — In this mode, the `ext_mode` DFT signal is active and the `int_mode` DFT signal is inactive. The top level drives input into the wrapper chains through the wrapper’s scan in on the core boundary.
- **Internal Mode** — In this mode, the `ext_mode` DFT signal is inactive and the `int_mode` DFT signal is active. The EDT logic drives all the scan chains inside the hierarchical physical block (both internal only and wrapper chains).

When using this configuration, you must control the corresponding `TDR` bits for mode switching. See “[Perform Wrapped Core Graybox Generation and ATPG Pattern Generation](#)” on page 181 for a detailed explanation.

Usage Guidelines

Use the following guidelines and criterion for configuring your third-party scan insertion tool for wrapper and scan stitching:

- For the hierarchical physical block you intend to perform scan insertion, you must have completed, you have completed the “[Perform Two-Pass DFT Insertion and Synthesis for Wrapped Cores](#)” on page 171 flow.
- You have identified the scan chains of the current core and how many among these scan chains should be wrapper chains. The rest of the scan chains can be specified and used as core chains.
- During wrapper analysis, you must exclude the following pins from any wrapper insertion:
 - IJTAG-related pins

- `edt_channel_input` pins
- `edt_channel_output` pins
- `scan_enable` pins
- Any clock pins
- The DFT signals for input and output wrapper scan enable that you have defined during [the second DFT insertion pass](#) should be used as scan enable signals for input and output wrapper chains, respectively, during wrapper analysis and insertion.
- To run ATPG after scan insertion, you must create and supply a test procedure file for the [Graybox generation step](#) to trace and control the wrapper chains. A test procedure file tells the tool how to clock the scan chains, and the tool automatically passes this to ATPG in the Tessent Shell flow. A graybox does not include an OCC and EDT logic for tracing and controlling wrapper chains; consequently, you must create the test procedure file manually when using the third-party scan insertion flow.

See “[Test Procedure File](#)” on page 283 for complete details on how you create and use a test procedure file. See also “[Example Test Procedure File](#)” on page 177.

- The Tessent Shell environment `get_dft_info_dictionary` command offers you a method to access the Tessent Database (TSDB) to use this information with your third-party scan insertion tool.

The command reads the scan information from the TSDB’s [dft_inserted_designs](#) directory, specifically in a Tcl dictionary file named:

[<design_name>.dft_info_dictionary](#)

The file contains the information about the DFT inserted in the design that must be considered during scan insertion when using a third-party scan insertion tool. See “[Example dft_info_dictionary File](#)” on page 178.

- The Tessent Shell environment provides a mechanism for generating an example usage script you can customize to work with your third-party scan insertion tool. In the Tessent Shell tool, invoke the following commands:

```
read_verilog design_netlist
source \
../tsdb_outdir/dft_inserted_designs/
design_name.last_DFT_insertion_design_id/
design_name.dft_info_dictionary
get_dft_info_dictionary -example_usage_script
```

After issuing these commands, the tool creates an example usage script. Use this script as a starting example to convert the dictionary into the specific commands used by your

third-party scan insertion tool. In the file, the tool inserts pound signs (#) with comments that specify which actions your third-party tool must perform. For example:

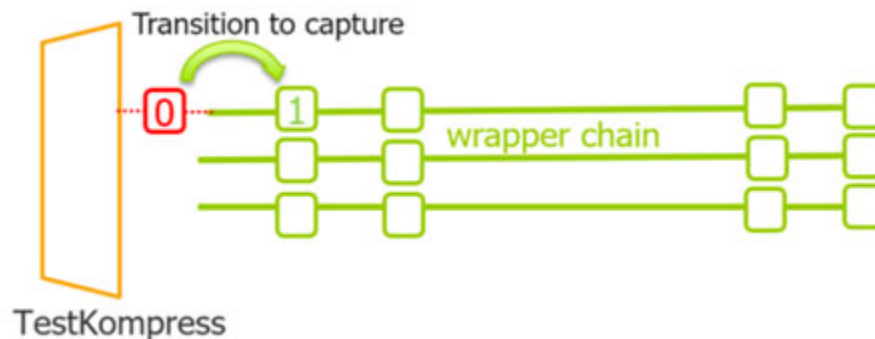
```
puts "Declare the Non-Scannable instances"

set non_scannable_instance_list [dict get $tessent_dft_info_dict
non_scannable_instance_list]
if {[llength $non_scannable_instance_list] > 0} {
    ### Command to declare the Non-Scannable instances
    add_nonscan_instances [get_instances
$non_scannable_instance_list]
}
...
```

After you add your third-party tool specific commands to the file, source the file to your scan insertion tool to finish the scan insertion and stitching.

- For the hierarchical transition fault model, you may need to insert one additional at-speed cell at the beginning of each wrapper chain in order to make the transition on the first cell of each wrapper chain. This way, you will achieve the best coverage. The method you use depends on the third-party insertion tool. [Figure 5-23](#) shows the logic.

Figure 5-23. At-Speed Flows in the Wrapper Chain



- If required and after you have inserted the scan using your third-party tool, proceed to [“Make Pre-ATPG Connections with Third-Party Scan for Wrapped Cores”](#) on page 179 if required.
- If no pre-ATPG connections are required, then load the scan-inserted netlist into Tessent Shell and write the modified netlist and scan information back to the Tessent Shell Database (TSDB).

Writing the Design Back to the TSDB

After completing scan insertion by your third-party scan insertion tool, you must write the scan-inserted netlist back to the TSDB to associate the netlist with Tessent DFT instruments by creating a softlink to the scan inserted netlist in TSDB. Doing this also creates the Tessent Core Description files and the ICL information.

Note

 If your design meets the criteria specified in [“Make Pre-ATPG Connections with Third-Party Scan for Wrapped Cores”](#) on page 179, then write the scan inserted netlist back to the TSDB after you have made the pre-ATPG connections.

Use the following procedure to save a scan-inserted netlist to the TSDB:

1. Set the context. (See lines 1-3.)
2. Set the TSDB location if not already set. (See line 6.)
3. Load the cell library. (See line 9.)
4. Load the scan-inserted netlist. This is the netlist modified by your third-party scan insertion tool and contains the scan cells. (See line 12.)
5. Load the supporting DFT files (except the netlist) from the last DFT insertion pass and set current design. (See lines 15-17.)
6. Set the design level. Make sure you specify “physical_block”. (See line 21.)
7. Write the design information to the TSDB and create the softlink. (See line 22.)

Example dofile:

```
1 # We use the design_id as "scan" and this will be used in all the ATPG
2 # runs that this design will be read in.
3 set_context patterns -ijtag -design_id <scan>
4
5 # Set the location of the TSDB. Default is the current working directory
6 set_tsdb_output_directory ../tsdb_outdir
7
8 # Read the Tessent Cell Library
9 read_cell_library ../../../../library/tessent/adk.tcelllib
10
11 # Read the scan inserted netlist and elaborate the design
12 read_verilog ../3.synthesize_rtl/<current_core>_scan.vg
13
14 # Read the -no_hdl from the last DFT insertion pass
15 read_design <current_core> -design_id <rtl2> -no_hdl -verbose
16 read_verilog ../../../../library/memories/SYNC_1RW_8Kx16.v
17 set_current_design <current_core>
18
19 # Specify the design level before writing out a softlink of the design in
20 # TSDB
21 set_design_level physical_block
22 write_design -tsdb -softlink_netlist -verbose
23 exit
```


Example Test Procedure File

```
set time scale 1.000000 ns ;
timeplate gen_tp1 =
    force_pi 0 ;
    measure_po 10 ;
    pulse_clock 20 10 ;
    pulse clock2 20 10;
    period 40 ;
end;
procedure shift =
    scan_group grp1 ;
    timeplate gen_tp1 ;
    // cycle 0 starts at time 0
    cycle =
        force_sci ;
        measure_sco ;
        pulse clock1;
        pulse clock2;
    end;
end;

procedure load_unload =
    scan_group grp1 ;
    timeplate gen_tp1 ;
    // cycle 0 starts at time 0
    cycle =
        force clock1 0;
        force clock2 0;
        force scan_enable 1 ;
    end ;
    apply shift 76;
end;
```

Example dft_info_dictionary File

```
set tessent_dft_info_dict {
  version 1
  dft_signals {
    dft_signal_name {
      connection_node_name node_name
      connection_node_type port|pin
      forced_value_in_pre_scan_drc 0|1
    }
  }
  modules_with_chains {
    module_name {
      pre_scan_drc_on_boundary_only 0|1
      module_type normal|memory|occ
      is_hard_module 0|1
      internal_scan_only 0|1
      allow_scan_out_retiming 0|1
      instance_list {inst_name ...}
      scan_en_ports|ltest_en_ports|set_reset_ports {
        port_name { active_polarity 0|1
        }
      }
      clock_ports {
        port_name {
          off_state 0|1
        }
      }
      clock_out_ports_or_pins {
        port_or_pin_name {}
      }
      scan_chains {
        scan_chain_name {
          length auto|int
          scan_in_port port_name
          scan_in_clock_name port_name
          scan_in_clock_inversion 0|1
          scan_out_port port_name
          scan_out_clock_name port_name
          scan_out_clock_inversion 0|1
        }
      }
    }
  }
  non_scannable_instance_list {inst_name ...}
  edt_instances {
    instance_name {
      edt_module_name module_name
      scan_chains {
        chain_name {
          scan_in_port port_name
          scan_out_port port_name
        }
      }
    }
  }
}
```

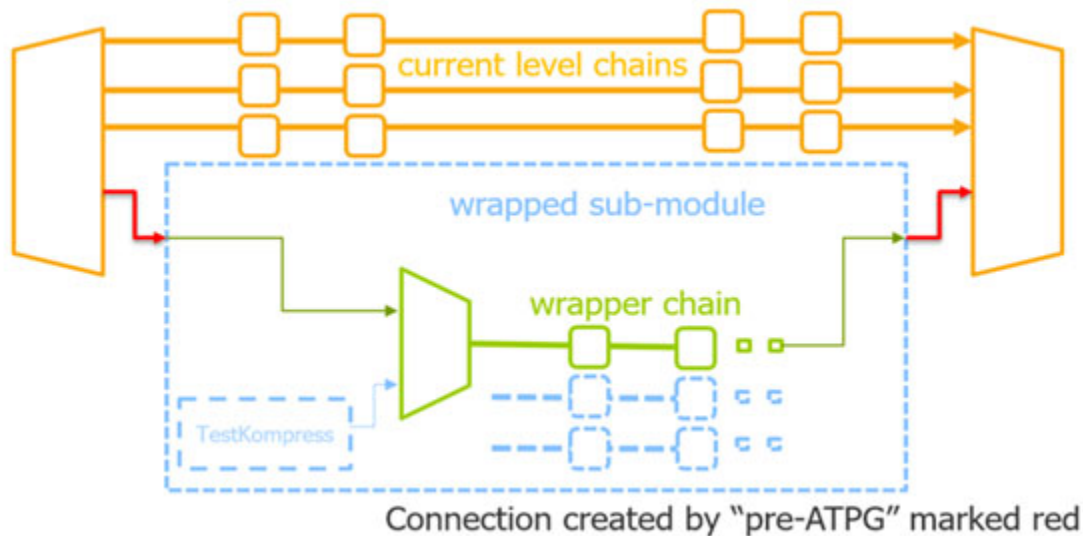
Make Pre-ATPG Connections with Third-Party Scan for Wrapped Cores

You perform this step in flow only if the wrapped core has at least one child wrapped core.

If your design contains no child wrapped cores, skip this step in the flow and proceed directly to [“Perform Wrapped Core Graybox Generation and ATPG Pattern Generation”](#) on page 181.

This procedure creates connections from the current level EDT logic to wrapped child cores' wrapper chains for paths depicted in red in [Figure 5-24](#).

Figure 5-24. Current Level Path to Child Core Wrapper Chains



By doing so, external mode logic and chains of child cores become part of current test coverage.

Note

The line numbers used in this procedure refer to the command flow dofile in [“Example Dofile for Pre-ATPG Connections for Wrapped Cores”](#) on page 180.

Prerequisites

- You must have performed the two-pass DFT insertion as described in [“Perform Two-Pass DFT Insertion and Synthesis for Wrapped Cores”](#) on page 171.
- You must have inserted scan using your third-party scan insertion tool per the guidelines provided in [“Third-Party Scan Insertion for Wrapped Cores”](#) on page 172.

Procedure

- Load the design. (See lines 1-7).
- Change to insertion mode. (See line 9).
- Make the connections. (See lines 12-26).

4. Save the design. (See line 28).
5. Write the design back to the TSDB. (See lines 30-66).

Examples

Example Dofile for Pre-ATPG Connections for Wrapped Cores

```
1  set_context dft -no_rtl
2
3  read_verilog ../3.synthesis/<current_design_name>_scan.vg
4
5  read_cell_library ../../../../library/tessent/adk.tcelllib
6
7  set_current_design <current_design_name>
8
9  set_system_mode insertion
10
11
12 # in gate level
13
14 # empty pins of edt logic were grounded, need to delete before connection
15 delete_connection <current_edt_instance>/edt_scan_out[]
16
17
18 delete_connection <current_edt_instance>/edt_scan_in[]
19
20
21 # connect edt scan-in/scan-out to ext_wsi/ext_wso of every chain
22 create_connection /<child_core_instance>/ext_wsi[] \
23                 <current_edt_instance>/edt_scan_in[]
24
25 create_connection /<child_core_instance>/ext_wso[] \
26                 <current_edt_instance>/edt_scan_out[]
27
28 write_design -output_file ../3.synthesis/<current_design_name>.vg
29
30 # We use the design_id as "scan" and this will be used in all the ATPG
31 # runs that this design will be read in.
32
33 set_context patterns -ijtag -design_id <scan>
34
35 # Set the location of the TSDB. Default is the current working directory
36 set_tsdb_output_directory ../tsdb_outdir
37
38 # Read the Tessent Cell Library
39 read_cell_library ../../../../library/tessent/adk.tcelllib
40
41 # Read the scan inserted netlist and elaborate the design
42 read_verilog ../3.synthesize_rtl/<current_core>_scan.vg
43
44 # Read the -no_hdl from the last DFT insertion pass
45
46 read_design <current_core> -design_id <rtl2> -no_hdl -verbose
47
48 read_verilog ../../../../library/memories/SYNC_1RW_8Kx16.v
49
50 set_current_design <current_core>
```

```
51
52 # Specify the design level before writing out a softlink of the design in
53 # TSDB
54 set_design_level physical_block
55 write_design -tsdb -softlink_netlist -verbose
56 exit
```

Perform Wrapped Core Graybox Generation and ATPG Pattern Generation

This step in the flow creates the graybox model of the current wrapped core and generates ATPG patterns. You must perform this step for each wrapped core in your design.

Prerequisites

- You must have completed the [“Perform Wrapped Core Graybox Generation and ATPG Pattern Generation”](#) on page 181 step for each wrapped core.
- You must have inserted scan with your third-party scan insertion tool per the guidelines cited in [“Third-Party Scan Insertion for Wrapped Cores”](#) on page 172 for each wrapped core and write the scan-inserted netlist back into the Tessent Shell Database—see [“Writing the Design Back to the TSDB.”](#)
- If required, make ATPG connections using the process outlined in [“Make Pre-ATPG Connections with Third-Party Scan for Wrapped Cores”](#) on page 179 for each wrapped core.

Procedure

1. [Perform ATPG Pattern Generation: Wrapped Core.](#)
 - a. Generate graybox model for the wrapped core. Add scan group of wrapper chains by importing the [test procedure file](#) you created during [Third-Party Scan Insertion for Wrapped Cores](#)—see [“Example Graybox Generation”](#) on page 182.
 - b. External pattern generation. You generate these patterns to check fault coverage for the wrapped core. You do not retarget these patterns. See [“Example External ATPG Pattern Generation.”](#)
 - c. Internal pattern generation. As long as the scan mode (scan chains and test logic) is stitched conforming to DRC, Tessent Shell will generate ATPG patterns formed in correct way.

See [“Example Internal ATPG Pattern Generation.”](#)
2. Save the design and write the patterns to the TSDB using the [write_tsdb_data](#) command.


```
write_tsdb_data -replace
```
3. Repeat the process for each wrapped core to generate transition patterns. All of the information is passed by the Tessent Shell through the TSDB.

Results

Once you have done this for each wrapped core, you need to perform scan insertion for the top chip. See “[Top Chip DFT Insertion with Third-Party Scan](#)” on page 184 for complete flow details.

Examples

Example Graybox Generation

```
set_context pattern -scan -design_id <scan>
set_tsdb_output_directory ../tsdb_outdir
read_cell_library ../../../../library/tessent/adk.tcelllib
read_design <current_design_name> -design_id <scan>
set_current_design <current_design_name>
# set memory modules black box
add_black_box -module SYNC_1RW_32x16_RC_BISR
add_black_box -module SYNC_1RW_32x4
add_clock 0 clock1
# Graybox generation requires core configured to external mode by
# setting DFT signal values:
set_static_dft_signal_values int_mode 0
set_static_dft_signal_values ext_mode 1
set_static_dft_signal_values int_ltest_en 0
set_static_dft_signal_values ext_ltest_en 1
# exclude edt_channels for graybox
set_attribute_value [get_ports *edt_channel*] \
                    -name ignore_for_graybox -value true
# import wrapper chain information by importing testproc file of
# wrapper chains
add_scan_group grp1 ../4.scan_insertion/wrapper.testproc
# add every wrapper chain from input pin to output pin
add_scan_chains chain1 grp1 ext_wsi[0] ext_wso[0]
add_scan_chains chain2 grp1 ext_wsi[1] ext_wso[1]
add_scan_chains chain3 grp1 ext_wsi[2] ext_wso[2]
set_system_mode analysis
# external mode information saved in tsdb
write_design -graybox -tsdb -verbose
exit
```

Example External ATPG Pattern Generation

```
set_context pattern -scan -design_id <scan>
set_tsdb_output_directory ../tsdb_outdir
read_cell_library ../../../../library/tessent/adk.tcelllib
read_design <current_design_name>-design_id <scan> -view graybox
set_current_design <current_design_name>
add_clock 0 clock
# create mode information for external ATPG
set_current_mode ext_multi_stuck -type external
# configure core to external mode
set_static_dft_signal_values int_mode 0
set_static_dft_signal_values ext_mode 1
set_static_dft_signal_values int_ltest_en 0
set_static_dft_signal_values ext_ltest_en 1
# for transition fault ATPG, replace stuck with transition
set_fault_type stuck
# import wrapper chain information
add_scan_group grp1 ../4.scan_insertion/wrapper.testproc
add_scan_chains chain1 grp1 ext_wsi[0] ext_wso[0]
add_scan_chains chain2 grp1 ext_wsi[1] ext_wso[1]
add_scan_chains chain3 grp1 ext_wsi[2] ext_wso[2]
set_system_mode analysis
create_pattern
write_tsdb_data -replace
exit
```

Example Internal ATPG Pattern Generation

```
set_context patterns -scan -design_id <scan>
set_tsdb_output_directory ../tsdb_outdir
read_cell_library ../../../../library/tessent/adk.tcelllib
set_tsdb_output_directory ../tsdb_outdir
read_cell_library ../../../../library/tessent/adk.tcelllib
read_design <current_design_name> -design_id <scan>
add_black_Box -module SYNC_1RW_32x16_RC_BISR
add_black_Box -module SYNC_1RW_32x4
set_current_design <current_design_name>
# current_mode will be used when add core instance at top level
set_current_mode edt_int_stuck -type internal
# import core instances of OCC, EDT and sib_sti if exist
add_core_instances -module <occ_instance_name> -parameter_values {}
add_core_instances -module <edt_instance_name>
add_core_instances -module <sib_sti_instance_name>
# for transition fault ATPG, replace stuck with transition
set_fault_type stuck
add_clock 0 clock -pulse_always
# configure core to internal mode
set_static_dft_signal_values int_mode 1
set_static_dft_signal_values ext_mode 0
set_static_dft_signal_values int_ltest_en 1
set_static_dft_signal_values ext_ltest_en 0
report_static_dft_signal_settings
set_system_mode analysis
create_pattern
write_tsdb_data -replace
exit
```

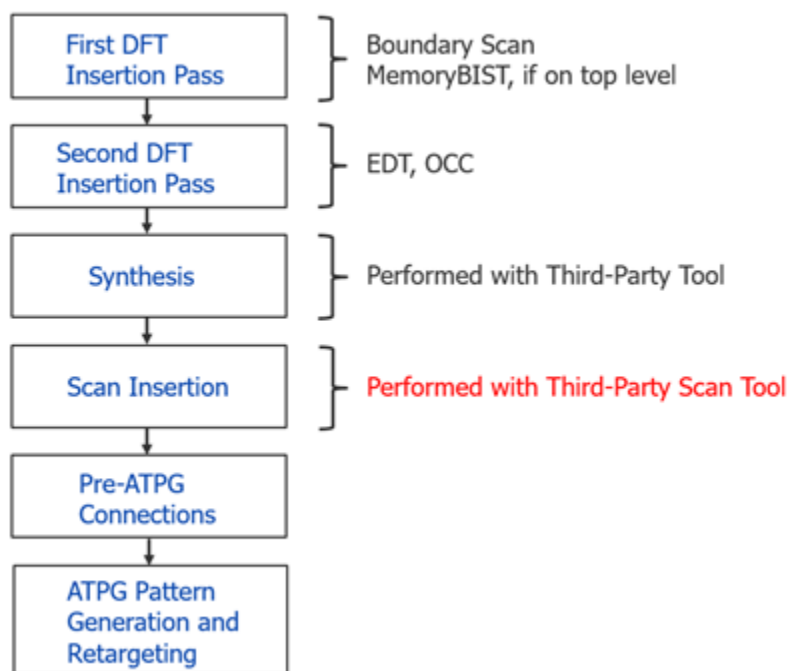
Top Chip DFT Insertion with Third-Party Scan

After performing the RTL and scan DFT insertion flow for each wrapped core in your design, you can perform the DFT insertion process for the top-level chip design using a third-party scan insertion tool to insert the scan.

See “[RTL and Scan DFT Insertion Flow for the Top Chip](#)” on page 150 for overall flow details, especially the Test Access Mechanism.

Figure 5-25. Two-Pass Insertion Flow for RTL, Top Level, and Third-Party Scan

*Perform after wrapped core
DFT insertion*



Perform Two-Pass Insertion and Synthesis for Top Chip	184
Perform Third-Party Scan Insertion for Top Chip	185
Make Pre-ATPG Connections with Third-Party Scan for Top Chip	187
Perform Top Chip Graybox Generation, ATPG Pattern Generation, and Wrapped Core Pattern Retargeting	189

Perform Two-Pass Insertion and Synthesis for Top Chip

After performing the RTL and scan DFT insertion flow for each wrapped core in your design, you can perform the DFT insertion process for the top-level chip design.

Prerequisites

- Before you perform the top level steps, all the child cores must be ready with their retargetable patterns. A spec form for the top level logic is also recommended to define number of scan chains and what DFT instruments to be inserted.
- You must have completed the [Perform Two-Pass DFT Insertion and Synthesis for Wrapped Cores](#) step for each wrapped core.
- You must have completed the [Third-Party Scan Insertion for Wrapped Cores](#) step for each wrapped core.
- If required, you must have completed the [Make Pre-ATPG Connections with Third-Party Scan for Wrapped Cores](#) step for each wrapped core.
- You must have completed the [Perform Wrapped Core Graybox Generation and ATPG Pattern Generation](#) for each wrapped core.

Procedure

1. [First DFT Insertion Pass: Performing Top-Level MemoryBIST and Boundary Scan](#). Ensure you set the design level to “chip” for the top level of the chip.
2. [Second DFT Insertion Pass: Top-Level EDT and OCC](#). Ensure you set the design level to “chip” for the top level of the chip.
3. [Perform Synthesis](#).

Results

You now have a design ready for top level scan insertion with your third-party scan insertion tool. Proceed to “[Perform Third-Party Scan Insertion for Top Chip](#)” on page 185.

Perform Third-Party Scan Insertion for Top Chip

The following guidance provides general instructions for using a third-party scan insertion tool to insert scan into the top chip of your design. Your actual process depends on the third-party scan insertion tool you use.

Usage Guidelines

- Top level logic does not require wrapper chains as all primary inputs and primary outputs are controllable.
- The Tessent Shell environment [get_dft_info_dictionary](#) command offers you a method to access the Tessent Database (TSDB) to use this information with your third-party scan insertion tool.

The command reads the scan information from the TSDB's *dft_inserted_designs* directory, specifically in a Tcl dictionary file named:

<design_name>.dft_info_dictionary

This file can be sourced to any Tcl script engine. The file contains the information about the DFT inserted in the design that must be considered during scan insertion when using a third-party scan insertion tool and contains the following sections:

- *dft_signals* — Contains all of the DFT signals.
 - *modules_with_chains* — Contains all modules that already scanned which means should be stitched into scan chains as sub-chains.
 - *non_scannable_instance_list* — Contains those instances set to non-scan during scan insertion.
 - *edt_instances* — Contains and describes the EDT modules that the scan changes connect to.
- The Tessent Shell environment provides a mechanism for generating an example usage script you can customize to work with your third-party scan insertion tool. In the Tessent Shell tool, invoke the following commands:

```
read_verilog design_netlist
source \
../tsdb_outdir/dft_inserted_designs/
design_name.last_DFT_insertion_design_id/
design_name.dft_info_dictionary
get_dft_info_dictionary -example_usage_script
```

After issuing these commands, the tool creates an example usage script. Use this script as a starting example to convert the dictionary into the specific commands used by your

third-party scan insertion tool. In the file, the tool inserts pound signs (#) with comments that specify which actions your third-party tool must perform. For example:

```
puts "Processing DFT signals"
set dft_signals_dict [dict get $tessent_dft_info_dict dft_signals]
puts "Setting up Static Dft Signals"
foreach dft_signal [dict keys $dft_signals_dict] {
  if {[dict exists $dft_signals_dict \
    $dft_signal forced_value_in_pre_scan_drc]} {
    set connection_node_name \
      [dict get $dft_signals_dict $dft_signal connection_node_name]
    set connection_node_type \
      [dict get $dft_signals_dict $dft_signal connection_node_type]
    set forced_value_in_pre_scan_drc \
      [dict get $dft_signals_dict $dft_signal \
        forced_value_in_pre_scan_drc]
    puts "--- Static Dft Signal $dft_signal ---"
    if {$connection_node_type eq "pin"} {
      ### Command to cut and force created pseudo port
      add_primary_input [get_pins [list $connection_node_name]] \
        -internal
      add_input_constraints \
        [get_pins [list $connection_node_name]] \
        -c${forced_value_in_pre_scan_drc}
    } else {
      ### Command tp force port
      add_input_constraints \
        [get_ports [list $connection_node_name]] \
        -c${forced_value_in_pre_scan_drc}
    }
  }
}
...

```

- If required and after you have inserted the scan using your third-party tool, proceed to [“Make Pre-ATPG Connections with Third-Party Scan for Top Chip”](#) on page 187 if required.
- When you have completed scan insertion, proceed to [“Perform Top Chip Graybox Generation, ATPG Pattern Generation, and Wrapped Core Pattern Retargeting”](#) on page 189.

Make Pre-ATPG Connections with Third-Party Scan for Top Chip

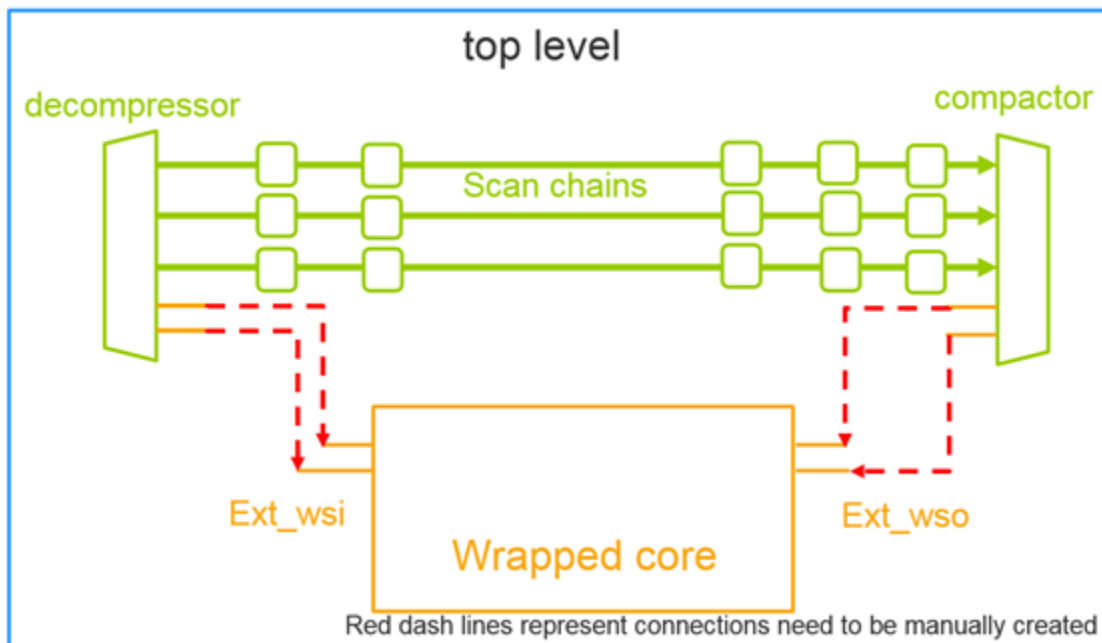
You perform this step in flow only if the wrapped core has at least one sub-wrapped core.

If your design contains no sub-wrapped cores, skip this step in the flow and proceed directly to [“Perform Top Chip Graybox Generation, ATPG Pattern Generation, and Wrapped Core Pattern Retargeting”](#) on page 189.

The most important logic to support hierarchical ATPG is the connection from top level EDT scan in ports to the primary inputs you created for each wrapper chain to cover external faults to all child cores.

Figure 5-26 illustrates this connection at an abstract level.

Figure 5-26. Top Level EDT Connection to Child Cores



Note

 The line numbers used in this procedure refer to the command flow dofile in “[Example Dofile for Pre-ATPG Connections for Top Chip](#)” on page 189.

Prerequisites

- You must have performed the two-pass DFT insertion as described in “[Perform Two-Pass Insertion and Synthesis for Top Chip](#)” on page 184.
- You must have inserted scan using your third-party scan insertion tool per the guidelines provided in “[Perform Third-Party Scan Insertion for Top Chip](#)” on page 185.

Procedure

1. Load the design. (See lines 1-9).
2. Change to insertion mode. (See line 12).
3. Make the connections. (See lines 12-32).
4. Save the design. (See line 36).
5. Write the design back to the TSDB. (See lines 38-40).

Examples

Example Dofile for Pre-ATPG Connections for Top Chip

```
1  # load the design
2  set_context dft -no_rtl
3  set_tsdb_output_directory ../tsdb_outdir
4  open_tsdb \ ../../cores_will_be_wrapped/<child_core_name>/tsdb_outdir
5  read_cell_library ../../library/tessent/adk.tcelllib
6  read_design <top_level_name> -design_id <rtl2> -no_hdl
7  read_verilog ../<synthesis_path>/<top_level_name>.vg
8  read_design <child_core_name> -design_id <scan> -view graybox
9  set_current_design <top_level_name>
10
11 # make the ATPG connections:
12 set_system_mode insertion
13
14 # disconnect empty scan out from ground
15 delete_connection /chip_top_rtl2_tessent_edt_c1_inst/edt_scan_out[20]
16 delete_connection /chip_top_rtl2_tessent_edt_c1_inst/edt_scan_out[21]
17 delete_connection /chip_top_rtl2_tessent_edt_c1_inst/edt_scan_out[22]
18 delete_connection /chip_top_rtl2_tessent_edt_c1_inst/edt_scan_out[23]
19 ...
20 # connect top level EDT scan in to child core primary input of wrapper
21 # chain
22 create_connection /chip_top_rtl2_tessent_edt_c1_inst/edt_scan_in[20]
23 /<child_core_instance_1>/ext_wsi[0]
24 create_connection /chip_top_rtl2_tessent_edt_c1_inst/edt_scan_in[21]
25 /<child_core_instance_1>/ext_wsi[1]
26 ...
27 # connect top level EDT scan out to child core primary output of wrapper
28 # chain
29 create_connection /chip_top_rtl2_tessent_edt_c1_inst/edt_scan_out[20]
30 /<child_core_instance_1>/ext_wso[0]
31 create_connection /chip_top_rtl2_tessent_edt_c1_inst/edt_scan_out[21]
32 /<child_core_instance_1>/ext_wso[1]
33 # repeat for all ext_wsi/wso of all child core instances
34
35 # save design after insertion
36 write_design -output_file <top_level_name>_scan.vg -replace
37
38 # Specify the design level before writing out a softlink of the design
39 set_design_level physical_block
40 write_design -tsdb -softlink_netlist -verbose
41 exit
```

Perform Top Chip Graybox Generation, ATPG Pattern Generation, and Wrapped Core Pattern Retargeting

This step in the flow creates the graybox model of the top and generates ATPG patterns.

Prerequisites

- You must have completed the [“Perform Two-Pass Insertion and Synthesis for Top Chip”](#) on page 184 step for the top chip.

- You must have inserted scan with your third-party scan insertion tool per the guidelines cited in “[Perform Third-Party Scan Insertion for Top Chip](#)” on page 185 for the top chip and write the scan-inserted netlist back into the Tessent Shell Database.
- If required, make ATPG connections using the process outlined in “[Make Pre-ATPG Connections with Third-Party Scan for Top Chip](#)” on page 187 on page 141 for top chip.

Procedure

1. Perform top level ATPG pattern generation using the “[Performing Top-Level ATPG Pattern Retargeting](#)” on page 162.
 - a. Top-level ATPG pattern generation. See [Example 5-13](#) on page 191. Repeat to generate transition patterns. Verification of these patterns are a must to ensure that the circuit functions.
 - b. Pattern retargeting of each child core. See [Example 5-14](#) on page 192

Depending on tester channel availability on how many cores can be run in parallel, the internal mode ATPG patterns from each of the lower-level cores can be retargeted to the chip-level top.

2. Save the design and write the patterns to the TSDB using the [write_tsdb_data](#) command.

```
write_tsdb_data -replace
```

Examples

Top Level ATPG Pattern Generation Example

Example 5-13. Example ATPG Pattern Generation for Top Chip

```
# Import design:
set_context pattern -scan -design_id <scan>
set_tsdb_output_directory ../tsdb_outdir

# this open_tsdb should import all tsdb of child cores
open_tsdb ../../wrapped_cores/<child_core_name>/tsdb_outdir
read_cell_library ../../../../library/tessent/adk.tcelllib

# make sure to import all child cores in graybox view
read_design <child_core_name> -design_id <scan> -view graybox
read_design <top_level_name> -design_id <scan>
set_current_design <design_name>
set_design_level top

# Import core instances we want to active:
add_core_instances -module chip_top_gate2_tessent_occ_INCLK
add_core_instances -module chip_top_gate2_tessent_occ_pll_clock_0
add_core_instances -module chip_top_gate2_tessent_edt_c1

# Configure top level DFT signal values to edt_mode and all wrapped child
# core instances to external mode:
set_static_dft_signal_values edt_mode 1
set_static_dft_signal_values ltest_en 1
set_static_dft_signal_values tck_occ_en 1
set_static_dft_signal_values int_ltest_en 1

# configure child core by set_test_setup_icall
set_static_dft_signal_values ext_mode 1 -instance <child_core_name1>
set_static_dft_signal_values ext_ltest_en 1 -instance <child_core_name1>

# repeat for all child core instances

# generate and save pattern:
set_fault_type stuck
set_system_mode analysis
create_patterns
write_tsdb_data -replace

# write parallel testbench and serial testbench for simulation
write_patterns <top_level_name>_parallel.v -verilog -parallel
write_pattenrs <top_level_name>_serial.v -verilog -serial
# Repeat to generate transition patterns. Verification of these patterns
# are a must to ensure the circuit functions.
```

Pattern Retargeting of Each Child Core Example

Example 5-14. Example Pattern Retargeting of Each Child Core

```
# Import design:
set_context pattern -scan_retargeting
set_tsdb_output_directory ../tsdb_outdir

# this open_tsdb should import all tsdb of child cores
open_tsdb ../../wrapped_cores/<child_core_name>/tsdb_outdir
read_cell_library ../../../../library/tessent/adk.tcelllib

# make sure to import all child cores except the one we are doing
# retargeting in graybox view
read_design <child_core_name> -design_id <scan> -view graybox
read_design <current_core_name> -design_id <scan> -view graybox
read_design <top_level_name> -design_id <scan>
set_current_design <top_level_name>
set_design_level top

# Read in the internal mode core description file of current child core
# by add_core_instance:
# -mode should match the mode while generating internal mode patterns
add_core_instances -instance <current_core_instance_name> \
    -core <current_core_name> -mode edt_int_stuck

# Configure top level DFT signal values to retargeting_mode for this exact
# child core:
set_current_mode retarget1 <current_child_core_name> stuck
# configure top level to retargeting mode for current child core
set_static_dft_signal_values retargeting1_mode 1

# generate and save pattern:
# for transition fault ATPG, replace stuck with transition
set_fault_type stuck
import_clocks
set_system_mode analysis
create_patterns
write_tsdb_data -replace

# write parallel testbench and serial testbench for simulation
write_patterns <current_child_core_name>_parallel.v -verilog \
    -parallel
write_patterns <current_child_core_name>_serial.v -verilog \
    -serial
```


Tessent Shell Post-Layout Validation Flow

Performing physical place and route on pre-layout netlists results in post-layout netlists you must validate before you proceeding to tape-out.

This flow assumes that you are familiar with the pre-layout flows as described in “[Tessent Shell Flow for Flat Designs](#)” and “[Tessent Shell Flow for Hierarchical Designs](#),” especially as related to performing ATPG pattern generation.

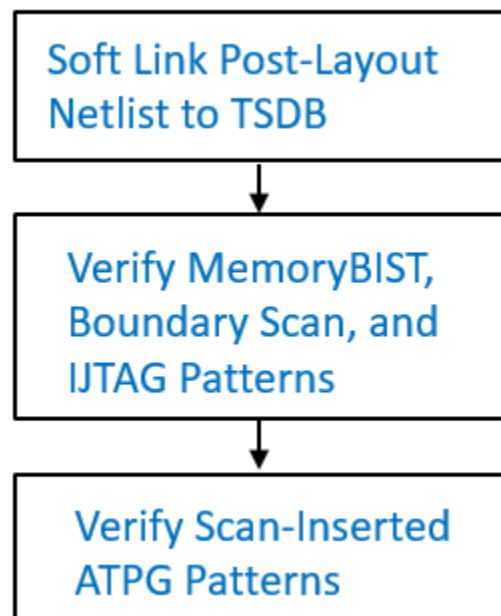
Overview of the Post-Layout Validation Flow	193
Soft Link TSDB and Post-Layout Netlist	194
Verify MemoryBIST, Boundary Scan and IJTAG Patterns	195
Verify the Scan-Inserted ATPG Patterns	196
Post-Layout Validation When You Have Ungrouped IJTAG/OCC/EDT Logic.....	198

Overview of the Post-Layout Validation Flow

You can validate the post-layout netlist for hierarchical core, hierarchical top and flat chip-level designs.

As shown in the following figure, you must first generate a soft link in the TSDB that points to the post-layout netlist. Then you verify the patterns for the MemoryBIST, boundary scan (if any) and IJTAG network, followed by verifying the post-scan-inserted ATPG patterns.

Figure 5-27. Post-Layout Validation Flow



This flow assumes that within the post-layout netlist, modules exist for the IJTAG network and inserted EDT and OCC instruments. That is, they remain distinct logical entities. Refer to “[Post-Layout Validation When You Have Ungrouped IJTAG/OCC/EDT Logic](#)” if you have ungrouped (unpreserved) logic.

Soft Link TSDB and Post-Layout Netlist

The soft link allows Tessent Shell to access the post-layout netlist for validation purposes. You are associating and linking the post-layout netlist (place-and-routed design) with all the pre-layout data files such as the ICL, TCD, and PDL.

Prerequisites

- You have previously performed the pre-layout flow so that you have the post-scan inserted DFT data files (ICL, PDL, TCD, and so on).
- You have post-layout netlist as a result of place and route.

Procedure

1. Load the design, including the post-layout netlist (lines 1-15). Ensure that you specify a unique design ID, such as “post_layout”.
2. Add any black boxes, as applicable, and set the design level, which can be chip, physical_block, or sub_block (lines 17-22).
3. Save the updated netlist (line 23). Ensure that you use the -softlink_netlist option with the [write_design](#) command. Using this switch references the post-layout netlist rather than copies it into the TSDB. This prevents duplication and allows the post-layout netlist to be updated without the need to repeat this step.

Examples

The following example generates an updated netlist for a wrapped core named processor_core that includes a soft link to the core’s post-layout netlist.

```
1  set_context patterns -ijtag -design_id post_layout
2
3  # Set the location of the TSDB
4  set_tsdb_output_directory ../tsdb_core
5
6  # Read the Tessent Cell Library
7  read_cell_library ../../../../library/tessent/adk.tcelllib
8
9  # Read the post-layout netlist and elaborate the design
10 read_verilog ../netlist/processor_core_layout.vg
11
12 # Read in the scan-inserted design data files generated during pre-layout
13 # The -no_hdl switch loads all relevant data except the original netlist
14 read_design processor_core -design_id gate -no_hdl -verbose
15
16 set_current_design processor_core
```

```
17
18 add_black_boxes -modules { \
19                         SYNC_1RW_8Kx16 \
20                     }
21
22 # Specify the design level before writing out the softlink
23 set_design_level physical_block
24 write_design -tsdb -softlink_netlist -verbose
25 exit
```

Verify MemoryBIST, Boundary Scan and IJTAG Patterns

Create and validate the MemoryBIST, boundary scan (if present), and IJTAG network patterns by creating and processing a Patterns Specification with the `create_patterns_specification` and `process_patterns_specification` commands.

Prerequisites

- You have generated a soft link in the TSDB that points to the post-layout netlist as described in “[Soft Link TSDB and Post-Layout Netlist](#).”

Procedure

1. Load the design by using `read_design` and pointing to the soft-linked post-layout netlist (lines 1-9).
2. If needed, add black boxes, and then check the design rules (lines 11-14).
3. Create, simulate and check the testbenches (see lines 16-24). Refer to `create_patterns_specification` and `process_patterns_specification` in the *Tessent Shell Reference Manual* for details.

Examples

Verify the Patterns with a new Patterns Specification

The following example verifies the patterns for a hierarchical core, `processor_core`.

```
1  set_context_patterns -ijtag -design_id post_layout
2  set_tsdb_output_directory ../tsdb_core
3
4  # Reading the tessent cell library
5  read_cell_library ../../../../library/tessent/adk.tcelllib
6
7  # Reading the soft-linked post-layout netlist from the tsdb_core
8  read_design processor_core -design_id post_layout -verbose
9  set_current_design processor_core
10
11 add_black_boxes -modules { \
12                     SYNC_1RW_8Kx16 \
13                 }
14 check_design_rules
15
```

```

16 create_patterns_specification
17 process_patterns_specification
18
19 set_simulation_library_sources -v ../../../../library/memory/
  SYNC_1RW_8Kx16.v
20 -v ../../../../library/verilog/adk.v
21
22 # Disable clock monitoring when running simulation on post-layout netlist
23 run_testbench_simulations -simulation_macro_definitions
  MGC_DISABLE_CLOCK_MONITOR
24 check_testbench_simulations -report_status
25 exit

```

Verify the Patterns with a Customized Patterns Specification from Pre-Layout Signoff

You may have customized a patterns specification during pre-layout signoff. You can read in this patterns specification that was stored in the TSDB during pre-layout signoff. Use the `read_config_data` command instead of the `create_patterns_specification` command.

The following example shows how to read in a customized patterns specification from pre-layout netlist. The pre-layout patterns specification “processor_core_gate.patterns_spec_signoff” was used to create the patterns on the gate-level scan-inserted netlist.

```

set_context patterns -ijtag -design_id after_layout
set_tsdb_output_directory ../tsdb_core
# Reading the tessent cell library
# Read the soft-linked post-layout netlist and elaborate the design...
check_design_rules
read_config_data ../tsdb_core/patterns/ \
  processor_core_gate.patterns_spec_signoff
process_patterns_specification
# Read in the required libraries and simulate
...

```

Verify the Scan-Inserted ATPG Patterns

Perform ATPG pattern generation on the post-layout netlist.

Prerequisites

- In the SDC that was created during [ICL extraction](#) for the pre-layout DFT insertion flow, set the `tessent_get_preserve_instances` proc to `add_core_instances` when you do not need grayboxes. For example, when you are working with flat designs.

For wrapped cores, set the `tessent_get_preserve_instances` proc to `icl_extraction` to automatically include ICL instances in the grayboxes.

The SDC file is located in the TSDB directory under `dft_inserted_designs`. Refer to [“Timing Constraints \(SDC\)”](#) for more information.

Procedure

1. Load the design (lines 1-9). Ensure that you set the context to patterns -scan and read in the soft-linked post-layout netlist.
2. Set the current mode (lines 11-12). Specify a different name than that used during scan insertion and used during pre-layout pattern generation.
3. Perform the remainder of the ATPG pattern generation flow (see lines 14-35).

Examples

The following example shows how to verify scan-inserted ATPG patterns by using the patterns -scan context and a post-layout netlist. This example shows the flow for a flat design. For hierarchical designs, refer to [“Perform ATPG Pattern Generation: Wrapped Core”](#) for specifics related to wrapped cores and [“Top-Level ATPG Pattern Generation Example”](#) for a top-level example.

```
1  set_context patterns -scan
2  read_cell_library ../library/tessent/adk.tcelllib
3  read_cell_library ../library/mem_ver/memory.lib
4  # Point to the TSDB directory
5  set_tsdb_output_directory ../tsdb_rtl
6
7  # Reading the post-layout netlist
8  read_design cpu_top -design_id post_layout
9  set_current_design cpu_top
10
11 # Use a unique mode name
12 set_current_mode edt_stuck_final
13
14 report_dft_signals
15 # If Tessent Scan was used for scan insertion, can use the scan
   #configuration mode
16 import_scan_mode edt_mode
17 # Set the following DFT Signal values to use the boundary scan chain to
   #apply/capture values that would normally use the I/O pads
18 set_static_dft_signal_values int_ltest_en 1
19 set_static_dft_signal_values output_pad_disable 1
20 # Set the following DFT signal value to apply the shift_capture_clock to
   #the scan-tested network during capture phase of ATPG
21 set_static_dft_signal_value tck_occ_en 1
22 report_static_dft_signal_settings
23
24 set_system_mode analysis
25
26 # Generation of ATPG patterns
27 create_patterns
28 report_statistics -detailed_analysis
29 write_tsdb_data -replace
30
31 # Patterns written out for simulation
32 write_patterns patterns/cpu_top_stuck_parallel.v -verilog -parallel \
   -replace -scan -parameter_list {SIM_KEEP_PATH 1}
33 write_patterns patterns/cpu_top_stuck_serial.v -verilog -serial -replace \
   -parameter_list {SIM_KEEP_PATH 1}
```

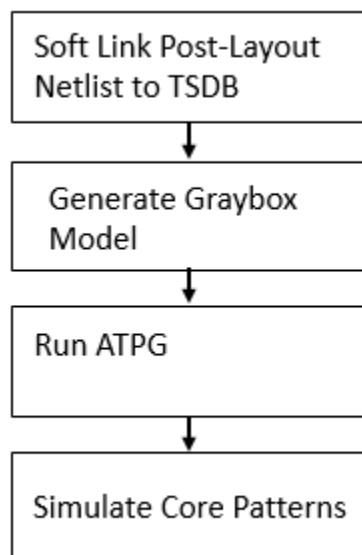
```
34 # Writing the STIL pattern for tester
35 write_patterns patterns/cpu_top_stuck.stil -stil -replace
36 exit
```

Post-Layout Validation When You Have Ungrouped IJTAG/OCC/EDT Logic

During layout, IJTAG, OCC, and EDT logic can become ungrouped, which means the hierarchy around this logic is dissolved so that the gates that were inside of the ungrouped instances become part of the next higher instance. Ungrouped test logic during layout can cause some of the automated setup for pattern generation to no longer operate seamlessly.

The automated flow relies on preservation of Tessent test logic's hierarchical names. Use the following post-layout validation flow to update the TSDB with the post-layout netlist and set up the design for ATPG. The flow assumes knowledge of the ATPG pattern generation flow for physical blocks as described in [“Perform ATPG Pattern Generation: Wrapped Core”](#).

Figure 5-28. Post-Layout Validation Flow with Ungrouped IJTAG/OCC/EDT Logic



The best way to avoid complications related to ungrouping in layout is to use the `tessent_get_preserve_instances` procedures in the generated SDC file to identify which instances must be preserved based on their intended uses. See the [“Synthesis Helper Procs”](#) section in the *Tessent Shell User's Manual*.

To use the `add_core_instances` command during ATPG or other post-layout steps, the hierarchy of the OCC and EDT logic must be preserved. The IJTAG logic nodes only need to be preserved if you plan to rerun ICL extraction on the post-layout netlist, but in most cases there is no need to do this.

Prerequisites

- If you do not preserve the instances, you must write out a TCD file for every mode of operation at the core level before you perform layout. In some cases, you may not need to generate patterns until after layout, but even then, you must run ATPG before layout to generate the core-level TCD files. Failure to perform this task could result in R14 or R15 rule check errors.

Procedure

1. Soft link the post-layout netlist to the TSDB. Refer to [“Soft Link TSDB and Post-Layout Netlist”](#) for more information.
2. Generate the graybox model. Graybox models are only required for hierarchical cores, not for hierarchical top or flat designs. (The line numbers in this step refer to the example [“Generate the Graybox Model”](#) on page 200.)
 - a. When loading the design (line 4), specify the same design that you used to write the post-layout netlist to the TSDB, for example “post_layout”.

Using the same design ID for the graybox model that you used for the post-layout netlist allows Tessent to access the full design view or the graybox model with the same design ID.
 - b. Read the core description for external mode using the [add_core_instances](#) command (line 9). Since you already ran the pre-layout ATPG step and saved the TCD file to the TSDB, the [add_core_instances](#) command can read the existing TCD file and add the core instances that are active in external mode.
 - c. Use [analyze_graybox](#) to generate the graybox (line 13). If the IJTAG network was ungrouped in layout, the [analyze_graybox](#) command can automatically find and preserve the IJTAG SRI network as long as the default names of the logic have not changed.
3. If you are running in hierarchical mode, run ATPG on the core’s internal mode of the wrapped core to generate the ATPG patterns that you will retarget at the top level of the chip. If you are running in hierarchical top or in flat mode, run ATPG. (The line numbers in this example refer to example [“Run ATPG on the Core’s Internal Mode”](#) on page 201.)
 - a. Specify a unique ATPG mode name with the [set_current_mode](#) command (line 9). Append the name with “_post_layout” or “_final” to clarify which mode to use for silicon diagnosis. Set the current mode type to “internal”.
 - b. Add the core instances using the design ID and mode from the pre-layout step (line 14). This loads the TCD file that contains the information for the design’s core instances without the need to match them to test logic instances that may have been ungrouped during layout.
 - c. You can use the [read_faults](#) command to merge the fault list from running external mode to find the total overall fault coverage of the wrapped core (line 36).

When you run ATPG in internal mode for transition patterns, note the following:

- Specify a unique name for the ATPG run with the `set_current_mode` command. For example, `edt_int_tdf_final`.
- Use the `add_core_instances` command to read the transition mode TCD. For example:

```
add_core_instance -current_design -design_id gate -mode
edt_int_transition
```

- Ensure that you set the correct fault type:

```
set_fault_type transition
```

- You can optimize the number of capture cycles used by the OCCs by specifying the optional `capture_window_size` parameter. The following command specifies a `capture_window_size` of 2:

```
set_core_instance_parameters -instrument_type occ
-parameter_values [list capture_window_size 2]
```

4. Run Verilog simulation of the core-level ATPG patterns.

Performing this task ensures that the patterns will function as desired when they are retargeted at the parent level. For parallel load patterns, as specified by the `write_patterns -parallel` command, simulate all the patterns. For serial load patterns, a handful of patterns are sufficient; the runtime for simulating gate-level serial load patterns is significant. The `set_pattern_filtering` command (line 27) is used to reduce the number of serial patterns saved for the simulation.

Examples

Generate the Graybox Model

The following example creates a graybox model for a post-layout processor_core design and saves the data under the same design ID.

Figure 5-29. Generate Graybox Example

```
1 set_context patterns -scan -design_id post_layout
2 set_tsdb_output_directory ../tsdb_outdir
3 read_cell_library ../../../../library/standard_cells/tessent/adk.tcelllib
4 read_design_processor_core -design_id post_layout -verbose
5 read_verilog ../../../../library/memories/SYNC_1RW_8Kx16.v -interface_only
6 set_current_design processor_core
7 # Use the add_core_instances command to read the TCD file.
8 #import_scan_mode ext_mode
9 add_core_instances -current_design -design_id gate
10 check_design_rules
11 report_scan_cells
12 # Create and write the updated graybox model to the TSDB.
13 analyze_graybox
14 write_design -tsdb -graybox -verbose
```



```
15 exit
```

Run ATPG on the Core's Internal Mode

Figure 5-30. Run ATPG on the Core's Internal Mode Example

```
1 set_context patterns -scan -design_id post_layout
2 set_tsdb_output_directory ../tsdb_outdirRun/adk.tcelllib
3 read_design processor_core -design_id post_layout -verbose
4 read_verilog ../../../../library/memories/SYNC_1RW_8Kx16.v -interface_only
5 set_current_design processor_core
6 # Specify the current mode using a different name than what was used
7 # during scan insertion or pre-layout
8
9 set_current_mode edt_int_stuck_final -type internal
10
11 # Use the -design_id and -mode from pre-layout to read in the TCD of that
12 # mode
13
14 add_core_instance -current_design -design_id gate -mode edt_int_stuck
15 set_system_mode analysis
16 add_fault -all
17 report_statistics -detail
18 create_patterns
19 report_statistics -detail
20 # Store TCD, flat_model, fault list and patDB format files in the TSDB
21 # directory
22
23 write_tsdb_data -replace
24
25 # Write Verilog patterns for simulation
26 write_patterns patterns/processor_core_stuck_parallel.v -verilog -
parallel -replace -parameter_list {SIM_KEEP_PATH 1}
27 set_pattern_filtering -sample_per_type 2
28 write_patterns patterns/processor_core_stuck_serial.v -verilog -serial -
replace -parameter_list {SIM_KEEP_PATH 1}
29 # Optional Step - Can run in external mode and calculate the fault
30 # coverage of the core(both Internal and External) as described below.
31 # In order to understand the coverage of the faults testable by Internal
32 # mode it is necessary to eliminate the undetected faults that would
33 # otherwise be detected in External mode. This is done by merging the
34 # fault list from running the graybox in External mode with read_faults:
35
36 #read_faults -mode ext_multi_stuck -fault_type stuck -merge -verbose
37
38 # Final coverage of the core that includes both Internal and External
39 # modes
40 # report_statistics -detail
41 exit
```

Hybrid TK/LBIST Flow for Flat Designs

Tessent Shell supports inserting LogicBIST controllers that share IP resources with EDT controllers to reduce hardware overhead. The process for performing this task is known as the hybrid TK/LBIST flow. In this flow, Tessent Shell automatically adds hybrid logic to the EDT controllers for IP resource sharing. The shared LogicBIST and EDT IP is often referred to as hybrid TK/LBIST IPs, or simply hybrid IPs.

This discussion builds on the Tessent Shell RTL and scan DFT insertion flow as described in “[Tessent Shell Flow for Flat Designs](#)” on page 104. Refer to [Hybrid TK/LBIST Flow User’s Manual](#) for details about the hybrid TK/LBIST flow and inserted architecture.

Tip

 This flow increments the basic Tessent Shell RTL and scan DFT insertion flow. To aid comprehension, ensure that you have reviewed the test case for flat designs.

Refer to the following test case for a detailed usage example of the flow described in this section. This test case illustrates hybrid IP insertion into a flat design with MemoryBIST in the first DFT insertion pass and EDT, OCC, and LogicBIST in the second DFT insertion pass.

```
tessent_example_hybrid_tk_lbist_flow_<software_version>.lbist
```

You can access this test case by navigating to the following directory:

```
<software_release_tree>/share/UsageExamples/
```

RTL and Scan DFT Insertion Flow With Hybrid TK/LBIST.....	203
First DFT Insertion Pass: Hybrid TK/LBIST.....	203
Second DFT Insertion Pass: Hybrid TK/LBIST.....	207
Perform Test Point Insertion	212
Perform Scan Chain Insertion	214
Perform ATPG Pattern Generation: Hybrid TK/LBIST	216
Perform LogicBIST Fault Simulation	219
Perform IJTAG Pattern Generation	222

RTL and Scan DFT Insertion Flow With Hybrid TK/LBIST

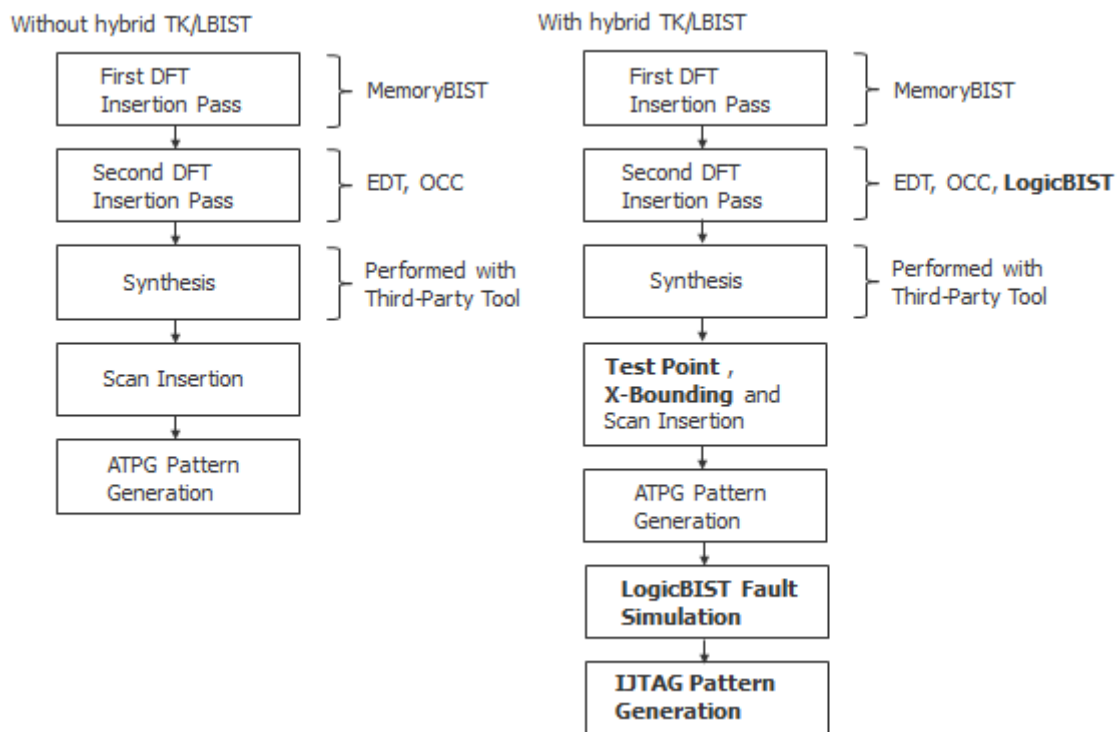
Within the two-pass DFT insertion process for a flat design, insert the hybrid TK/LBIST IP during the second DFT insertion pass.

Adding LogicBIST to your Tessent Shell flow introduces new steps that are described in this section.

Note

The test case that illustrates the hybrid TK/LBIST flow does not include boundary scan insertion. Refer to “[First DFT Insertion Pass: MemoryBIST and Boundary Scan](#)” on page 107 for information about inserting boundary scan in the first DFT insertion pass.

Figure 5-31. Two-Pass Insertion Flow With Hybrid TK/LBIST



First DFT Insertion Pass: Hybrid TK/LBIST

When performing the hybrid TK/LBIST flow to insert the hybrid IP, perform the first pass of the two-pass DFT and scan insertion process as usual—insert MemoryBIST and boundary scan. In addition, for the hybrid TK/LBIST flow, enable controller chain mode (CCM) test by specifying a dedicated control signal.

CCM allows you to generate ATPG patterns that target the hybrid-IP logic so that you can test the test logic itself. Refer to “[Controller Chain Mode](#)” in the *Hybrid TK/LBIST User’s Manual* for details.

Refer to “[First DFT Insertion Pass: MemoryBIST and Boundary Scan](#)” on page 107 for additional information.

Note



The line numbers used in this procedure refer to the command flow dofile in [Example 5-15](#) on page 205.

Procedure

1. Load the RTL design data and set the design parameters. (See lines 1-11.)

Note



“rtl1” is the recommended naming convention for the design ID for the first insertion pass, but you can specify any name, as desired. Refer to “[Load the Design](#)” for more information about setting the design ID. The default design ID for the current test case is “rtl.”

2. Add a DFT signal to use for controller chain mode. (See lines 13-16.) You must register the DFT signal name and then add the signal.

By default, Tessent Shell adds a pin at the current design level to control CCM. You can save this pin by using a TDR register to control CCM; do this by specifying the `add_dft_signal -create_with_tdr` switch.

Whether or not you are inserting MemoryBIST or boundary scan, you must add the CCM DFT signal in the first DFT insertion pass because its generated hardware will be used in the second pass when you insert the hybrid IP.

3. Set the [set_dft_specification_requirements](#) command to “on” for `-memory_bist`. (See lines 18-19.)
4. Define the design clocks. (See lines 21-23.)
5. Check the design rules and set the system mode to analysis. (See lines 25-26.)
6. Create the DFT specification. (See lines 28-31.)
7. Generate the DFT hardware, IJTAG network connectivity, and test patterns. (See lines 33-43.)
8. Run simulations to verify the design. (See lines 45-50.)

Examples

Example 1

The following dofile example shows a typical command flow as detailed in the procedure above. The highlighted command lines are unique to the process for inserting hybrid IP in a two-pass DFT insertion process.

Example 5-15. Dofile Example for First Pass, Hybrid TK/LBIST With MemoryBIST

```
1  # Load the design
2  set_context dft -rtl
3  set_tsdb_output_directory ../tsdb_outdir
4  read_verilog ../rtl/piccpu_rtl.v
5  read_cell_library ../lib/tessent/adk.tcelllib \
6    ../lib/tessent/picdram.atpglib
7  set_design_source -format tcd_memory -y ../rtl -extension memlib
8
9  set_current_design piccpu
10
11 set_design_level physical_block
12
13 # Add DFT signal for controller chain mode test
14 register_static_dft_signal_names HTKLB_CCM_EN \
15   -usage logic_test_control -reset_value 0
16 add_dft_signal HTKLB_CCM_EN -create_with_tdr
17
18 # Specify the DFT requirements
19 set_dft_specification_requirements -memory_test on
20
21 # Define memory clock
22 add_clocks 0 ramclk -period 100ns
23 add_clocks 0 clk -period 100ns
24
25 check_design_rules
26 set_system_mode analysis
27
28 # Create and report the DFT specification
29 # This test case reads in the DFT specification from a separate file
30 read_config_data mbist_ip.dftspec
31 report_config_data
32
33 # Generate and insert the hardware
34 process_dft_specification
35
36 extract_icl
37
38 # Generate MemoryBIST and IJTAG network verification patterns
39 create_patterns_specification
40
41 report_config_data
42
43 process_patterns_specification
44
45 # Run and check test bench simulations
46 set_simulation_library_sources -v {../lib/verilog/adk.v} \
```

```
47 -y ../lib/verilog -extension v
48
49 run_testbench_simulations
50 check_testbench_simulations -report_status
51
52 exit
```

Example 2

Typically, you insert MemoryBIST and/or boundary scan in the first DFT insertion pass before inserting the hybrid IP (and OCC) in the second DFT insertion pass. The following sample dofile shows a first DFT insertion pass without MemoryBIST or boundary scan, in which you are adding the DFT signal for controller chain mode test.

Example 5-16. Dofile Example for First DFT Pass, Hybrid TK/LBIST Only

```
# Load the design
set_context dft -rtl -design_id rtl1
set_tsdb_output_directory ../tsdb_outdir
set_design_sources -format tcd_memory -y ../rtl -extension memlib
read_verilog ../rtl/piccpu_rtl.v -verbose
read_cell_library ../library/tessent/adk.tcelllib \
    ../rtl/picdram.atpglib
set_current_design piccpu
set_design_level physical_block

# Specify the clocks
add_clocks 0 clk -period 100ns
add_clocks 0 ramclk -period 100ns

# Add DFT signal for controller chain mode test
register_static_dft_signal_names HTKLB_CCM_EN -usage global_dft_control \
    default_value_in_all_test 0 -reset_value 0
add_dft_signal HTKLB_CCM_EN -create_with_tdr

check_design_rules

# Create DFT specification
set_spec [create_dft_specification]
set_config_value -in $spec user_rtl_cells on

process_dft_specification

extract_icl

# Generate patterns
create_patterns_specification

# Create patterns specification and create simulation test benches
process_patterns_specification

# Run and check testbench simulations
set_simulation_library_sources -v ../library/verilog/adk.v \
    -v ../library/memory/ram.v -y ../library/verilog -extensionv
run_testbench_simulations -simulator_option +nowarnTSCALEexit
```

Second DFT Insertion Pass: Hybrid TK/LBIST

In the second DFT insertion pass with the hybrid TK/LBIST flow, insert the EDT, OCC, and LogicBIST instruments.

Note



The line numbers used in this procedure refer to the command flow dofile in [Example 5-17](#) on page 208 unless otherwise noted.

Procedure

1. [Load the Design](#). (See lines 1-6.)
2. [Specify and Verify the DFT Requirements](#). (See lines 8-28.) The following additional step is important for the hybrid TK/LBIST flow.

Note



Hybrid TK/LBIST has additional DFT requirements for the clock signals.

- a. Lines 23-24. Create the LogicBIST test point and X-bounding signals with a TDR.
3. Add a core instance for the MemoryBIST mini-OCC for LogicBIST test. (See lines 30-31.) This is required when you have inserted MemoryBIST in the first DFT insertion pass.

```
add_core_instances -instances [get_instances  
*_tessent_sib_sti_inst]
```

4. [Create the DFT Specification](#). (See lines 38-40.) The following additional step is important for the hybrid TK/LBIST flow.
- a. Include a SIB for LogicBIST by specifying lbist in the create_dft_specification command.

```
create_dft_specification -sri_sib_list {edt occ lbist}
```

5. [Generate the EDT and OCC Hardware](#), plus the LogicBIST hardware. (See lines 47-52.) The following steps are important for inserting the LogicBIST hardware. See “[Example 2: DFT Specification Example for Second DFT Insertion Pass with Hybrid TK/LBIST](#)” on page 210.
- a. For OCC, set the static clock control to external and the capture trigger to capture_en.

When static_clock_control is set to external, the OCC will have an N-bit input (where N equals the number of shift register bits), which will be driven by the NcpIndexDecoder. This is what gives the OCC programmability for LogicBIST.

When `capture_trigger` is set to `capture_en`, the OCC is capable of handling a free-running slow clock. In LogicBIST applications, the slow clock is a free-running clock (whereas for ATPG this comes from a top-level tester-controlled clock).

For details about OCC for the hybrid TK/LBIST flow, refer to “[Tessent OCC TK/LBIST Flow](#)” in the *Hybrid TK/LBIST Flow User’s Manual*.

- b. Specify a [LogicBist](#) wrapper that contains both a Controller wrapper and an `NcpIndexDecoder` wrapper. This wrapper causes Tessent Shell to automatically convert the EDT controller into a hybrid controller for the hybrid TK/LBIST flow.

In the Controller/Connections wrapper, you must specify the `controller_chain_enable` property if you are using a TDR to control CCM. Specify the full path to the pin or port name.

The `NcpIndexDecoder` wrapper specifies the name capture procedures (NCPs) for LogicBIST test. For details, refer to “[NCP Index Decoder](#)” in the *Hybrid TK/LBIST Flow User’s Manual*.

- c. In the EDT/Controller wrapper, define the [LogicBistOptions](#) if you are not using the default values.

When you specify the LogicBIST wrapper, the tool adds a `LogicBistOptions` wrapper to the EDT controller automatically populated with default values.

The `misr_input_ratio` property provides a way to specify the MISR size. The automatic (default) ratio results in the lowest hardware with a small MISR.

You can control how many chains are masked by a single mask register bit using the `chain_mask_register_ratio` property. By default, the ratio is 1:1; there are as many bits in the chain mask register as there are number of scan chains.

Specify the low-power shift options specifically for hybrid IP usage separately from the low-power shift options for the EDT controller.

6. [Extract the ICL Module Description](#). (See line 54.)
7. [Generate ICL Patterns and Run Simulation](#). (See lines 56-63.)

Examples

Example 1

The following dofile example shows a typical command flow. The highlighted command lines are unique to the process for inserting Tessent Shell LogicBIST and hybrid IP instruments in a two-pass DFT insertion process. In this example, the dofile reads in the DFT specification that defines the instruments to be inserted.

Example 5-17. Dofile Flow for the Second DFT Insertion Pass: Hybrid TK/LBIST

```
1 set_context dft -rtl -design_id rtl2
2 set_tsdb_output_directory ../tsdb_outdir
```



```

3 read_design piccpu -design_id rtl1
4 read_cell_library ../lib/tessent/adk.tcelllib \
5   ../lib/tessent/picdram.atpglib
6 set_current_design piccpu
7
8 # Add the DFT signals
9 # For scan test MemoryBIST-related logic
10 # Define tck_occ_en to access mini-OCC during LogicBIST
11 add_dft_signal ltest_en memory_bypass_en tck_occ_en
12
13 # Scan test shift/capture and edt clocks are driven from the
14 # test clock (saves a port)
15 add_dft_signal scan_en edt_update -source_node {scan_en edt_update }
16 add_dft_signal test_clock -source_node test_clock
17 add_dft_signal edt_clock shift_capture_clock -create_from_other_signals
18 # Alternatively, they can be directly driven from a primary port as
19 # follows
20 # add_dft_signal edt_clock shift_capture_clock -source_node {edt_clock
21 # shift_capture_clock}
22
23 # Required DFT signals for hybrid TK/LBIST
24 add_dft_signals control_test_point_en observe_test_point_en X_bounding_en
25
26 add_dft_signals int_ltest_en ext_ltest_en
27
28 set_dft_specification_requirements -logic_test on
29
30 # Add the MemoryBIST mini-OCC for LogicBIST test
31 add_core_instances -instances [get_instances *_tessent_sib_sti_inst]
32
33 # add_clock clk -period 100ns
34 # Need to define it because of ICL clock port tracing
35
36 check_design_rules
37
38 # Create DFT specification
39 # Populated later in this dofile
40 set_spec [create_dft_specification -sri_sib_list {occ edt lbist } ]
41
42 # You can set this property if there are no library cells
43 # set_config_value -in $spec use_rtl_cells on
44
45 report_config_data $spec
46
47 # Read in the DFT specification data
48 # This test case reads in the DFT specification from a separate file
49 # This file is illustrated in the next example
50 read_config_data logic_instruments.dftspec -in_wrapper $spec -replace
51
52 process_dft_specification
53
54 extract_icl
55
56 create_patterns_specification
57 process_patterns_specification
58
59 set_simulation_library_sources -v {../lib/verilog/adk.v} \
60   -y ../lib/verilog -extension v

```

```

61
62 run_testbench_simulations
63 check_testbench_simulations -report_status
64
65 exit

```

Example 2: DFT Specification Example for Second DFT Insertion Pass with Hybrid TK/LBIST

The following example illustrates a DFT specification that includes the wrappers for the LogicBIST controller and additional LogicBIST options for the EDT controller.

```

1  Occ {
2    ihtag_host_interface: Sib(occ);
3    capture_trigger: capture_en;
4    static_clock_control: both;
5    Controller(clk) {
6      clock_intercept_node: clk;
7    }
8    Controller(ramclk) {
9      clock_intercept_node: ramclk;
10   }
11 }
12
13 # Include the LogicBIST controller
14 # Example LogicBIST only; modify for your design requirements
15 LogicBist {
16   ihtag_host_interface : Sib(lbist);
17   Controller(c0) {
18     burn_in : on ;
19     pre_post_shift_dead_cycles : 8 ;
20     SingleChainForDiagnosis { Present : on ; }
21     ShiftCycles {max: 40; hardware_default : 1024;}
22     CaptureCycles {max: 4;}
23 # Hardware default is max
24     PatternCount {max: 10000; hardware_default : 1024;}
25 # Hardware default is 0
26     WarmupPatternCount { max : 512;}
27     ControllerChain {
28       present : on;
29       clock : tck;
30     }
31     Connections {
32       shift_clock_src: clk;
33       controller_chain_enable : piccpu_rtl_tessent_tdr_sri_ctrl_inst/\
34         HTKLB_CCM_EN ;
35     }
36   }
37   NcpIndexDecoder{
38     Ncp(clk_occ_ncp) {
39       cycle(0): OCC(clk);
40       cycle(1): OCC(clk);
41     }
42     Ncp(ramclk_occ_ncp) {
43       cycle(0): OCC(ramclk);
44       cycle(1): OCC(ramclk);
45     }
46 # References to piccpu_rtl_tessent_sib_1 only applicable when

```

```

47 # inserting MemoryBIST
48     Ncp(ALL_occ_ncp) {
49         cycle(0): OCC(clk) , OCC(ramclk) , piccpu_rtl_tessent_sib_1;
50     }
51     Ncp(sti_occ_ncp) {
52         cycle(0): piccpu_rtl_tessent_sib_1 ;
53         cycle(1): piccpu_rtl_tessent_sib_1 ;
54     }
55 }
56 }
57
58 # Example EDT only; modify for your design requirements
59 # Note the additional LogicBIST options for the hybrid TK/LBIST flow
60 EDT {
61     ijttag_host_interface : Sib(edt);
62     Controller (c0) {
63         longest_chain_range : 20, 60;
64         scan_chain_count : 10;
65         input_channel_count : 1;
66         output_channel_count : 1;
67         ShiftPowerOptions {
68             present : on ;
69             min_switching_threshold_percentage : 15 ;
70         }
71         LogicBistOptions {
72             misr_input_ratio : 1 ;
73             chain_mask_register_ratio : 1 ;
74             ShiftPowerOptions {
75                 present : on ;
76                 default_operation : disabled ;
77                 SwitchingThresholdPercentage { min : 25 ; }
78             }
79         }
80     }
81 }

```

Example 3: DFT Signals Required for the Hybrid TK/LBIST Flow

The following dofile snippet shows the DFT signals required if you are performing the hybrid TK/LBIST flow only (without MemoryBIST or boundary scan in the first DFT insertion pass).

```

# For logic test
add_dft_signals ltest_en -create_with_tdr
add_dft_signals scan_en edt_update -source_node \
    { scan_en edt_update }
# You can create edt_clock and shift_clock from a test clock
add_dft_signals test_clock -source_node { scan_test_clock }
add_dft_signals edt_clock shift_capture_clock -create_from_other_signals

add_dft_signals int_ltest_en ext_ltest_en

# Required for TK/LBIST IP
add_dft_signals observe_test_point_en -create_with_tdr
add_dft_signals control_test_point_en -create_with_tdr
add_dft_signals x_bounding_en -create_with_tdr

```

Perform Test Point Insertion

The two-pass DFT insertion flow with hybrid TK/LBIST includes a step for test point insertion prior to performing scan chain insertion. Inserting test points increases the testability of a design by improving controllability and observability during scan testing.

Note



Prior to test point insertion, perform synthesis as described in section “[Perform Synthesis](#)” on page 122.”

The tool inserts observe points that enable the tool to observe test responses during scan cycles and control points that activate and receive pseudo-random values during built-in self test. For details, refer to “[Test Points for LBIST Test Coverage Improvement](#)” in the *Tessent Scan and ATPG User’s Manual*.

Before inserting the test points, specify a DFT signal that will enable X-bounding signals. X-bounding is the process of preventing signals originating from X-generating nets (from non-scan cells, black boxes, and primary inputs) from reaching the scan cells. The tool inserts a MUX with an X-bounding enable signal that is controlled by a top-level pin. X-bounding ensures that only valid binary values propagate through the scan cells during test. For details, refer to “[X-Bounding](#)” in the *Hybrid TK/LBIST Flow User’s Manual*.

Procedure

1. Specify the following command to set the DFT context for test point insertion:

```
set_context dft -test_point -no_rtl -design_id gate1
```

For test point insertion, you must use a gate-level netlist. Ensure that you specify a design ID with a unique name from the design ID for scan chain insertion that you will perform after test point insertion.

2. Load the cell libraries.

```
read_cell_library ../lib/tessent/adk.tcelllib ../rtl/  
picdram.atpglib
```

3. Specify the same output directory that you used in the first and second DFT passes.

```
set_tsdb_output_directory ../tsdb_outdir
```

4. Load the synthesized netlist. For example:

```
read_verilog ../03.synthesis/piccpu.vg
```

5. Load the design data for the DFT hardware you previously inserted. For example:

```
read_design piccpu -design_id rtl2 -no_hdl -verbose
```

6. Set the maximum number of test points you want to add. For example:

```
set_test_point_analysis_options -total_number 50
```

Typically, the maximum number of test points should be 1%-2% of the number of flops in the design.

7. Set the test point-related insertion options.

```
set_test_point_insertion_options -observe_point_share 5
```

The control_test_point_en and observe_test_point_en enable signals are required for test point insertion, and they are automatically inferred from the control_test_point_en and observe_test_point_en DFT signals you previously added.

8. Specify the type of test points you want to insert into the design.

```
set_test_point_types { lbist_test_coverage edt_pattern_count }
```

Specify both LogicBIST test coverage and EDT pattern count test point types so that the tool will generate test points to reduce pattern count and improve random pattern testability.

9. Specify to insert test logic on the set and reset signals to make them controllable when you insert scan chains.

```
set_test_logic -set on -reset on
```

This command increases test coverage.

10. Prohibit test point insertion for Tessent-inserted scan resource instruments (SRIs)—EDT, OCC, and LogicBIST.

```
add_notest_point [ get_instance *tessent* ]
```

You can only insert test points into scan tested instruments (STIs).

11. Analyze and optimize the test points.

```
analyze_test_points
```

Tessent Shell returns a log file that lists the test points that will be inserted into the tool. As desired, you can write out a test point dofile that lists the test point locations. You can use this dofile to edit the test points you want to insert.

```
write_test_point_dofile tpi.dofile -all -replace
```

12. Insert the test point and scan logic.

```
insert_test_logic
```

To generate a script to use with third-party test point insertion tools, use the following command to target the insertion script for Design Compiler.

```
insert_test_logic -write_in_tsdb off -write_insertion_script \  
testpoint_dc.tcl -insertion dc -replace
```

Examples

The following dofile shows a command flow for test point insertion.

Example 5-18. Dofile Example for Test Point Insertion

```
set_context dft -test_point -no_rtl -design_id gate  
read_cell_library ../lib/tessent/adk.tcelllib ../rtl/picdram.atpglib  
set_tsdb_output_directory ../tsdb_outdir  
read_verilog ../03.synthesis/piccpu.vg  
read_design piccpu -design_id rtl2 -no_hdl -verbose  
  
set_current_design piccpu  
add_input_constraint reset -C0  
  
set_test_point_analysis_options -total_number 50  
  
# Following command inferred when X-bounding enable DFT signal was added  
# -xbounding_enable [ get_dft_signal x_bounding_en ]  
set_test_point_insertion_options -observe_point_share 5  
set_test_point_types { lbist_test_coverage edt_pattern_count }  
set_test_logic -set on -reset on  
# no test points in Tessent-inserted IPs  
add_notest_point [ get_instance *tessent* ]  
  
set_system_mode analysis  
  
analyze_test_points  
  
insert_test_logic  
report_test_logic  
  
exit
```

Perform Scan Chain Insertion

Scan chain insertion for the hybrid TK/LBIST flow includes additional commands for connecting the scan enable signal, performing DRC, specifying non-scannable design objects, and X-bounding.

Note



The line numbers used in this procedure refer to the command flow dofile in [Example 5-19](#) on page 215.

Procedure

1. Load the design data and set the design parameters. (See lines 1-6.) Ensure that you specify a unique design ID name from the name you used for test point insertion.

2. Set DRC handling to issue warnings for E9 and E11 DRC checks. (See lines 8-10.)

```
set_drc_handling E09 W
set_drc_handling E11 W
```

These DRC checks look for possible bus contention issues. By default, Tessent Shell ignores these rules. Set these checks to issue warnings so that the X-bounding algorithm will check them and fix any X-source contention issues.

3. Specify the pins/ports to exclude from X-bounding. (See lines 12-13.) For example:

```
set_xbounding_options -exclude { reset }
```

Exclude the pins/ports that are guaranteed to have a known value during fault simulation and will never propagate an X value to the MISR. The tool will not insert X-bounding muxes at the specified pins/ports, or at the logic that is driven by these pins.

4. Run X-source analysis with the `analyze_xbounding` command to identify memory elements that might capture an unknown during LogicBIST. (See lines 17-19.)
5. Perform wrapper analysis and insertion. (See lines 21-34.)

For details, refer to “[Perform Scan Chain Insertion](#)” on page 141.

6. Connect the scan chains to the EDT block, analyze the scan chains, and insert the scan chains. (See lines 36-50.)

For details, refer to “[Perform Scan Chain Insertion](#)” on page 141.

Examples

The following dofile example shows a typical command flow. The highlighted command lines are unique to the process for inserting Tessent Shell LogicBIST and hybrid IP instruments in a two-pass DFT insertion process.

Example 5-19. Dofile Example for Scan Chain Insertion: Hybrid TK/LBIST

```
1 set_context dft -scan -design_id gate2
2 read_cell_library ../lib/tessent/adk.tcelllib \
3   ../lib/tessent/picdram.atpglib
4 set_tsdb_output_directory ../tsdb_outdir
5 # Reading in test point-inserted design
6 read_design piccpu -design_identifier gate1 -verbose
7
8 # Enable DRCs for X-bounding
9 set_drc_handling E09 w
10 set_drc_handling E11 w
11
12 # Set x-bounding options
13 set_xbounding_options -exclude {reset}
14
15 set_system_mode analysis
16
17 # Run X-source analysis
18 analyze_xbounding
```

```

19 report_xbounding -verbose -ignored_x_sources on
20
21 ## Perform wrapper analysis and insertion
22 # Exclude the edt_channel in and out ports from wrapper analysis
23 # The ijtag_* edt_update ports are automatically excluded
24 set_wrapper_analysis_options -exclude_ports \
25     [ get_ports {*_edt_channels_*} ]
26
27 # Add a new wrapper dedicated cell on reset
28 set_dedicated_wrapper_cell_options on -ports { reset }
29 set_wrapper_analysis_options -input_fanout_flop_threshold 100 \
30     -output_fanin_flop_threshold 40
31
32 # Perform wrapper cell analysis
33 analyze_wrapper_cells
34 report_wrapper_cells -Verbose
35
36 # Find the edt_instance
37 # Instance of module *edt_lbist_c0
38 set edt_instance [get_instances -of_icl_instances [get_icl_instances \
39     -filter tessent_instrument_type==mentor::edt]]
40
41 # Connect scan chains to the EDT signals
42 add_scan_mode int_mode -type internal -edt_instances $edt_instance
43 add_scan_mode ext_mode -type external -chain_count 2
44 analyze_scan_chains
45
46 analyze_scan_chains
47 report_scan_chains
48
49 # Insert scan chains and write the scan-inserted design into the TSDB
50 insert_test_logic
51
52 report_scan_chains
53 report_scan_cells
54
55 exit

```

Perform ATPG Pattern Generation: Hybrid TK/LBIST

ATPG pattern generation for the hybrid TK/LBIST flow includes a process for generating controller chain mode patterns to test the TK/LBIST logic itself.

For details, refer to “[Performing Pattern Generation for CCM in the TSDB Flow](#)” in the *Hybrid TK/LBIST Flow User’s Manual*.

Note



Perform EDT pattern generation as described in “[Perform ATPG Pattern Generation](#)” on page 124.

Note

 The line numbers used in this procedure refer to the command flow dofile in [Example 5-20](#) on page 217.

Procedure

1. [Load the Design](#). (See lines 1-12.)
2. Enable CCM. (See lines 16-17.)

```
set_static_dft_signal_values HTKLB_CCM_EN 1
```

You had added a DFT signal for ccm_en during IP creation, which is now being configured. If this is a port (default), then this command is not required.

3. Define the scan chain for CCM. (See lines 19-22.)
4. Specify tck or edt_clock for CCM, depending on which you are using in your implementation. (See lines 24-25.)
5. Define the pin constraints. (See lines 33-37.)
You must disable core clock and reset activity. If ccm_en was not added as a DFT signal, then also include a pin constraint for it (enabled).
6. For retargeting, specify to pulse the CCM clock during shift. (See line 39.) The following command is only required when you use edt_clock for CCM and edt_clock is a top-level port, or when you use tck for CCM.

```
set_procedure_retargeting_options -pulse_during_shift edt_clock
```

The tool automatically generates a test procedure file that configures the scan_en and shift_capture_clock DFT signals. If the edt_clock is derived from test_clock, do not specify the set_procedure_retargeting_options command.

7. Specify the set_current_mode command with a unique name to indicate that you are generating CCM patterns. (See line 41.) For example:

```
set_current_mode ccm_stuck
```

8. Create and write the patterns. (See lines 43-60.)

Examples

The following example generates CCM ATPG patterns. The highlighted statements illustrate additional considerations for CCM.

Example 5-20. Dofile Example for ATPG with Controller Chain Mode: Hybrid TK/LBIST

```
1  ### DESIGN VARIABLES
2  set DesignName  piccpu
```

```
3 set DesignLevel physical_block
4
5 set_context patterns -scan
6 set_tsdb_output_directory ../tsdb_outdir
7 read_cell_library ../lib/tessent/adk.tcelllib \
8   ../lib/tessent/picdram.atpglib
9 set_design_source -format tcd_memory -y ../lib/tessent -extension memlib
10
11 # Read in the scan-inserted design
12 read_design $DesignName -design_id gate2
13
14 report_dft_signals
15
16 # Enable CCM
17 set_static_dft_signal_values HTKLB_CCM_EN 1
18
19 add_scan_groups grp1
20 # These are the required scan chains for CCM
21 add_scan_chains ccm_chain grp1 control_chain_scan_in \
22   control_chain_scan_out
23
24 # Add edt_clock or tck as the primary clock source for CCM
25 add_clock 0 tck
26
27 # Specify clocks driven by primary-input ports; this is optional
28 add_clocks 0 clk
29 add_clocks 0 ramclk
30 add_clocks 0 reset
31 add_clocks 0 test_clock
32
33 # Define the pin constraints
34 add_input_constraints clk -c0
35 add_input_constraints ramclk -c0
36 add_input_constraints reset -c0
37 add_input_constraints test_clock -c0
38
39 set_procedure_retargeting_options -pulse_during_shift edt_clock
40
41 set_current_mode ccm_stuck
42
43 set_system_mode analysis
44 add_fault -module [ get_module *tessent* ]
45 report_clocks
46
47 create_patterns
48 report_statistics -detailed_analysis
49
50 report_statistics -instance piccpu_rtl2_tessent_lbist
51 report_statistics -instance \
52   piccpu_rtl2_tessent_lbist_ncp_index_decoder_inst
53 report_statistics -instance piccpu_rtl2_tessent_single_chain_mode_logic
54 report_statistics -instance piccpu_rtl2_tessent_edt_lbist_c0_inst
55
56 write_tsdb_data -replace
57 write_patterns ${DesignName}_ccm_stuck_parallel.v -verilog -parallel \
58   -replace -parameter_list {SIM_KEEP_PATH 1}
59 write_patterns ${DesignName}_ccm_stuck_serial.v -verilog -serial \
60   -replace -parameter_list {SIM_KEEP_PATH 1}
```

```
61
62 exit
```

Perform LogicBIST Fault Simulation

Perform LogicBIST fault simulation on the scan-inserted, gate-level design. LogicBIST fault simulation generates pseudo-random, parallel pattern sets.

Refer to “[Parallel Versus Serial Patterns](#)” in the *Tessent Scan and ATPG User’s Manual* for more information.

Note

 The line numbers used in this procedure refer to the command flow dofile in [Example 5-21](#) on page 220.

Procedure

1. [Load the Design](#). (See lines 1-5.)
2. Set the current test mode to a unique name for the new pattern set you will be creating that will test the hybrid IP. (See line 7.)
3. Import the core’s internal mode data. (See line 9.)

Tessent Shell imports the scan-inserted design data for the EDT and OCC logic. For LogicBIST fault simulation, the tool requires EDT for PRPG/compactor/MISR configuration and chain tracing, and OCC for the clocks. Using the [import_scan_mode](#) command assumes that you used Tessent Scan to perform scan chain stitching.

For LogicBIST fault simulation, only the internal mode data is valid.

4. Add the hybrid TK/LBIST core instances. (See lines 11-12.) For example:

```
add_core_instances -instance piccpu_rtl2_tessent_lbist
```

For the hybrid TK/LBIST flow, you must explicitly add the LogicBIST controller core.

5. Specify the order of the NCPs. (See lines 15-18.)

The IP generation process produces a dofile that is stored within the *_lbist_ncp_index_decoder.instrument directory in the TSDB. This file contains the list of NCPs in the order they were specified for IP creation. Specify the full pathname to this dofile.

6. Specify the capture procedure names with the [set_lbist_controller_options](#) command. (See lines 20-22.)

Be sure to include the names of all the NCPs that will be used in LogicBIST mode and their active percentages.

7. Define the DFT signals. (See lines 27-41.) In addition to enabling the logic test controls `ltest_en` and `int_ltest_en`, for the hybrid TK/LBIST flow, you must specify the following signals:

```
set_static_dft_signal_values control_test_point_en 1
set_static_dft_signal_values observe_test_point_en 1
set_static_dft_signal_values xbounding_en 1
set_static_dft_signal_values tck_occ_en 1
set_static_dft_signal_values HTKLB_CCM_EN 0
```

8. Read in the NCP testproc file. (See lines 47-50.)

The NCP testproc file defines how Tessent Shell should pulse the clocks for simulation. The IP generation process produces a testproc file that is stored within the `*_lbist_ncp_index_decoder.instrument` directory in the TSDB. Specify the full pathname to this file.

9. Set the PLL external capture clocks if your design has a PLL that is a driving clock but the PLL itself is driven by external clocks. (See lines 52-55.)

This indicates the number of cycles the NCPs take with respect to the LogicBIST controller clock, as specified with the `-fixed_cycles` switch.

10. Specify the maximum number of pseudo-random patterns you want the tool to simulate. (See line 57.)
11. Set the pattern source to LogicBIST and execute fault simulation. (See line 60.)
12. Write out the parallel test bench and save all the data into the TSDB. (See lines 62-66.)

Examples

The following dofile example shows a typical command flow as detailed in the procedure listed above. Highlighted text illustrates additional considerations for the hybrid TK/LBIST flow.

Example 5-21. Dofile Example for LogicBIST Fault Simulation

```
1 set_context pattern -scan -design_id gate2
2 set_tsdb_output_directory ../tsdb_outdir
3 read_cell_library ../lib/tessent/adk.tcelllib ../rtl/picdram.atpglib
4 read_design piccpu
5 set_current_design piccpu
6
7 set_current_mode lbist
8
9 import_scan_mode int_mode
10
11 # Explicitly add the LogicBIST controller core
12 add_core_instances -instance piccpu rtl2 tessent_lbist
13 # EDT and OCC are loaded with the int_mode imported scan mode
14
15 # Read the NCP order
16 dofile ../tsdb_outdir/instruments/ \
17     piccpu_rtl2_lbist_ncp_index_decoder.instrument/ \
```

```

18  piccpu_rtl2_tessent_lbist_ncp_index_decoder.dofile
19
20 # Define the number of patterns per NCP capture window
21 set_lbist_controller_options -capture_procedure {clk_occ_ncp 70 \
22     ramclk_occ_ncp 10 ALL_occ_ncp 10 sti_occ_ncp 10}
23
24 add_input_constraint test_clock -c0
25 add_nofault [get_instance *tessent*]
26
27 # Set the DFT signals
28 set_static_dft_signal_values ltest_en 1
29 # The following DFT signal is inferred by ltest_en 1 if have memory
30 # set_static_dft_signal_value memory_bypass_en 1
31
32 set_static_dft_signal_values control_test_point_en 1
33 set_static_dft_signal_values observe_test_point_en 1
34 set_static_dft_signal_values x_bounding_en 1
35
36 set_static_dft_signal_values int_mode 1
37 # The following DFT signal is inferred by int_mode 1
38 # set_static_dft_signal_values int_ltest_en 1
39
40 set_static_dft_signal_values tck_occ_en 1
41 set_static_dft_signal_values HTKLB_CCM_EN 0
42
43 report_static_dft_signal_settings
44
45 set_system_mode analysis
46
47 # Read the NCP testproc file
48 read_procfile ../tsdb_outdir/instruments/ \
49     piccpu_rtl2_lbist_ncp_index_decoder.instrument/ \
50     piccpu_rtl2_tessent_lbist_ncp_index_decoder.testproc
51
52 # Specify the number of cycles the NCPs take with respect to the LogicBIST
53 # controller clock
54 # This ensures that the tool runs the testproc correctly
55 set_external_capture_options -fixed_cycles 4
56
57 set_random_patterns 1000
58 add_faults -all
59
60 simulate_patterns -source bist -store_patterns all
61
62 # Write the parallel pattern Verilog testbench
63 write_patterns piccpu_lbist_parallel.v -verilog -parallel -replace \
64     -parameter_list {SIM_KEEP_PATH 1}
65 # Save the TCD, PatternDB, flat model, and fault list to the TSDB
66 write_tsdb_data -replace
67
68 exit

```

Perform IJTAG Pattern Generation

Perform IJTAG pattern retargeting from the extracted ICL module description for the design. Tessent Shell translates (or retargets) the PDL commands so that you can control the LogicBIST controller through the TAP controller/IJTAG infrastructure.

Note



Do not confuse IJTAG retargeting with ATPG (or scan) pattern retargeting as described in “[Hierarchical DFT Terminology](#)” on page 129.

Procedure

1. [Load the Design](#). (See lines 1-5.) The following step is important for the hybrid TK/LBIST flow:
 - a. Set the context to patterns -ijtag and set the design ID to the name of the scan-inserted, gate-level netlist generated during scan chain insertion.

When you specify the -ijtag switch, Tessent Shell automatically accesses the ICL module description for the current design, which enables IJTAG retargeting mode.
2. Define clocks and constraints. (See lines 7-10.)
3. Generate and validate the IJTAG patterns for the design. (See lines 14-16.)
4. Run and check the test bench simulations. (See lines 18-23.) The following step is important for the hybrid TK/LBIST flow:
 - a. Specify the simulator option to keep all the logic gates for simulation. The following example applies to Questa:

```
run_testbench_simulations -simulator_options \  
    { -voptargs="+acc" }
```

For correct pattern simulation, use the applicable simulator option to ensure that the necessary design logic is not optimized away during elaboration.

Tessent Shell simulates the logic test operations only, which means the test bench does not connect all the pins in the design. The tool issues warnings for the unconnected pins. To filter these warnings out, you can use the `run_testbench_simulations -simulation_option +nowarnTSCALE` option.

Examples

The following dofile shows a command flow to generate IJTAG patterns for LogicBIST. Highlighted text illustrates additional considerations for the hybrid TK/LBIST flow.

```
1  set_context patterns -ijtag -design_id gate2  
2  read_cell_library ../lib/tessent/adk.tcelllib ../rtl/picdram.atpglib  
3  set_tsdb_output_directory ../tsdb_outdir  
4  read_design piccpu  
5  set_current_design piccpu
```

```
6
7 # This is the clock driving the internal PLL
8 add_clocks 0 clk -period 100ns
9 # Scan enable has to be tied to low
10 add_input_constraint scan_en -c0
11
12 set_system_mode analysis
13
14 read_config_data ./lbist_mbist_pattern.patspec
15
16 process_patterns_specification
17
18 set_simulation_library_sources -v { ../lib/verilog/adk.v \
19   ../lib/verilog/picdram.v ../lib/verilog/SYNC_1R1W_256x16.v }
20
21 run_testbench_simulation -simulator_options { -voptargs="+acc" \
22   -quiet} -wait
23 check_testbench_simulations -report_status
24
25 exit
```

Running Multi-Load ATPG on Memories for Wrapped Cores with Built-In Self Repair

If your design has wrapped cores that include repairable memories and you want to test the logic around the memories at speed, then you use multi-load ATPG patterns.

Overview of Multi-Load ATPG on Memories for Wrapped Cores With Built-in Self Repair 224

Performing Multi-Load ATPG Pattern Generation. 225

Performing Multi-Load Top-Level ATPG Pattern Retargeting 228

Overview of Multi-Load ATPG on Memories for Wrapped Cores With Built-in Self Repair

When generating ATPG multi-load patterns for memories with Built-In Self Repair (BISR), the memories inside the wrapped cores are tested first. If any memories need to be repaired due to defects that are present inside the memory, then the BISR registers are programmed such that the contents of these registers vary based on the repair solution that is stored in the fuses at the parent level.

To generate multi-load ATPG patterns, the BISR registers need to be excluded from retargetable ATPG patterns that you created for the wrapped core. At the wrapped core level, a test_setup_iCall will reset the BISR registers, resulting in all spare elements being unallocated before ATPG is executed. Only the memory control ports and data ports participate in the ATPG patterns; the memory repair ports are excluded. If memory defects are detected and repaired via the BISR registers, the retargeted ATPG patterns will also pass.

You must provide an ATPG model of the eFuse during pattern generation in order to meet the E14 design rule requirements. If such a model is not available, you can instead provide an input constraint on the fuseValue output pin of the eFuse interface module. To do this, create a pseudo-port associated with the fuseValue output pin of the fuse box interface instance. Then, add a constant 0 input constraint on the “fuseValue” pseudo-port, as shown in the following example:

```
add_primary_inputs chip_rtl_tessent_mbisr_controller_inst/ \
    chip_rtl_tessent_mbisr_generic_fusebox_interface_instance/fuseValue \
    -pseudo_port_name fuseValue
add_input_constraints -c0 fuseValue
```

For more details, refer to the [add_primary_inputs](#) and [add_input_constraints](#) commands.

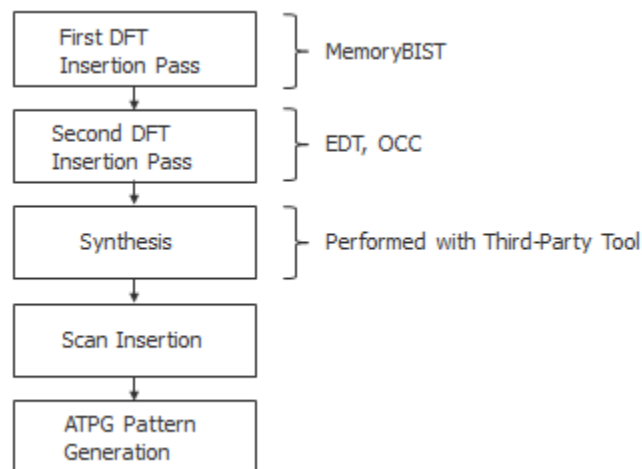
Performing Multi-Load ATPG Pattern Generation

When you need to generate multi-load ATPG patterns when the memory is not bypassed but used during ATPG there is no difference in the flow steps used for inserting DFT. You use this procedure to bypass the memories.

The DFT insertion at the wrapped core level for memories with BISRs is similar to the two-pass pre-scan DFT insertion process for wrapped cores with the primary difference being the ATPG pattern generation. There are no changes to creating the graybox model, and running the wrapped core in external mode for stuck and transition patterns by bypassing the memories. There are no changes for running the wrapped core in internal mode for stuck and transition patterns by bypassing the memories.

This discussion assumes that you are familiar with the two-pass pre-scan DFT insertion flow as described in “[RTL and Scan DFT Insertion Flow for Physical Blocks](#)” on page 133, especially as related to performing ATPG pattern generation. [Figure 5-32](#) shows this flow.

Figure 5-32. Two-Pass Insertion Flow for RTL, Wrapped Cores



Note that the `init_bisr_chains` iProc contains Tcl code you can use to initialize the BISR registers at any level (core or chip-level) as follows:

- At the wrapped core level, the iProc performs an asynchronous clear of the BISR registers.
- At the chip-level, the iProc will initiate a FuseBox controller power-up emulation.

The power-up emulation will clear the BISR registers if the fuse box has not been programmed, or will initialize the BISR registers with the repair data if memory repair information was programmed in the fuse box. See “[Creating and Inserting the BISR Controller](#)” in *Tessent MemoryBIST User’s Manual*.

Note

 The line numbers used in this procedure refer to the command flow dofile in [Example 5-22](#) on page 227.

Prerequisites

- You have performed the “[RTL and Scan DFT Insertion Flow for Physical Blocks](#)” on page 133 up to the ATPG pattern generation step.

Procedure

1. Load the Tessent Cell Library for the memory. (See lines 1-9.)
2. Load the design. (See lines 11-13.)
3. Set the DFT Signal memory_bypass to 0, so memory is not bypassed. (See lines 24-26.)
4. Load the PDL for the Memory BISR chains, which has an iProc (init_bisr_chains) that is called with [set_test_setup_icall](#) -non_retargetable. (See lines 28-35.)

Note

 Note that the init_bisr_chains iProc contains Tcl code you can use to initialize the BISR registers at any level (core or chip-level) as follows: at the wrapped core level, the iProc performs an asynchronous clear of the BISR registers; and at the chip-level, the iProc will initiate a FuseBox controller power-up emulation.

5. Generate the multi-load ATPG patterns. (See lines 39-40.)
6. Write the design data and patterns to the TSDB. (See lines 42-44.)
7. Write out the Verilog test benches for simulation. (See lines 47-50.)

Results

At the wrapped core level, you inject a fault in the repairable memory, then run the multi-load ATPG Pattern and it should fail. This check shows that if a repairable memory has defect and it is not repaired then when you retarget the multi-load ATPG patterns from the top-level then it will fail.

You must run the memoryBIST inside the wrapped core to determine if the memory is repairable. If it is, then the repairable information from Built-In Redundancy Analysis (BIRA) to BISR must be stored and the repair performed by storing the repairable contents using the fusebox repair solution that you use at the top-level.

Examples

The following example for the repairable memory “MGC_SYNC_1RW_1024x8_C” generates ATPG patterns that will be run by the memory.

Example 5-22. Multi-Load ATPG Pattern Generation for Memories with BISR

```

1  set_context patterns -scan
2
3  # Set the location of the TSDB. Default is the current working directory.
4  set_tsdb_output_directory ../tsdb_outdir
5
6  # Read Tessent Library
7  read_cell_library ../../../../library/standard_cells/tessent/adk.tcelllib
8  # Read the Tessent Cell Library for the memory
9  read_cell_library ../../../../library/memories/MGC_SYNC_1RW_1024x8_C.atpglib
10
11 # Read in the scan inserted netlist/design
12 read_design processor -design_id gate -verbose
13 set_current_design processor
14
15 # Specify the current mode using a different name than what was used
16 # with the add_scan_mode command
17 set_current_mode edt_int_multi_load_atpg -type internal
18 report_dft_signals
19
20 # Extract the scan chains using the internal mode specified during
21 # scan insertion with the add_scan_mode command
22 import_scan_mode int_mode -fast_capture_mode on
23
24 # Memory is used during ATPG, by setting the DFTSignal memory_bypass
25 # to "0"
26 set_static_dft_signal_values memory_bypass_en 0
27
28 # Read in the PDL for the memory BISR
29 source \
30 ../tsdb_outdir/instruments/processor_rtl1_mbisr.instrument/\
31 processor_rtl1_tessent_mbisr.pdl
32
33 # Specify the iProc init_bisr_chains for the memory BISR as
34 # non_retargetable
35 set_test_setup_icall init_bisr_chains -non_retargetable
36
37 report_statistics -detail
38 # Generate patterns
39 create_patterns
40 report_statistics -detail
41
42 # Store TCD, flat_model, fault list and patDB format files in the TSDB
43 # directory
44 write_tsdb_data -replace
45
46 # Write test benches for Verilog simulation
47 write_patterns patterns/processor_multi_load_parallel.v \
48   -verilog -parallel -replace -parameter_list {SIM_KEEP_PATH 1}
49 write_patterns patterns/processor_multi_load_serial.v
50   -verilog -serial -replace -parameter_list {SIM_KEEP_PATH 1}
51 exit

```

At the wrapped core level, you inject a fault in the repairable memory, then run the multi-load ATPG Pattern and it should fail. This check shows that if a repairable memory has defect and it

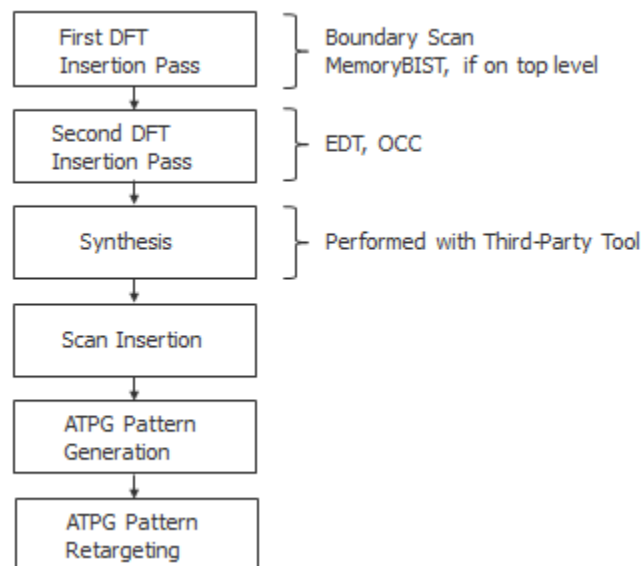
is not repaired then when you retarget the multi-load ATPG patterns from the top-level then it will fail.

Performing Multi-Load Top-Level ATPG Pattern Retargeting

When you have wrapped cores with repairable memories, then at the chip-top level you need to insert a memory BISR controller. This can be done along with the TAP, boundary scan, and MemoryBIST insertion if there are any memories present at the top-level.

This discussion assumes that you are familiar with the RTL and scan DFT insertion flow for the top as described in “[RTL and Scan DFT Insertion Flow for the Top Chip](#)” on page 150, especially as related to performing ATPG pattern retargeting. [Figure 5-33](#) shows this flow.

Figure 5-33. Two-Pass Insertion Flow for RTL, Top Level



For retargeting multi-load ATPG patterns where the memories are not bypassed from a lower level wrapped core that has repairable memories, specify the correct retargeting mode signal. Subsequently, use the [add_core_instances](#) command to load the specific core with the correct ATPG mode that you want to retarget.

Finally, you need to source the PDL of the BISR chain with the [iProc](#) `init_bisr_chains`. This PDL was created when the BISR controller RTL was generated at the chip-level. The `iProc` detects the presence of the fuse box controller and initiates a `PowerUpEmulation`. When the `iProc` is called and no fuse box controller is found, the `iProc` performs an asynchronous clear of the BISR chains using the primary inputs (`bisr_reset` port).

Note

 The line numbers used in this procedure refer to the command flow dofile in [Example 5-23](#) on page 229.

Prerequisites

- You have performed the “[RTL and Scan DFT Insertion Flow for the Top Chip](#)” on page 150 up to the ATPG retargeting step.

Procedure

1. Set the context to pattern retargeting. (See lines 1-2.)
2. Specify the TSDB directory and open the TSDB. (See lines 4-8.)
3. Read the Tessent Cell Library. (See lines 10-11.)
4. Load the Verilog design. (See lines 13-15.)
5. Set the retargeting from the lower-level patterns. (See lines 20-24.)
6. Read the PDL of the memoryBISR chains that has the iProc “init_bisr_chains”. This PDL is different than the one created at the wrapped core level. (See lines 30-34.)
7. Change mode to analysis. (See line 36.)

Examples

The following example retargets the ATPG pattern from the lower level to the chip-level.

Example 5-23. ATPG Pattern Retargeting for Memories with BISR

```

1  # Set the context to retarget ATPG Patterns from lower level child cores
2  set_context pattern -scan_retargeting
3
4  # Point to the TSDB directory
5  set_tsdb_output_directory ../tsdb_outdir
6
7  # Open all the TSDB of the child cores
8  open_tsdb ../../wrapped_cores/processor/tsdb_outdir
9
10 # Read the tessent cell library
11 read_cell_library ../../library/standard_cells/tessent/adk.tcelllib
12
13 # Read the verilog
14 read_design chip_top -design_id gate
15 read_design processor -design_id gate -view graybox -verbose
16
17 set_current_design chip_top
18
19
20 # Retarget Transition patterns from processor
21 set_current_mode retarget1_processor_multi_load
22 set_static_dft_signal_values retargeting1_mode 1
23 add_core_instances -instances {processor_inst1 processor_inst2} \

```

```
24  -core processor -mode edt_int_multi_load_atpg
25
26 report_core_descriptions
27 import_clocks -verbose
28 report_clocks
29
30 # Read the PDL of the MBISR chains that has the iProc "init_bisr_chains"
31 # that needs to be called before running the ATPG pattern.
32 source ../tsdb_outdir/instruments/chip_top_rtl1_mbisr.instrument/\
33 chip_top_rtl1_tessent_mbisr.pdl
34 set_test_setup_icall init_bisr_chains -front
35
36 set_system_mode analysis
37 report_clocks
38
39 # write the TCD file for chip-level in the TSDB outdir
40 write_tsdb_data -replace
41 # Read the patterns to be retargeted
42 read_patterns -module processor -fault_type transition
43 set_external_capture_options -pll_cycles 5 [lindex [get_timeplate_list] 0]
44
45
46 # Write Verilog patterns for simulation
47 write_patterns patterns/processor_edt_multi_load_retargeted.v -verilog \
48 -parallel -replace -begin 0 -end 7 -scan -parameter_list {SIM_KEEP_PATH 1}
49 write_patterns patterns/processor_edt_multi_load_retargeted_serial.v \
50 -verilog -serial -replace -Begin 0 -End 2 \
51 -parameter_list {SIM_KEEP_PATH 1}
52 exit
```

Built in Self Repair (BISR) in Hierarchical Tessent MemoryBIST Flow

You insert BISR into a hierarchical design using the process outlined in this section.

See also the following related section:

- [“First DFT Insertion Pass: Performing Top-Level MemoryBIST and Boundary Scan”](#) on page 153.

This section contains the following topics:

Overview of BISR	231
Performing Block Level BISR Insertion	233
Performing Chip Level BISR Insertion	236
Verifying Block and Chip Level BISR	239

Overview of BISR

The BISR registers are used to control the repair ports of repairable memory.

The BISR logic insertion tasks are as follows:

- Inserting BISR chains in a block
- Connecting a BISR controller to existing BISR chains
- Connecting a BISR controller to an external fuse box
- Connecting a BISR controller to system logic

You perform the first two tasks in a bottom-up, hierarchical design methodology where BISR chains are inserted in lower-level blocks before being connected at the chip top level to the BISR controller. This is the most frequently used method for inserting the repair logic.

Typically, you complete the BISR insertion task with the memoryBIST insertion in the same pass at the block or chip level. If the BISR insertion task is carried out separately from the memory BIST insertion, you must perform this after you have completed all memoryBIST insertion tasks at the block or chip level in a single pass.

This BISR flow omits the additional BISR capabilities of Repair Sharing and Fast BISR Loading, which are discussed in detail in the following sections of the *Tessent MemoryBIST User's Manual*:

- [Repair Sharing](#)
- [Fast BISR Loading](#)

If you use Tessent Scan, do not scan test the BISR hardware if you plan to generate multi-load patterns for logic test. Tessent Scan does this automatically.

In contrast, if you use a third-party scan insertion tool, then the `non_scannable_instance_list` in the `dft_info_dictionary` should be honored and is located in the TSDB in the `dft_inserted_designs` directory—see “[RTL and DFT Insertion Flow with Third-Party Scan](#)” on page 169.

Performing Block Level BISR Insertion

The insertion of BISR chains is automatic if memories instantiated in the design have spare resources described in their memory library file. BISR registers associated to repairable memories are connected together to form scan chains.

The tasks are as follows:

Assigning Memories to Power Domains	233
Controlling the BISR Chain Order	234
Disabling Insertion of BISR Registers	234
Excluding Child Block BISR Chains	234

Assigning Memories to Power Domains

By default, all repairable memories are part of the same power domain, and the tool creates a single BISR chain. If, however, more than one power domain exists, multiple BISR chains are required.

The tool determines the power domain of a memory instance by its `bisr_power_domain_name` attribute, which you specify in the following two ways:

1. Reading a CPF or UPF file corresponding to the design using the [read_cpf](#) or [read_upf](#) command. This is the recommended method.
2. Manually setting the `bisr_power_domain_name` attribute using the [set_attribute_value](#) command.

The following example defines power domains with UPF:

UPF:

```
create_power_domain pd_A
create_power_domain pd_AA -element {mem3 mem4}
create_power_domain pd_AB -element {mem5 mem6}
```

Generate BISR segment order file:

```
BisrSegmentOrderSpecification {  
  PowerDomainGroup(pd_AA) {  
    OrderedElements {  
      mem3;  
      mem4;  
    }  
  }  
  PowerDomainGroup(pd_AB) {  
    OrderedElements {  
      mem5;  
      mem6;  
    }  
  }  
}
```

For additional implementation details, see “[Inserting BISR Chains in a Block](#)” in the *Tessent MemoryBIST User's Manual*.

Controlling the BISR Chain Order

BISR chains are connected according to the content of the `BisrSegmentOrderSpecification` wrapper containing a list of memory instances defining the BISR chain order.

This file is generated automatically when [check_design_rules](#) successfully completes. When a DEF file is provided, the BISR segments are ordered using an algorithm that optimizes the routing based on the memory coordinates. If a DEF file is not provided, the memories are sorted alphabetically within each power domain group.

If you wish to change the generated BISR change order, the you can manually modify the `<design_name>.bisr_segment_order` file and change the order of memory instance names before executing the [process_dft_specification](#) command.

Disabling Insertion of BISR Registers

It is possible to disable the generation of BISR registers for specific memory instances.

You do this by issuing the [set_memory_instance_options](#) command as follows:

```
set_memory_instance_options memory_inst -use_in_memory_bisr_dft_specification off
```

This is option is rarely used.

Excluding Child Block BISR Chains

When you do not implement memory repair in a parent design but integrate sub-blocks or physical blocks that already contain BISR chains, you must not connect or use these chains, and you must properly tied the chains off.

Note



This is not a typical use model and is rarely implemented.

If needed, refer to “[Inserting BISR Chains in a Block](#)” and “[Excluding Child Block BISR Chains](#)” in the *Tessent MemoryBIST User's Manual* for more implementation details.

Performing Chip Level BISR Insertion

As with the block level, the insertion of BISR chains is automatic if memories instantiated in the design at the chip level have spare resources.

There are also additional tasks that you must do when inserting chip level BISR as follows:

Choosing a Functional Repair Clock	236
Connecting BISR controller to Existing BISR Chains	236
Connecting BISR controller to an External Fuse Box	237
Connecting BISR Controller to System Logic	238

Choosing a Functional Repair Clock

The distributed architecture and conservative clocking methodology used for self-repair require some care in the selection of the functional repair clock used to apply repair during chip power up. Mentor Graphics recommends that you use a functional clock of 10 MHz or less in that functional mode.

For implementation details, refer to “[Inserting BISR Chains in a Block](#)” in the *Tessent MemoryBIST User's Manual*.

Connecting BISR controller to Existing BISR Chains

As with block level BISR insertion, BISR chains are connected according to the content of the `BisrSegmentOrderSpecification` wrapper containing a list of memory instances defining the BISR chain order as well as lower level blocks containing pre-inserted BISR chains.

The tool automatically generates this file when the [check_design_rules](#) command successfully completes.

The following example shows two instances of the same block (blockA, instances blockA_clka_i1 and blockA_clkb_i2), which has two BISR chains partitioned by two power domains (pd_AA and pd_AB):

```
BisrSegmentOrderSpecification {
  PowerDomainGroup(pd_AA) {
    OrderedElements {
      blockA_clka_i1/pd_AA_bisr_si;
      blockA_clkb_i2/pd_AA_bisr_si;
    }
  }
  PowerDomainGroup(pd_AB) {
    OrderedElements {
      blockA_clka_i1/pd_AB_bisr_si;
      blockA_clkb_i2/pd_AB_bisr_si;
    }
  }
}
```

As with the block level BISR insertion, if you change the generated BISR chain order, the `<design_name>.bisr_segment_order` file, you can manually modify this file and change the order of memory instance names before executing the [process_dft_specification](#) command.

Connecting BISR controller to an External Fuse Box

The BISR controller hardware is created at the top level of the chip automatically.

The hardware implements several functions and can be used to perform the following operations:

- Compress the repair information and write the result into the fuse box
- Decompress the fuse box contents and shift the contents into the chip BISR chain
- Initialize or observe BISR chain content via the TAP
- Read and program fuses via the TAP

The BISR controller is accessed using the TAP. The BISR chain control ports are automatically connected to the BISR controller. The BISR controller is also connected to the fuse box.

A typical implementation of BISR is with an external fuse box.

Note



A BISR controller with an *internal* fuse box is supported. For details, refer to “[Implementing and Verifying Memory Repair](#)” in the *Tessent MemoryBIST User's Manual*.

The design instance for the external fuse box must already be instantiated in the design. Typically, the fuse box is instantiated within a module that also contains interface logic. All input ports of the module should be tied off, and the output ports should be left open. When

executing the [process_dft_specification](#) command, the tool disconnects all input ports and then reconnects the ports to the BISR controller module.

When using external fuse box, set the `fuse_box_location` property in the `MemoryBisr/Controller` wrapper to `external`. The `fuse_box_interface_module` property located in the same wrapper can be used to specify the library module for the external fuse box. If this is not specified, the library module will be inferred from the design instance specified in the `design_instance` property of `ExternalFuseBoxOptions` wrapper. If neither of these properties are specified and only a single `tcd_fusebox` file exists in the design, the fuse box module will be inferred from this `tcd_fusebox` description.

The core description for the external fuse box can automatically be read in during module matching using the “[set_design_sources -format tcd_fusebox](#)” command or directly using the [read_core_descriptions](#) command. Refer to the [FuseBoxInterface](#) core description section of *Tessent MemoryBIST User's Manual* for more implementation details.

If the instantiated module has a core description with a `FuseBoxInterface` wrapper, then connections between the fuse box controller and the fuse box interface will be done automatically. If a core description is not available, or not complete, explicit connections can be made in the `MemoryBisr/Controller/ExternalFuseBoxOptions/ConnectionOverrides` wrapper. See “[Connecting the BISR Controller to an External Fuse Box](#)” in the *Tessent MemoryBIST User's Manual* for more implementation details.

Connecting BISR Controller to System Logic

Typically, system logic will be connected to the BISR controller for initiating memory repair and monitoring the progress of the operation. All connections are specified in the `DftSpecification` configuration file under the `MemoryBisr:Controller`.

- The BISR controller input `clk` must be driven by an appropriate functional clock. The connection is made by specifying the `repair_clock_connection` property in the `DftSpecification/MemoryBisr/Controller` wrapper.
- The BISR controller input `resetN` is the signal used to reset the BISR chain(s) and initiate memory repair. The connection is made by specifying the `repair_trigger_connection` property in the `DftSpecification/MemoryBisr/Controller` wrapper.

Verifying Block and Chip Level BISR

You must verify the BISR at both the block and chip level in order to guarantee functional hardware.

Block Level BISR Verification	239
Chip Level BISR Verification.....	239

Block Level BISR Verification

The default signoff is the PatternsSpecification you generate with Tessent Shell.

You generate this [PatternsSpecification](#) using the [create_patterns_specification](#) command, which tests the connectivity of the BISR chain using a pattern named MemoryBisr_BisrChainAccess.

You can additionally create future verification patterns. As a best practice, these verification patterns are as follows:

- Executing Fault-Inserted MemoryBIST
- Performing BIRA-to-BISR Capture
 - Scan External BISR Chain into the Internal BISR Chain
- Executing Post-Repair Memory BIST

For additional implementation details, refer to the “[Verify BISR at the Block Level](#)” section of the *Tessent MemoryBIST User's Manual*.

Chip Level BISR Verification

The default signoff is the PatternsSpecification you generate with Tessent Shell.

The default signoff [PatternsSpecification](#) you generate using the [create_patterns_specification](#) command for the BISR hardware verifies the FuseBoxAccess, BisrChainAccess, and Autonomous modes of operation of the BISR controller and BISR chains via the TAP.

You do not need to run memoryBIST with fault-inserted memories at the top level of the chip. This type of verification is better performed at the block level where memoryBIST can be run on the full address space of all memories.

For additional implementation details, refer to the “[Top-Level Verification and Pattern Generation](#)” section of the *Tessent MemoryBIST User's Manual*.

Chapter 6

TSDB Data Flow for the Tessent Shell Flow

The Tessent Shell Database (TSDB) is a repository for all the files and directories that Tessent Shell generates. The TSDB enhances flow automation by acting as the central location where Tessent can access the data it requires for the current task, whether that task be reading in a design, performing DRC, inserting logic test hardware, or performing ATPG pattern generation.

The TSDB structure aids data management between steps in a process even if you are not performing these steps within the Tessent Shell platform. If the steps are performed within Tessent Shell, then specifying the correct design ID automatically ensures that Tessent uses the correct file inputs for the current task.

Refer to “[Tessent Shell Database \(TSDB\)](#)” in the *Tessent Shell Reference Manual* for details about the TSDB directory structure and contents.

This chapter builds on the material discussed in the sections “[Tessent Shell Flow for Flat Designs](#)” and “[Tessent Shell Flow for Hierarchical Designs](#)” regarding the RTL and scan DFT insertion flow. During the RTL and scan DFT insertion process, Tessent Shell generates many output files and directories that it accesses later in the flow as data inputs. This chapter illustrates the data flow through each step of the RTL and scan DFT insertion flow.

Core-Level or Flat TSDB Data Flow	241
Top-Level TSDB Data Flow	246

Core-Level or Flat TSDB Data Flow

The core-level TSDB data flow applies to wrapped cores in a hierarchical DFT insertion flow. In the bottom-up insertion process, you process the wrapped cores before the top-level chip.

Refer to “[Tessent Shell Flow for Hierarchical Designs](#)” for details.

With the addition of boundary scan in the first DFT insertion pass and the exclusion of graybox modeling, this data flow also applies to flat and DFT-inserted designs unless the core includes embedded pad IO macros that need boundary scan to be inserted as well. Refer to “[Tessent Shell Flow for Flat Designs](#)” for details.

Table 6-1. Core-Level TSDB Data Flow Inputs and Outputs

Task	Input	Output
DFT Insertion: First Pass Design ID for output: rtl1	• RTL netlist • Libraries • Required DFT signals • MemoryBIST requirements • For flat: boundary scan requirements also	• Modified RTL + new RTL • ICL for IJTAG network • TCD: clocks, DFT signals • ICL for Memory + PDL • For flat: boundary scan also
DFT Insertion: Second Pass Design ID for output: rtl2	• rtl1 design data • Libraries • Required DFT signals • EDT and OCC requirements	• Modified RTL + new RTL • ICL for IJTAG network • TCD: clocks, DFT signals • ICL for OCC and EDT + PDL
Synthesis Third-party tool	• Output from write_design_import_script command (use rtl2 design data)	• Synthesized gate-level netlist
Scan Chain Insertion Design ID for output: gate	• Synthesized gate-level netlist • Scan modes • TCD, ICL, PDL from rtl2	• Scan-stitched gate-level netlist • ICL, PDL • TCD with scan modes • Graybox model (not flat)
ATPG Pattern Generation	• gate scan-inserted design data • ATPG run name • Scan mode	• Flat model • Fault list • Patterns database • TCD

During the first DFT insertion pass, provide the required files for your DFT implementation at this stage. This can include the RTL netlist, libraries for MemoryBIST insertion and boundary scan, and gate-level cells that require a Tessent cell library.

Tessent Shell generates the dft_inserted_designs, instruments, and patterns directories within the TSDB you specified. By default, Tessent Shell generates the TSDB in the current working directory if you do not specify a location. For details about these directories and the TSDB, refer to “[Tessent Shell Database \(TSDB\)](#)” in the *Tessent Shell Reference Manual*.

Tessent modifies the RTL netlist for the design into which the first-pass instrument hardware needs to be inserted. This hardware may include a MemoryBIST controller, BAP interface, and IJTAG network. In the flat DFT implementation, it may also include boundary scan and a TAP controller. Tessent Shell generates new RTL for the newly inserted DFT instruments.

In addition, Tessent Shell produces the TCD, ICL, and PDL for the design and the inserted instruments. As shown in [Figure 6-1](#), the design-level files and modified RTL are stored within the dft_inserted_designs subdirectory for this insertion pass (design ID “rtl1”). However, the

new RTL, TCD, ICL and PDL files for each inserted instrument are stored in subdirectories within the instruments directory.

The patterns directory stores the patterns associated with the rtl1 design ID in an associated subdirectory.

Tip

i To facilitate data management, save each design (whether flat, core, sub-block, or chip) in its own TSDB. This is the recommended practice when using Tessent Shell for DFT insertion.

Figure 6-1. TSDB Data Flow, Core Level, First Insertion Pass

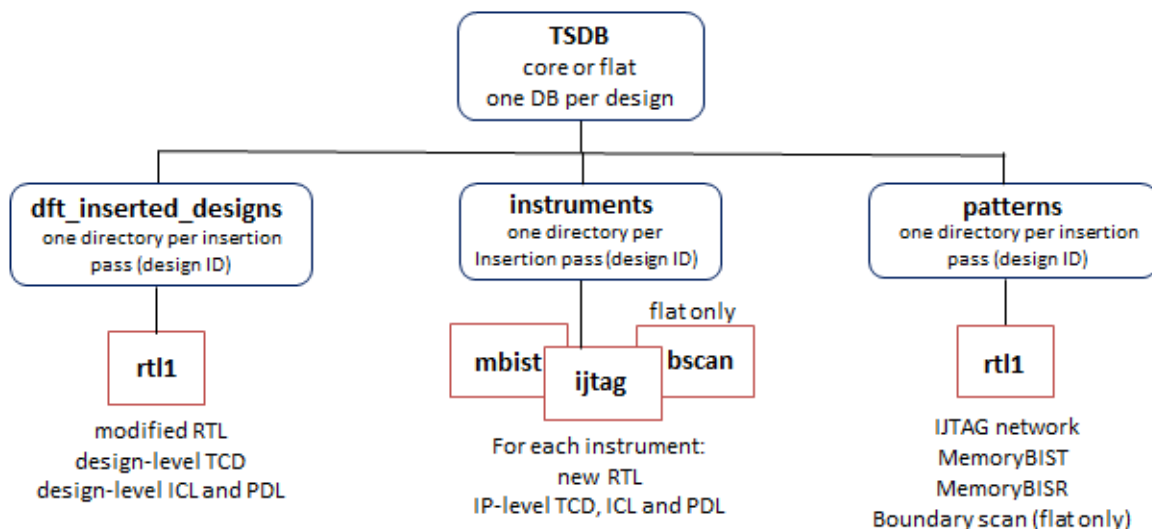


Figure 6-2 shows the data flow for the second DFT insertion pass. Tessent uses the design data that was saved as rtl1 in the first pass as input for the second pass.

The relevant design RTL, TCD, ICL and PDL files from the first DFT insertion pass are automatically read in when you specify the read_design command as described in “[Load the Design](#).” You only need to supply a library, if required, and any DFT input requirement for the DFT instruments you are inserting during this pass.

The hardware you insert in this pass, such as EDT and OCC, is stored in the instruments directory. Separate directories are created for each type of hardware inserted in each insertion pass.

The patterns directory stores the patterns associated with the rtl2 design ID in an associated subdirectory.

Figure 6-2. TSDB Data Flow, Core Level, Second Insertion Pass

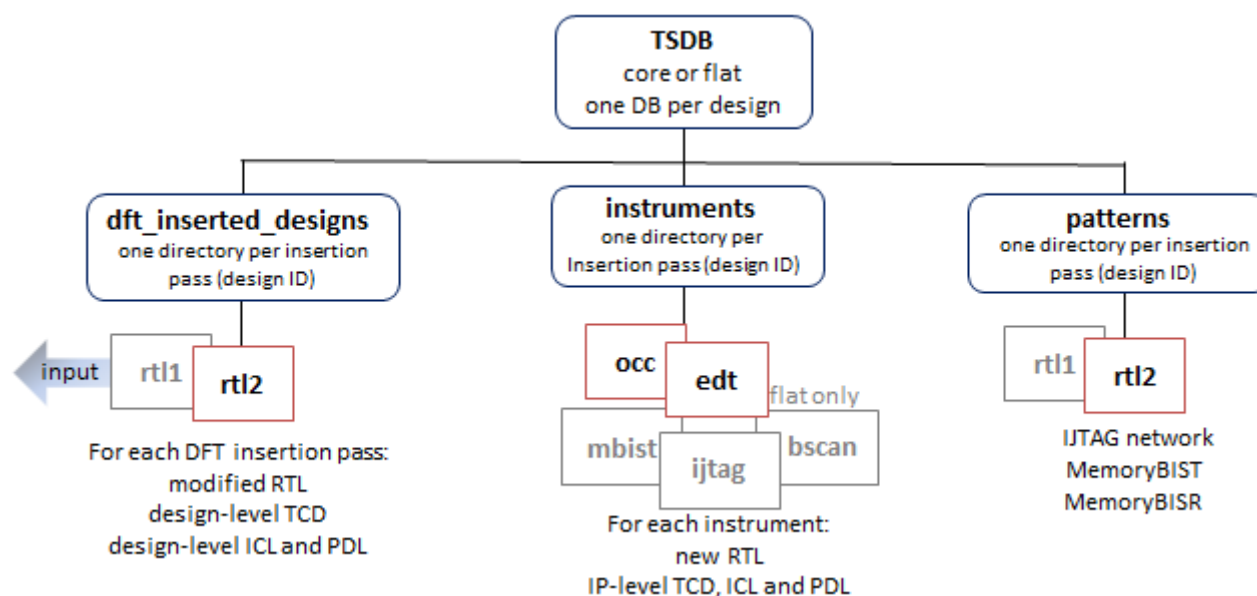


Figure 6-3 shows the data flow for scan chain insertion. After synthesis, which you perform using a third-party tool, you have a synthesized gate-level netlist. This netlist is the input for scan chain insertion along with the design-level TCD, ICL, and PDL from the rtl2 design generated during the second DFT insertion pass.

For wrapped cores, Tessent performs wrapper analysis along with scan chain insertion, whereas for flat designs, Tessent performs scan chain replacement and stitching. For information about using Tessent Scan for scan insertion, refer to “[Internal Scan and Test Circuitry Insertion](#)” in the *Tessent Scan and ATPG User’s Manual*.

Scan insertion does not insert instruments, so the instruments and patterns directories are not utilized in this step.

Figure 6-3. TSDB Data Flow, Core Level, Scan Insertion

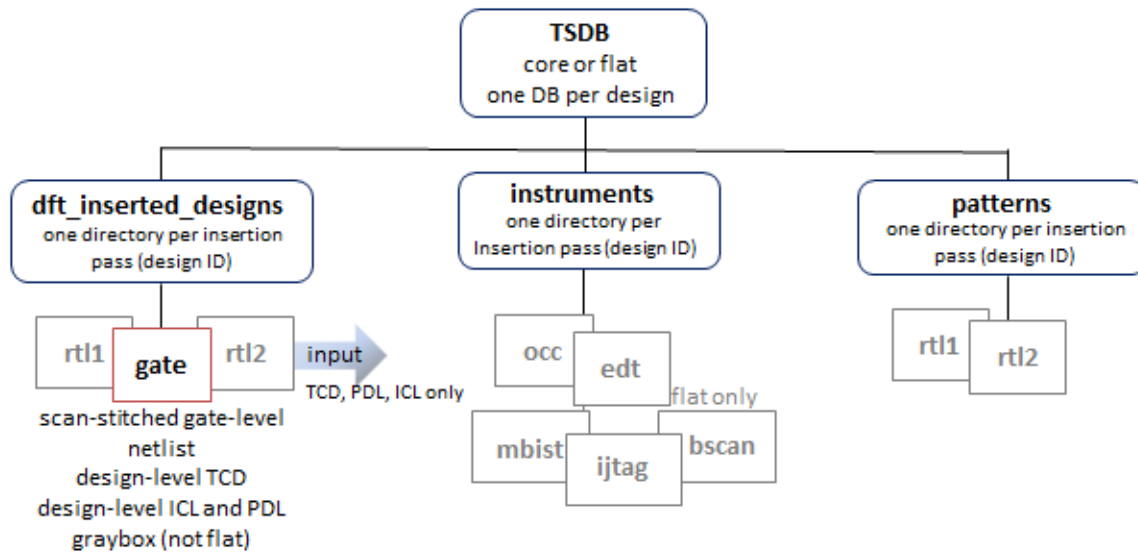
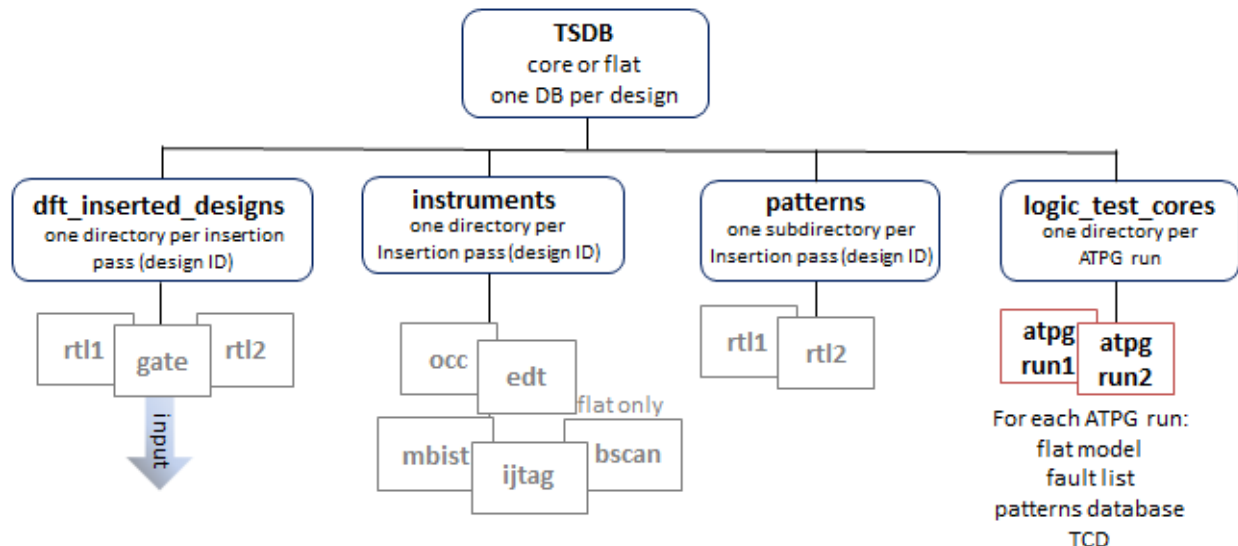


Figure 6-4 shows that the input to ATPG is the scan-inserted netlist and all the supporting files such as TCD, ICL and PDL files that were stored in the TSDB during the scan insertion pass (design_id “gate”). Tessent creates the logic_test_cores directory, which stores the output pattern data for each ATPG run, which can include runs for various test types and associated fault models as described in “[Fault Modeling Overview](#)” in the *Tessent Scan and ATPG User’s Manual*.

Before generating patterns for wrapped cores, Tessent creates a graybox model of the core. This model is stored using the same design ID as the one created during scan insertion (design ID “gate”), so that at the top-level you can either use the full design view or graybox view of the wrapped core. Refer to “[Perform ATPG Pattern Generation: Wrapped Core](#)” for more information.

Figure 6-4. TSDB Data Flow, Core Level, Pattern Generation



Top-Level TSDB Data Flow

The top-level TSDB data flow discussion applies to the hierarchical RTL and scan DFT insertion flow.

In the hierarchical DFT insertion flow, insert boundary scan—and MemoryBIST, if present—at the top level of a chip during the first DFT insertion pass.

At the chip level, Tessent uses as inputs the core-level design data that was stored in the core-level TSDBs for purposes of ATPG pattern retargeting. Refer to “[RTL and Scan DFT Insertion Flow for the Top Chip](#)” for details.

Table 6-2. Top-Level TSDB Data Flow Inputs and Outputs

Task	Input	Output
DFT Insertion: First Pass Design ID for output: rtl1	• RTL netlist • Libraries • Required DFT signals • Boundary scan requirements • TSDBs, lower core design data	• Modified RTL + new RTL • ICL for IJTAG network • TCD: clocks, DFT signals • ICL for boundary scan + PDL
DFT Insertion: Second Pass Design ID for output: rtl2	• rtl1 design data • Libraries • Required DFT signals • EDT and OCC requirements	• Modified RTL + new RTL • ICL for IJTAG network • TCD: clocks, DFT signals • ICL for OCC and EDT + PDL
Synthesis Third-party tool	• Output from write_design_import_script command (use rtl2 design data)	• Synthesized gate-level netlist
Scan Chain Insertion Design ID for output: gate	• Synthesized gate-level netlist • Scan modes • TCD, ICL, PDL from rtl2	• Scan-stitched gate-level netlist • ICL, PDL • TCD with scan modes
ATPG Pattern Generation	• Gate scan-inserted design data • ATPG run name • Scan mode	• Flat model • Fault list • Patterns database • TCD
ATPG Pattern Generation	• Gate scan-inserted design data • ATPG run name • Scan mode • Wrapped core ATPG pattern data, retargeted	• Flat model • Fault list • Patterns database • TCD

[Figure 6-5](#) shows the data flow for the first DFT insertion pass. Tessent modifies the RTL netlist for the design and generates new RTL for the boundary scan and MemoryBIST (if inserted)

hardware. In addition, it produces the TCD, ICL, and PDL for the design and the inserted instruments.

For integration at the top-level, Tessent uses as inputs the gate scan-inserted design data and interface view from the wrapped cores. This is done by opening the core TSDB directories and using the read_design command to read in the graybox model and TCD, ICL and PDL files.

The patterns directory stores the patterns associated with the rtl1 design ID in an associated subdirectory.

Tip

i To facilitate data management, save each design (whether flat, core, sub-block, or chip) in its own TSDB. This is the recommended practice when using Tessent Shell for DFT insertion. The following discussion assumes that you have one TSDB per design.

Figure 6-5. TSDB Data Flow, Top Level, First Insertion Pass

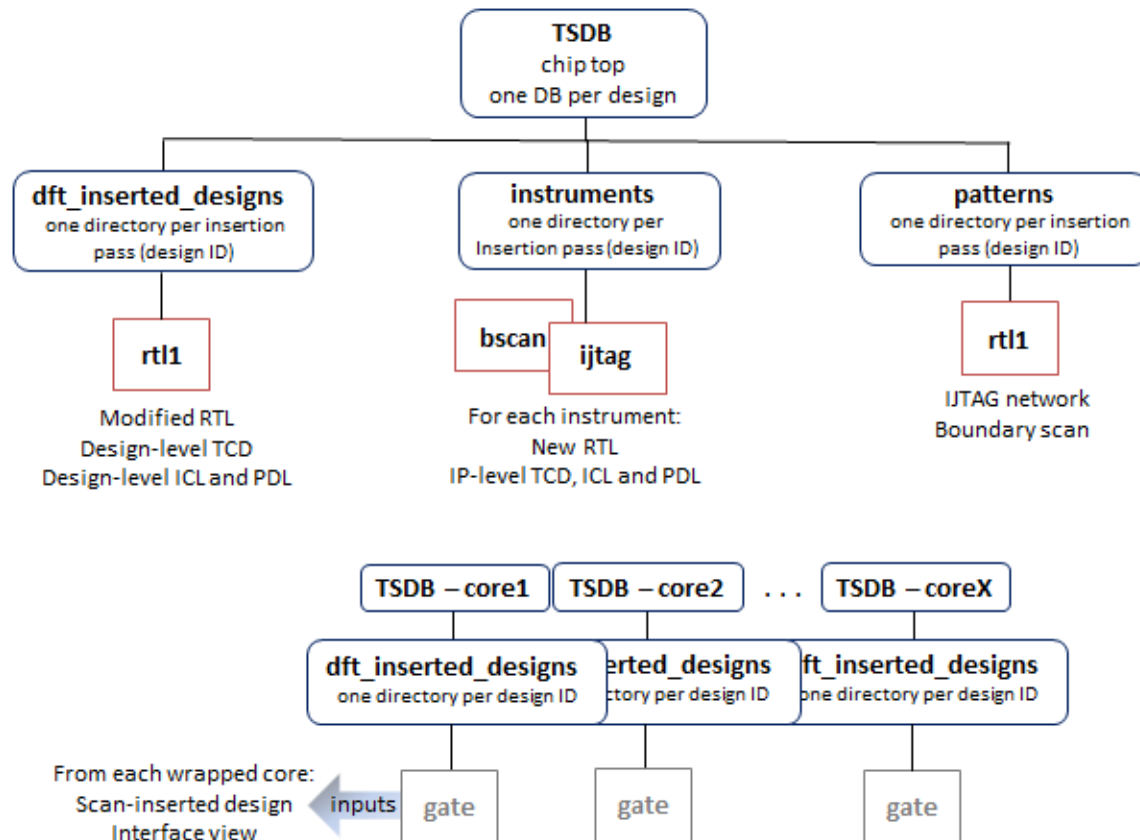


Figure 6-6 shows the data flow for the second DFT insertion pass. In addition to the other input requirements that you provide as shown in Table 6-2, Tessent uses the design data that was saved as rtl1 in the first pass, the gate scan-inserted design data, and graybox models from the wrapped cores.

Tessent saves the output design data for the EDT and OCC hardware in their applicable instruments subdirectories. The design-level TCD, ICL, PDL and modified RTL that includes the EDT, OCC, and IJTAG network is placed in the `dft_inserted_designs` subdirectory for this insertion pass (`rtl2`).

The patterns directory stores the patterns associated with the `rtl2` design ID in an associated subdirectory.

Figure 6-6. TSDB Data Flow, Top Level, Second Insertion Pass

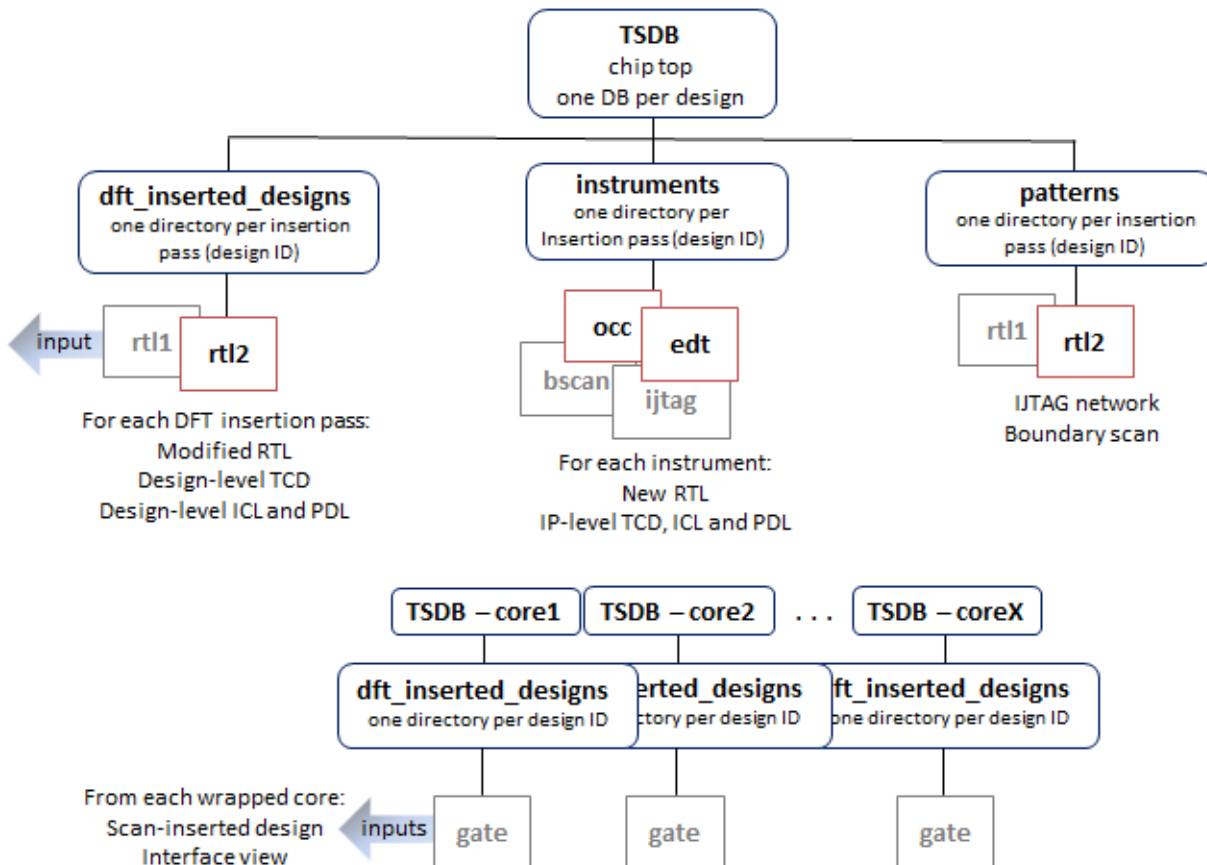


Figure 6-7 shows the data flow for scan chain insertion. Scan insertion does not insert instruments, so the instruments and patterns directories are not utilized in this step.

Figure 6-7. TSDB Data Flow, Top Level, Scan Insertion

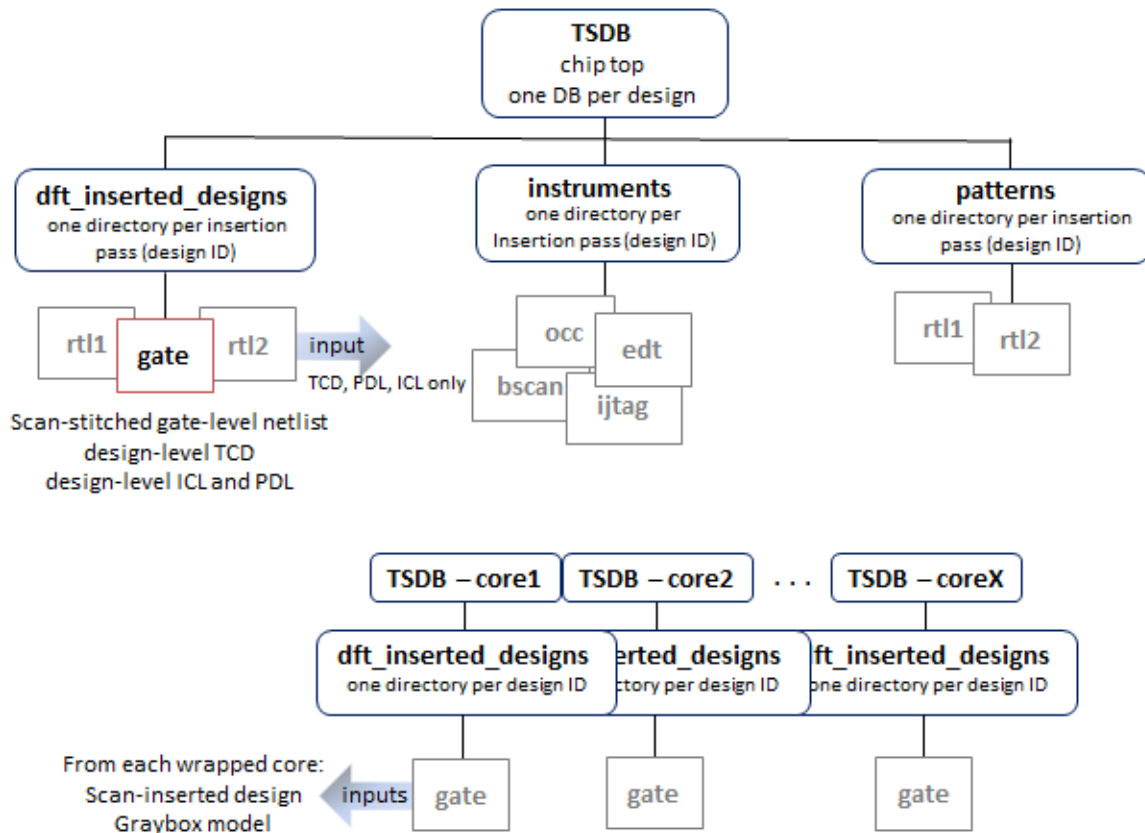
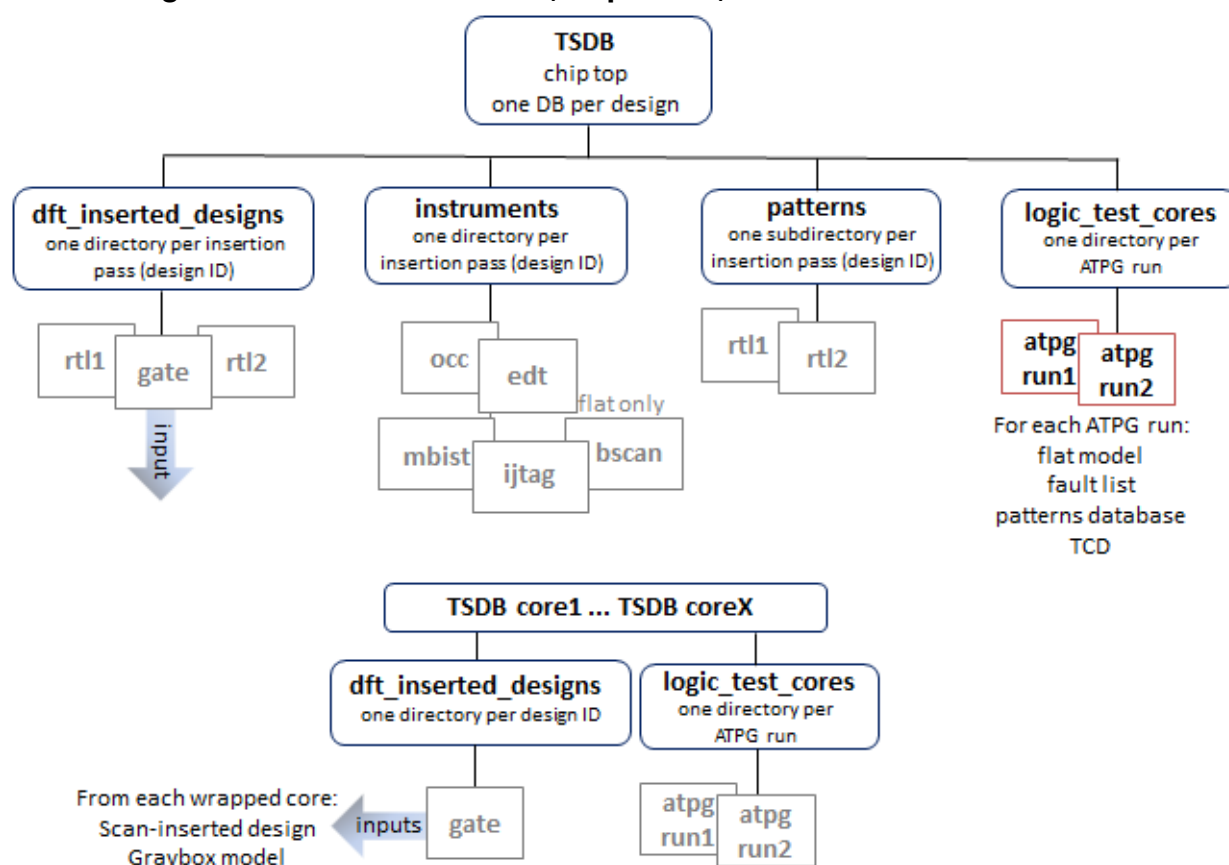


Figure 6-8 shows that output from ATPG pattern generation gets stored in the logic_test_cores directory. As inputs, Tessent uses the scan-inserted design data for the chip and for the cores.

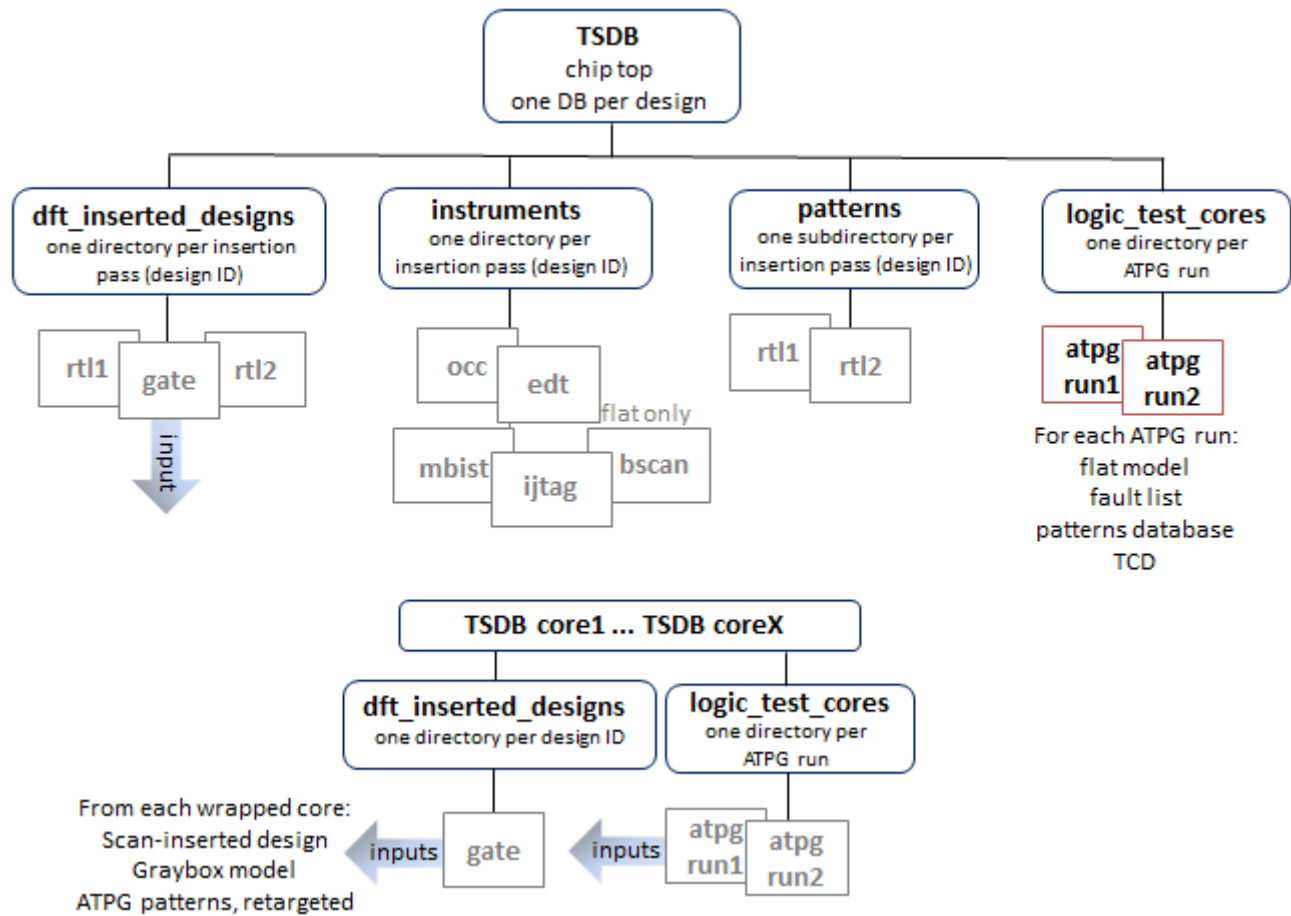
Figure 6-8. TSDB Data Flow, Top Level, ATPG Pattern Generation



As the final step for the top level in a hierarchical design, perform ATPG pattern retargeting of the core ATPG patterns as shown in “[Top-Level ATPG Pattern Generation Example](#).”

Figure 6-9 shows that you read in the ATPG patterns from the logic_test_cores directory from each of the core TSDB directories in addition to the scan-inserted design data for the chip and for the cores.

Figure 6-9. TSDB Data Flow, Top Level, ATPG Pattern Generation with Pattern Retargeting



Chapter 7

Tessent Examples and Solutions

This chapter contains Tessent solutions for specific DFT scenarios and problems. This includes applications and flows that solve common, difficult issues encountered in DFT.

The solutions are organized in the following sections:

How to Handle Clocks Sourced by Embedded PLLs During Logic Test	253
How to Design Capture Windows for Hybrid TK/LBIST	259
How to Use Boundary Scan in a Wrapped Core	261
TAP Configuration	264
Stand-alone TAP	264
TAP with IJTAG Host Scan Interface	266
Compliance Enable TAP with IJTAG Interface	267
Daisy chained TAP	269
Master TAP Insertion	270
Slave TAP Insertion	271
Connecting to a Third-Party TAP	273
How to Set Up Third-Party Synthesis	273
How to Set Up Support for Third-Party OCCs	276
How to Configure Files for Third-Party OCCs	276
Test Logic Insertion	277
Scan Insertion	279
Pattern Generation	279
Pattern Simulation	282

How to Handle Clocks Sourced by Embedded PLLs During Logic Test

Clocks sourced by an embedded PLL have local OCCs that are reused when testing parent physical regions. The flow is different when the parent physical regions are themselves wrapped cores.

Problem

For embedded PLLs, such as the one shown inside “corec” in [Figure 7-1](#), the OCC inserted on the VCO output of the PLL is used during the internal logic test modes of the core. It is also reused during the internal test modes of its parent physical regions.

When the PLL is inside a wrapped core that is itself the child of another wrapped core, you must take special steps to ensure that the OCC is still controllable by the scan chains when running logic test modes of the top level.

Solution

Wrapped Core Only Used in the Top Level

If the PLL is embedded inside a wrapped core that is only used inside the top level physical region, then you do not need to take any special steps. The standard flow handles this scenario automatically, as described in “[Tessent Shell Flow for Hierarchical Designs](#)” on page 128.

Wrapped Cores Used Inside Other Wrapped Cores

You must use the following procedure when the wrapped core in which the embedded PLL is located has another wrapped core above it, as shown in [Figure 7-1](#).

1. Add an extra DFT signal when you process the wrapped core that embeds the PLL in “[Second DFT Insertion Pass: Top-Level EDT and OCC](#)” on page 156 (for example, when processing “corec” in [Figure 7-1](#)) as follows:

```
add_dft_signals promoted_cells_mode
```

2. Create a new scan mode when you process the wrapped core that embeds the PLL in the “[Scan Chain Insertion](#)” step (for example, when processing “corec” in [Figure 7-1](#)) as follows:

```
set promoted_instances \
    [get_attributed_objects \
        -attribute_name wrapper_type_from_clock_source \
        -object_type instance]
if {[sizeof_collection $promoted_instances] > 0} {
    add_scan_mode promoted_cells_mode \
        -include_elements [get_scan_elements \
            -of_instances $promoted_instances]
}
```

The new scan mode contains the control flip-flops of the OCCs that need to be accessible from the top-level.

3. Modify the definition of the external scan mode when you process the parent wrapped core in the scan chain insertion step of the flow (for example, when processing “corea” in [Figure 7-1](#)).

```
set_attribute_value corea_i1 -name active_child_scan_mode \
    -value promoted_cells_mode

set ext_scan_elements [add_to_collection \
    [get_scan_elements -class wrapper] \
    [get_scan_elements -of_child_scan_modes promoted_cells_mode ]]

add_scan_mode ext_mode -chain_length 32 \
    -include_elements $ext_scan_elements
```

Modifying the external scan mode definition allows you to include the promoted scan chains from the child wrapped cores in the external chain of the current wrapped core. This step ensures that the control flip-flops of the embedded OCCs are accessible by the test modes of the top level.

4. Depending on the hierarchy of your design, do one of the following:
 - If your design only uses the wrapped core at the top level of the chip (such as “corea” in [Figure 7-1](#)), then no further modifications are required for the standard flow.
 - If your design embeds the wrapped core inside another wrapped core (for example, there was a layer of wrapped core between “corea” and “top” in [Figure 7-1](#)), you need to again create the promoted scan mode at the current level.

This step provides access to the OCC control bits in its external scan mode. The method shown in [step 3](#) would then apply again to that parent wrapped core as follows:

```
set_attribute_value corea_i1 -name active_child_scan_mode \
    -value promoted_cells_mode
add_scan_mode promoted_cells_mode \
    -include_elements [get_scan_elements \
        -of_child_scan_modes promoted_cells_mode]
```

Discussion

The example chip shown in the following figure illustrates functional clocking when a PLL is embedded inside a child physical region.

Figure 7-1. Example Chip with PLL Embedded Inside Lower Core

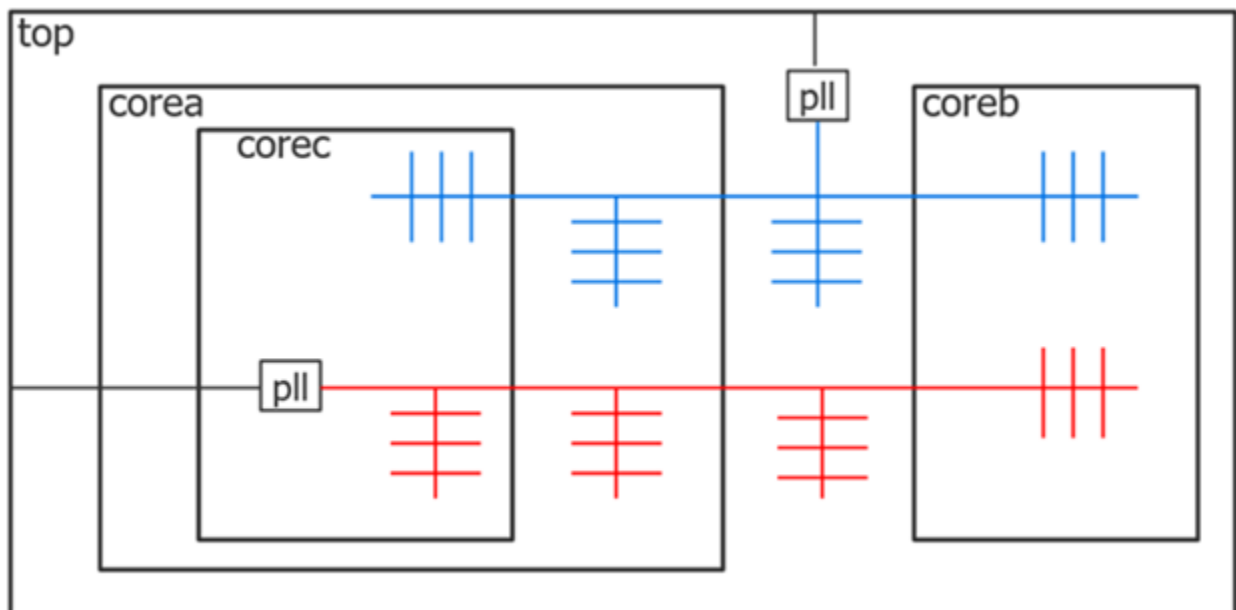
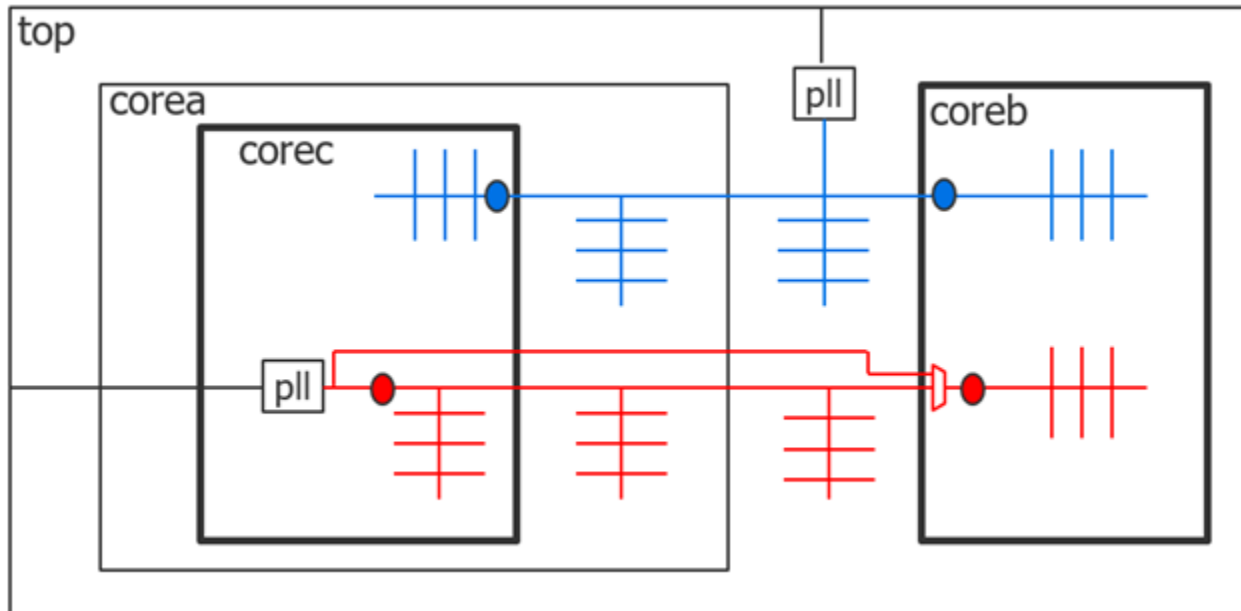


Figure 7-2 shows the location of the OCCs inserted inside “corec” and “coreb”. The two OCCs inside “corec” are active during its internal test modes in order to inject the shift clock during the shift cycles and to chop the functional clocks during capture cycles. The two OCCs inside coreb are also active during its internal test modes for the same reasons.

Figure 7-2. Active OCCs During Internal Test Modes of corec and coreb



If you want to run the internal test modes of “corec” in parallel with those in “coreb”, you need to provide a clock bypass path as shown in Figure 7-2, such that the free running output of the PLL can continue to source the red clock of “coreb” when the OCC inside “corec” is active.

The bypass clock path is not required. The tool issues an error if you try to re-target an internal test mode of “coreb” in parallel with an internal test mode of “corec” or “corea” and the bypass path is not present. The reason for this check is that the OCC at the input of the red domain of “coreb” expects and requires a free running clock when active. The red clock output of “corea” is not a free running clock when the red OCC inside “corec” is active. Instead, it is alternating between the shift clock and bursts of at-speed clock pulses.

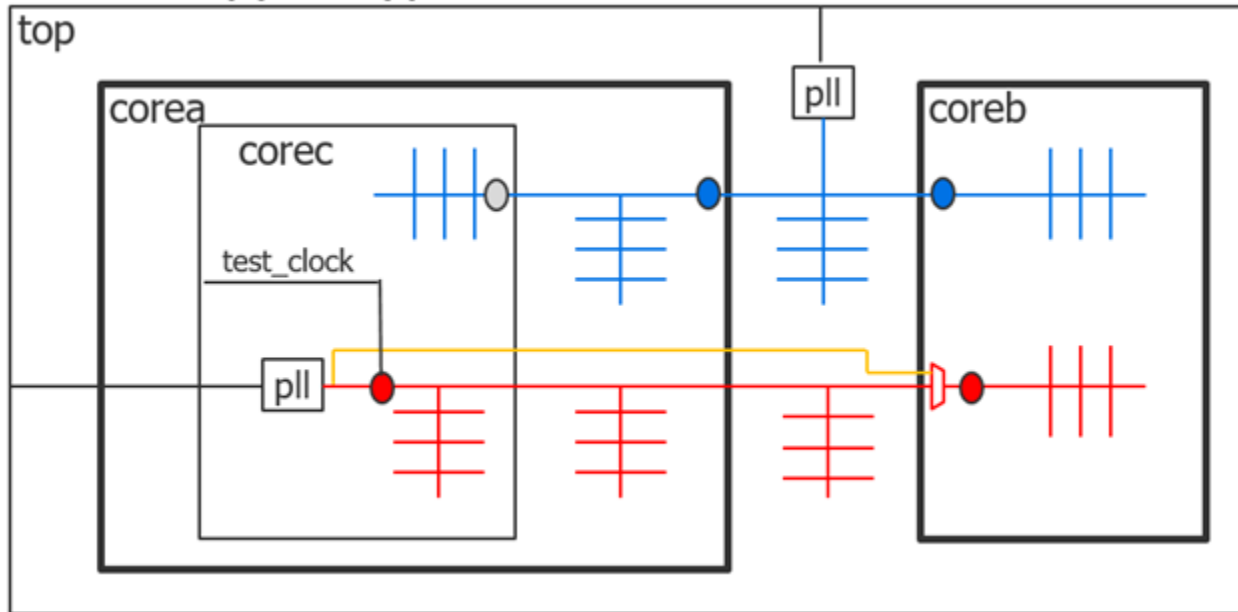
If you provide the bypass clock path, you reduce the overall chip test time as you can concurrently test “corec” or “corea” with “coreb”. If you want to insert the clock bypass path within the DFT insertion process, use a [process_dft_specification.post_insertion](#) callback to create the ports and make the connections. Use the [intercept_connection](#) command to insert the multiplexer inside “coreb”. The best option to control the select input of the multiplexer is to register and add a new DFT signal. See the [register_static_dft_signal_names](#) command for more information.

The [get_dft_signal](#) command in the [process_dft_specification.post_insertion](#) callback gets the connection point for the added DFT signal. If the bypass path is manually added in the golden RTL, leave the select input of the multiplexer tied low and connect it to the DFT signal during

the DFT process with the `add_dft_control_points` command. See “[Example 1](#)” of the `register_static_dft_signal_names` command description section for an example.

When you move up to “corea”, another OCC is inserted at the base of the blue clock domain to be used during its internal logic test modes, as shown in [Figure 7-3](#).

Figure 7-3. Active OCCs During Internal Test Modes of corea and coreb



The OCC on the blue domain inside “corec” is kept inactive, as it is during the functional mode, such that the clocking of the entire blue clock domain is controlled by the OCC located inside of “corea” at the base of the clock tree. The red OCC inside “corec” is active during the internal test mode of “corea” in order to control the scannable flip-flops on the red domain inside “corea”, as well as the wrapper flops on the red domain inside “corec”. Because the “fast_clock” input of that red OCC is not sourced by an input of “corec”, its scan chain is automatically promoted to its wrapper chains. This allows it to be under ATPG control when running the internal test mode of “corea”.

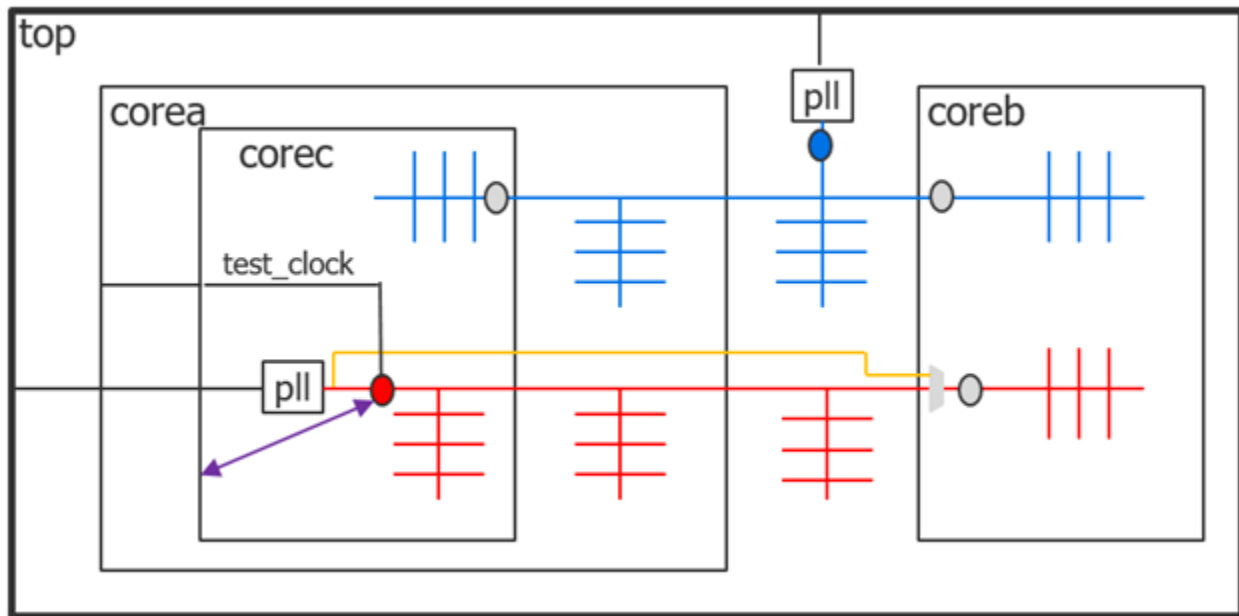
If “corec” was not embedded within another wrapped core (such as “coreb” that is directly instantiated in the top level), the handling is completely automated and no deviation from the standard flow, described under “[Tessent Shell Flow for Hierarchical Designs](#)” on page 128, is necessary. However, when the wrapped core containing the embedded PLL is inside another wrapped core, you must follow the steps described under “[Solution](#)” on page 254 to enable the embedded OCC inside “corec” to be under ATPG control at the top level.

Extra DFT Signals

Figure 7-4 shows the active OCCs when running the logic test mode of the top level. As in the previous case, the red OCC inside “corec” is active. This requires that the scan segment that contains the control flip-flops of the red OCC must be accessible by the scan chains of the top

level. For that reason, step 1 of the solution above shows how to add a new DFT signal called “promoted_scan_mode” to use as the enable for that scan chain configuration.

Figure 7-4. Active OCCs During Test Modes of Top Level



New Scan Mode

Step 2 shows how to create the scan mode that contains the control flip-flops of the promoted OCCs. The OCC instances that have their “fast_clock” input controlled by an internal source have an attribute named “wrapper_type_from_clock_source” set on them automatically. The [get_attributed_objects](#) command is used to find them and make them part of the special scan mode.

Promoted Scan Chains

When you get to the “corea” level, you need to promote the special scan chain on “corec” into the wrapper chain of “corea”, such that the control flip-flops of the red OCC are under ATPG control from the top level. This task is described in step 3.

Step 4 provides instructions for when “corea” is not directly instantiated into the top level, but instead is embedded within another wrapper core. In that case, you again need a special scan mode to collect the embedded OCC chains such that they can again be included in the wrapper chains of the parent wrapped core. Additionally, you again follow step 3 on the parent wrapped core to include the promoted scan mode of “corea” within its wrapper chain.

The purple line in Figure 7-4 represents the promoted scan chain that contains the control flip-flops of the red OCC. This scan segment is inserted in the “ext_mode” of “corea” in step 3 such that the OCC is part of the scan chains of the top level.

Related Topics

[On-Chip Clock Controller Design Description \[Tessent Scan and ATPG User's Manual\]](#)
[OCC \[Tessent Shell Reference Manual\]](#)

How to Design Capture Windows for Hybrid TK/LBIST

A capture window is a group of capture cycles defined by one or more clocks. Test logic can be configured to run a selected number of patterns for each capture window. This approach gives full control over the patterns to optimize test coverage and test power consumption.

Problem

The fundamental objectives for LBIST are increased test coverage, decreased test time, and lower power consumption for the test run. In a typical flow, the full set of data needed to perform optimal insertion to meet these objectives is not available until the gate-level netlist is ready. In practice, test circuitry is closely integrated into the design and the design suffers when test is treated as an ad-hoc component or inserted later in the gate-level netlist.

To address these issues and achieve desired test coverage and performance, the Hybrid TK/LBIST flow enables you to insert DFT logic at the RTL level, before the gate-level netlist is ready. Capture windows enable the flexibility to add test logic in the RTL and test accurately.

Solution

The solution is provided in two parts:

- How to design capture windows (which clocks and how many pulses from each clock).
- How many patterns to run per capture domain to achieve the targeted coverage.

Define Capture Windows

If the clocking structure is known and determined during the insertion of the IP, then defining the capture windows is a straight-forward process. To define capture windows for this flow, follow the examples under “[NCP Index Decoder](#)” in the *Hybrid TK/LBIST Flow User's Manual*.

If the clocking structure is not yet defined or finalized, use the following procedure:

1. Create a gate-level netlist using quick synthesis.
2. Run ATPG with the following constraints:
 - a. Use the same setup and constraints used for LBIST.

- b. Set the number of random patterns:

```
set_random_pattern integer
```

where the integer argument is the number of patterns that are planned for use in LBIST.

- c. Run pattern simulation:

```
simulate_patterns -source random -store all
```

- d. Report the test coverage per clock domain:

```
report_statistics -clock all
```

- e. Report the generated patterns:

```
report_patterns
```

- f. Design your capture windows using information from the ATPG run and the report_statistics command.

- The ATPG run helps you identify the test clock domains in the design.
- The report_statistics command helps you identify the number of clock pulses per clock domain

- g. Determine how many patterns to run for each capture domain to achieve the targeted coverage.

In the following example, you can identify that the design has three clock domains, with possible interaction between CLK2 and CLK3. The highest faults per domain is at CLK2 at 78%, followed by CLK3 at 43%.

Clock Domain Summary	% faults (total)	Test Coverage (total relevant)
CLK3_OCC	43.07%	99.13%
CLK2_OCC	78.80%	99.08%
CLK1_OCC	22.98%	98.17%

Patterns to Run per Capture Domain

For stuck-at-fault, try to use as many clock domains in the capture window as possible. This saves test time. Use the “Define Capture Windows” procedure above to select the number of patterns per capture window.

In the following example, to achieve the best coverage with the lowest number of patterns, a two cycle capture window should be used for 20% of the patterns and more than 80% of patterns should be used by a single CLK2_OCC/CLK1_OCC clock pulse capture window

patt. #	pre-filtering pattern #	type	cycles	loads	capture_clock_sequence
0	0	basic	1	1	[CLK1_OCC, *]
1	1	basic	1	1	[CLK2_OCC, *]
...	...	...			
1	400	basic	1	1	[CLK2_OCC, *]
2	401	clock_sequential	2	1	[CLK2_OCC, *] [CLK3_OCC, *]
3	405	clock_sequential	2	1	[CLK2_OCC, *] [CLK3_OCC, *]
3	500	clock_sequential	2	1	[CLK2_OCC, *] [CLK3_OCC, *]

Note: [*] = "SHIFT_CLOCK", which is a pulse-in-capture clock.

How to Use Boundary Scan in a Wrapped Core

You must perform a specific procedure if a wrapped core has embedded pads that require boundary scan. Care must be taken during the insertion of the boundary scan and during ATPG.

Problem

Pads embedded inside a wrapped core must be identified such that boundary scan cells can be added. The resulting boundary scan cells must also be made visible to the tool in order for them to be included in scan chains and to apply scan patterns.

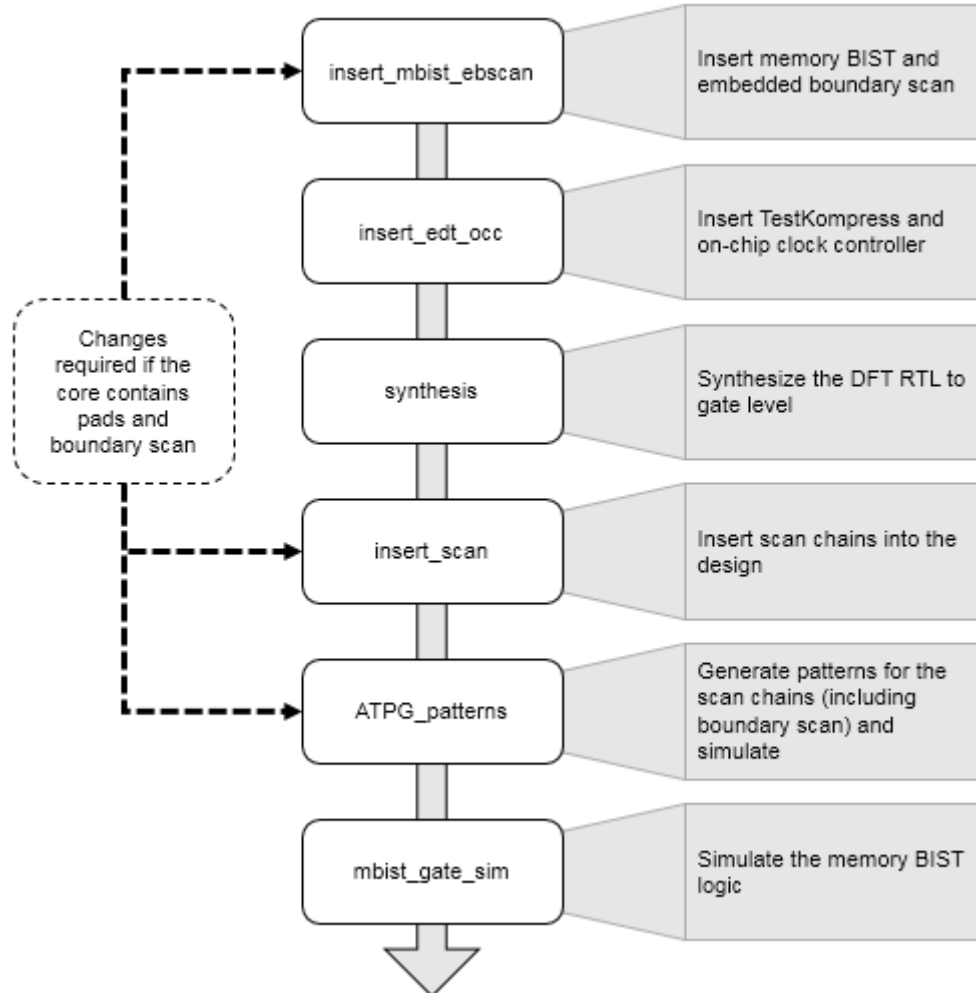
Solution

The solutions are given for wrapped core and chip-level flows.

Wrapped Core Flow

The following figure shows the standard command flow for creating a wrapped core that includes embedded boundary scan:

Figure 7-5. Wrapped Core Boundary Scan Flow



If the core contains pads and boundary scan, you must include the following commands in the wrapped core flow shown in [Figure 7-5](#) on page 262:

- Insert memory BIST and embedded boundary scan using during the `insert_mbist_ebscan` step as follows:
 - a. Specify DFT requirements to insert memory test and embedded boundary scan:

```
set_dft_specification_requirements -memory_test on \  
-boundary_scan on
```

- b. Insert embedded boundary scan cells and identify the pads:

```
set_boundary_scan_port_options \
  -pad_io_ports [list taclk ta_cci0a ta_cci0b ta_cci1a ta_cci1b
    ta_cci2a ta_cci2b ta_out0 ta_out1 ta_out2]
```

Note



The order specified is the order in which the boundary scan cells are inserted.

- Specify not to add any dedicated wrapper cells on embedded pad IO during the insert_scan step:

```
set_dedicated_wrapper_cell_options off \
  -ports {ta_out0 ta_out1 ta_out2}

set_dedicated_wrapper_cell_options off \
  -ports {ta_cci0a taclk ta_cci0b ta_cci1a ta_cci1b ta_cci2a
    ta_cci2b}
```

- Preserve the boundary scan instances in the graybox during the ATPG_patterns (graybox generation) step.

```
set_preserve_bscan {}

set_preserve_bscan \
  [get_instances -hier *_tessent_bscan_logical_group_*]
analyze_graybox -preserve_instances $preserve_bscan
write_design -tsdb -graybox -verbose
```

Chip-Level Flow

For boundary scan at the chip-level, follow the standard chip-level flow by inserting boundary scan as the first step in the flow. There are no specific additions for embedded boundary scan at this phase of the flow. The boundary scan segments from the wrapped core are picked up automatically during chip-level boundary scan insertion.

TAP Configuration

Tessent Shell typically relies on an IEEE 1149.1-based Test Access Port (TAP) as the primary access mechanism to the DFT it inserts. When boundary scan is implemented, the Tessent TAP fully complies with IEEE 1149.1 standard requirements, however many other possible configurations are possible.

This section provides a quick reference to the various TAP insertion styles that are supported, how to specify them, and what to expect from such implementations.

Stand-alone TAP	264
TAP with IJTAG Host Scan Interface.....	266
Compliance Enable TAP with IJTAG Interface.....	267
Daisy chained TAP	269
Master TAP Insertion	270
Slave TAP Insertion.....	271
Connecting to a Third-Party TAP	273

Stand-alone TAP

Insert a simple stand-alone TAP in a design.

Solution

1. Set the design level to “chip”.
2. Use the following dofile example:

```
set DFT [create_dft_specification]
read_config_data -in $DFT -from_string {
  IjtagNetwork {
    HostScanInterface(ijtag) {
      Interface {
        tck : tck;
      }
      Tap(single) {
      }
    }
  }
}
process_dft_specification
```

The generated TAP connects to the trst, tck, tms, tdi, and tdo pads that were present in the pre-DFT design.

Discussion

If the TAP pins in the current design do not use the default names (trst, tck, tms, tdi, or tdo), then they can be mapped using one of the following:

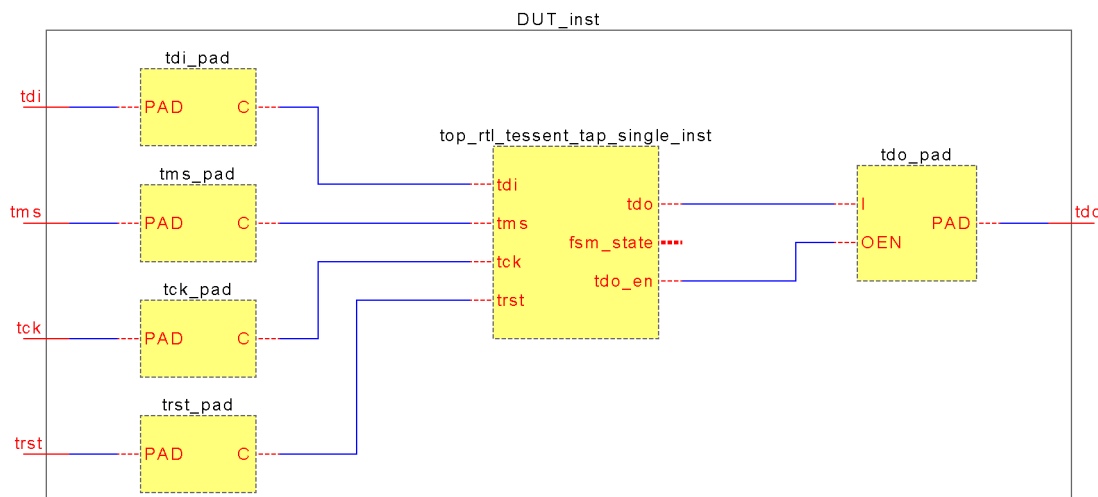
- The `DftSpecification::IjtagNetwork::HostScanInterface::Interface` wrapper.
- The following dofile command:

```
set_attribute_value portname -name function \  
-value [trst | tck | tms | tdi | tdo]
```

The tool issues errors if the TAP pads are not yet present in the current design. If the design is only temporary, you can specify the “`set_design_level sub_block`” command instead. Note that many TAP-specific DRCs are not run in such a case and other side effects may result. You should read an updated design that includes TAP pads as soon as possible.

The TAP generated in this example can be further enhanced with additional IR opcodes, IJTAG host scan interfaces, and decoded outputs to enable downstream logic, such as another TAP.

The following is a schematic representation of this example:



Related Topics

[TAP with IJTAG Host Scan Interface](#)

[Compliance Enable TAP with IJTAG Interface](#)

[Daisy chained TAP](#)

[Master TAP Insertion](#)

[Slave TAP Insertion](#)

[Connecting to a Third-Party TAP](#)

TAP with IJTAG Host Scan Interface

Insert a TAP equipped with an IJTAG host scan interface. An IJTAG network can then be connected to the TAP in a subsequent test insertion phase.

Solution

1. Set the design level to “chip”.
2. Use the following dofile example:

```
set DFT [create_dft_specification]
read_config_data -in $DFT -from_string {
  IjtagNetwork {
    HostScanInterface(ijtag) {
      Interface {
        tck : tck;
      }
      Tap(single) {
        HostIjtag(1) {
        }
      }
    }
  }
}
process_dft_specification
```

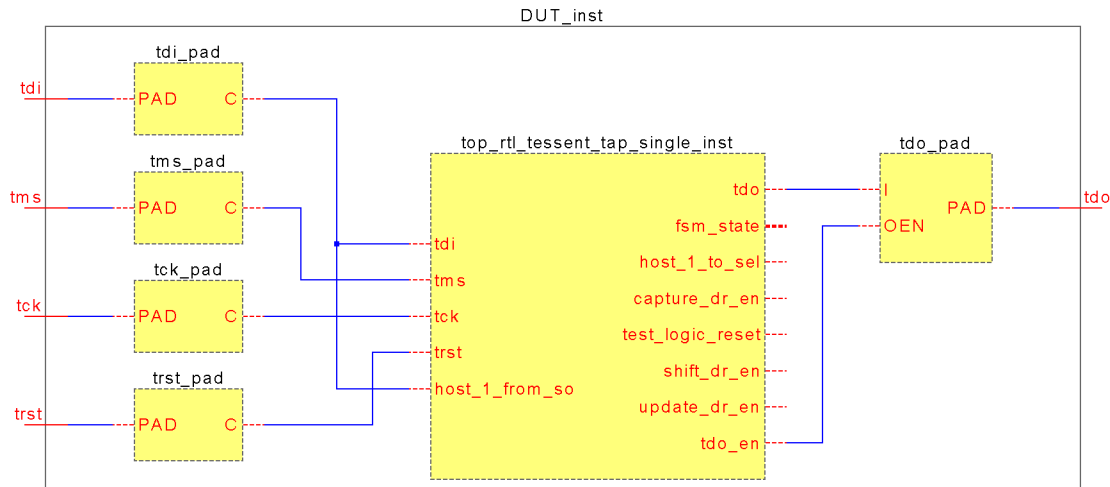
Discussion

The host scan interface on the generated TAP can be used to control any type of IJTAG-compliant network.

The host scan interface provides a `test_logic_reset` output to reset downstream logic; it asserts the `host_hostname_to_sel` output to 1 when accessing the client IJTAG network.

The `capture_dr_en`, `shift_dr_en` and `update_dr_en` outputs are consumed by the client IJTAG network or instruments after qualification with the above `host_hostname_to_sel` port.

The following is a schematic representation of this example:



Related Topics

[Stand-alone TAP](#)

Compliance Enable TAP with IJTAG Interface

Insert a TAP in a design with compliance enable (CE) decoding logic. The CE decoding logic ensures that the TAP can only be accessed if all of the specified values are present on the primary inputs.

The same scheme is also used when the TAP pins are shared with functional pins. The CE pins ensure that the TAP are only be activated during intended test modes and not accidentally, for example, while the functional pins toggle during normal operation.

Solution

1. Set the design level to “chip”.
2. Ensure that at least one primary input pad is set to 0 or 1 to enable the TAP logic.
3. Use the following dofile example:

```
set DFT [ create_dft_specification \
    -active_low_compliance_enables {tap_en0} \
    -active_high_compliance_enables {tap_en1} ]
read_config_data -in $DFT/IjtagNetwork/HostScanInterface(tap) \
    -from_string {
        Tap(CE_decoded) {
            HostIjtag(1) {
            }
        }
    }
process_dft_specification
```

Discussion

Note how the CE pins are specified as options to the `create_dft_specification` command.

The CE decoding module relies on the CE pin values to select the active TDO and TDO_EN signals and route both to the primary TDO output pad.

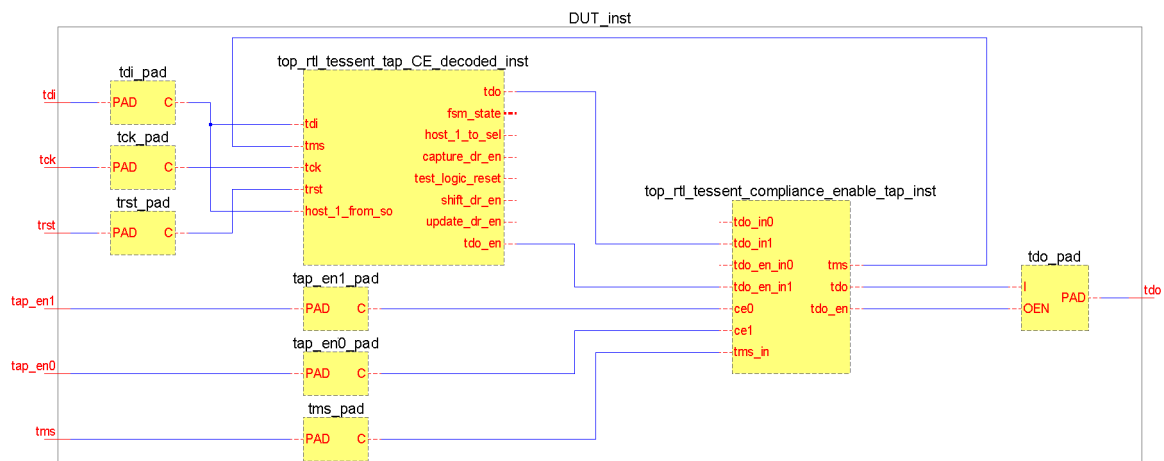
The same CE module also gates the TMS signal such that when the proper values are not present, the TMS input on the TAP is kept to 0. This normally has the effect of “parking” the TAP in one of the stable FSM states (refer to the IEEE 1149.1 FSM state diagram for details).

Do not mix-up the CE decoding module input port names with expected CE values! For instance, in the above diagram the “ce0” input is actually sourced by the CE pin driven at 1; the “ce1” input is connected to the CE pin driven at 0. Potentially very confusing – so be aware.

The general rule is that if n CE inputs are specified, the CE decoding logic will be created with input ports ranging from `ce<n-1>` down to `ce0`.

If internal CE nodes have to be used (i.e. when the pre-DFT design already contains decoding logic and hookup points to connect the new TAP to), declare those hierarchical nodes in a new `DftSpecification::IjtagNetwork::HostScanInterface::Interface` wrapper. The `Tap(<name>)` wrapper then has to be put within the very same interface wrapper

The following is a schematic representation of this example:



Related Topics

[Stand-alone TAP](#)

Daisychained TAP

This example inserts a second TAP in series with an existing one. The TRST, TCK, and TMS signals are shared between both TAPs, which implies that the complete JTAG chain (for example, top-level TDI to TDO) always shifts through both TAPs.

Solution

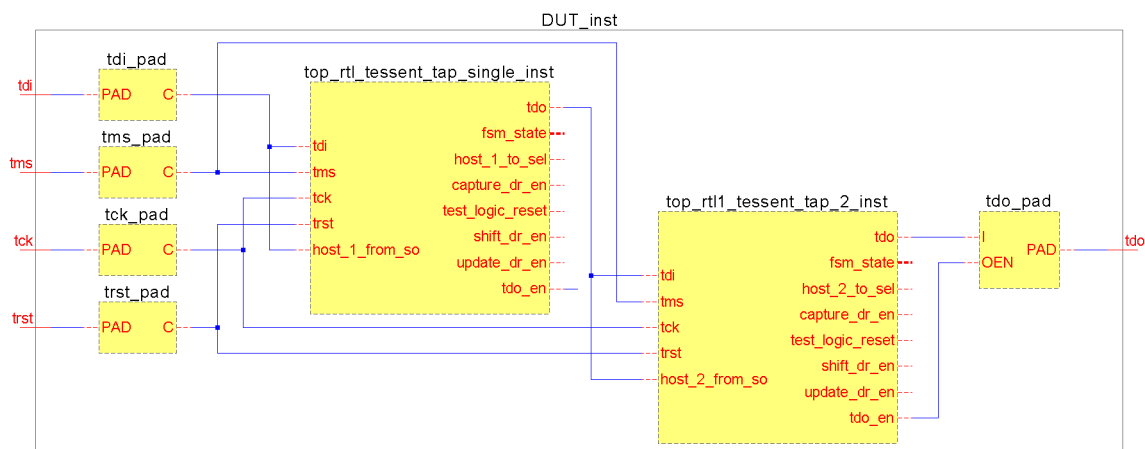
1. Set the design level to “chip”.
2. Use the following dofile example:

```
set DFT [create_dft_specification]
read_config_data -in $DFT -from_string {
  IjtagNetwork {
    HostScanInterface(tap) {
      Interface {
        tck : tck;
        daisy_chain_with_existing_client : on;
      }
      Tap(2) {
        HostIjtag(2) {
        }
      }
    }
  }
}
process_dft_specification
```

Discussion

The first TAP in this example has a name that starts with “top_rtl_”, while the second TAP begins with “top_rtl1_”. These names are used because this step is typically done as a subsequent insertion pass, such that the current design already contains at least one valid TAP.

The following is a schematic representation of this example:



Related Topics

[Stand-alone TAP](#)

Master TAP Insertion

Insert a TAP in a design that does not currently contain a TAP. The generated TAP is used as a master TAP that contains output(s) to enable or disable slave TAP(s) that are inserted in subsequent insertion passes.

Solution

1. Set the design level to “chip”
2. Create one or several DataOut ports on the master TAP, such that a slave TAP is enabled when the corresponding DataOut port is asserted.
3. Use the following dofile example:

```
set DFT [create_dft_specification]
read_config_data -in $DFT -from_string {
  IjtagNetwork {
    HostScanInterface(tap) {
      Interface {
        tck : tck;
      }
      Tap(master) {
        HostIjtag(1) {
        }
        DataOutPorts {
          count : 1;
        }
      }
    }
  }
}
process_dft_specification
```

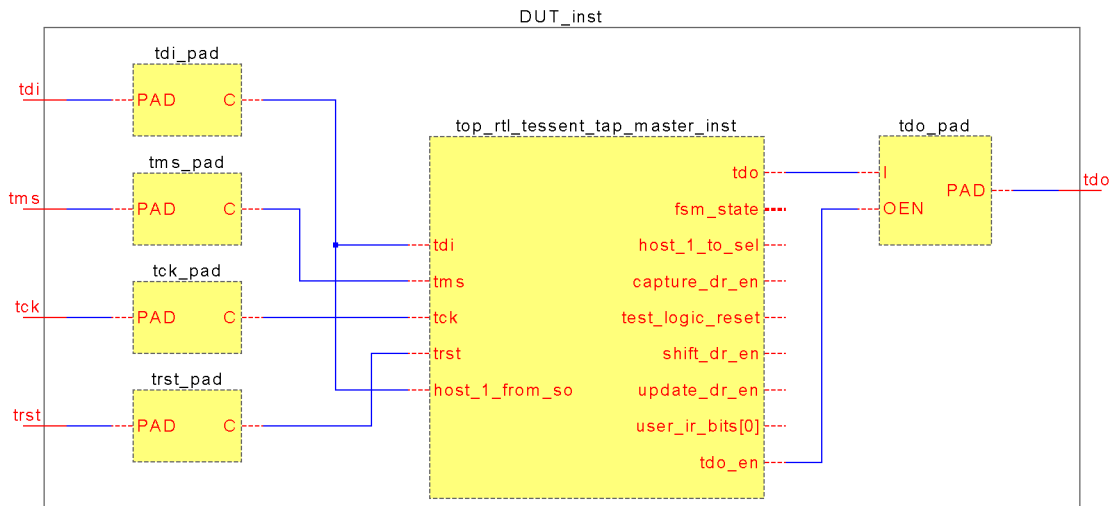
Discussion

Other than the new user_ir_bits[0] output port that was created using the DataOutPorts wrapper, this TAP is functionally identical to the stand-alone TAP with a host scan interface.

Note that the added DataOutPort ports are considered as TAP IR bits. If you need to create these bits as DR bits, insert a TDR on the existing host scan interface using the following command:

```
create_dft_specification -existing_ijtag_host_scan_in hierarchical_pin
```

The following is a schematic representation of this example:



Related Topics

[Stand-alone TAP](#)

[Slave TAP Insertion](#)

Slave TAP Insertion

Generate a master TAP, a 2-to1 scan mux, and a slave TAP. By default, the master TAP is enabled.

Solution

1. Set the design level to “chip”

2. Use the following dofile example:

```
set DFT [create_dft_specification]
read_config_data -in $DFT -from_string {
  IjtagNetwork {
    HostScanInterface(tap) {
      Interface {
        tck : tck;
      }
      Tap(master) {
        HostIjtag(0) {
        }
        DataOutPorts {
          count : 1 ;
        }
      }
      ScanMux(1) {
        select : Tap(master)/DataOut(0) ;
        Input(1) {
          Tap(slave) {
            HostIjtag(1) {
            }
          }
        }
      }
    }
  }
}
process_dft_specification
```

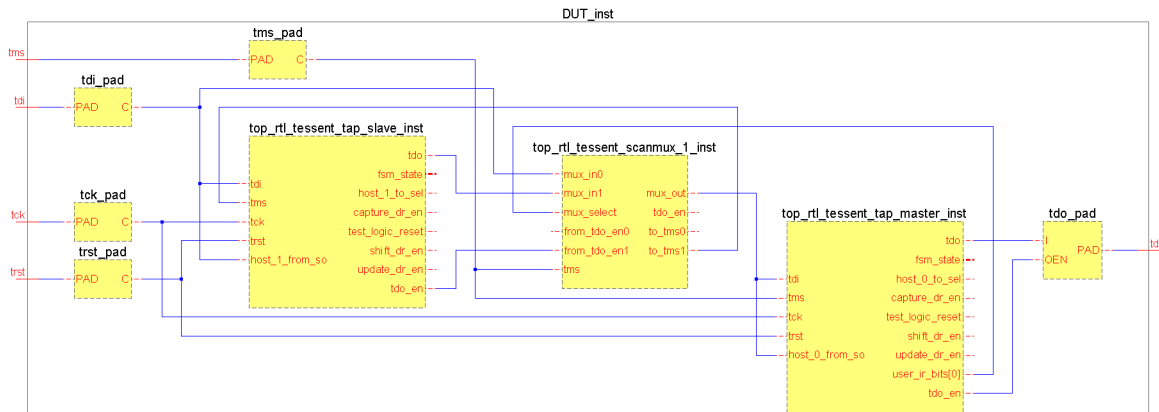
Discussion

To trace through TAP slave implementations, it is easier to start from the TDO output and gradually trace back toward the primary TDI input.

The inserted ScanMux selects the TDI input by default and routes it to its own mux_out output.

The mux_select input comes from the master TAP's user_ir_bits[0] output. When this output transitions to 1, the slave TAP is muxed in, such that the complete JTAG scan path goes through both TAPs.

The following is a schematic representation of this example:



Related Topics

[Stand-alone TAP](#)

[Master TAP Insertion](#)

Connecting to a Third-Party TAP

Connecting an IJTAG network or instruments to a third-party TAP requires reading the Verilog module of that TAP along with its ICL before setting the current design in Tessent Shell.

Solution

When creating the DFT specification, specify the `-existing_ijtag_host_scan_in` port option and point to the ScanInPort on the third-party TAP that receives the scanned-out data from the inserted IJTAG network or instruments. The DFT specification is then created accordingly.

Related Topics

[Stand-alone TAP](#)

How to Set Up Third-Party Synthesis

You must synthesize DFT logic generated in the Tessent Shell flow in order to perform scan insertion, test pattern generation, and other processes.

Problem

You must define constraints in synthesis tools to enable accurate gate level representations of the RTL. These constraints are primarily made up of clock definitions and timing constraints.

Solution

Use the following procedures to set up constraints for Synopsys and Cadence synthesis tools within the Tessent Shell flow.

Synopsys

The following procedure provides a high-level overview of the synthesis flow for Synopsys. For complete details, refer to “[Example Design Compiler Synthesis Script](#)” on page 505 for an example.

1. Source the SDC file generated by the tool.

The generated SDC file is located in the TSDB of the current design under the design ID to which the DFT was inserted and the ICL was extracted.

2. Set and redefine the Tessent Tcl Variables.

The SDC file includes variables that need to be set in order for the script to operate correctly. For example, the period of the TCK clock needs to be set. Depending on your DFT flow, you may need to define additional variables. The variables that you must set are easily identified in the SDC script file.

3. Verify the declaration of the functional clocks.

You must define all functional clocks. The tool automatically generates clock definitions based on the information from your DFT insertion script. You must review these definitions for accuracy.

4. Redefine other Tessent Tcl variables.

You must review and customize any additional variables. For example, a variable must be set that defines the input delay for the primary pins. This setting is based on the pre-defined TCK period and a custom multiplier value. The following is an example:

```
set tessent_input_delay [expr 0.3 * $tessent_tck_period]
```

Note

 The value of `tessent_tck_period` might depend on the maximum tck clock frequency that can be applied to the circuit. See the “[IJTAG Network Performance Optimization](#)” section in the *Tessent IJTAG User’s Manual* showing how to maximize the frequency of the IJTAG network test clock.

5. Load the design into your synthesis tool.

The Tessent Shell tool automatically creates a script to import your design. The tool then sources the generated script elaborates the design.

6. Apply the SDC constraints.

At this stage of the flow, the SDC constraints for the DFT logic are applied by sourcing the appropriate procedure. The procedures are described under the [extract_sdc](#) command of the *Tessent Shell Reference Manual*.

7. Prepare the DFT logic for synthesis.

You must set up the DFT logic by configuring synthesis tool settings to ensure proper synthesis. For example, some instances need to be preserved during synthesis to ensure that the gate-level analysis functions correctly. The following statements are examples of configuration settings:

```
set_app_var  
compile_enable_constant_propagation_with_no_boundary_opt false  
set_preserve_instances [tessent_get_preserve_instances icl_extract]  
set_boundary_optimization $preserve_instances false  
set_ungroup $preserve_instances false  
set_app_var compile_seqmap_propagate_high_effort false  
set_app_var compile_delete_unloaded_sequential_cells false  
set_boundary_optimization [tessent_get_optimize_instances] true  
set_size_only -all_instances [tessent_get_size_only_instances]
```

8. Synthesize your design.

The synthesis script compiles the design to create a gate level representation of the RTL.

9. Write out the SDC and the final gate-level netlist.

The final SDC is a combination of the functional and the DFT constraints.

Cadence

For synthesis with the Cadence Encounter tool, follow the procedure for the Synopsys tool as documented above, but ensure that you adjust steps 3 through 6 to match the script shown in “[Example Encounter Synthesis Script](#)” on page 507. These specific steps are different because the tools use different commands to run synthesis. Otherwise, the flow and results are the same.

How to Set Up Support for Third-Party OCCs

Tessent tools provide automation that enables you to insert Tessent OCCs as well as define and configure them for ATPG. If the design contains third-party OCCs, Tessent tools can set up and operate the OCC throughout the flow by reading and processing files that describe the operation of the OCC.

The following topics describe the process for defining and configuring third-party OCCs.

How to Configure Files for Third-Party OCCs	276
Test Logic Insertion	277
Scan Insertion	279
Pattern Generation	279
Pattern Simulation	282

How to Configure Files for Third-Party OCCs

In order to automate the flow as much as possible, the following files set up and control how the tool interacts with the OCC.

Solution

You must place the ICL and PDL files in the same directory as the Verilog of the OCC using the same file base name. For this example, the OCC Verilog module is in a file named *third_party_occ.v*.

Moving the ICL and PDL files into the same directory as the Verilog enables the tool to automatically load them and carry the information throughout the flow.

You must prepare the following files as described:

- **ICL** — Provide an ICL file based on the IEEE 1687 IJTAG standard to describe the ports on the OCC that need to be controlled by IJTAG during test. Name the file *third_party_occ.icl* and place it in the same directory as the Verilog file.
- **PDL** — Provide a PDL file based on the IEEE 1687 IJTAG standard to describe the procedures that configure the third-party OCC. Name the file *third_party_occ.pdl* and place it in the same directory as the Verilog file.
- **TCD Scan** — Provide a TCD Scan file based on the Tessent Core description language to describe the OCC's programming shift register (chain segment) that needs to be connected to the design's scan chains during scan insertion.

The following is an example TCD Scan file for a third-party OCC. The sub-chain port information must be included in the ScanChain wrapper and the Clock and ScanEn wrappers in order to define the polarity of each port.

```

Core(third_party_occ) {
  Scan {
    module_type           : occ;
    allow_scan_out_retiming : 0;
    is_hard_module         : 1;
    traceable              : 0;
    pre_scan_drc_on_boundary_only : 1;
    Clock(slow_clock) {
      off_state : 1'b0;
    }
    ClockOut(clock_out_mux/y) {
      slow_clock_input : slow_clock;
      fast_clock_input : fast_clock;
    }
    ScanEn(scan_en) {
      active_polarity : 1'b1;
    }
    ScanChain {
      length      : 4;
      scan_in_port : scan_in;
      scan_out_port : scan_out;
      scan_in_clock : slow_clock;
      scan_out_clock : ~slow_clock;
    }
  }
}

```

- **Clock Control Definitions** — Provide a test procedure file that contains Clock Control Definitions (CCDs) for the OCC instances in the design. For details on the format of CCDs, refer to [“Clock Control Definition”](#) on page 317.

Test Logic Insertion

During test logic insertion (for example, EDT), the tool reads and processes the ICL and PDL files and includes this information in design files stored in the TSDB. Additionally, any OCC ports described in the ICL file that need to be controlled by IJTAG are connected to the generated IJTAG network such that the OCCs can be easily configured in subsequent steps.

See [“How to Configure Files for Third-Party OCCs”](#) on page 276 for instructions on setting up the ICL and PDL files.

Solution

The following EDT example uses a post-insertion procedure to connect the `slow_clock` port of the OCC to the newly-generated `shift_capture_clock` DFT signal. This enables the same clock source to be used for the OCC and EDT logic. The post-insertion script is executed after the `process_dft_specification` command creates all other logic defined in the DFT Specification. For detailed information on how to create and use a post-insertion procedure, refer to [“process_dft_specification.post_insertion”](#) in the *Tessent Shell Reference Manual*.

```
set_context dft -rtl
set_tsdb_output_directory ../tsdb_outdir

# Read the design and OCC module. The ICL and PDL files with the
# same base name will be automatically read from the same directory
read_verilog ../rtl/RDS_process_with_occ.v
read_verilog ../rtl/third_party_occ.v

set_current_design RDS_process
set_design_level physical_block

# Define DFT Signals for EDT and scan test.
add_dft_signals scan_en edt_update test_clock -source_node \
    {scan_en_r edt_update test_clock_r}
add_dft_signals edt_clock shift_capture_clock -create_from_other_signals
set_dft_specification_requirements -logic_test on
add_clocks clock1 -period 3ns
add_clocks clock2 -period 4ns

check_design_rules

# Create the DFT Spec for adding EDT
set_spec [create_dft_specification -sri_sib_list {edt} ]
read_config_data -in $spec -from_string {
    EDT {
        ijtag_host_interface : Sib(edt);
        Controller (c1) {
            longest_chain_range : 50, 65;
            scan_chain_count : 60;
            input_channel_count : 2;
            output_channel_count : 2;
        }
    }
}

# Post-insertion procedure to change the existing connection to
# third-party OCC's slow_clock port and connect to the shift_capture_clock
# DFT Signal
proc process_dft_specification.post_insertion {RDS_process args} {
    set occ_insts [get_instances -of_module third_party_occ -silent]
    if {[sizeof_collection $occ_insts] > 0} {
        set slow_clock_pins [get_pins slow_clock -of_instances $occ_insts]
        foreach_in_collection occ_pin $slow_clock_pins {
            delete_connection $occ_pin
            create_connection [get_dft_signal shift_capture_clock] $occ_pin
        }
    }
}
process_dft_specification
extract_icl

# Create a synthesis script of all RTL (original and newly created)
write_design_import_script use_in_synthesis.tcl -use_relative_path_to . \
-replace
exit
```

Scan Insertion

You must configure the tool to read the OCC files before running scan insertion.

See “[How to Configure Files for Third-Party OCCs](#)” on page 276 for instructions on setting up the OCC files.

Solution

When inserting scan into the post-synthesis netlist, specify the location of the TCD Scan file using the `set_design_sources` command. The tool uses the description of the OCCs’ scan segment to stitch them into the design’s scan chains.

The following is an example:

```
set_context dft -scan
set_tsdb_output_directory ../tsdb_outdir

# Specify the location of the TCD Scan file that describes the OCC's scan
# segment
set_design_sources -format tcd_scan -Y ../rtl -extension tcd_scan

# Read the cell library and synthesized netlist from DC Shell
read_cell_library ../library/tessent/adk.tcelllib
read_verilog ../2.synthesis/RDS_process_synthesized.vg

# Use read_design to read in information (DFT signals, etc.) performed in
# previous pass
read_design RDS_process -design_identifier rtl -no_hdl
set_current_design RDS_process
set_system_mode analysis

# Add a scan mode and specify EDT instances to which scan chains should be
# connected
set edt_instance [get_instances -of_icl_instances [get_icl_instances \
                                                    -filter tessent_instrument_type==mentor::edt]]
add_scan_mode edt -edt_instance $edt_instance
analyze_scan_chains
insert_test_logic
exit
```

Pattern Generation

You must configure the tool to read the OCC files before generating patterns.

See “[How to Configure Files for Third-Party OCCs](#)” on page 276 for instructions on setting up the OCC files.

Solution

For stuck-at ATPG, much of the setup is automatically imported using the `import_scan_mode` command. Additional clock definitions and settings for the OCC should be provided for the OCC's asynchronous source clocks as well as the clocks on the output of the OCC instances.

```
set_context patterns -scan
set_tsdb_output_directory ../tsdb_outdir

# Read the cell library and design
read_cell_library ../library/tessent/adk.tcelllib
read_design RDS_process -design_id gate
set_current_design RDS_process

# Specify a test mode for stuck-at ATPG
set_current_mode edt_stuck

# Import scan, EDT, and clock configuration for ATPG
import_scan_mode edt

# Define external and OCC clocks
set_clock_options test_clock_r -pulse_in_capture on
set_occ_insts [get_instances -of_module third_party_occ* -silent]
if {[sizeof_collection $occ_insts] > 0} {
    foreach_in_collection occ_inst $occ_insts {
        set inst_name [get_single_name $occ_inst]
        add_clocks [get_pins ${inst_name}/clock_out_mux/y] -capture_only
    }
    # Execute iProc for third_party OCC to set default values (test_mode = 1)
    set_test_setup_icall "${inst_name}.setup" -append
}
set_system_mode analysis

# Specify a procedure file containing clock control definition for the OCC
# instances
read_procfile third_party_occ_clock_controls.proc
create_patterns
write_tsdb_data -replace
write_patterns patterns/RDS_process_stuck_parallel.v -verilog -parallel \
    -replace -parameter_list {SIM_KEEP_PATH 1}
set_pattern_filtering -sample_per_type 2
write_patterns patterns/RDS_process_stuck_serial.v -verilog -serial \
    -replace -parameter_list {SIM_KEEP_PATH 1}
```

In the ANALYSIS mode, specify a procedure file that contains the clock control definitions for the OCC instances.

For transition fault ATPG, a number of changes to the dofile are highlighted in the following example. Define any additional internal clocks that are needed for the third-party OCC, similar to those defined in the example (for example, if the OCC internally gates the fast clock during transition test).

Additionally, specify any parameters to the OCC's iCalls that must be set to configure the OCC for transition/fast-capture test.

Finally, you must constrain the design's I/O which are not used for transition test.

```
set_context patterns -scan
set_tsdb_output_directory ../tsdb_outdir

# Read the cell library and design
read_cell_library ../library/tessent/adk.tcelllib
read_design RDS_process -design_id gate
set_current_design RDS_process

# Specify a test mode for transition ATPG
set_current_mode edt_transition

# Import scan, EDT, and clock configuration for ATPG
import_scan_mode edt
import_clocks

# Define external and OCC clocks
set_clock_options test_clock_r -pulse_in_capture on
set_occ_insts [get_instances -of_module third_party_occ* -silent]
if {[sizeof_collection $occ_insts] > 0} {
    foreach_in_collection occ_inst $occ_insts {
        set inst_name [get_single_name $occ_inst]
        add_clocks [get_pins ${inst_name}/clock_out_mux/y] -capture_only
        add_clocks \
            [get_pins ${inst_name}/occ_control/cgc_SHIFT_REG_CLK/clkg] \
            -pulse_in_capture
    }
}

# Execute iProc for third_party OCC and enable fast capture mode for OCCs
set_test_setup_icall "${inst_name}.setup fast_capture_mode 1" \
    -append

}

add_input_constraints -hold
add_output_masks -all

set_system_mode analysis

# Specify a procedure file containing clock control definition for the OCC
# instances
read_procfile third_party_occ_clock_controls.proc

set_fault_type transition
set_external_capture_options -pll_cycles 5 [lindex [get_timeplate_list] 0]

create_patterns
write_tsdb_data -replace
write_patterns patterns/RDS_process_transition_parallel.v -verilog \
    -parallel -replace -parameter_list {SIM_KEEP_PATH 1}
set_pattern_filtering -sample_per_type 2
write_patterns patterns/RDS_process_transition_serial.v -verilog \
    -serial -replace -parameter_list {SIM_KEEP_PATH 1}
```

Pattern Simulation

You must run a Verilog simulation of the generated patterns to ensure no mismatches are reported and that the patterns function as expected during the tester application.

Solution

For parallel load patterns specified by the “write_patterns -parallel” command, simulate all the patterns.

For serial load patterns, a handful of patterns are sufficient since the run time for simulating serial load patterns can be significant.

Chapter 8

Test Procedure File

Test procedure files specify how the scan circuitry within a design operates. The scan circuitry operation is specified using previously-defined scan clocks and other control signals. To operate the scan circuitry in your design, you must define the scan circuitry and provide a test procedure file to describe its operation. The design rules checking (DRC) process, which occurs when you exit setup mode, performs extensive checking to ensure the scan circuitry operates correctly.

Test Procedure File Creation	284
Test Procedure File Syntax	284
Test Procedure File Structure	289
#include Statement	289
Set Statement	290
Alias Definition	293
Timing Variables	295
Timeplate Definition	298
Multiple-Pulse Clocks	304
Always Block	306
Procedure Definition	307
Clock Control Definition	317
The Procedures	326
Test_Setup (Optional)	327
Shift (Required)	330
Alternate Shift Procedure (Optional)	332
Load_Unload (Required)	334
Shadow_Control (Optional)	337
Master_Observe (Sometimes Required)	339
Shadow_Observe (Optional)	339
Seq_Transparent (Optional)	340
Clock (Optional)	342
Skew_Load (Optional)	343
Clock_run (Optional)	344
Capture Procedures (Optional)	346
Clock_po (Optional)	351
Ram_sequential (Optional)	351
Ram_passthru (Optional)	353
Clock_sequential (Optional)	353
Init_force (Optional)	354
Test_end (Optional, all ATPG tools)	355
Sub_procedure	356

Additional Support for Test Procedure Files	358
Creating Test Procedure Files for End Measure Mode	359
Serial Register Load and Unload for LogicBIST and ATPG	363
Register Load and Unload Use Models	363
Static Versus Dynamic Register Variables.....	363
Test Procedure File Modifications	364
Dofile Modifications.....	367
Serial Load and Unload DRC Rules.....	370
Notes About Using the stil2mgc Tool.....	376
Extraction of Strobe Timing Information from STIL (SPF).....	376
The STIL ClockStructures Block	376
Test Procedure File Commands and Output Formats.....	377

Test Procedure File Creation

You insert scan circuitry and create test procedure files for ATPG operations using the “patterns-scan” context of Tessent Shell. If your design already contains scan circuitry, you will need to create a test procedure file to describe its operation either by hand or with Tessent Shell.

You can specify a test procedure file in setup mode using the `add_scan_groups` command. The tools can also read in procedure files by using the `read_procfile` command or the `write_patterns` command when not in setup mode. When you load more than one test procedure file, the tool merges the timing and procedure data.

You can also have the **stil2mgc** tool translate a STIL Procedure File (SPF) into a dofile and test procedure file. This tool produces a dofile that defines clocks, scan chains, scan groups, and pin constraints. This tool also creates test procedure files with a timeplate and the following standard scan procedures: `test_setup`, `load_unload`, and `shift`. For more information about **stil2mgc**, refer to “[Notes About Using the stil2mgc Tool](#)” on page 376.

The following subsections describe the syntax and rules of test procedure files, give examples for the various types of scan architectures, and outline the checking that determines whether the circuitry is operating correctly.

Test Procedure File Syntax

The test procedure file uses common syntactical conventions such as bold and italic fonts, and reserved characters. The file supports Tcl conditional statements.

Syntax Conventions

The following syntax conventions are used in this chapter:

- **Bold** — Indicates a keyword. Enter the keyword exactly as shown.

- *Italic* — Indicates lexical elements such as identifiers, strings, or numbers. Replace the italicized word with the appropriate name or integer.
- | — A vertical bar or pipe character indicates a logical “OR” as in “select foo OR foo_not”.
- [] — Square brackets indicate optional elements. Do not include the brackets.
- ... — An ellipsis indicates a repeatable item or set.

Reserved Characters

If you have a pin or pathname that uses a reserved punctuation character, you must enclose that name in double quotes. See [Table 8-1](#) for a list of reserved punctuation characters.

For example, the following statement is illegal because it uses the exclamation point outside of double quotes.

```
force /inst_my_adder_1/xclk_header!x1!x1/op1[9] 1
```

The signal name contains a reserved punctuation character, the exclamation point (!), so it must be enclosed inside double quotes. The correct syntax would be:

```
force "/inst_my_adder_1/xclk_header!x1!x1/op1[9]" 1
```

Table 8-1. Reserved Punctuation Characters

Name	Character
Ampersand/AND	&
Caret/Circumflex/XOR	^
Comma	,
Equals	=
Exclamation mark	!
Left/Opening brace	{
Left/Opening parenthesis	(
Right/Closing brace	}
Right/Closing parenthesis	)
Semicolon	;
Vertical bar/OR	

Throughout this chapter, value = 0, 1, X, or Z.

Using Tcl in the Test Procedure File

Procedure files support the Tcl conditional statements “if”, “else”, and “elseif” using the following syntax:

```
if { tcl_expr } {  
    procedure file statements  
}  
elseif { tcl_expr } {  
    procedure file statements  
}  
else {  
    procedure file statements  
}
```

Where a “tcl_expr” is any Boolean Tcl expression that uses Tcl variables, dofile variables, or environment variables. Just as when doing variable substitution in the procedure file, other Tcl statements and defining Tcl variables are not supported. All variables must be defined in the dofile or from the shell as environment variables.

The body of these Tcl conditional statements should contain only legal procedure file syntax, not any other Tcl statements. The Tcl conditional statements are treated as preprocessor statements in the procedure file parser. They are not preserved in the tool after parsing is finished; only the procedure file code selected by the evaluation of the Tcl “if” expression is stored in the tool. Therefore, when using `write_procfile` to write out the procedure file, none of the Tcl conditional statements are present, and the procedure file code not used is also not present. For more information refer to “[The Tessent Tcl Interface](#)” on page 491.

Introductory Test Procedure File Example

The following is an example of a simple test procedure file.

```
// Comments use "//" characters
//
// Set the base time increment for use in all timeplates
set time scale 1.0 ns;

// Define the strobe time for the measure statements
set strobe_window time 1;

// This design uses a single timeplate, named "tpl", for all
// vectors.

timeplate tpl =
    force_pi 0;
    measure_po 1;
    pulse CLK0_7 2 1;
    pulse CLK8_15 2 1;
    period 4;
end;

// The shift and load_unload procedures define how the design
// must be configured to allow shifting data through the scan
// chains. The procedures define the timeplate that will be
// used and the scan group that it will reference.

procedure shift =
    scan_group grp1;
    timeplate tpl;
    cycle =
        force_sci;
        measure_sco;
        pulse CLK8_15;
        pulse CLK0_7;
    end;
end;

procedure load_unload =
    scan_group grp1;
    timeplate tpl;
    cycle=
        force CLEAR 0;
        force CLK0_7 0;
        force CLK8_15 0;
        force scen1 1;
    end;
    apply shift 8;
end;

// The capture procedure is a "non-scan" procedure. This
// procedure describes the timeplate that will be used for the
// capture cycle. It also defines the number of cycles that
// will be used in the capture cycle. In this example there is
// just one cycle.

procedure capture =
    timeplate tpl;
    cycle =
        force_pi;
        measure_po;
```

```
        pulse_capture_clock;  
    end;  
end;
```


Test Procedure File Structure

The test procedure file consists of many structural elements that display in a specific order, starting with `#include` statements. Some of these elements are required and others are optional.

```
#include "<file_name>";
[set_statement ...]
[alias_definition ...]
[timing_variables ...]
timeplate_definition           // includes pulse clock statements
always_block
procedure_definition
clock control definition
```

#include Statement	289
Set Statement	290
Alias Definition	293
Timing Variables	295
Timeplate Definition	298
Multiple-Pulse Clocks	304
Always Block	306
Procedure Definition	307
Clock Control Definition	317

#include Statement

The “`#include`” statement specifies that the tool read test procedure data from a specified file.

The following rules apply to `#include` statements and files:

- The “`#include`” statement can occur anywhere in the file, and multiple “`#include`” statements can occur in one file. For example:

```
#include "foo.proc";
```
- The filename to be included must be enclosed in double quotes, and the statement must be followed by a semicolon.
- All timeplates and procedure rules apply to the statements placed in `#include` files.
- Included files can use the “`#include`” statement to include other files, up to a maximum include depth of 512. If you later use the `write_procfile` command write out procedure data, the “`#include`” statements are not preserved, and the tool writes all procedure data to a single file.

Set Statement

The Set statements define specific parameters used throughout the test procedure file.

The following statements are available:

```
set time scale tscale;  
set strobe_window time window_width;  
set default_timeplate timeplate_name;  
set autoforce off;
```

set time scale Statement

Defines the time scale and unit. The “set time scale” statement must be at the beginning of the procedure file, before any timeplate or procedure definition. If you do not specify the time scale, the default value is 1 ns.

The tool applies the time scale and unit to the test procedure file and timeplates. The *tscale* you specify can be any real number. Time values in the timeplate, however, must be integers, representing whole time scale units. If you find you are specifying fractional times in the timeplate, you must reduce the time scale unit so you can specify integer time values in the timeplate. For example, the following would result in a syntax error:

```
set time scale 1 ns ;  
set strobe_window time 1 ;  
  
timeplate fast_clk_tp =  
    force_pi 0 ;  
    measure_po 0.500 ;  
    pulse CLKA 0.750 1.50 ;  
    pulse CLKB 0.750 1.50 ;  
    period 3.000 ;  
end ;
```

To correct the syntax, you could change the time scale to picoseconds, and adjust the time value to meet the scale as follows:

```
set time scale 10 ps ;  
set strobe_window time 1 ;  
  
timeplate fast_clk_tp =  
    force_pi 0 ;  
    measure_po 50 ;  
    pulse CLKA 75 150 ;  
    pulse CLKB 75 150 ;  
    period 300 ;  
end ;
```

The units supported are ms, us, ns, ps, and fs.

The tool translates the time scale in the procedure file into a Verilog ‘timescale directive in the Verilog testbench when writing patterns in Verilog format.

If the time scale number you specify in the test procedure file is 1 or larger, the resulting Verilog ‘timescale directive has the same time unit (resolution) and time precision. For example, “set time scale 1 ns ;” would result in this Verilog directive:

```
`timescale 1ns / 1ns
```

If you want the testbench to have smaller precision than resolution, there are several ways to designate this:

- Specify a time scale number of less than 1 in the procedure file. For example, “set time scale 0.5 ns ;” produces this Verilog directive:

```
`timescale 1ns / 100ps
```

- Add *non-zero* significant bits to the time scale in the procedure file. For example, “set time scale 10.05 ns ;” produces this Verilog directive:

```
`timescale 1ns / 10ps
```

- Add trailing zeros as significant bits for an asynchronous clock period or pattern_set period (when creating IJTAG patterns). For example, an “add_clocks -period 10.00ns” command produces this Verilog directive:

```
`timescale 1ns / 10ps
```

- Use the SIM_PRECISION parameter file keyword. For example, “SIM_PRECISION 0.5ns;” produces this Verilog directive:

```
`timescale 1ns / 100ps
```

The precision in the Verilog testbench can originate from any of the previous sources, and the tool uses the smallest specified precision when writing out the Verilog testbench.

The resolution in the Verilog testbench originates from the procedure file. When you use multiple procedure files, the various “set time scale” statements can specify different values, and the tool uses the smallest specified resolution when writing the Verilog testbench.

set_strobe_window time Statement

Defines the strobe-window width. The “set_strobe_window time” statement must be at the beginning of the procedure file, before any timeplate or procedure definition. If you do not specify the strobe_window time, it will default to the maximum allowable size. For example, if you look at a timeplate, and if there is a 10 ns window between the measure_po event and the next event (or end of timeplate), then that is what the strobe window will be. If there are multiple timeplates, then the smallest strobe window from the timeplates is the maximum allowable strobe window.

Some tester formats measure primary outputs (POs) at the exact time that you specify with the measure_po statement in the timeplate. However, other tester formats, such as STIL, require

that output measurements occur during a specified window of time (*window_width*). For WGL, this statement changes the strobe window in the output file.

A strobe window can only stretch from the *measure_po* time to the end of the cycle or the next force or pulse event. For example, if you issue a *measure_po* at time 10 and the rising edge of a pulse at time 30, the strobe window can only be a maximum of 20. *Strobe_window* lets you know that, starting at the *measure_po* time, the primary output should be stable for the time specified by the strobe window.

Note



Strobe_window only affects the following formats: STIL, TSTL2, and WGL.

set default_timeplate Statement

Specifies a timeplate that can be used for any procedure definition that does not explicitly specify a timeplate. The referenced *timeplate_name* must be defined prior to the Set Default_timeplate statement in the procedure file.

set autoforce Statement

An optional statement that controls the behavior for automatically adding force events. If included, this statement must appear at the beginning of the procedure file, prior to any procedures.

By default (without this statement), if a constrained pin is not forced to the constrained value in the *test_setup* procedure, a force event is automatically added to the first cycle of the *test_setup* procedure. If the *test_setup* procedure starts with a call to a subprocedure, then the force event is added to the first cycle following the subprocedure. If a force event already exists in the *test_setup* procedure for a constrained pin, then no additional force event will be added for that pin.

When you include the “set autoforce off” statement, the tool will not add any force events on the constrained pins to the test setup procedure.

Including the “set autoforce off” statement also prevents the tool from forcing clock pins to the inactive state at the beginning of the *load_unload* procedure. The statement also disables copying the last value forced on an input port in *test_setup* and prevents it from becoming a force at the start of the *load_unload* procedure.

The “set autoforce off;” statement also turns off auto forcing Z values on bidis in the *load_unload* procedure—see [Load_Unload \(Required\)](#).

Alias Definition

The Alias definition groups multiple signal names or cell paths into a single alias name. Signal Alias statements are useful in procedures or timeplates where multiple signals need to be assigned to the same value at the same time.

Cell Alias statements are used to group cell paths into a single alias name. You must define aliases before using them. The definition can occur at any place in the procedure file outside of a timeplate or procedure definition.

Note



When saving STIL2005, CTL, or Structural_STIL patterns, all aliases defined in the procedure file will be defined as SignalGroups in the resulting STIL file.

There is a predefined alias available for specifying all bidirectional pins. The “_ALL_BIDI” keyword may be useful for forcing all bidirectional pins to a specified value without having to identify each individual pin. For example:

```
force _ALL_BIDI Z;
```

In using a cell Alias statement to group cell paths from condition statements into a single alias name, it is possible to override a condition statement in a named capture procedure with a subsequent condition statement that occurs in the same place (global condition, or local to a specific cycle). A condition statement can only override a previous condition if the first condition is specified using an alias name, and if the second condition is specified without using an Alias statement.

Tip



When using multiple named capture procedures where each procedure requires many condition statements, it is helpful to group cells into a common name and apply the condition statement once to the entire group of cells, and then override specific cells that need a different value than what was applied to the group. This frees you from having to enter numerous condition statements for each named capture procedure, while only a handful of the cells require different values for each procedure.

The Alias definition has the following format:

```
alias alias_name = pin_name [, pin_name ...];
```

or

```
alias alias_name = cell_name [, cell_name ...];
```

- ***alias_name***

A string that specifies the name of the alias.

- ***pin_name***

A repeatable string that specifies the pin name to associate with the alias name.

- ***cell_name***

A repeatable string that specifies the cell name to associate with the alias name.

Alias Examples

This example groups two signal names into a single alias name.

```
alias my_group = T, U;
```

This next example shows how a named capture procedure should look when using an Alias statement for condition cells. The example sets each of four cells to a value of 0, and then the fourth cell (/inst_3/blockb/reg_2/Q) is overridden with a value of 1.

```
alias cond_cells = "/inst_0/blocka/reg_1/Q", "/inst_1/blocka/reg_1/Q",  
                  "/inst_2/blocka/reg_1/Q", "/inst_3/blockb/reg_2/Q";  
timeplate tp1 =  
    force_pi 0;  
    measure_po 10;  
    pulse ref_clk 50 50;  
    period 100;  
end;  
procedure capture capture1 =  
    timeplate tp1;  
    condition cond_cells 0;  
    condition /inst_3/blockb/reg_2/Q 1;  
    // overrides condition in previous statement  
    cycle =  
        force_scan_en 0;  
        force_ctrl_a 1;  
        force_pi;  
        pulse ref_clk;  
    end;  
    cycle =  
        force_pi;  
        measure_po;  
        pulse ref_clk;  
    end;  
end;
```

This example shows how to define a user-defined bus in a procedure file:

```
alias wdata = "D[0]", "D[1]", "D[2]", "D[3]";
```

And this shows how to assign a value to that user-defined bus:

```
procedure sub_procedure subpro1 =  
    scan_group grp1 ;  
    timeplate shift_tp ;  
    cycle =  
        force wdata 0010 ;  
        pulse ref_clk ;  
    end;  
end;
```

Timing Variables

Two timing variable block definitions allow a procedure file to express timing using variables and equations, and to have this equation-based timing preserved in the tool and reproduced in the correct syntax in pattern output files.

Test data languages such as WGL and STIL have the ability to express time values in the timing blocks as numerical values or as equations based on variables. Using equation-based timing allows one value to be specified for a global attribute, such as the test cycle period, while other values are derived from this using equations.

The two timing block definitions are called “timing_variables” and “variables”. In the “timing_variables” block, variables can be defined and assigned timing values. These values will be expressed in the time scale which is already specified by the Set Time Scale statement. The “timing_variables” block must be defined before the timeplate definitions.

The “variables” block is used to define variables that are not time values and have no units associated with them. These variables can only be assigned integer numbers, and can be used as scaling multipliers in the timing equations.

The variables in the “timing_variables” block can also be assigned timing equations instead of time values. These equations are simple mathematical equations which can use either timing values or previously defined *variables* or *timing variables* as operands.

Note



The event statements in the timeplate definition block accept timing values and timing variables.

When saving patterns in the Verilog, WGL, and STIL supported formats, the waveform tables in these formats will be written using the equations and variables, and the variables will be defined in the appropriate definition blocks which exist in each format. When saving patterns in formats that don't support equation-based timing, the equations will be computed and the timing information will be specified as the resulting numeric values in the pattern file. Setting the ALL_FLATTEN_TIMING parameter file keyword to 1 will cause Verilog, WGL, and STIL outputs to compute the timing equations and use only the resulting numeric values in the output

files. Any equation that does not compute to an integer value will be rounded to the nearest integer value.

The “timing_variables” block has the following syntax:

```
variables =  
    variable_name = integer;  
    [variable_name = integer; ...]  
end;  
  
timing_variables =  
    variable_name = time_or_equation;  
    [variable_name = time_or_equation; ...]  
end;
```

Note that *time_or_equation* can either be an integer time value or a time equation. A time equation is expressed using operators and operands. An operator is one of +, -, *, or /. An operand can be a time value or a variable name (time or scaling variable). The multiplication and division operators (* and /) take precedence over the addition and subtraction operators (+ and -). You can use parenthesis to group operations for precedence.

Note



In the timeplate definitions, any place where a time value can be used, a timing variable is also allowed. A scaling variable from the “variables” block cannot be used in a timeplate definition. These can only be used in time equations.

Variable names can be any identifier except for reserved keywords used in the procedure file syntax (such as “period” and “force_pi”). The variable names must conform to the rules that apply to all identifiers used in the procedure file (alpha numeric string, starting with an alpha character, and no reserved punctuation marks). If reserved characters or reserved words are used in a variable name, the name must be enclosed in quotes.

Equation-Based Timing Example

The following is a partial example of using equation-based timing.


```
set time scale 1.0 ns;
variables =
    v_scale = 1;
end;

timing_variables =
    t_period = 100;
    t_force = 0;
    t_meas = ((t_period * 0.1) * v_scale );
    t_rise = ((t_period / 2) * v_scale );
    t_width = ((t_period * 0.2) * v_scale);
end;

timeplate tp1 =
    force_pi t_force;
    measure_po t_meas;
    pulse ref_clk t_rise t_width;
    period t_period;
end;
```

This is how the timing example above would be represented in the STIL output:

```
Spec STUCK_spec {
    Category STUCK_cat {
        v_scale = '1';
        t_period = '100ns';
        t_force = '0ns';
        t_meas = '(t_period*0.1)*v_scale';
        t_rise = '(t_period/2)*v_scale';
        t_width = '(t_period*0.2)*v_scale';
    }
}

Timing STUCK_timing {
    WaveformTable tset_tp1 {
        Period 't_period';
        Waveforms {
            input_time_gen_0 { 01 { 't_force' D/U; }}
            input_time_gen_1 { 01 { '0ns' D; 't_rise' D/U;
                                   't_rise+t_width' D;}}
            _po_ { LHX { '0ns' X; 't_meas' 1/h/x; 't_rise' X;}}
        }
    }
}
```

This is how the timing example would be represented in the WGL output:

```
equationsheet STUCK_sheet
  exprset STUCK_set
    v_scale := 1.0;
    t_period := 100nS;
    t_force := 0nS;
    t_meas := (t_period * 0.1) * v_scale;
    t_rise := (t_period / 2) * v_scale;
    t_width := (t_period * 0.2) * v_scale;
    _tp1_fall_1 := t_rise + t_width;
  end
end

timeplate tp1 period t_period
  "input_a" := input [t_force:S];
  ...
  "output_z" := output [0nS:X, t_meas:Q, t_rise:X];
  ...
  "refclk" := input [0nS:D, t_rise:S, _tp1_fall_1:D];
end
```

Timeplate Definition

The timeplate definition describes a single tester cycle and specifies where in that cycle all event edges are placed.

You must define all timeplates before they are referenced. A procedure file must have at least one timeplate definition. All clocks must be defined in the timeplate definition. The timeplate definition has the following format:

```
timeplate timeplate_name =
  timeplate_statement
  [timeplate_statement ...]
  period time;
end;
```

The following list contains available timeplate_statement statements. The timeplate definition should contain at least the force_pi and measure_po statements. You are not required to include pulse statements for the clocks. But if you do not “pulse” any of the clocks, the tool uses two cycles to pulse a clock, resulting in larger patterns.

Note that the tool uses the pulse_clock statement rather than individual pulse statements when generating default procedures.

```
timeplate_statement:  
  offstate pin_name off_state;  
  force_pi time;  
  bidi_force_pi time;  
  measure_po time;  
  bidi_measure_po time;  
  force pin_name time;  
  measure pin_name time;  
  pulse pin_name time width [, time width];  
  pulse_clock time width [, time width];
```

Note

 In “timeplate_statement” definitions, you can use timing variables instead of time values. For more information, refer to “[Timing Variables](#)” on page 295.

The following is a list of statements in the timeplate definition:

- ***timeplate_name***

A string that specifies the name of the timeplate.

- ***offstate pin_name off_state***

A literal and double string that specifies the inactive, off-state value (0 or 1) for a specific named primary input pin that will be pulsed within this timeplate but is not defined as a clock pin by the add_clocks command. The complex timeplates are most useful in the shift procedure where a non-clock pin must be pulsed while still maintaining a single cycle in the shift procedure.

This statement must occur before all other timeplate_statement statements. This statement is required for any pin that is not defined as a clock pin by the add_clocks command but will be pulsed within this timeplate.

Note

 An “offstate” statement does not automatically force *pin_name* to its off state at time 0. For that to occur, you must force or pulse *pin_name* appropriately in a procedure.

- ***force_pi time***

A literal and integer pair that specifies the force time for all primary inputs.

- ***bidi_force_pi time***

A literal and integer pair that specifies the force time for all bidirectional pins. This statement allows the bi-directional pins to be forced after applying the tri-state control signal, so the system avoids bus contention. This statement overrides “force_pi” and “measure_po”.

- ***measure_po time***

A literal and integer pair that specifies the time at which the tool measures (or strobes) the primary outputs.

- ***bidi_measure_po time***

A literal and integer pair that specifies the time at which the tool measures (or strobes) the bidirectional pins. This statement overrides “force_pi” and “measure_po”.

- ***force pin_name time***

A literal, string, and integer that specifies the force time for a specific named pin.

Note



This force time overrides the force time specified in force_pi for this specific pin.

- ***measure pin_name time***

A literal, string, and integer that specifies the measure time for a specific named pin. You can use a “measure” statement in timeplates only to specify a measure time for a pin.

Note



This measure time overrides the measure time specified in measure_po for this specific pin.

- ***pulse pin_name time width*** [, *time width*]...

A literal, string, and repeatable integer set that specifies the pulse timing for a specific named pin.

pin_name — String that refers to a pin in the design. Valid pins must meet one of the following conditions:

Defined as a clock pin using the add_clocks command.

Not defined as a clock pin, but has a pulse signal and an offstate specified by the “offstate” statement.

time — Integer that defines the offset from time 0 to the leading edge of the pulse.

width — Integer that defines the width of the pulse.

To define a multiple-pulse waveform, include multiple time and width pairs separated by a comma.

All pulses must occur within the tester cycle period. You define this period using the “period” keyword.

Note

 Multiple pulses are only supported for the following output formats: Verilog, WGL, STIL, STIL2005, CTL, FJTDL, MITDL, and TSTL2. Additionally, the TSTL2 output format does not support more than two pulses.

For MITDL format there is restriction that multiple pulse timing must be a cyclical repetition of the first pulse. Consequently, multi-pulse and double-pulse timing in the procedure file only works in the MITDL output without an error if the timing fits the restrictions of the MITDL syntax.

- **pulse_clock** *time width* [, *time width*]...

A literal and integer set that specifies the pulse timing for all signals defined as clocks, unless another statement such as a “force” or “pulse” exists for a particular clock signal. This is similar to the `force_pi` statement, which specifies the timing for ports that are not explicitly overridden by a force statement for those specific ports.

time — Integer that defines the offset from time 0 to the leading edge of the pulse for the first pulse. For subsequent edges in a multi pulse clock, time equals the time of the previous leading edge plus the period of the clock.

width — Integer that defines the width of the pulse.

You can use the **pulse_clock** statement to ensure that any added clocks (especially internal clocks) automatically have defined timing.

You may have multiple `pulse_clock` statements within a timeplate definition, as long as each statement has a different number of offset and width pairs. If two **pulse_clock** statements in the timeplate definition have the same number of offset and width pairs, the tool issues an error.

For more information on multiple-pulse clocks, see “[Multiple-Pulse Clocks](#).”

All pulses must occur within the tester cycle period. You define this period using the “period” keyword.

For example (1x pulse_clock):

```
timing_variables =
  tester_period = 10;
  strobe_1 = (0.96 * tester_period);
  t_time = (0.25 * tester_period);
  t_width = (0.5 * tester_period);
end;

timeplate tesseract_ijtag =
  force_pi 0 ;
  measure_po strobe_1;
  force tck 0;
  pulse_clock t_time t_width;
  period tester_period;
end;
```

- **period *time***

A literal and integer pair that defines the period of a tester cycle. This statement ensures that the cycle contains sufficient time, after the last force event, for the circuit to stabilize. The time you specify should be greater than or equal to the final event time.

Example 1

```
timeplate tp1 =  
    force_pi 0;  
    pulse T 30 30;  
    pulse R 30 30;  
    measure_po 90;  
    period 100;  
end;
```

Example 2

The following example shows a shift procedure that pulses b_clk with an off-state value of 0. The timeplate tp_shift defines the off-state for pin b_clk. The b_clk pin is not declared as a clock in the ATPG tool.

```
timeplate tp_shift =  
    offstate b_clk 0;  
    force_pi 0;  
    measure_po 10;  
    pulse clk 50 30;  
    pulse b_clk 140 50;  
    period 200;  
end;  
  
procedure shift =  
    timeplate tp_shift;  
    cycle =  
        force_sci;  
        measure_sco;  
        pulse clk;  
        pulse b_clk;  
    end;  
end;
```

Example 3

In the following example, the pin b_clk is not declared as a clock in the ATPG tool. However, in the shift procedure, the user needs the pin to be pulsed twice with an offstate of 0.

```
timeplate tp_shift =
  offstate b_clk 0;
  force_pi 0;
  measure_po 10;
  pulse clk 50 30;
  pulse b_clk 40 50, 140 50;
  period 200;
end;

procedure shift =
  timeplate tp_shift;
  cycle =
    force_sci;
    measure_sco;
    pulse clk;
    pulse b_clk;
  end;
end;
```

Multiple-Pulse Clocks

You can use the **pulse_clock** statement to handle multiple-pulse clock timing and still use a single generic timeplate template definition in various flows. The **pulse_clock** statement handles multiple-pulse timing definitions in the same manner as the **pulse** statement.

The pulse_clock Statement	304
Inferred Timing	305
Differences Between Default add_clock and 1x Multiplier Clock.	306

The pulse_clock Statement

In a timeplate statement, each **pulse_clock** statement has a different number of integer pairs for the offset and width of the clocks. The different statements represent 1x, 2x, 3x, and so on, pulse timing for clocks. The frequency multiplier that you specify for the clocks identifies the appropriate pulse timing that will be used for them.

When you specify a multiplier for a clock and no **pulse_clock** statement exists for that clock, the tool creates inferred timing for the clock using the timeplate period. The tool issues an error when a port-specific **pulse** or **force** statement exists for the clock and the timing in the statement does not specify the correct number of pulse statements to match the frequency multiplier of the clock.

The following example has multiple **pulse_clock** statements. It is a timeplate that includes both a port specific **pulse** statement for a clock and the generic **pulse_clock** statements for all other clocks. The first **pulse_clock** statement sets the default clock timing for this timeplate and contains a single pair of numbers (offset and width). Clocks other than “SlowClockA” that do not have frequency multipliers use this timing. Clocks with 2x multipliers use the second **pulse_clock** statement, and clocks with 4x multipliers use the third **pulse_clock** statement. The tool uses inferred timing for clocks specified with any other frequency multiplier and issues an error if "SlowClockA" has a frequency multiplier of 2x or more.

```
timeplate tp1 =  
    force_pi 0;  
    measure_po 95;  
    pulse SlowClockA 20 50;  
    pulse_clock 30 30;  
    pulse_clock 13 25, 63 25;  
    pulse_clock 6 12, 31 12, 56 12, 81,12;  
    period 100;  
end;
```

The following example shows a timeplate with one port-specific pulse and one **pulse_clock** statement for 4x multiplier timing. Clocks other than “ClockA” that do not have a frequency multiplier use force_pi timing. Inferred timing only occurs when clocks have frequency multipliers, which means other clocks still use NRZ timing when you use multiple cycles to create the clock pulse.


```
timeplate tp1 =
  force_pi 0;
  measure_po 95;
  pulse ClockA 20 50;
  pulse_clock 6 12, 31 12, 56 12, 81, 12;
  period 100;
end;
```

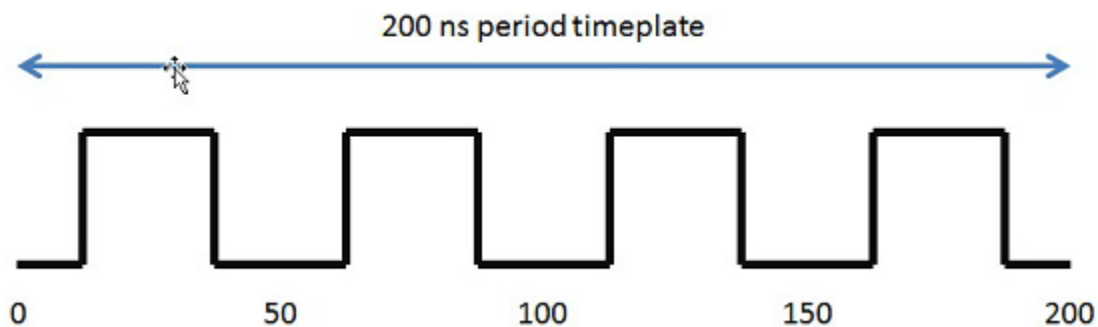
Inferred Timing

Based on the period of the timeplate, the tool creates inferred timing for frequency multiplied clocks when there are no **pulse_clock** statements in the timeplate with the correct number of clock edges for the clock.

The total period of a clock pulse is the period of the timeplate divided by the frequency multiplier of the clock. The offset for the leading edge of the first clock pulse is one-fourth of the period of the clock. For example, the inferred timing for a 4x clock in a timeplate with a period of 200 is equivalent to the following pulse_clock definition:

```
pulse_clock 13 25, 63 25, 113 25, 163 25
```

Figure 8-1. 200ns Timing Waveform



The first edge is one-fourth the period of the clock rounded to the nearest integer, and the width of the pulse is one-half the period of the clock. Each subsequent edge is the previous leading edge plus the period of the clock.

The tool bases the inferred timing for a 1x frequency-multiplied clock on any other clock timing found for a single pulse clock; otherwise, it will use the inferred timing formula.

When you specify a timeplate period and a clock frequency multiplier that combine to create inferred timing that is too small to be integer timing, the tool automatically reduces the procedure file timescale and scales all timing numbers to larger values.

For example, suppose the timescale for the procedure file is 1ns, the period of the timeplate is 5, and the clock frequency multiplier is 10. The tool automatically adjusts the timescale to 100ps and the period of the timeplate becomes 50. It adjusts all of the other numbers accordingly. In this case, the tool multiplies them by 10.

Differences Between Default add_clock and 1x Multiplier Clock

Default add_clock clocks use the time specified by **pulse_clock statements** with one pulse or by pin-specific **pulse** statements. However, frequency-multiplied clocks use the timing specified for them only if they match the number of pulses. If this is not the case, the tool uses inferred timing and issues an error if there are mismatches.

Clocks specified with a frequency multiplier of one behave differently than default clocks. For this reason, you must specify the timing for a 1x frequency-multiplied clock with a single clock pulse. The tool does not use NRZ timing with multiple cycles or forced timing edges, and it issues an error if the timeplate specifies a double-pulse or multiple-pulse timing for the 1x clock.

Always Block

This optional block definition specifies events that happen in all cycles of all procedures. Because the always block specifies events for all cycles, it will be used with all timeplates and does not require a timeplate to be referenced in the block. Also, any signal that is pulsed in the always block must have a pulse waveform in all timeplate definitions.

If you defined any pulse-always clocks using the **add_clocks** command, an always block is automatically created in the procedure file, if one does not already exist, and a pulse statement added for each clock. Similarly, if you pulse a clock signal in the always block, the signal is automatically defined as a pulse-always clock. For more information, refer to the [add_clocks](#) description in the *Tessent Shell Reference Manual*.

Note



Pulse-always clocks are not automatically pulsed in a named capture procedure. The clocks must be pulsed explicitly.

All events specified in the always block will be subject to rules checks that apply to each procedure. In other words, the events in the always block will be added to each cycle of each procedure, and all DRC rules still apply to these events.

When saving patterns that preserve the structure of the procedures as macros (such as the CTL pattern file, or structural STIL pattern file), the events in the always block will be placed in the cycles of each procedure. The always block will not be present in the structural pattern file as a macro or procedure.

Always Block Syntax

The always block has the following syntax.

```
always =  
    always_statement ;  
    [always_statement ; ... ]  
end ;
```

The *always_statement* is defined as one of the following.

```
pulse pin_name ;  
force pin_name value ;
```

Always Block Example

The following is a partial example of an always block.

```
set time scale 1,0 ns ;  
  
timeplate tp1 =  
    force_pi 0 ;  
    measure_po 10 ;  
    pulse ref_clk 20 20, 60 20 ;  
    pulse shift_clk 50 20 ;  
    period 100 ;  
end ;  
always =  
    pulse ref_clk ;  
end ;  
  
procedure shift =  
    timeplate tp1 ;  
    cycle =  
        force_sci ;  
        measure_sco ;  
        pulse shift_clk ;  
    end ;  
end ;
```

Procedure Definition

The procedure definition is the heart of the procedure file. The procedure defines precisely how the scan circuitry operates.

All procedure definitions contain one or more *cycle* definitions. Each cycle definition in the procedure specifies a vector; each statement in the cycle specifies which events occur in that vector. The timeplate being used specifies any timing associated with that vector. The following is a list of rules for writing procedure definitions:

- If more than one timeplate is defined, you can assign a specific timeplate for each procedure definition or for each cycle within the procedure definitions. You must assign a timeplate at some point within a procedure definition.
- You must group all procedure statements, except `scan_group`, `timeplate`, `apply`, and `loop`, into cycle statements.

- You cannot specify time values in cycle statements.
- You cannot specify loop statements within cycle statements.
- The order of events within a cycle definition does not matter. The assigned timeplate specifies the order.
- Within a procedure definition, you can specify a scan group.
- Each scan group needs a unique test procedure file. You associate the test procedure file with the scan group when you specify the [add_scan_groups](#) command.
- Text following “//” is a comment and is ignored.
- You can include blank lines.
- You define a procedure type for a particular scan group (with the exception of the seq_transparent and clock procedures) only once in a test procedure file.
- You can only have a single test_setup procedure, even if you define multiple scan groups for your design.

The procedure definition has the following general format, but note that certain statements are restricted to certain procedures.

```

procedure procedure_type [proc_name] =
    [scan_group scan_group_name;]
    proc_statement [proc_statement ...]
end;

proc_statement:
    [timeplate timeplate_name;]
    cycle =
        cycle_statement [cycle_statement ...]
    end;
    annotate "quoted string";
    apply proc_name #times;
    loop loop_count =
        cycle =
            cycle_statement [ cycle_statement ...]
        end;
    end;

    cycle_statement:
        force_pi;
        bidi_force_pi;
        force_sci;
        force_sci_equiv;
        measure_po;
        bidi_measure_po;
        measure_sco;
        restore_pi;
        restore_bidi;
        bidi_force_off;
        pulse_capture_clock;
        force_capture_clock_on ;
        force_capture_clock_off ;
        pulse_read_clock;
        pulse_write_clock;
        force pin_name value;
        expect pin_name value;
        condition cell_name value;
        measure pin_name;
        initialize instance_name [value];
        pulse pin_name;
        timeplate timeplate_name;
        annotate "quoted string";

```

- ***procedure_type***

A string that specifies the type of procedure that follows. The following list contains valid procedures types:

- | | | |
|------------------|--------------------|-------------------|
| • test_setup | • capture | • seq_transparent |
| • shift | • clock_po | • test_end |
| • load_unload | • ram_sequential | • clock |
| • shadow_control | • ram_passthru | • sequential |
| • master_observe | • clock_sequential | • skew_load |

- `shadow_observe`
- `init_force`
- `sub_procedure`

For more information, refer to “[The Procedures](#)” on page 326.

- *proc_name*

An optional string that specifies the user-defined name of the procedure. Since you can specify multiple `seq_transparent` and `clock` procedures in a test procedure file, these procedure types require explicit procedure names, *proc_name*, for each procedure that you define.

- **scan_group** *scan_group_name*

A literal and string pair that specifies a scan group within a scan procedure. Since some of the scan procedures are scan group specific, you can specify scan groups within scan procedures. This makes it possible to define the scan procedures (`shift`, `load_unload`) for multiple scan groups within the same procedure file. You can then specify this file on the [add_scan_groups](#) command for each scan group in this file. If you use the [read_procfile](#) command to read a procedure file, you must include this statement. However, if you use the `add_scan_groups` command, this statement is optional since the group is specified on the command line. When the tool writes out a procedure file, it produces the `scan_group` statement.

Note



The `scan_group_name` argument is case-sensitive if the netlist used is case-sensitive.

- **timeplate** *timeplate_name*

A literal and string pair that specifies the name of the timeplate the procedure uses.

A timeplate statement at the beginning of the procedure, outside of the cycle definitions, is the timeplate used by the entire procedure, if no other timeplates are referenced.

A timeplate statement within a cycle is the timeplate used for that cycle and all other subsequent cycles until another timeplate statement is encountered. For more information about timeplates, refer to “[Timeplate Definition](#)” on page 298.

- **annotate** “*quoted string*”;

A literal and string pair that reports the Verilog testbench annotations during simulation.

The `annotate` statement is optional and must always include a quoted string. All procedures can be annotated, including sub-procedures. The `annotate` statement can occur inside or outside of cycle blocks, including before the first cycle or after the last cycle.

This example shows the annotate statement in the beginning of a cycle along with the cycle timeplate statement, before any event statements:

```
CYCLE =
  [ TIMEPLATE tp_name ; ]
  [ ANNOTATE "quoted string" ; ]
  event_statement;
...
END ;
```

The following is an example of using the annotate statement outside of a cycle block:

```
procedure test_setup =
  timeplate tp1 ;
  annotate "Before first cycle" ;
  cycle =
    ...
  end;
  annotate "start sub procedure" ;
  apply mySub 1 ;
  ...
end;
```

The following is an example of an annotate statement used in a test_setup procedure, and how this will appear in a STIL pattern file.

```
Procedure test_setup =
  timeplate tp1;
  cycle =
    annotate "first cycle in test_setup" ;
    force reset 1;
    force clock 0;
  end;
  cycle =
    annotate "next annotation" ;
    force reset 0;
  end;
...
```

This is a segment of the resulting STIL pattern file:

```
W tset_tp1;
V { _pi_ = 0X11XXX;
    _po_ = XXXX;
}
Ann {* Begin chain test *}
Ann {* first cycle in test_setup *}
V { ...
}
Ann {* next annotation *}
V { ...
}
```

- label ***"quoted_string"*** ;

A literal and string pair for including pattern labels in saved patterns. As with the annotation statement, you can have one label statement per cycle in a procedure definition. The quoted_string becomes a pattern label for the vector that corresponds to that procedure cycle.

Only the STIL and WGL pattern formats support a pattern label statement. For pattern file formats that don't support a pattern label, the label is present as an annotation statement that has the string "label:" added at the beginning of the label string.

For the simulation testbench, the label is also present as an annotation that has the string "label:" added at the beginning, and the annotation is echoed when the patterns are simulated. You can use the existing parameter file keyword SIM_ANNOTATE_QUIET to turn off echoing the annotations and labels while simulating.

Each pattern label is a unique identifier, with its vector count appended to the end of the label string.

This statement can be used at the start of any cycle, just like an annotation statement. A cycle cannot contain both a label and an annotation statement.

The following example shows how to use the label statement within the test_setup procedure:

```
procedure test_setup =
  timeplate my_tp;
  cycle =
    force my_sig 0;
  end;
  cycle =
    label "end of test_setup" ;
    force my_sig 1;
  end;
end;
The previous example produces the following STIL vectors:
V { _pi_ = ...;
}
"end of test_setup_1": V { _pi_ = ...;
}
```

- **apply *proc_name* #times**

A literal and double-argument string that tells the tool to apply the specified procedure the specified number of times. You must use the apply shift statement at least once in the load_unload procedure. For the apply shift statement, you should enter a proper #times parameter, otherwise you will get a warning message.

If required, you must enter the apply shadow_control statement immediately after the apply shift procedure statement, and you must set the #times argument to 1. The apply statement is only valid outside of the cycle blocks because it specifies another group of cycles within another procedure to be added at that point.

- **loop *loop_count***

A literal and integer pair that specifies the loop count and is followed by a block of statements. The loop procedure statement takes the loop count and causes all cycles within the loop block to be repeated by the number of times specified by the count. For example, the following test procedure file excerpt specifies 3 cycles within the loop that are each repeated 20 times:

```
procedure test_setup =  
  timeplate tp1 ;  
  cycle =  
    force_pi;  
    measure_po;  
  end;  
  loop 20 =  
    cycle =  
    end;  
    cycle =  
      pulse tck;  
    end;  
    cycle =  
    end;  
  end; // end loop  
end;
```

Nesting loops within other loops is permitted. For example, the following test procedure file excerpt causes tck to be pulsed 20 times and clk_a to be pulsed 100 times:

```
procedure test_setup =  
  timeplate tp1 ;  
  cycle =  
    force_pi;  
    measure_po;  
  end;  
  loop 20 =  
    cycle =  
    end;  
    cycle =  
      pulse tck;  
    end;  
    loop 5 =  
      cycle =  
      pulse clk_a;  
      end;  
    end; // end inside loop  
    cycle =  
    end;  
  end; // end outside loop  
end;
```

This statement can be used in procedures but must be specified outside of the cycle statement.

The loop statement is preserved in the flat model when the tool writes the model and is also present in the TCD files.

When writing out the patterns in tester pattern formats, the loops are preserved where possible, and unrolled if the syntax of the pattern file does not support loops. Specifying the [ALL_NO_LOOP](#) parameter keyword unrolls loops in the pattern files in similar fashion to sub procedures that are applied more than once. Using the loop statement to repeat a certain number of cycles N times is exactly equivalent to putting those cycles within a sub procedure, then applying that procedure N times.

- **cycle_statement**

The following list describes valid cycle_statement keywords. Cycle_statements cannot contain time values.

- **force_pi**
A literal that specifies for the tool to force all primary inputs.
- **bidi_force_pi**
A literal that specifies for the tool to force all bidirectional pins.
- **force_sci**
A literal that specifies for the tool, in the shift procedure, to place values on the scan chain inputs, thus implementing scan cell controllability.
- **force_sci_equiv**
A literal that acts the same as the force_sci statement, except that it also forces all pins equivalent to the scan input pins. Using this statement places the complement value on the associated differential pin of a scan input during scan loading. This statement is necessary because the test procedures do not consider pin equivalence relationships (those specified with add_input_constraints -equivalent).
- **measure_po**
A literal that specifies for the tool to measure or strobe the primary outputs.
- **bidi_measure_po**
A literal that specifies for the tool to measure or strobe the bidirectional pins.
- **measure_sco**
A literal that specifies for the tool, in the shift procedure, to measure scan output values, thus implementing scan cell observability. In End Measure Mode (refer to “[Creating Test Procedure Files for End Measure Mode](#)” on page 359), measure_sco is also used in the load_unload procedure.
- **restore_pi**
A literal that returns primary inputs to their original states (prior to this procedure’s execution). You use the restore_pi statement at the end of a seq_transparent procedure.

- restore_bidi

A literal that returns bidirectional pins to their original states (prior to this procedure's execution). You use the restore_bidi statement at the end of a "clock" procedure.

- bidi_force_off

A literal that specifies for the tool to force all unconstrained bidirectional pins off.

- pulse_capture_clock

A literal that specifies for the tool to pulse the capture clock.

- force_capture_clock_on

A literal that specifies the cycle when the capture clock goes active. This statement and the force_capture_clock_off statement can be used in place of the pulse_capture_clock statement.

The "_on" refers to the active state of the clock, which is not necessarily the high binary value. This statement is used only with the non-scan procedures and cannot be mixed with the following statements in the same procedure:

- pulse_capture_clock
- pulse_write_clock
- pulse_read_clock

- force_capture_clock_off

A literal that specifies the cycle when the capture clock goes inactive. This statement and the force_capture_clock_on statement can be used in place of the pulse_capture_clock statement.

The "_off" refers to the inactive state of the clock, which is not necessarily the low binary value. This statement is used only with the non-scan procedures and cannot be mixed with the following statements in the same procedure:

- pulse_capture_clock
- pulse_write_clock
- pulse_read_clock

- pulse_read_clock

A literal that specifies for the tool to pulse the RAM read clock.

- pulse_write_clock

A literal that specifies for the tool to pulse the RAM write clock.

- force_pin_name value

A literal and double string that forces the specified value of 0, 1, X, or Z on the specified pin. The pin names you specify must be valid pin pathnames for primary inputs.

- expect name value

A literal and double string that causes the tool to expect the specified value of 0, 1, X, or Z on the specified internal pin or port. The default value is X. You can use “expect” statements only in the test_setup and test_end procedure. Internal pins are checked by DRC and are compared in the testbench. For ports, the tool validates the values in the testbench, the DRCs, and in the tester pattern formats.

- condition cell_name value

A literal and double string that you use at the beginning of a seq_transparent procedure to identify the necessary scan cell states (conditions) to establish transparency in non-scan cells. You identify the scan cell by the pin pathname associated with the output of its state element. The path from the defined pin to the scan cell must only contain buffers and inverters. The value argument sets the value at the specified pin pathname, which may be inverted relative to the associated scan cell value.

- measure pin_name

A literal and string pair that specifies for the tool to measure the value of the named pin. You can use a “measure” statement in the capture procedure only to specify a measure on a pin in a different cycle than the measure_po event.

- initialize instance_name value

A literal and string pair that initializes the named memory element to the value given. This statement is particularly useful for initializing the finite state machine in the TAP controller of boundary scan circuitry, when the TAP does not contain the TRST signal. Once set to a binary state, the TCK and TMS pins can place the finite state machine in a desired state. If not set, these pins remain at X.

If you do not specify a value, the tool chooses a random value to assign to all latches and flip-flops with the specified instance name.

- pulse pin_name

A literal and string pair that specifies for the tool to pulse the named clock pin.

- observe_method value

A literal and string pair set to a value of master, slave, or shadow, to specify for a specific observe method to be defined for each named capture procedure.

The following example shows how to use a `cycle_statement` to force scan inputs and measure scan outputs:

```
procedure shift =  
  scan_group grp1;  
  timeplate tp1;  
  cycle =  
    force_sci;  
    measure_sco;  
    pulse T;  
  end;  
end;
```

Clock Control Definition

You can manually create clock control definitions in the test procedure file.

For complete information about when you use this definition, refer to “[Support for Internal Clock Control](#)” in the *Tessent Scan and ATPG User’s Manual*.

ATPG Restrictions

The following restrictions apply to ATPG when clock control definitions are enabled:

- Clock_PO patterns are disabled.
- In undefined cycles, the internal clock is assumed to be off, even if the source clock pulses.
- Source clocks are pulsed regardless of clock restrictions. Any false paths should be explicitly defined with DC or the [add_false_paths](#) command.
- External clocks without clock control definitions are controlled through top-level pins.
- Clock control definitions applied to a clock defined as equivalent also applies to all associated equivalent clocks.
- Timeplate definitions apply only to external clocks.
- If you use the [set_clock_restriction](#) -same_clocks_between_loads command, you must use one of the following definitions to pulse the controlled clock:
 - {ATPG_SEQUENCE, END}
 - {ATPG_SEQUENCE <N> <M>, END} with N starting from 0. The generated test pattern includes M+1 capture cycles between the scan loading operation and the scan unloading operation.
 - <ATPG_CYCLE <i>, END} with i=0 defined

If none of these statements are present a warning displays during ATPG about clock controls that cannot be applied due to clock restrictions.

Rules for Clock Control Definitions

The following rules apply to the clock control definitions in the test procedure file:

- Global control conditions and source clocks defined for equivalent clocks must be the same.
- When a clock is forced off, it cannot be used as a source in the same definition.
- A FORCE statement cannot force a clock pin to an on state.
- Clock control cannot be applied to an asynchronous free-running clock.
- Condition statements cannot be applied to non-scan state elements.
- You must specify a condition to turn on the internal clock; otherwise, it is assumed to be off.
- When multiple sequence clock control definitions are defined for the same clock, they must use mutually exclusive pulse conditions as follows:
 - The clock control condition to pulse a clock in sequence mode must be mutually exclusive with the clock control condition for the same clock in per-cycle mode.
 - The condition to pulse a clock in sequence mode must be mutually exclusive with the condition to pulse the same clock by using another sequence mode.

Keywords

The following is a list of keywords used in clock control statement:

- **ATPG_CYCLE *cycle_number***

A literal and integer that specifies a test pattern capture cycle to map the clock control to. The specified capture cycle values start from 0, which corresponds to the first capture cycle after scan loading.

Multiple ATPG_CYCLE definitions can be declared to pulse the internal clock at the same capture cycle with different sets of conditions.

Use a FORCE statement to turn the clock off, or the clock continues to pulse when the conditions are satisfied.

The specified and actual capture cycles may differ—see “[Capture Cycle Determination](#)” in the *Tessent Scan and ATPG User’s Manual*.

- **ATPG_SEQUENCE *N M***

A literal and an integer pair that specify a range of capture cycles for clock pulsing from N to M consecutively.

Define the condition to pulse the clock continuously from capture cycle N ($N \geq 0$) to capture cycle M ($M \geq N$) right after scan loading. If N is greater than 0, the clock is automatically set to off state from the first capture cycle right after scan loading to the capture cycle N - 1. When the generated test pattern includes more than M capture cycles after scan loading, the clock is set to off state from the M + 1 capture cycle to the last capture cycle.

Multiple ATPG_SEQUENCE definitions can be declared as necessary.

Use a FORCE statement to turn the clock off, or the clock continues to pulse when the conditions are satisfied.

The specified and actual capture cycles may differ—see “[Capture Cycle Determination](#)” in the *Tessent Scan and ATPG User’s Manual*.

- **ATPG_SEQUENCE**

A literal that specifies clock pulsing. When a condition list is provided, the controlled clock pulses in all capture cycles in the pattern when the conditions are met. When checking the off conditions for cycles outside the capture window, the conditions listed in this special atpg_sequence will be ignored.

When there is no condition list, the controlled clock pulses unconditionally in every capture cycle between scan loading.

Multiple ATPG_SEQUENCE definitions can be declared as necessary.

Use a FORCE statement to turn the clock off, or the clock continues to pulse when the conditions are satisfied.

The specified and actual capture cycles may differ—see “[Capture Cycle Determination](#)” in the *Tessent Scan and ATPG User’s Manual*.

- **CLOCK_CONTROL *pin_pathname***

A literal and string value that specifies the pin pathname of the PI for the internal clock. The specified pin must be an existing clock. Internal clocks must also be defined with the [add_clocks](#) command.

- **CONDITION *cell_name value***

An optional literal and double string that specifies necessary scan cell states (conditions). The scan cell is specified by the pin pathname associated with the output of its state element. The value argument specifies the value loaded into the scan cell at the end of shift.

The specified and actual capture cycles may differ—see “[Capture Cycle Determination](#)” in the *Tessent Scan and ATPG User’s Manual*.

- **FORCE** *pin_pathname value*

A literal and double string that forces a value of 0, 1, or Z on a specified pin. The specified pin names must be valid pin pathnames for primary inputs. This keyword is used to force necessary pins off during capture cycles when the controlled clock is pulsed.

- **SOURCE_CLOCK** *pin_pathname...*

A literal and repeatable string that specifies one or more source clocks to drive the internal clock logic to pulse in the specified capture cycle(s). If no source clock is specified, the source clock is assumed to be an always-capture clock that pulses in every capture cycle.

- **END**

Required literal the specifies the end of an ATPG_CYCLE or ATPG_SEQUENCE block, or at the end of the clock control definition.

Global Condition Statements

Global conditions are **CONDITION** statements that are accessible in every scope of a clock control definition. You can use global conditions to define default conditions within a clock control definition.

For example, you can specify a **CONDITION** statement for all ATPG_CYCLE/ATPG_SEQUENCE blocks within a definition. Then you can define a **CONDITION** statement within individual ATPG_CYCLE/ATPG_SEQUENCE blocks to override the global **CONDITION** variable when necessary.

The following example uses local conditions (*italicized*) to define some of the control bits necessary for each scan cell to pulse the clock. The global conditions define other conditions that must be satisfied for all clock cycles.

```
CLOCK_CONTROL /clk_ctrl/int_clk1 =  
    SOURCE_CLOCK ref_clk;  
    CONDITION /clk_ctrl/enable_1/q 0;  
    CONDITION /clk_ctrl/enable_2/q 0;  
    ATPG_CYCLE 0 =  
        CONDITION /clk_ctrl/F0/q 1;  
    END;  
    ATPG_CYCLE 1 =  
        CONDITION /clk_ctrl/F1/q 1;  
    END;  
    ATPG_CYCLE 2 =  
        CONDITION /clk_ctrl/F2/q 1;  
    END;  
    ATPG_CYCLE 3 =  
        CONDITION /clk_ctrl/F3/q 1;  
    END  
END;
```


The previous example is equivalent to the following:

```
CLOCK_CONTROL /clk_ctrl/int_clk1 =
SOURCE_CLOCK ref_clk;
ATPG_CYCLE 0 =
    CONDITION /clk_ctrl/F0/q 1;
    CONDITION /clk_ctrl/enable_1/q 0;
    CONDITION /clk_ctrl/enable_2/q 0;
END;
ATPG_CYCLE 1 =
    CONDITION /clk_ctrl/F1/q 1;
    CONDITION /clk_ctrl/enable_1/q 0;
    CONDITION /clk_ctrl/enable_2/q 0;
END;
ATPG_CYCLE 2 =
    CONDITION /clk_ctrl/F2/q 1;
    CONDITION /clk_ctrl/enable_1/q 0;
    CONDITION /clk_ctrl/enable_2/q 0;
END;
ATPG_CYCLE 3 =
    CONDITION /clk_ctrl/F3/q 1;
    CONDITION /clk_ctrl/enable_1/q 0;
    CONDITION /clk_ctrl/enable_2/q 0;
END
END;
```

The previous example demonstrates the importance of ensuring that global conditions do not conflict with local conditions. To further illustrate this point, consider the following incorrect definition of global conditions:

```
// Example of incorrect definition of global conditions
CLOCK_CONTROL /clk_ctrl/int_clk1 =
SOURCE_CLOCK ref_clk;
    CONDITION /clk_ctrl/F0/q 0;
ATPG_CYCLE 0 =
    CONDITION /clk_ctrl/F0/q 1;
END;
ATPG_CYCLE 1 =
    CONDITION /clk_ctrl/F1/q 1;
END;
ATPG_CYCLE 2 =
    CONDITION /clk_ctrl/F2/q 1;
END;
ATPG_CYCLE 3 =
    CONDITION /clk_ctrl/F3/q 1;
END
END;
```

On the surface, it may seem correct that the condition in ATPG_CYCLE 0 will override the global condition while the other cycles can still be satisfied. However, after the global condition is expanded to all cycles, the clock control definition looks like this:

```
// Example of incorrect definition of global conditions
CLOCK_CONTROL /clk_ctrl/int_clk1 =
SOURCE_CLOCK ref_clk;
  ATPG_CYCLE 0 =
    CONDITION /clk_ctrl/F0/q 1;
  END;
  ATPG_CYCLE 1 =
    CONDITION /clk_ctrl/F1/q 1;
    CONDITION /clk_ctrl/F0/q 0;
  END;
  ATPG_CYCLE 2 =
    CONDITION /clk_ctrl/F2/q 1;
    CONDITION /clk_ctrl/F0/q 0;
  END;
  ATPG_CYCLE 3 =
    CONDITION /clk_ctrl/F3/q 1;
    CONDITION /clk_ctrl/F0/q 0;
  END
END;
```

You can now see that it would not be possible to pulse the clock in ATPG_CYCLE 0 while also pulsing it in any other cycle. The tool can load only one value into /clk_ctrl/F0, so it can either pulse the clock in cycle 0 by loading a 1 or pulse it in another cycle by loading a 0.

Per-Cycle Clock Control Definition Example

The following example defines per-cycle clock control for two internal clocks (/top/core1/clk1 and /top/core1/clk2):

```
CLOCK_CONTROL /top/core1/clk1 =
  ATPG_CYCLE 0 =
    CONDITION /pll_ctl/cell_0/Q 1;
  END;
  ATPG_CYCLE 1 =
    CONDITION /pll_ctl/cell_1/Q 1;
    CONDITION /pll_ctl/cell_4/Q 0;
  //both conditions must be satisfied for clock to pulse in
  //capture cycle 1
  END;
END;
CLOCK_CONTROL /top/core1/clk2 =
  ATPG_CYCLE 0 =
    CONDITION /pll_ctl/cell_2/Q 1;
  END;
  ATPG_CYCLE 1 =
    CONDITION /pll_ctl/cell_3/Q 1;
    CONDITION /pll_ctl/cell_5/Q 1;
  END;
END;
```

Sequence Clock Control Definition Example

The following example defines sequence clock control for two internal clocks (/top/core/clk1 and /top/core1/clk2) derived from the source clock clk_src:

```
CLOCK_CONTROL /top/core1/clk1 =
    SOURCE_CLOCK clk_src;
    ATPG_SEQUENCE 0 1 =
    // Pulses 2 consecutive cycles if the scan cell
    // is loaded with 1, and the source clock is pulsed.
    CONDITION /pll_ctl/cell_1/Q 1;
    END;
END;
CLOCK_CONTROL /top/core1/clk2 =
    SOURCE_CLOCK clk_src;
    ATPG_SEQUENCE 0 1=
    CONDITION /pll_ctl/cell_2/Q 1;
    END;
END;
```

The following example defines sequence clock control for two internal clocks (/top/core/clk1 and /top/core1/clk2) derived from the source clock clk_src. The clock pulses in all capture cycles when the conditions are met.

```
CLOCK_CONTROL /top/core1/clk1 =
    SOURCE_CLOCK clk_src;
    ATPG_SEQUENCE =
    // Pulse clock in all capture cycles if the scan cell
    // is loaded with 1, and the source clock is pulsed.
    CONDITION /pll_ctl/cell_1/Q 1;
    END;
END;
CLOCK_CONTROL /top/core1/clk2 =
    SOURCE_CLOCK clk_src;
    ATPG_SEQUENCE =
    CONDITION /pll_ctl/cell_2/Q 1;
    END;
END;
```

The following example pulses clock /top/core1/clk1 unconditionally in every capture cycle between scan loading:

```
CLOCK_CONTROL /top/core1/clk1 =
    ATPG_SEQUENCE =
    // empty body
    END;
END;
```

The following example defines a multi-sequence clock control definition:

```
CLOCK_CONTROL /top/core/clk1_int =  
    SOURCE_CLOCK /clk1;  
    ATPG_SEQUENCE 0 2 =  
        CONDITION /pll/ctl_1/Q 1;  
        FORCE ENABLE_1 1;  
    END;  
    ATPG_SEQUENCE 3 4 =  
        CONDITION /pll/ctl_1/Q 0;  
        FORCE ENABLE_1 1;  
    END;  
END;
```

Exclusive conditions ensure that only one sequence block is applied per capture cycle (otherwise, no sequence is applied). If no cycle numbers are specified for sequence clock control, the clock pulses in every capture cycle when conditions are loaded.

Multiple Sets of Conditions for the Same Cycle Example

The following example shows that if a clock can be pulsed in a particular cycle or sequence of cycles when there are multiple sets of conditions where any one set can activate the clock for that cycle, the same cycle can be defined multiple times:

```
CLOCK_CONTROL /top/core1/clk1 =  
    ATPG_CYCLE 0 =  
        CONDITION /pll_ctl/cell_1/Q 1;  
    END;  
    ATPG_CYCLE 0 =  
        CONDITION /pll_ctl/cell_2/Q 1;  
    END;  
END;
```

The previous example shows that /top/core1/clk1 can be pulsed in ATPG_CYCLE 0 when any set of the specified conditions are met. This demonstrates the case where loading a '1' into either /pll_ctl/cell_1 or /pll_ctl/cell_2 pulses the clock in cycle 0.

Similarly, the following example defines multiple sets of conditions for the same sequence of cycles, which can overlap. The sequence of cycles must have mutually exclusive conditions to ensure conditions for each ATPG_SEQUENCE can be satisfied without conflicting with other sequences.

```
CLOCK_CONTROL /top/core1/clk1 =
  ATPG_SEQUENCE 0 2 =
    CONDITION /pll_ctl/cell_1/Q 1;
    CONDITION /pll_ctl/cell_2/Q 0;
    CONDITION /pll_ctl/cell_3/Q 0;
  END;
  ATPG_SEQUENCE 0 3 =
    CONDITION /pll_ctl/cell_1/Q 0;
    CONDITION /pll_ctl/cell_2/Q 1;
    CONDITION /pll_ctl/cell_3/Q 0;
  END;
  ATPG_SEQUENCE 1 4 =
    CONDITION /pll_ctl/cell_1/Q 0;
    CONDITION /pll_ctl/cell_2/Q 0;
    CONDITION /pll_ctl/cell_3/Q 1;
  END;
END;
```

Source Clocks with Different Frequencies Example

The following example defines source clocks that have different frequencies when using clock control definitions:

```
timeplate _default_WFT_ =
  force_pi 0 ;
  measure_po 40 ;
  pulse clk1 45 10;
  pulse ref_clock 15 5, 40 5, 65 5, 90 5;
  pulse clocks_02/my_controller/U2/Z 45 10;
  pulse clocks_03/my_controller/U2/Z 45 10;
  pulse clocks_04/my_controller/U2/Z 45 10;
  period 100 ;
end;

procedure capture =
  timeplate _default_WFT_;
  cycle =
    force_pi ;
    measure_po ;
    pulse_capture_clock ;
  end;
end;
```

In this example, for one pulse of clk1, there are 4 pulses of ref_clock, specifically the ref_clock frequency is 4 times the frequency of clk1.

The Procedures

The test procedure file contains scan and clock procedures, and non-scan procedures. The scan and clock-related procedures inform the tool how to operate the scan chain and pulse clocks. The non-scan procedures can represent any type of pattern that the tool produces.

You can use the non-scan procedures to specify in which cycles of the procedure “potential events” happen. A potential event is an event that the ATPG engine may or may not have created to cover a certain fault.

To avoid DRC violations, each non-scan procedure must contain the proper statements in the correct order with the timing from the timeplate. The statements in a non-scan procedure can be spread over any number of cycles using a different timeplate for each cycle if needed.

A basic pattern consists of loading the scan chains, a default capture procedure, followed by unloading the scan chains; however, you do not specify the loading and unloading of scan chains in non-scan procedures. The following shows the basic pattern for non-scan procedures.

Basic Pattern

Force primary inputs
Measure primary outputs
Pulse capture clock

All example procedures shown in this section use one of the following two timeplates, unless otherwise stated:

```
timeplate tp1 =
  force_pi 0;
  measure_po 10;
  pulse scan_clk 30 10;
  pulse sys_clk 30 10;
  period 50;
end;

timeplate tp2 =
  force_pi 0;
  measure_po 10;
  pulse scan_mclk 15 10;
  pulse scan_sclk 30 10;
  period 50;
end;
```

Test_Setup (Optional)	327
Shift (Required)	330
Alternate Shift Procedure (Optional)	332
Load_Unload (Required)	334
Shadow_Control (Optional)	337

Master_Observe (Sometimes Required)	339
Shadow_Observe (Optional).....	339
Seq_Transparent (Optional).....	340
Clock (Optional).....	342
Skew_Load (Optional).....	343
Clock_run (Optional)	344
Capture Procedures (Optional)	346
Clock_po (Optional)	351
Ram_sequential (Optional).....	351
Ram_passthru (Optional)	353
Clock_sequential (Optional).....	353
Init_force (Optional)	354
Test_end (Optional, all ATPG tools)	355
Sub_procedure	356

Test_Setup (Optional)

This optional procedure, which can only contain force, pulse, init, and expect event statements, sets non-scan elements to the desired states for the load_unload procedure. You may use this procedure only once for all scan groups, and it appears only once at the beginning of the test pattern set.

This procedure is particularly useful for initializing boundary scan circuitry. For an example using this procedure to set up boundary scan circuitry, refer to “[Pattern Generation for a Boundary Scan Circuit](#)” in the *Tessent Scan and ATPG User’s Manual*.

IJTAG Embedded Instruments

In the ATPG context patterns -scan, IJTAG is allowed in test_setup and test_end procedures only. It is neither allowed in any other procedure, nor in ATPG’s analysis mode. Also, only [iReset](#) and [iCall](#) commands are allowed in the test procedures.

For detailed information, see “[How to Set Up Embedded Instruments Through Test Procedures](#)” in the *Tessent IJTAG User’s Manual*.

Bidirectional Scan Out Pins

The value of all bidirectional scan out pins must be forced to the Z state (indicating it is operating in “output” mode) to properly sensitize the scan chain. When reading in a test procedure file, the tool automatically adds force events to the beginning of the load_unload procedure to force all bidi pins to Z.

Bidi pins that are clocks or constrained pins are not forced to a Z, as they were already forced to the off-state or the constrained values. Bidi scan-in pins are also not forced to a Z. Any bidi pin already forced later in the load_unload procedure will not be forced to a Z. Any bidi forced to a specific value in the test_setup procedure will instead be forced to this value instead of a Z.

Like previous automatic force values, these can be disabled by putting the “set autoforce off;” statement at the beginning of the procedure file.

Pin Constraints

If you use the add_input_constraints command to set pin constraints, be aware this command only forces pins during capture. To constrain these pins during test_setup, you should include the same pin constraints in the test_setup procedure. This will ensure the pins are in the same state for loading the first pattern as for loading all subsequent patterns.

If you do not properly constrain the pins prior to the end of the test_setup procedure, the tool automatically constrains them by inserting a cycle statement in the test_setup procedure. However, this automatic handling may not insert the events with the timing you want. Also, the automatic handling is not included in DRC.

If you have defined input constraints but have not provided a test_setup procedure, the tool will automatically generate a test_setup procedure to force those pins to their constrained values.

You can use both the [write_procfile](#) and the [report_procedures](#) commands to see the contents of the test_setup procedure the tool has generated. The write_procfile command writes existing procedure and timing data to a specified file. The report_procedures command writes the information to the screen.

Example 1

The following is an example using a sub_procedure. In this example, the signal named C will retain its value of 1 during the test unless it is forced to a different value in a later cycle, by another procedure, or it is overwritten by WGL patterns.

```
procedure sub_procedure initialize =  
    template soc_timeplate ;  
    cycle =  
        force C 1;  
    end;  
end;
```

The following example shows how to apply the previous sub_procedure. For more information, see “[Sub_procedure](#)” on page 356.


```

procedure test_setup =
    timeplate soc_timeplate;
    cycle =
        force test_en 1; // force test_en 1
        force chip_en 0; // force chip_en to 0
    end;
    apply initialize 10 ; // force C to 1 for 10 cycles
end;

```

Example 2

The following example shows a way to apply initialization cycles to a memory. The RST signal is active for the first 128 cycles, then it is deactivated in the next cycle (cycle 129).

```

procedure sub_procedure reset_mem =
    timeplate soc_timeplate ;
    cycle =
        force RST 1;
    end;
end;
procedure test_setup =
    timeplate soc_timeplate;
    apply reset_mem 128;
    cycle =
        force RST 0; // deactivate RST
    end;
end;

```

Example 3

The following example shows a way to use an expect statement in a test_setup procedure. The output signal (DFT) is expected to 1 in the first cycle and X in the remaining cycle. Please note that “expect” statements do not work the same as a force or pulse statement. When none is present, it is assumed to mean do not measure.

```

procedure test_setup =
    timeplate soc_timeplate;
    cycle =
        expect DFT 1 ;
    end;
end;

```

Example 4

This example shows a way to start pulsing a clock in a test_setup procedure. The SYSCLK starts pulsing at cycle number 2 until the end of test.

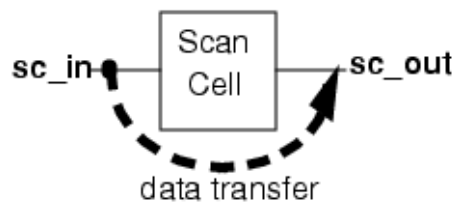
```
timeplate soc_timeplate =  
    force pi;  
    measure_po 90;  
    pulse SYSCLK 50 50;  
    period 100;  
end;  
  
procedure test_setup =  
    timeplate soc_timeplate;  
    cycle =  
        force RST_L 0;  
    end;  
    cycle =  
        pulse SYSCLK;  
    end;  
end;
```

Shift (Required)

This required procedure describes how to shift data one position down the scan chain by forcing the scan input, toggling the clock(s), and strobing the scan output.

Figure 8-2 shows the data flow process for the shift procedure.

Figure 8-2. Shift Procedure



Within this procedure, you must use the `force_sci`, or `force_sci_equiv`, and the `measure_sco` event statements. You can also use the `force` and `pulse` event statements. A shift procedure can contain more than one cycle, although not all pattern formats can support multiple cycles and parallel load. Pattern formats that do not support multiple cycles are any parallel format other than STIL and Verilog. If you use `write_patterns` to write out one of these other parallel formats with a multicycle shift procedure, the command generates an AG11 error.

The times at which the timeplate used by the shift procedure applies the `force_sci` and `measure_sco` commands must allow proper operation of the `load_unload` procedure. The `measure_sco` will occur at the `measure_po` time specified in the timeplate. The `force_sci` will occur at the `force_pi` time specified in the timeplate.

The following are examples of the shift procedure for both mux-DFF and LSSD architectures.

Mux-DFF Example

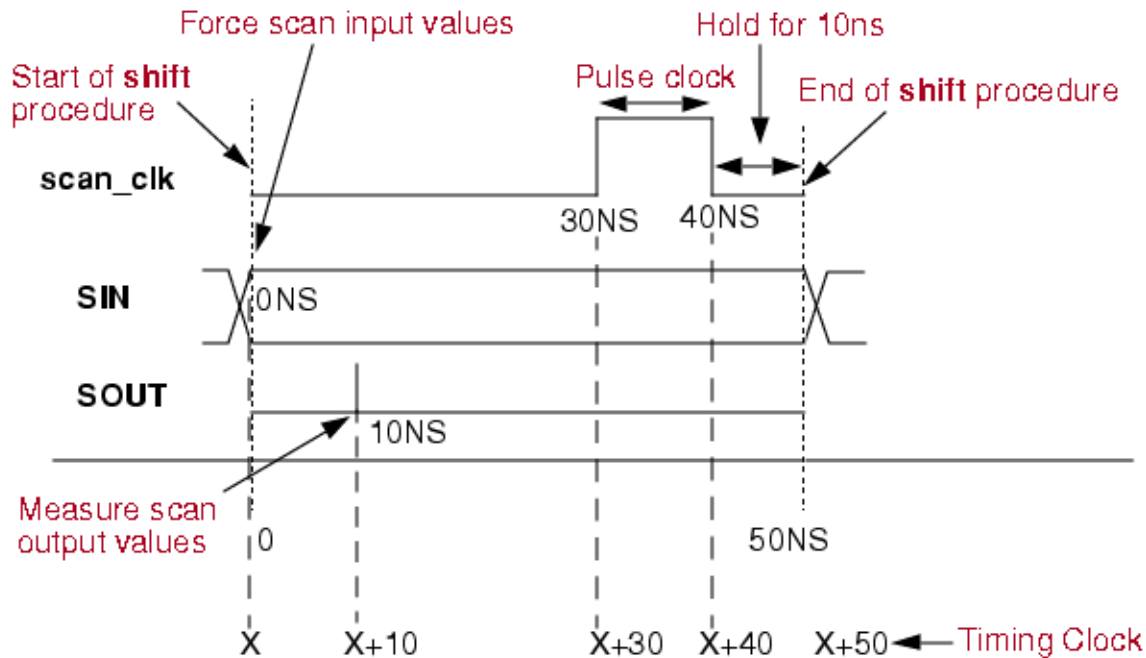
```
procedure shift =  
  timeplate tp1;  
  cycle =  
    // force scan chain input  
    force_sci;  
    // measure scan chain output  
    measure_sco;  
    // pulse the scan clock  
    pulse scan_clk;  
  end;  
end;
```

LSSD Example

```
procedure shift =  
  timeplate tp2;  
  cycle =  
    // force scan chain input  
    force_sci;  
    // measure scan chain output  
    measure_sco;  
    // pulse master clock  
    pulse scan_mclk;  
    // pulse slave clock  
    pulse scan_sclk;  
  end;  
end;
```

[Figure 8-3](#) graphically displays the waveforms for the clock pin, the scan-in pin, and the scan-out pin derived from the Mux-DFF shift procedure example. This timing diagram shows one scan chain shift cycle, assuming the time unit is 1ns.

Figure 8-3. Timing Diagram for Shift Procedure



The procedure contains four scan events: forces scan input values at 0ns, strobes (or measures) scan output values at 10ns, pulses the scan clock `scan_clk` (turning it on at 30ns and off at 40ns), and holds the state of the last event until the procedure finishes at 50ns.

A timing clock monitors when each significant event occurs. If the timing clock is at X when the shift procedure begins, the timing clock assigns those four events with time values X, X+10, X+30, and X+40. When the shift procedure finishes, the timing clock advances to X+50. The shift cycle ending time becomes the starting time for the next shift cycle.

Alternate Shift Procedure (Optional)

When using on-chip clock generators, such as programmable PLLs, it is sometimes necessary to change values on input (control) signals to the clock generator a cycle or two before the change in generated clocking schemes is realized. When the shift clocks for a scan chain are also provided by the on-chip clock generator, it is sometimes not possible to reprogram the clock generator near the end of the scan chain shifting in order to stop the shift clock and prepare for the capture clocks. To accomplish this you might want to use an alternative shift procedure.

Alternate shift procedures have names, as described in the following paragraph. Alternate shift procedures can only be used for single shifts (a pre shift or a post shift), and there must be one un-named normal shift (*shift*) as the main shift in the required `load_unload` procedure. See [Load_Unload \(Required\)](#).

The shift procedure allows for an optional name following the shift procedure type. For each scan group, one shift procedure must be defined that has the default name of shift. For each scan group, additional alternate shift procedures can be defined as long as each has a unique name.

Each shift procedure is required to contain a `force_sci` or `force_sci_equiv` statement and a `measure_sco` statement.

Syntax

```
procedure shift [ procedure_name ] =  
...  
end ;
```

Example

The following is a partial example of how the alternate shift procedure might be used in a procedure file for a scan chain with a length of 100.

```
timeplate tp1 =
  force_pi 0;
  measure_po 10;
  pulse ref_clk 50 50;
  period 100;
end;
procedure shift =
  timeplate tp1;
  scan_group grp1;
  cycle =
    force ctrl_a 1;
    force_sci;
    measure_sco;
    pulse ref_clk;
  end;
end;
procedure shift shift_last =
  timeplate tp1;
  scan_group grp1;
  cycle =
    force ctrl_a 0;
    force_sci;
    measure_sco;
    pulse ref_clk;
  end;
end;
procedure load_unload =
  timeplate tp1;
  scan_group grp1;
  cycle =
    force ref_clk 0;
    force scan_en 1;
    force ctrl_a 1;
  end;
  apply shift 98;
  apply shift_last 1;
  apply shift_last 1;
end;
```

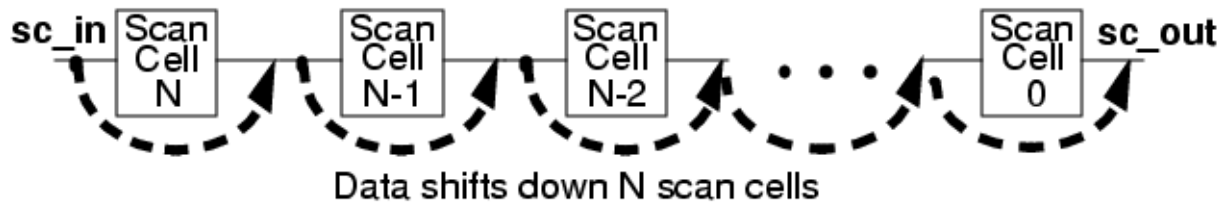
Load_Unload (Required)

This required procedure describes how to load and unload the scan chains in the scan group. To load the scan chain, you must force the circuit into the appropriate state for the start of the shift sequence. This includes forcing clocks, resets, RAM write control signals, and any other signals that need to be at their off states for scan chain loading. Also, if a reset signal is defined as a clock, and pin constrained to its off state in the dofile, it needs to again be forced to its off state in the load_unload and named capture procedures in order to avoid a P34 DRC.

Offstate for clock pins, constrained pin values, and other pins that have values forced in the test_setup procedure are automatically added as force statements to the beginning of the load_unload procedure (if not present); this helps reduce DRC failures.

Figure 8-4 shows the data flow for the load_unload procedure.

Figure 8-4. Load_Unload Procedure



If the scan out pin is bidirectional, you must force its value to the Z state (indicating it is operating in “output” mode) to properly sensitize the scan chain. If there is a scan enable signal, you must force it on to enable the scan chain prior to the shift. You then use the apply shift statement to specify the number of shift cycles (which equals the number of scan elements in the chain). If you have optionally included the shadow_control procedure (which if used, immediately follows the shift procedure), you must also include the apply command.

The following list includes the basic statements in the load_unload procedure:

Mux-DFF Example

```
procedure load_unload =
  timeplate tp1;
  cycle =
    // force clocks off
    force RST 0;
    force CLK 0;
    // activate scanning mode
    force scan_en 1;
  end;
  // shift data thru each of 7 cells
  apply shift 7;
end;
```

LSSD Example

```
procedure load_unload =
  timeplate tp2;
  cycle =
    // force all clocks off
    force RST 0;
    force CLK 0;
    force scan_sclk 0;
    force scan_mclk 0;
  end;
  // apply shift procedure 7 times
  apply shift 7;
end;
```

The timing for the load_unload procedure is generally straightforward. The load_unload procedure contains the apply statement. Therefore, the total time for a load_unload procedure

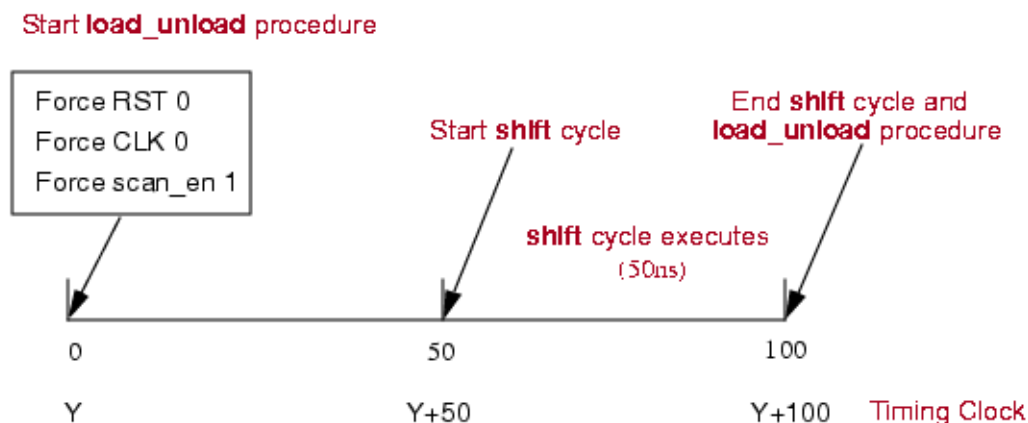
includes the time specified by the timeplate being used plus the time required to execute the apply cycles.

For example, examine the following load_unload procedure, using the example shift procedure in the previous section.

```
procedure load_unload =  
    timeplate tp1;  
    cycle =  
        force RST 0;  
        force CLK 0;  
        force scan_en 1;  
    end;  
    apply shift 1;  
end;
```

The timeplate of the load_unload procedure specifies the period is 50ns. However, the load_unload procedure includes an apply statement that executes one shift procedure. The shift procedure requires an additional 50ns. Thus, the load_unload procedure actually requires a total time of 100ns, as shown in [Figure 8-5](#).

Figure 8-5. Timing Diagram for Load_Unload Procedure



Within the load_unload procedure, after the completion of the cycle block, the shift procedure starts at 50ns, executes for 50ns, and ends at 100ns. Thus, the load_unload procedure also ends at 100ns.

As with the shift procedure, the timing clock determines the event times for the load_unload procedure. If the timing clock is at Y when the load_unload procedure begins, the first three events happen at time Y. When the apply cycle executes, the timing clock advances to Y+50, which is when the shift procedure begins. As mentioned previously, the shift procedure requires 50 time units. Therefore, when the apply cycle finishes, the timing clock reads Y+100.

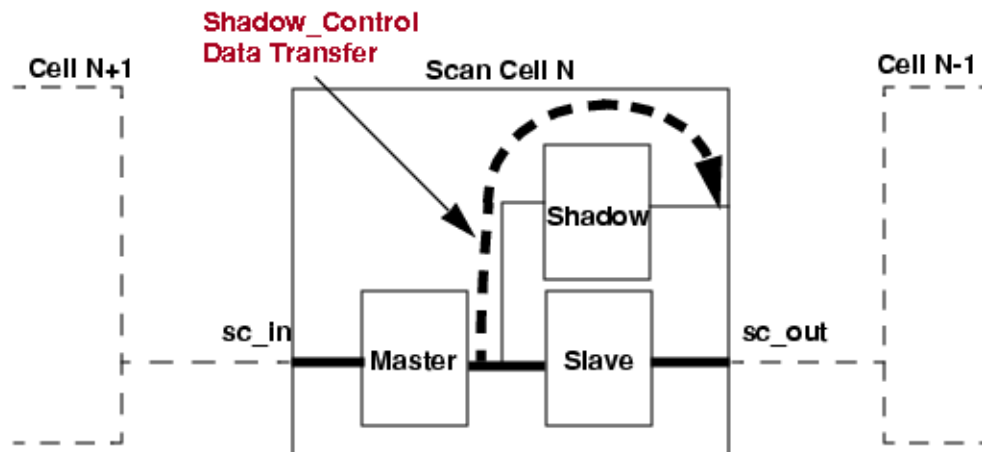
Because it is the last event in the load_unload procedure, the end of the apply cycle determines the end of the load_unload procedure.

Shadow_Control (Optional)

The optional shadow_control procedure, which may only contain force and pulse event statements, describes how to load the contents of a scan cell into the associated shadow.

If you use this procedure, you must also apply the shadow_control command in the load_unload procedure. This procedure must not disturb the contents of any of the scan cells. [Figure 8-6](#) shows the data flow for the shadow_control procedure.

Figure 8-6. Shadow_Control Procedure



The following procedure file example demonstrates the syntax for applying a shadow_control procedure within a load_unload procedure:

```
proc shift =
    force_sci          0;
    measure_sco        0;
    force SK2          1 1;
    force SK2          0 2;
    force SK1          1 3;
    force SK1          0 4;
end;

proc load_unload =
    force WE          0 0;
    force ABC         1 0;
    force COMB_B      1 0;
    force SK2         0 0;
    force CLK         0 0;
    force SK1         0 0;
    force sh_clk      0 0;
    apply shift       5 1;
    apply shadow_control 1 2;
end;

proc shadow_control =
    force sh_clk      1 1;
    force sh_clk      0 2;
end;

proc master_observe =
    force WE          0 0;
    force SK2         0 0;
    force CLK         0 0;
    force SK1         0 0;
    force sh_clk      0 0;
    force SK1         1 1;
    force SK1         0 2;
end;

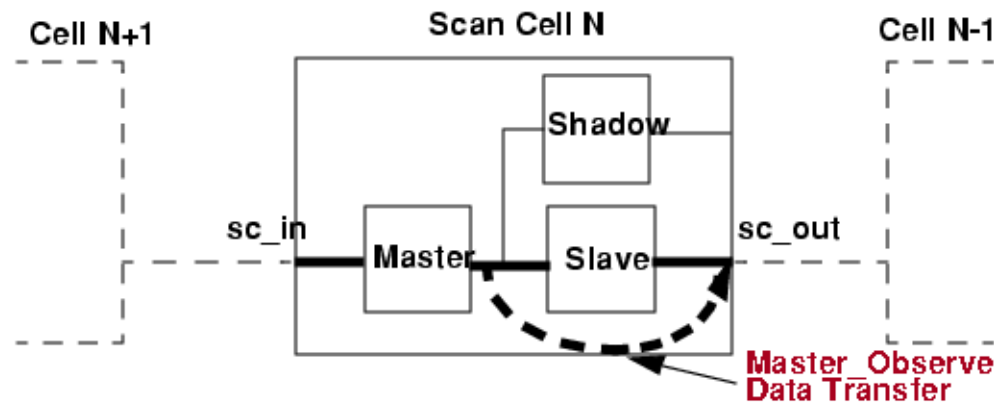
proc shadow_observe =
    force WE          0 0;
    force EN          1 0;
    force SK2         0 0;
    force CLK         0 0;
    force SK1         0 0;
    force sh_clk      0 0;
    force CLK         1 1;
    force CLK         0 2;
    force SK1         1 3;
    force SK1         0 4;
end;
```

Master_Observe (Sometimes Required)

The master_observe procedure, which may only contain force and pulse event statements, describes how to place the contents of a master into the output of its scan cell, where you can observe it by using the unload operation.

Figure 8-7 shows the data flow for the master_observe procedure.

Figure 8-7. Master_Observe Procedure



You do not need to use this procedure if the master element's output is the output of the scan cell. The D1 rule ensures this procedure does not disturb master memory element's contents. You can override this requirement by changing the D1 rule handling. The following example shows a master_observe procedure for the LSSD architecture:

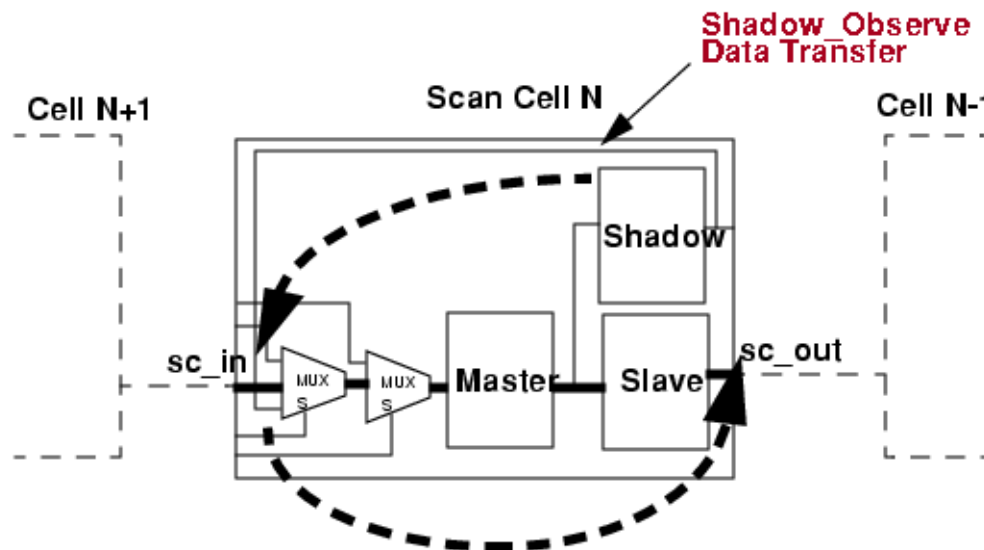
```
// LSSD architecture example
procedure master_observe =
  timeplate tp1;
  cycle =
    // Force all clocks off
    force scan_sclk 0;
    force scan_mclk 0;
    force rst      0;
    force clk      0;
    // Pulse the slave clock
    pulse scan_sclk;
  end;
end;
```

Shadow_Observe (Optional)

The optional shadow_observe procedure, which may only contain force and pulse event statements, describes how to place the contents of a shadow into the output of its scan cell, assuming the circuitry of the scan cell allows the transfer of data in this way. Once the data is at the scan cell output, you can observe it by applying the unload command. This procedure allows the shadow to be used as an observation point in the design.

Figure 8-8 shows the data flow of the shadow_observe procedure.

Figure 8-8. Shadow_Observe Procedure



Seq_Transparent (Optional)

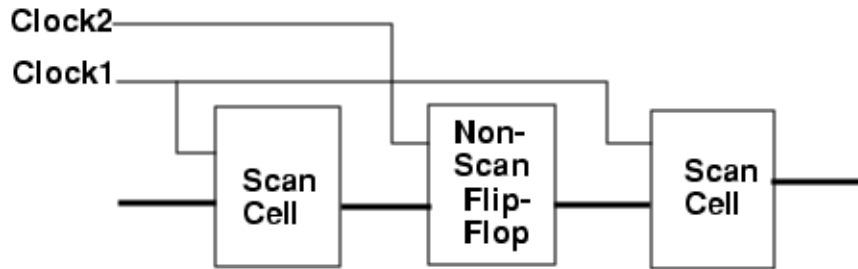
The seq_transparent procedure (optional for “patterns -scan” context) identifies how to make non-scan cells and RAM read ports functionally behave transparently. This procedure activates the clock inputs of non-scan cell inputs, thus pulsing data through the cells “transparently.”

All clocks must be at their off-states and constrained pins at their constrained states before applying the seq_transparent procedure. The procedure must immediately follow a force of all the primary inputs. For more information on the sequential transparent operation, refer to “[Sequential Transparent Patterns](#)” in the *Tessent Scan and ATPG User’s Manual*.

You can use multiple clock cycles to create the sequential transparent conditions. You may define up to 32 different seq_transparent procedures within a procedure file. When simulation mode is set to RAM_sequential, each force_all statement in the pattern file can use any of the possible seq_transparent procedure choices. In “patterns -scan” context, the tool treats non-scan state elements that cannot use the sequential transparent procedures as tie-X gates.

There may be occasions when you would want to use seq_transparent procedures when the design contains no scan chains. In this case, you would use the add_scan_groups command, specifying the name “dummy” for the group name; the tool would expect the specified test procedure file to contain only the timeplate and seq_transparent procedure. For more information, refer to the [add_scan_groups](#) command reference page in the Tessent Shell Reference Manual. [Figure 8-9](#) shows some circuitry that could benefit from a seq_transparent procedure.

Figure 8-9. Sequential Transparent Circuitry Example



The following example shows the seq_transparent procedure for [Figure 8-9](#).

```

timeplate tp1=
  force_pi 0;
  measure_po 0;
  pulse clock1 30 10;
  pulse clock2 30 10;
  period 50;
end;

procedure seq_transparent tran1=
  timeplate tp1;
  cycle=
    force clock1 0;
    force clock2 0;
    force reset 1;
  end;
  cycle=
    pulse clock2;
  end;
  cycle=
    restore_pi;
  end;
end;

```

The basic stimuli necessary to create transparent behavior for the non-scan flip-flop shown in [Figure 8-9](#) are:

- Force all clocks off
- Pulse non-scan cell clock Clock2
- Restore primary inputs to original values

In more complex situations, you may need to set primary inputs to certain values, place conditions on scan cells, pulse multiple clocks, and so on.

You can use the report_seq_transparent_procedures command to display data defined by the seq_transparent procedures. For more information, refer to the [report_seq_transparent_procedures](#) description in the Tessent Shell Reference Manual.

Clock (Optional)

The clock procedure (optional in “patterns -scan” context) provides flexible clock handling during the test procedures. Using clock procedures, instead of pulsing a single clock during a capture cycle, you can serially exercise multiple clocks and force non-clock pins that do not affect captured data.

The following example shows a clock procedure used to operate two clocks in sequence:

```
procedure clock clock_procl =  
  timeplate tp3;  
  cycle =  
    pulse clk1;    // Pulse first clock  
  end;  
  cycle =  
    pulse clk2;    // Pulse second clock  
  end;  
end;
```

Clock procedures must abide by the following rules:

- The procedure must activate at least one clock.
- If you define multiple clock procedures, only one of these procedures can activate a specific clock.
- The procedure events cannot violate pin constraints or equivalence conditions.
- The procedure can only force non-clock pins, if they do not affect data captured into state elements, whose clocks may activate later in the procedure.
- Multiple clocks that activate serially cannot logically interact.
- The procedure must follow all standard rules for both clock and non-clock pin usage.
- Each clock procedure must have a unique name.
- If a state element can change state during the procedure, the element must be stable when all clocks are off and pins are constrained.
- *Transparent_capturecells* are stable state elements that can capture data during the procedure and whose new data can affect other state elements later in the procedure. Design rule D10 ensures that these cells do not connect to state elements that capture old data or propagate data to primary outputs.
- The procedure must set all bidirectional pins to their input mode prior to executing the `restore_bidis` statement.

If a clock procedure contains a `restore_bidis` statement, the tool cannot use sequential ATPG. This may cause a problem if you set the tool for multiple clock compression because multiple clock compression uses sequential ATPG.

Skew_Load (Optional)

The optional `skew_load` procedure propagates the output value of the preceding scan cell into the master memory element of the current cell without changing the slave, for all scan cells. Using only force and pulse event statements, this procedure defines how to apply an additional pulse of the master shift clock after the scan chains are loaded.

Figure 8-10 shows the data flow of the `skew_load` procedure.

Figure 8-10. Skew_Load Procedure

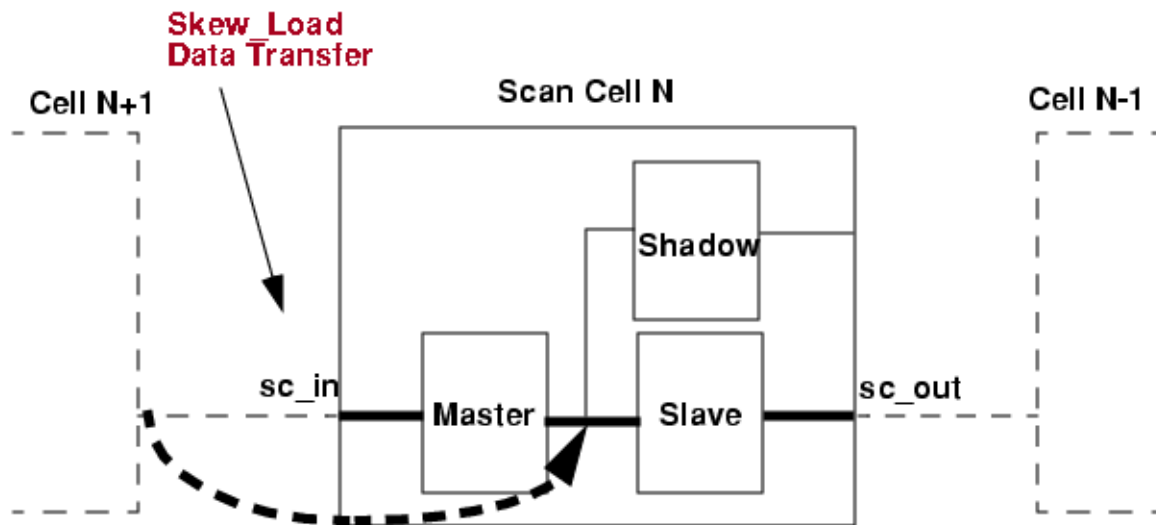
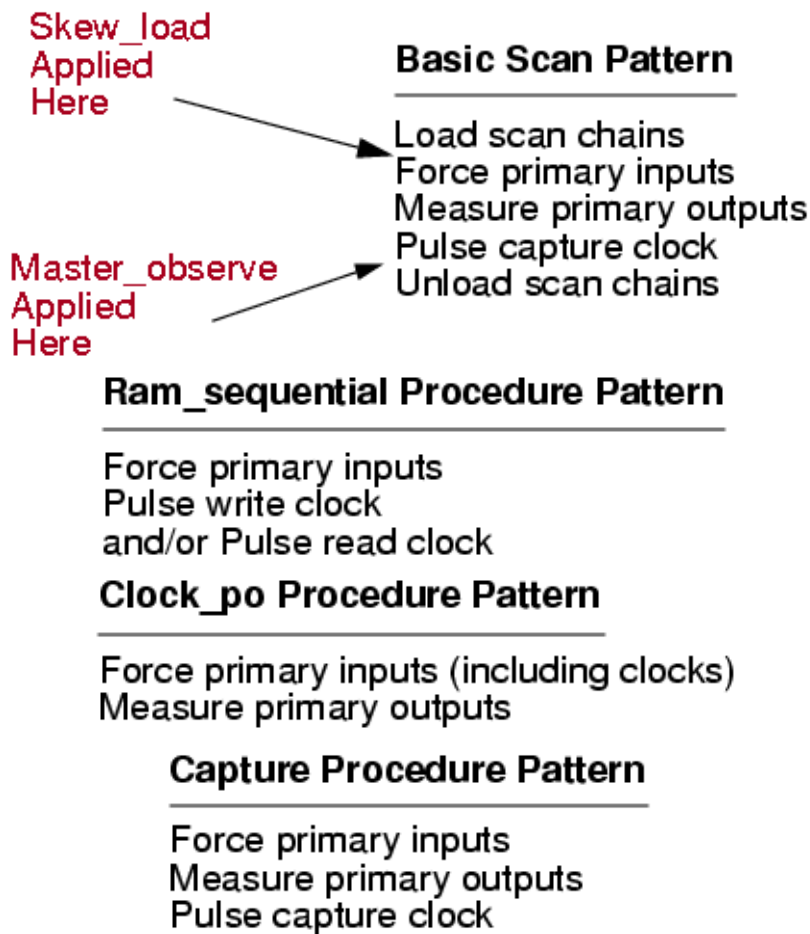


Figure 8-11 shows where you apply the `skew_load` procedure and the `master_observe` procedure within the basic scan pattern events.

Figure 8-11. Skew_load applied within Pattern



Clock_run (Optional)

For every controller, or concurrent controller group, you can write a `clock_run` procedure, if needed. The `clock_run` procedure has both an internal mode as well as an external mode.

You can specify only one `clock_run` procedure per controller or concurrent group; however, you do not need to specify a separate procedure for each controller instance. The same procedure can be used for multiple controllers. You need to specify a separate procedure for a controller instance only if it maps to a different set of internal clocks.

In case of controllers running concurrently, and some of these controllers clocks are driven by PLL internal clocks, the `clock_run` procedure is required per concurrent group. It is not required for every BIST controller participating in the group to have its clock driven by a PLL internal clock. For some controllers, their clocks can be driven by a PLL reference clock or even by a system clock.

The tool relies on you to control the PLL control signal. This can be achieved by forcing the PLL control signal to a proper value in a test_setup procedure and in external mode of clock_run procedure as well (it depends on the PLL model behavior).

A clock_run procedure has to have a N-to-1 or 1-to-N ratio between internal and external cycles; that is, either the internal mode has to have only one cycle, or the external mode has to have only one cycle. You cannot have, for example, two external cycles and three internal cycles.

Capture Procedures (Optional)

There are three types of capture procedures. These procedures are optional in the “patterns -scan” context.

- The *default capture procedure* is an optional capture procedure, without a name, that provides information on how the series of capture events are broken into cycles and which timeplates these cycles use. The default capture procedure is defined in the procfile as part of the scan group definition or internally derived by the tool when you do not define one. If you need to create or edit a default capture procedure, see “[Clock_po \(Optional\)](#)” in this chapter.
- The *named capture procedure* is an optional capture procedure, with a name, that is used to define explicit clock cycles. You can create multiple named capture procedures, each with a unique name, using the [create_capture_procedures](#) command. If you need to manually create or edit named capture procedures, see “[Rules for Creating and Editing Named Capture Procedures](#)” in this chapter. For information on using named capture procedures to create at-speed test patterns, see “[At-Speed Test With Named Capture Procedures](#)” in the *Tessent Scan and ATPG User’s Manual*.
- The *external_capture procedure* is an optional capture procedure used for all capture cycles between each scan load, even when the pattern is a multi-load pattern. External_capture procedures are used by the “[set_external_capture_options -capture_procedure](#)” command. External_capture procedures that are used with this command have several restrictions:
 - The procedure can only have one `force_pi` statement and no `measure_po` statements. This is because in order to use the “[set_external_capture_options -capture_procedure](#)” switch, the patterns to be saved must be `hold_pi` and `mask_po` patterns. The statements in the capture procedure must match up to this.
 - Unlike named capture procedures, the external_capture procedure cannot have any `load_cycles` as it is meant to be used between each scan load of a pattern. The external_capture cannot contain any events on internal signals.
 - When using the `-capture_procedure` switch with `set_external_capture_options`, all clocks in the design that are internally connected (don't trace to just a cut point), must be controlled. In addition, the clocks must either be constrained, always-capture, controlled by a `clock_control` definition, or the clock must have an event in the external_capture procedure.

Rules for Creating and Editing a Default Capture Procedure	347
Rules for Creating and Editing Named Capture Procedures	347
Slow and Load Types in the Cycle Statement	349
launch_capture_pair Statement	350

Rules for Creating and Editing a Default Capture Procedure

There are several issues to consider when working with default capture procedures.

- The default procedure may only contain `force_pi`, `measure_po`, `pulse_capture_clock`, `bidir_force`, `bidir_force_pi`, `bidir_force_off`, and `bidir_measure_po` event statements that represent the non-scan activity for a normal pattern. There is no overlap between the capture procedure and the existing clock procedure.
- Use the `pulse_capture_clock` statement in the default capture procedure to indicate in which cycle one or more capture clocks should be pulsed.
- Do not specify any complex clocking that needs to be described for capture clocks or other clocks in the default capture procedure; specify it in the clock procedure or by using a named capture procedure.
- Do not specify any type of pin or ATPG constraint in the default capture procedure. For example, specifying that a certain pin is to be held at a certain state in the default capture procedure does not restrict the ATPG engine from applying different values to that pin. However, you can use the `bidir_force` and `bidir_force_pi` statements in the default capture procedure to force all bidirectional pins off in one cycle and force the ATPG values on the bidirectional pins in the next cycle.

Rules for Creating and Editing Named Capture Procedures

There are several issues to consider when working with named capture procedures.

- A named capture procedure may only contain `force_pi`, `measure_po`, `observe_method`, `pulse` (named clock), and `condition` statements.
- If you use mode definitions, all cycles in a procedure must be defined within mode definitions. Use the keyword “mode” with two mode blocks: “internal” and “external”. Use the `mode_internal` definition to describe what happens on the internal side of the on-chip PLL. Use the `mode_external` definition to describe what happens on the external side of the on-chip PLL.
- All events in a named capture procedure that use modes must be duplicated in both modes. The only difference is that the internal mode uses only internal clocks and the external mode uses only external clocks. The number of cycles and timeplates used can be different as long as the total period of both modes is the same.
- Signal events used in both internal and external modes must happen at the same time. Examples of these events are `force_pi`, `measure_po`, and other signal forces, but also include clocks that can be used in both modes.

- If a `measure_po` statement is used, it can only appear in the last cycle of the internal mode and must occur before the last clock pulse. If no `measure_po` statement is used, the tool issues a warning that the primary outputs will not be observed.
- The cumulative time from the start of the first cycle to the `measure_po` must be the same in both modes.
- The external mode cannot pulse any internal clocks or force any internal control signals.
- A `force_pi` statement needs to appear in the first cycle of both modes and occur before the first pulse of a clock.
- If an external clock goes to the PLL and to other internal circuitry, a C2 DRC violation is issued.
- At-speed cycles need to be continuous; that is, a named capture procedure cannot have more than one at-speed clocking subsequence.
- All defined real clocks (excluding internal clocks) must be forced to off state first in the `mode_internal` definition.

For more information, see “[Internal and External Modes Definition](#)” in the *Tessent Scan and ATPG User’s Manual*.

- Do not use the `pulse_capture_clock` statement in a named capture procedure. The clocks used are explicitly pulsed.
- If you want to specify the internal conditions that need to be met at certain scan cells in order to enable a clock sequence, use the condition statement at the beginning of the cycle statement in the named capture procedure.
- If you want to define a specific observe method for each named capture procedure, use the `observe_method` statement in the named capture procedure; otherwise, the ATPG engine automatically selects master, slave, or shadow observation.

Note

 The [write_patterns](#) command allows you to save internal or external clock patterns. Internal clock patterns can be used to simulate the DUT without having the PLL modeled, while the external patterns only exercise the PLL external clocks and control signals. Internal patterns are the default for ASCII and binary formats, and external patterns are the default for tester formats.

- If you generate patterns using a named capture procedure that has both internal and external modes and you save them in STIL or WGL format, you must use the `write_patterns` command’s “internal” option to read them back into the tool (for example, to use in diagnosis). For more information and for information about special considerations that apply to LBIST mode in the TK/LBIST Hybrid flow, refer to the `-Mode_internal` and `-Mode_external` switches for the [write_patterns](#) command in the *Tessent Shell Reference Manual*.

DRC rules W20 through W36 check named capture procedures. If a DRC error prevents use of a capture procedure, the run aborts.

Slow and Load Types in the Cycle Statement

Optionally, you can add a “slow” or a “load” type to the cycle definition.

For example:

```
cycle slow =  
...  
end;
```

- The slow cycle indicates that at-speed faults cannot be launched or captured. The tool must know which at-speed cycles are slow in order to get accurate at-speed fault coverage simulation numbers; therefore, be sure to include “slow” when defining cycles that are not at-speed cycles in an at-speed capture procedure.

Note

 At-speed cycles need to be continuous; that is, a named capture procedure cannot have more than one at-speed clocking subsequence.

- The load cycle indicates that the cycle is always preceded by an extra scan load. The first cycle in a named capture procedure is always a load (with or without the load type designation), so you typically apply “load” to subsequent cycles. An at-speed launch cycle can be a load cycle; however, none of the cycles that follow in the at-speed sequence, up to and including the capture cycle, can be load cycles.

Note

 To get extra loads, you must enable the tool’s multiple load and clock sequential capabilities by issuing the [set_pattern_type](#) command with “-multiple load on” and “-sequential <2 or greater>”. For more information, see “[Multiple Load Patterns](#)” in the *Tessent Scan and ATPG User’s Manual*.

The following example illustrates the “slow” and “load” attributes:

```
procedure capture multi_load_example =  
  timeplate tpl;  
  // first cycle is always a load, with or without load type designation  
  cycle slow =  
    force_pi;  
    force wr_enable 1;  
    pulse int_clk1;  
  end;  
  cycle slow load =  
    pulse int_clk1;  
  end;  
  cycle =  
    force re_enable 1;  
    pulse int_clk1; // launch clock  
  end;  
  cycle =  
    pulse int_clk1; // capture clock  
  end;  
end; // end of capture procedure
```

launch_capture_pair Statement

Optionally, you can add one or more “launch_capture_pair” statements to the beginning of a named capture procedure. This statement defines legal at-speed launch and capture points in non-adjacent cycles. If you do not use the launch_capture_pair statement, the tool will launch and capture only in adjacent cycles. If at least one launch and capture clock pair is defined, the launch and capture points are derived from the defined launch and capture clock pairs.

Note



This statement is only supported when using a named capture procedure to perform test generation.

The syntax of the launch_capture_pair statement is as follows:

```
launch_capture_pair <launch_clock_pin_name> <capture_clock_pin_name>;
```

Where:

- **launch_clock_pin_name** is the clock used to launch the transition.
- **capture_clock_pin_name** is the clock used to capture the transition.

The launch clock cycle is used to check the transition condition. The capture clock cycle is used to capture the transition fault effect. The cycles between the launch clock and capture clock must be at-speed cycles. They cannot include any slow cycles between them. The faults to be tested by the named capture procedure with the defined launch and capture clock pair are the faults that can be launched by the launch clock and captured by the capture clock defined in the launch_capture_pair statement.

The following is an example of the launch_capture_pair statement:

```
procedure capture c1_c1 =  
  launch_capture_pair c1 c1;  
  
  cycle = // cycle 1  
    force_pi;  
    force c1 0;  
    force c2 0;  
    force c3 0;  
    pulse c1;  
  end;  
  
  cycle = // cycle 2  
    pulse c2;  
  end;  
  
  cycle = // cycle 3  
    pulse c1;  
    pulse c3;  
  end;  
end; // end of capture procedure
```

In this example, a valid launch can happen in cycle 1. A valid capture can happen in cycle 3 only with c1 as the capture clock. A launch in cycle 1 and a capture in cycle 2 is not used for fault detection. The faults to be tested by this named capture procedure are the faults that can be launched and captured by clock c1.

Clock_po (Optional)

The clock_po procedure (optional in “patterns -scan” context), which can contain only force_pi, measure_po, bidi_force_pi, and bidi_force_off event statements, represents the non-scan activity for a clock PO pattern. Use this procedure instead of the capture procedure.

Note that with this procedure, you must use a timeplate that does not pulse the clocks.

The following shows the pattern for the clock_po procedure pattern.

Clock_po Procedure Pattern

Force primary inputs (including clocks)
Measure primary outputs

Ram_sequential (Optional)

The ram_sequential procedure (optional for “patterns -scan” context), which may only contain force_pi, pulse_write_clock, pulse_read_clock, bidi_force_pi, and bidi_force_off event

statements, represents the RAM sequential events in a RAM sequential pattern. Use this procedure with the capture procedure.

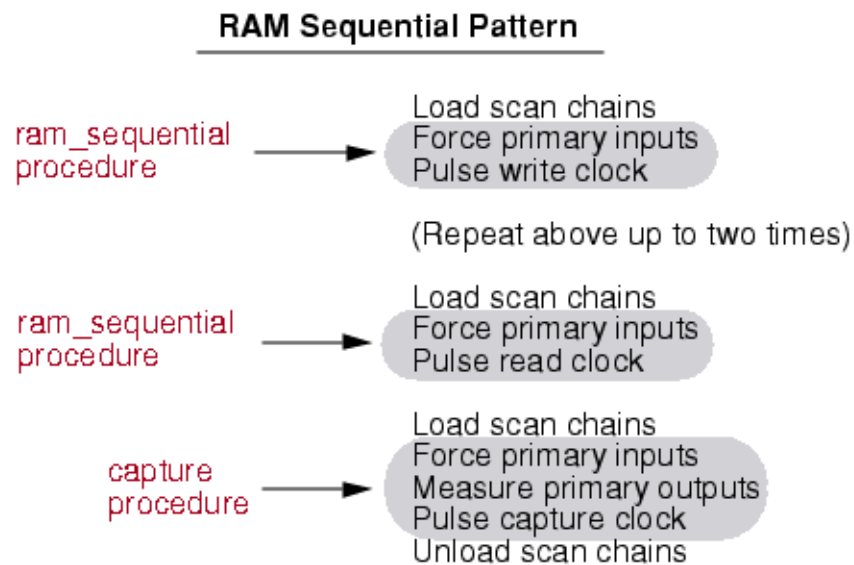
The following illustrates the basic ram_sequential procedure pattern format.

Ram_sequential Procedure Pattern

Force primary inputs
Pulse write clock
and/or Pulse read clock

Figure 8-12 shows an entire RAM sequential pattern, which illustrates where the ram_sequential and capture procedures are used.

Figure 8-12. Full Ram Sequential Pattern



Clock_sequential Procedure Pattern

Force primary inputs
Pulse write clock
and/or Pulse read clock
and/or Pulse capture clock

Ram_passthru Procedure Pattern

Force primary inputs
Pulse write clock
Pulse read clock
Measure primary outputs
Pulse capture clock

Ram_passthru (Optional)

The ram_passthru procedure (optional in “patterns -scan” context), which may only contain force_pi, measure_po, pulse_write_clock, pulse_capture_clock, bidi_force_pi, bidi_force_off, and bidi_measure_po event statements, represents the non-scan activity for a RAM passthrough pattern. Use this procedure instead of the capture procedure.

The following shows the ram_passthru procedure pattern.

Ram_passthru Procedure Pattern

Force primary inputs
Pulse write clock
Pulse read clock
Measure primary outputs
Pulse capture clock

Clock_sequential (Optional)

The clock_sequential procedure (optional for “patterns -scan” context), which may only contain force_pi, pulse_write_clock, pulse_read_clock, pulse_capture_clock, bidi_force_pi, and bidi_force_off event statements, represents the clock sequential events in a clock sequential pattern. Use this procedure with the capture procedure.

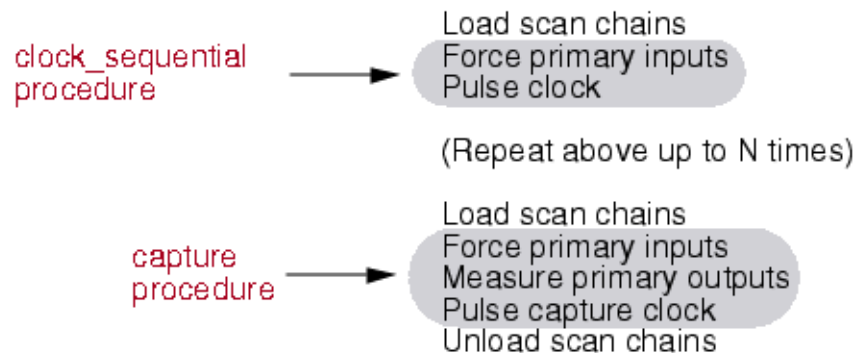
The following shows the clock_sequential procedure pattern.

Clock_sequential Procedure Pattern

Force primary inputs
Pulse write clock
and/or Pulse read clock
and/or Pulse capture clock

Figure 8-13 shows an entire clock sequential pattern, which illustrates where the clock_sequential and capture procedures are used.

Figure 8-13. Full Clock Sequential Pattern
Clock Sequential Pattern



Init_force (Optional)

The `init_force` procedure (optional for “patterns -scan” context), which may only contain `force_pi` event statements, represents the force cycle that is used in an ATPG pattern that targets a transition fault. The transition must be launched off of the last scan chain shift. This procedure is used when the fault type is set to transition fault and either the depth is set to 2 or less or the ATPG engines fail to find a sequential pattern that can cover this transition fault. Use this procedure with the capture procedure.

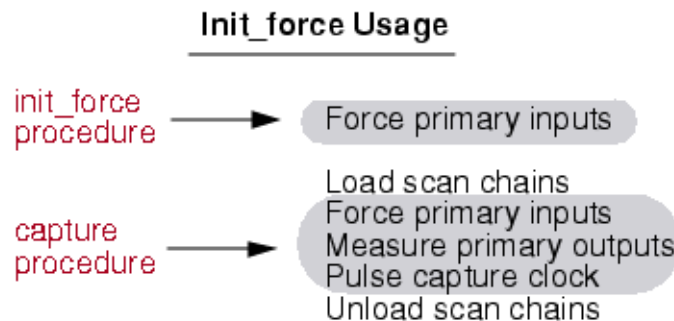
The following illustrates the format of the `init_force` procedure pattern.

Init_force Procedure Pattern

Force primary inputs

Figure 8-14 shows the pattern which uses the `init_force` procedure.

Figure 8-14. Init_force Procedure Usage



Test_end (Optional, all ATPG tools)

The optional test_end procedure is used to add a sequence of events to the end of a test pattern set.

The test_end procedure may only contain force and pulse event statements (see the following exception), and can only be defined once for all scan groups. When saving patterns, the test_end procedure will be applied to the end of each pattern set saved.

The following shows the general pattern for the test_end procedure pattern.

Test_end Procedure Pattern

Force primary inputs
Pulse clocks

IJTAG Embedded Instruments

In the ATPG context patterns -scan, IJTAG is allowed in test_setup and test_end procedures only. It is neither allowed in any other procedure, nor in ATPG's analysis mode. Also, only [iReset](#) and [iCall](#) commands are allowed in the test procedures.

For detailed information, see "[How to Set Up Embedded Instruments Through Test Procedures](#)" in the *Tessent IJTAG User's Manual*.

Example 1: Using test_end in a Procedure File

The following is a partial example of how the test_end procedure might be used in a procedure file:

```
timeplate tp1 =
  force_pi 0;
  measure_po 10;
  pulse ref_clk 50 50;
  period 100;
end;

procedure test_end =
  timeplate tp1;
  cycle =
    force ctrl_a 1;
    force tms 0;
    pulse ref_clk;
  end;
end;
```

Example 2: Test_end Procedure with Timeplate

The following is an example test_end procedure with its corresponding timeplate:

```
timeplate tp4 =
    force_pi 0;
    pulse TCK 10 10;
    measure_po 30;
    period 40;
end;

procedure test_end =
    timeplate tp4;
    cycle =
        // TMS = 1, change to select-DR state
        force TDI 1;
        force TMS 1;
        pulse TCK;
    end;

    cycle =
        // TMS = 0, change to capture-DR state

...
    cycle =
        // Scan out signature (MISR has length of 4)
        force TDI 1;
        force TMS 0;
        pulse TCK;
    end;
    cycle =
        force TDI 1;
        force TMS 0;
        pulse TCK ;
    end;
...
end;
```

Sub_procedure

The sub_procedure procedure eliminates the need to insert duplicate actions within a procedure. Once you have defined a sub_procedure, you can specify this procedure within other procedures using the apply statement.

You can also set the tool to reissue the sub_procedure as many times as needed by specifying the repeat_count. Because the repeat_count is required when using apply sub_procedure, you must enter a minimum of 1 for this parameter.

Sub_procedure Looping

Sub_procedure looping is used to reduce the size of pattern files. The default behavior of the sub_procedure is to use “loops” or “repeats” in all applicable pattern formats to repeat the contents of the sub_procedure N times, where N is greater than 1.

Disabling Sub_procedure Looping

If you want the event data in a sub_procedure to be expanded and represented as N sets of vectors in the pattern file, where N is the number of times the sub_procedure is applied, use the “[ALL_NO_LOOP 1](#)” parameter file keyword to disable the use of “loop” or “repeat” statements.

For example, if the test_setup procedure has the following statement:

```
apply pulse_bclock 1000;
```

The vector data for the sub_procedure “pulse_bclock” would be expanded to be 1000 vectors. The default for the ALL_NO_LOOP keyword is off (0).

Sub_procedure Definition Format

The sub_procedure definition has the following format.

```
procedure sub_procedure my_subprocedure =  
    timeplate tpl;  
    cycle =  
        force_pi;  
        measure_po;  
    end;  
end;
```

Using the Sub_procedure in a Procedure

The following is an example of how to use the sub_procedure in a procedure.

```
procedure shift =  
    scan_group grp1;  
    timeplate tpl;  
    apply my_subprocedure 4;  
    cycle =  
        force_sci;  
        measure_sco;  
        pulse T;  
    end;  
end;
```

Note



You must first define a sub_procedure before using it in a procedure. Next, you can apply a sub_procedure within any procedure type. Also, you cannot use a sub_procedure within the “cycle =” and “end;” statements.

Additional Support for Test Procedure Files

Tessent Shell provides additional support so that you can use environmental variable, merge, default template capabilities with your test procedure files.

Tcl Variables Used for Substitution and Conditional Selection

The procfile is allowed to include Tcl variables to provide parameterized values within the procfile statements. The Tcl variables must be set in the global Tcl namespace. If you are setting the variables within a proc, make sure to use `::` as a prefix to the variable name such that it is set in the global namespace. For example, use `"set ::my_period 4"` such that `$period` exists when processing the proc files.

The Tcl variable can also be used to evaluate the condition of a Tcl if command located inside the procfile. The element of a list of ports in the procfile must be separated by commas. If the port names are escaped identifiers, they must be enclosed in quotes. Use the method shown in the following example to convert a list of port names into a quoted comma-separated list needed in the proc file:

Command invoked in the dofile:

```
set ::add_clocks_timing 1
set ::edges "25 75"
set ::port_list [get_name_list [get_clocks -type sync_source]
set ::all_clocks [string cat {"} [join $port_list {"", "}] {""}]
set_procfile_name ./myproc
```

Given the content of the `./myproc` is:

```
alias all_clocks = $all_clocks;
timeplate mytimeplate =
    if {$add_clocks_timing} {
        pulse all_clocks $edges;
    }
end;
```

The [report_procedures](#) command will show:

```
alias all_clocks = "clk1", "clk2", clk3";
timeplate mytimeplate =
    pulse all_clocks 25 75;
end;
```

Merging Procedure Files

It is possible to specify more than one procedure file for a design. You can specify a procedure file with the `add_scan_groups` command or with the `read_procfile` command. You need to supply (to the ATPG tool) a minimum set of information in the procedure file with the `add_scan_groups` command. You must supply all event information for the scan procedures.

However, after leaving setup mode, it is possible to specify non-scan procedures, timeplates, and new timing for the scan procedures by reading in an additional procedure file with the `read_procf` command. Specifying new information for the same design, from more than one procedure file, is known as “merging the procedure files.” To properly merge the information from multiple procedure files, the Vector Interfaces code follows these rules:

- All scan procedures that you will use must be specified in the procedure file that you load with the `add_scan_groups` command.
- If you load a procedure that contains nothing but the procedure name, a timeplate name, and an optional scan group, it is a template procedure. If a procedure already exists by that name for that scan group (if it is a group-specific procedure), then the timeplate is mapped onto the existing procedure. If no procedure already exists with that name, the tool stores the template procedure for future use.
- If you load a new complete procedure (not a template) and a procedure already exists by that name for the specified scan group (if applicable), the new procedure overwrites the existing one.
- In both cases, when a procedure overwrites an existing one, or if a new timeplate is mapped to an old procedure, the tool checks the procedures to make sure that the sequence of events in the new procedure does not differ from the old procedure.

Default Information Provided by the Tool

When you issue the `write_patterns` command, the tool checks to make sure that all procedures and timeplates needed to save the patterns in the specified format are present.

If there are any missing non-scan procedures, the tool creates default procedures and issues a warning. For example, in cases where there are `ram_sequential` patterns that need to be saved and no `ram_sequential` procedure was supplied, the tool automatically creates a default procedure.

For any procedures that are created or that do not have a timeplate specified, the default timeplate is mapped to these procedures, if it is set. You can set the default timeplate by using the `set default_timeplate` statement previously described in the “[Set Statement](#)” section. If you use this statement, the timeplate specified when creating default procedures is used. If the default procedure needs to be created and no default timeplate has been set, then the first timeplate specified is used. If no timeplates are specified, a default timeplate is created as well.

Creating Test Procedure Files for End Measure Mode

You can create test procedure files that enable end measure mode. End measure mode refers to the special handling that the Vector Interfaces code needs to move the measure to the end of the shift and capture cycle.

Prerequisites

- A test procedure file.

Procedure

1. Create a new timeplate that measures the outputs after the clock pulse.
2. Change the timeplate for the shift and load_unload to point to the new timeplate.
3. Add the measure_sco statement to the load_unload procedure.
4. Make sure all shift procedures have the measure_sco statement after the shift clock. When end measure mode is enabled, the measure_sco statement measures the next value from the output of the scan chain. The very first value for the output of the scan chain is measured by a measure_sco statement in the load_unload procedure.
5. Change the timeplate for the capture cycle by breaking it into two cycles. Move the capture clock to the second cycle of the capture procedure to allow the measure at the end. In the first cycle, the force_pi and measure_po are performed. In the second cycle, the capture clock is pulsed. When using end measure mode, a measure cannot be performed after the capture clock.

Examples

```

set time scale 1.000000 ns ;
set strobe_window time 10 ;
timeplate gen_tp1 =
    force_pi 0 ;
    measure_po 10 ;
    pulse clk 20 10;
    pulse edt_clock 20 10;
    pulse ramclk 20 10;
    period 40 ;
end;
// CREATE A NEW TIMEPLATE THAT MEASURES AFTER THE CLOCK PULSE
timeplate gen_tp2 =
    force_pi 0 ;
    //      measure_po 10 ;
    pulse clk 20 10;
    pulse edt_clock 20 10;
    pulse ramclk 20 10;
    measure_po 35 ;          // <== NEW MEASURE STATEMENT
    period 40 ;
end;
// FOR CAPTURE SPLIT INTO TWO CYCLES
procedure capture =
    timeplate gen_tp1 ;
    cycle =
        force_pi ;
        measure_po ;
    end ;
    cycle =
        pulse_capture_clock ;
    end;
end;
// FOR THE SHIFT AND LOAD_UNLOAD, USE THE NEW TIMEPLATE
procedure shift =
    scan_group grp1 ;
    timeplate gen_tp2 ; //<=== NEW TIMEPLATE
    cycle =
        force_sci ;
        force edt_update 0 ;
        measure_sco ;
        pulse clk ;
        pulse edt_clock ;
    end;
end;
// ADD A MEASURE_SCO TO THE LOAD_UNLOAD PROCEDURE
procedure load_unload =
    scan_group grp1 ;
    timeplate gen_tp2 ; //<=== NEW TIMEPLATE
    cycle =
        force clk 0 ;
        force edt_bypass 0 ;
        force edt_clock 0 ;
        force edt_update 1 ;
        force ramclk 0 ;
        force scan_en 1 ;
        pulse edt_clock ;
    measure_sco;          //<=== NEW MEASURE STATEMENT

```

```
        end ;  
        apply shift 39;  
    end;  
    procedure test_setup =  
        timeplate gen_tp1 ;  
        cycle =  
            force edt_clock 0 ;  
    end;  
end;
```

Serial Register Load and Unload for LogicBIST and ATPG

You can use the `test_setup` or `test_end` test procedure file procedures to serially load or unload control registers in the circuit under test. Additionally, this section discusses the commands you must add to your dofile to use this functionality.

Serial register load and unload targets the following flows:

- **Logic BIST** — Used to serially unload certain registers (for example, MISRs) using the `test_end` procedure.
- **Low Pin Count Test** — Used to serially load registers with test information through a serial access port such as a JTAG TAP controller.

Register Load and Unload Use Models	363
Static Versus Dynamic Register Variables	363
Test Procedure File Modifications	364
Dofile Modifications.....	367
Serial Load and Unload DRC Rules	370

Register Load and Unload Use Models

Using serial register load and unload allows you to identify a load/unload register variable value in the test procedure file and define this variable, either static or dynamic, using commands in your Tessent dofile. By using this functionality, you can load control registers by using a type of load procedure where the data value for the register can be passed as a string of values, and the load register uses a shift mechanism to show how this string of data values is shifted into the register.

To serially load/unload register value variables, you must make modifications to your test procedure file and your Tessent dofile.

The register value variable is used in the test procedure file to load the value into a register through the data input pin, or unload a value from a register through the data output pin. The load/unload procedures support pin inversions, but you must explicitly specify this.

The registers to be loaded/unloaded can be cascaded such that one load or unload operation loads or unloads multiple values, for example PRPG/MISR values.

Static Versus Dynamic Register Variables

You can define both static and dynamic register value variables.

- Static variable — A specific register value variable string you specify during setup mode. Once set, you cannot change this variable.
- Dynamic variable — A register value variable string that you can define or change later, and can use tool-specific switches.

Static Variable

A static register variable is one where the value is assigned to the variable when the variable is defined (using the [add_register_value](#)), and this value then cannot be changed. This static value is used by DRC when simulating the procedures. A static variable cannot be set using tool specific switches to link the variable to tool computed values, as these may change.

Dynamic Variable

A dynamic register value variable uses tool-specific switches and arguments to link the variable to a value computed by the tool. If you use a dynamic variable, then you must specify the register's width unless the tool knows this value at the time DRC is run (for example, PRPG size).

This method is used for defining a variable that has a tool-specific computed value that is available when patterns are saved and needs to be loaded or unloaded by the test_setup or test_end procedures. The value defaults to all X bits, and you can specify the actual value can non-Setup mode by using the set_register_value command.

If no value is specified the first time DRC is invoked after adding a register value variable, the tool considers the variable to be dynamic for DRC and any future invocation of DRC.

Test Procedure File Modifications

In the test procedure file, the load_unload_registers Procedure and shift Keyword are used.

load_unload_registers Procedure

The **load_unload_registers** procedure is a named procedure; consequently, you can have multiple occurrences of this procedure, corresponding to multiple groups of registers that need to be loaded and/or unloaded. The load_unload_registers procedure can only be called from the following procedures:

- **test_setup**
- **test_end**

The load_unload_registers procedure can load, unload, or both. Additionally, one application of the procedure can load/unload multiple cascaded registers.

When the test procedure file explicitly calls the `load_unload_registers` procedure from either the `test_setup` or `test_end` procedures, the values to load or unload are passed to the procedure using the string of binary values (value hard-coded in the procedure application) or the register value variables.

shift Keyword

The `load_unload_registers` procedure uses the **shift** keyword to define a block of events that result in one shift operation. This is similar to the IEEE STIL syntax where the shift keyword defines a shift block within a load or unload procedure. It is analogous to embedding the shift procedure into the `load_unload` procedure.

Apply Statement Extension

The **apply** statement in the procedure file syntax is extended in order to accept a statement that associates a set of string values or register value variables with a particular input pin, output pin, or alias. This pin or alias must then be used within the shift statement and have a “#” character associated with it. This character is a placeholder for the values that will be loaded or unloaded using the string of values passed to the procedure, or the internally generated values associated with the value name.

Test Procedure File Syntax

The following illustrates using the `load_unload_registers` procedure and shift keyword:

```
procedure load_unload_registers procedure_name =  
    ...  
    [ cycle blocks ]  
    shift =  
        cycle blocks  
    end ;  
    [ cycle blocks ]  
end ;
```

The `load_unload_registers` procedure must have a shift block defined, which has one or more cycles used to shift the data into the data input pin or the data out of the data output pin. The procedure can also optionally have cycles which precede or follow the shift block, to be used to put the circuit into shift mode or finish the shift mode when done, similar to how a `load_unload` procedure is used.

Event Statements

Within a shift block, at least one event statements in a `load_unload_registers` procedure must use the “#” character to denote where the shift data passed into the procedure is used.

The event statement must be either a “force” event or an “expect” event. An event statement with the “#” character can also occur in the cycles preceding or following the “shift” block in

order to express pre-shifts and post-shifts for loading the register. This event statement has the following syntax:

```
<force | expect > pin_or_alias_name # ;
```

The apply statement which is currently used to call shift and sub_procedure procedures and also calls the load_unload_registers procedure. When calling these procedures, however, the number of times argument in the apply statement is replaced with one or more value assignments to the data in and/or data out pins.

```
apply procedure_name [ #times | shift_data_assignment  
[ , shift_data_assignment ...] ] ;
```

A shift_data_assignment has the following syntax:

```
identifier = < value_string | register_value_variable >  
[<value_string | register_value_variable>...]
```

where identifier is either a pin name or an alias name.

The identifier used in the shift_data_assignment must match an identifier used within the procedure in one of the event statements with the “#” character.

Register Value Strings

A register value string (value_string) is one of the following:

- A binary string of 0's, 1's or X's, where the length of the string determines the number of shifts to load the register.
- A string of a different radix as long as the Verilog syntax of identifying the radix and width are used, such as “32'h” for a 32 bit hexadecimal value.

If a register_value_variable is used in the shift_data_assignment, then a value computed by the tool for that register_value_variable (as bound within the dofile) or hard-coded in the dofile is loaded or unloaded at that time and the shift length is also provided by the tool. It is possible for more than one value_string or register_value_variable to be assigned to an identifier in one shift_data_assignment. In this case, the extra values are separated by spaces. This allows multiple shorter values to be shifted into one register group.

The typical usage is that each register_value_variable corresponds to one register being loaded, and the specification of multiple variables is used when those registers are cascaded and loaded/unload on after another.

Alias Names

It is possible to use an alias name in the procedure type for loading shift data, even if that alias name refers to multiple pins. If this is the case, the number of bits assigned by the “#” character

for each shift is equal to the width of the alias being used. The length of the value string being passed to the procedure must be a multiple of the width of the alias.

Loading or unloading an alias can be used if performing parallel load/unload and no shift is required. For example, if each MISR bit is connected to a separate primary output. Even if no shifting is required, the functionality is still useful to bind the expected values to the signature computed by the tool.

If multiple `shift_data_assignments` are passed to a procedure, then all of them must have the same shift length such that each pin being loaded/unloaded requires the same number of cycles to load/unload all the data. No padding is performed by the tool.

If a “measure” event is used in one of these procedures to unload a register, then these measure values can be compared in the final patterns and the Verilog testbench.

Dofile Modifications

You define register value variables using commands you issue to the tool interactively or in a dofile. The value variables are subsequently referenced in the procedure file.

You use the following commands to perform these operations:

- `add_register_value`
- `set_register_value`
- `delete_register_value`
- `report_register_value`

Addition of a Register Value Variable

The `add_register_value` command defines the register value variables. You can define the variables as either static value variables or dynamic value variables.

You can only use this command in setup mode. You must define the register value variables in the dofile *before* the variables are used in a test procedure file to load values into the registers.

Static Value Variables

You specify a static value variable by stating a specific value string. This value variable then has this value string as a constant value.

You must specify this value in setup mode; the value is considered to be static by default, and the value is present when simulating procedures in DRC.

```
add_register_value value_name {value_string [-Radix {Binary | Decimal |  
Octal | hexadecimal} -Width integer]} optional_arguments
```

The *value_name* is a user-specified identifier, and the *value_string* is a state string in a particular radix. The default is binary radix.

If the `-Radix` switch is used to change this to a different radix, then a register width must also be specified using the `-Width` switch.

Dynamic Value Variables

You specify a dynamic value variable using the following variation of the command:

```
add_register_value value_name value_string -Dynamic -Width integer  
optional_arguments
```

The *value_string* is optional and defaults to all X bits if not specified.

You can subsequently enter the actual value once you have exited setup mode using the [set_register_value](#) command. If no value is specified the first time DRC is invoked after adding a register value variable, it will be considered as dynamic in this and any future invocation of DRC.

Data Pin Inversions

When using either a static or dynamic variable, you can optionally specify inversions on the input and output data pins by using the `-INput_pin_inversion` and `-OUtput_pin_inversion` switches, respectively.

These are optional switches you use to specify that the data value in this variable should be inverted before it is loaded into the register through the `load_unload_register` procedure.

Definition of a Dynamic Register Value Variable

You can define a dynamic register value variable by using tool-specific switches and arguments to link the variable to a value computed by the tool. These variables are dynamic, and the width must be specified unless the width is known to the tool at the time DRC is run.

This method is used for defining a variable that has a tool-specific computed value that will be available when patterns are saved and needs to be loaded or unloaded by the `test_setup` or `test_end` procedures.

In this case, you use the `add_register_value` command with the `-Width` switch and omit the value while in setup mode. Once you have exited setup mode, you can use the [set_register_value](#) command to set the variable:

```
set_register_value value_name value_string [-RADix {Binary | HEx | OCtal |  
Decimal}]
```

The name of the register value and its width are known prior to parsing the procedure file and running DRCs. The value will be all X bits for DRC.

This command can only be used for a register value that was defined without a value string, and the width of the value string must match the width specified in the [add_register_value](#) command.

If the value overflows the width specified, an error is issued.

By default, the bits extracted by force/measure “#” in the procedure are from MSB to LSB. If the value specified should be shifted in/out in the opposite order, with the LSB bits applied/measured first, use the “-LSB_shifted_first” optional switch.

Deletion of a Register Value Variable

The [delete_register_value](#) command deletes all or a specified register value variable. You can only use this command in setup mode.

Register Value Variable Reports

The [report_register_value](#) command provides a detailed report of the register value variables to stdout or, optionally, a file.

Serial Load and Unload DRC Rules

The P13 and P54, P66, and W5 DRC rules are applied to the procedures and syntax in the section “Procedure Examples.”

The P1 “syntax error” message will be used to catch many potential issues, such as specifying a “#times” value for applying a `load_unload_registers` procedure instead of the `shift_data_assignment`.

P13 and P54	370
P66	370
W5	371
Procedure Examples	372

P13 and P54

The P13 and P54 rules are used to check the shift assignment statements when calling a `load_unload_registers` procedure.

If the shift assignment uses a value string that is not properly formatted or contains illegal characters for that radix, then a P13 is issued.

```
Error: Invalid state value state string. (P13)
```

If the shift assignment references an undefined register value variable name, a P54 is issued.

```
Error: Undefined identifier identifier_string referenced by signal_name.  
(P54)
```

P66

The P66 rule is used to check for missing statements within a `load_unload_registers` procedure.

For example, if no Shift block is specified, or if an event statement using the “#” character is not present for each `shift_assignment` passed to the `load_unload_registers` procedure, then a P66 is issued.

```
Error: Procedure procedure_name is missing required statement_string  
statement. (P66)
```

The following examples illustrate the types of P66 DRC errors you could encounter.

Example 1

In the following example, the tool issues this error if there is no shift block within the `load_unload_registers` procedure:

```
Error: Procedure procedure_name is missing required shift statement.  
(P66)
```

Example 2

In the following example, the tool issues this error if there are no events in the `load_unload_register` procedure that use the “#” substitute character.

```
Error: Procedure procedure_name is missing required substitute event  
statement. (P66)
```

Example 3

In the following example, the tool issues this error if an `apply` statement that uses a `load_unload_registers` procedure has a shift data assignment that uses a signal that does not appear in the `load_unload_registers` procedure with the substitute character “#”.

```
Error: Procedure procedure_name is missing event using shift assign  
signal_name statement. (P66)
```

W5

The W5 DRC error is used to flag any extra events or statements in a `load_unload_registers` procedure, or any events that are not legal.

For example, if an event type other than Force or Expect is used with the “#” substitute character, then a W05 rule will be issued. If the `load_unload_registers` procedure contains more than one shift block, this rule will be issued. If there is a Force or Expect statement using a “#” character for a signal name that is not being passed to the procedure as a shift assignment, this rule will be issued. The W05 rule is used when shift assignments for a particular Apply statement do not all have the same length.

```
Error: Procedure procedure_name has an illegal event statement or event  
order (event_statement_string) (W5)
```

Example 1

The following error is issued if an event in the `load_unload_register` procedure uses the “#” substitute character, but no shift data for this signal is passed into the procedure when it is called “extra shift block”:

```
<force | measure> signal_name # without matching shift assign data
```

Example 2

The following error is issued if there is more than one shift block in the load_unload_registers procedure.

```
"extra shift block"
```

Example 3

The following error is issued if more than one event of the same type in the shift block uses the same signal name with the “#” substitute character.

```
"too many events using shift assign signal_name"
```

Example 4

The following error is issued for an apply statement that uses a load_unload_registers procedure but has no shift data assignments in the apply statement.

```
"apply with no shift data assignment"
```

Example 5

The following error is issued for an apply statement that uses a load_unload_registers procedure and has more than one shift assignment, however, the target of the shift assignments are aliases and they are not the same width.

```
"unmatched shift assign signal width"
```

Example 6

This is issued for an apply statement that uses a load_unload_registers procedure and has more than one shift assignment and the assignments have different shift lengths.

```
"unmatched shift lengths"
```

Procedure Examples

The definition of the load_unload_registers procedure would look as follows. Notice how this procedure uses the “shift =” statement and the “force tdi #” statement to denote the shifting and where the string of data is applied, one bit at a time.

```

procedure load_unload_registers load_prpg1 =
    timeplate tp1 ;
    cycle =
        force prpg_select 1 ;
        ... // setup tap controller to load prpg
    end;
    shift =
        cycle =
            force tdi # ;
            pulse tck ;
        end;
    end;
end;

```

This procedure could then be applied in the test_setup procedure to initialize the PRPG, specifying the string of bits to apply to “tdi” during shifting.

```

procedure test_setup =
    timeplate tp1 ;
    cycle =
        force clk1 0 ;
        force clk2 1 ;
        force tck 0 ;
        ...
    end;
    apply load_prpg1 tdi = 000000000000000000000001 ;
    cycle =
        ...
    end;
end;

```

For loading a register with the shift length during test_setup, using the values computed by the tool, the procedure definition would look the same, but how the procedure is called is slightly different. The following example both loads and unloads the register, as this same procedure could be used in a test_end procedure to unload the shift length value. The dofile for this example contains the following command:

add_register_value length_val -shift_length -width 16

The procedure file would contain the following:

```
procedure load_unload_registers load_unload_length =
  timeplate tp1 ;
  cycle =
    force clk1 0 ;
    ...
  end;
  shift =
    cycle =
      force tdi # ;
      expect tdo # ;
      pulse tck ;
    end;
  end;
end;
procedure test_setup =
  timeplate tp1 ;
  cycle =
    ...
  end;
  apply load_unload_length tdi = length_val, tdo = XXXXXXXXXXXXXXXXXXXX;
  cycle =
    ...
  end;
end;
```

This next example uses an alias to group three tdi signals into one alias, and also adds a post shift cycle to the load_unload_registers procedure. When this procedure is called, the data passed to it will be consumed three bits at a time, with each bit being shifted into tdi1, tdi2, and tdi3 in order. The total number of shifts applied in the “shift” block will be the total length of the value string divided by three, and then minus one for the post shift. Each shift will consumes three bits, and the final cycle adds a post shift which will assign the last three bits. The length of the value string being passed to this procedure must be a multiple of three.

```
alias TDI_GRP = tdi1, tdi2, tdi3 ;
procedure load_unload_registers load_reg1 =
  timeplate tp1 ;
  cycle =
    force clk1 0 ;
    ...
  end;
  shift =
    cycle =
      force TDI_GRP # ;
      pulse tck ;
    end;
  end;
  cycle =
    force TDI_GRP # ;
    pulse tck;
  end;
end;
```

This final example shows how to use a dofile commands to setup a user-defined register value that will be used to store total number of scan patterns when the final patterns are saved.

```
add_register_value scan_pat_count -pattern_count scan_test -width 24
```

Notes About Using the stil2mgc Tool

You can use the **stil2mgc** tool to create a dofile and procedure file. The **stil2mgc** tool reads a STIL Procedure File (SPF) and then creates a dofile and test procedure file.

The dofile defines clocks, scan chains, scan groups, and pin constraints. The test procedure file contains a timeplate and the following standard scan procedures: test_setup, load_unload, and shift. For more information about this tool, refer to the [stil2mgc](#) command description in the *Tessent Shell Reference Manual*.

Extraction of Strobe Timing Information from STIL (SPF).....	376
The STIL ClockStructures Block.....	376

Extraction of Strobe Timing Information from STIL (SPF)

In the STIL WaveformTable, strobe window and edge strobe are indicated by which event characters are used for the measure timing. Using the ‘H’ and ‘L’ characters indicates that the measure is an edge strobe, while using the ‘h’ and ‘l’ character indicates a window strobe with the length of the strobe window being determined by the following X event in the event list for that WaveformCharacter.

If the window strobe events (‘h’ or ‘l’) are used for a measure event, then the strobe window is calculated taking into account the strobe window indicated by these events. The shortest strobe window calculated is the one used for the procedure file.

If “-edge_strobe_processing on” is specified to the tool, and the edge strobe events (‘H’ or ‘L’) are used for a measure event, then the strobe window is set to 0 in the procedure file to indicate edge strobe timing.

If the “-edge_strobe_processing” option is not used or is set to off, and edge strobe events (‘H’ and ‘L’) are used for a measure event, then the existing behavior is maintained where the strobe window is calculated based on the next force event or the period of the Waveform table. The strobe window is not set to 0.

The STIL ClockStructures Block

When parsing STIL Procedure Files (SPF), stil2mgc recognizes the Synopsys-defined ClockStructures block and uses this information to create clock_control definitions. The Latency statement within the ClockStructures block is used to control the “set_external_capture_options” statement in the generated dofile.

Test Procedure File Commands and Output Formats

The test procedure file is supported by a set of commands and a set of output formats.

Test Procedure File Tool Commands

The following table provides a summary of the tool commands that support the use of test procedure files. For a detailed description of each command, refer to the corresponding command reference page in the *Tessent Shell Reference Manual*.

Table 8-2. Procedure File Tool Command Summary

Command	Description
add_scan_groups	Adds a scan group using the scan procedures in the named procedure file.
read_procfile	Reads a new procedure file in non-setup mode. Merges new procedure and timing data with existing data loaded from previous procedure files.
write_procfile	Writes out existing procedure and timing data as the named procedure file.
write_patterns	Loads a cycle before saving patterns and merges the new data with the existing data.
report_procedures	Reports (displays) a named procedure to the screen. The -All switch displays all procedures to the screen.
report_timeplates	Reports (displays) a named timeplate to the screen. The -All switch displays all timeplates to the screen.

Test Procedure File Output Formats

The test procedure file format supports the following output formats:

- Fujitsu TDL
- Mitsubishi TDL
- STIL
- TI TDL
- WGL
- TSTL2
- VERILOG

The `-PROcfile` switch causes the `write_patterns` command to get its timing information from the procedure file. For more information, refer to the [write_patterns](#) command in the *Tessent Shell Reference Manual*.

Chapter 9

Tessent Visualizer

Tessent Visualizer is a graphical user interface for Tessent Shell. This chapter describes the various features of Tessent Visualizer and helps you understand how to work with those features.

Overview	379
Invoking Tessent Visualizer	380
Framework Overview	381
Common Features	385
Tessent Visualizer GUI Features	386
Schematics	392
Tessent Shell Attributes	405
Gate Report	407
Tessent Visualizer Preferences	407
Windows Overview	411
Hierarchical Schematic Window	411
Flat Schematic Window	421
Instance Browser	424
Cell Library Browser	428
DRC Browser	429
Pin Data Window	432
Transcript Window	434
Text/HDL Viewer	437
Diagnosis Report Viewer	437
Search Features	440
Usage Examples	443

Overview

With Tessent Visualizer, you can use schematics, browsers, tables, and other reports to analyze and understand DFT issues. By cross-referencing between these different viewpoints of your data and performing various analyses on them, you can more quickly arrive at solutions to those issues.

Invoking Tessent Visualizer

There are several ways to invoke Tessent Visualizer, depending on the task you want to perform. Tessent Visualizer can be launched directly from the Tessent Shell application or remotely from a different machine or a different terminal on the same machine.

Note

 To use the legacy DFTVisualizer interface instead, invoke Tessent Shell with the `-dftvis` switch, or set the environment variable “ENABLE_TESSENT_VISUALIZER” to 0 before invoking Tessent Shell. The legacy DFTVisualizer interface will be available for a limited time to facilitate the transition to Tessent Visualizer.

Prerequisites

- The minimum operating system requirements are one of the following:
 - CentOS 6.7 or later
 - CentOS 7
 - RHEL 6.7 or later
 - RHEL 7
 - SLES 12
- The DISPLAY environment variable must be set correctly.

Procedure

1. Set a context using the [set_context](#) command.
2. Load a design using the [read_verilog](#) command.
3. Read a library using the [read_cell_library](#) command.
4. Set the current design using the [set_current_design](#) command.

Note

 You can invoke Tessent Visualizer without issuing these commands, but most of the user interface will be disabled until you load a design and any required libraries, and specify the top level. You can also issue these commands from the Tessent Visualizer Transcript window after you invoke the tool.

5. Invoke Tessent Visualizer using one of the following methods:
 - Explicitly on the same machine Tessent Shell is running on:
 - [open_visualizer](#)

```
ANALYSIS> open_visualizer
// Note: Tessent Visualizer server started on host 'wrkhrs01'
// and listening on port 57061.
// Client authorization code: 1234.
// Note: Tessent Visualizer client successfully started and
// connected to the server.
```

The port and authorization code information enable you to connect to a Tessent Visualizer session from a different machine as described below.

- Implicitly through one of the following commands:
 - [add_schematic_callout](#)
 - [add_schematic_objects](#)
 - [add_schematic_path](#)
 - [add_schematic_connections](#)
 - [analyze_drc_violation](#)
 - [display_diagnosis_report](#)
- Explicitly on a different machine. Tessent Visualizer is a client-server application. Tessent Shell is the server, and Tessent Visualizer is a client of that server.

To start the server in Tessent Shell, invoke the `open_visualizer` command with the `-server_only` option. Optionally, specify the TCP/IP port using the `-tcp_port` option, and a 4-digit authorization code with the `-authorization_code` option. Valid port numbers are from 1024 to 65535.

To start the client, use the **visualizer** startup script:

```
$TESSENT_HOME/bin/visualizer
```

The **visualizer** executable opens a Remote Connection dialog box that prompts for the host, port, and authentication code.

Alternately, specify this information on the command line. See [visualizer](#) in the *Tessent Shell Reference Manual* for more information.

Note

The IP address and port of the remote machine must be accessible to the client machine.

Framework Overview

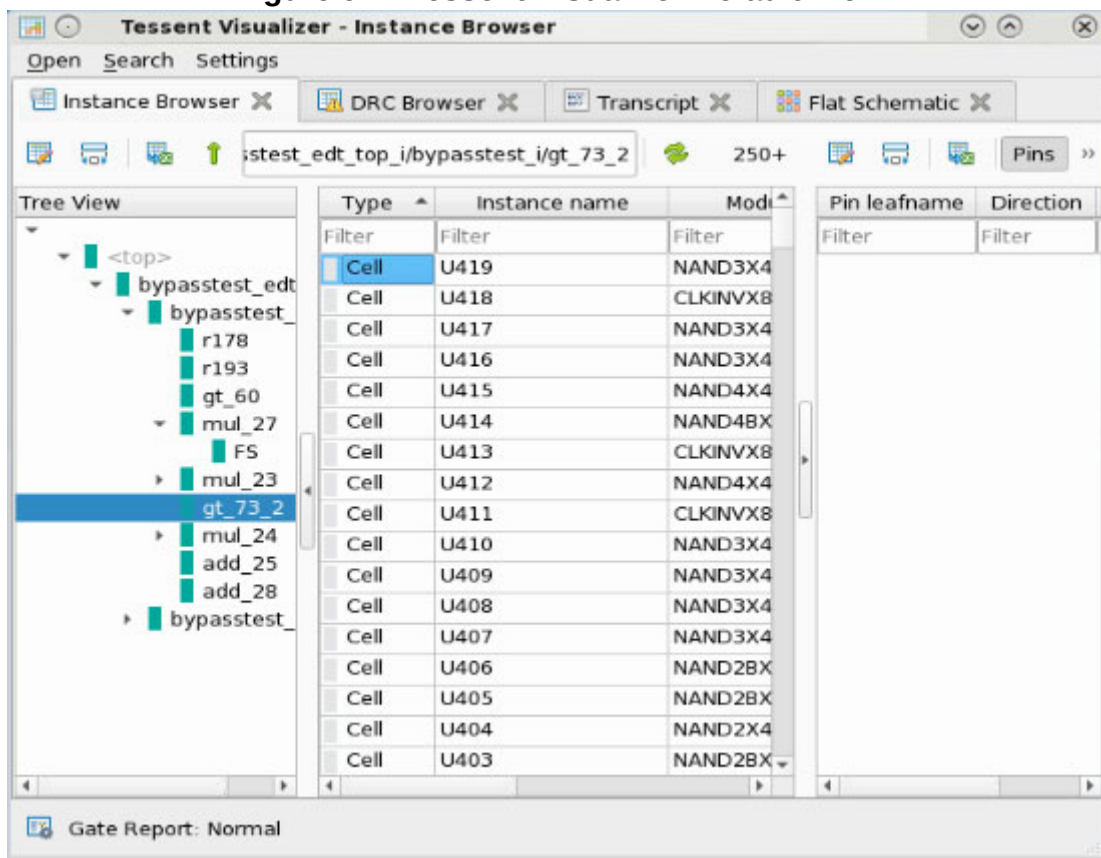
Tessent Visualizer provides multiple tabs for viewing and debugging design and simulation data in multiple windows. Each functional task is placed as a tab. You can cycle through the tabs with Ctrl-Tab or reorder them by dragging and dropping.

The Tessent Visualizer framework consists of:

- **Windows** — Windows contain one or two panes, a global top menu, and a status bar. The menu and status bar are identical across multiple windows; only the pane content differs.
- **Panes** — There are one or two panes per window. By default, one pane provides multiple tabs. Windows can be split with drag and drop into two panes, each having tabs.
- **Tabs** — Grouped within a pane, you can view and select multiple tabs.

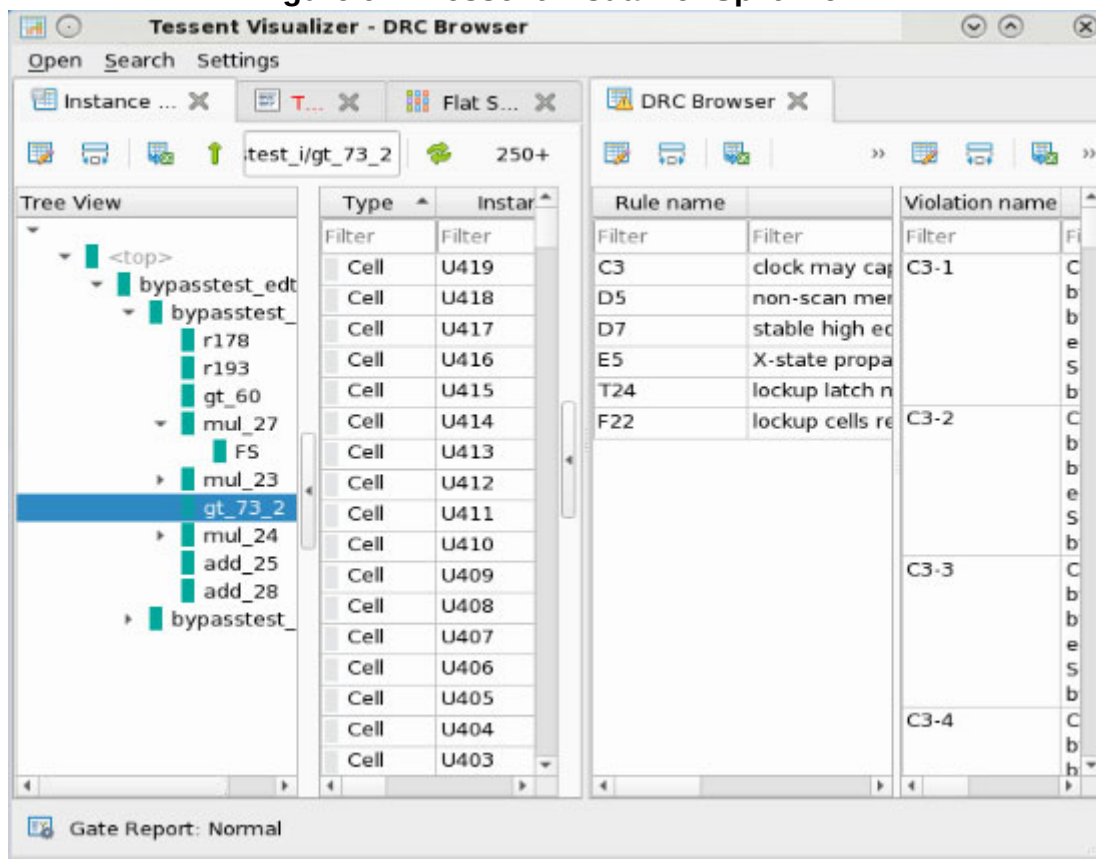
Basic Window Operations

Figure 9-1. Tessent Visualizer Default View



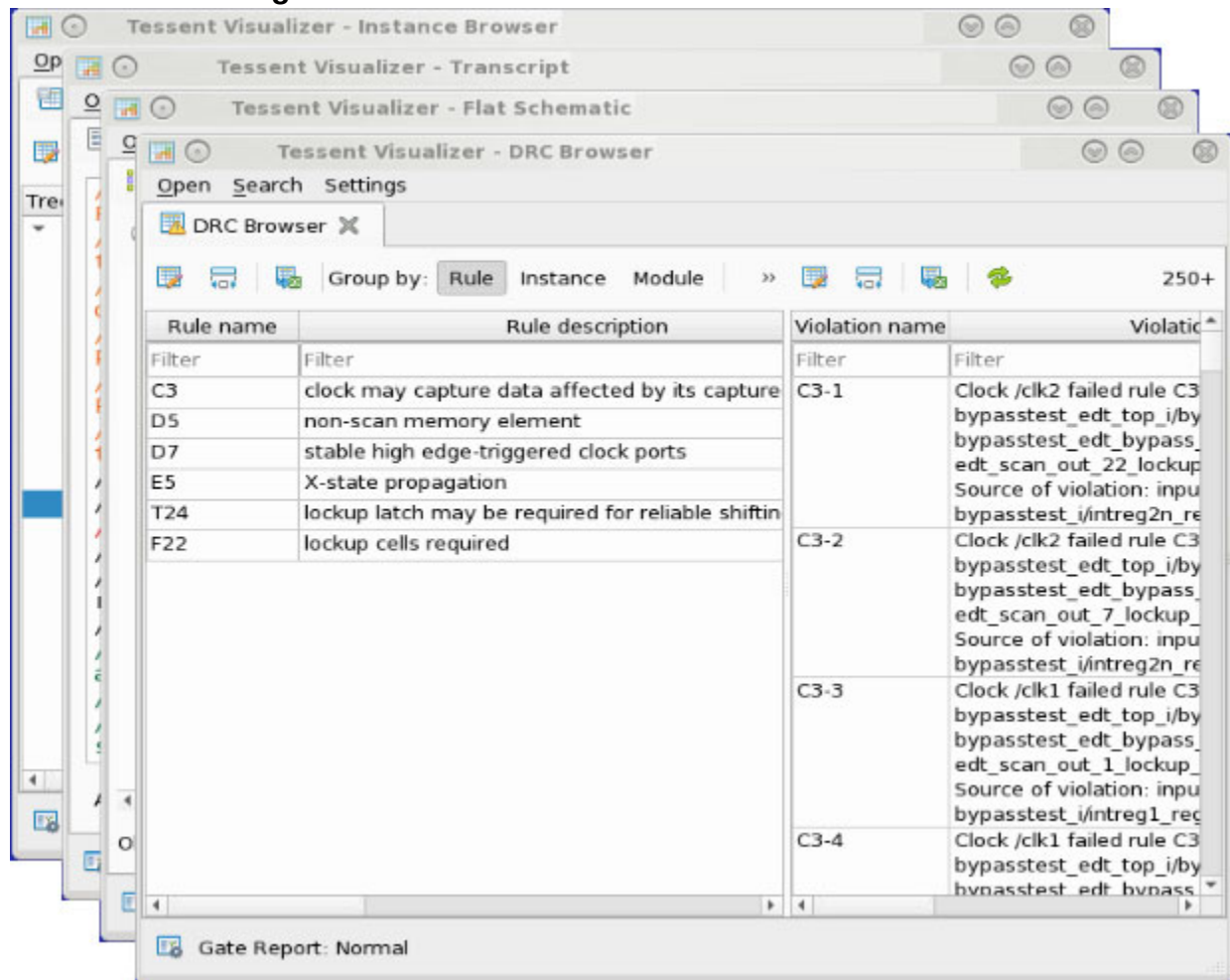
One main window can contain two separate sets of tabs if desired (split view). To create a split in the main window, drag one of the tabs to either side. Drag and drop a tab to move it from one side of the split to the other. There are at most two work areas per main window.

Figure 9-2. Tessent Visualizer Split View



Any tab can be undocked from a window as a separate (floating) window, and redocked as necessary. These have the full functionality of the original window, with access to features such as global settings and the capability to offer a split view. To undock a tab, double-click the tab or drag and drop it away from the parent window. To redock it, drag it back to the parent window (or any other Tessent Visualizer window) and drop it as a tab in that window.

Figure 9-3. Tessent Visualizer Undocked Windows



In addition to clicking the “X”, you can close the tab by middle-clicking it or by choosing **Close tab** from the right-click popup menu.

Common Features

The Tessent Visualizer tools contain the following features in common:

Tessent Visualizer GUI Features	386
Schematics	392
Tessent Shell Attributes.....	405
Gate Report	407
Tessent Visualizer Preferences	407

Tessent Visualizer GUI Features

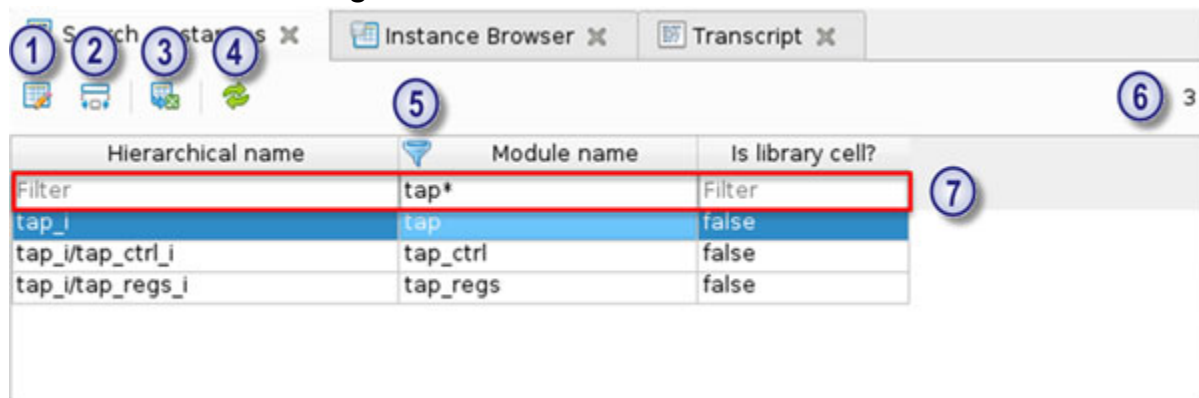
The following features are common to many tabs and windows in Tessent Visualizer:

Common Toolbar Features	386
Tables	387
Columns and Filters Editor	388
Filters	389
Other Common Features	391

Common Toolbar Features

Some toolbar features are common to Tessent Visualizer functions.

Figure 9-4. Common Toolbar Features



Item	Description
①	Columns and Filters Editor. See “ Columns and Filters Editor ” on page 388.
②	Column Width icon. This fits the column widths to accommodate the size of the displayed data.
③	Export Table icon. Table data can be exported in Excel format or as comma-separated values (CSV).
④	Refresh/Abort icon. This resubmits the query that created the table, or halts the loading of data.
⑤	A Funnel icon. This indicates that filtering is active.
⑥	Loaded row counter. If this number includes a “+” suffix, additional data exists in Tessent Shell. Fetch this additional data by using the window scrollbar.

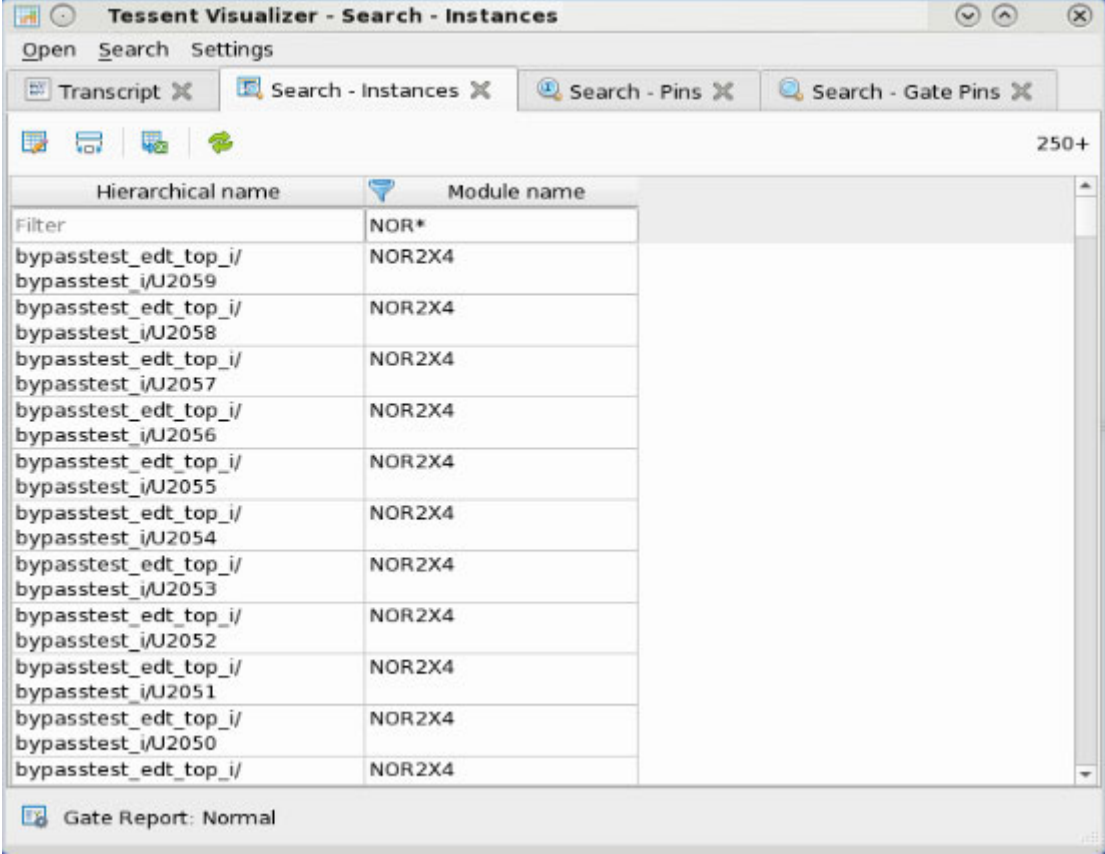
Item	Description
7	Filter fields. These control which data appears.

Tables

Many of the reports provided by Tessent Visualizer appear in tables. These tables are configurable, and data can be filtered dynamically by various criteria.

The following figure shows an example of the tabular report used in the “Search - Instances” window.

Figure 9-5. Search - Instances: Tabular Report



The screenshot shows the 'Tessent Visualizer - Search - Instances' window. It has a menu bar with 'Open', 'Search', and 'Settings'. Below the menu bar are tabs for 'Transcript', 'Search - Instances', 'Search - Pins', and 'Search - Gate Pins'. The 'Search - Instances' tab is active, showing a table with two columns: 'Hierarchical name' and 'Module name'. A filter 'NOR*' is applied to the 'Module name' column, indicated by a funnel icon. The table lists 15 rows of data, all with 'NOR2X4' in the 'Module name' column. The 'Hierarchical name' column contains paths like 'bypass_test_edt_top_i/' followed by instance IDs (e.g., 'iU2059'). A status bar at the bottom shows 'Gate Report: Normal'.

Hierarchical name	Module name
Filter	NOR*
bypass_test_edt_top_i/ bypass_test_iU2059	NOR2X4
bypass_test_edt_top_i/ bypass_test_iU2058	NOR2X4
bypass_test_edt_top_i/ bypass_test_iU2057	NOR2X4
bypass_test_edt_top_i/ bypass_test_iU2056	NOR2X4
bypass_test_edt_top_i/ bypass_test_iU2055	NOR2X4
bypass_test_edt_top_i/ bypass_test_iU2054	NOR2X4
bypass_test_edt_top_i/ bypass_test_iU2053	NOR2X4
bypass_test_edt_top_i/ bypass_test_iU2052	NOR2X4
bypass_test_edt_top_i/ bypass_test_iU2051	NOR2X4
bypass_test_edt_top_i/ bypass_test_iU2050	NOR2X4
bypass_test_edt_top_i/ bypass_test_iU2050	NOR2X4

Note

The filter funnel symbol only appears when a filter is applied.

Select one or more rows in the Tessent Visualizer window and perform various compatible actions on them using the popup menus available by right-clicking. Incompatible actions are dimmed on these menus.

Most report tables in Tessent Visualizer use a spreadsheet-like format with customizable columns, a toolbar, and filter fields for each column. Customize the data types included in the table with the Columns and Filters Editor (see “Columns and Filters Editor” on page 388).

Export table data with the Export Table icon (see Figure 9-4 on page 386). Choose standard CSV or Microsoft Excel compatible CSV from the Export Table dialog box that appears.

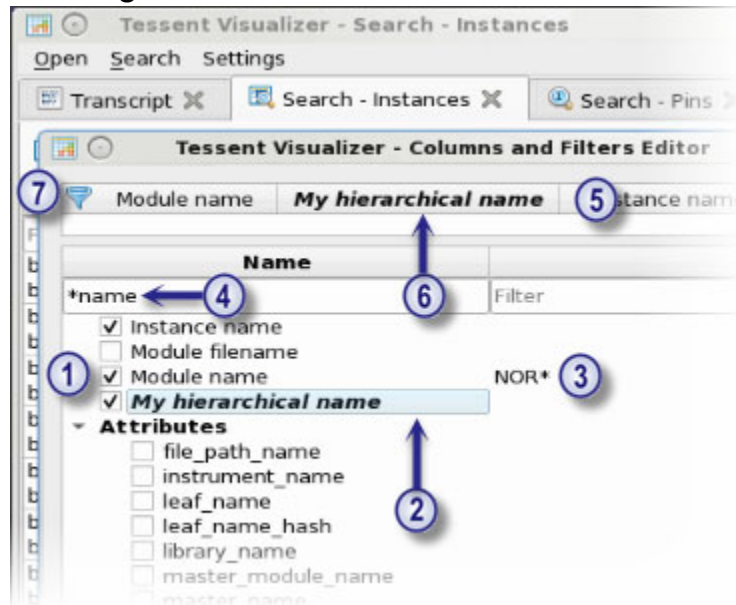
Add line numbers to tables from the Preferences window. (See “Tessent Visualizer Preferences” on page 407.) This feature is turned off by default.

Columns and Filters Editor

Use the Columns and Filters Editor to control the content and format of tables.

The following figure shows features of the Columns and Filters Editor:

Figure 9-6. Columns and Filters Editor



Item	Description
1	Table columns are added or removed by checking or unchecking the boxes associated with column names.
2	Column label names can be customized by double-clicking the name representing the column. The name change is indicated by a bold italic font.
3	Column filters can be added or edited by double-clicking the filter cell.
4	Specific data types can be searched for by using simple string matching in the relevant column.

Item	Description
5	Columns can be reordered by dragging and dropping the header items at the top of the Column and Filters Editor window.
6	Customized column label names are indicated by bold italics in the top-of-window header items as well as in the Column Editor tree.
7	The funnel icon indicates that filtering is being applied on a column.

The affected report table refreshes when you accept the modifications indicated by the Columns and Filters Editor by clicking **OK**. By default, up to 250 items are loaded. Scroll down to request more. Change the default threshold from the Preferences window. See [“Tessent Visualizer Preferences”](#) on page 407.

Filters

Filters control which data is displayed in tables.

The filter field is located directly beneath the column headers, as shown in [“Common Toolbar Features”](#) on page 386. Tessent Visualizer uses the following filters:

- **Exact match** — A string literal or value, optionally prepended with ‘=’. This restricts the display of data to those objects matching the string or value. String literals containing spaces and characters such as ‘=’ should be quoted with single quotation marks; for example ‘x = y’.
- **Exact inverse match** — A string literal or value prepended with ‘!=’. This restricts the display of data to those objects not matching the string or value.
- **Logical operators** — “AND”, “OR”, and “NOT”. Examples:
 - “!= abc AND != xyz” restricts the display of data to all objects that are not named “abc” and are not named “xyz”.
 - “abc OR xyz” restricts the display of data to those objects named “abc” or “xyz”.
 - “NOT” is a special logical modifier and requires an explicit “REGEXP” or “GLOB” (see following).
- **Wildcard matching** — The keyword “GLOB” with “?” (which represents a single character) and “*” (which represents zero or more characters). Examples:
 - “GLOB ‘*abc’” restricts the display of data to all objects with names ending in the string “abc”.
 - “GLOB ‘*abc*’” restricts the display of data to all objects with names that contain the string “abc”.

- `"GLOB 'abc?xyz'"` restricts the display of data to all objects with seven-character names that begin with the string "abc", followed by a single character, and end with the string "xyz".
- `"GLOB '?!'"` will match any non-empty string.

Note

 In GLOB filters, if there is no empty space as part of the expression, the keyword "GLOB" and the single quotation marks are optional. Example: `"*abc*"`

- **Numerical comparison operators** — Data that includes numerical information can be filtered using standard numerical comparison operators (<, <=, >, >=).
- **Standard regular expressions** — Data can be filtered using regular expressions by using the construct `"REGEXP 'RE'"`. For example, `"REGEXP '.*clk_\d+$'"` restricts the display of data to all objects with names ending in the string "clk_" followed one or more decimal digits.

Note

 In this filter, the keyword "REGEXP" and the single quotation marks are required.

Table 9-1. Filtering Summary

Filter Type	Filter Format	Examples
Exact match	<i>value</i> <i>=value</i>	<code>abc</code> <code>=xyz</code> <code>=12</code>
Exact match (inverse)	<i>!=value</i>	<code>!=abc</code>
Numerical operators	< <= > >=	<code>>10</code> <code>>20 AND <= 50</code>
Wildcards	<code>GLOB 'expression'</code>	<code>GLOB 'abc*'</code> <code>GLOB '*xyz*'</code> <code>GLOB 'abc?xyz'</code> <code>GLOB 'abcdef??'</code>
Regular expression	<code>REGEXP 'expression'</code>	<code>REGEXP '^abc.*'</code> <code>REGEXP '.*_\d0\$'</code>

Table 9-1. Filtering Summary (cont.)

Filter Type	Filter Format	Examples
Logical operators	AND OR NOT GLOB <i>'expression'</i> NOT REGEXP <i>'expression'</i>	<code>abc OR xyz</code> <code>!=abc AND != xyz</code> <code>NOT GLOB 'abc*'</code> <code>NOT REGEXP '^abc.*'</code>

Note

 Filters applied to table columns are processed in Tessent Shell on the complete data model, no matter how many resulting rows appear in the Tessent Visualizer tabular reports.

Other Common Features

Other common features to Tessent Visualizer functions include column ordering, data sorting, colors, context menus, and session state saving.

Column Ordering

Change the ordering of columns in tables by clicking the column title cell and dragging the column left or right. Resize columns to fit their cells' contents by clicking the column resizer in the toolbar or by double-clicking the column separators.

Data Sorting

The default order of rows in tabular data is determined by the underlying data structure in Tessent Shell. In some tables, you can sort this data by clicking the column title cell. The ordering cycles through each of ascending, descending, and the default ordering.

Tip

 For better performance, limit the set of visible data with filtering before using the sorting function.

Row Highlighting

Highlighted table rows indicate objects shown in schematic windows. You can also use a Boolean value in the 'Displayed' column for easy filtering of objects that are currently in a particular schematic.

Figure 9-7. Selection Colors

Hierarchical name	 Module name	Is library cell?
Filter	tap*	Filter
tap_i	tap	false
tap_i/tap_ctrl_i	tap_ctrl	false
tap_i/tap_regs_i	tap_regs	false

The dark highlighting in this figure shows the selected object (in this case a row). The light highlighting shows the active cell. Most right mouse button popup menus operate on the selected object (dark). Copy and paste actions (Ctrl-C and Ctrl-V) work only on the active cell (light).

Ctrl-right-click to select multiple adjacent objects in a table. Ctrl-left-click to select multiple nonadjacent objects.

Context Menus

Access popup context menus for selected objects, or objects currently under the mouse pointer, by right-clicking. The actions and options available in those context menus depends on the type of objects they relate to.

Saving the Session State

General user settings such as the tabs appearing in a window and preferences are stored in a hidden file in your home directory. Some more specific actions such as instance highlighting and filtering will persist in memory until the Tessent Visualizer server is closed. For example: if you close a Tessent Visualizer client and later start a new one connected to the same server, the session will appear as you left it.

Schematics

Tessent Visualizer schematic windows contain features that enable you to inspect and navigate your design.

Figure 9-8 shows an example flat schematic window in Tessent Visualizer. The following GUI objects are highlighted:

- **Toolbar** — Control how objects are displayed in the schematic window and export schematic objects for use by other tools.
- **Address Bar** — Displays the name of the currently-selected object. You can also use the address bar to add new objects to the schematic by name.
- **Selection Buttons for Context Table** — Select which context table you want to display.

Figure 9-8. Schematic Window Elements (1 of 3)

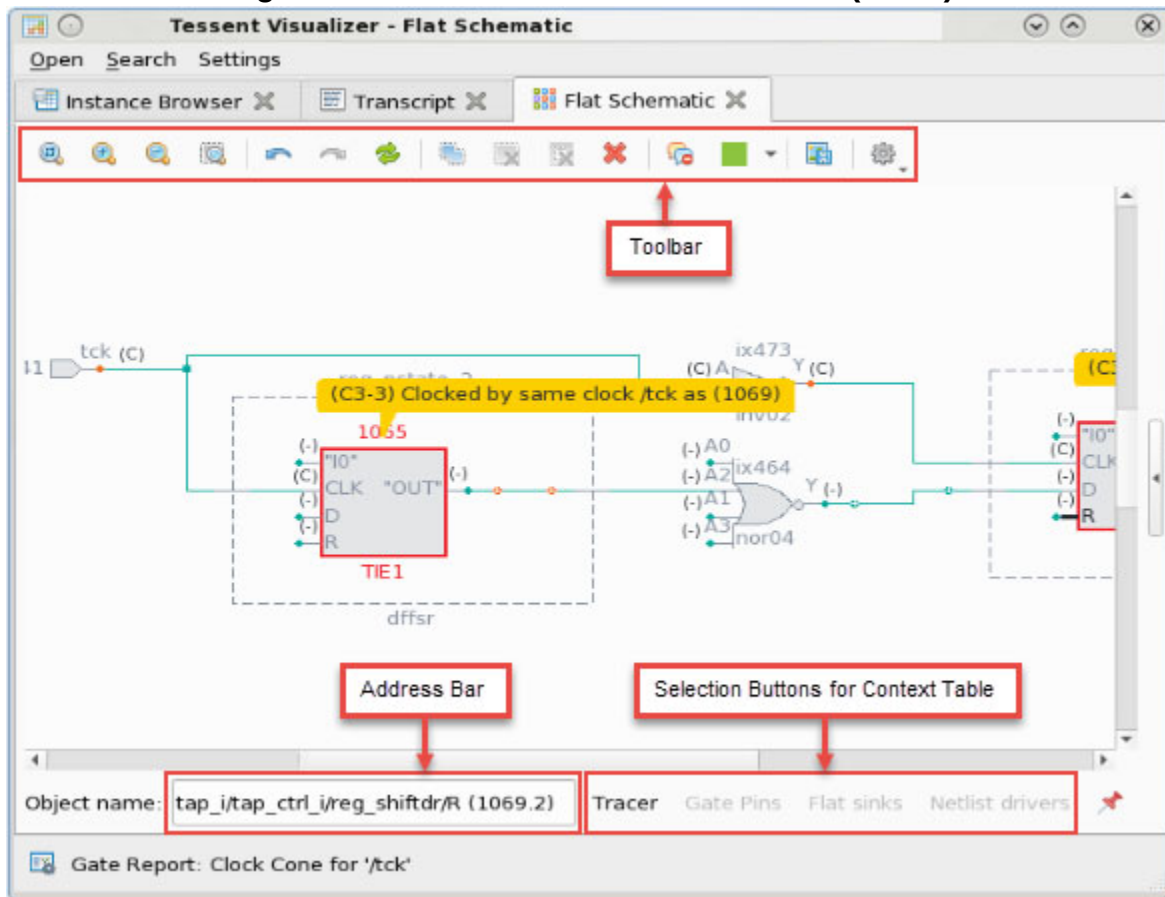


Figure 9-9 illustrates schematic window common features for a selected object.

- **Attributes Table for the Selected Object** — Displays information about the currently-selected object.
- **Vertical Splitter** — Click or slide left and right to reveal or conceal the attributes table and the graphical overview.
- **Horizontal Splitter** — Slide up and down to resize the attributes table and the graphical overview.
- **Overview Window** — Show all objects that have been added to the schematic window. The currently zoomed view is highlighted.

Figure 9-9. Schematic Window Elements (2 of 3)

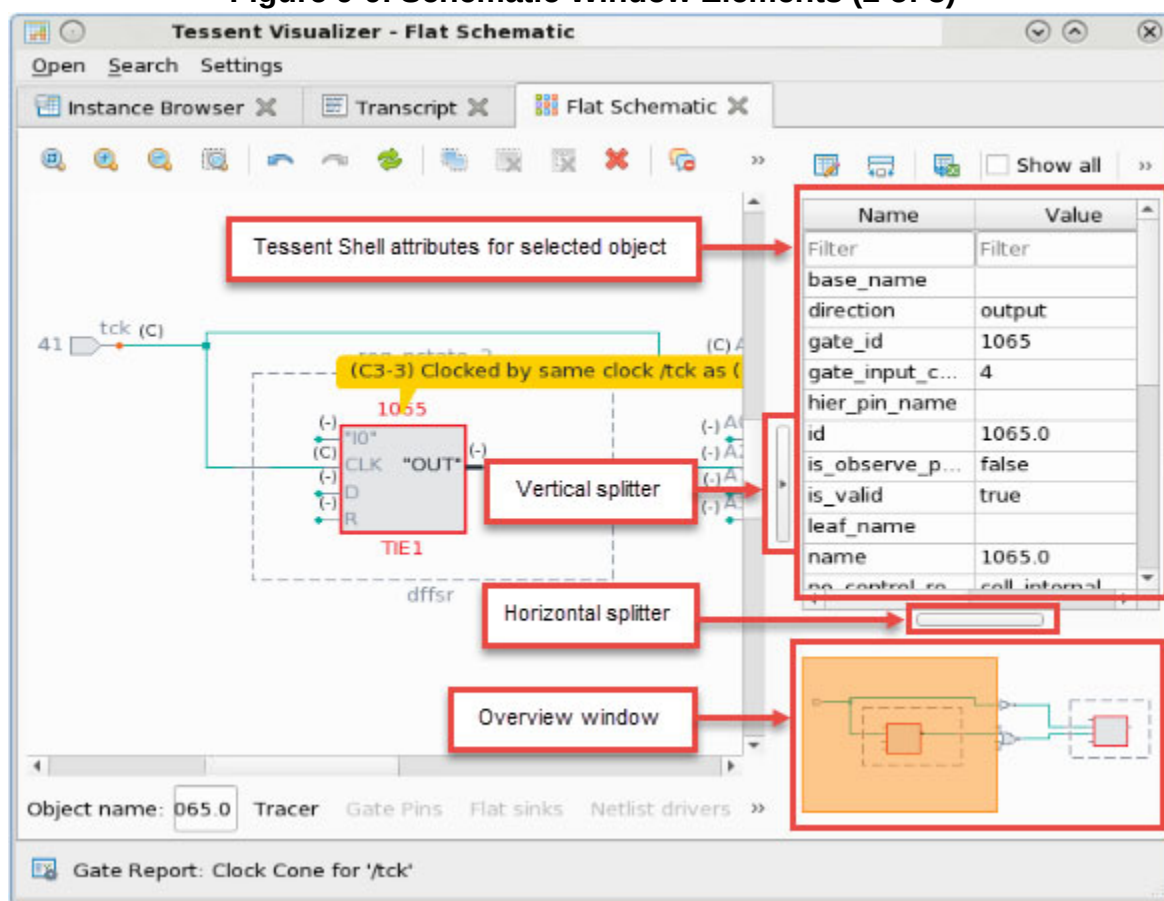
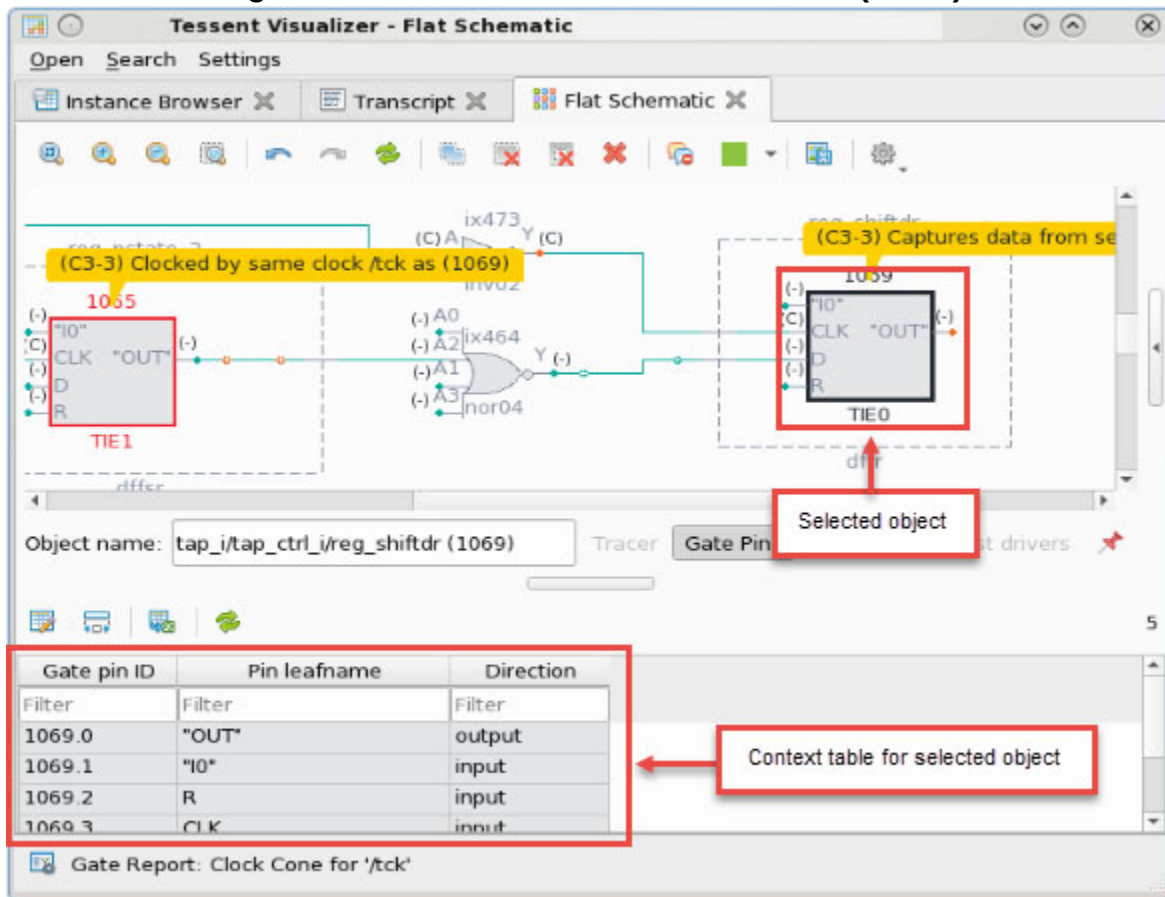


Figure 9-10 shows an example of a context table listing the data for the selected object.

- **Context Table** — Shows information about the currently-selected object. The specific data in this table depends on the object selected.

Figure 9-10. Schematic Window Elements (3 of 3)



The following sections describe Tessent Visualizer schematic window features in more detail.

Toolbar

The toolbar at the top of the schematic window includes the following:

Table 9-2. Actions Available From the Schematic Window Toolbar

Tool	Description
	Zoom all: fit the current view of the schematic in the window.
	Zoom in.
	Zoom out.
	Zoom selected: zoom in on and center the selected object or objects.
	Undo: undo the previous action.

Table 9-2. Actions Available From the Schematic Window Toolbar (cont.)

Tool	Description
	Redo: redo the previous action.
	Redraw: refresh the schematic view.
	Select all: select all objects in the schematic view.
	Delete selected: delete all selected objects from the schematic view.
	Delete unselected: delete all objects from the schematic view except those currently selected.
	Delete all: delete all objects from the schematic view.
	Collapse callouts: collapse all callout messages to their icon representations, or uncollapse if currently collapsed.
	Mark / Unmark: highlight the currently selected object or objects with the currently-selected color, or clear the highlighting on the currently selected object. If the color you want to use is already chosen here, you can also use Ctrl-M to highlight the selected object(s).
	Export schematic: save the current view of the schematic in PDF or PostScript format.
	Options: set schematic-specific options (different schematic types have different options in this submenu).

Dragging and Dropping

Use the mouse to drag and drop objects from browsers or tables into schematic tabs, or from one schematic tab to another, as shown in [Figure 9-11](#) on page 397 and [Figure 9-12](#) on page 398.

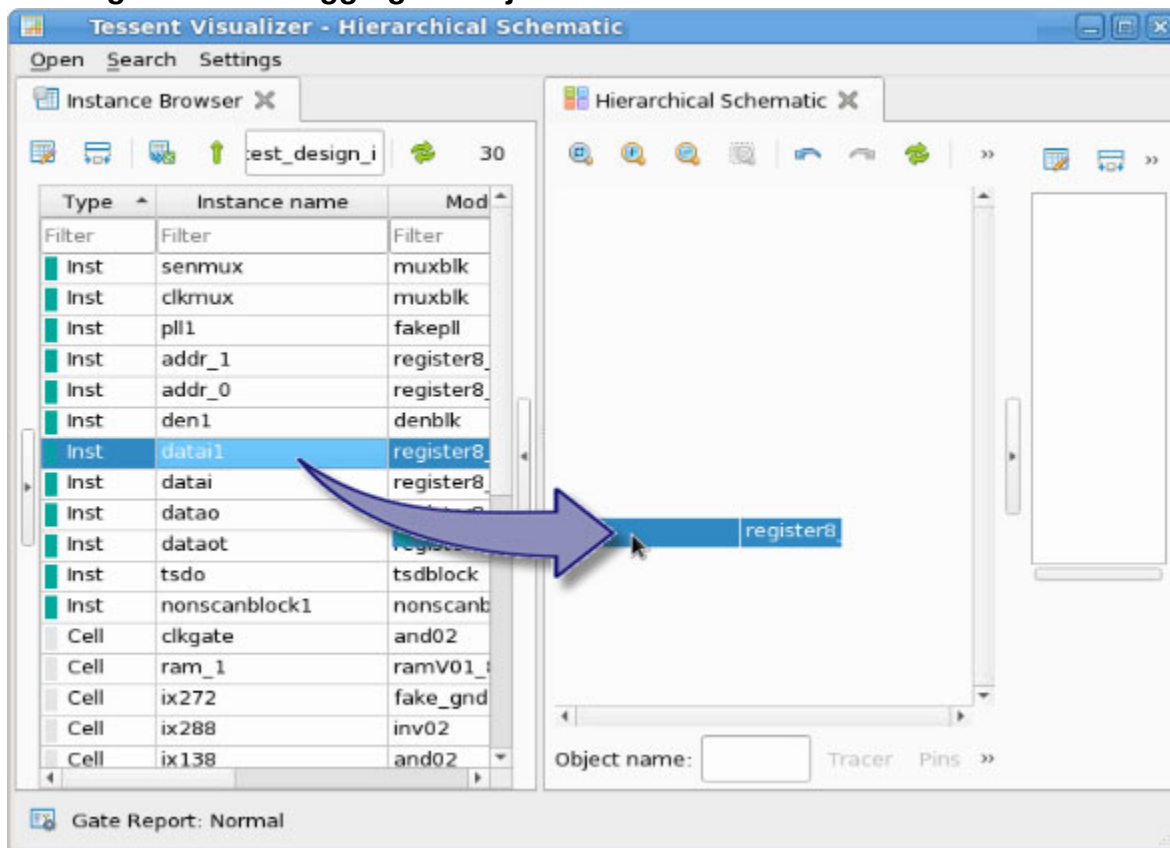
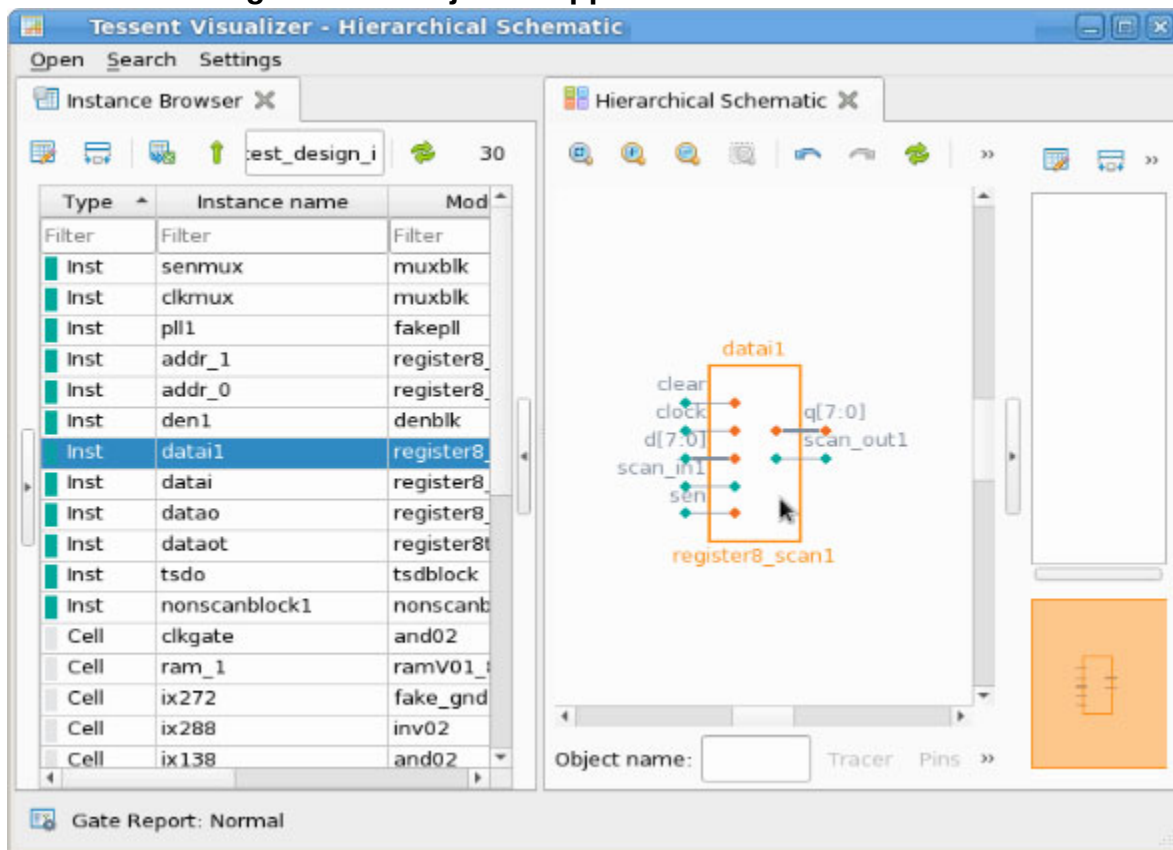
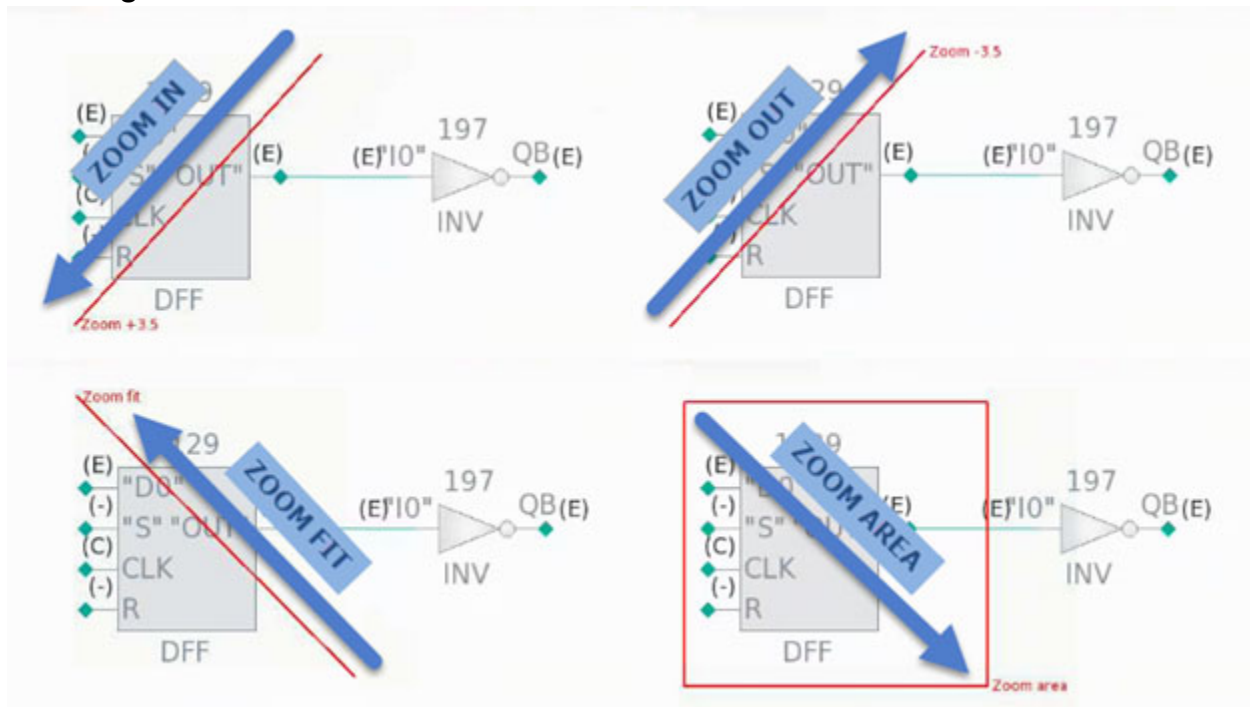
Figure 9-11. Dragging an Object From a Browser to a Schematic Tab

Figure 9-12. Object Dropped Into a Schematic Tab



Mouse Gestures

Zoom in or out within schematic windows using the left mouse button. The following figure summarizes the four mouse gestures (stroke commands) available to perform a zoom.

Figure 9-13. Mouse Gestures in Tessent Visualizer Schematic Windows

You can also zoom in and out using the scroll wheel of your mouse.

Additional mouse gestures are available:

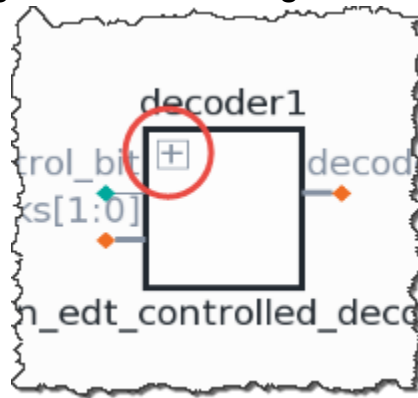
- Shift-click and drag with the left mouse button: area select.
- Click and drag with the scroll wheel: pan the view.

Customize the mapping of mouse gestures to specific mouse buttons and keyboard modifiers in the Preferences window. See “[Tessent Visualizer Preferences](#)” on page 407.

Folding and Unfolding

Both cell grouping boxes (in the Flat Schematic window) and instances (in the Hierarchical Schematic window) can be folded and unfolded (collapsed and uncollapsed). Double click the grouping box or instance to do this. You can also unfold hierarchical instances by clicking the “+” symbol at the top-left corner as shown in [Figure 9-14](#) on page 400.

Figure 9-14. Unfolding an Instance



Attributes Table

The Attributes table lists the Tessent Shell attributes for the selected schematic objects. This is equivalent to issuing the [report_attributes](#) command. By default, only attributes with non-default values or with the “show default” flag set are shown. Click the “Show all” check box to display all properties.

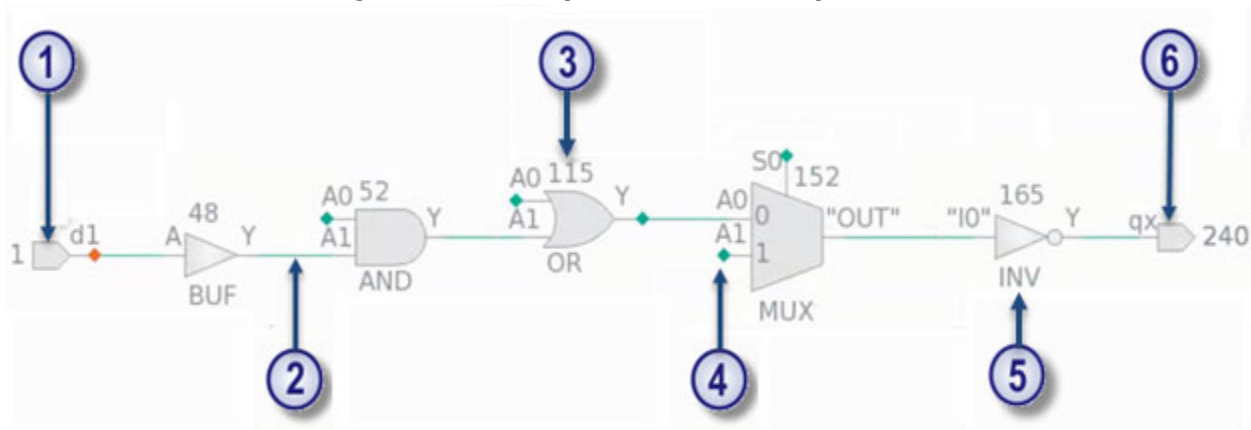
Figure 9-15. Attributes Table

GUI	Name	Value	Inhe
F...	Filter	Filter	Filter
	cell_type	dff	modul
	has_no_definition	false	modul
	is_created	false	modul
	is_elaborated	true	modul
	is_hard_module	true	modul
	is_modified	false	modul
	is_non_editable	true	
	is_non_editable_reas...	hard_module	
	is_unsaved	false	modul
	is_valid	true	
	language	tessent_lib	modul
	leaf_name	reg_serial_output	
	master_module_name	dff	
	module_name	dff	
	name	bsr_i1/ bsc_v_dout_6/ reg_serial_output	
	object_id	instance:16	
	object_type	instance	

Schematic Symbols

The following figure shows common symbols and properties in Tessent Visualizer schematic windows.

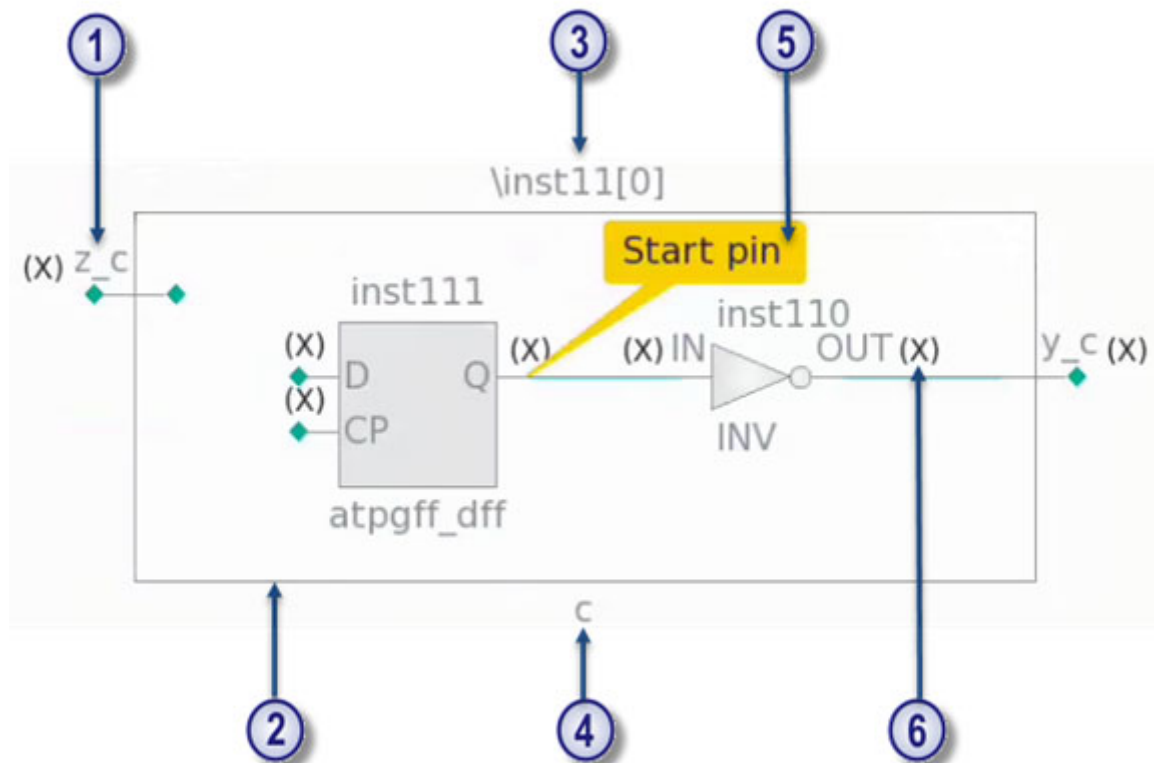
Figure 9-16. Properties and Components



Item	Description
①	Primary Input.
②	Net.
③	Gate ID or Instance Name.
④	Pin.
⑤	Module Name.
⑥	Primary Output.

In addition to conventional schematic symbols, Tessent Visualizer schematics depict complex components (library cells) enclosed by rectangles. Callouts convey information about specific design objects in certain contexts, and simulation data is associated with design pins when appropriate.

Figure 9-17. Complex Component With Callout and Pin Data



Item	Description
①	Pin Name.
②	Complex Component (hierarchical instance).
③	Instance Name (leaf name only).
④	Module Name.
⑤	Callout.
⑥	Data.

Tip

 Move the callout objects in the schematic window with Ctrl-LMB (left mouse button).

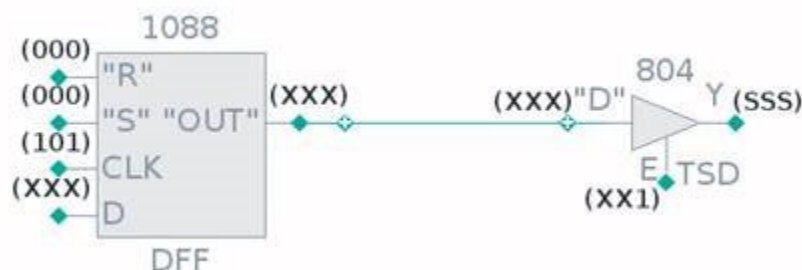
Marker symbols indicate other schematic features:

Table 9-3. Marker Symbols

Symbol	Description
	Green diamond: single path (for connectivity tracing).
	Orange diamond: multiple paths (for connectivity tracing).
	Blue diamond: bidirectional port.
	Red triangle: coercion (connection against declared direction).
	Blue square: RTL connection (no connection in hierarchical model).
	Orange polygon: pseudo-port (primary input/output added by the user or tool).
	Diamond with "+": collapsed buffers and/or inverters. Click this symbol to uncollapse.
	Diamond with "-": uncollapsed buffers and/or inverters. Click this symbol to collapse.
	Circle with "X": no connection.
	Scissors: cut point (connections cut or disconnected by the user or the tool).
	Collapsed callout.

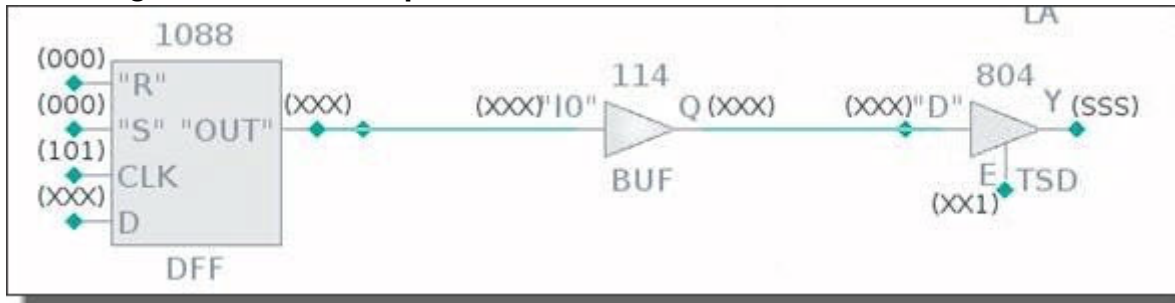
The collapsed/uncollapsed symbols are shown in either green or orange. Green indicates that there are no hidden fanouts associated with the collapsed objects:

Figure 9-18. Collapsed Buffers/Inverters With no Hidden Fanout



After uncollapsing, there is no additional fanout:

Figure 9-19. Uncollapsed Buffers/Inverters With no Hidden Fanout



Orange indicates that there is additional fanout hidden by the buffer/inverter collapsing:

Figure 9-20. Collapsed Buffers/Inverters With Hidden Fanout

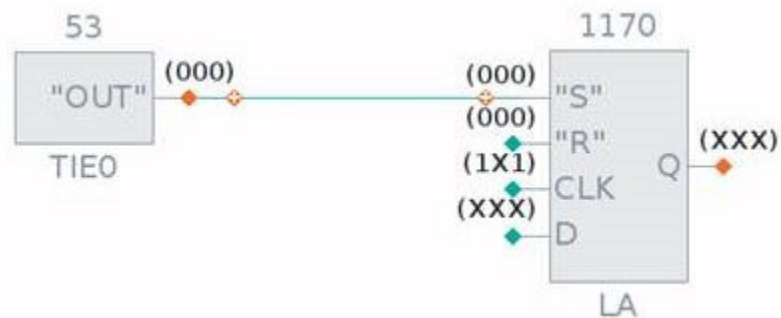
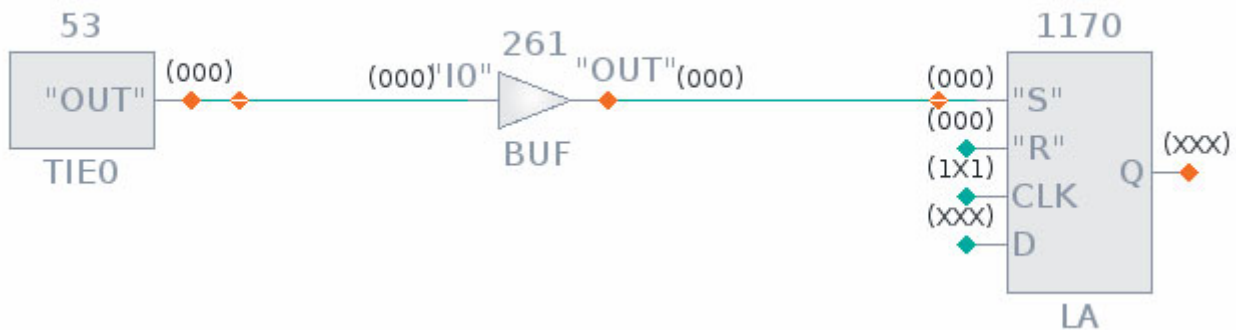
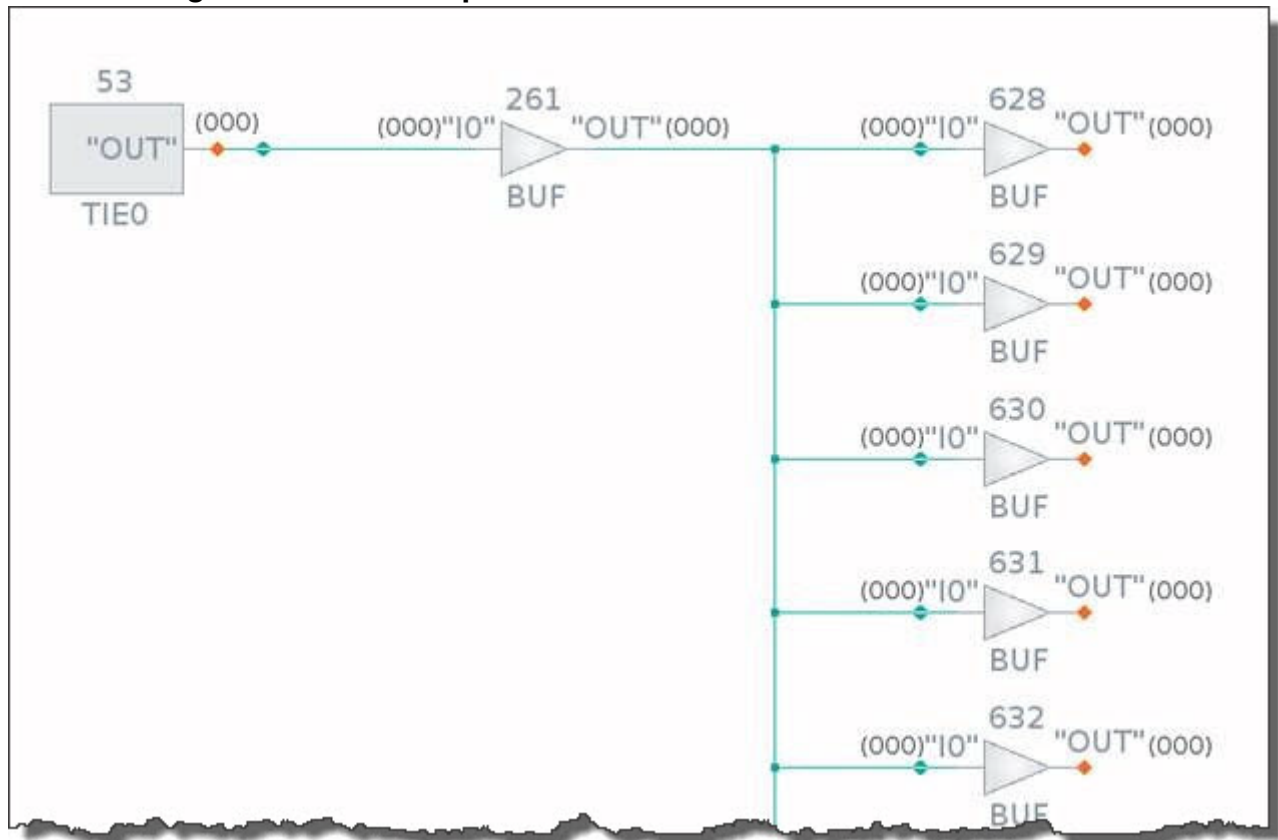


Figure 9-21. Uncollapsed Buffers/Inverters With Hidden Fanout



Additional objects appear after adding the fanouts to the schematic, and the color of the markers changes to green:

Figure 9-22. Uncollapsed Buffers/Inverters With Fanout Revealed



Note

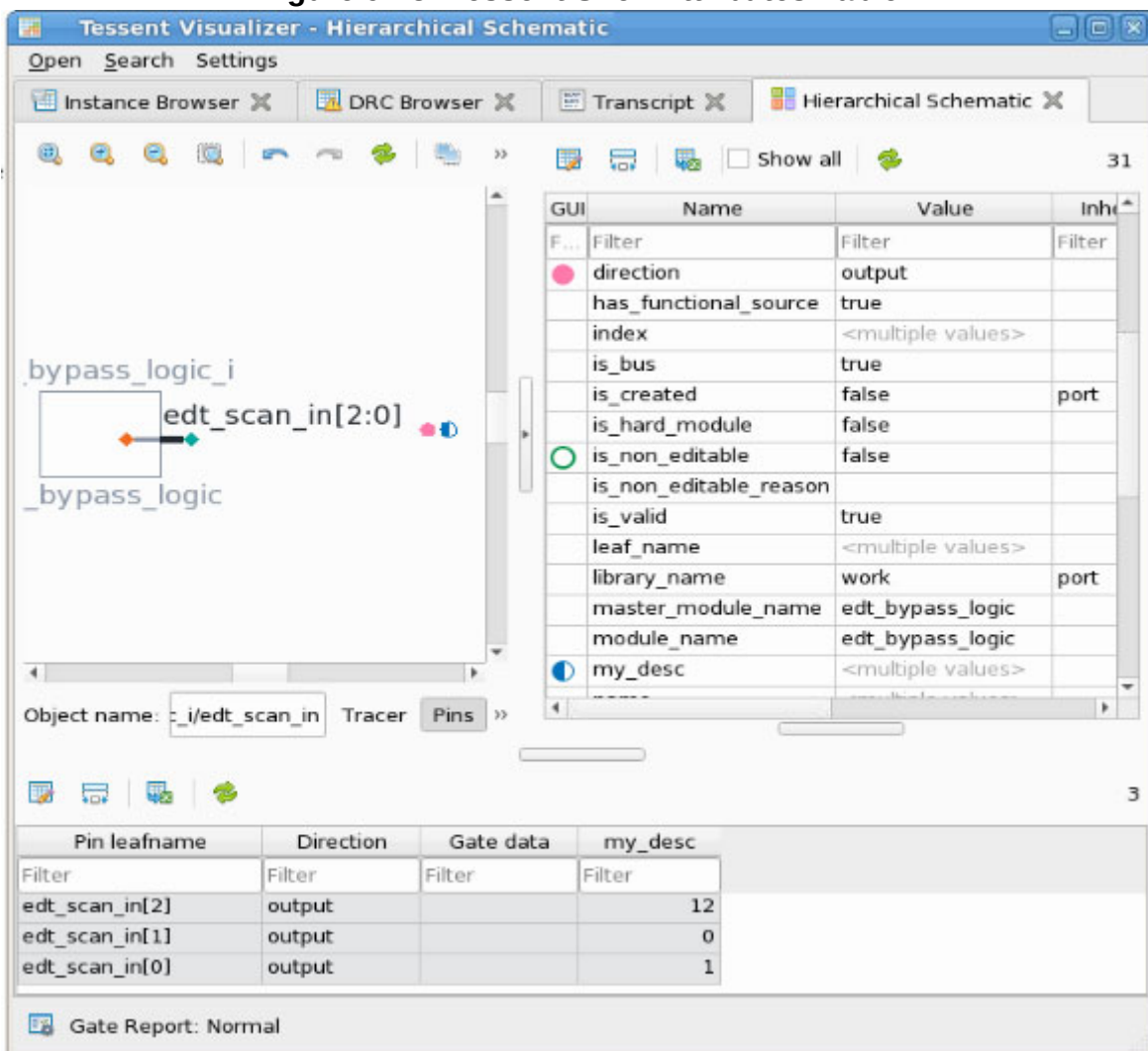
Only buffers and inverters added implicitly to the schematic as a result of tracing are collapsible. Buffers and inverters added explicitly by name or gate ID are not collapsible.

Tessent Shell Attributes

Tessent Visualizer includes functionality to view Tessent Shell attributes and annotate the schematic window with them.

As shown in the overview of the schematic window in [Figure 9-9](#) on page 394, the Tessent Shell attributes for the currently selected object in the schematic window are shown in a table. More detail is described here and illustrated in [Figure 9-23](#) on page 406.

Figure 9-23. Tessent Shell Attributes Table



By default, only those attributes with non-default values, or which were registered with the “-show_default” option, are shown. Click the “Show all” checkbox to show all attributes, including those with default values.

Double-click a cell in the GUI column to select a color in the GUI marking index and show a corresponding marker near schematic objects which have that attribute set to a non-default value. Hover the mouse pointer over that marker in the schematic window to show a tooltip with the attribute name and value.

Note

For boolean attributes, the marker is only shown if the value is set to “true.”

An empty circle in the GUI column indicates that the attribute is set to the default value. A corresponding marker is not displayed in the schematic window in this case. A half-filled circle

indicates that not all of the components of the selected object (for example, a bus) are set to the same value.

Gate Report

Click the gear icon at the bottom of any window to launch a Gate Report settings dialog box. This is a GUI front end for the [set_gate_report](#) command.

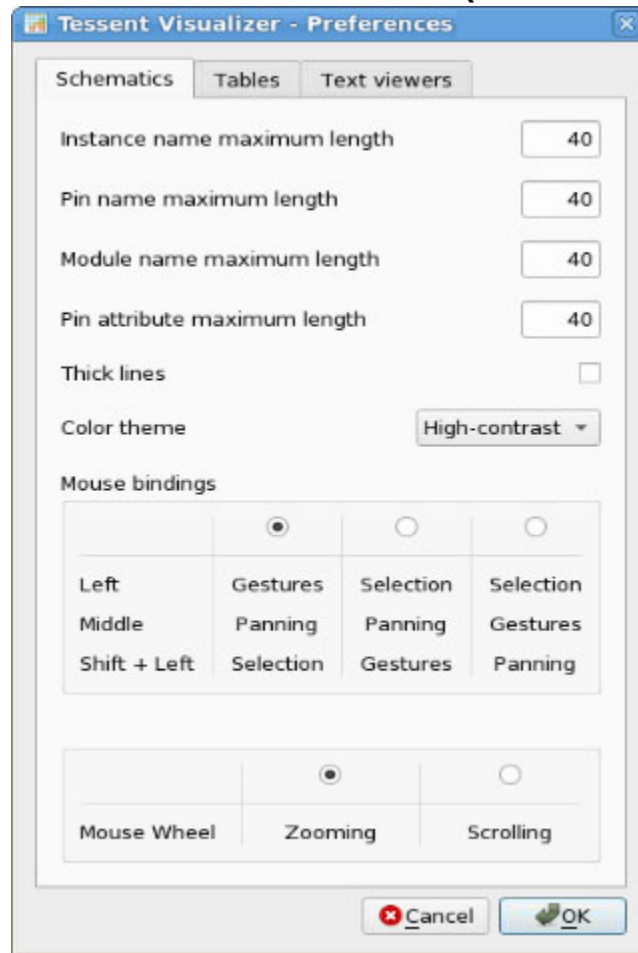
Tessent Visualizer Preferences

Customize Tessent Visualizer with the Preferences dialog box, found under the **Settings** menu at the top of the main window.

The Preferences dialog box contains three tabs: **Schematics**, **Tables**, and **Text viewers**.

Use the Schematics tab to define how schematic windows look and operate. Set maximum lengths for names, choose the color theme, set the width of net representations, and redefine how the mouse works when interacting with schematic windows.

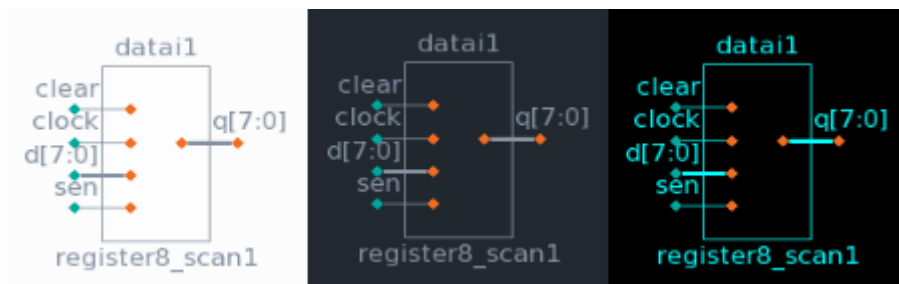
Figure 9-24. Preferences Window (Schematic Tab)



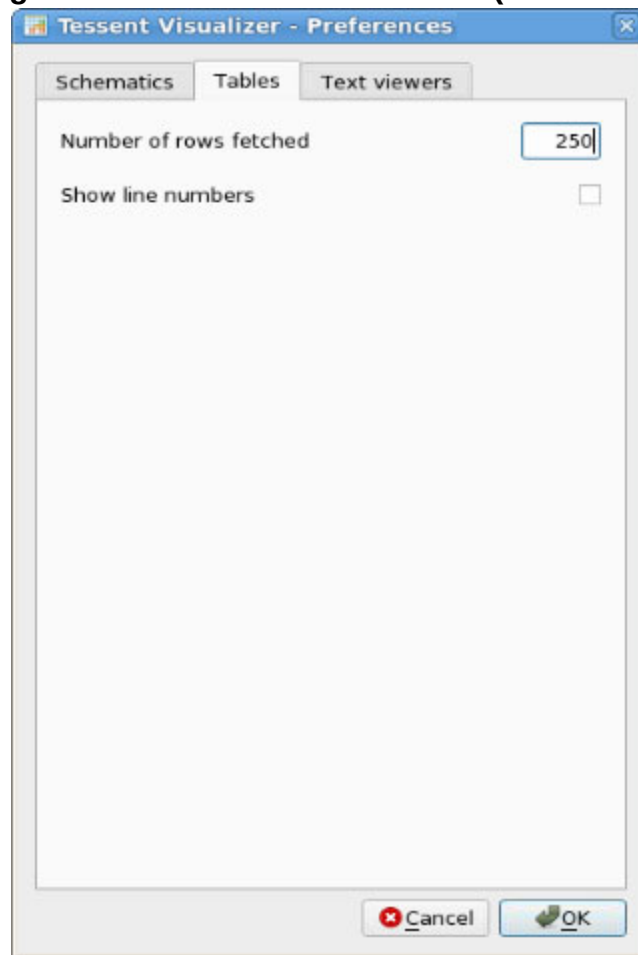
There are three predefined color themes available in Tessent Visualizer schematic windows: light, dark, and high-contrast. Set the color scheme with the Preferences menu or the `set_visualizer_preferences` command:

`set_visualizer_preferences -color_theme light|dark|high-contrast`

Figure 9-25. Light, Dark, and High-Contrast Color Themes in Tessent Visualizer Schematic Windows



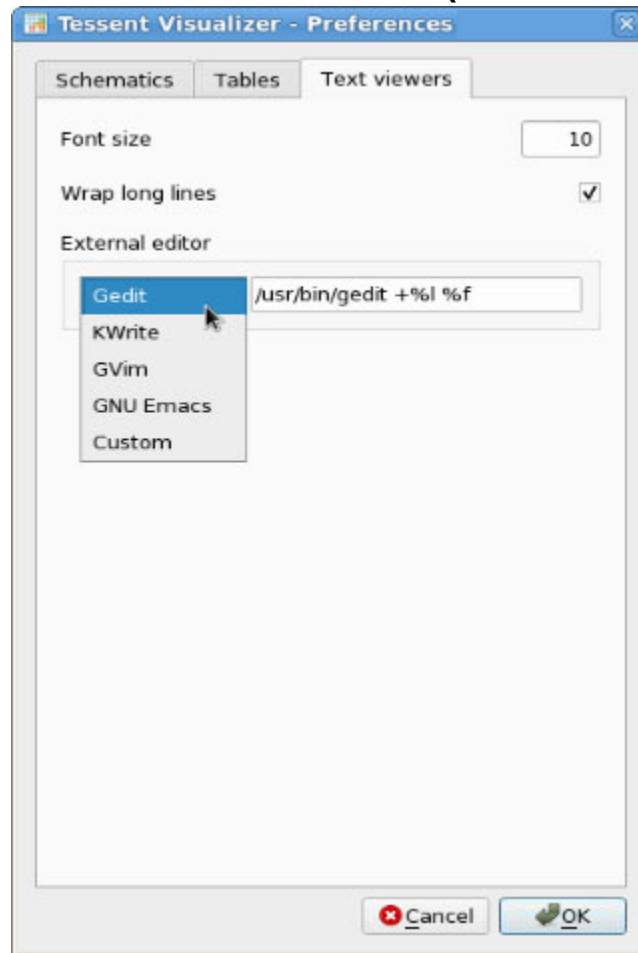
Use the Tables tab to set the maximum number of rows fetched in tables or to show line numbers in tables.

Figure 9-26. Preferences Window (Tables Tab)

Use the Text viewers tab to set the font size and line-wrapping policy in the Text Viewer, and to define an external text editor that can be launched from the Text/HDL Viewer tab. Choose from several common text editors, or define your own preferred one. The variables “%l” (line number) and “%f” (filename) are provided for use as arguments to the text editor executable. For example, this text editor definition opens the file currently in the Text/HDL Viewer tab in the gvim editor with the cursor positioned at the current line:

```
gvim +%l %f
```

Figure 9-27. Preferences Window (Text Viewers Tab)



Caution

 Tessent Visualizer invokes whichever executable you specify as the external text editor. Ensure that a valid and working option is set here.

Windows Overview

The Tessent Visualizer windows enable you to explore your design, trace nets, obtain reports about design objects, cross-reference those objects between windows, display or hide design objects, and export your entire design or portions of your design to various file types.

There are three menus available at the top of the main window: **Open**, **Search**, and **Settings**. From the **Search** menu, search for hierarchical pins, gate pins, and instances. Set preferences and obtain version information from the **Settings** menu. Open the following tabbed windows from the **Open** menu:

Hierarchical Schematic Window	411
Flat Schematic Window	421
Instance Browser	424
Cell Library Browser	428
DRC Browser	429
Pin Data Window	432
Transcript Window	434
Text/HDL Viewer	437
Diagnosis Report Viewer	437

Hierarchical Schematic Window

From the Hierarchical Schematic window, navigate your design by tracing fanout and fan-in, selecting various components, and obtaining information about those paths and components.

Figure 9-28. Hierarchical Schematic Window

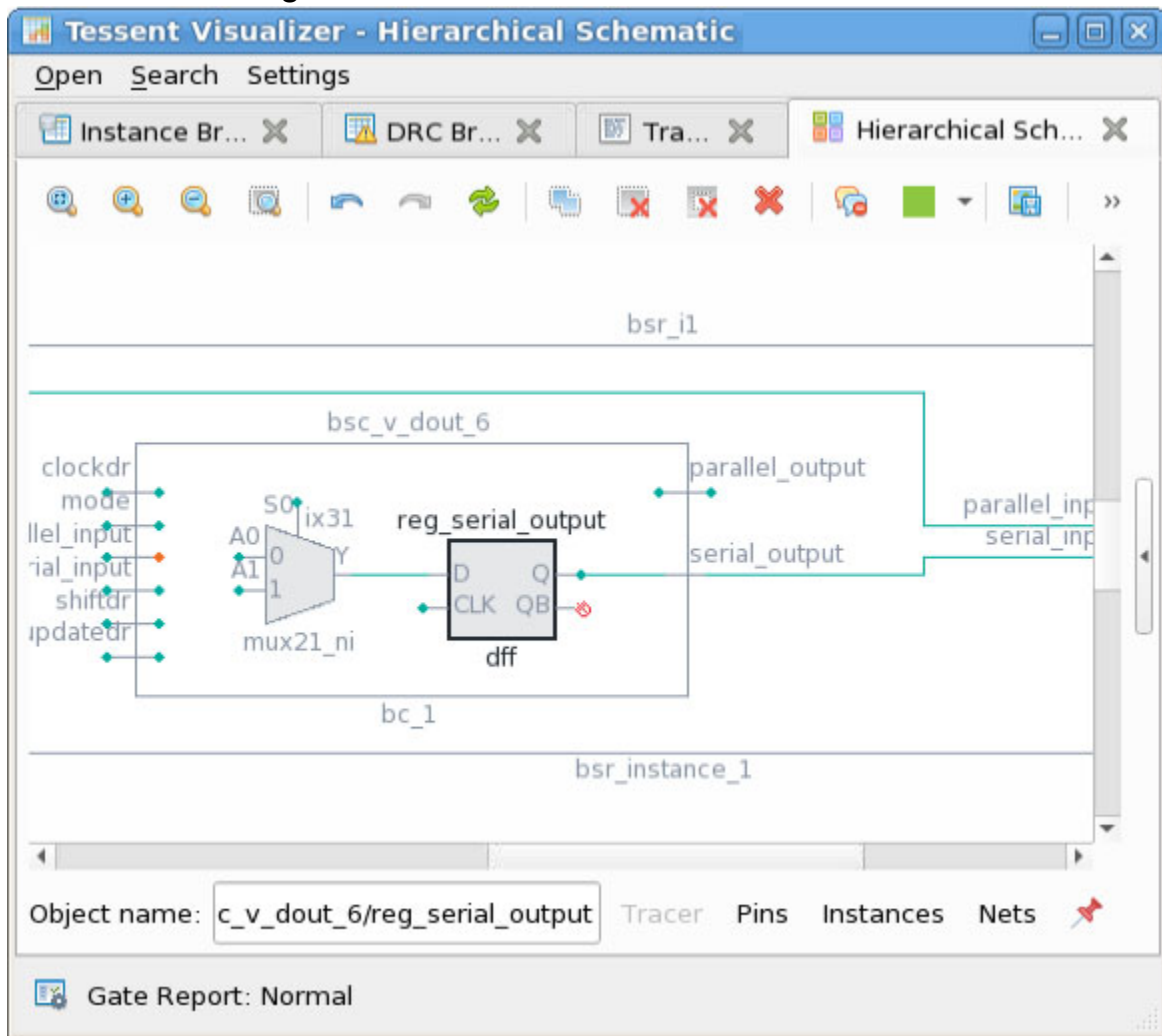


Figure 9-29. Hierarchical Schematic Window With Overview Window and Properties Table

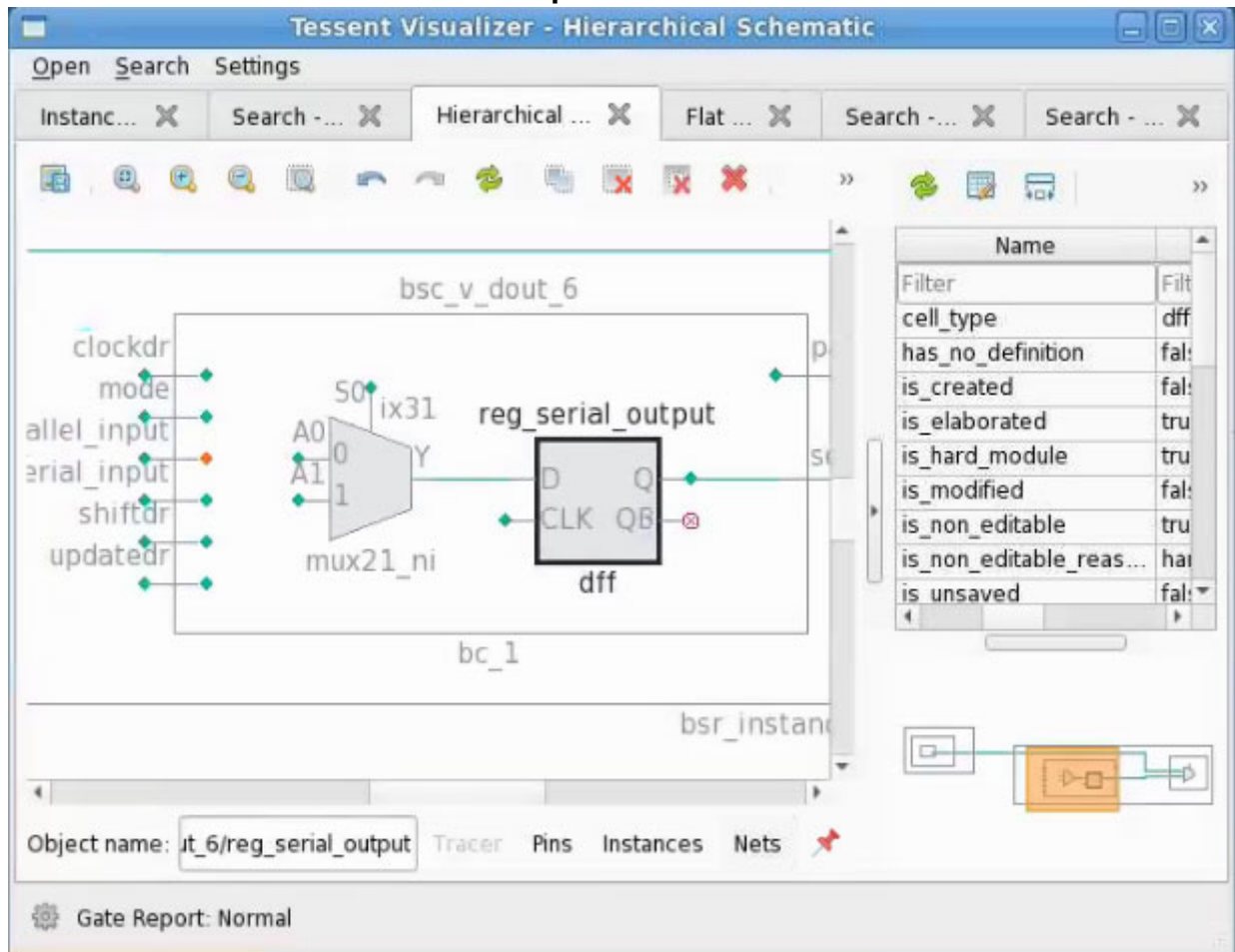
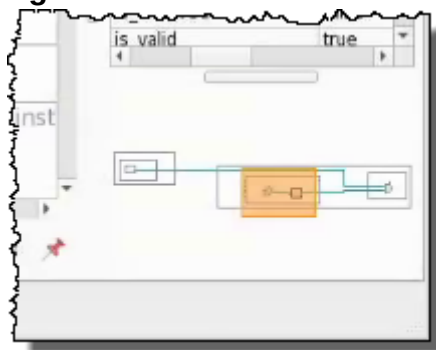


Figure 9-30. Overview Window



Note

Move and resize the highlighted box in the overview window to control the view in the main part of the hierarchical schematic window.

In addition to the tools available from the toolbar, invoke the following by right-clicking and choosing an action from the popup window, depending on the selected objects:

- Show the selected object(s) in the flat schematic window.
- Show the HDL definition for the selected object in the Text / HDL Viewer window.
- Show the HDL instantiation for the selected object in the Text / HDL Viewer window.
- Show all pins on the instance.
- Show the instances as parent in the instance browser window.
- Delete all selected object(s) from the schematic.
- Delete all unselected object(s) from the schematic.
- Delete unconnected pins from the schematic.
- Trace backward.
- Report gates.

The following figures show the Hierarchical Schematic window with all pins shown on the selected instance, and again with unconnected pins hidden:

Figure 9-31. Hierarchical Instance With All Pins Shown

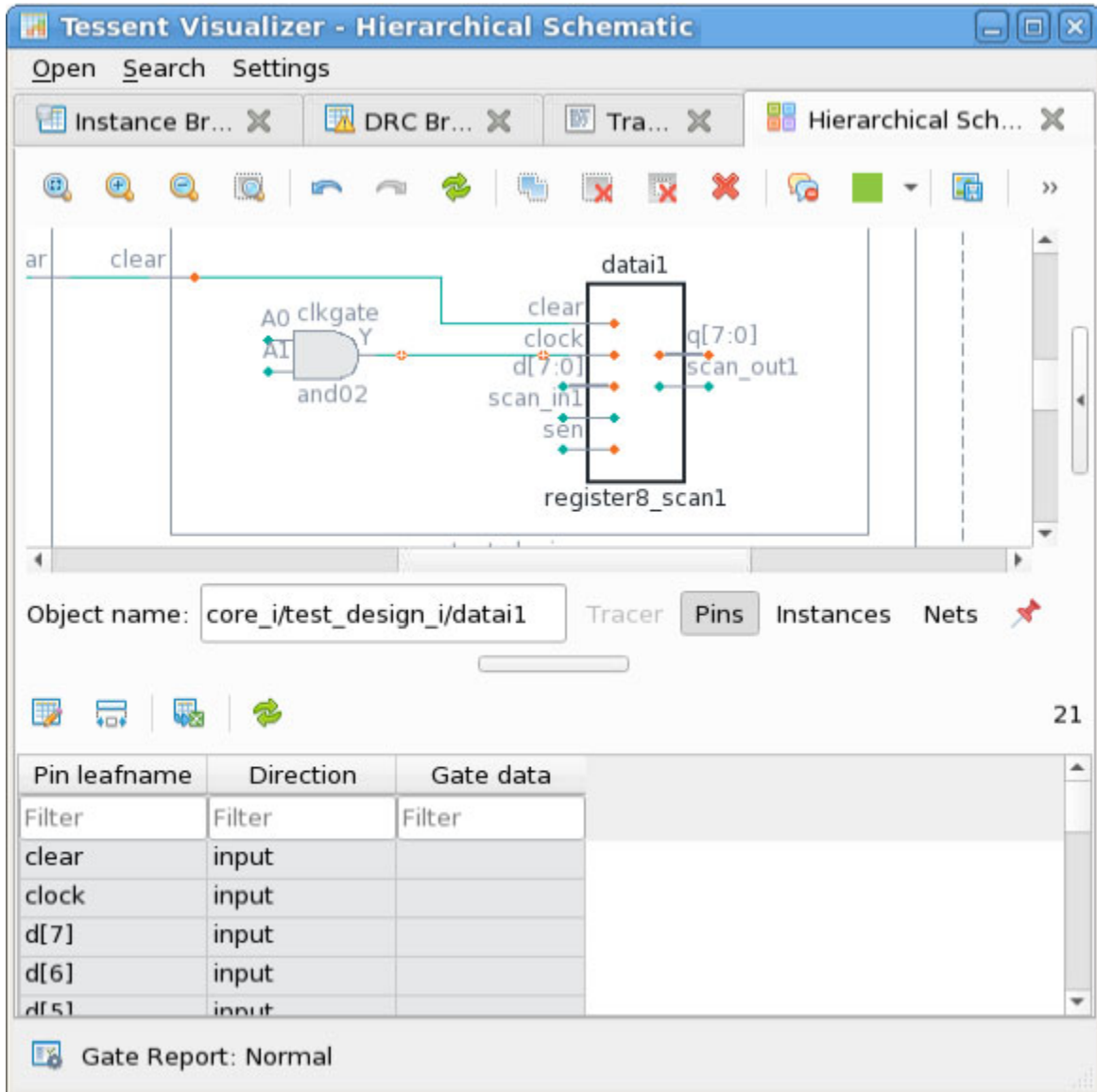
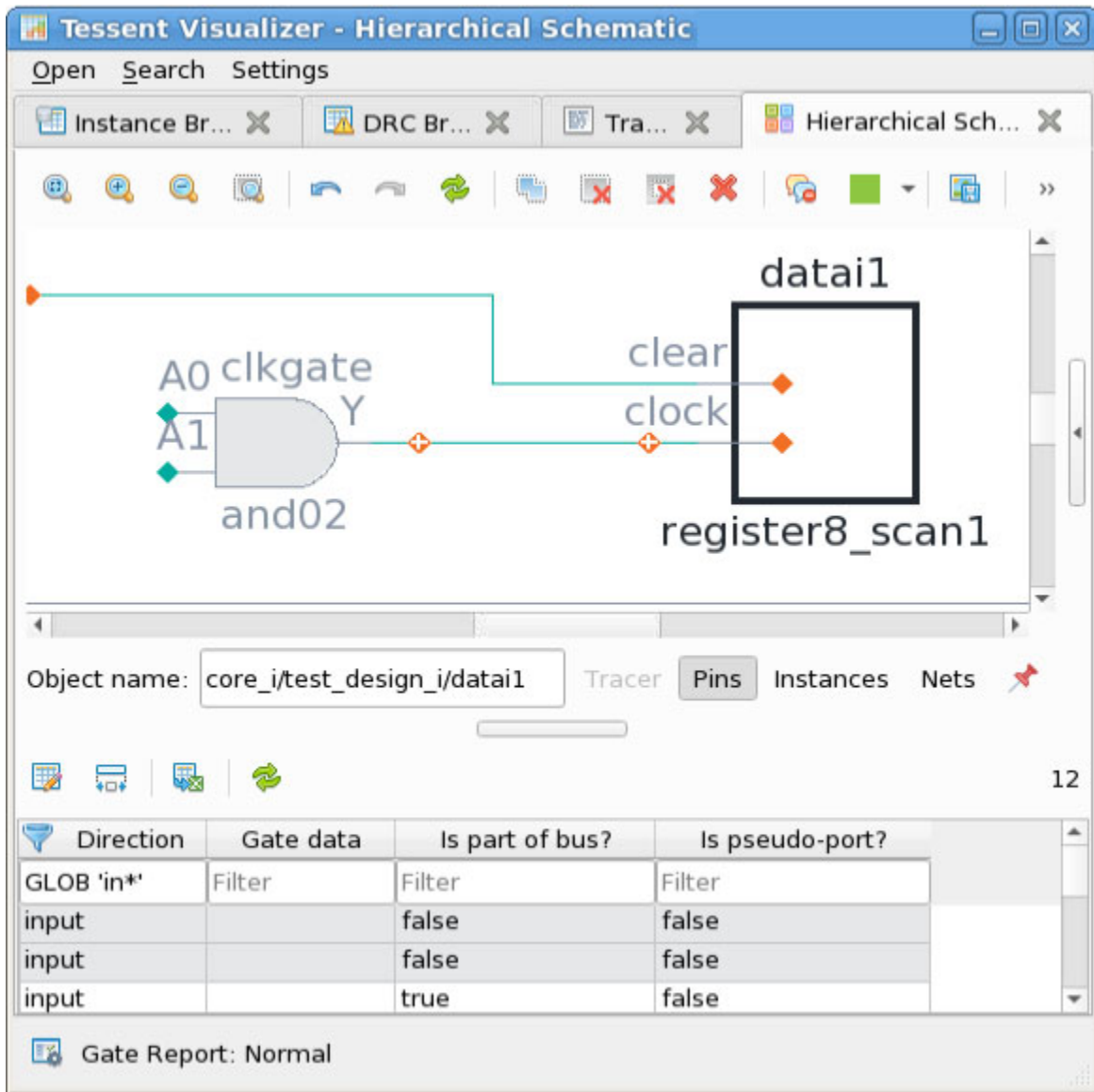


Figure 9-32. Hierarchical Instance With Unconnected Pins Deleted From the Schematic



The following two tables relate specifically to the Hierarchical Schematic window:

- **Instances Table** — Displays information about the instances contained within a selected instance.
- **Nets Table** — Displays information about the nets of a selected instance.

Figure 9-33. Hierarchical Schematic Instances Table

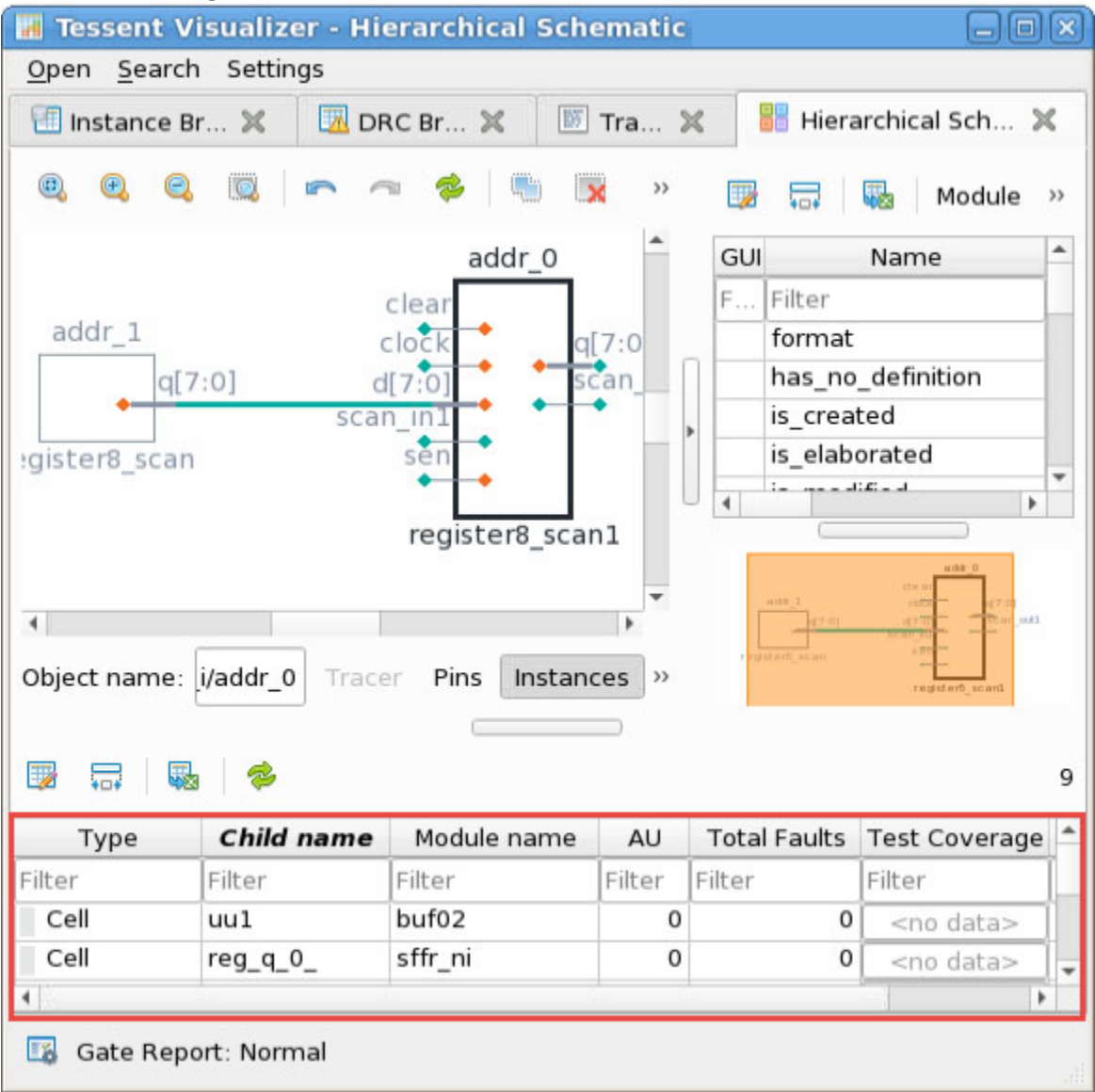
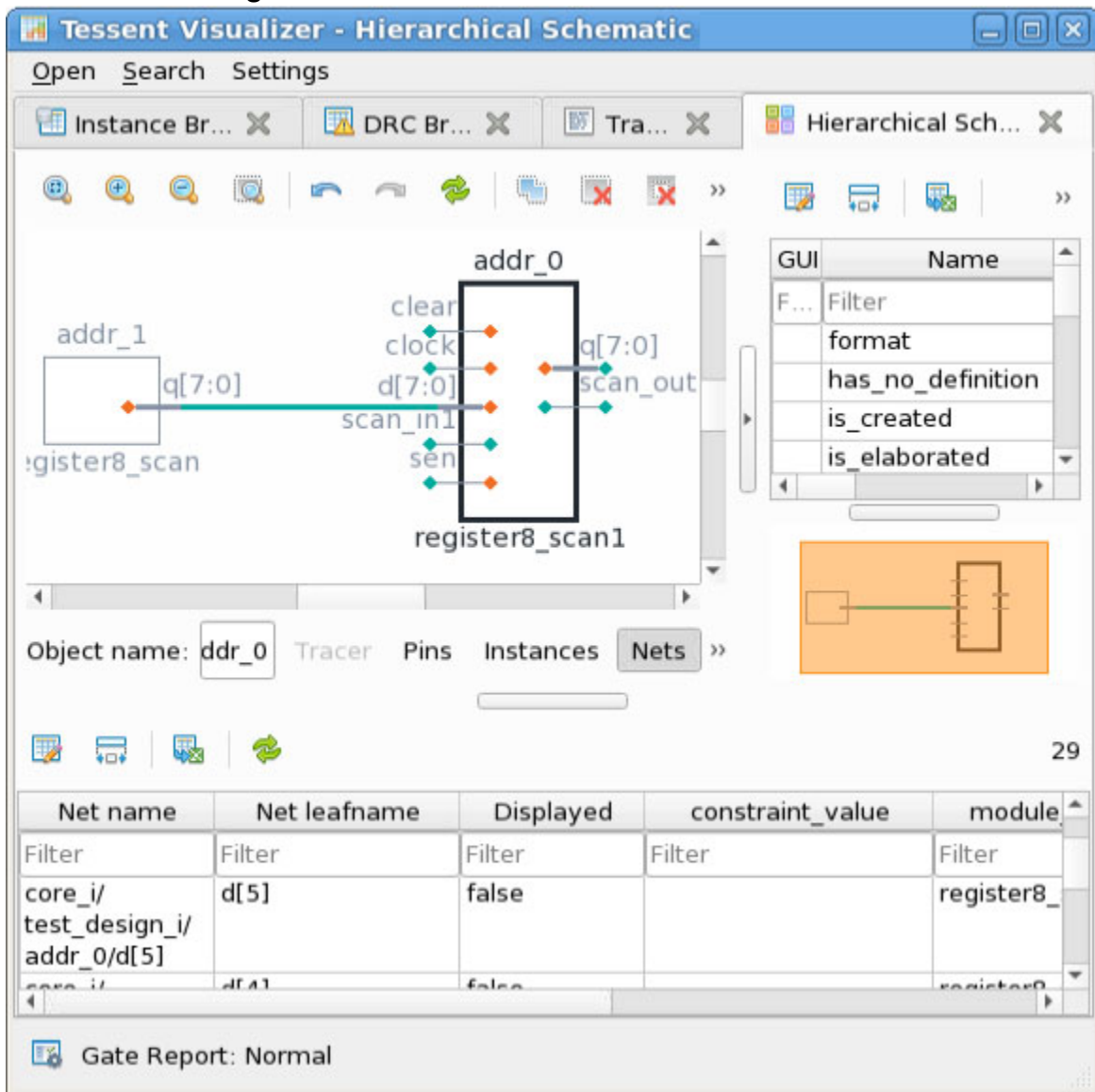
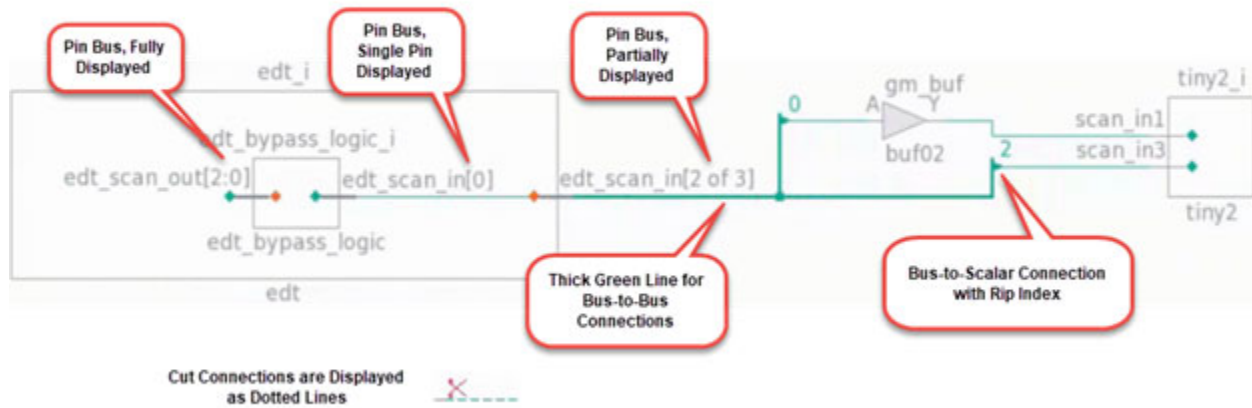


Figure 9-34. Hierarchical Schematic Nets Table



The following figure shows several symbols specific to the Hierarchical Schematic window:

Figure 9-35. Hierarchical Schematic Symbols



Design hierarchy is shown with rectangles surrounding the objects in a design instance, and library cells are shown without rectangles and filled with gray. The leaf-level instance names appear above each instance, and the module or cell names appear below them.

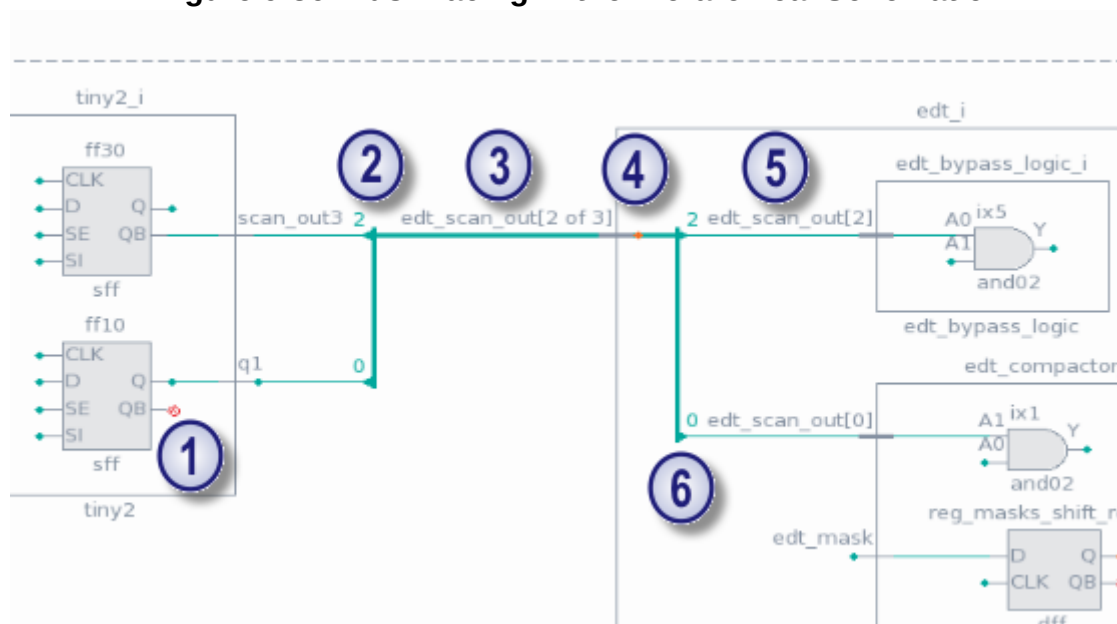
Buses

Both complete and partial buses can be displayed in the Hierarchical Schematic window. Add or remove bus bins in the display from any table showing hierarchical pins — for example, the Pin Information table in the Instance Browser, the pins table at the bottom of the hierarchical schematic window (see [Figure 9-32](#) on page 416), etc.

The display of bus pins affects the operation of the tracing function, since the tracer starts from displayed bus pins. Choose individual pins to trace from the pin table directly, or by applying filtering for the pins of interest.

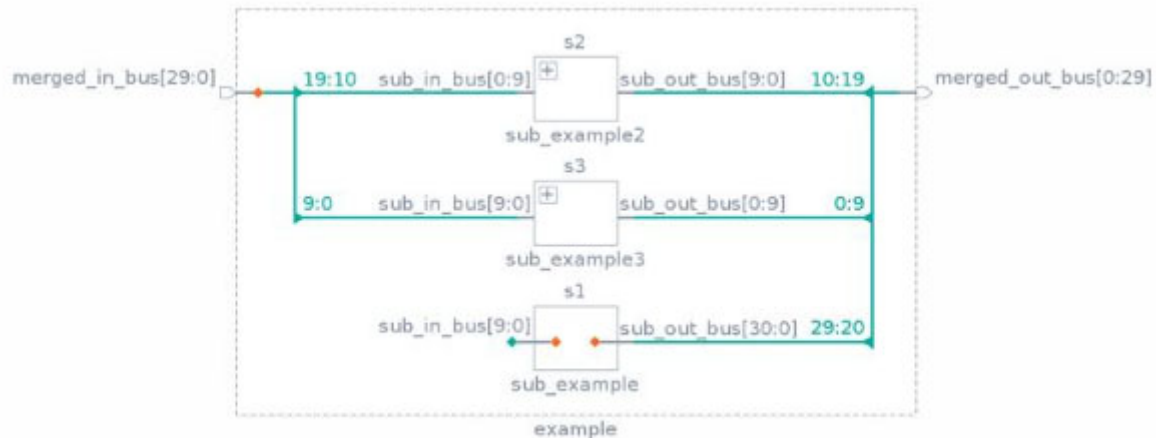
Several naming and symbol conventions are used for buses in schematic windows. See [Figure 9-36](#) on page 420 for examples.

Figure 9-36. Bus Tracing in the Hierarchical Schematic



Item	Description
①	Unconnected pin (red circle with “X”). Not specific to buses but included here for illustration.
②	Bus-to-scalar connection with rip index (“2” in this case).
③	Generated name for a partial bus (two bits of a three-bit bus in this case).
④	Orange diamond representing multiple paths.
⑤	Generated name of a partial bus with a single bit displayed (bit index2 in this case).
⑥	Green diamond representing a single path.

Buses can be split into sub-ranges as shown in [Figure 9-37](#) on page 421. In this case, ten bits of the 31-bit output bus from instance s1 are merged with all ten bits of the output buses from instances s2 and s3 to form the 30-bit merged_out_bus.

Figure 9-37. Bus Sub-Ranges**Note**

 This schematic does not indicate which ten bits of the output bus from s1 are connected. Use the Tracer table to determine this.

Other Symbols

There are a few additional marker symbols used to indicate other hierarchical schematic features:

Table 9-4. Hierarchical Schematic Marker Symbols

Symbol	Description
	Blue square: RTL connection (no connection in hierarchical model)
	Pin tied high (power or 1'b1)
	Pin tied low (ground or 1'b0)
	Red triangle: coercion (connection against declared port direction)

Flat Schematic Window

The Flat Schematic window shows a representation of the design after design flattening. Use the Flat Schematic window to navigate your design by tracing fanout and fan-in, and obtaining information about design objects you select.

Cell library instances in the design appear in the flat schematic window as gates. Cells that consist of a single gate appear with the conventional symbols for those gates, but those that

consist of multiple gates appear either with or without a bounding box showing this grouping. To turn this display option on and off, use the “Cell grouping” checkbox in the Options menu available from the toolbar.

Many features of the flat schematic window are similar to those in the [Hierarchical Schematic Window](#) window.

Figure 9-38. Flat Schematic Window (Cell Grouping On)

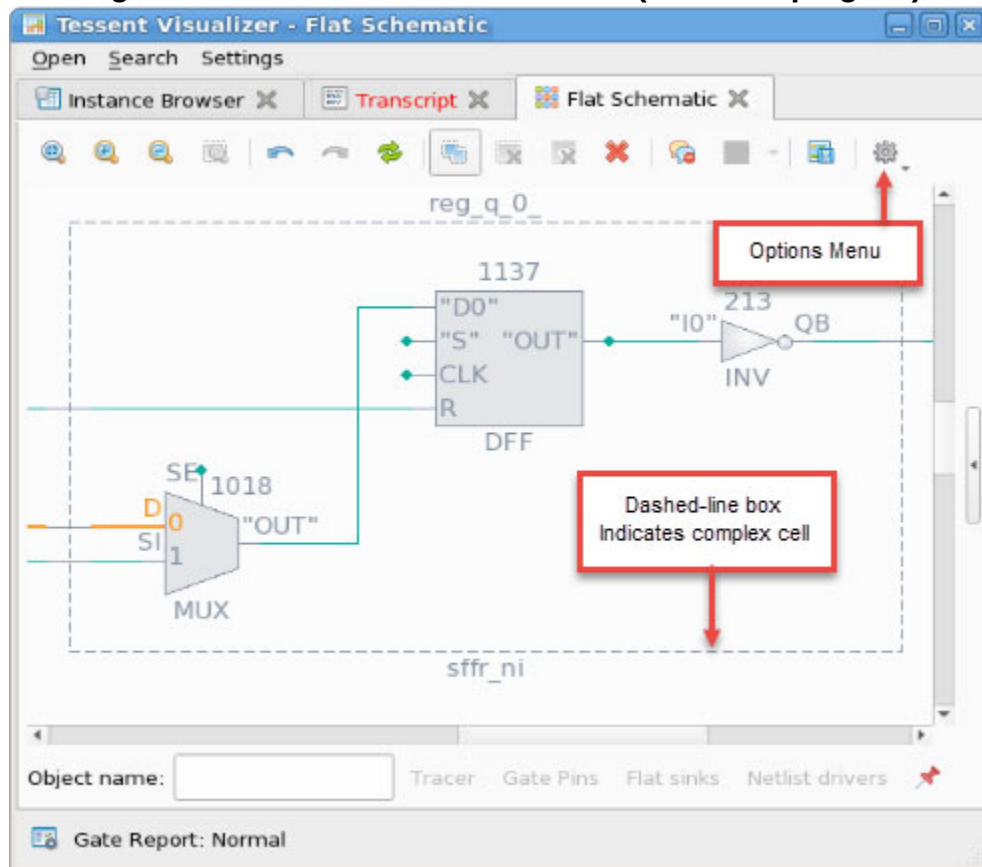
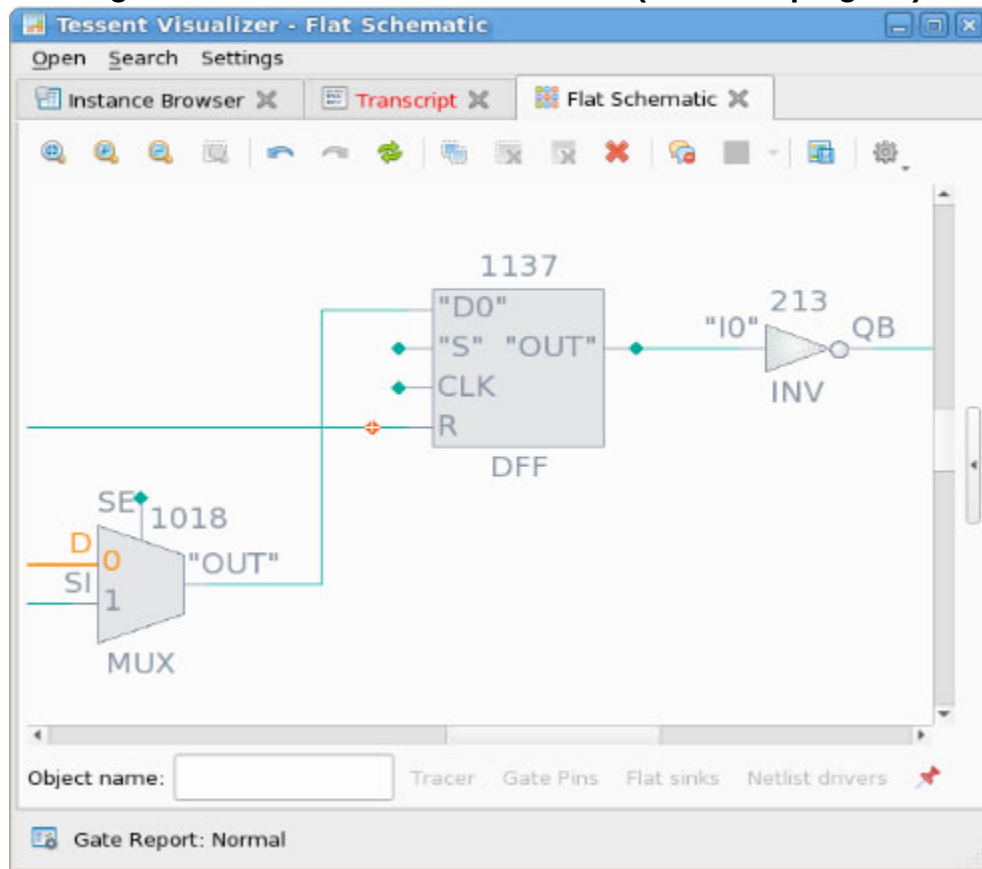
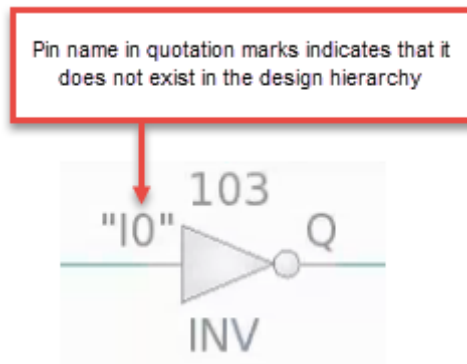


Figure 9-39. Flat Schematic Window (Cell Grouping Off)**Note**

When cell grouping is turned on, instances shown on the schematic are identified by their module or primitive names (below the symbol) and leaf-level cell library names (above the symbol). When it is turned off, they are identified by their module/primitive names and their gate IDs from the flat model.

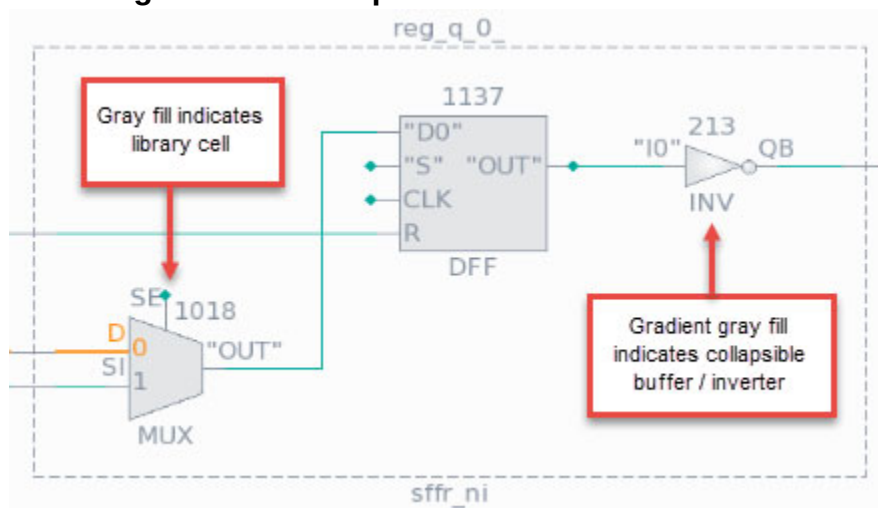
Pin names on instances appear in quotation marks when the pin is not part of the design hierarchy.

Figure 9-40. Non-Hierarchical Pin



By default, sequences of buffers and inverters are collapsed to simplify the representation of your design in the schematic window. Use the checkbox in the “Options” menu from the toolbar to control this. Buffers and inverters that are collapsible are indicated by a gradient gray fill.

Figure 9-41. Collapsible Buffers and Inverters



Instance Browser

Use the Instance Browser to navigate your design and obtain reports about its elements. It is analogous to a file browser, which shows directories and files in a similar manner.

The Instance Browser consists of four major elements:

- **Tree View** — a pane that shows the instances in your design hierarchy directly.
- **Child instances table** — a pane that displays information about the child instances of the instance currently selected in the tree view or the name display/entry bar.

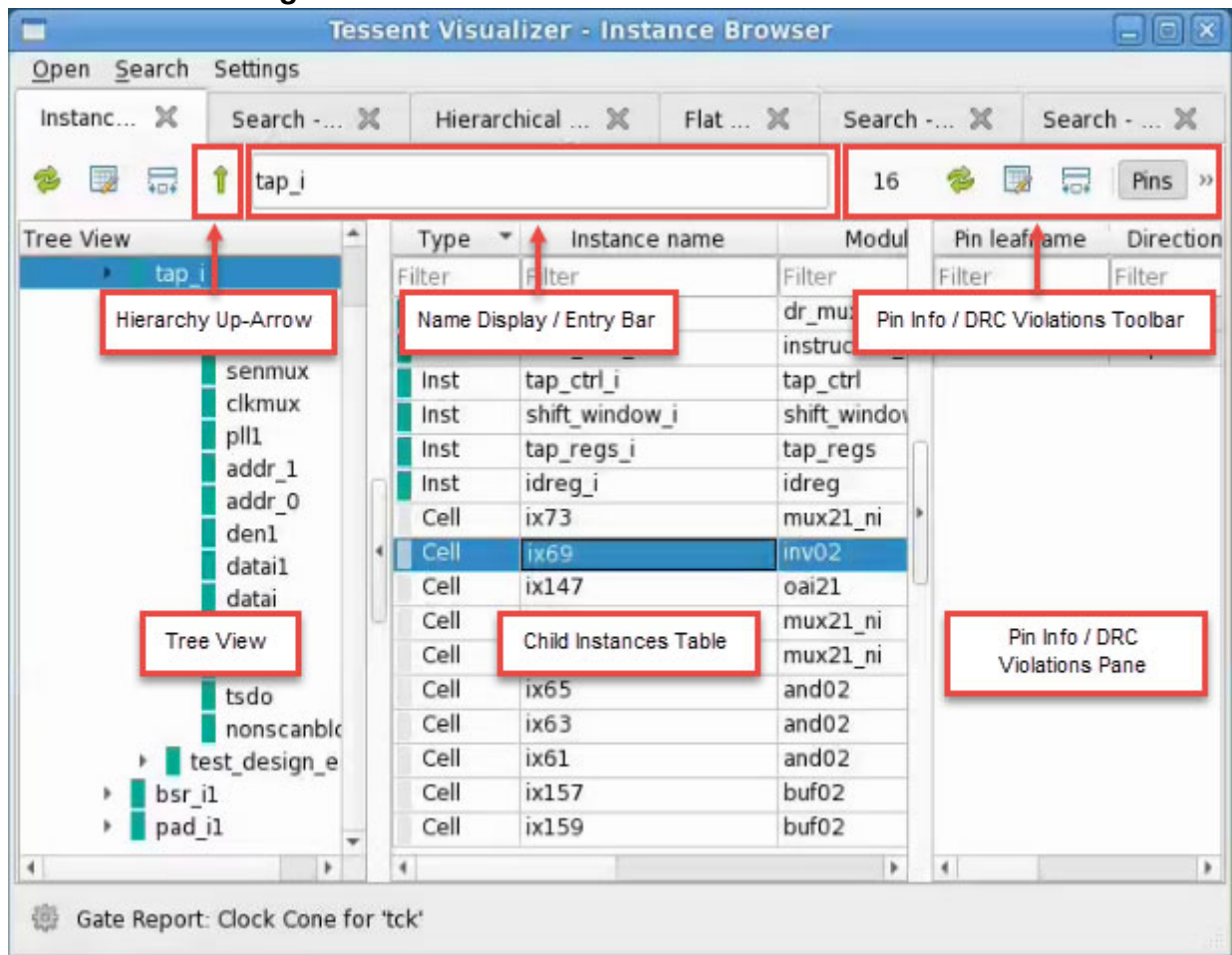
- **Address bar** — a text field that shows the instance currently selected in the tree view and the parent instance for instances displayed in the Child instances table. Type an instance name in this field to select an instance directly.
- **Pin information or DRC violations pane** — these are listed for the instance currently selected in the child instances report table.

Click an instance in the tree view to display that instance's contents in the child instances table. Double-clicking an instance in the tree view both displays the instance's contents in the child instances table and opens the next level of hierarchy in the tree view.

Select an instance of any type in the child instances table to display pin or DRC information for that instance in the third pane. Double-click, or use the Enter key, to open the next level of hierarchy (if any) in the child instances table. Press Backspace key to navigate up one level of hierarchy.

Show instances of cells or modules in the hierarchical schematic window, the flat schematic window, or the text/HDL viewer window by right-clicking the cell or module in the tree view or child instances table and choosing the appropriate menu option from the popup menu. Add a pin to the pin data table by right-clicking the pin in the pin information table and choosing **Add to Pin Data**.

Figure 9-42. Tessent Visualizer Instance Browser

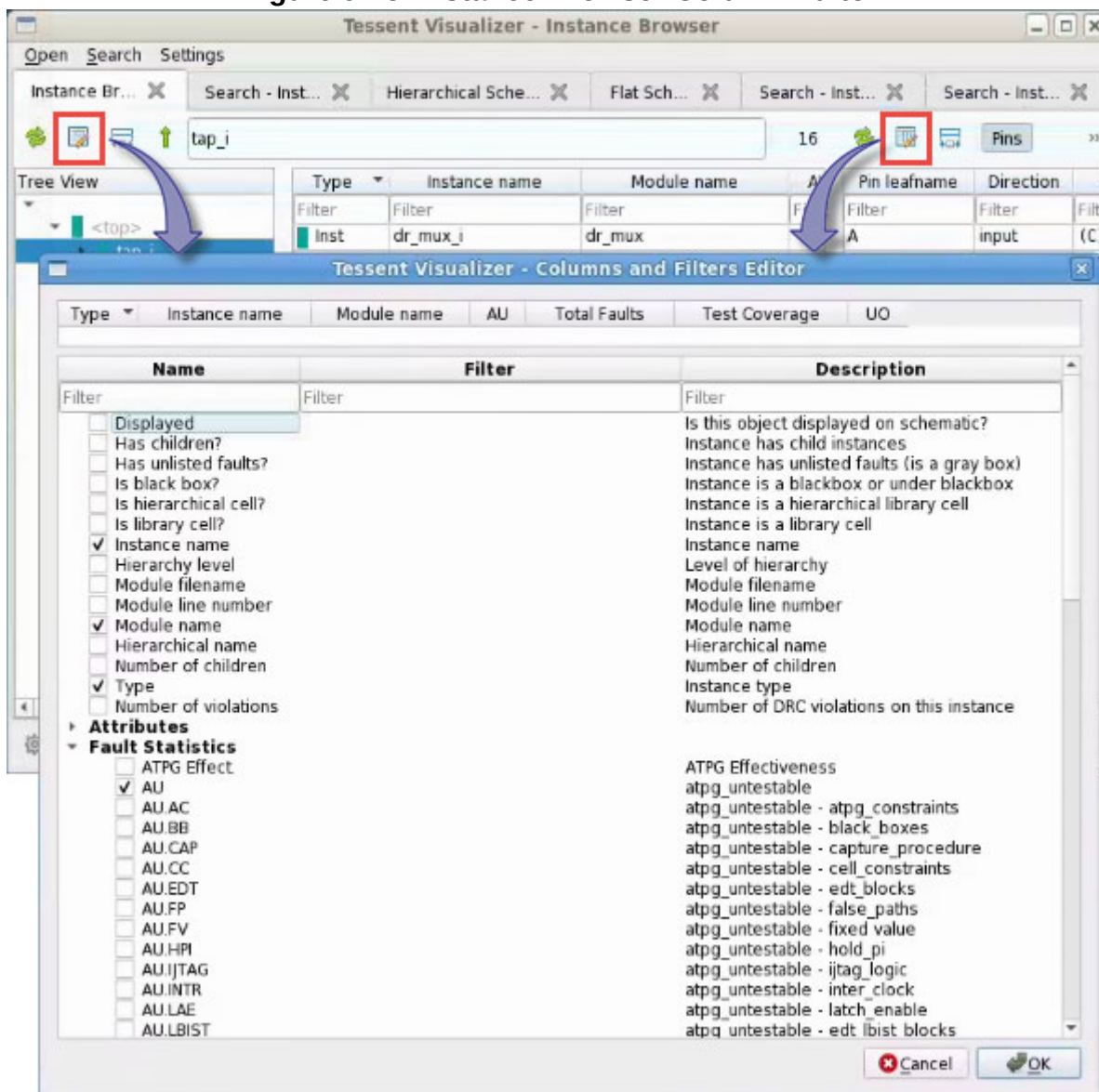


The child instances table and pin/DRC violation table include toolbars that provide the common controls described in the “[Common Features](#)” on page 385 section, as well as an up-arrow icon you can use to rise one level in the design hierarchy and buttons you can use to switch between pin information and DRC violations in the table.

The name display/entry bar is located between the two toolbars.

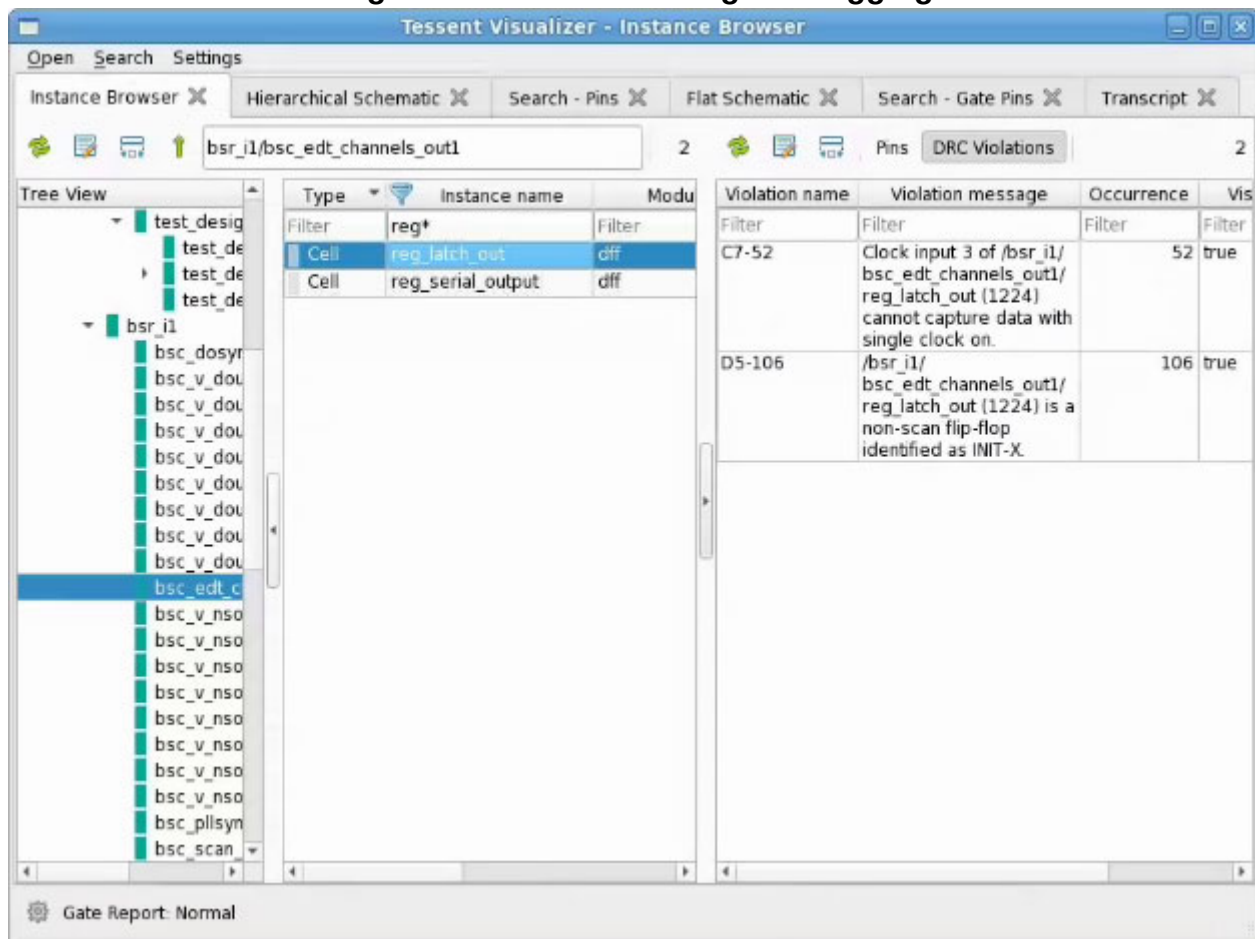
As previously noted, two column editors are available from the instance browser toolbars. Use these as shown in [Figure 9-43](#) on page 427 to control the types of data that appear in the table columns and define filters to include or exclude specific entries in the table rows.

Figure 9-43. Instance Browser Column Editor



To perform fault coverage debugging directly in the design browser, use the sorting and filtering functions as shown in [Figure 9-44](#) on page 428 to investigate faults and discover instances with problematic coverage statistics. As shown in this example, you can select a module in the Tree View, apply a filter (reg* in this case) to find instances of interest, and populate the DRC Violations pane by right-clicking a particular instance.

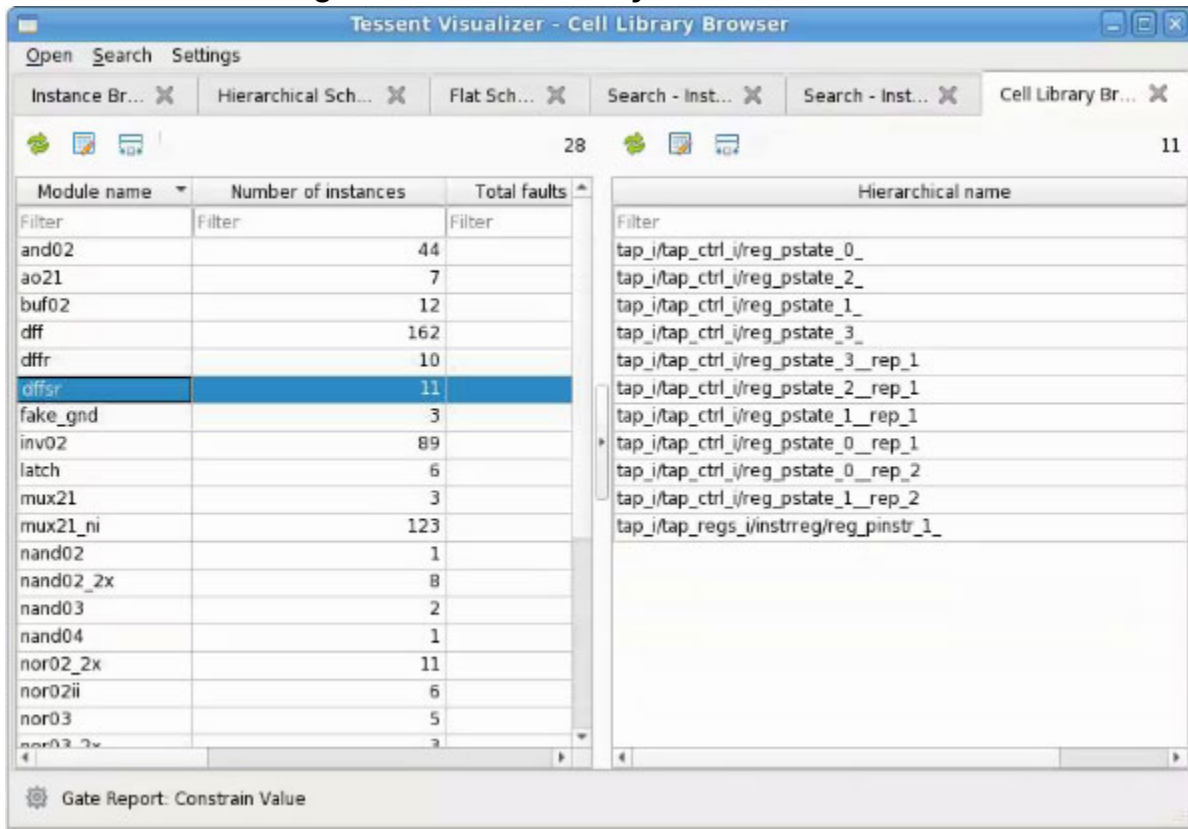
Figure 9-44. Fault Coverage Debugging



Cell Library Browser

Use the Cell Library Browser to debug test coverage and fault coverage loss from the perspective of library cells. The Cell Library Browser complements the Instance Browser view for each hierarchical instance. Customize the presentation to show various types of data and to filter that data for items of interest.

The Cell Library Browser window consists of two panes. The left pane shows the cell library itself. It can include data such as the module name, statistics including the number of instances in the design, and fault coverage. The right pane shows the instantiations of those library cells in the design. You can add columns to display data such as the attributes of those instantiations, the hierarchical name, the parent module, and fault information.

Figure 9-45. Cell Library Browser Window

Click in a row in the left pane to show the instantiation information for that cell type in the right pane. Right-click an instance name in the table in the right pane to get access to all actions available for hierarchical instances.

DRC Browser

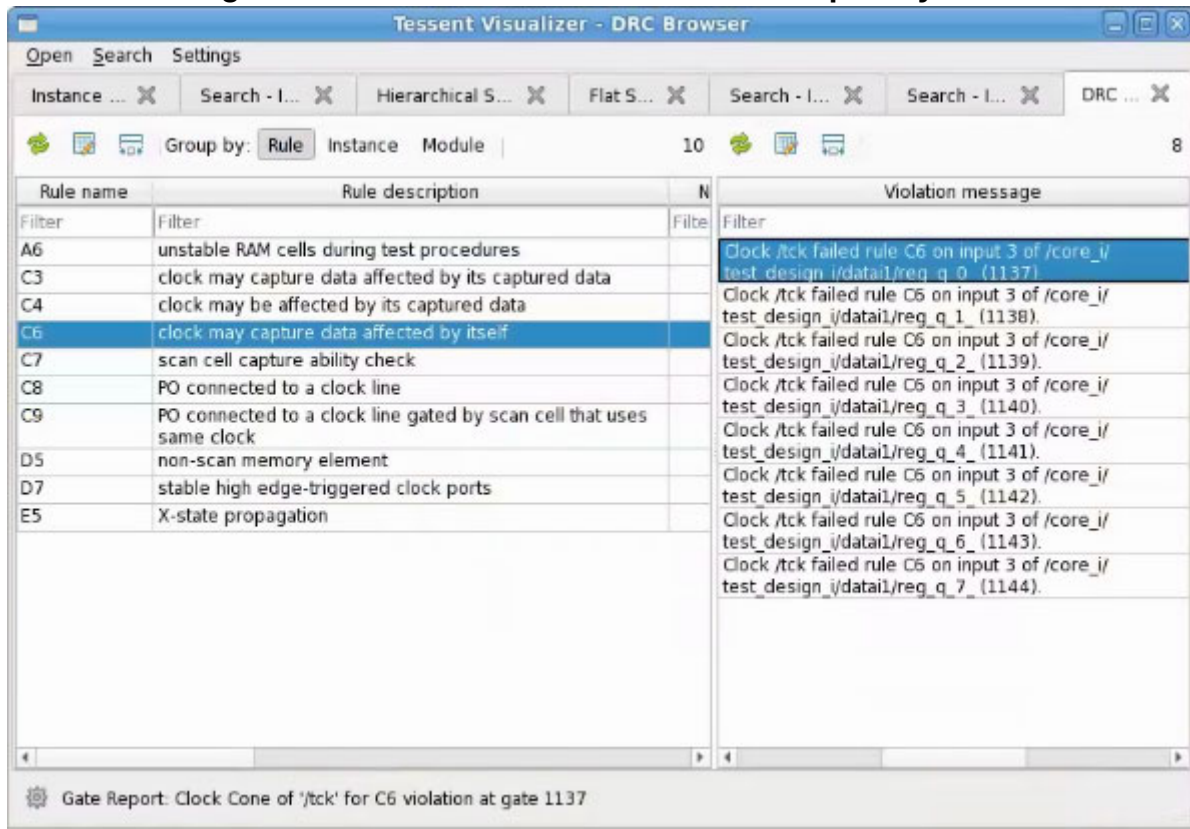
The DRC Browser is a tabular reporting tool for exploring the rule violations in your design. Customize the browser to group and filter DRC violations by various criteria.

The DRC Browser window consists of two panes. The left pane enables you to group the violation list by DRC rule (the default), instance, or module. The number of elements in the group appears in the upper-right corner of the left pane.

The right pane shows specific information about the violations associated with the rule, instance, or module selected in the left pane, such as:

- The violation name
- A description of the violation
- The occurrence number
- An indication of whether the violation is visualizable

Figure 9-46. DRC Browser With Data Grouped By Rule



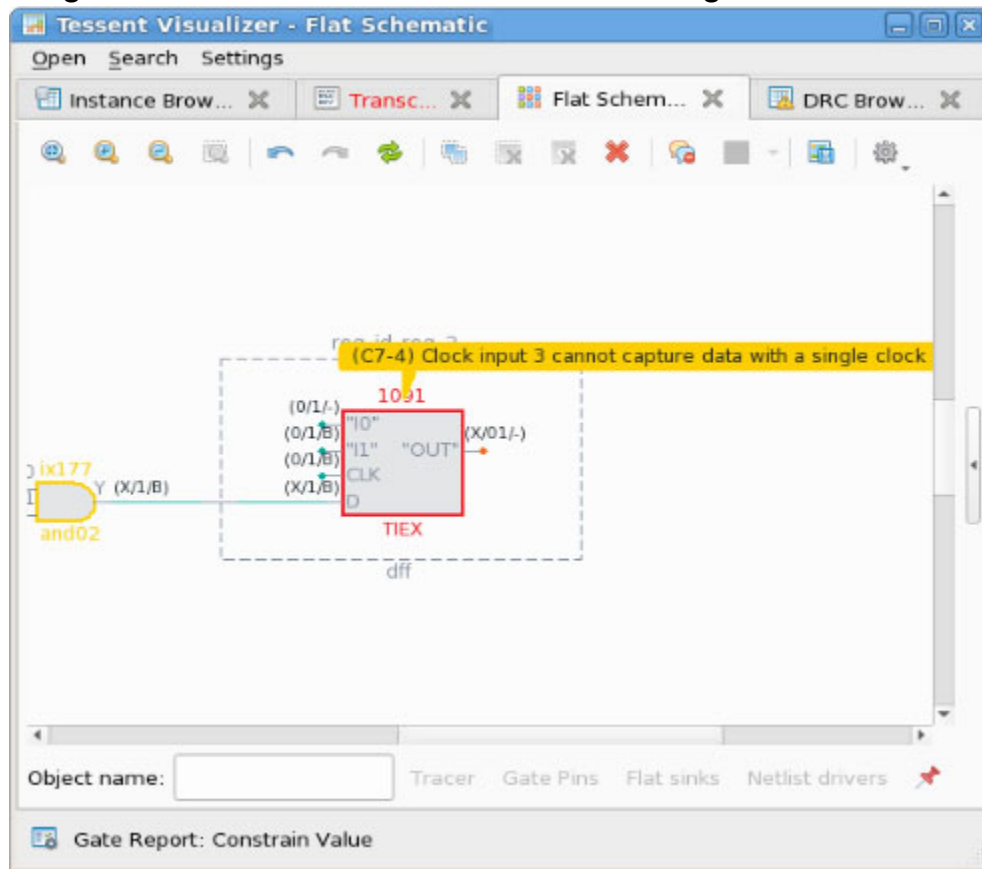
If you group by instance or module, data for the violations in the selected instance or module is displayed.

Show the specifics for a visualizable rule violation in a schematic window (as shown in [Figure 9-47](#) on page 431) or the text/HDL viewer window by double-clicking a row in the right pane of the DRC Browser window, or by right-clicking and choosing the appropriate menu item from the popup window.

Note

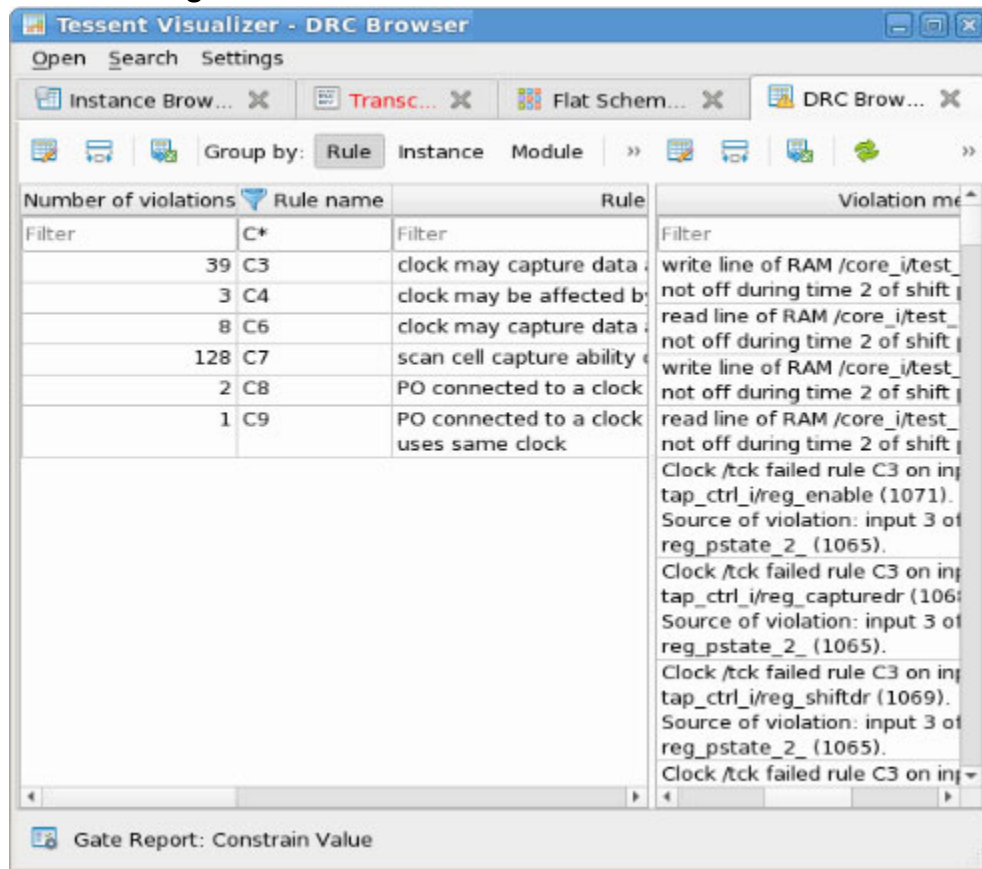


The method described above is equivalent to invoking the [analyze_drc_violation](#) command in Tessent Shell or in the Tessent Visualizer Transcript window.

Figure 9-47. Flat Schematic Window Showing a DRC Violation

Each pane of the DRC Browser window also includes the standard table control toolbar options as described in the “[Common Features](#)” on page 385 section. For example, [Figure 9-48](#) on page 432 shows the DRC Browser with a filter for C rules applied.

Figure 9-48. DRC Browser Window With Filter

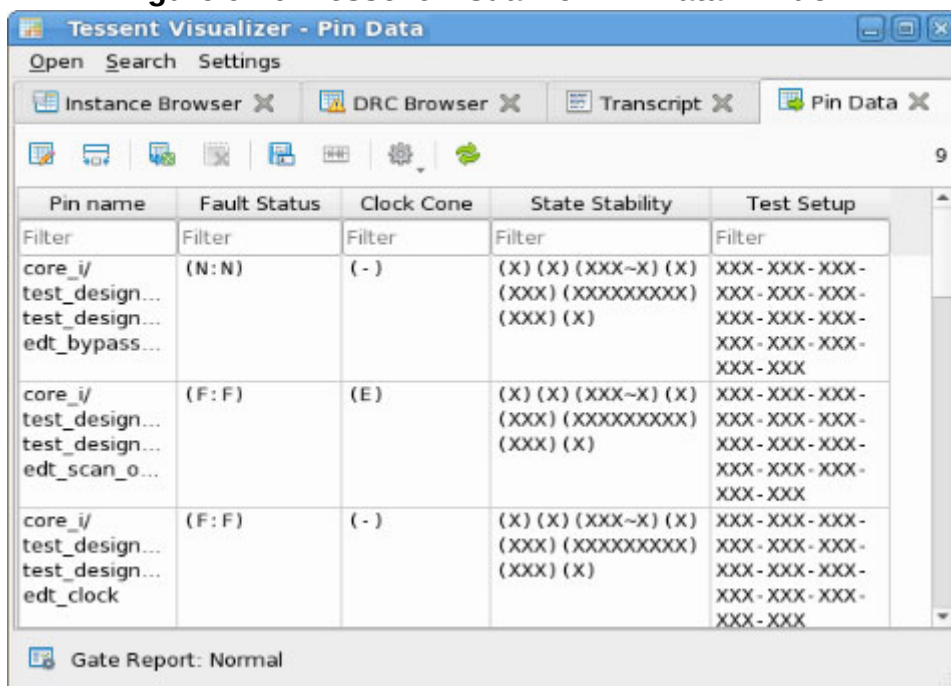


Pin Data Window

The Pin Data window contains a tabular report of a selection of pins in your design. This table shows gate report information you select with the column editor in the window's toolbar.

Add pins selected in any Tessent Visualizer window to the Pin Data window by right-clicking the pins and choosing **Add to Pin Data**. The toolbar in the Pin Data window enables you to refresh the data, fit the column widths to the cell contents, and remove rows from the table.

Figure 9-49. Tessent Visualizer Pin Data Window

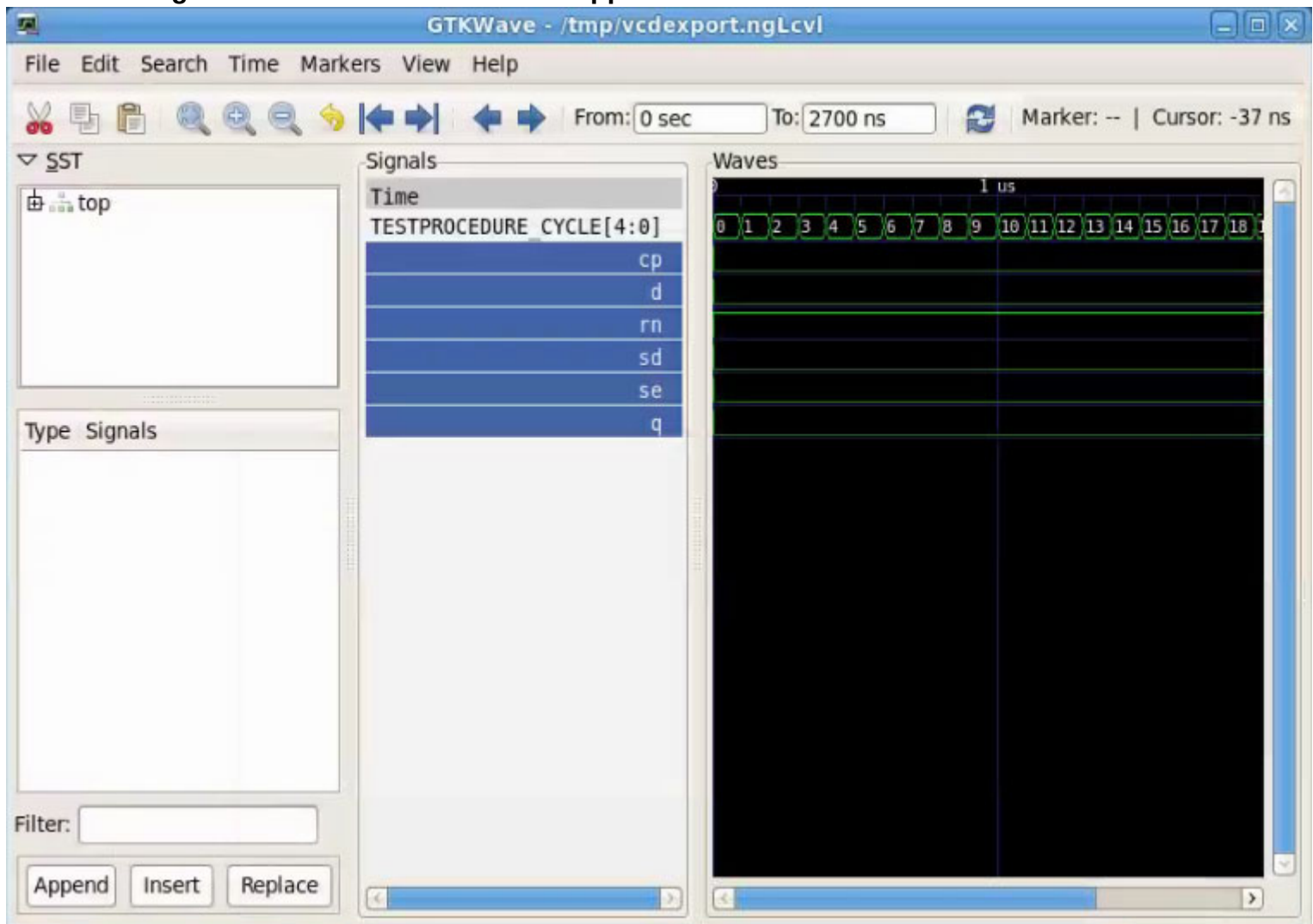


Use the toolbar in the Pin Data window to control the tabular display of the pin data. You can also export the Test Setup or Test End procedures for pins selected in the table to a VCD file or visualize them directly with a GTKWave viewer application from the toolbar.

Note

GTKWave is a third-party application downloadable from <http://gtkwave.sourceforge.net/>.

Figure 9-50. A GTKWave Viewer Application Invoked From the Pin Data Viewer



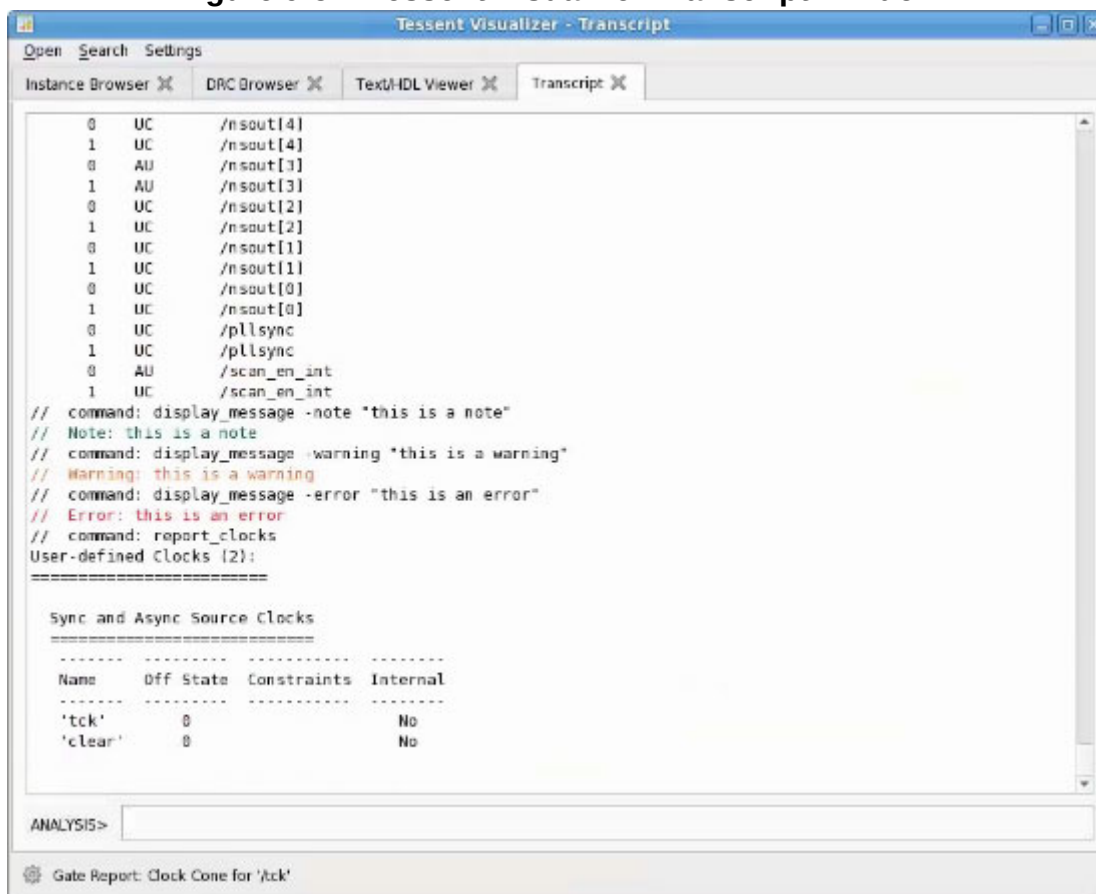
Transcript Window

The Tessent Visualizer Transcript window provides access to the Tessent Shell command line and a record of commands issued as well as responses to those commands.

Text in the Transcript window is highlighted as shown in [Figure 9-51](#) on page 435:

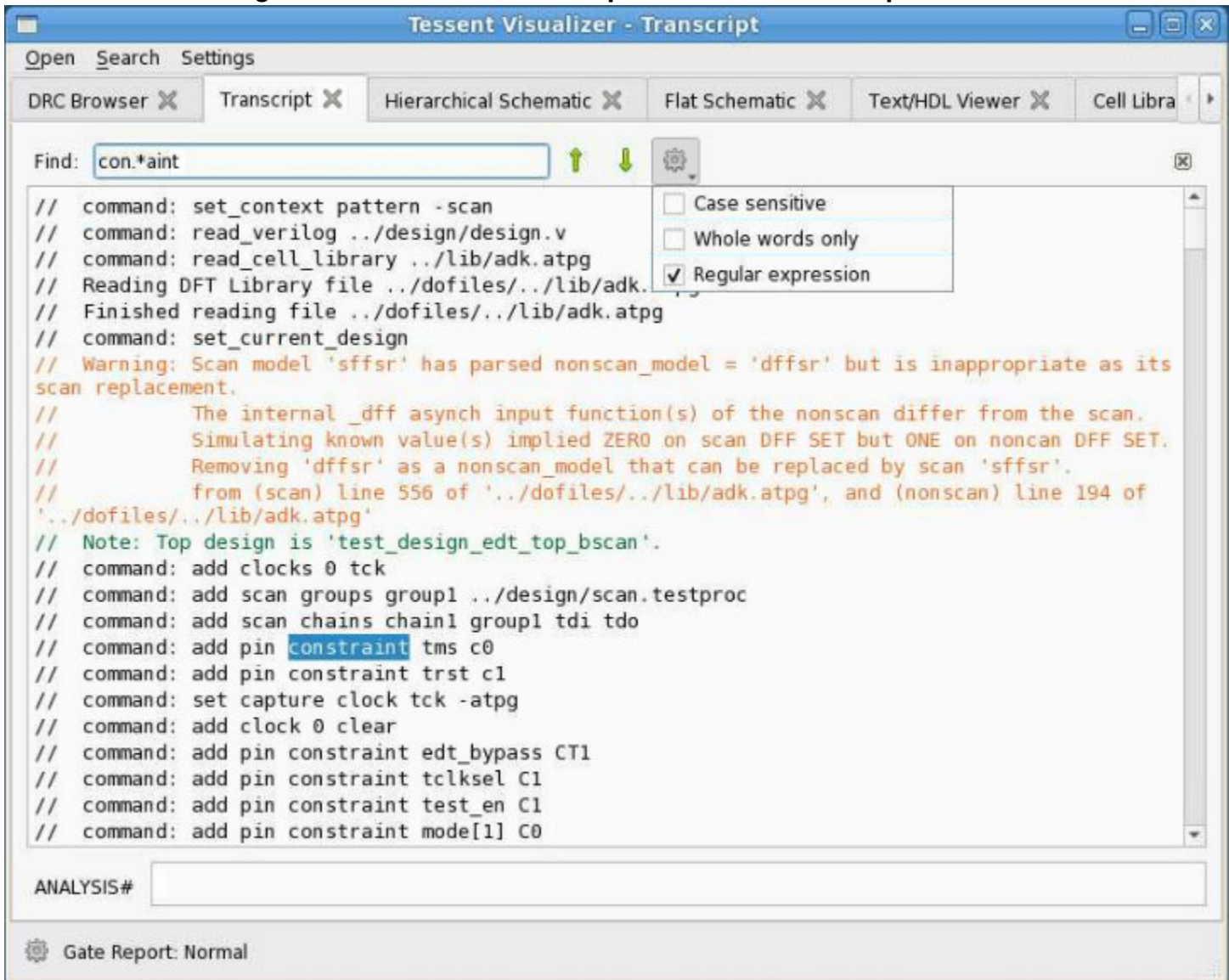
- **Green** — informational notes
- **Orange** — warning messages
- **Red** — error messages

Figure 9-51. Tessent Visualizer Transcript Window



To search for text within the Transcript window, press Ctrl-F and type the search terms in the text box provided as shown in [Figure 9-52](#) on page 436. Use the options available from the gear menu to make your search case sensitive or limit it to whole words, or to use standard regular expressions. Use the up-arrow and down-arrow icons, or Ctrl-G and Ctrl-Shift-G, to search forward or backward in the Transcript window.

Figure 9-52. Tessent Transcript Window Search Capabilities



Issue Tessent Shell commands directly from the Transcript window, with the same features as the standard Tessent Shell command line. (For example, command history using the up- and down-arrow keys, command completion using the Tab key, and multi-line commands.)

Press Ctrl-Enter to enable multi-line command mode. The background color of the command entry box changes to light blue to indicate that multi-line command mode is active. Use Shift-Enter to insert a new line and the arrow keys to navigate over the multi-line command.

Select text with the left mouse button and copy it to the clipboard by using the right-click popup menu. The right-click popup menu also includes an item to select all text in the Transcript window.

Text/HDL Viewer

The Text/HDL Viewer enables you to examine the HDL description of your design or cell library.

Open a tab for the Text/HDL Viewer directly from the **Open** menu in the main window, or by right-clicking objects in the Instance Browser, Cell Library Browser, or Schematic window and choosing **Show HDL definition** from the popup menu. When you open the Text/HDL Viewer from the popup menu for an object, the Viewer highlights the appropriate line in the HDL when the definition or instantiation appears.

Show HDL definitions and instantiations of instances in your design using the **Search** menu in the main window and right-clicking the results of your search to display a popup menu.

Within the Text/HDL Viewer window, select text and copy it for insertion in other contexts (such as an external text editor) by pressing Ctrl-C or right-clicking and selecting **Copy** from the popup window.

The names of the files opened by the Text/HDL Viewer are shown on tabs at the bottom of the viewer window.

To open the current file in an external text editor, right-click the tab and select **Open in external editor** from the popup window. Use the [Tessent Visualizer Preferences](#) dialog box to set the external text editor. If the specified text editor cannot be opened, the tool issues an error when you try to open the file.

To copy the file path of the current file to the system clipboard, right-click the tab and select **Copy file path** from the popup window.

Diagnosis Report Viewer

Use the Diagnosis Report Viewer window to open reports created with the `write_diagnosis` command.

Open the Diagnosis Report Viewer by clicking **Open** from the main window menu. Open a diagnosis report using the toolbar at the top of the Diagnosis Report Viewer window or by invoking the [display_diagnosis_report](#) command. You can open multiple reports using either of these methods. Each appears in its own tab. Select either text view or table view mode from the same toolbar.

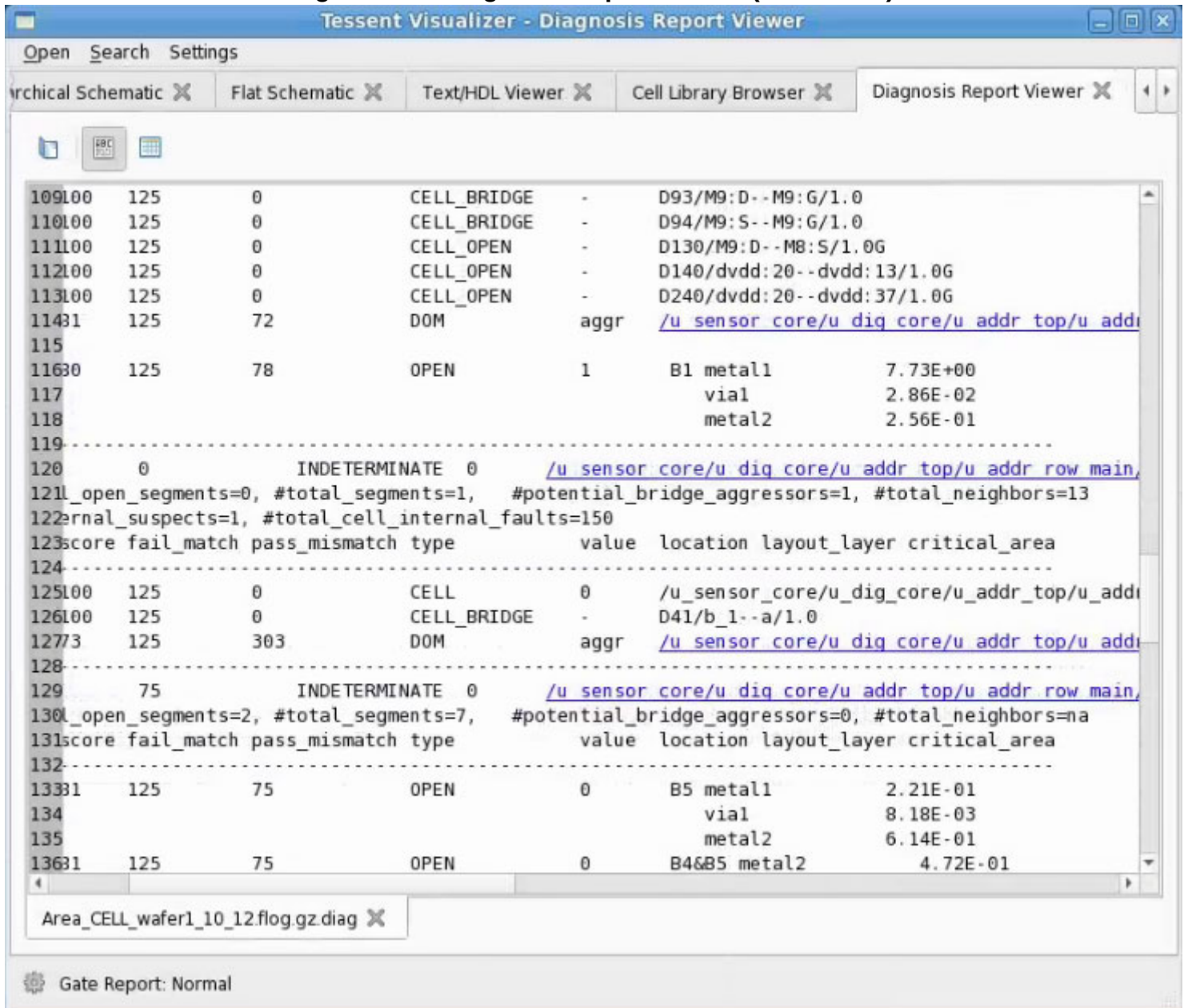
In text view mode, the diagnosis report itself appears in the window, annotated with hypertext links for each design object affected. These hypertext links refer to the following:

- Symptoms
- Suspects

- Sub-suspects
- Pin and net objects associated with suspects

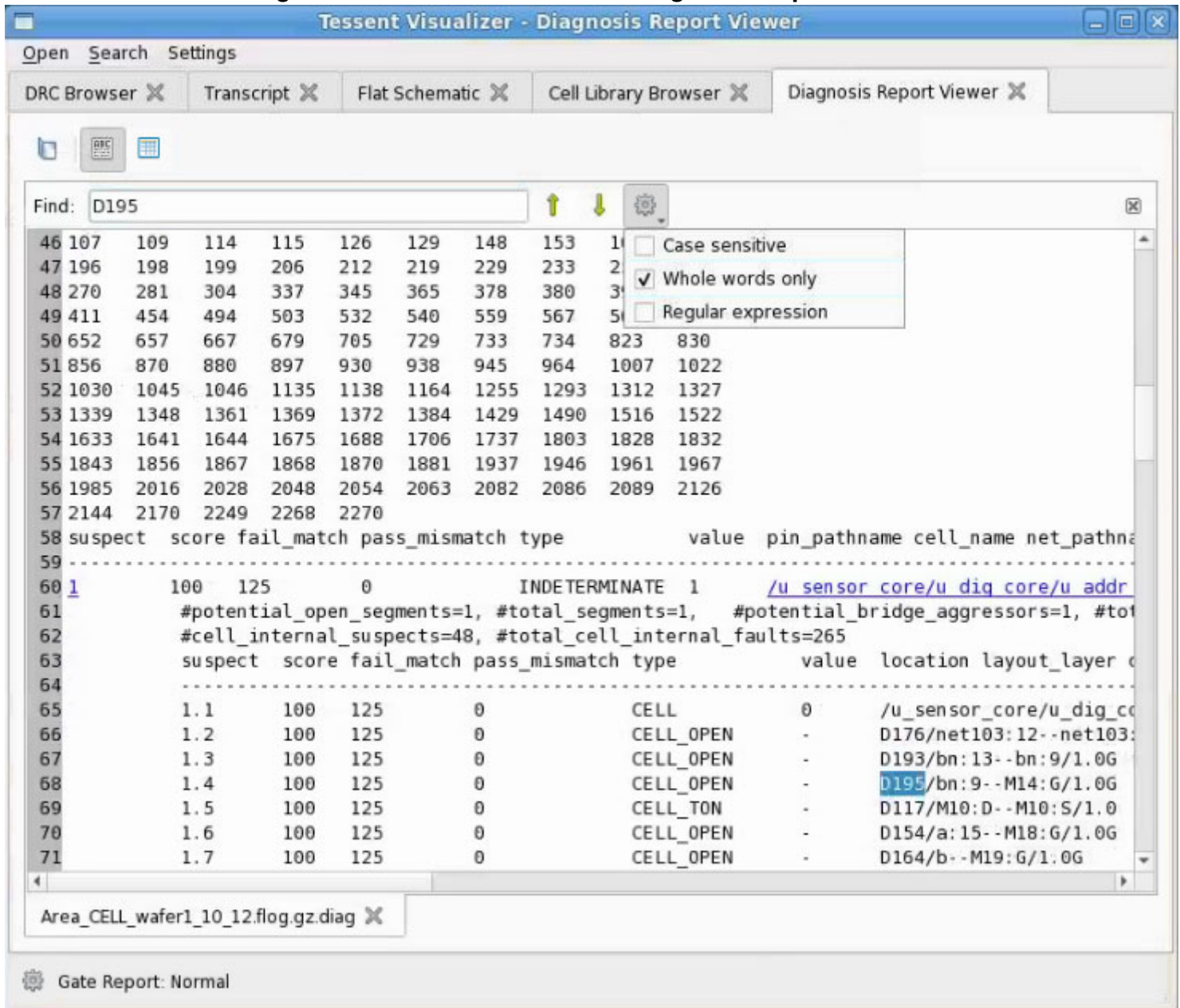
Click a hypertext link to show the indicated object in the Hierarchical Schematic window.
Right-click to bring up a context menu for other actions available to examine these objects.

Figure 9-53. Diagnosis Report Viewer (Text Mode)



Use Ctrl-F to search for text in the Diagnosis Report Viewer. Click the gear icon in the toolbar next to the search box to limit your search to whole words, to make it case sensitive, or to use regular expressions.

Figure 9-54. Text Search in the Diagnosis Report Viewer



In table view mode, two tables appear in the window. The upper table contains information about design objects affected by faults in the diagnosis report. Double-click an object in the table, or right-click and choose **Show on Hierarchical Schematic** to show it in the Hierarchical Schematic window. Select a suspect ID in the upper table to see information about the associated physical suspects in the lower table.

Figure 9-55. Diagnosis Report Viewer (Table View)

The screenshot shows the 'Tessent Visualizer - Diagnosis Report Viewer' window. The top menu bar includes 'Open', 'Search', and 'Settings'. Below the menu bar are tabs for 'Architectural Schematic', 'Flat Schematic', 'Text/HDL Viewer', 'Cell Library Browser', and 'Diagnosis Report Viewer'. The main area displays a table with columns: 'Symptom id', 'Match', 'Pass mismatch', 'Pins', 'Nets', and 'Cell names'. The table contains two rows of data. Below the table, there is a section for 'Locations' with columns 'Match', 'Pass mismatch', and 'Locations'. This section also contains three rows of data. At the bottom of the window, there is a status bar that reads 'Gate Report: Normal'.

Symptom id	Match	Pass mismatch	Pins	Nets	Cell names
1	125	0	/u_sensor_core/u_dig_core/ u_addr_top/u_addr_row_main/ u_sequencer/u2795/z	/u_sensor_core/ u_dig_core/ u_addr_top/ u_addr_row_main/ u_sequencer/n1999	aoi422_01_std_thk_dnd
	125	75	/u_sensor_core/u_dig_core/ u_addr_top/u_addr_row_main/ u_sequencer/u3390/a	/u_sensor_core/ u_dig_core/ u_addr_top/ u_addr_row_main/	xor2_01_std_thk_dnd

Match	Pass mismatch	Locations
125	0	/u_sensor_core/u_dig_core/u_addr_top/u_addr_row_main/u_sequencer/u2795
125	0	D41/b_1--a/1.0
125	303	/u_sensor_core/u_dig_core/u_addr_top/u_addr_row_main/u_sequencer/n6749

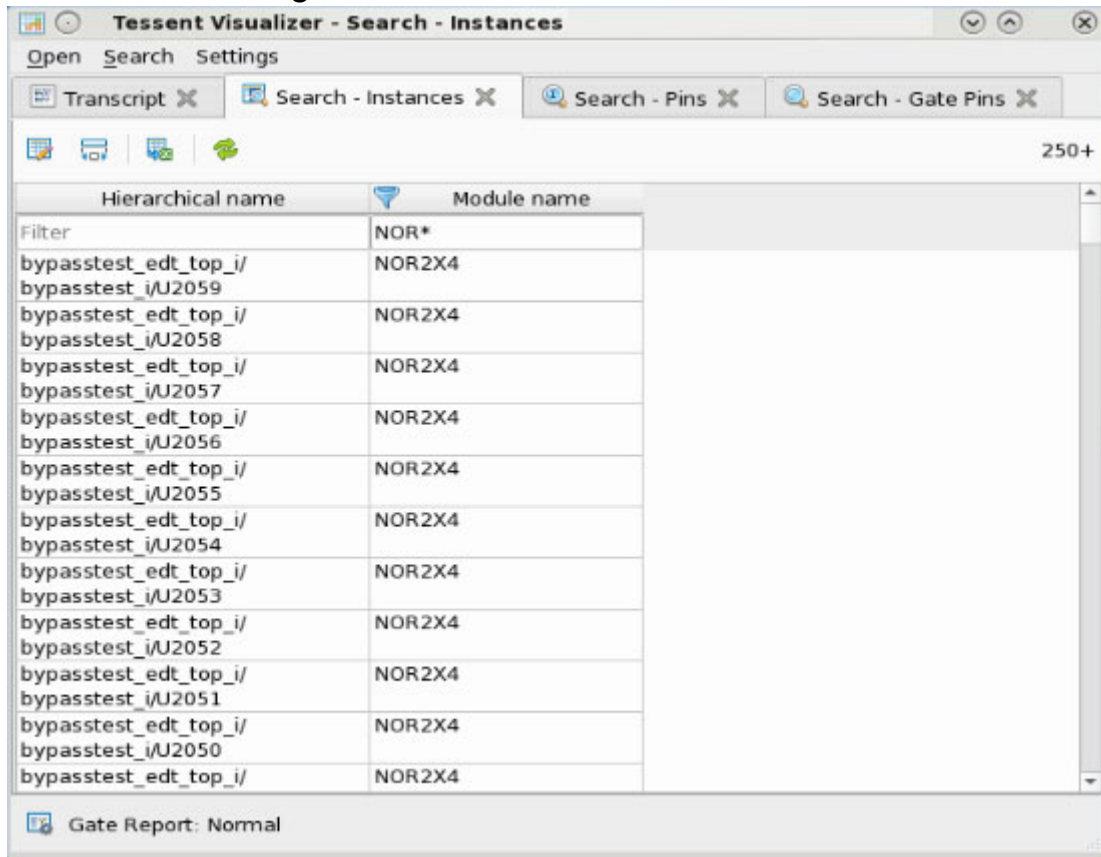
Area_CELL_wafer1_10_12.flog.gz.diag

Gate Report: Normal

Search Features

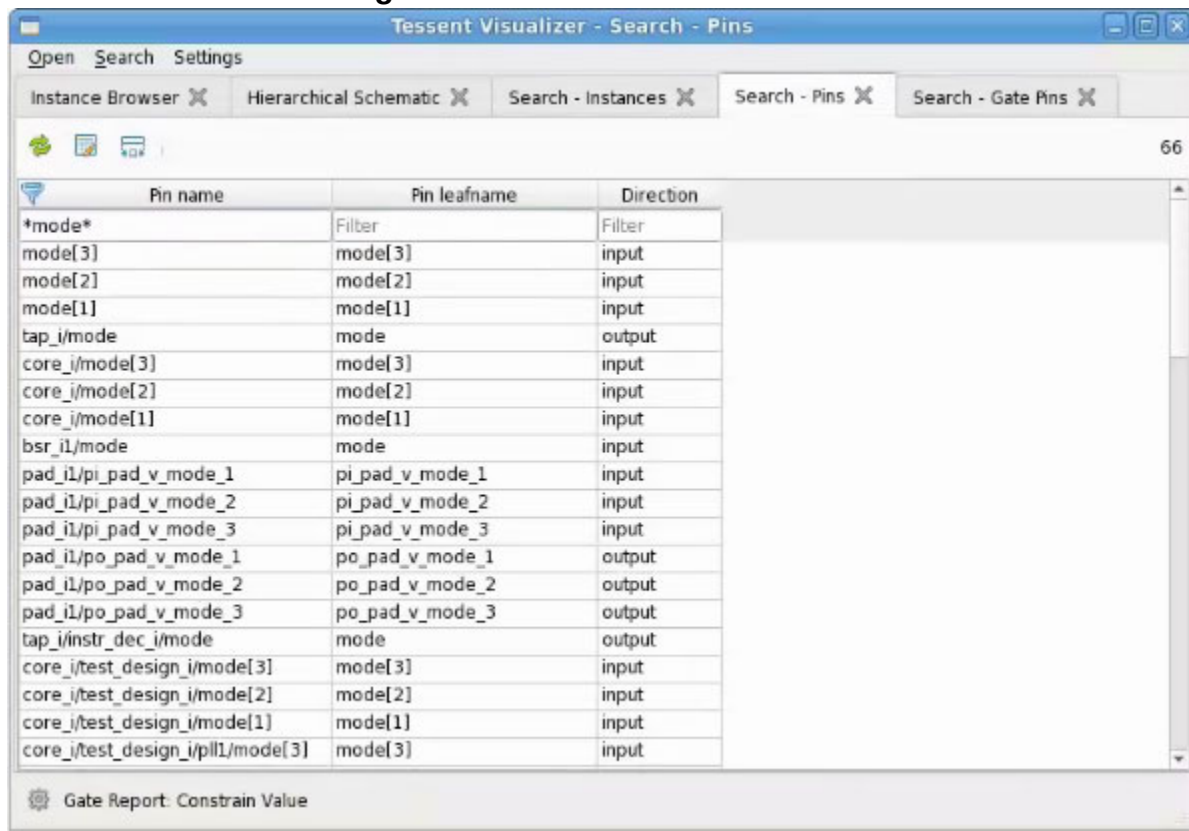
From the main Tessent Visualizer window menu, you can open the **Search** menu to search for instances, pins, or gate pins in your design.

Figure 9-56. Search Window: Instances



The preceding figure shows a search for all instances in the design that begin with the string “NOR” by using the filter functionality of the table.

Figure 9-57. Search Window: Pins



Pin name	Pin leafname	Direction
mode	Filter	Filter
mode[3]	mode[3]	input
mode[2]	mode[2]	input
mode[1]	mode[1]	input
tap_i/mode	mode	output
core_i/mode[3]	mode[3]	input
core_i/mode[2]	mode[2]	input
core_i/mode[1]	mode[1]	input
bsr_i1/mode	mode	input
pad_i1/pi_pad_v_mode_1	pi_pad_v_mode_1	input
pad_i1/pi_pad_v_mode_2	pi_pad_v_mode_2	input
pad_i1/pi_pad_v_mode_3	pi_pad_v_mode_3	input
pad_i1/po_pad_v_mode_1	po_pad_v_mode_1	output
pad_i1/po_pad_v_mode_2	po_pad_v_mode_2	output
pad_i1/po_pad_v_mode_3	po_pad_v_mode_3	output
tap_i/instr_dec_i/mode	mode	output
core_i/test_design_i/mode[3]	mode[3]	input
core_i/test_design_i/mode[2]	mode[2]	input
core_i/test_design_i/mode[1]	mode[1]	input
core_i/test_design_i/pil1/mode[3]	mode[3]	input

Gate Report: Constrain Value

The preceding figure shows a search for all hierarchical pins with the string “mode” in the pin name.

Figure 9-58. Search Window: Gate Pins

Gate id	Gate pin ID	Pin pathname	Direction
Filter	Filter	bsr_i1/*Q	Filter
371	371.0	bsr_i1/bsc_v_dout_1/reg_serial_output/Q	output
373	373.0	bsr_i1/bsc_v_dout_0/reg_latch_out/Q	output
375	375.0	bsr_i1/bsc_v_dout_0/reg_serial_output/Q	output
377	377.0	bsr_i1/bsc_edt_channels_out1/reg_latch_out/Q	output
379	379.0	bsr_i1/bsc_edt_channels_out1/reg_serial_output/Q	output
381	381.0	bsr_i1/bsc_v_nsout_7/reg_latch_out/Q	output
383	383.0	bsr_i1/bsc_v_nsout_7/reg_serial_output/Q	output
385	385.0	bsr_i1/bsc_v_nsout_6/reg_latch_out/Q	output
387	387.0	bsr_i1/bsc_v_nsout_6/reg_serial_output/Q	output
389	389.0	bsr_i1/bsc_v_nsout_5/reg_latch_out/Q	output
391	391.0	bsr_i1/bsc_v_nsout_5/reg_serial_output/Q	output
393	393.0	bsr_i1/bsc_v_nsout_4/reg_latch_out/Q	output
395	395.0	bsr_i1/bsc_v_nsout_4/reg_serial_output/Q	output
397	397.0	bsr_i1/bsc_v_nsout_3/reg_latch_out/Q	output
399	399.0	bsr_i1/bsc_v_nsout_3/reg_serial_output/Q	output
401	401.0	bsr_i1/bsc_v_nsout_2/reg_latch_out/Q	output
403	403.0	bsr_i1/bsc_v_nsout_2/reg_serial_output/Q	output
405	405.0	bsr_i1/bsc_v_nsout_1/reg_latch_out/Q	output

Gate Report: Constrain Value

The preceding figure shows a search for Q pins in all instances in the “bsr_i1” hierarchy.

You can select one or more rows in a filtered search table. Then, use the right-click popup menu to show those objects in a schematic window, add them to the pin data (pin searches), or show the HDL definition (instance searches).

See “[Common Features](#)” on page 385 for information about interacting with these tables in Tessent Visualizer.

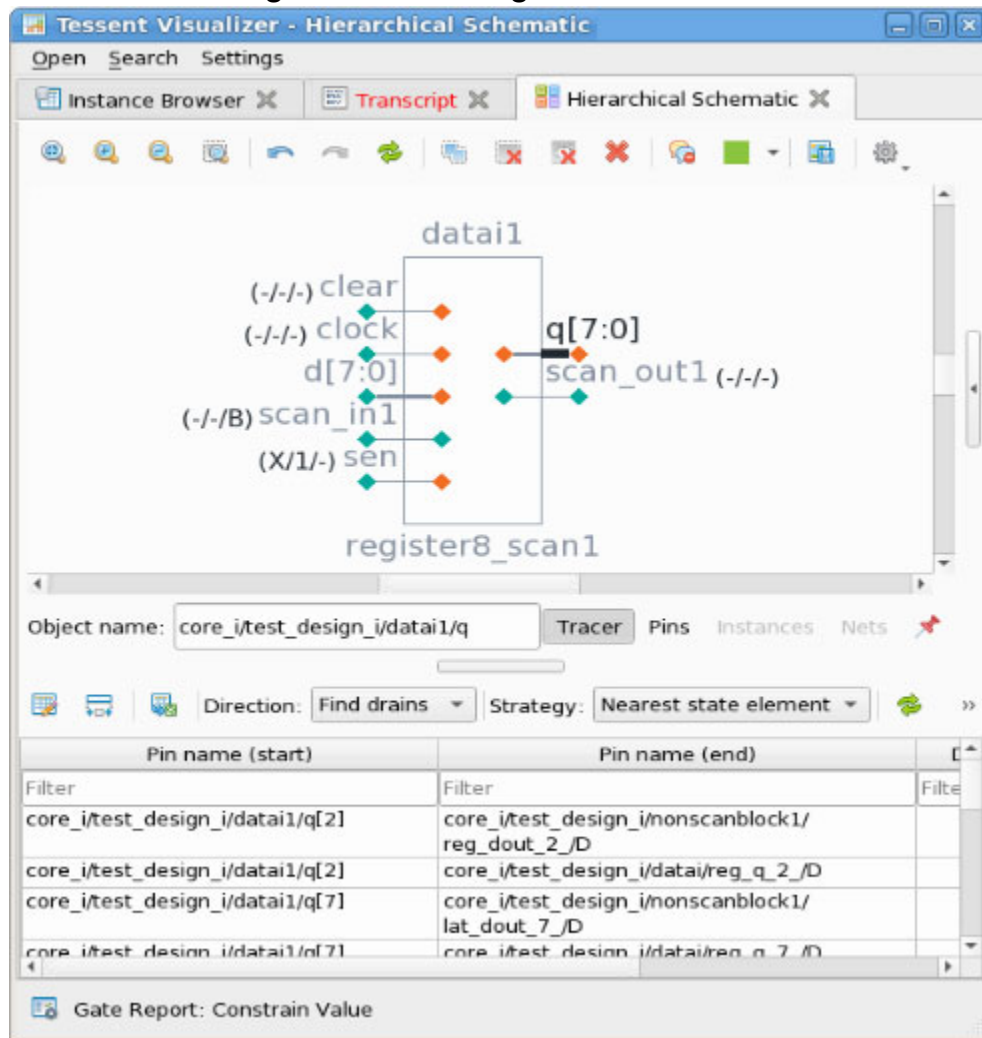
Usage Examples

This section provides some examples of using Tessent Visualizer to solve various design problems.

Listing All State Elements or Outputs from a Node

To see all the possible state elements or observation points from a node in your design, click the node in a schematic window and change the trace strategy to “Nearest State Element.” This gives you a listing of all the endpoints and the exact distance (number of gates in the path) from the source.

Figure 9-59. Listing State Elements

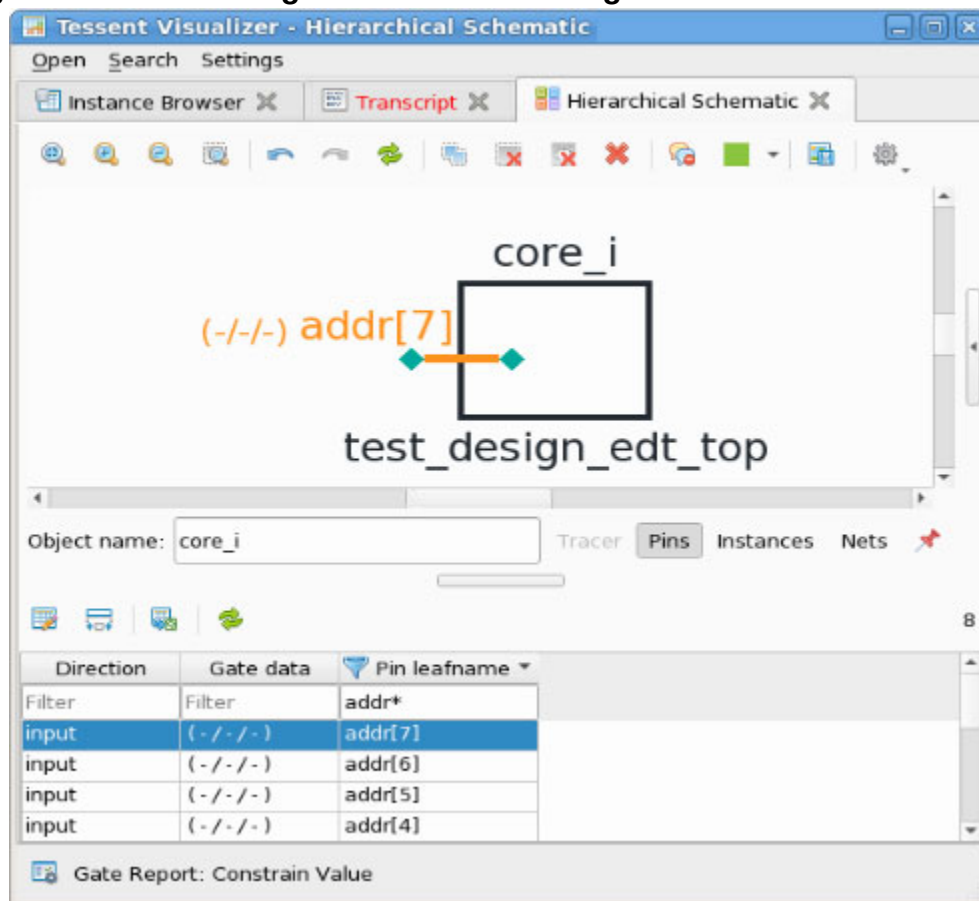


Adding Pins to Instances in Schematic Windows

When you add an instance to a schematic window directly or by tracing to it, only the pins that have connections are shown. This feature simplifies the schematic to focus on pins of interest and improves run time.

To add a particular pin to any instance, use the following steps:

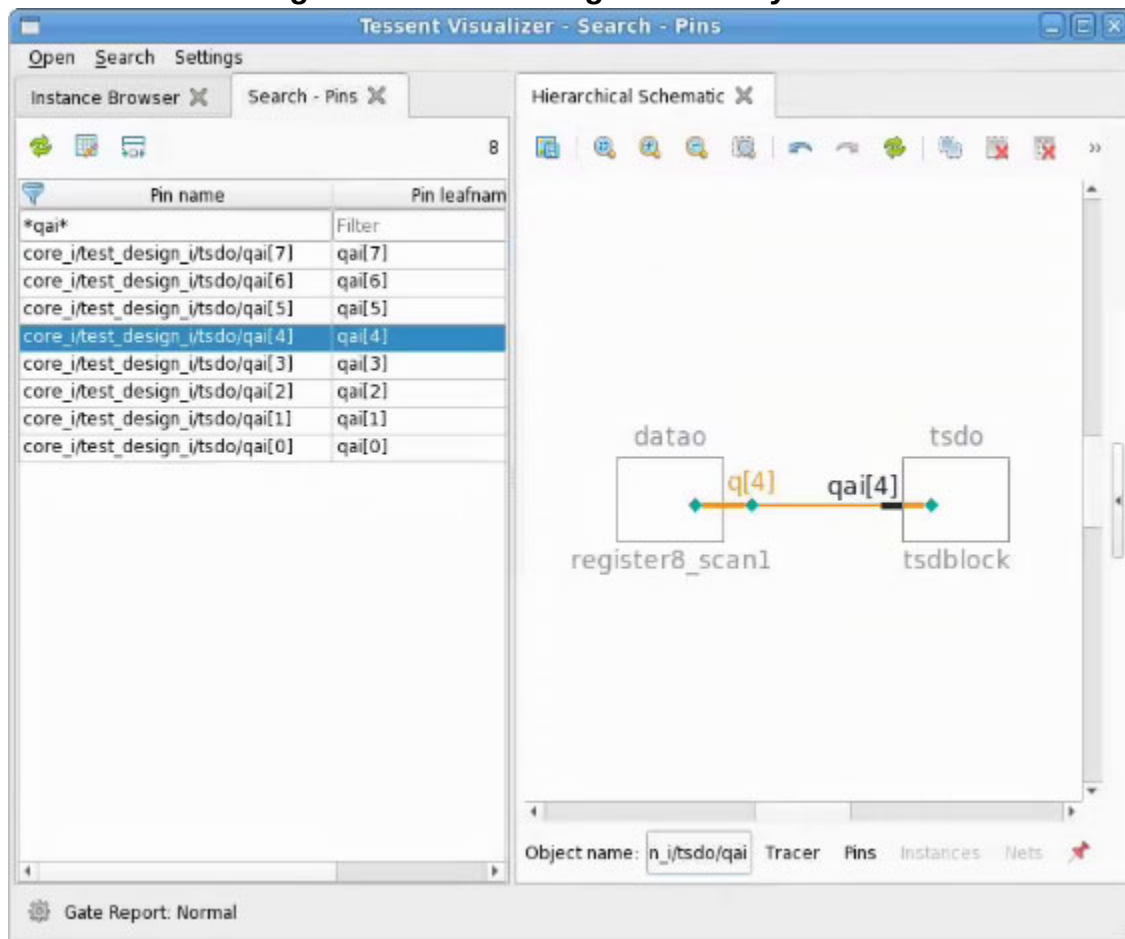
1. Select the instance of interest.
2. Click the “Pins” option in the tracer tool.
3. Use the filter feature to search for the pin desired.
4. Add the pin to the schematic by double-clicking or dragging it onto the schematic.

Figure 9-60. Searching for a Pin and Adding it to the Schematic Window

Searching for Pins by Name

If you are uncertain about exactly where a signal is in the hierarchy of your design, search for the pin by name using the Search - Pins feature with appropriate filtering. This example shows a search for the “qai” bus and subsequent selection of an element of that bus, which has appears in the hierarchical schematic. The signal can then be traced as desired.

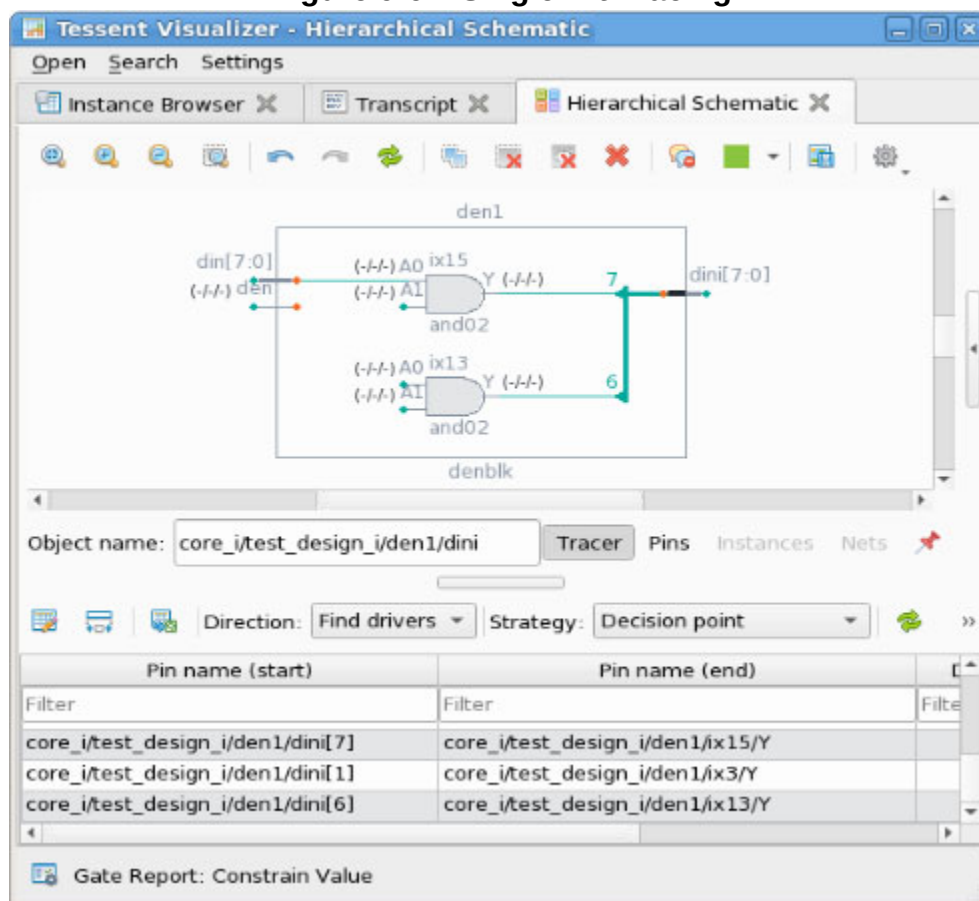
Figure 9-61. Searching for a Pin by Name



Tracing Single Bits of a Bus

When tracing a single bit of a bus, the bit is split from the bus and does not need to be selected multiple times. When tracing a net that recombines into a bus, the bit indices are shown in the schematic window.

Figure 9-62. Single Bit Tracing



Chapter 10

Configuration Data Visualizer

The Configuration Data Visualizer enables you to view specifications and to modify the configuration tree elements and their properties using a graphical interface.

Modifying the Contents of the Configuration Data Visualizer	449
Adding a Test Data Register to a SIB Example	450
Adding a Multiplexer to a SIB Example	451
Customizing the DFT Specification for EDT	452

Modifying the Contents of the Configuration Data Visualizer

View and modify specifications with the Configuration Data Visualizer.

Prerequisites

- Tessent Shell is invoked.

Procedure

1. Open the Configuration Data Visualizer by issuing the [display_specification](#) command to view and modify a DftSpecification. The specification can be one that was read by using the [read_config_data](#) command, or created with the “display_specification -create” command.

The Configuration Data Visualizer window opens and displays any existing configuration data.

2. Select an element from the Configuration Tree and choose an **Add** > menu item from the right-click popup menu.

The **Add** submenu displays a context-sensitive list of elements that shows the types of elements you can add based on the element you selected.

3. Specify the parameters for the newly added element in the Configuration Options panel:
 - Enter the values of the element’s listed properties. Predefined values appear when click you the  icon.
 - Click **Interface[+]** to define parameters of the element’s interface.
 - Click **DataInPorts[+]** and **DataOutPorts[+]** to define the number of ports on the element, their names, and their connections.

- Click **Attributes[+]** to define arbitrary attributes on the element's ICL module.
4. Click **Apply** to update the specification.
 5. When you have created and fully defined all of the elements that you want to add, click the  icon to validate the current specification and look for errors. You can also issue the “process_dft_specification -validate_only” command, which is an equivalent action.
 6. After you have validated the specification and resolved all errors, click the  icon to process the specification. You can also issue the process_dft_specification command, which is an equivalent action.

Results

New elements have been created and defined new in your specification, and those changes have been validated. The specified elements have been created by processing the specification.

Related Topics

[Adding a Test Data Register to a SIB Example](#)

[Adding a Multiplexer to a SIB Example](#)

Adding a Test Data Register to a SIB Example

Add a Test Data Register (TDR) to a SIB with the Configuration Data Visualizer.

Prerequisites

- Tessent Shell is invoked.

Procedure

1. Open the Configuration Data Visualizer from the Tessent Shell command line.
2. Select the SIB to which you want to add the TDR.
3. Right-click and choose **Add > Tdr**.
Change its position using the **Move Up** and **Move Down** menu items.
4. In the Configuration Options panel, enter a name in the ID text box. Type appropriate values in any of the other displayed fields.
5. Click  and  and modify the value of the “count” property in each to change the number of ports on the TDR.
6. Click **Apply** to update the specification.

Results

A TDR with the appropriate number of ports has been added to a SIB.

Related Topics

[Modifying the Contents of the Configuration Data Visualizer](#)

Adding a Multiplexer to a SIB Example

Add a ScanMux to a SIB with the Configuration Data Visualizer.

Prerequisites

- Tessent Shell is invoked.

Procedure

1. Open the Configuration Data Visualizer from the Tessent Shell command line.
2. Add a ScanMux by selecting the SIB to which you want to add it, then right-click and choose **Add > ScanMux**.

Change its position using the **Move Up** and **Move Down** menu items.

- a. Name the ScanMux by typing a name in the id field of the Configuration Options panel. Specify the desired value of any of the other displayed fields.
 - b. Define the ScanMux connections by clicking **Interface[+]** and typing the connection for the MUX select line. If a TDR is defined, you can use any TDR data output port to control the mux select signal.
 - c. Click **Apply** to update.
3. Add the first input port to the new ScanMux by right-clicking it and choosing **Add > Input**.
 - a. Select the new input port and type a number in the int text box of the Configuration Options panel.
 - b. Click **Apply** to update.
 - c. Add a TDR that will be accessed when this mux input port is selected by right-clicking the Input port and choosing **Add > Tdr**.
 - i. Name the TDR by selecting it and typing a name in the ID field of the Configuration Options panel.
 - ii. Connect the TDR to the design by doing the following:
 - a. Click the **DataInPorts[+]** button, click the **Connections[+]** button, and then click the  icon. Type the range that matches the design instance data bus size into the range field (for example, “7:0”), and type the pathname to the design instance in the pin_name field.

- b. Click the **DataOutPorts[+]** button, click the **Connections[+]** button, and then click the  icon. Type the range that matches the design instance data bus size into the range field, and type the pathname to the design instance in the pin_name field.
4. Define the connections for the second input port of the ScanMux using the instructions in Step 3.

Results

A ScanMux has been created and the select signal has been defined. Two Input wrappers with TDRs defined have been added to the ScanMux.

Related Topics

[Modifying the Contents of the Configuration Data Visualizer](#)

Customizing the DFT Specification for EDT

In the pre-scan DFT insertion flow, you create a DFT specification for inserting the EDT hardware. You may need to customize the DFT specification that Tessent Shell generates to suit your design requirements. Edit the DFT specification with the Configuration Data Visualizer.

Prerequisites

- You have created a DFT specification for inserting EDT. This procedure assumes that you specified the create_dft_specification command with the -sri_sib_list option.
- Tessent Shell is invoked.

Procedure

1. Open a DFT specification in the Configuration Data Visualizer by invoking the display_specification command.
2. In the Configuration Tree pane, right-click the DftSpecification wrapper and then choose **Add > EDT**.
3. In the ijtag_host_interface text box, type **Sib(edt)**, and then click **Apply**.
4. Right-click the EDT wrapper and choose **Add > Controller**.
5. Select the Controller wrapper, and then type the name of the controller in the id text box in the **Configuration Options** pane.
6. In the **Configuration Options** pane, type values in the following text boxes to configure how you want Tessent Shell to create the EDT controller hardware, and then click **Apply**.
 - a. **longest_chain_range** (two number boxes)
 - b. **scan_chain_count**

- c. **input_channel_count**
 - d. **output_channel_count**
7. If you are inserting EDT at the chip level (design_level chip), do the following:
 - a. Click the plus sign next to the Controller wrapper, right-click the **Connections** wrapper, and then choose **Add > EdtChannelsIn** and **Add > EdtChannelsOut** as many times as required for the EDT controller.
 - b. Within the Connection wrapper, select one of the input or output channels you created, and type the pin name for the selected channel in the port_pin_name text box in the **Configuration Options** pane. Click **Apply**.
 - c. Repeat the previous step to name all the input and output channels you created.
 8. (Optional) View the edited DFT specification by invoking the report_config_data command, and then invoking the read_config_data command to add the edited DFT specification to a dofile for later reuse in an automated flow.

Results

To view the modified DFT specification, specify the report_config_data command. For example:

```
report_config_data $spec
```

```
DftSpecification(cpu_top,rtl2) +{
  IjtagNetwork +{
    HostScanInterface(sri) +{
      Interface {
        design_instance : cpu_top_rtl_tessent_sib_sti_inst;
        scan_interface : client;
      }
    }
    Sib(sri) +{
      Attributes {
        tessent_dft_function : scan_resource_instrument_host;
      }
    }
    Tdr(sri_ctrl) {
      Attributes {
        tessent_dft_function : scan_resource_instrument_dft_control;
      }
    }
    Sib(occ) {
    }
    Sib(edt) {
    }
  }
}
EDT +{
  ijtag_host_interface : Sib(edt);
  Controller(c1) +{
    longest_chain_range : 65, 100;
    scan_chain_count : 85;
    input_channel_count : 3;
    output_channel_count : 3;
    Connections +{
      EdtChannelsIn(1) {
        port_pin_name : edt_channels_in[0];
      }
      EdtChannelsIn(2) {
        port_pin_name : edt_channels_in[1];
      }
      EdtChannelsOut(1) {
        port_pin_name : edt_channels_out[0];
      }
      EdtChannelsIn(3) {
        port_pin_name : edt_channels_in[2];
      }
      EdtChannelsOut(2) {
        port_pin_name : edt_channels_out[1];
      }
      EdtChannelsOut(3) {
        port_pin_name : edt_channels_out[2];
      }
    }
  }
}
```

Examples

Use the `read_config_data` command to save the modifications to the DFT specification into a dofile for re-use.

```
read_config_data -in $spec -from_string {
  EDT {
    ijttag_host_interface : Sib(edt);
    Controller (c1) {
      longest_chain_range : 65, 100;
      scan_chain_count : 85;
      input_channel_count : 3;
      output_channel_count : 3;
      Connections +{
        EdtChannelsIn(1) {
          port_pin_name : edt_channels_in[1];
        }
        EdtChannelsOut(1) {
          port_pin_name : edt_channels_out[0];
        }
        EdtChannelsIn(3) {
          port_pin_name : edt_channels_in[2];
        }
        EdtChannelsOut(2) {
          port_pin_name : edt_channels_out[1];
        }
        EdtChannelsOut(3) {
          port_pin_name : edt_channels_out[2];
        }
      }
    }
  }
}
```


Chapter 11

Timing Constraints (SDC)

This chapter describes the Synopsys Design Constraints (SDC) generated by Tessent Shell, and how you use it to constrain the test mode of the Embedded Test hardware generated by the tool.

Generating and Using SDC for Tessent Shell Embedded Test IP.....	458
SDC File Generation with Tessent Shell	458
SDC Design Synthesis with Tessent Shell	460
Preparation Step 1: Sourcing SDC File	460
Preparation Step 2: Setting and Redefining Tessent Tcl Variables	460
Preparation Step 3: Verifying the Declaration of Functional Clocks	462
Preparation Step 4: Redefining Other Tessent Tcl Variables	463
Synthesis Step 1: Loading the Design Into Your Synthesis Tool	464
Synthesis Step 2: Applying the SDC Constraints	464
Synthesis Step 3: Preparing the DFT Logic for Synthesis	464
Synthesis Step 4: Synthesizing Your Design	465
Synthesis Step 5: Writing Out Your Final SDC	465
Synthesis Step 6: Writing Out Your Final Netlist	466
SDC for Modal Static Timing Analysis	467
Checking Your Functional Logic Alone	467
Checking Your Embedded Test Logic	467
VHDL and Mixed Language Designs	469
VHDL Generate Blocks	469
SDC File Contents	470
tessent_set_default_variables	470
tessent_create_functional_clocks	471
tessent_<design_name>_set_dft_signals	471
tessent_constrain_<design_name>_mentor_ltest_disable	472
tessent_constrain_<design_name>_non_modal	472
tessent_<design_name>_kill_functional_paths	472
IJTAG Instrument	473
LOGICTEST Instruments	473
MemoryBIST Instrument	485
BoundaryScan Instrument	488
Cell and Pin Mapping Procs	488
Synthesis Helper Procs	489

Generating and Using SDC for Tessent Shell Embedded Test IP

We guide you through an example SDC flow, from generation of the Tessent SDC file, through adjusting the variables used within the file, to using it in your design synthesis.

First, you learn how to generate an SDC file for your design, which may be at the chip or the physical block level. SDCs for sub blocks are automatically merged by Tessent with the constraints of their parent physical block.

Then we show you how to adjust variables in the SDC file to match your particular synthesis setup or requirements. There is no need however, to ever directly edit the generated SDC files.

Next, you see how to use the generated SDC file in a step-by-step example synthesis flow, followed by additional notes on Static Timing Analysis (STA), mixed-language and VHDL specific issues.

Key SDC procs are then explained. These procedures are used by Tessent instruments (like MBIST or OCC) or maybe useful to use in your own synthesis scripts.

The appendices show full examples of using and adjusting Tessent SDC files in the Synopsys Design Compiler and Cadence Encounter synthesis environments, respectively. Also a complete example for using Tessent SDC files for STA is given in “[Example PrimeTime STA Script](#)” on page 510.

Note: The SDC described in this manual is to be used in conjunction with your functional SDC for the synthesis of your design after Tessent Shell RTL insertion and the static timing analysis of your design after synthesis. The Tessent Shell gate insertion flow uses a different set of SDC constraints during the [run_synthesis](#) command, but the generated SDC should still be used for layout.

SDC File Generation with Tessent Shell

You generate your design SDC file using Tessent Shell.

In the standard Tessent Shell flow, the tool creates the SDC for the current design when you execute the [extract_icl](#) command, which does the following:

- Stores the SDC in Tcl procs
- Writes the SDC to a single file named *design_name.sdc*, where *design_name* is the name of the current design loaded in Tessent Shell

The SDC file is located next to the extracted ICL, which is typically in the [dft_inserted_designs](#) directory as follows:

```
${tsdb}/dft_inserted_designs/${design_name}_${design_id}.dft_inserted_design
```

Every instrument type (for example, MBIST, IJTAG) of the current design and all sub-blocks that provide SDC constraints are represented by a separate proc in the SDC file. Constraints for logictest-related instruments such as OCC, EDT, or LBIST, including logictest-related DFT signals, are all grouped under similar placeholder “ltest” instrument procs.

There is no need to call each instrument’s individual SDC proc. Tessent Shell provides user procs, which are customized to call all relevant SDC procs for your design.

If you encounter a problem with SDC generation preventing ICL extraction, you can temporarily skip SDC generation by using the following command options:

```
extract_icl -skip_sdc_extraction
```

To re-generate your SDC file for a given design and to take advantage of changes in the SDC constraints in a newer version that does not reflect a hardware change, you can extract the SDC of your design without running ICL extraction by loading the desired design’s full view and executing the Tessent Shell [extract_sdc](#) command. You should load the interface view of all physical blocks of your design. For example:

```
read_design design_name -design_identifier design_id -view full  
extract_sdc
```

When it is time to use the SDC, locating your SDC file requires the following:

- Your latest TSDB location for the design you wish to synthesize/analyze.
- Your design’s name.
- The *design_id* of the latest insertion pass of your design.

For example, if your design “myChip” used only a single TSDB directory, the default “tsdb_outdir”, then your SDC files would be located in the following places:

- For the processed DftSpecification(myChip,rtl1):

```
tsdb_outdir/dft_inserted_designs/myChip_rtl1.dft_inserted_design/myChip.sdc
```

- For the processed DftSpecification(myChip,rtl2):

```
tsdb_outdir/dft_inserted_designs/myChip_rtl2.dft_inserted_design/myChip.sdc
```

Note



The latest SDC file is always a superset of the previous one. If you use a two-pass flow, you only need to load the latest file.

SDC Design Synthesis with Tessent Shell

What follows is a list of steps you must follow during your synthesis flow to properly handle Tessent Embedded Test IP. We use a non-modal approach to constrain the inserted test logic. This means that we let both functional and test clocks through muxes and we do not set any constant anywhere. This lets the synthesis tools optimize both the functional and test timing paths simultaneously.

These steps do not describe the entire flow, but, rather, only highlight typical steps affected by the presence of Tessent IP. Design Compiler commands are used as an example: see “[Example Design Compiler Synthesis Script](#)” on page 505 and “[Example Encounter Synthesis Script](#)” on page 507 for complete example scripts.

Preparation Step 1: Sourcing SDC File	460
Preparation Step 2: Setting and Redefining Tessent Tcl Variables	460
Preparation Step 3: Verifying the Declaration of Functional Clocks	462
Preparation Step 4: Redefining Other Tessent Tcl Variables	463
Synthesis Step 1: Loading the Design Into Your Synthesis Tool	464
Synthesis Step 2: Applying the SDC Constraints	464
Synthesis Step 3: Preparing the DFT Logic for Synthesis	464
Synthesis Step 4: Synthesizing Your Design	465
Synthesis Step 5: Writing Out Your Final SDC	465
Synthesis Step 6: Writing Out Your Final Netlist	466

Preparation Step 1: Sourcing SDC File

The first step is to source the Tessent Shell tool-generated SDC file, associated to the last insertion pass, from within your synthesis script. We suggest creating central variables for the TSDB location and design name:

```
set design_name myChip
set tsdb ../../../../golden_repo/${design_name}_tsdb
set design_id rtl2
source \
${tsdb}/dft_inserted_designs/${design_name}_${design_id}.dft_inserted_design \
/${design_name}.sdc
```

Preparation Step 2: Setting and Redefining Tessent Tcl Variables

The Tessent Shell tool-generated SDC file makes extensive use of Tcl variables. This section discusses the redefinition of those variables

Their definitions are contained in the proc `tessent_set_default_variables`. You must call this procedure to set all Tessent variables to their design dependent default value. All other Tessent SDC-related procs make use of these variables. This procedure call is mandatory:

```
tessent_set_default_variables
```

Tessent SDC variables can and should be redefined to better suit your setup and design. For example, you can change the TCK period from the default of 100.0ns to any value by redefining the variable `tessent_tck_period`.

```
set tessent_tck_period 80.0
```

Note


The value of `tessent_tck_period` might depend on the maximum tck clock frequency that can be applied to the circuit. See the “[IJTAG Network Performance Optimization](#)” section in the *Tessent IJTAG User’s Manual* showing how to maximize the frequency of the IJTAG network test clock.

It is important to understand that you do not need to edit the Tessent Shell-generated SDC file, but simply use the ‘set’ command in your synthesis script to overwrite the value of the Tessent Shell tool-specific SDC variable `tessent_tck_period`. Refer to “[Example Design Compiler Synthesis Script](#)” on page 505 and “[Example Encounter Synthesis Script](#)” on page 507 for a full synthesis script examples.

When the design contains LogicBIST instruments, create the `shift_clock_src` of the LogicBIST controller and indicate its name to the SDC procs by using the `tessent_lbist_shift_clock_src` TCL array variable. Creating this clock and setting its name with the `tessent_lbist_shift_clock_src` variable is mandatory.

Specify this variable as follows:

```
set tessent_lbist_shift_clock_src(lbist_inst<index>) clock_name
```

For example:

```
create_clock -name lbist_clock pll/clk_out_2 -period 20  
set tessent_lbist_shift_clock_src(lbist_inst0) lbist_clock
```

If multiple controllers use the same clock, create the clock once only. However, you must still specify the `tessent_lbist_shift_clock_src` variable for each controller with the same clock name. Starting at 0, specify as many `lbist_inst` indices as the number of LogicBIST controllers in the current design level minus one. For example, for a design with two LogicBIST controllers, set the `tessent_lbist_shift_clock_src(lbist_inst0)` and `tessent_lbist_shift_clock_src(lbist_inst1)` variables.

You can find the mapping of the `lbist_inst<index>` identifier to the instance path name of the LogicBIST controllers in the `tessent_lbist_mapping` TCL array variable inside the `tessent_set_default_variables` SDC proc. When the clock is created with a different name than

its source, specify the name of the clock as specified with -name option of the create_clock SDC command. Existence of this TCL variable and the clock it refers to is rule checked by the generated SDC.

Similarly, when the design contains InSystemTest instruments, indicate the clocks to SDC by using the tessent_ist_clock TCL array variable. You can find the mapping from the ist_inst<index> identifier to the InSystemTest controller instance path name in the tessent_ist_mapping variable.

Another key variable to possibly overwrite is the tessent_clock_mapping array explained in the next section. Other, much less frequently used variables are discussed in [“Preparation Step 4: Redefining Other Tessent Tcl Variables”](#) on page 463.

Preparation Step 3: Verifying the Declaration of Functional Clocks

In the Tessent Shell tool’s system mode ‘setup’, you may have specified asynchronous clocks, reference clocks and/or branch clocks. Some might be functional clock domain bases of some instruments, namely MemoryBist. They could be used in some timing constraints.

This information is stored in the array tessent_clock_mapping. It is defined in the generated SDC file, within the proc tessent_set_default_variables. It has the following form:

```
array set tessent_clock_mapping {
    <TessentShell ClockLabel> <FunctionalSDC ClockLabel>
    [...]
}
```

This array is used by the Tessent Shell tool-generated SDC constraints to refer to your functional clocks. They do so only through a remapped “tessent_clock_mapping(<TessentShell ClockLabel>)” array element. By default, in tessent_set_default_variables, <TessentShell ClockLabel> and <FunctionalSDC ClockLabel> are identical.

Note



It is imperative that you examine this array and look for cases where this default mapping must be updated. Overriding names will not be necessary if you have used the “-label” option of the [add_clocks](#) command in Tessent Shell to match the “-name” value in the SDC.

If you need to update one of the tessent_clock_mapping() array values, please do so in your main synthesis script, right below the call to the tessent_set_default_variables proc. For example, if you had a clock defined with a label “CK25” in Tessent Shell, but in your SDC constraints the same clock is defined with a name “PIN_CK25”, you need to specify:

```
set tessent_clock_mapping(CK25) PIN_CK25
```

The new definition of `tessent_clock_mapping(CLK25)` will override the default one contained in `tessent_set_default_variables`.

It is important to understand that you do not need to edit the Tessent-generated SDC file, but simply use the ‘set’ command in your synthesis script to overwrite the value of the Tessent SDC variable array element you want to change. Alternatively, if many of your Tessent defined clocks and respective SDC clocks have different names, you might want to redefine a block of the array:

```
set tessent_clock_mapping(CLK25) pCLK25
set tessent_clock_mapping(CLK_PLL_REF) pCLK100
```

OR

```
array set tessent_clock_mapping {
  CLK25 pCLK25
  CLK_PLL_REF pCLK100
}
```

IMPORTANT: You do not need to define `tessent_clock_mapping()` entries for clocks that you have not specified in Tessent Shell. Those will not be used in the timing constraints.

Preparation Step 4: Redefining Other Tessent Tcl Variables

The `tessent_tck_period` and `tessent_clock_mapping` array variables are the most important Tessent SDC variables you should double check and overwrite as needed. Tessent SDC uses a number of additional variables that are all defined and set to a default value in the `tessent_set_default_variables` proc.

For example, you can independently change the input and output delay (from the default, 25% of the `tessent_tck_period`) or change the hierarchy separator character. Please see the header of the generated SDC file and the embedded comments in the `tessent_set_default_variables` proc, explaining each such variable. Within the synthesis tool, you can use the TCL command “`info body tessent_set_default_variables`” to see what variables are being defined.

Note



The value of `tessent_tck_period` might depend on the maximum tck clock frequency that can be applied to the circuit. See the “[IJTAG Network Performance Optimization](#)” section in the *Tessent IJTAG User’s Manual* showing how to maximize the frequency of the IJTAG network test clock.

Synthesis Step 1: Loading the Design Into Your Synthesis Tool

The most effective way of loading a design, which was edited by Tessent Shell with embedded Tessent test logic, into the synthesis tool is to use Tessent Shell-generated supporting files.

If you are using Design Compiler to synthesize your design, you can use the Tessent Shell command `write_design_import_script`. This command will write out a single file containing analyze commands to read in every file which was read for the current design in memory in the right order of dependency, and from the right location, i.e either your golden source file or the Tessent edited file from the TSDB. You can use this command right after the [extract_icl](#) command of your last RTL insertion pass or simply load the latest design view with the command `read_design` and then use it.

You can then set the current design using:

```
source ${design_name}.dc_shell_import_script
elaborate [...]
```

Synthesis Step 2: Applying the SDC Constraints

If you use Design Compiler, you need to set the variable “`timing_enable_multiple_clocks_per_reg`” to true. This is needed to properly constrain MemoryBIST in the presence of MemoryBIST clock muxing.

```
set_app_var timing_enable_multiple_clocks_per_reg true
```

To apply the Tessent Shell-generated SDC constraints, run the merged non_modal proc:

```
tessent_constrain_${design_name}_non_modal
```

This Tessent SDC proc will in turn call all instrument non-modal procs as needed by your design and its sub-blocks. There is nothing else for you to do than call this one proc to apply all non-modal SDC constraints.

Synthesis Step 3: Preparing the DFT Logic for Synthesis

Once your design is loaded, you need to make sure to preserve instances of Tessent Shell-inserted “persistent” cells and possibly instrument instances, depending on your downstream needs.

Refer to “[Synthesis Helper Procs](#)” on page 489 for more information on the different options.

Persistent cells should not be optimized as Tessent Shell SDC uses them as anchor points for layout and STA, and different instances need to be preserved depending on if you will do scan

insertion with Tessent Shell or more dft insertion. Procs are provided in the .sdc file to generate a list of all instances that need to be preserved in different ways:
tessent_get_preserve_instances, tessent_get_optimize_instances and
tessent_get_size_only_instances.

First you need to prevent boundary optimization of DFT objects using the proc
tessent_get_preserve_instances:

```
set_app_var compile_enable_constant_propagation_with_no_boundary_opt false
set_preserve_instances [tessent_get_preserve_instances scan_insertion]
set_boundary_optimization $preserve_instances false
set_ungroup $preserve_instances false
set_app_var compile_seqmap_propagate_high_effort false
set_app_var compile_delete_unloaded_sequential_cells false
```

Then, since boundary optimization applies hierarchically, you re-enable boundary optimization of the child instances of DFT objects using tessent_get_optimize_instances:

```
set_boundary_optimization [tessent_get_optimize_instances] true
```

Cells from your provided Tessent Cell Library instantiated in Tessent Shell must also be preserved but can be resized to allow critical data path timing optimization during synthesis. You can get them using tessent_get_size_only_instances:

```
set_size_only -all_instances [tessent_get_size_only_instances]
```

Tool Limitation: set_size_only does not have the desired effect in Cadence tools. See [“Example Encounter Synthesis Script”](#) on page 507 for an example of the suggested procedure.

If your design contains shared bus assemblies, you can save significant area by ungrouping their content, with the exception of the MemoryBIST controller. Refer to the “Synthesis Step 3” portions in [“Example Design Compiler Synthesis Script”](#) and [“Example Encounter Synthesis Script”](#) for examples of the suggested procedures.

Synthesis Step 4: Synthesizing Your Design

You can now proceed with the compilation of your design:

```
link
check_design
compile -boundary_optimization [...]
```

Synthesis Step 5: Writing Out Your Final SDC

Use the write_sdc command after this step to export the merged SDC (Functional and Tessent Shell) for use with your layout tool:

```
write_sdc ${design_name}.merged_sdc
```

Synthesis Step 6: Writing Out Your Final Netlist

You can now proceed with writing out your synthesized netlist.

```
write -format verilog -output ${design_name}.vg ${design_name} -hier
```

SDC for Modal Static Timing Analysis

Below are guidelines to help you perform static timing analysis of the embedded test hardware that was inserted by Tessent Shell.

Checking Your Functional Logic Alone	467
Checking Your Embedded Test Logic.....	467

Checking Your Functional Logic Alone

If you want to only focus on functional logic and completely disable all Tessent Shell embedded test IP, include the following steps in your master functional STA script.

First, if flops in your cell library do not propagate constant settings through their set or reset pin to their data output pin, you need to set a PrimeTime variable:

```
set case_analysis_sequential_propagation always
```

This will allow disabling all embedded test modes with just a few asserts of the TAP's reset port.

For example:

```
-----
... loading your functional constraints ....
set case_analysis_sequential_propagation always
# Hold the DFT in reset and kill TCK to make sure only functional logic is
# active.
set_case_analysis TRST 0
set_case_analysis TCK 0
-----
```

You will also need to disable your scan circuit timing by forcing your scan_enable pin to its inactive value, and by issuing a set_false_path to/from all your lockup cells and your pipeline flops. Those can normally be found with regular expressions such as:

```
[get_pins -hier <standard naming>*/d]
[get_pins -hier <standard naming>*/q]
```

You might also have to turn your BISR clock or reset pin off, if present, like in:

```
set_case_analysis bisr_clk      0
set_case_analysis bisr_reset    0
```

Checking Your Embedded Test Logic

You can find a self-documenting example script of how to run STA using Tessent Shell generated hardware.

Refer to “[Example PrimeTime STA Script](#)” on page 510. Carefully read the instructions in this script before proceeding.

The script executes the following:

- Loads the generated SDC file and initializes some Tcl variables used by Tessent Shell SDC constraints.
- Sets some Primetime control variables to their required value.
- Sources user input files if present in the same directory as follows:
 - *user_timing_data.tcl*
 - *user_remapping_procs.tcl*
 - *load_design.pt*
- Loads your final netlist.
- Sequentially runs all EmbeddedTest modes present in your design, each separated by a “reset_design” command.
- Runs an example “report_constraints” command for each mode and redirects its output to a file.

VHDL and Mixed Language Designs

Synthesis tools have similar but sometimes different internal representations of elaborated designs. Here are some instructions on how to deal with those differences to be able to use the SDC constraints generated by Tessent Shell.

VHDL Generate Blocks..... 469

VHDL Generate Blocks

Required tool settings and other mixed-language issues.

Synopsys Design Compiler

Constraints applying to cells and pins in VHDL generate loops are problematic to apply in `dc_shell`. The default naming scheme of the internally synthesized design is different for Verilog and VHDL. To illustrate, here is a Verilog path of an instance within 2 nested generate for loops: `blk[0].jblk[0].i0`. The equivalent configuration in VHDL would be changed during `dc_shell`'s GTECH pre-synthesis and would be named `i0_0_0`. Since Tessent Shell's SDC does not "flatten" generate loops, if you have VHDL generate loops in your design, you must use the following setting in `dc_shell` to restore the loop naming:

```
set_app_var hdlin_enable_upf_compatible_naming true
```

This variable will restore VHDL generate loop naming to follow Verilog style: `blk(0)/jblk(0)/i0`

Cadence Encounter and Genus

There are no special steps for these two synthesis tools to be able to apply Tessent Shell generated SDC constraints to a mixed language design.

Dealing with Unrolled VHDL Generate for Loops

Tessent Shell tries to minimize unrolling of generate loops. If VHDL generate loops are unrolled then one "if-generate" block is generated for each loop iteration. The trailing closing square brace, that would normally be changed to an underscore, has to be truncated because of VHDL language restrictions. This means that functional SDC constraints pointing to cells or pins within those generate blocks will not match even if special characters are mapped to wildcards. Ex: the instance path `blk[0].i0` which would need to be unrolled would be written out as `blk_0.i0`.

For this reason, we recommend using the provided procs `tessent_get_pins` and `tessent_get_cells`, they will be able to match your functional instance path "`blk[0].i0`" to `blk_0.i0` using a caching algorithm that searches for generate blocks in the path and tries to match with our unrolling strategy if the path does not match outright. Refer to the "[Cell and Pin Mapping Procs](#)" on page 488 to see the procs' description.

SDC File Contents

For your convenience and information, here is a description of the procs contained in the generated SDC file. Note that many of these procs are called automatically by other procs in the file. You do not need to call all of these procs.

tessent_set_default_variables	470
tessent_create_functional_clocks	471
tessent_<design_name>_set_dft_signals	471
tessent_constrain_<design_name>_mentor_ltest_disable	472
tessent_constrain_<design_name>_non_modal	472
tessent_<design_name>_kill_functional_paths	472
IJTAG Instrument	473
LOGICTEST Instruments	473
MemoryBIST Instrument	485
BoundaryScan Instrument	488
Cell and Pin Mapping Procs	488
Synthesis Helper Procs	489

tessent_set_default_variables

This proc defines the initial/default value of all global variables used in all procs of the sdc file. The variables contained in this proc are there for you to customize the constraints without having to edit the constraints themselves. A good example of this would be the variable “tessent_tck_period”. It currently defaults to 100.0 (nanoseconds). If your TCK clock has a different period than 100ns, you can overwrite the value of this variable after having called the proc tessent_set_default_variables but before calling the constraint procs.

Note

 The value of tessent_tck_period might depend on the maximum tck clock frequency that can be applied to the circuit. See the “[IJTAG Network Performance Optimization](#)” section in the *Tessent IJTAG User’s Manual* showing how to maximize the frequency of the IJTAG network test clock.

The mapping between LogicBIST and InSystemTest controller IDs and their instance path names is available in this proc. Use the mapping information to indicate the clocks of these instruments to the other ltest SDC procs as described in “[Preparation Step 2: Setting and Redefining Tessent Tcl Variables](#)” on page 460.

This proc must always be the first proc to be run.

tessent_create_functional_clocks

This proc is generated for your convenience. It contains the `sdcreate_clock` and `create_generated_clock` commands to define the various functional clocks needed by the instruments constrained by the `sd` procs. Typically, your functional clocks would already be defined by your functional SDC. If that is the case, you should override the default values of the `tessent_clock_mapping` array to match the clock names you used in the definition of your clocks.

This proc would typically not be called in your synthesis SDC.

Required clocks are determined with ICL tracing. `create_clock` constraints are added for all clock sources like top level `ClockPort` ports and source `ToClockPort` ports (embedded crystals). `create_generated_clock` constraints are added for all generated clocks (add_clocks - reference), branch clocks and unidentified ICL `ToClockPort` ports (typically PLL's). If your design contains an ICL instrument with a `ToClockPort` that should not require a new generated clock, please set the `exclude_from_sdc` attribute on the port.

tessent_<design_name>_set_dft_signals

This procedure forces all your specified static `dft_signal` sources to either reset or get their default value during `all_test`, depending on the proc's argument.

```
argument mode ::= reset | logic_off
```

- `logic_off` — Recommended argument for running non-logictest STA modes such as memoryBIST. But if you added Mentor TestKompress, your `sd` file will also get the proc “`tessent_constrain_<design_name>_mentor_ltest_disable`”. You then only need to call that proc for running the other non-logictest modes such as modal memoryBIST.
- `reset` — Disables all DFT-inserted logic.

If you are using functional-only mode, you can optionally call the proc with the “reset” argument under one of the following circumstances:

1. Many static `dft` signals come from top-level ports.
2. Although all static `dft` signals sourced by the Mentor Graphics IJTAG Test Data Registers (TDR) reset when the IJTAG reset port is asserted, the timing tool would not propagate that assert through the TDR flops. That may happen in Synopsys Primetime when the control variable “`case_analysis_sequential_propagation`” is set to “never”.

tessent_constrain_<design_name>_mentor_ltest_disable

This proc disables all Mentor Logictest logic in your design. You call this proc when setting up exclusive STA mode for your IJTAG, BoundaryScan or memoryBIST, and you do not want to bother about ltest logic and scan chains timing at the same time.

You can also invoke it right after the tessent_constrain_<design_name>_non_modal proc in order to get non-modal constraints just for functional/ijtag/bscan/memoryBist (disabling logictest) if you plan to apply two different mode scripts during either synthesis and/or layout. The second mode would time logictest-only shift modes, for which you would invoke the proc "tessent_constrain_<design_name>_mentor_ltest_shift". See this proc for more details.

tessent_constrain_<design_name>_non_modal

This proc contains calls to the non_modal procs of all constrained instruments. They are documented below. For synthesis, this is the only constraint proc you need to call.

tessent_<design_name>_kill_functional_paths

This procedure disables all timing paths involving non-Tessent flops. It allows running test-only timing analysis without having to fix functional paths violations with your own functional SDC constraints. It helps making functional mode STA and Mentor DFT mode STA fully orthogonal, and reduces the Mentor DFT mode STA run-time.

You can optionally run this procedure when you know that your functional logic has many intra-domain timing paths that cannot meet default single-cycle path timing with the clocks being set by tessent_create_functional_clocks and tessent_constrain_* procs.

Disabling some sequential cells in your design might accidentally block the test clocks feeding your inserted instruments. Examples of such cells are Integrated Clock Gating (ICG) cells or flops that belong to clock dividers. You can fix this by specifying those cells either by module name or by instance names.

By module name:

```
set ClockSeqCellModuleRegExp "<regularExpression>"
```

By instance name:

```
set ClockSeqCellInstanceRegExp "<regularExpression>"
```


The procedure optionally creates a report file containing a sorted list of all disabled flops instance names. To enable this option, you need to define the following variable prior to calling `tessent_kill_functional_paths`:

```
set CreateDisabledFlopsReport 1
```

IJTAG Instrument

Tessent IJTAG provides a single proc:

- `tessent_constrain_<design_name>_mentor_ijtag_non_modal`
 - This proc would typically not be called directly in your synthesis script, it is called as part of `tessent_constrain_<design_name>_non_modal`.

This proc contains the clock creation for your TCK clock and configurable input and output delays for created ports at the `sub_block` and `physical_block` design levels.

It also creates an asynchronous clock group with all TCK clocks. It is used to prevent synthesis from balancing paths between TCK and functional clocks. This clock group could be recreated with additional clocks if your design contains Tessent BoundaryScan and/or Tessent OCC hardware. This is managed via the “`tessent_tck_clock_list`” global variable. This variable is built from different procs (`ijtag` & `jtag_bscan`) and used to create an asynchronous clock group. If you have your own TCK generated clocks that are not used in the Tessent Shell SDC constraints, we suggest adding their name to this list right after calling `tessent_set_default_variables`.

```
tessent_set_default_variables  
lappend tessent_tck_clocks_list my_tck_generate_clock
```

At the sub-block and physical block levels, constraints are added to reflect the timing of the IJTAG protocol. The IJTAG reset port is declared a false path and the IJTAG select port's hold paths are declared false and its setup paths are given two cycles with a `muticycle_path` constraint.

LOGICTEST Instruments

The section lists the logic test-based procs. LogicBIST and InSystemTest-related procs listed below are included only when LogicBIST and InSystemTest controllers are present in the design.

1. Non-modal constraints, to be merged with other DFT instrument constraints, for synthesis or layout. See this proc description for a discussion on single-mode synthesis/layout vs multi-mode synthesis/layout flow:
 - [tessent_constrain_<design_name>_mentor_ltest_non_modal](#)
 - `tessent_constrain_<design_name>_mentor_in_system_test_non_modal`

2. Modal procs, used for signoff STA and optionally for synthesis or layout constraining:
 - `tessent_constrain_<design_name>_mentor_ltest_modal_shift`
 - `tessent_constrain_<design_name>_mentor_ltest_disable`
 - `tessent_constrain_<design_name>_mentor_ltest_modal_lbist_shift`
3. Procs only for per-mode signoff STA constraints:
 - `tessent_constrain_<design_name>_mentor_ltest_edt_shift`
 - `tessent_constrain_<design_name>_mentor_ltest_bypass_shift`
 - `tessent_constrain_<design_name>_mentor_ltest_single_bypass_chain_shift`
 - `tessent_constrain_<design_name>_mentor_edt_slow_capture`
 - `tessent_constrain_<design_name>_mentor_ltest_edt_fast_capture`
 - `tessent_constrain_<design_name>_mentor_ltest_modal_lbist_shift`
 - `tessent_constrain_<design_name>_mentor_ltest_modal_lbist_capture`
 - `tessent_constrain_<design_name>_mentor_ltest_modal_lbist_setup`
 - `tessent_constrain_<design_name>_mentor_ltest_modal_lbist_single_chain`
 - `tessent_constrain_<design_name>_mentor_ltest_modal_lbist_controller_chain`
4. Procs sub-invoked by the above procs.
 - `tessent_constrain_<design_name>_mentor_ltest_create_clocks`
 - `tessent_constrain_<design_name>_mentor_ltest_set_pin_delays`

tessent_constrain_<design_name>_mentor_ltest_create_clocks

This proc creates the slow-speed test clocks that will be used during scan mode. There are a few possible clock configurations:

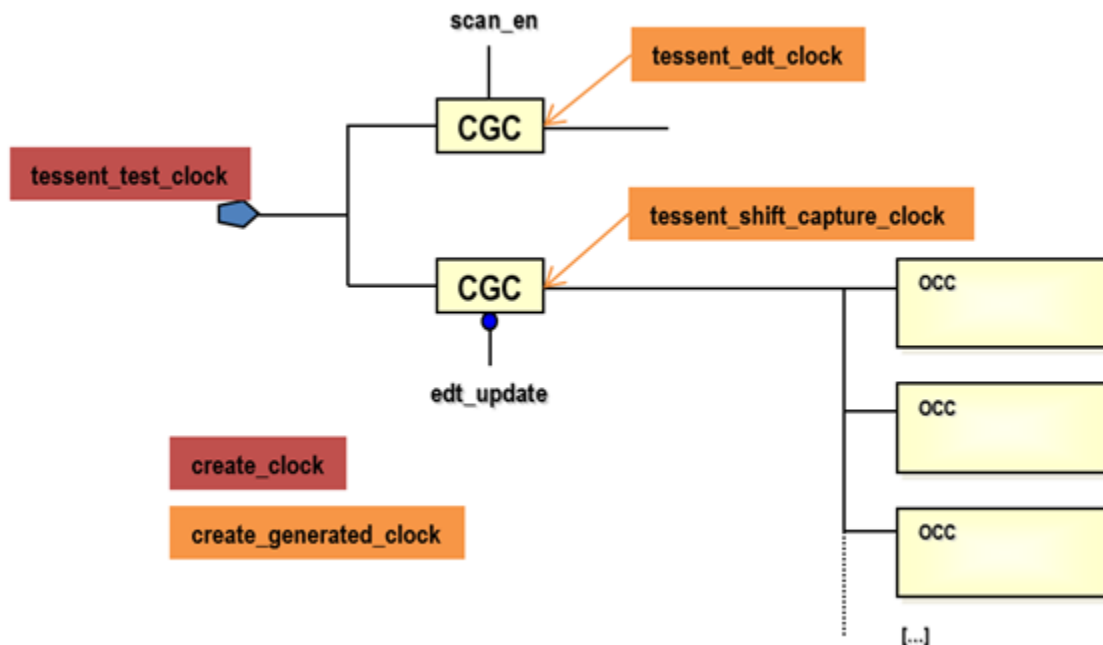
- **tessent_test_clock** — This is a DFT signal in Tessent Shell. It should be your tester's synchronous clock and is used to create the `shift_capture_clock` and the `edt_clock`.
- **tessent_edt_clock** — This clock can either be a primary input port or it can be generated from the `test_clock` and is used to time the shift paths of the EDT logic.
- **tessent_shift_capture_clock** — This clock can either be a primary input port or it can be generated from the `test_clock` and is the main scan mode clock propagating to your functional logic through the OCCs.
- **tessent_virtual_slow_clock** — A virtual slow clock of the same frequency as your scan or EDT shift clock, used to time all top-level ports that directly interface with your EDT

controllers and your scan chains, through the “set_input_delay” and “set_output_delay” constraints.

- **tessent_lbist_shift_clock_src** — This clock is typically an internally generated free-running clock used as the source for LogicBIST tests.

When using LogicBIST, the clock gaters (CGC) shown in the following figure for `tessent_edt_clock` and `tessent_shift_capture_clock` are located inside the LogicBIST controller, and the clock source automatically updates to this new location. This clock is created by the user outside of the Tessent-generated SDC.

Figure 11-1. tessent_test_clock and tessent_shift_capture_clock Creation



This proc is also called by every SDC proc that propagates the `edt_clock` and/or `shift` clocks, specifically:

- `tessent_constrain_<design_name>_mentor_ltest_non_modal`
- `tessent_constrain_<design_name>_mentor_ltest_shift`
- `tessent_constrain_<design_name>_mentor_ltest_modal_slow_capture`
- `tessent_constrain_<design_name>_mentor_ltest_modal_lbist_shift`
- `tessent_constrain_<design_name>_mentor_ltest_modal_lbist_capture`

The following is an example of the created clocks you would get if you ran the following commands:

```
add_dft_signals test_clock -source_nodes [get_ports test_clock]
```

add_dft_signals {edt_clock shift_capture_clock} -create_from_other_signals

```
proc tessent_constrain_myDesign_mentor_ltest_create_clocks {} {  
    global tessent_slow_clock_period  
    # test_clock: <<< This clock will also propagate through the  
    #               shift_capture_clock clock gating gate.  
    create_clock [get_ports test_clock] \  
        -add -period [expr $tessent_slow_clock_period] \  
        -name tessent_test_clock  
    # edt clock:  
    create_generated_clock -source [get_ports test_clock] \  
        [tessent_get_pins tessent_persistent_cell_edt_clock/clkg] \  
        -divide_by 1 \  
        -name tessent_edt_clock \  
        -add -master_clock tessent_test_clock  
    # Virtual slow clock, for use with input/output delay constraints in  
    # scan or edt mode:  
    create_clock \  
        -period $tessent_slow_clock_period \  
        -name tessent_virtual_slow_clock  
}
```

tessent_constrain_<design_name>_mentor_ltest_edt_fast_capture

This proc must be called in your STA to constrain the EDT in fast capture mode. Logictest fast capture mode is the one that detects your at-speed transition faults. It counts on user SDC to provide the capture clocks and timing exceptions that would actually time the capture paths. Typically this would be your original functional SDC constraints, but those might still require some adjustments if your test conditions (clock frequencies, false paths) slightly differ from what they are in functional mode. Obviously, you will also need to remove any constraint that resets the IJTAG network or ties the scan_enable or test_enable control pins to zero.

If present, the following dft signals are turned off:

- scan_enable (putting the circuit in capture mode)
- control_test_point_en (these hold during capture)

The proc also allows you to check that some dft signals such as x_bounding_en, memory_bypass_en, observe_testpoint_en are doing their isolation job correctly. You can optionally force them to a desired value by setting global Tcl variables prior to calling the proc.

The following demonstrates how this is done (annotations are in red):

```
global memory_bypass_en_value <<< set this variable if needed in
                                   your master script
if {[info exists memory_bypass_en_value]} {
    set_case_analysis $memory_bypass_en_value \
        [tessent_get_pins eltl_mbist_tessent_tdr_sri_ctrl_inst/
tessent_persistent_cell_memory_bypass_en/Y]
}
global x_bounding_en_value <<< set this variable if needed in
                              your master script
if {[info exists x_bounding_en_value]} {
    set_case_analysis $x_bounding_en_value \
        [tessent_get_pins eltl_dft_tessent_tdr_sri_ctrl_inst/
tessent_persistent_cell_x_bounding_en/Y]
}
global observe_test_point_en_value <<< set this variable if needed
                                     in your master script
if {[info exists observe_test_point_en_value]} {
    set_case_analysis $observe_test_point_en_value \
        [tessent_get_pins eltl_dft_tessent_tdr_sri_ctrl_inst/
tessent_persistent_cell_observe_test_point_en/Y]
}
```

Finally, the proc also disables any other timing paths that are ignored during capture, such as those going through the EDT channel pins.

tessent_constrain_<design_name>_mentor_edt_slow_capture

This proc must be called in your STA to constrain the EDT in slow capture mode.

The proc does the following:

- Forces the test_enable dft signals on.
- Declares the test_clock.
- By default, relaxes the hold of the test_clock to itself unless you use a user-defined Tcl variable that is in the proc. Refer to the Note below.
- Disables ignored timing paths.
- Optionally forces your scan_enable to zero, or, if you set the global Tcl variable “tessent_timing_scan_en_in_slow_capture”, set an MCP using your global Tcl variables tessent_scan_en_setup_extra_cycles and tessent_scan_en_hold_extra_cycles.
- Set an MCP for your edt_update signal, if present, using your global variables tessent_edt_update_setup_extra_cycles and tessent_edt_update_hold_extra_cycles.

Note

 Just like with the above <prefix>shift_ltest_non_modal proc, propagating test_clock may create false capture timing paths across asynchronous domains as single-cycle paths of test_clock. This may or may not be a problem, given that test_clock fanout is balanced and running at slow speed. By default, this proc assumes that all valid intra-domain hold timing paths were already covered by your functional modes STA, and therefore sets a multicycle_path of 1 hold to all paths from test_clock to itself. Remember that all scan mode shift paths are properly covered for both setup and hold by the above defined “xxx_ltest_shift” mode proc. If you need to, you can still force the proc to revert to zero-hold constraint by setting the Tcl variable tessent_time_hold_in_slow_capture to value 1 in your master calling script.

tessent_constrain_<design_name>_mentor_ltest_modal_shift

This proc sets the circuit in scan shift mode. It covers both EDT shift modes and all available bypass modes, assuming you will run them all with the same shift clock frequency. If you need more specific timing analysis, you can choose to apply the following procs in separate STA runs. These procs sub-invoke the tessent_constrain_<design_name>_mentor_ltest_shift proc.

- tessent_constrain_<design_name>_mentor_ltest_edt_shift
- tessent_constrain_<design_name>_mentor_ltest_shift_bypass
- tessent_constrain_<design_name>_mentor_ltest_single_bypass_chain_shift

The three above modal shift procs all sub-invoke the tessent_constrain_<design_name>_mentor_ltest_shift proc, which applies a set of common constraints required for shifting. Then they only need to complete their mode setting by forcing the edt_bypass and single_chain_bypass signals to the required constant value.

proc tessent_constrain_<design_name>_mentor_ltest_shift does the following:

- Creates the shift clocks.
- Applies set_input/output_delays to scan control pins.
- Forces dft signals ltest_en and scan_en to 1.
- Applies any LogicBIST mode constraints as applicable.

Note

 You can save on total STA time and flow complexity if you already plan to apply the same test_clock (with the same period) for all shift and bypass modes. In this case you only need to call the tessent_constrain_<design_name>_mentor_ltest_shift proc directly in place of all above three procs.

Note

 If you plan to feed your synthesis and/or layout tool with two separate mode scripts (functional/dft and logictest), the `tessent_constrain_<design_name>_mentor_ltest_shift` proc is the one you need to apply for the logictest mode. That way, all possible shift paths will be timed, and since `scan_enable` is forced active, it will prevent the existence of a potentially large number of false capture timing paths across your functional logic. It will also cover all your possible scan chain configurations at once, including the internal and external mode for cores. See later discussion on logictest single-mode or dual mode usage—see [tessent_constrain_<design_name>_mentor_ltest_non_modal](#).

tessent_constrain_<design_name>_mentor_ltest_modal_lbist_shift

You can configure timing analysis for this mode to run with either `shift_clock_src` or `test_clock`. The default is `shift_clock_src`. Add the optional `test_clock` argument when calling this SDC TCL proc to analyze timing with `test_clock`.

The proc does the following:

- Adds multi-cycle path exceptions on dynamic signals from the LogicBIST controller to design scan cells and hybrid EDT blocks. These include LogicBIST mode scan enable, `prpg_en`, `misr_en`, and LogicBIST synchronous reset for the hardware default mode of operation.
- Adds case analysis on the mux, which chooses between the top-level scan enable for ATPG and the LogicBIST controller-generated scan enable to enable only the LogicBIST mode paths.
- Adds case analysis to set `lbist_en` to active.
- Disables single chain mode scan chain concatenation paths.
- Disables paths from `edt_chain_mask` masking registers to the scan cells. This constraint is required because even though this register is configured using `tck` as the source, the clock net connected to the `edt_lbist_clock` signal carries both `edt_clock` and `shift_clock_src` clocks.

tessent_constrain_<design_name>_mentor_ltest_modal_lbist_capture

This mode is similar to `tessent_constrain_<design_name>_mentor_ltest_modal_lbist_shift`, except that the LogicBIST mode scan enable is constrained to off.

tessent_constrain_<design_name>_mentor_ltest_modal_lbist_setup

This mode propagates `tck` through the JTAG network within the LogicBIST controller and hybrid EDT blocks to time the LogicBIST test_setup paths that initialize registers such as PRPG and `edt_chain_mask`, as well as test_end paths that read the MISR signature.

This mode does not propagate tck to the design scan cells because it is only intended to check the IJTAG network paths.

tessent_constrain_<design_name>_mentor_ltest_modal_lbist_single_chain

This mode is available when single chain mode logic that enables LogicBIST diagnosis is enabled during IP generation. This mode propagates tck through the IJTAG network paths and design scan cells, including the concatenation of the internal scan chains for single-chain shifting. The LogicBIST scan enable is constrained to 1 to time only the shift paths in the design with the tck signal.

tessent_constrain_<design_name>_mentor_ltest_modal_lbist_controller_chain

This mode is available when controller chain mode (CCM) logic is enabled during IP generation. When CCM is present, all of the previously described LogicBIST modes disable the controller chain logic by constraining the ccm_en signal to off. This mode tests the CCM paths by constraining the ccm_en signal to on. The clock for this mode is either tck or test_clock, as specified during IP creation.

This mode only tests the LogicBIST and hybrid EDT controller blocks; clocks are not propagated to design scan cells.

Paths that are normally disabled during the LogicBIST operation modes described above become valid single-cycle paths in CCM and are timed accordingly.

- Signals such as lbist_en are not constrained to 1 because paths from the TDR register in the LogicBIST controller to the hybrid EDT blocks that use this signal are tested with ATPG in CCM. Paths from static signals—such as x_bounding_en, test_point_en, and capture_per_cycle—to the design scan cells are correctly excluded because no clocks are propagated to the scan cells.
- Paths from dynamic control signals that are declared as multi-cycle for other modes—such as scan_en, prpg_en, and misr_en—that go from the LogicBIST controller to the EDT blocks are tested as valid single-cycle paths.
- Paths from IJTAG network nodes such as SIBs are timed as valid single-cycle paths of the CCM clock.
- Input and output pin delays from top-level ports to EDT blocks and CCM scan IO ports are included for this mode. This differs from the LogicBIST shift and capture modes, which do not have any active paths from or to design ports.

tessent_constrain_<design_name>_mentor_ltest_non_modal

This procedure provides SDC timing constraints to add to your combined functional-dft non-modal timing scripts for one-pass synthesis or layout. Its contents is merged with your both your

functional constraints and all other Tessent controller's non-modal constraints. It is called by the umbrella proc “tessent_constrain_<design_name>_non_modal” proc.

The procedure calls the previously described procs:

- tessent_constrain_<design_name>_mentor_ltest_create_clocks
- tessent_constrain_<design_name>_mentor_ltest_set_pin_delays

It also:

- Defines a specific clock group (using 'set_clock_groups') for all logictest slow clocks, isolating them from your declared functional clocks.
- Adds 'set_false_path' constraints from each of your primary ports assigned to a static dft signals such as 'all_test', 'ltest_en' 'edt_mode'. Note that if the same signals come instead from an internal IJAG Test Data Register (TDR), then the “set_clock_groups” command between tessent_tck and the scan clocks replaces the individual set_false_path commands below.
- Provides constraints to prevent “tck” and your functional clocks to propagate to unwanted paths inside your mentor OCCs, as well as constraints that prevent false timing violations on the select pin of clock muxes inside that OCC.
- Adds an optional set_multicycle_path constraint from your primary 'scan_en' port and “edt_update” signal, based on your setting of the global variables:
 - tessent_scan_en_hold_extra_cycles
 - tessent_scan_en_setup_extra_cycles
 - tessent_edt_update_hold_extra_cycles
 - tessent_edt_update_setup_extra_cycles

which all default to zero, meaning single-cycle paths of slow clock.

- When using LogicBIST, includes a 3-to-1 shift_clock_select mux at the clock structure root of the LogicBIST controller. This mux allows any one of three clocks—shift_clock_src, test_clock or tck—to be used as the source clock for LogicBIST test.
 - Blocks tck propagation through the shift_clock_select mux into the design scan cells. This removes analysis of the slowest LogicBIST mode of operation using tck to reduce timing analysis run time.
 - Propagates both shift_clock_src and test_clock to the design scan cells, but all interactions between them are blocked.

- Adds the following LogicBIST-related exceptions:
 - Declares multi-cycle paths from LogicBIST scan enable generation logic to the design scan cells. The number of cycles are based on the `pre_post_shift_dead_cycles` IP generation parameter and defaults to 8.
 - Declares multi-cycle paths from logic that generates the `prpg_en`, `misr_en` and LogicBIST synchronous reset for hardware default mode operation signals to the hybrid EDT blocks. The number of cycles are based on the `pre_post_shift_dead_cycles` IP generation parameter.
 - Disables paths from static control registers in the LogicBIST controller—such as `lbist_en` and `x_bounding_en`—to design scan cells and hybrid EDT blocks. This is implicitly performed by declaring `tck` as asynchronous to other clocks in the IJTAG non-modal proc. When CCM is implemented with the EDT clock as the CCM clock, explicit false paths are added to disable such paths.
- Excludes single chain mode logic scan chain concatenation paths through case analysis on the single bypass chain control TDR output.
- When CCM is implemented with `test_clock` as the CCM clock, the `test_clock` propagates to all IJTAG scan elements on the `tck` domain. Constraints are added to disable such paths to the `tessent_lbist_shift_clock_src` clock.

Single-Mode vs Dual-Mode Constraining in Synthesis/Layout

Important: As it stands in the current release, leaving the `scan_enable` signal free to toggle in this proc is known to create false `shift_clock`-frequency timing paths across your functional design, which may or may not affect timing closure or quality of results in layout. Mentor customer's experience greatly varies on this front. Here are some examples of such false paths:

1. Scan chains intra-domain shift-only paths are constrained at the speed of the functional clock.
2. As a result of creating only one `shift_capture_clock` source and letting it propagate to all functional clock domains, all cross-domain capture paths become single-cycle of the shift clock, regardless of whether they are asynchronous in functional mode or not.

This said, these new timing paths do not instantly condemn your layout or physical synthesis tool to fail timing closure or perform a bad P&R job, knowing that clock tree synthesis also balances the `test_clock` fanout by default. Non-physical synthesis is generally not a problem.

On the other hand, re-constraining of all those bogus timing paths would typically require a very large number of design-dependent timing exceptions, which [extract_sdc](#) is not able to provide at this point. Applying them could also slow down some layout tools considerably.

The alternative to single-mode constraining is to apply individual mode scripts at different times in the synthesis or layout tool. It is a well-known solution that has been applied by most Mentor customers historically.

1. Functional/Dft mode:
 - Apply your functional constraints
 - Invoke `proc constrain_<design_name>_non_modal`
 - Invoke `proc tessent_constrain_<design_name>_mentor_ltest_disable`
2. Logictest-only mode, covering all EDT scan configurations, such as EDT shift and EDT bypass shift:
 - Invoke `proc tessent_constrain_<design_name>_mentor_ltest_modal_shift`
3. If your design contains hybrid EDT/LBIST, another logictest-only mode to cover the LBIST shift configuration:
 - Invoke `proc tessent_constrain_<design_name>_mentor_ltest_modal_lbist_shift`

This script cannot be combined with EDT scan configuration script because these modes are not compatible; they propagate different shift clocks, have different case analysis constraints on the `lbist_en` signal, and have different timing requirements for scan enable with respect to the shift clock.

tessent_constrain_<design_name>_mentor_ltest_set_pin_delays

This procedure assigns the clock “`tessent_virtual_slow_clock`” to your top-level ports that directly interface with your EDT controllers or your scan chains during scan or EDT mode, using the “`set_input_delay`” and “`set_output_delay`” timing constraints.

This proc is called by every SDC proc which propagate the edt and/or shift clocks, that is:

- `tessent_constrain_<design_name>_mentor_ltest_non_modal`
- `tessent_constrain_<design_name>_mentor_ltest_shift`
- `tessent_constrain_<design_name>_mentor_ltest_modal_slow_capture`

Input/Output Pin Delay Constraints

Input and output delay constraints are declared for the EDT control and channel pins. For example:

```
# channel_input:
set_input_delay $tessent_scan_input_delay \
                -clock tessent_virtual_slow_clock \
                [get_ports {my_edt_channels_in[0]}]

# channel_output:
set_output_delay $tessent_scan_output_delay \
                -clock tessent_virtual_slow_clock \
                [get_ports {my_edt_channels_out[0]}]

# edt_update:
set_input_delay $tessent_scan_input_delay \
                -clock tessent_virtual_slow_clock \
                [get_ports edt_update]
```

DFT Signals Handling in ltest STA Procs

All EDT-related modal STA procs below force the following dft signals, when present, to their active value:

- all_test (active in all tests)
- ltest_en (active in logictest only)

For example:

```
set_case_analysis 1 [get_ports all_test]
set_case_analysis 1 [get_ports ltest_en]
```

The same procs will leave all other logictest-related dft signals toggling, and will add a false_path constraint if they come from a primary port. For example:

```
set_false_path -from [get_ports async_set_reset_static_disable]
set_false_path -from [get_ports control_test_point_en]
set_false_path -from [get_ports ext_ltest_en]
set_false_path -from [get_ports ext_mode]
```

Note that if the same signals come instead from an internal IJAG Test Data Register (TDR), then the “set_clock_groups” command between tessent_tck and the scan clocks replaces the individual set_false_path commands.

Procedure “tessent_constrain_<module_name>_mentor_ltest_edt_fast_capture”, on the other hand, will optionally assert some of these dft_signals, under control of a user-declared variable.

MemoryBIST Instrument

Tessent MemoryBIST provides the following procs:

- `tessent_constrain_<design_name>_mentor_memory_bist_non_modal` for chip synthesis
 - This proc would typically not be called in your synthesis script, it is called as part of `tessent_constrain_<design_name>_non_modal`.
- `tessent_constrain_<design_name>_mentor_memory_bist_modal` for STA
 - This proc must be called in your STA script if you want to constrain the MemoryBist mode.

- `tessent_<design_name>_top_set_dft_signals logic_off`

Specify this call if your design also feature logictest-based dft signals that need to be turned off in membist STA mode.

- `tessent_mbist_set_ai_timing_mode`

This procedure assures all functional clocks are defined and properly propagated, but leaves the asynchronous interface free of constraints so they can be formally analyzed by procedure `tessent_mbist_report_controller_ai_timing`.

- `tessent_mbist_report_ai_timing [-verbose 1|0] [-tck_period time_in_ns]`

This procedure runs `tessent_mbist_report_controller_ai_timing` once for every MemoryBIST controller in the current design.

- `tessent_mbist_report_controller_ai_timing [-verbose 1|0] [-controller_id id] [-tck_period time_in_ns]`

This procedure accurately measures the BAP AI timing paths to the specified MemoryBIST controller. The covered AI signals are: BIST_SHIFT, BIST_SI, BIST_SO and BIST_HOLD.

Due to the asynchronous nature of this interface, the signals listed above cannot be accurately timed with simple SDC constraints. Although the circuit is designed to always work with a low TCK frequency, this procedure validates that it will work at the frequency you specified.

Each AI signal timing margin depends on a combination of the TCK period, the controller's BIST_CLK period, and the skew related to the TCK branch reaching that AI TCK transition detector. We recommend running this procedure only once with your worst case operating conditions. In most cases, AI signal timing is expected to largely exceed their setup timing requirements. Hold timing is never a problem due to the design of the interface.

They contain many multicycle path constraints to and from controller registers aiming to minimize the impact of the MemoryBIST DFT on your timing closure.

Multi-Clock Memories

Additional timing exceptions were added for synthesis if you have multi-port memories functionally driven by multiple clocks. A mux is inserted in the memory interface to allow both clock ports of the memory to be driven using a single clock during the Memory BIST mode. The insertion of this clock multiplexer adds different timing modes between functional and test modes which must be described correctly in the SDC file.

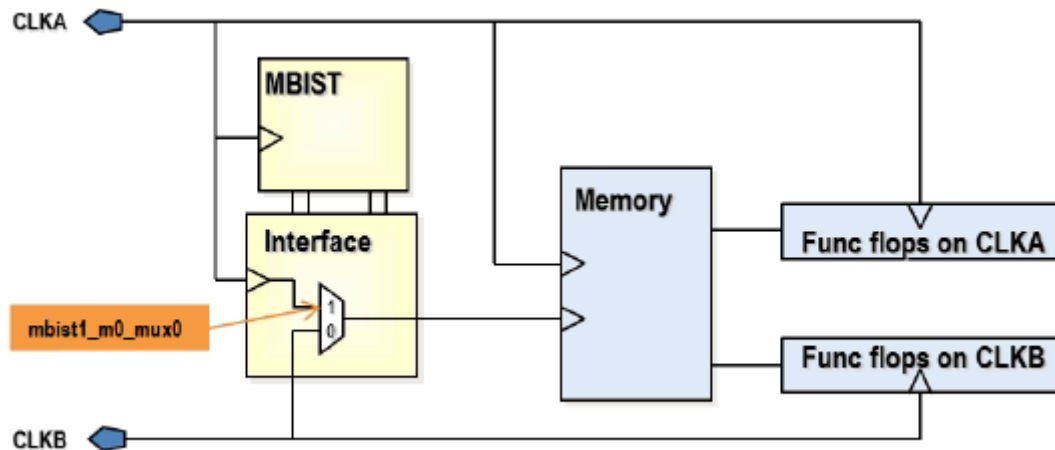
Note

 If you wish to skip the memoryBIST constraints of clock muxing (creating the generated clock and false paths) for multi-clock memories, set the variable `tessent_apply_mbist_mux_constraints` to “0” in the `tessent_set_default_variable` proc. This variable should be set to “0” in two cases:

- For synthesis: When the BIST clock used by the MemoryBIST tests is declared as false path with the memory’s functional clock in the user’s SDC script.
 - For STA: When verifying the timing of the scan modes.
-

In the following, we explain key points of the MemoryBist constraints to handle these modes harmoniously. Suppose the situation where CLKA and CLKB are two of your functional clocks that drive two clock ports of a single memory. The MemoryBIST logic will be inserted and will run on CLKA. CLKA will be multiplexed onto the CLKB branch of the memory during tests.

Figure 11-2. Generated Clock Muxed Multi-Clock Memories



First, we need to create a generated clock on the input of the clock mux. In our example, this clock is called `mbist1_m0_mux0`.

```
create_generated_clock [tessent_get_pins
$tessent_memory_bist_mapping(mbist1_m3)/tessent_persistent_cell_MUX1/b] \
-name mbist1_m0_mux0 \
-source [get_ports CLK_A] \
-add -master_clock $tessent_clock_mapping(CLK_A) \
-divide_by 1
```

Creating this clock at the input of the clock mux (instead of the output) allows all input clocks to propagate through the mux and times the following interactions precisely:

- Timing paths between the memory BIST logic and the memory
- Timing paths between the functional flops and the memory

With the mbist1_m0_mux0 defined, we can now define clock-based exceptions to declare as false any interaction between CLKB and the memory BIST flops, and between mbist1_m0_mux0 and the functional flops on CLKB. Those false paths are described using the following SDC commands:

```
set nonMBISTclocks [remove_from_collection [all_clocks] [get_clocks
"$tessent_clock_mapping(CLK_A) mbist1_m0_mux0 "]]
set_false_path -from [get_clocks mbist1_m0_mux0] \
-to $nonMBISTclocks
set_false_path -from $nonMBISTclocks \
-to [get_clocks mbist1_m0_mux0]
set_false_path -from [get_clocks $tessent_clock_mapping(CLK_B)] \
-to [tessent_get_cells [concat \
$tessent_memory_bist_mapping(mbist1_m3)/SCAN_OBS_FLOPS* \
$tessent_memory_bist_mapping(mbist1_m3)/MBISTPG_STATUS/GO_ID_REG* \
$tessent_memory_bist_mapping(mbist1_m3)/FREEZE_STOP_ERROR*]]
set_false_path -from [tessent_get_cells [concat \
$tessent_memory_bist_mapping(mbist1_m3)/Q1_SCAN_IN* \
$tessent_memory_bist_mapping(mbist1_m3)/Q2_SCAN_IN* \
$tessent_memory_bist_mapping(mbist1_m3)/BIST_INPUT_SELECT*]] \
-to [get_clocks $tessent_clock_mapping(CLK_B)]
set_false_path -from [tessent_get_cells [concat \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_PORT_COUNTER/PORT_COUNTER* \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_FSM/STATE* \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_POINTER_CNTRL/
EXECUTE_OP_SELECT_CMD** \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_SIGNAL_GEN/JCNT* \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_SIGNAL_GEN/OPSET_SELECT_REG* \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_DATA_GEN/WDATA_REG* \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_DATA_GEN/X_ADDR_BIT_SEL_REG* \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_DATA_GEN/Y_ADDR_BIT_SEL_REG* \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_ADD_GEN/AX_ADD_REG* \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_ADD_GEN/AY_ADD_REG* \
$tessent_memory_bist_mapping(mbist1)/MBISTPG_OPTION/BIST_EN_RETIME2* \
$tessent_memory_bist_mapping(mbist1)/MEM_SELECT_REG*]] \
-to [get_clocks $tessent_clock_mapping(CLK_B)]
```

By making these timing exceptions, we are precisely timing the memory BIST interaction with the memory using CLKA reaching the two-clock ports of the memory and the functional logic to memory interaction using the CLKB reaching the memory clock port. The added logic is precisely timed to simplify timing closure. These constraints will make physical design tools

aware of the mux on the clock port of the memory and they will place the mux in a location where it does not affect timing.

BoundaryScan Instrument

Tessent BoundaryScan provides the following proc:

- `tessent_constrain_<design_name>_mentor_jtag_bscan_non_modal`
 - This proc would typically not be called directly in your synthesis script, it is called as part of `tessent_constrain_<design_name>_non_modal`.

It creates generated clocks from each BoundaryScan interface and relaxes the timing between them.

Cell and Pin Mapping Procs

Since the Tessent SDC constraints are made to be usable pre- and post-synthesis, a certain mapping needs to be done in order to find all pins and cells with the pre-synthesis instance path.

It is done with the following procs:

- `tessent_get_pins`
- `tessent_get_cells`
- `tessent_get_flops`
- `tessent_map_to_verilog`
- `tessent_remap_vhdl_path_list`

`tessent_get_pins` and `tessent_get_cells` are wrapper procs of the original SDC commands `get_pins` and `get_cells`, they are used throughout the Tessent SDC. They both use `tessent_map_to_verilog` and `tessent_remap_vhdl_path_list` procs as needed. You can and should use the `tessent_get_pins` and `tessent_get_cells` procs (as opposed to `get_pins` and `get_cells`) in your functional SDC if you have a design with unrolled VHDL generate loops and you have problems applying your functional SDC on the RTL. See [“Dealing with Unrolled VHDL Generate for Loops”](#) on page 469 for more information.

The `tessent_get_flops` proc runs the `tessent_get_cells` proc and then filters the result collection for sequential elements. The proc uses a filtering method appropriate for the current tool.

The `tessent_map_to_verilog` proc adds wildcarding to pre-synthesis instance paths to match post-synthesis instance paths for layout and STA. For example, the instance path `core_inst/my_reg[0]` would return the pattern `core_inst/my_reg?0?`. This is to be able to match both the RTL instance path and the post-synthesis path `core_inst/my_reg_0_` even if “change_names -rules Verilog” was used. It also maps the slash (/) coming from Tessent Shell instance paths to

any user-defined “tessent_hierarchy_separator”. This proc is called by tessent_get_cells and tessent_get_pins. You should not need to call this proc directly.

The tessent_remap_vhdl_path_list proc is used when the instance path cannot be found with the simple mapping. It should only be needed when Tessent Shell has unrolled some VHDL generate loops to do uniquified insertion in the instances within them. See “[Dealing with Unrolled VHDL Generate for Loops](#)” on page 469 for more information. The proc supports finding cells/instances that have a trimmed last character, for example a closing brace or a question mark. This proc is called by tessent_get_cells and tessent_get_pins if needed. You should not need to call this proc directly. You can also use the proc in the unlikely event that your synthesis tool changed the names of cells in an unexpected way such as trimming any trailing closing brace, i.e. “my_reg[0]” changed to “my_reg_0” instead of the expected “my_reg_0_”

If you require additional mapping capabilities, the tessent_map_to_verilog proc provides a method for custom mapping of regular expressions/substitutions that run prior to the tool’s mapping process. To use this, you must add your own mapping by defining the global array variable.

For example:

```
global tessent_custom_mapping_regsub
array set tessent_custom_mapping_regsub {
  {\| (/| \| $) } {\| }
}
```

This example maps all RTL instance paths from the SDC file to remove any closing bracket preceding a hierarchy separator (/) or at the end of the path (space for end of list element, \$ for end of list). You would need this mapping expression during your STA run if the synthesis tool was trimming the closing brace of registers after a change_name. For example, my_reg[0] -> my_reg_0 instead of my_reg_0_ as expected.

Synthesis Helper Procs

To be able to apply Tessent Shell SDC constraints on your post-synthesis netlist, some steps are required to preserve the boundary of certain instruments as well as to preserve certain “persistent” cells, which must not be optimized away.

Three procs are provided to return the design instances which need to be boundary preserved, shall be optimized or must be entirely preserved:

- tessent_get_preserve_instances *preservation_intent*
- tessent_get_optimize_instances
- tessent_get_size_only_instances

`tessent_get_preserve_instances` *preservation_intent* returns a collection of instances whose boundaries must be preserved. It must be provided an argument to establish the future use of the netlist and determine which instances need to be preserved. The “select” value you need to choose depends on if you intend on using your post-synthesis netlist with our tools. Here are the allowed values of the “select” argument, they are in increasing order of inclusiveness:

- [add_core_instances](#)

This usage selection returns instances that need to be preserved for applying SDC constraints for STA to your post-synthesis design, and instances required for the TCD automation flow with ATPG.

- `scan_insertion`

This usage selection returns all instances of the previous selection, plus any instance of modules having existing scan segments described in a [TCD scan file](#) and non-scan instances (ICL attribute `keep_active_during_scan_test=true`). This selection is required if you intend to do scan insertion on your design with Tessent Shell or a third party tool.

- `icl_extract`

This usage selection returns all instances of the two previous selections, plus any instance of modules providing an ICL definition. This context is needed if you intend to go through the DftSpecification flow again, this time with your gate level netlist. ICL extraction requires that all ICL instances be present and traceable in the design.

The `tessent_get_optimize_instances` proc returns a collection of child instances within Tessent instruments whose boundaries are allowed to be optimized. This should be used when your synthesis tools propagates boundary optimization attributes downward hierarchically.

The `tessent_get_size_only_instances` proc returns a collection of instances of all persistent cells. These cells are required to be intact to apply SDC constraints for layout and STA. However, the synthesis tool is allowed to change the size of the driving stage as needed.

Appendix A

The Tessent Tcl Interface

The Tessent Shell tool provides a Tcl-based command interface.

General Tcl Guidelines in Tessent Shell	491
Guidelines for Modifying Existing Dofiles for Use with Tcl	493
Special Tcl Characters.....	495
Custom Tcl Packages in Tessent Shell.....	497
Tcl Resources	498

General Tcl Guidelines in Tessent Shell

If Tcl procedures are available in separate files, you can source these files from within the tool or dofiles. You can also place Tcl procedures in a *.tool_startup* file so that they are available for use. Tcl file input/output is also supported.

You should be aware of the following guidelines and behaviors when using Tcl in Tessent Shell:

- You can use Tcl variables interchangeably with legacy Tessent tool variables and environment variables.
- You should use Tcl syntax for setting and referencing variables, including using `$env(ENVARNAME)` for accessing environment variables.

For example, if the value of environment variable “foo” equals “mode1” (`$foo = mode1`), you can compare this value to a Tcl variable value using the following syntax:

```
set bar mode2
if {$env(foo) == $bar} {puts Match} else {puts "No match"}
```

Note



Use Tcl namespaces to avoid creating procedures that conflict with existing tool or Tcl commands.

- When processing comments within a dofile, the tool does not write comments preceded with “//” characters to the transcript. However, the tool does write comments that are preceded with a pound sign (#) (the Tcl comment delimiter) to the transcript.
- If a variable can contain a command string or be empty, attempting to execute by referencing `$variable` results in an error if the variable is empty.

- When a tool command error occurs, the command interpreter prints the error message and does not return anything.
- You can use the [catch_output](#) command to issue a specified tool command line and prevent command errors from aborting an enclosing dofile or Tcl proc. The `catch_output` command can optionally capture the output of a command or the returned result.
- When a command error occurs nested inside a Tcl construct or Tcl proc, you can obtain additional information about the error by issuing the following command:

> set errorInfo

This command prints the value of the `$errorInfo` Tcl variable, which may contain a traceback of the nested Tcl calls so that you can determine the root cause of the error.

Difference Between the Dofile Command and the Tcl Source Command

You normally use the tool's [dofile](#) command to execute a file of tool commands. The Tcl “source” command also executes a file of commands, but you should only use it to load Tcl procs, set Tcl variables, or do other strictly Tcl commands.

You should use the `dofile` command to execute a command file containing tool commands for the following reasons:

- The `dofile` command is affected by the [set_dofile_abort](#) command which provides you with the ability to specify whether the tool aborts when an error condition is detected. The Tcl “source” command is not affected by the `set_dofile_abort` command.
- The `dofile` command transcripts the commands to the shell and logfile. The Tcl “source” command always stops execution if any command in the file encounters an error.

Example 1

In this example, the `add_scan_chains` command defines every scan chain in the design. This command is sometimes used hundreds of times and makes the dofile very long. Using Tcl, you can shorten the dofile substantially by using a loop construct as shown here:

```
for {set xx 1} {$xx < 257} {incr xx} {  
    add_scan_chains int chain$xx group1 /top/edt_si$xx /top/edt_so$xx  
}
```

For the values of `xx` from 1 up to 256, the tool executes a separate `add_scan_chains` command for each value. The transcript and logfile will contain all 256 `add_scan_chains` commands (preceded with “// subcommand: ”).

Example 2

The following example controls the flow of creating test patterns and uses a variable to execute different commands based on the variable's value:

```
if {$mode == stuck} {
    set_fault_type stuck
    . . .
} elseif {$mode == transition} {
    set_fault_type transition
    set_pattern_type -sequential 2
    . . .
}
```

Example 3

Module hierarchies may become ungrouped through either synthesis or layout and ATPG may operate on a flattened view of the design. Automated DRCs will find and trace the EDT IP and subsequently will find and trace the flattened scan chains automatically. In the following example, we are capturing the output of the [report_scan_chains](#) command and placing it into the variable "rsc" by using the [catch_output](#) command. The foreach loop iterates on the report_scan_chains output, capturing the chain name, group name and input and output connections in the variables "chain", "group", "input", and "output," respectively. This information is written into the file *add_chains.txt*.

```
set out [open "add_chains.txt" w]
catch_output {report_scan_chains} -output rsc
set rscl [split $rsc "\n"]

foreach a $rscl {
    puts "NEWLINE =\t$a"
    regexp {.*chain = (.*)\s+group = (.*)\s+input = \'(.*)\'\s+output = \
'(.*)\'\s+.*} $a match chain group input output
    puts $out "add_scan_chains -internal $chain $group \{$input\} \{$output\
}"
}

close $out
```

Guidelines for Modifying Existing Dofiles for Use with Tcl

When using an existing dofile with the Tcl interface, you should evaluate the dofile for issues that could cause incorrect evaluation by the Tessent Tcl interpreter.

[Table A-1](#) provides guidance for correcting common dofile issues.

The most common issues you can run across with an existing dofile is accounting for Tcl special characters. For more information, refer to [Table A-2](#) on page 495, which provides a list of typically-used Tcl special characters.

Table A-1. Common Dofile Issues and Solutions

Dofile Issue	Solution
Stopping dofile execution at a specific point	Use the native Tcl “error” command to stop execution of a dofile at any point. The “error” command requires a message string which the tool outputs. But to avoid any message you can use an empty string: error ""
Escaping Tcl special characters	Use the following techniques to escape Tcl special characters: • Double quotes (" ") group tokens but allow \$variable and [command] evaluations. • Braces ({ }) also group tokens and disable \$variable and [command] evaluations. • Brackets ([]) implement command substitution and are used to nest or embed commands. • A backslash (\) escapes the next character. Use this to tame a Tcl special character such as \$, [, {, or ".
Using dollar signs in pathnames	A dollar sign (\$) specifies variable substitution. In some netlists, pathnames (for example, <i>foo/pin\$p7</i>) can contain the dollar sign. When using pathnames with dollar signs, enclose the pathname with braces ({ }) to prevent the tool from substituting the value as shown in the following example: report_gates {foo/pin\$p7}
Escaping quotation marks	In Tcl, quotation marks (" ") instruct the Tcl interpreter to treat the enclosed words as a single argument. For example: puts " Hello World " If embedded quotes are required, you must use backslashes (\) to escape the embedded double quotes. For example: puts " Hello \"World\" " Otherwise, the tool issues an error message.
Using brackets	Brackets ([]) implement command substitution and are used to nest or embed commands. A command and its arguments enclosed in square brackets is evaluated and its result inserted in place in the enclosing command.

Table A-1. Common Dofile Issues and Solutions (cont.)

Dofile Issue	Solution
Optional single quotes	Optional single quotes (' ') are no longer valid for Tessent commands. For example, the following produces an error: <pre>set_design_sources '-v MODB.v -v MODC.v'</pre> Correct this by using no quotes, double quotes, or braces: <pre>set_design_sources -v MODB.v -v MODC.v</pre> <pre>set_design_sources "-v MODB.v -v MODC.v"</pre> <pre>set_design_sources {-v MODB.v -v MODC.v}</pre>
Environment variables	You should use Tcl syntax for setting and referencing variables, including using <code>\$env(ENVARNAME)</code> for accessing environment variables. For example, if the value of environment variable “foo” equals “mode1” (<code>\$foo = mode1</code>), you can compare this value to a Tcl variable value using the following syntax: <pre>set bar mode2 if {\$env(foo) == \$bar} {puts "Match"} else {puts "No match"}</pre>

Special Tcl Characters

In Tcl scripts, you often see characters used for special purposes. For a complete list of special characters, you should consult a Tcl resource.

[Table A-2](#) lists and describes the more common special characters you can encounter when reading the examples in this manual. See the additional resources described in [“Tcl Resources”](#) on page 498.

Table A-2. Common Tcl Characters

Character	Description
;	The semicolon terminates the previous command, allowing you to place more than one command on the same line.
\	Used at the end of a line, the backslash continues a command on the following line.
\\$	The backslash with other special characters, like a dollar sign, instructs the Tcl interpreter to treat the character literally.
\n	The backslash with the letter “n” instructs the Tcl interpreter to create a new line.
\$	The dollar sign in front of a variable name instructs the Tcl interpreter to access the value stored in the variable.

Table A-2. Common Tcl Characters (cont.)

Character	Description
[]	Square brackets group a command and its arguments, instructing the Tcl interpreter to treat everything within the brackets as a single syntactical object. You use square brackets to write nested commands. For example: set chain_report [report_scan_chains -subchains -verbose]
{ }	Curly braces instruct the Tcl interpreter to treat the enclosed words as a single string. The Tcl interpreter accepts the string as is, without performing any variable substitution or evaluation. For example, to create a string that contains special characters such as \$ or \: set my_string {This book costs \$25.98.}
" "	Quotes instruct the Tcl interpreter to treat the enclosed words as a single string. However, when the Tcl interpreter encounters variables or commands within string in quotes, it evaluates the variables and commands to generate a string. For example, to create a string that displays a final cost calculated by adding two numbers: set my_string "This book costs \[\$expr \$price + \$tax]"

Table A-2. Common Tcl Characters (cont.)

Character	Description
#	The pound sign (#) indicates a comment and directs the Tcl compiler to not evaluate the rest of the line. When using the pound sign, you must use it where a command starts, and at the beginning of a command, not within a command. • Tricky Point 1: Evaluate does not equal parse. Despite the pound sign, the comment below gives an error because Tcl detects an open lexical clause. <pre># if (some condition) { if { new text condition } { ... }</pre> • Tricky Point 2: The apparent beginning of a line is not always the beginning of a command: <pre># This is a comment</pre> In the following code snippet, the line beginning with “# -type” is not a comment, because the line right above it has a line continuation character (\). In fact, the “#” confuses the Tcl interpreter, resulting in an error when it attempts to create the tk_messageBox. <pre>tk_messageBox -message "The diagnosis report was successfully written." \ # -type ok</pre> # and this is also a comment. This one will span \ multiple lines, even without the # at the beginning \ of the second and third lines. In general, it is good practice to begin all comment lines with a #. • Tricky Point 3: The beginning of a command is not always at the beginning of a line: Usually, you begin new commands at the beginning of a line. That is, the first non-space character is the first character of the command name. However, you can combine multiple commands into one line using the semicolon “;” to designate the end of the previous command: <pre>set myname "John Doe" ; set this_string "next command" set yourname "Ted Smith" ; # this is a comment</pre>

Custom Tcl Packages in Tessent Shell

Tessent Shell supports several standard techniques for adding your own Tcl packages with the Tcl “package require” command, which you can issue from Tessent Shell.

You can use any of the following ways to specify directory locations of Tcl packages. Any directory that contains a Tcl package must also contain a *pkgIndex.tcl* file within its hierarchy. The following methods are listed in order of the precedence in effect if there is more than one package with the same name:

- Default location for Tessent Shell Tcl packages. You can place your package underneath a directory named *tessent_plugin/packages* that is located at the top of your Tessent install tree. When Tessent Shell finds a package in this directory upon invocation, it appends the directory path to *tessent_plugin/packages* to the *auto_path* Tcl global variable, if it exists.
- `TESENT_PLUGIN_PATH` environment variable. You can set this variable to a colon-separated list of directory paths that contain Tcl packages in a *packages* subdirectory. When Tessent Shell finds a package, it appends the directory path to *packages* to the *auto_path* Tcl global variable, if it exists.
- *auto_path* Tcl global variable. You can issue the following command in Tessent Shell to specify a package location:

```
> lappend auto_path pathToTclPackageDir
```

- `TCLLIBPATH` environment variable. You can set this variable to a space-separated list of directory paths that contain Tcl packages. Note that a *packages* subdirectory is not required under any of the paths.

Note

Tessent Shell ignores the `TCL_LIBRARY` environment variable.

Tcl Resources

The following website is a place to start in your search for the reference material that works best for you. It is not an endorsement of any book or website.

<http://www.tcl.tk/>

Appendix B

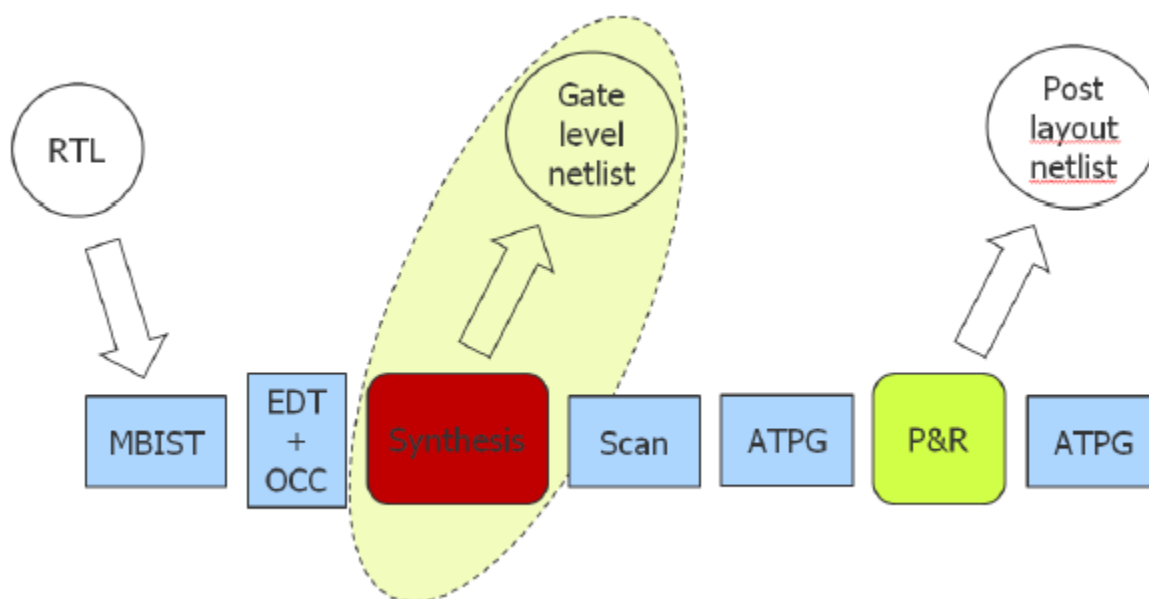
Synthesis Guidelines for RTL Designs with Tessent Inserted DFT

This appendix provides guidelines for you to use when synthesizing your design with Tessent created IP RTL with a third-party synthesis tool, specifically Synopsys DC Ultra™.

In certain cases, you may experience problems with synthesis optimization using DC Ultra that manifests itself by propagating constants even if the boundary optimization is false; this results in optimizing away some of the DFT logic that is not connected until scan insertion, but keeping the ports. This creates a situation where some of the necessary Tessent logic is eliminated during optimization, which breaks the design.

Figure B-1 illustrates where in the design flow this problem occurs.

Figure B-1. Problem Occurrence Creating the Gate Level Netlist



This problem does not appear to be present with the basic DC compile command; it seems to only be an issue when using the compile_ultra command in certain tool versions.

DFT Insertion with Tessent	500
Synthesis Guidelines	501

DFT Insertion with Tessent

When the Tessent tools create the IP logic to add to the design, the logic includes some persistent instances that are used as anchor points for items such as clocks, control signals, constraints, and so on. For automation, the tool uses modules and port matching, and, consequently, requires that some specific modules be preserved after synthesis and place and route.

There are two types of persistent instances to be aware of for synthesis:

- Cells
 - Requires the use of a `set_size_only` command.
- Design Modules
 - Requires `ungroup` to be set to off.
 - Requires boundary optimization to be set to off.

Boundaries of all instances of modules with Tessent Core Descriptions (TCDs) need to be preserved, which means the following:

- No boundary optimization.
- No ungrouping.
- No new ports.
- No logic optimization.

The logic test module types that require this include EDT, LBIST, OCC, STI SIB, Bscan interface, and any module with `tcd_scan` (pre-existing scan chain). ICL extraction uses modules and ports of ICL modules so they also need to be preserved. Note that it is allowed to not preserve the IJTAG instance boundaries, which allows the synthesis tool to obtain a better optimization result. You can use the ICL file from RTL in post synthesis and place and route.

The boundary optimization option during synthesis is global in the sense that it applies to every design hierarchy on and below the specified module or instance. Hence, if you want to globally optimize your design, you'd specify it on the root (top) module and then boundary optimization will run through the entire design doing its work (assuming that there are no “don't touch” cores, which were already synthesized earlier). The problem is that valid constrained or not (yet) connected ports or nets get optimized away. To prevent this, you have to tell the DC tool not to apply this boundary optimization to selected modules or instances, like those Tessent created IP pieces previously listed above.

- DC has the following two modes:
 - Basic compile (using the *compile* command):
 - Will not cause any problem as the tool does not ungroup and optimize boundaries.
 - Constants are not optimized away.
 - Optimization On (using *compile_ultra* command):
 - The synthesis tool will do its best to remove any dead logic (either due to constraints propagation or missing loads or unused).
 - Starting with 2013.12, DC propagates constraints even if boundary optimization is set false for design instances.

As this issue is becoming more prevalent, there is an informational message from DC Ultra when using the *compile_ultra* command that reports the following:

```
Information: Starting from 2013.12 release, constant propagation is
enabled even when boundary optimization is disabled. (OPT-1318)
```

If you receive this message, you may not fully understand the ramifications or how to correctly synthesize the Tessent IP in this environment. If not properly handled, you may see additional information messages from the tool similar to the following:

```
Information: Removing unused design 'corea_rtl_tessent_occ_shift_reg_1'.
(OPT-1055)
Information: The register
'corea_rtl_tessent_occ_clk_inst/occ_control/scan_out_reg' is a constant
and will be removed. (OPT-1206)
Information: Ungrouping hierarchy tessent_persistent_cell_edt_clock
before Pass 1 (OPT-776)
```

Synthesis Guidelines

When using DC Ultra, there are two key commands to be aware of to synthesize correctly with the Tessent IP.

The first is to turn off constant propagation with no boundary optimization by setting an application variable. This variable works globally and is used with the following syntax:

```
set_app_var compile_enable_constant_propagation_with_no_boundary_opt false
```

The second key command for instance level control is to use the following compile directive before using the *compile_ultra* command to turn off the constant propagation for cells:

```
set_compile_directives -constant_propagation false [get_cells <hier-cell-name>]
```

Or for pin-level granularity, use the following command:

```
set_compile_directives -constant_propagation false [get_pins <hier-pin-name>]
```

Other means of restricting the synthesis tool from changing and optimizing instances and levels of hierarchy include the *set_size_only* command and ungrouping. When *set_size_only* is used for a specified list of leaf cells, the tool can only change their drive strength, or sizing. The *ungroup*, *group*, *set_ungroup*, and *uniquify* commands can be used to control how the tool removes (or not) levels of hierarchy and how reused blocks are uniquified during synthesis. The *dont_touch* attribute is also effective for adding to designs, sub designs, and cells to prevent them from being ungrouped during optimization.

Different guidelines apply to the type of flow you are using, whether a bottom-up synthesis flow, or a top-down approach. Here are the main steps to use for incorporating Tessent RTL IP with the design data using DC Ultra.

Bottom-Up Flow Using DC Ultra

- Read full design in synthesis tool.
- Set current design to each DFT IP and compile.
- Set don't touch or set size only to all the DFT IP.
- Set current design TOP.
- Read functional SDC.
- Read DFT SDC.
- Set ungroup false to all DFT IP and persistent modules.
- Set boundary optimization false to all DFT IP and persistent modules.
- Set size only to all persistent cells.
- Disable constant propagation during boundary optimization.
- Compile ultra.

Top-Down Flow Using DC Ultra

- Read full design in synthesis tool.
- Read functional SDC.
- Read DFT SDC.
- Set ungroup false to all DFT IP and persistent modules.
- Set boundary optimization false to all DFT IP and persistent modules.
- Set size only to all persistent cells.

- Disable constant propagation during boundary optimization.
- Compile ultra.

Appendix C

Example Scripts using Tessent Tool-Generated SDC

This section provides example scripts for use when using Tessent Shell tool-generated SDC.

Example Design Compiler Synthesis Script	505
Example Encounter Synthesis Script.....	507
Example PrimeTime STA Script	510

Example Design Compiler Synthesis Script

The following is an example synthesis script to use with Design Compiler.

```
# Preparation Step 1: Sourcing SDC File
set design_name top
set tsdb ../tsdb_outdir

source \
${tsdb}/dft_inserted_designs/${design_name}_rtl.dft_inserted_design/ \
${design_name}.sdc

# Preparation Step 2: Setting and Redefining Tessent Tcl Variables
tessent_set_default_variables
set tessent_tck_period 80.0

# Uncomment the following line if you wish to skip the memoryBIST
# constraints of clock muxing for multi-clock memories:
#set tessent_apply_mbist_mux_constraints 0

# Preparation Step 3: Verifying the Declaration of Functional Clocks
# Define functional clocks

create_clock [get_ports pCLK25] -name pCLK25 -period 25.0
create_clock [get_ports pCLK100] -name pCLK100 -period 100.0
# Set clock names in mapping array
array set tessent_clock_mapping {
    CLK25 pCLK25
    CLK_PLL_REF pCLK100
}
# Preparation Step 4: Redefining Other Tessent Tcl Variables
# set tessent_input_delay [expr 0.3 * $tessent_tck_period]

# Synthesis Step 1: Loading the Design Into Your Synthesis Tool
source ${design_name}.dc_shell_import_script
elaborate $design_name

# Synthesis Step 2: Applying the SDC Constraints
set_app_var timing_enable_multiple_clocks_per_reg true
tessent_constrain_${design_name}_non_modal
```

```
# Synthesis Step 3: Preparing the DFT Logic for Synthesis
set_app_var compile_enable_constant_propagation_with_no_boundary_opt false
set preserve_instances [tessent_get_preserve_instances icl_extract]
set_boundary_optimization $preserve_instances false
set_ungroup $preserve_instances false
set_app_var compile_seqmap_propagate_high_effort false
set_app_var compile_delete_unloaded_sequential_cells false

set_boundary_optimization [tessent_get_optimize_instances] true

set_size_only -all_instances [tessent_get_size_only_instances]

# shared bus assembly ungrouping
foreach_in_collection assembly \
  [get_designs -hierarchical *_tessent_mbist*_shared_bus_assembly] { \
    current_design $assembly \
    set_ungroup \
    [get_cells -filter "ref_name!~*_tessent_mbist*_controller*"] true \
  }
current_design $design_name

# Synthesis Step 4: Synthesizing Your Design
link
check_design
compile -boundary_optimization

# Synthesis Step 5: Writing Out Your Final SDC
write_sdc ${design_name}.merged_sdc

# Synthesis Step 6: Writing Out Your Final Netlist
write -format verilog -output ${design_name}.vg ${design_name} -hier
```

Example Encounter Synthesis Script

This section provides an example synthesis script to use with Encounter.

Example Scripts using Tessent Tool-Generated SDC

Example Encounter Synthesis Script

```
# Preparation Step 1: Sourcing SDC File
set design_name top
set tsdb ../tsdb_outdir
source
source ${tsdb}/dft_inserted_designs/${design_name}_rtl.dft_inserted_design/ \
${design_name}.sdc

# Preparation Step 2: Setting and Redefining Tessent Tcl Variables
tessent_set_default_variables
set tessent_tck_period 80.0

# Uncomment the following line if you wish to skip the memoryBIST
# constraints of clock muxing for multi-clock memories:
#set tessent_apply_mbist_mux_constraints 0

# Preparation Step 3: Verifying the Declaration of Functional Clocks
# Define functional clocks
create_clock [get_ports pCLK25] -name pCLK25 -period 25.0
create_clock [get_ports pCLK100] -name pCLK100 -period 100.0
# Set clock names in mapping array
array set tessent_clock_mapping {
    CLK25 pCLK25
    CLK_PLL_REF pCLK100
}

# Preparation Step 4: Redefining Other Tessent Tcl Variables
# set tessent_input_delay [expr 0.3 * $tessent_tck_period]

# Synthesis Step 1: Loading the Design Into Your Synthesis Tool
source ${design_name}.encounter_import_script
elaborate
cd designs/${design_name}

# Synthesis Step 2: Applying the SDC Constraints
tessent_constrain_${design_name}_non_modal
```

```
# Synthesis Step 3: Preparing the DFT Logic for Synthesis
proc do_set_boundary_optimization { instance_collection state } {
    set C {}
    # Find sub-instances because 'boundary_opto' attribute is not inherited
    # downward (unlike 'dc_shell')
    set instancesCollection {}
    foreach_in_collection inst $instance_collection {
        if { [sizeof_collection [set subInstancesCollection [find . -instance \
            [vname $inst]]]]> 0 } {
            append_to_collection instancesCollection $subInstancesCollection
        }
    }

    # In 'encounter', a collection is really just a list...
    set instancesCollection [lsort -unique $instancesCollection]

    # Boundary optimization can only be applied to subdesign.
    # Build a list of the subdesign names for each instance.
    foreach_in_collection inst $instancesCollection {
        if { [set moduleName [get_attribute module $inst]] eq "" } { continue }
        lappend C $moduleName
    }

    if { [llength $C] == 0 } { return }

    # Apply boundary optimization
    set C [lsort -unique $C]
    puts "Setting boundary optimization '$state' on modules:\n "
    puts [join $C "\n "]
    set_attribute boundary_opto -quiet $state $C

    # Preserve the footprint of DFT objects
    # when boundary optimization is not set.
    if { !$state } {
        puts "Preserve footprint on pins of:\n "
        puts [join $instancesCollection "\n "]
        foreach_in_collection inst $instancesCollection {
            set_attribute preserve true [find $inst -pin *]
        }
    }
    return $C
}
```

```
set preserveInstances [tessent_get_preserve_instances icl_extract]
do_set_boundary_optimization $preserveInstances false

set optimizeInstances [tessent_get_optimize_instances]
do_set_boundary_optimization $optimizeInstances true

set set_size_only_collection [tessent_get_size_only_instances]
if { [sizeof_collection $set_size_only_collection] != 0 } {
    foreach {preserveValue isSequential} [list map_size_ok true size_ok false] {
        set instancesListFiltered [filter sequential $isSequential
$set_size_only_collection]
        if { [llength $instancesListFiltered] == 0 } { continue }
        puts "Setting preserve attribute '$preserveValue' to '$isSequential' on:\n "
        puts [join [lsort $instancesListFiltered] "\n "]
        set_attribute preserve $preserveValue $instancesListFiltered
    }
}
# Ungroup shared bus assemblies
set shared_bus_assembly_instances \
    [filter subdesign *_tessent_mbist_*_shared_bus_assembly \
    [find . -instance *]]
set shared_bus_ungroup_instances [list]
foreach assembly_inst $shared_bus_assembly_instances {\
    set shared_bus_ungroup_instances \
        [concat $shared_bus_ungroup_instances \
            [filter -invert subdesign *_tessent_mbist_*_controller* \
                [find $assembly_inst -instance -maxdepth 2 *]]] \
}
if { [llength $shared_bus_ungroup_instances] > 0 } { \
    ungroup $shared_bus_ungroup_instances \
}
# Synthesis Step 4: Synthesizing Your Design
synthesize -to_mapped

# Synthesis Step 5: Writing Out Your Final SDC
redirect ${design_name}.merged_sdc { write_sdc $design_name }

# Synthesis Step 6: Writing Out Your Final Netlist
redirect ${design_name}.vg { write_hdl $design_name }
```

Example PrimeTime STA Script

The following section provides an example STA script to use with PrimeTime.

```
# RunAndReport
#
proc RunAndReport { mode } {
    update_timing
    redirect violations_${mode}.report {report_constraint -all_violators}
}
set tsdb ./tsdb_outdir
set design_name top
set design_id rtl
set netlist ${design_name}.vg
# Allow asserts on flop reset pins to propagate to their Q output.
set case_analysis_sequential_propagation always

# Disable timing through the enable side of clock gating cells,
# so as to prevent clock reconverging paths.
set timing_clock_gating_propagate_enable false

# Source Tessent Shell generated sdc file and set default variables
source \
${tsdb}/dft_inserted_designs/${design_name}_${design_id}.dft_inserted_design \
/${design_name}.sdc
tessent_set_default_variables

# Optionally override the above TCL variables values using "set" commands.  For example:
# set tessent_slow_clock_period 20
# set tessent_tck_period 50
# array set tessent_clock_mapping {
#     tessent_tck my_tck
#     CLK3 CLK_F300
# }

# Change your hierarchy separator variable if not appropriate
set tessent_hierarchy_separator "/"

# Override data in above "tessent_set_default_variables" by creating your
# own timing data file and placing it at the same level as this script.
if {[file exists user_timing_data.tcl]} {
    source user_timing_data.tcl
}
```

Example Scripts using Tessent Tool-Generated SDC

Example PrimeTime STA Script

```
# You can provide your own path remapping procedures file
# (tessent_get_xxx). You only need to copy and edit the procedures you
# want to change and put them in file ./user_remapping_procs.tcl.
if {[file exists user_remapping_procs.tcl]} {
    source user_remapping_procs.tcl
}

# If you have fake cells or cells that are not part of your library
# you can provide this file to load them.
if {[file exists load_lib_cells.tcl]} {
    source load_lib_cells.tcl
}

# Create the script "load_design.pt" only if you need to load your design
# with a different command than the simple "read_verilog" below.
if {[file exists load_design.pt]} {
    source load_design.pt
} else {
    read_verilog $netlist
}
current_design $design_name
link_design

#####
#
#                               Mentor DFT Mode
#
#####
echo "\n\nAnalysis in Mentor DFT mode.***** \n" ;
# DFT mode covers all Mentor-inserted DFT IP (ijtag network, bscan, memoryBist, BISR),
# but NOT scan or edt mode.
reset_design
LoadBackAnnotationData

# Apply constraints for STA.
tessent_create_functional_clocks
tessent_constrain_${design_name}_mentor_ijtag_non_modal
tessent_constrain_${design_name}_mentor_memory_bist_modal
tessent_constrain_${design_name}_mentor_memory_bisr_non_modal
tessent_constrain_${design_name}_mentor_jtag_bscan_non_modal
tessent_constrain_${design_name}_mentor_ltest_disable

tessent_kill_functional_paths

RunAndReport MENTOR_DFT_MODE
```



```
#####
# Formally check your BAP Asynchronous interface timing paths
#####
echo "\n\nChecking your BAP asynchronous interface paths timing***** \n" ;
echo "See the results in output report file AITiming.report"
reset_design
tessent_set_default_variables
tessent_mbist_set_ai_timing_mode
update_timing
# set "-verbose" to 1 for a more detailed timing report
# change the "-tck_period" value for analysis of different shift speeds
redirect AITiming.report { tessent_mbist_report_ai_timing -tck_period $tessent_tck_period /
    -verbose 1 }
```

The following steps assume you have EDT IP in your design:

```
#####
#                               Combined Edt Shift Modes
#####
reset_design
LoadBackAnnotationData
tessent_constrain_${design_name}_mentor_ltest_modal_shift
set_propagated_clock [all_clocks]
RunAndReport SHIFT

#####
#                               EDT Slow CaptureMode
#####
reset_design
LoadBackAnnotationData
set tessent_time_scan_en_in_slow_capture 1
set tessent_scan_en_setup_extra_cycles 1
set tessent_scan_en_hold_extra_cycles 1
set tessent_edt_update_setup_extra_cycles 1
set tessent_edt_update_hold_extra_cycles 1
tessent_constrain_${design_name}_mentor_ltest_modal_edt_slow_capture
set_propagated_clock [all_clocks]
RunAndReport SLOW_CAPTURE

#####
#                               EDT Fast Capture Mode
#####
# Important: The following variable setting is important in order to allow at-speed coverage
# of your Mentor OCC's internal shift register and clock gaters paths with scan_enable=0:
set case_analysis_sequential_propagation never
reset_design
LoadBackAnnotationData
<load your functional constraints here, including all clock definitions and functional timing
exceptions>
set memory_bypass_en_value 1
set x_bounding_en_value 1
set observe_test_point_en_value 0
tessent_constrain_${design_name}_mentor_ltest_modal_edt_fast_capture
set_propagated_clock [all_clocks]
RunAndReport FAST_CAPTURE
```

The LogicBIST modes described below are required when using the hybrid TK/LBIST flow.

You can execute LogicBIST tests using one of three test clock sources chosen at pattern generation time: `shift_clock_src`, `test_clock`, or `TCK`. Because `TCK` is a slow clock, STA verification is not performed explicitly when using this clock source for LogicBIST; the tool verifies all paths with one of the other two clocks, except for the clock network differences prior to the LogicBIST controller.

The default clock is `shift_clock_src`, which is typically connected to an internally generated, free-running clock because this is the most suitable source for running LogicBIST in-system test.

If you generate LogicBIST patterns to run with `test_clock` as the LogicBIST clock source, specify “`test_clock`” as an argument in the following procs:

```
tessent_constrain_${design_name}_mentor_ltest_modal_lbist_shift test_clock
tessent_constrain_${design_name}_mentor_ltest_modal_lbist_capture test_clock
```

The tool supports the values `ltest_clock` and `edt_clock` as aliases for “`test_clock`”.

If you generate two sets of LogicBIST patterns to run LogicBIST tests, execute the LogicBIST shift mode proc described below twice: first with the `shift_clock_src` argument and second with `test_clock` argument.

For LogicBIST shift mode and LogicBIST capture mode using the `shift_clock_src` argument, create the `shift_clock_src` clock in the timing tool and indicate its name using the Tessent SDC TCL variable `tessent_lbist_shift_clock_src`, as mentioned in [“Preparation Step 2: Setting and Redefining Tessent Tcl Variables”](#) on page 460.

```
#####
#                               LogicBIST Shift Mode
#####
reset_design
LoadBackAnnotationData
tessent_constrain_${design_name}_mentor_ltest_modal_lbist_shift
set_propagated_clock [all_clocks]
RunAndReport LBIST_SHIFT
```

Unlike EDT, which uses separate fast and slow capture modes, LogicBIST uses a combined capture mode. The NCPs and fault type used during fault simulation determine whether LogicBIST test targets stuck-at or transition faults. These NCPs can use single or multiple clock pulses sourced from either the LogicBIST clock or functional clock, as determined by the `NcpIndexDecoder` and `OCC` settings. Similar to the shift mode, use either one or both of `shift_clock_src` and `test_clock` arguments for analysis. The value for `observe_test_point_en` should reflect the setting used during fault simulation. Control test points are typically enabled for both stuck-at and transition tests.

```
#####
#                               LogicBIST Capture Mode
#####
reset_design
LoadBackAnnotationData
tessent_constrain_${design_name}_mentor_ltest_modal_lbist_capture
<when LogicBIST test targets transition faults, load your functional constraints here,
including all clock definitions and functional timing exceptions>
set memory_bypass_en_value 1
set x_bounding_en_value 1
# When observe_test_points are disabled for transition test:
set observe_test_point_en_value 0 ;
set_propagated_clock [all_clocks]
RunAndReport LBIST_CAPTURE
```

The following proc analyzes timing for the IJTAG network paths used to initialize the BIST controller and read the MISR values after test is complete. This mode uses TCK.

```
#####
#                               LogicBIST Setup (IJTAG Network Paths)
#####
reset_design
LoadBackAnnotationData
set_propagated_clock [all_clocks]
tessent_constrain_${design_name}_mentor_ltest_modal_lbist_setup
RunAndReport LBIST_SETUP
```

The following proc analyzes timing for the single chain-based diagnosis mode. This mode uses TCK for reading all the scan cells through the IJTAG network.

```
#####
#                               LogicBIST Single Chain Diagnosis
#####
reset_design
LoadBackAnnotationData
set_propagated_clock [all_clocks]
tessent_constrain_${design_name}_mentor_ltest_modal_lbist_single_chain
RunAndReport LBIST_SINGLE_CHAIN
```

When you implement the optional controller chain mode (CCM), the following proc analyzes timing for CCM. In this mode, the tool removes timing exceptions between the LogicBIST controller and the hybrid TK/LBIST blocks (as seen in the shift and capture modes) to reflect CCM pattern generation setup.

```
#####
#                               LogicBIST Controller Chain Mode
#####
reset_design
LoadBackAnnotationData
set_propagated_clock [all_clocks]
tessent_constrain_${design_name}_mentor_ltest_modal_lbist_controller_chain
RunAndReport LBIST_CONTROLLER_CHAIN
```


Appendix D

Clocking Architecture Examples

The clocking architecture in designs may vary, and they play a role during test planning, especially for hierarchical test.

This appendix provides examples of common clocking architectures and how to insert the on-chip clock controllers (OCCs) given those architectures. The examples assume that you are performing hierarchical test as described in “[DFT Architecture Guidelines for Hierarchical Designs](#)” on page 85”.

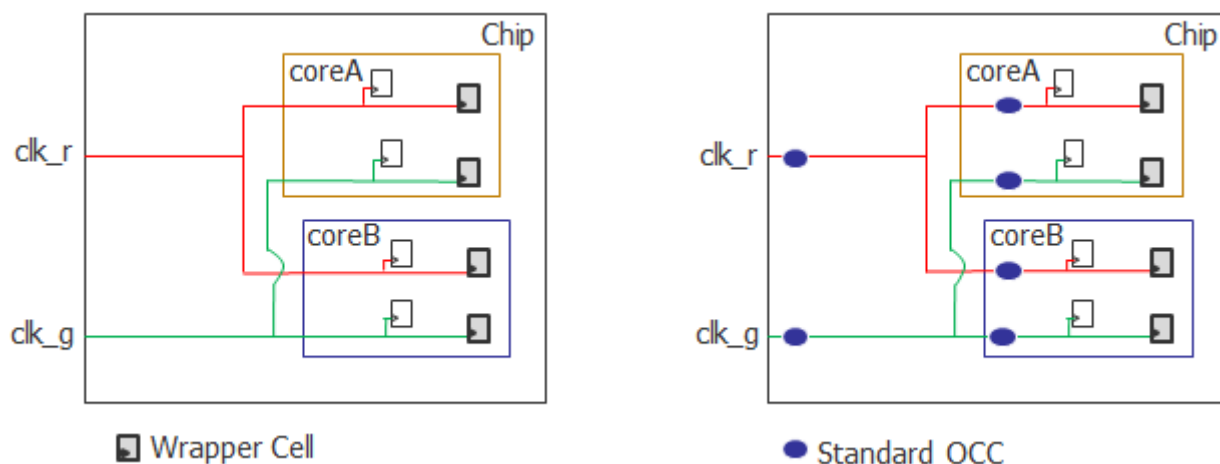
For information about the OCCs supported by Tessent, refer to “[Tessent On-Chip Clock Controller](#)” in the *Tessent Scan and ATPG User’s Manual*.

Clocks Driven by Primary Inputs	517
Clock Generators Outside the Cores	518
Clock Generators Inside the Cores	518
Clock Sourced From A Core With Embedded PLL	519
Clock Mesh Synthesis	520

Clocks Driven by Primary Inputs

In the following figure, coreA and coreB are wrapped cores, and the clk_r and clk_g clocks are asynchronous to each other. Insert standard OCCs inside coreA and coreB for both clk_r and clk_g ports. At the chip-level, insert standard OCCs on the clk_r and clk_g ports.

Figure D-1. OCCs for Clocks Driven by Primary Inputs

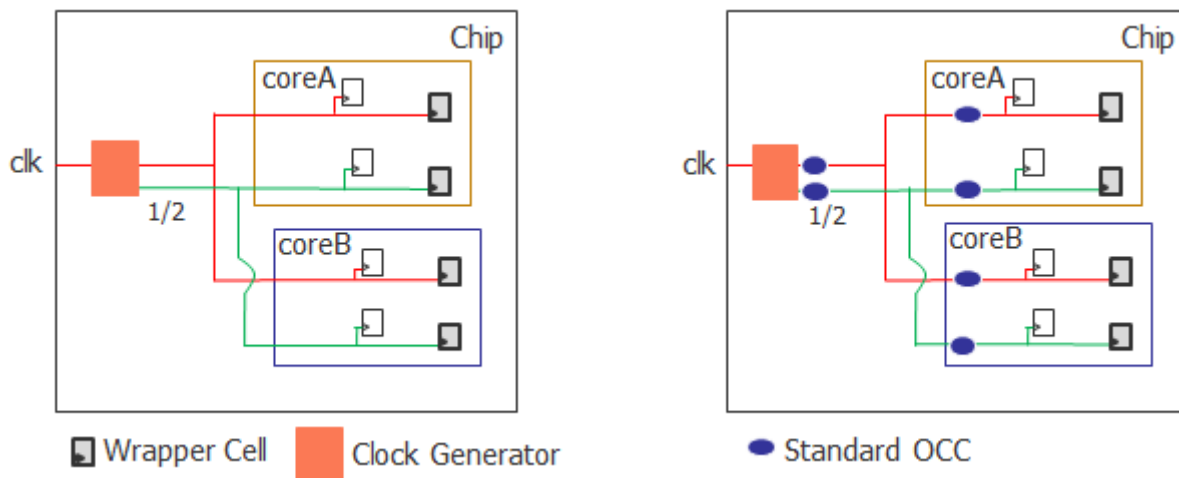


During intest mode for the wrapped cores, the OCCs inside the cores are active and the OCCs outside the cores are inactive. During extest, the OCCs outside the cores are active and the OCCs inside the cores are inactive.

Clock Generators Outside the Cores

In the following figure, coreA and coreB are wrapped cores, and a clock generator at the chip level divides the clock frequency by half to generate the green clock. The red and green clocks are asynchronous to each other. Insert standard OCCs inside coreA and coreB for both the red and green clocks. At the chip-level, insert standard OCCs on both the red and green clocks generated at the output of the clock generator.

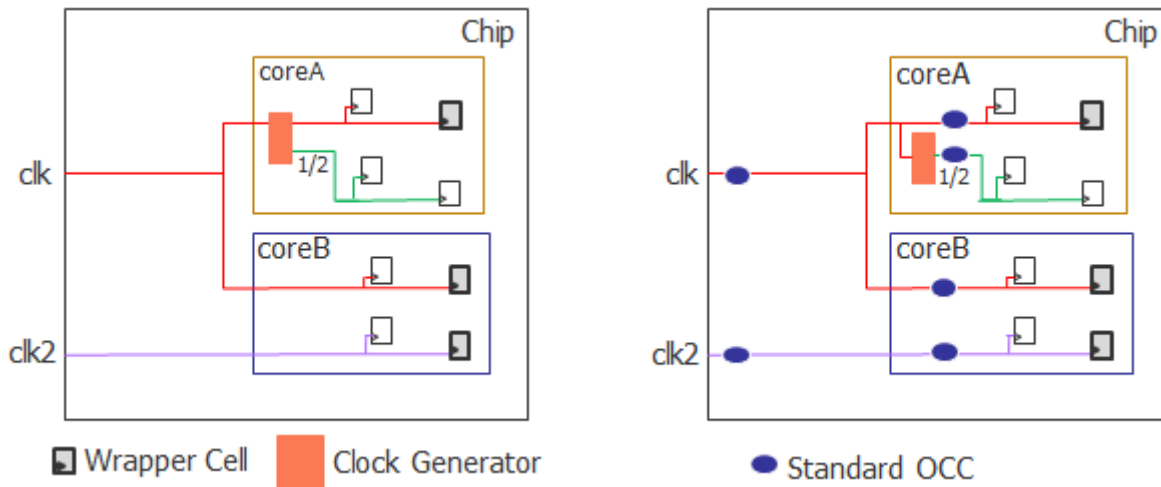
Figure D-2. OCCs With Clock Generators at Chip Level, Asynchronous Clocks



Clock Generators Inside the Cores

In the following example, coreA and coreB are wrapped cores and a clock generator is located inside coreA. The red and green clocks are asynchronous to each other inside coreA, so you would insert standard OCCs inside coreA. In addition, the red and purple clocks are asynchronous to each other, so you insert standard OCCs for these clocks inside coreB as well as at the chip-level.

Figure D-3. OCCs With Clock Generators Inside the Cores, Asynchronous Clocks

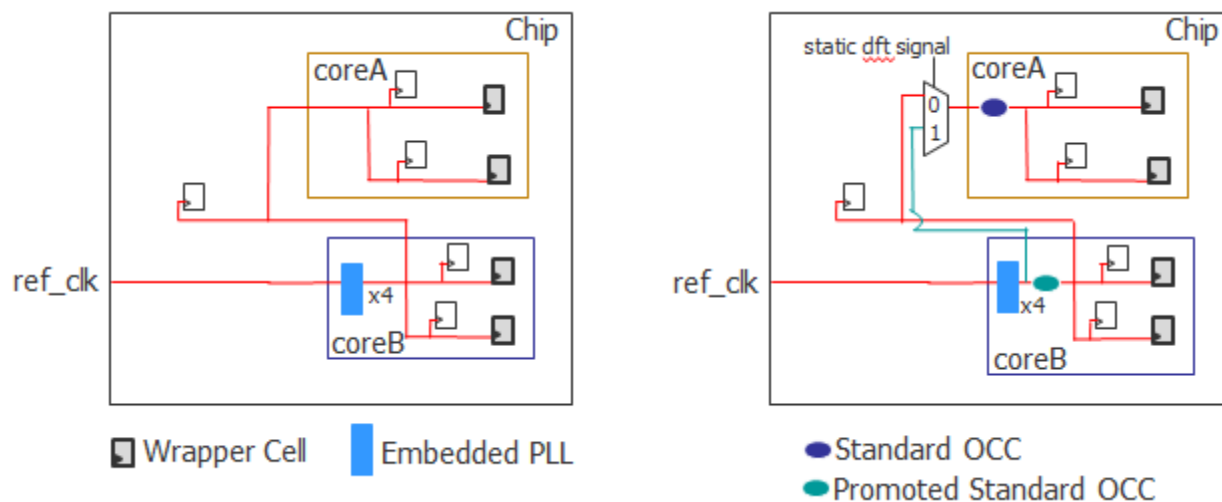


During intest mode for the wrapped cores, the OCCs inside the cores are active, and the OCCs outside the cores are inactive. During extest, the OCCs outside the cores are active, and the OCCs inside the cores are inactive.

Clock Sourced From A Core With Embedded PLL

In the following figure, coreA and coreB are wrapped cores, and coreB contains an embedded PLL module. coreB sources the clock that feeds both coreA and the chip. In this case, you would retarget coreA and coreB at the same time.

Figure D-4. Clock Sourced From A Core With Embedded PLL, With MUX



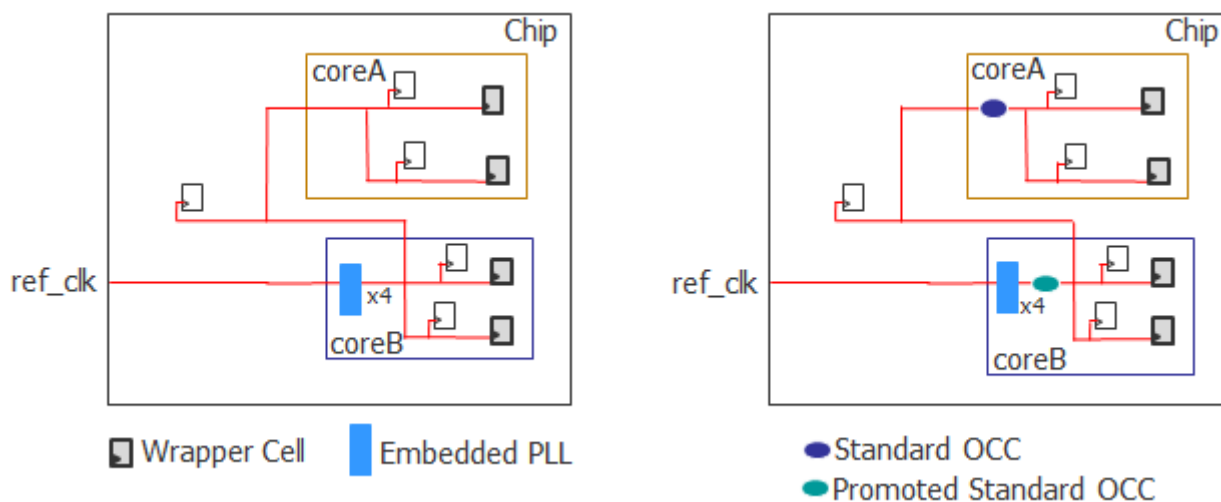
When you are retargeting coreA and coreB at the same time, you must add a mux. Use a clock net before reaching the OCC inside coreB as a clock to feed the OCC inside coreA to avoid having back-to-back OCCs. To retarget both coreA and coreB at the same time, the OCCs inside coreA and coreB must be active at the same time.

Insert standard OCCs inside the cores. The standard OCC inside the core with the embedded PLL (coreB) gets promoted so that it is also included in the external chains. When in external mode, the OCC inside coreB can be reused. For details, refer to “[How to Handle Clocks Sourced by Embedded PLLs During Logic Test](#)” on page 253.

During intest, the standard OCCs inside coreA and coreB are active. The mux is set to 1 to avoid cascading OCCs. During extest, only the promoted standard OCC within coreB is active, and the mux is held at 0.

In the following example, you are retargeting coreA and coreB in separate runs, so there is no need to insert a mux. During intest of coreA, only coreA’s OCC is active, and then during intest of coreB, only coreB’s OCC is active. During extest of the cores, only the promoted standard OCC within coreB is active.

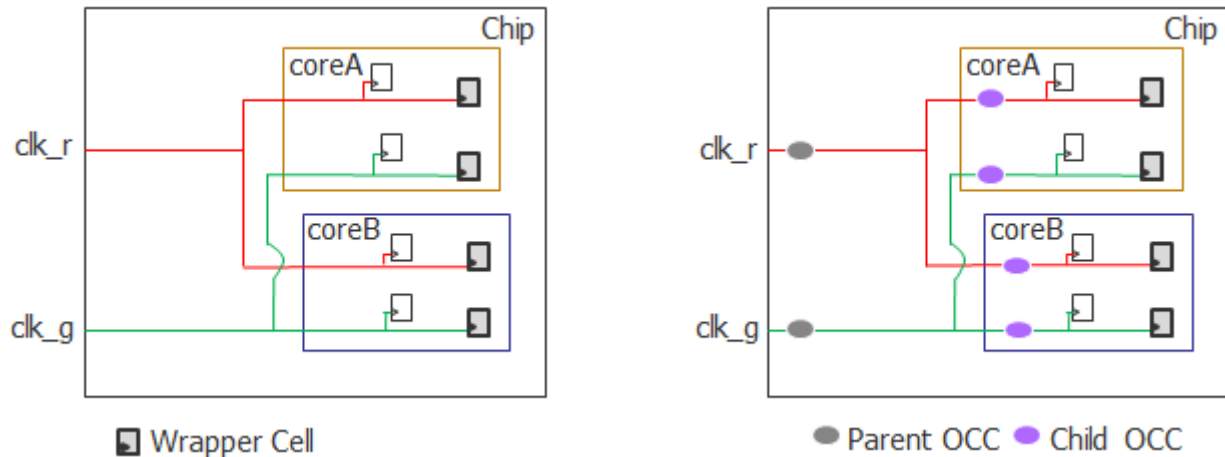
Figure D-5. Clock Sourced From A Core With Embedded PLL, Without MUX



Clock Mesh Synthesis

The most common method for clock balancing is clock tree synthesis. However, you can also use clock mesh synthesis to balance the clocks. For clock mesh, rather than standard OCCs, use parent OCCs at the chip level and child OCCs inside the wrapped cores. In the following figure, coreA and coreB are wrapped cores, and the clk_r and clk_g clocks are asynchronous to each other.

Figure D-6. Clock Mesh Synthesis



Core-level child OCCs have additional timing requirements because a single clock on the boundary of the wrapped core is used for shift as well as for slow and fast capture. For details, refer to “[Core OCC Recommendation](#)” in the *Tessent Scan and ATPG User’s Manual*. Using SDC for timing closure is described in “[Timing Constraints \(SDC\)](#)” on page 457.

During intest of coreA and coreB, the child OCCs are active, and the parent OCCs are in parent mode. During extest of coreA and coreB, the parent OCCs are in standard mode and the child OCCs are inactive.

Tessent Shell-based products currently do not generate scripts for use with Synopsys Formality. You can, however, set constraints in your design that are used with Formality.

Constraints for Formality Scripts 523

Constraints for Formality Scripts

The following guidance is provided to aid you in manually creating a script for with Formality.

Golden RTL Versus DFT Inserted RTL

Set the following:

- Set the circuit in functional mode. This is basically holding the IJTAG network and/or TAP in reset.
- If you are at the chip level, set a constant 0 on TRST. If you are at the sub_block/physical_block level, set a constant 1 on the ijtag_reset port(s).
- In both design levels, if your design has DftSignals coming from ports, they need to be set constant to their reset values. DftSignals from the TDR will automatically be handled by the reset.
- If you are at the sub_block or physical_block level, prevent the verification of ports created by Tessent Shell or they will report violations.
- Find all the instruments and SIBs that the Tessent tools inserted and assert the ijtag_reset pins to their active value.
- Since the ICL network may already be present our SIBs could result in mismatches on the ICL network scan path. The SIB scan in/outs would probably have to be set as “don't verify points”.

Pre-Layout Versus Post-Layout

Ensure the layout did not break DFT functionality as well. The only thing to ignore is if you have done scan insertion; scan chain reordering will report violations. The scan_en signal needs to be set to a constant low. Nothing else should be constrained on the inserted DFT.

After scan chain reordering, during layout, you may see mismatches on the lockup latches that Tessent Shell adds that become stop/anchor points in the scan DEF file. That is, during Tessent

Shell scan insertion, lockup elements are inserted at the end of the scan chains and marked as stop points in the scan DEF file, which means that they are not reordered.

```
GPIO_2/\p2out_reg[1]  ( IN SI ) ( OUT Q )  
+ STOP GPIO_2/ts_1_lockup_latchn_clkc6_intno266_i D\
```

When this occurs and you perform formal verification on the pre- versus post layout, the tool issues warnings that these lockups are no longer connected to the same functional flop as previously. This is not an issue with the other scan flops because you can disable the scan path by setting the scan enable to 0. However, the lockup is a simple flop/latch without scan-in.

If you encounter this situation, use the following workaround: Specify `set_dont_verify_point` for the retiming flops in both the reference and implemented designs to see if Formality passes. To find the retiming flops, introspect the design. For example:

```
get_instances ts_1_lockup*
```

EDT

Constraining `scan_en` to 0 (the same as requirement for scan insertion) is sufficient to proceed with equivalence checking. The formal verification tool will report some unmatched nodes like the newly-added EDT pins (*clock/update/bypass/low_power/channel*) and registers inside the EDT logic.

OCC

If the OCC has an `IjtagInterface`, or if the `test_mode` pin of the OCC is connected to an `IjtagNetwork`, then keeping the `ijtag` network in reset state is sufficient for OCC. Otherwise (that is, the `test_mode` port is connected to a top-level port), this port needs to be constrained to 0.

Appendix F

Transitioning from the Classic Point Tools

This appendix provides information to help you make the optional transition from the classic Tessent point tools to Tessent Shell. The point tool application commands are forward-compatible, so your dofiles work properly in Tessent Shell with only minor modifications. The main differences involved in using Tessent Shell is that you have to set the context before doing anything else, and then you must load the netlist and library.

Note that the classic Tessent point tools are available in the current release, so all of your existing dofiles and startup scripts continue to work as before. However, Mentor Graphics recommends that you start planning your transition now because each release of Tessent Shell contains additional features not available in the classic point tools, and future releases of Tessent Shell will continue to add new features that are not available in the classic point tools.

For information about specific point tools refer to the following sections:

Transitioning from the Classic FastScan Point Tool	525
Transitioning from the Classic TestKompress Point Tool.....	526
Transitioning from the Classic DFTAdvisor Tool	527
Transitioning from the Classic Diagnosis Tool.....	528

Transitioning from the Classic FastScan Point Tool

Beginning with the v2012.3 release, you can optionally transition from the classic FastScan point tool to Tessent Shell. Tessent Shell provides all of the commands available in the classic FastScan point tools (invoked with the “fastscan” shell command) plus additional features.

Note that the classic Tessent point tools are available in the current release, so all of your existing dofiles and startup scripts continue to work as before. However, Mentor Graphics recommends that you start planning your transition now because each release of Tessent Shell contains additional features not available in the classic point tools, and future releases of Tessent Shell will continue to add new features that are not available in the classic point tools.

If you are already familiar with classic FastScan, the main differences involved in using Tessent Shell is that you have to set the context before doing anything else, and then you must load the Verilog netlist and library:

```
set_context patterns -scan
read_verilog netlist_name
read_cell_library library_name
set_current_design
```

Another difference with using Tessent Shell is that several system modes used with classic FastScan (atpg, good, fault) have been replaced with a single system mode (called analysis). This means that “set_system_mode atpg” invocations (as well as transitions to good or fault modes) are replaced by “set_system_mode analysis.” To facilitate your transition to Tessent Shell, the set_system_mode command continues to accept the atpg/good/fault arguments, but the tool actually switches to analysis mode.

Also, it is highly recommended for any simulation to replace the old “set_pattern_source external” and “run” commands with the new read_patterns and simulate_patterns commands. Unlike the “run” command, which has slightly different behavior in the good/fault/atpg system modes, the simulate_patterns command allows you to explicitly control which pattern set to simulate and which patterns (if any) to store in the internal pattern set.

Transitioning from the Classic TestKompress Point Tool

Beginning with the v2012.3 release, you can optionally transition from the classic TestKompress point tool to Tessent Shell. Tessent Shell provides all of the commands available in the classic point tool (invoked with the “testkompress” shell command) for EDT IP creation and test pattern generation plus additional features.

Note that the classic Tessent point tools are available in the current release, so all of your existing dofiles and startup scripts continue to work as before. However, Mentor Graphics recommends that you start planning your transition now because each release of Tessent Shell contains additional features not available in the classic point tools, and future releases of Tessent Shell will continue to add new features that are not available in the classic point tools.

The following list describes how Tessent Shell is different from the classic TestKompress tool for creating EDT IP:

- Before issuing any commands, set the context to “dft -edt”:

```
set_context dft -edt
```

Once you have set the context, all commands in setup mode of classic TestKompress are available in the setup mode of Tessent Shell. All commands available in atpg mode of classic TestKompress (IP creation phase) are available in analysis mode of Tessent Shell.

- Use the existing `write_edt_files` command to generate all EDT files, including RTL, dofiles, the test procedure file, and synthesis and timing verification scripts. You can issue this command only when in analysis mode. Note that the `write_edt_files` command does not write out the EDT IP netlist, so you must do this with the `write_design` command.
- For internal IP location flow-based EDT insertion with classic TestKompress, the tool writes out all EDT files, inserts EDT IP into the design, and automatically changes to insertion mode. You then have the ability to further edit the netlist, but you must explicitly save changes with the `write_design` command.

Unlike classic TestKompress, Tessent Shell does not automatically exit after executing a `write_edt_files` command, so you must explicitly exit when ready. This is so that after IP insertion into the design, you can perform further design editing before writing out the design, or configure how to write out the design using the `write_design` command. Also note that Tessent Shell does not write out the netlist as part of the `write_edt_files` command, so you must write out the netlist using the `write_design` command.

- The classic TestKompress command `write_edt_files` has a switch named “-insertion tk.” In Tessent Shell, this switch is renamed “-insertion ts.”

The following list describes how Tessent Shell is different from the classic TestKompress tool for generating patterns:

- Before generating compressed patterns, set the context to “patterns -scan”:

`set_context patterns -scan`

Once you’ve set the context, all commands in the setup mode of classic TestKompress are available in setup mode of Tessent Shell. And all commands available in the atpg mode of classic TestKompress (Test Pattern Generation phase) are available in the analysis mode of Tessent Shell.

Transitioning from the Classic DFTAdvisor Tool

Beginning with the v2012.3 release, you can optionally transition from the classic DFTAdvisor point tool to Tessent Shell. Tessent Shell and Tessent Scan provide all of the commands available in the classic DFTAdvisor tool (invoked with the “dftadvisor” shell command), plus additional features.

Note that the classic Tessent point tools are available in the current release, so all of your existing dofiles and startup scripts continue to work as before. However, Mentor Graphics recommends that you start planning your transition now because each release of Tessent Shell contains additional features not available in the classic point tools, and future releases of Tessent Shell will continue to add new features that are not available in the classic point tools.

The following lists the differences between the classic DFTAdvisor tool and Tessent Scan:

- You no longer need to use the “run” command. Analysis that was previously done using the “run” command is now done automatically after DRC or as part of the [analyze_wrapper_cells](#) and [insert_test_logic](#) commands.
- After issuing `insert_test_logic` in analysis mode, the tool updates the design and then changes to insertion mode. This allows you to optionally perform further design editing before writing out the netlist with the `write_design` command.
- There is no longer a system mode called Dft. Use analysis mode instead.
- Before you issue any of the classic DFTAdvisor commands, you must first set the context:

```
set_context dft -scan
```

Transitioning from the Classic Diagnosis Tool

Beginning with the v2012.3 release, you can optionally transition from the classic Diagnosis point tool to Tessent Shell. Tessent Shell provides all of the commands available in the classic Diagnosis tool (invoked with the “tessent -diagnosis” shell command), plus additional features.

Note that the classic Tessent point tools is available in the current release, so all of your existing dofiles and startup scripts continue to work as before. However, Mentor Graphics recommends that you start planning your transition now because each release of Tessent Shell contains additional features not available in the classic point tools, and future releases of Tessent Shell will continue to add new features that are not available in the classic point tools.

After invoking Tessent Shell with the “tessent -shell” command, at a minimum, you must perform the following operations in sequence to enable scan diagnosis in Tessent Shell:

1. Set the context before doing anything else:

```
set_context patterns -scan_diagnosis
```

2. Read the flat model of your design:

```
read_flat_model flat_model_name
```

Previously, you specified the flattened design with the “tessent -diagnosis” command. Now you must use the [read_flat_model](#) command.

There are several ways to get help when setting up and using Tessent software tools. Depending on your need, help is available from documentation, online command help, and Mentor Graphics Support.

The Tessent Documentation System	529
Mentor Support Services.....	530

The Tessent Documentation System

At the center of the documentation system is the InfoHub that supports both PDF and HTML content. From the InfoHub, you can access all locally installed product documentation, system administration documentation, videos, and tutorials. For users who want to use PDF, you have a PDF bookcase file that provides access to all the installed PDF files.

For information on defining default HTML browsers, setting up browser options, and setting the default PDF viewer, refer to the “[Documentation Options](#)” in the *Mentor Documentation System* manual.

You can access the documentation in the following ways:

- **Shell Command** — On Linux platforms, enter **mgcdocs** at the shell prompt or invoke a Tessent tool with the **-manual** invocation switch.
- **File System** — Access the Tessent InfoHub or PDF bookcase directly from your file system, without invoking a Tessent tool. For example:

HTML:

```
firefox $MGC_DFT/docs/infohubs/index.html
```

PDF

```
acroread $MGC_DFT/docs/pdfdocs/_bk_tessent.pdf
```

- **Application Online Help** — ou can get contextual online help within most Tessent tools by using the “**help -manual**” tool command. For example:

> help dofile -manual

This command opens the appropriate reference manual at the “dofile” command description.

Mentor Support Services

Mentor provides a range of industry-leading support services that keep design teams productive and up-to-date with Mentor products.

A Mentor support contract includes the following:

- **Software Updates** — Get the latest releases and product enhancements to keep your environment current.
- **Mentor Graphics Support Center** — Access our online knowledge base, personalized to your Mentor products.
- **Support Forums** — Learn, share, and connect with other Mentor users.
- **Technical Support** — Collaborate with Mentor support engineers to solve complex design challenges.
- **Regular Communications** — Receive the latest knowledge base articles and announcements for your Mentor products.
- **Mentor Ideas** — Share ideas and vote for your favorites to shape future products.

More information is available here:

<https://support.mentor.com>

If your site is under a current support contract, but you do not have a Support Center login, register today:

<https://support.mentor.com/register>

— Symbols —

#include statement, 289

— A —

Alias statements, 293

Always block, 306

Always block example, 307

Always block statements
 always_statement, 307
 force, 307
 pulse, 307

Always block syntax, 307

— C —

Clock procedure, 342

Clock_run procedure, 344

Cycle statements
 bidi_force_off, 315
 bidi_force_pi, 314
 bidi_measure_po, 314
 condition, 316
 expect, 316
 force, 316
 force_pi, 314
 force_sci, 314
 force_sci_equiv, 314
 initialize, 316
 measure, 316
 measure_po, 314
 measure_sco, 314
 observe_method, 316
 pulse, 316
 pulse_capture_clock, 315
 pulse_read_clock, 315
 pulse_write_clock, 315
 restore_bidi, 315
 restore_pi, 314

— E —

EMPTY, 342, 343

Environment Variables

 TESSENT_LICENSE_ORDER, 28

Environment variables

 application-specific, 28

 MGCDFT_STARTUP, 28

external capture procedure, 346

— I —

Include statement, 289

— N —

Named capture procedure, 346

— P —

Procedure statements

 apply, 312

 cycle, 307

 label, 312

 scan_group, 310

 timeplate, 310

— S —

Sequential_transparent procedure, 340

Set statement, 290

Set statements

 default_timeplate, 292

 strobe_window time, 291

 time scale, 290

Shift procedure, 330

 alternate, 332

Sub_procedure, 356

— T —

Test procedure file

 defined, 284

 DRC checking, 284

 statements, 290

 timing variables, 295

Test procedures

 capture, 346

 clock_po, 351

- clock_sequential, [353](#)
- init_force, [354](#)
- load_unload, [334](#)
- master_observe, [339](#)
- ram_passthru, [353](#)
- ram_sequential, [351](#)
- shadow_control, [337](#)
- shadow_observe, [340](#)
- shift, [330](#)
- skew_load, [343](#)
- sub_procedure, [356](#)
- test_end, [355](#)
- test_setup, [327](#)
- Test_end procedure, [355](#)
- Time scale
 - setting, [290](#)
- Timeplate statements
 - bidi_force_pi, [299](#)
 - bidi_measure_po, [300](#)
 - force, [300](#)
 - force_pi, [299](#)
 - measure, [300](#)
 - measure_po, [300](#)
 - offstate, [299](#)
 - period, [302](#)
 - pulse, [300](#)
 - pulse_clock, [301](#)
- Transparent_capture cells, [342](#)
- tscale, [290](#)

Third-Party Information

Provides information on open source and third-party software that may be included in Tessent products.

For third-party information, refer to the [Third-Party Software for Tessent Products](#) document. Additional open source and third-party software information may be found in <your_Mentor_Graphics_documentation_directory>/legal

End-User License Agreement with EDA Software Supplemental Terms

Use of software (including any updates) and/or hardware is subject to the End-User License Agreement together with the Mentor Graphics EDA Software Supplement Terms. You can view and print a copy of this agreement at:

mentor.com/eula

